

LNCS 14983

Joaquin Garcia-Alfaro
Rafał Kozik
Michał Choraś
Sokratis Katsikas (Eds.)

Computer Security – ESORICS 2024

29th European Symposium
on Research in Computer Security
Bydgoszcz, Poland, September 16–20, 2024
Proceedings, Part II

2 Part II

 Springer

Lecture Notes in Computer Science

14983

Founding Editors

Gerhard Goos

Juris Hartmanis

Editorial Board Members

Elisa Bertino, *Purdue University, West Lafayette, IN, USA*

Wen Gao, *Peking University, Beijing, China*

Bernhard Steffen , *TU Dortmund University, Dortmund, Germany*

Moti Yung , *Columbia University, New York, NY, USA*

The series Lecture Notes in Computer Science (LNCS), including its subseries Lecture Notes in Artificial Intelligence (LNAI) and Lecture Notes in Bioinformatics (LNBI), has established itself as a medium for the publication of new developments in computer science and information technology research, teaching, and education.

LNCS enjoys close cooperation with the computer science R & D community, the series counts many renowned academics among its volume editors and paper authors, and collaborates with prestigious societies. Its mission is to serve this international community by providing an invaluable service, mainly focused on the publication of conference and workshop proceedings and postproceedings. LNCS commenced publication in 1973.

Joaquin Garcia-Alfaro · Rafał Kozik ·
Michał Choraś · Sokratis Katsikas
Editors

Computer Security – ESORICS 2024

29th European Symposium
on Research in Computer Security
Bydgoszcz, Poland, September 16–20, 2024
Proceedings, Part II

Editors

Joaquin Garcia-Alfaro 
Institut Polytechnique de Paris
Palaiseau, France

Michał Choraś 
Bydgoszcz University of Science
and Technology
Bydgoszcz, Poland

Rafał Kozik 
Bydgoszcz University of Science
and Technology
Bydgoszcz, Poland

Sokratis Katsikas 
Norwegian University of Science
and Technology - NTNU
Gjøvik, Norway

ISSN 0302-9743

ISSN 1611-3349 (electronic)

Lecture Notes in Computer Science

ISBN 978-3-031-70889-3

ISBN 978-3-031-70890-9 (eBook)

<https://doi.org/10.1007/978-3-031-70890-9>

© The Editor(s) (if applicable) and The Author(s), under exclusive license
to Springer Nature Switzerland AG 2024

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

If disposing of this product, please recycle the paper.

Preface

After a decade, the 29th edition of the prestigious ESORICS (European Symposium on Research in Computer Security) conference finally circled back to Poland, this time landing in the city of Bydgoszcz. Both the hosting country and the city itself have experienced rapid development in recent years, especially in the IT domain, and security in particular. Right now, computer security is more than just a hot topic – it is a vital necessity for Poland. As one of the most cyber-attacked countries in the world, Poland finds itself on the frontline of cybersecurity developments and research, making every advancement in this field not just important, but critical.

Bydgoszcz boasts a rich history of math, science and security technology, being the birthplace of Marian Adam Rejewski. In 1932, this brilliant Polish mathematician and cryptologist cracked the German military Enigma cipher machine, with documents provided by French military intelligence.

Currently, Bydgoszcz is a renowned academic hub, with one of the fastest-developing technical universities, namely Bydgoszcz University of Science and Technology (Politechnika Bydgoska im. J.J. Śniadeckich – PBS). This institution is at the forefront of cyber-research, as its Institute of Telecommunications and Computer Science leads the charge in cybersecurity, AI, machine learning and disinformation detection. With a strong track record in scientific dissemination and international projects, PBS is putting Bydgoszcz on the map as a centre of cutting-edge technological innovation.

Given this vibrant backdrop, we were thrilled to host a renowned and reputable computer security conference. As program committee chairs (Joaquin Garcia-Alfaro and Rafał Kozik) and general chairs (Michał Choraś and Sokratis Katsikas), we are proud to present this four-volume set of conference proceedings, published by Springer.

As anticipated, the response was overwhelming. We received an abundance of high-quality submissions from all over the world. Thanks to the tireless efforts of our program committee and reviewers, we selected a subset of those. As for the numbers, 535 submissions were reviewed (201 in the winter phase, and 334 in the spring phase). The program committee accepted 34 papers in the winter phase and 52 papers in the spring phase. In total, the program committee accepted 86 out of 535 submissions (16% acceptance) to constitute the program and LNCS proceedings of ESORICS 2024. Each paper received at least three single-blind reviews, some of the papers four reviews, and we had many comments and discussions between program committee members to ensure a fair and high-quality selection process. The program was enriched by two plenary talks by:

- Josef Pieprzyk from the Institute of Computer Science (Polish Academy of Sciences) and Data61, CSIRO, Sydney, Australia who came back to Bydgoszcz to his Alma Mater university he left for Australia decades ago,
- Wojciech Mazurczyk from Warsaw University of Technology, Warsaw, Poland.

The proceedings consist of high-quality papers in the following domains:

- AI and ML for security
- Security of AI and ML
- Network and web security
- Hardware and cloud security
- Privacy and personal data protection
- Software and systems security
- Applied cryptography
- Attacks and defenses

We would like to express our sincere gratitude to:

- Authors and contributors: without high-quality submissions from the authors, the success of the conference would not have been possible.
- PC members and additional reviewers: for the effort they put into the evaluation and high-quality in-depth reviews. Thanks to this collective effort we were able to send out notifications without a single day of delay.
- Publicity Chairs: Paria Shirani from the University of Ottawa, Canada, and Wenjuan Li from Hong Kong Polytechnic University, China for their efforts in spreading the word about ESORICS 2024.
- Web Chair Aleksandra Pawlicka from the University of Warsaw for her efforts and continuous and quick updates of the website.
- Workshops Chair Marek Pawlicki from Bydgoszcz University of Science and Technology for more than handling workshops organization, but also for being involved in all the critical organizational aspects.
- The Rector of Bydgoszcz University of Science and Technology, Marek Adamski, for his continuous support and efforts from the start of our ESORICS organizational journey.
- Organization Chair Tomasz Marciniak (also the Dean of the Faculty of Telecommunications and Computer Science) and Sponsor/Promotion Chair Michał Grzybowski for addressing numerous logistical, financial and organizational issues.

We would also like to thank the Springer team for their help in publishing the four-volume proceedings effectively, efficiently and on time. Thanks go as well to the EasyChair team, for all their help and support with the submissions platform.

In closing, we believe that ESORICS 2024 was an overall success and we hope that all attendees will remember their stay in Bydgoszcz, Poland at the ESORICS 2024 conference.

July 2024

Joaquin Garcia-Alfaro
Rafał Kozik
Michał Choraś
Sokratis Katsikas

Organization

General Chairs

Michał Choraś	Bydgoszcz University of Science and Technology, Poland
Sokratis Katsikas	Norwegian University of Science and Technology, Norway

Program Committee Chairs

Joaquin Garcia-Alfaro	Institut Polytechnique de Paris, France
Rafał Kozik	Bydgoszcz University of Science and Technology, Poland

Organization Chair

Tomasz Marciniak	Bydgoszcz University of Science and Technology, Poland
------------------	---

Workshops Chair

Marek Pawlicki	Bydgoszcz University of Science and Technology, Poland
----------------	---

Publicity Chairs

Paria Shirani	University of Ottawa, Canada
Wenjuan Li	Hong Kong Polytechnic University, China

Sponsor Chair

Michał Grzybowski	Fundacja Pischingera and Bydgoszcz University of Science and Technology, Poland
-------------------	--

Web Chair

Aleksandra Pawlicka

University of Warsaw, Poland

Steering Committee

Joachim Biskup

University of Dortmund, Germany

Frederic Cuppens

Polytechnique Montréal, Canada

Sabrina De Capitani di Vimercati

Università degli Studi di Milano, Italy

Joaquin Garcia-Alfaro (Chair)

Institut Polytechnique de Paris, France

Dieter Gollmann

Hamburg University of Technology, Germany

Sushil Jajodia

George Mason University, USA

Sokratis Katsikas

Norwegian University of Science and Technology,
Norway

Mirek Kutylowski

Wrocław University of Technology, Poland

Javier Lopez

Universidad de Málaga, Spain

Jean-Jacques Quisquater

Université catholique de Louvain, Belgium

Peter Y A Ryan

University of Luxembourg, Luxembourg

Pierangela Samarati

Università degli Studi di Milano, Italy

Einar Sneekkenes

Norwegian University of Science and Technology,
Norway

Michael Waidner

Technische Universität Darmstadt, Germany

Edgar Weippl

University of Vienna & SBA Research, Austria

Program Committee

Eric Alata

LAAS-CNRS, France

Massimiliano Albanese

George Mason University, USA

Cristina Alcaraz

University of Malaga, Spain

Magnus Almgren (Round 2)

Chalmers University of Technology, Sweden

Abdelrahman Aly (Round 2)

Cryptography Research Centre, Technology
Innovation Institute, United Arab Emirates

Giovanni Apruzzese

University of Liechtenstein, Liechtenstein

Mikael Asplund

Linköping University, Sweden

Vijay Atluri

Rutgers University, USA

Daniel Augot

Inria Saclay, France

Alessandro Barenghi (Round 2)

Politecnico di Milano, Italy

Sebastien Bardin (Round 2)

CEA List, France

Ken Barker

University of Calgary, Canada

Giampaolo Bella

University of Catania, Italy

Elisa Bertino	Purdue University, USA
Giuseppe Bianchi	Università di Roma Tor Vergata, Italy
Alex Biryukov	University of Luxembourg, Luxembourg
Jorge Blasco-Alis	Universidad Politécnica de Madrid, Spain
Carlo Blundo	Università degli Studi di Salerno, Italy
Rainer Böhme	University of Innsbruck, Austria
Nora Boulahia-Cuppens	Polytechnique Montréal, Canada
Pino Caballero-Gil (Round 2)	University of La Laguna, Spain
Xavier Carpent (Round 2)	University of Nottingham, UK
Mauro Conti	University of Padua, Italy & Delft University of Technology, The Netherlands
Scott Coull	Google, USA
Bruno Crispo	University of Trento, Italy
Frédéric Cuppens	Polytechnique Montréal, Canada
Mila Dalla Preda	University of Verona, Italy
Tooska Dargahi (Round 2)	Manchester Metropolitan University, UK
Sanchari Das	University of Denver, USA
Sabrina De Capitani di Vimercati	University of Milan, Italy
Hervé Debar	Institut Polytechnique de Paris, France
Jose Maria de Fuentes	Universidad Carlos III, Spain
Roberto Di Pietro	King Abdullah University of Science and Technology, Saudi Arabia
Xuhua Ding	Singapore Management University, Singapore
Josep Domingo-Ferrer	Universitat Rovira i Virgili, Spain
Jannik Dreier (Round 2)	Université de Lorraine, France
François Dupressoir	University of Bristol, UK
Andreas Ekelhart (Round 2)	Secure Business Austria, Austria
Anna Lisa Ferrara	Università degli studi del Molise, Italy
Steven Furnell (Round 1)	University of Nottingham, UK
Olga Gadyatskaya	University of Leiden, The Netherlands
Debin Gao (Round 2)	Singapore Management University, Singapore
Yansong Gao (Round 2)	CSIRO/Data61, Australia
Essam Ghadafi (Round 2)	Newcastle University, UK
Giorgio Giacinto	University of Cagliari, Italy
Alberto Giarretta	Örebro Universitet, Sweden
Dieter Gollmann	Hamburg University of Technology, Germany
Lorena González Manzano	Universidad Carlos III, Spain
Dimitris Gritzalis (Round 2)	Athens University of Economics & Business, Greece
Berk Gulmezoglu	Iowa State University, USA
Xueyuan Han (Round 2)	Stellar Cyber, USA
Hannes Hartenstein	Karlsruhe Institute of Technology, Germany

Javier Herranz (Round 2)	Universitat Politècnica de Catalunya, Spain
Jordi Herrera-Joancomarti	Universitat Autònoma de Barcelona, Spain
Lucca Hirschi	Inria, France
Chung Hwan Kim	University of Texas at Dallas, USA
Hugo Jonker	Open University of the Netherlands, The Netherlands
Mohamed Kaaniche	LAAS-CNRS, France
Stefan Katzenbeisser	University of Passau, Germany
Jörg Keller (Round 2)	FernUniversität in Hagen, Germany
Hiroaki Kikuchi	Meiji University, Japan
Hyoungshick Kim	Sungkyunkwan University, South Korea
Jihye Kim (Round 1)	Kookmin University, South Korea
Marina Krotofil	European Commission, Switzerland
Christopher Kruegel	University of California, Santa Barbara, USA
Mirosław Kutylowski	NASK-National Research Institute, Poland
Alptekin Küpçü	Koç University, Turkey
Romain Laborde	Université Toulouse 3 Paul Sabatier, France
Pascal Lafourcade	LIMOS, Université Clermont Auvergne, France
Peeter Laud	Cybernetica AS, Estonia
Maryline Laurent	Institut Polytechnique de Paris, France
Kaitai Liang	Delft University of Technology, The Netherlands
Peng Liu	Pennsylvania State University, USA
Giovanni Livraga	University of Milan, Italy
Valeria Loscri	Inria, France
Wenjing Lou	Virginia Tech, USA
Eduard Marin	Telefónica Research, Spain
Jean-Yves Marion	Université de Lorraine, France
Gregorio Martinez-Perez	University of Murcia, Spain
Ivan Martinovic	University of Oxford, UK
Barbara Masucci (Round 2)	University of Salerno, Italy
Wojciech Mazurczyk	Warsaw University of Technology, Poland
Catherine Meadows	Naval Research Laboratory, USA
David Megías	Universitat Oberta de Catalunya, Spain
Weizhi Meng	Technical University of Denmark, Denmark
Chris Mitchell	Royal Holloway, University of London, UK
John C. Mitchell (Round 1)	Stanford University, USA
Haralambos Mouratidis	University of Essex, UK
Guillermo Navarro-Arribas	Autonomous University of Barcelona, Spain
Antonino Nocera	University of Pavia, Spain
Gabriele Oligeri	Hamad Bin Khalifa University, Qatar
Cristina Onete	Université de Limoges, France
Philippe Owezarski	LAAS-CNRS, France

Stefano Paraboschi	Università di Bergamo, Italy
Josef Pieprzyk (Round 2)	CSIRO/Data61, Australia
Joachim Posegga	University of Passau, Germany
Jean-Jacques Quisquater	Université catholique de Louvain, Belgium
Kai Rannenberg	Goethe University Frankfurt, Germany
Indrakshi Ray	Colorado State University, USA
Peter Rønne	University of Luxembourg, Luxembourg
Giovanni Russello (Round 1)	University of Auckland, New Zealand
Indrajit Ray (Round 2)	Colorado State University, USA
Peter Y. A. Ryan	University of Luxembourg, Luxembourg
Reihaneh Safavi-Naini	University of Calgary, Canada
Pierangela Samarati	Università degli Studi di Milano, Italy
David Sanchez	Universitat Rovira i Virgili, Spain
Neetesh Saxena (Round 2)	Cardiff University, UK
Dominique Schröder	Universität Erlangen-Nürnberg, Germany
Joerg Schwenk	Ruhr-Universität Bochum, Germany
Savio Sciancalepore	Eindhoven University of Technology, The Netherlands
Siamak Shahandashti	University of York, UK
Claudio Soriente	NEC Laboratories Europe, Germany
Alessandro Sorniotti	IBM Research Europe, Switzerland
Angelo Spognardi	Sapienza Università di Roma, Italy
Riccardo Spolaor	Shandong University, China
Thorsten Strufe	Karlsruhe Institute of Technology, Germany
Florence Sèdes	Université Paul Sabatier, France
Nadia Tawbi	Laval University, Canada
Vicenç Torra	Umeå University, Sweden
Daniele Venturi	Sapienza University of Rome, Italy
Haining Wang	Virginia Tech, USA
Cong Wang	City University of Hong Kong, China
Edgar Weippl	University of Vienna, Austria
Avishai Wool	Tel Aviv University, Israel
Stefano Zanero	Politecnico di Milano, Italy
Nicola Zannone	Eindhoven University of Technology, The Netherlands
Urko Zurutuza	Mondragon Unibertsitatea, Spain

Additional Reviewers

Abdelgawad, Mahmoud	Courtney, Emily
Abedi, Kamyar	Cozza, Vittoria
Afzal, Zeeshan	D'Onghia, Mario
Ahmed, Faisal	Dai, Xushu
Almutairi, Abeer	Daniele, Cristian
Alqurashi, Saja	Dargahi, Tooska
Amoussou-Guenou, Yackolley	Dedousis, Panagiotis
Andriotis, Panagiotis	Dehez Clementi, Marina
Antognazza, Francesco	Del Vasto Terrientes, Luis
Arazzi, Marco	Di Paolo, Edoardo
Atalay, Tolga	Diemunsch, Vincent
Avizheh, Sepideh	Dipta, Debopriya Roy
Baecker, Ruben	Dolati, Mahdi
Bamiloshin, Michael	Droll, Jan
Barat, Md Mohaimin Al	Eichhammer, Philipp
Barenghi, Alessandro	Ekelhart, Andreas
Bashir, Shadaab Kawnain	El Fray, Imed
Bassetti, Enrico	El Kassem, Nada
Battarbee, Kit	Esposito, Sergio
Belguith, Sana	Fabi, Michele
Benaloh, Josh	Facchinetti, Dario
Benes, Martin	Fallahi, Martin
Beretta, Michele	Farasat, Talaya
Bezawada, Bruhadeshwar	Fdhila, Walid
Birba, Aubin	Finogina, Tamara
Biswas, Chinmoy	Fries, Heinrich
Blanco-Justicia, Alberto	Galdi, Clemente
Bodaghi, Omid	Gangwal, Ankit
Briongos, Samira	Garcia, Victor
Caporaso, Pasquale	Garrigues, Carles
Caprolu, Maurantonio	Gegenhuber, Gabriel Karl
Carminati, Michele	George, Aleena Elsa
Carpent, Xavier	George, Dominik
Castiglione, Gianpietro	Gerhart, Paul
Casula, Roberto	Ghadafi, Essam
Chen, Depeng	Gogov, Boris
Cheng, Hao	Golinelli, Matteo
Chu, Hien	Gollmann, Dieter
Chu, Hien Thi Thu	Gorbett, Matt
Cicala, Fabrizio	Grisafi, Michele
Cihangiroglu, Mert	Groszschädl, Johann
Cimato, Stelvio	Grundmann, Matthias
Cinal, Adrian	Guerra-Balboa, Patricia

Guo, Xiaonan
Haffar, Rami
Haffey, Preston
Haque, Md Shahedul
Herz, Johanna
Hofer, Nora
Holzbauer, Florian
Homayouni, Hajar
Huang, Mengdie
Jacob, Florian
Jacomme, Charlie
Jafari, AmirHossein
Jafari, Amirhossein
Jebreel, Najeeb
Jin, Heng
Katsis, Charalampos
Kersten, Leon
Khan, Safiullah
Korichi, Youcef
Krejci, Philippe
Lazrig, Ibrahim
Leinweber, Marc
Lerch-Hostalot, Daniel
Lesniak, Mateusz
Leventopoulos, Sozon
Li, Adrian Shuai
Li, Jiaxin
Liang, Yuan
Litzinger, Sebastian
Liu, Guannan
Lombard-Platet, Marius
Luo, Nanqing
Lybarger, Kevin
Maehren, Marcel
Mahmoud, Dhekra
Mallordy, Lola-Baie
Marcadet, Gael
Marchiori, Francesco
Martinez, Sergio
Masucci, Barbara
Maugeri, Marcello
Meng, Fei
Merchant, Farhad
Mostafiz, Mir Imtiaz
Mouheb, Djedjiga

Nabi, Mahmudun
Nair, Vinod P.
Nam, Ihyun
Neal, Christopher
Nguyen, Hieu
Nicolazzo, Serena
Niknia, Ahad
Nowroozi, Ehsan
Oldani, Gianluca
Olivier-Anclin, Charles
Panja, Somnath
Patrignani, Marco
Pedrouzo, Alberto
Perez Kempner, Octavio
Pisu, Lorenzo
Podder, Rakesh
Pornin, Thomas
Preatoni, Riccardo
Pucher, Michael
Puthuvath, Vinod
Quaglia, Francesco
Raciti, Mario
Ramponi, Carlo
Regano, Leonardo
Saadeh, Angelo
Sanna, Alessandro
Sanna, Silvia Lucia
Sargolzaei Javan, Morteza
Schmid, Martin
Schmidbauer, Tobias
Schriftwieser, Sebastian
Senn, Judith
Sha, Kailun
Shahriar, Md Hasan
Sharma, Pragya
Sheikhalishahi, Mina
Shirazi, Hossein
Siniscalchi, Luisa
Skandylas, Charilaos
Skrobot, Marjan
Sobolewski, Oliwer
Soi, Diego
Son, Seonghun
Soroush, Najmeh
Sotgiu, Angelo

Spiekermann, Daniel
Stengele, Oliver
Stergiopoulos, George
Stifter, Nicholas
Sun, Shihua
Talukder, Rakibul
Thomas, Julian
Todd, James
Todt, Julian
Tronnier, Frederic
Tseng, Pei-Yu
Tsoumas, Bill
Udovenko, Aleksei
Virvilis, Nick
Wang, Haizhou

Wang, Rui
Wang, Xinda
Wang, Ziyang
Wechta, Gabriel
Yu, Hexuan
Zampieri, Marcos
Zhang, Chaoyu
Zhang, Guoming
Zhang, Lan
Zhang, Yiwei
Zhang, Zhi
Zhang, Zhikun
Zhao, Yi
Zhou, Ming

Contents – Part II

Network, Web, Hardware and Cloud

PRETT2: Discovering HTTP/2 DoS Vulnerabilities via Protocol Reverse Engineering	3
<i>Choongin Lee, Isa Jafarov, Sven Dietrich, and Heejo Lee</i>	
TurboTLS: TLS Connection Establishment with 1 Less Round Trip	24
<i>Carlos Aguilar-Melchoro, Thomas Bailleux, Jason Goertzen, Adrien Guinet, David Joseph, and Douglas Stebila</i>	
From Fingerprint to Footprint: Characterizing the Dependencies in Encrypted DNS Infrastructures	45
<i>Baiyang Li, Yujia Zhu, Yong Ding, Yong Sun, Yuedong Zhang, Qingyun Liu, and Li Guo</i>	
Netfuzzlib: Adding First-Class Fuzzing Support to Network Protocol Implementations	65
<i>Jeroen Robben and Mathy Vanhoef</i>	
BOOTRIST: Detecting and Isolating Mercurial Cores at the Booting Stage	85
<i>Yihao Luo, Yunjie Deng, Jingquan Ge, Zhenyu Ning, and Fengwei Zhang</i>	
Social Control and Interactivity in Anonymous Public Events	104
<i>Md Mushfekur Rahman and Philip W. L. Fong</i>	
Faster Post-quantum TLS 1.3 Based on ML-KEM: Implementation and Assessment	123
<i>Jieyu Zheng, Haoliang Zhu, Yifan Dong, Zhenyu Song, Zhenhao Zhang, Yafang Yang, and Yunlei Zhao</i>	
CRATOR a CRAWler for TOR: Turning Dark Web Pages into Open Source Intelligence	144
<i>Daniel De Pascale, Giuseppe Cascavilla, Damian A. Tamburri, and Willem Jan Van Den Heuvel</i>	
ProxyKiller: An Anonymous Proxy Traffic Attack Model Based on Traffic Behavior Graphs	162
<i>Hongbo Xu, Zhenyu Cheng, Shuhao Li, Chenxu Wang, Peishuai Sun, Jiang Xie, and Qingyun Liu</i>	

Confidential and Verifiable Machine Learning Delegations on the Cloud	182
<i>Wenxuan Wu, Soamar Homsî, and Yupeng Zhang</i>	
Process Identity-Based Firewalling	202
<i>Radu Mantu, Mihai Chiroiu, and Costin Raiciu</i>	
Privacy and Personal Data Protection	
Privacy-Preserving Regular Expression Matching Using TNFA	225
<i>Ning Luo, Chenkai Weng, Jaspal Singh, Gefei Tan, Mariana Raykova, and Ruzica Piskac</i>	
Nemesis: Combating Abusive Information in Encrypted Messaging with Private Reporting	247
<i>Rui Lian, Yulong Ming, Chengjun Cai, Yifeng Zheng, Cong Wang, and Xiaohua Jia</i>	
ARPSO: An OIDC-Compatible Privacy-Preserving SSO Scheme Based on RP Anonymization	268
<i>Junlin He, Lingguang Lei, Yewu Wang, Pingjian Wang, and Jiwu Jing</i>	
DPC: Filtering Out Patch-Based Poisoned Samples with Differential Privacy	289
<i>Yukun Yan, Peng Tang, Rui Chen, Qilong Han, and Ruochen Du</i>	
Private Decision Tree Evaluation with Malicious Security via Function Secret Sharing	310
<i>Jiaxuan Fu, Ke Cheng, Yuheng Xia, Anxiao Song, Qianxing Li, and Yulong Shen</i>	
Review the Cuckoo Hash-Based Unbalanced Private Set Union: Leakage, Fix, and Optimization	331
<i>Keyang Liu, Xingxin Li, and Tsuyoshi Takagi</i>	
BMI: Bounded Mutual Information for Efficient Privacy-Preserving Feature Selection	353
<i>David Eklund, Alfonso Iacovazzi, Han Wang, Apostolos Pyrgelis, and Shahid Raza</i>	
Task-Specific Knowledge Distillation with Differential Privacy in LLMs	374
<i>Sonakshi Garg and Vicenç Torra</i>	
Exploiting Internal Randomness for Privacy in Vertical Federated Learning	390
<i>Yulian Sun, Li Duan, Ricardo Mendes, Derui Zhu, Yue Xia, Yong Li, and Asja Fischer</i>	

Code Voting: When Simplicity Meets Security	410
<i>Véronique Cortier, Alexandre Debant, and Florian Moser</i>	
Speedy Privacy-Preserving Skyline Queries on Outsourced Data	430
<i>Yu Chen, Lin Liu, Rongmao Chen, Shaojing Fu, Yuexiang Yang,</i> <i>Jiangyong Shi, and Liangzhong He</i>	
RedactBuster: Entity Type Recognition from Redacted Documents	451
<i>Mirco Beltrame, Mauro Conti, Pierpaolo Guglielmin,</i> <i>Francesco Marchiori, and Gabriele Orazi</i>	
Author Index	471

Network, Web, Hardware and Cloud



PRETT2: Discovering HTTP/2 DoS Vulnerabilities via Protocol Reverse Engineering

Choongin Lee¹, Isa Jafarov², Sven Dietrich^{2(✉)}, and Heejo Lee^{1(✉)}

¹ Korea University, Seoul, Republic of Korea
{choonginlee, heejo}@korea.ac.kr

² City University of New York, New York, USA
ijafarov@gradcenter.cuny.edu, spock@ieee.org

Abstract. HTTP/2, enhancing data transmission speed over HTTP/1.1 with features such as flow control for stream multiplexing, has seen widespread adoption across major web servers. This has exposed numerous vulnerabilities, with denial of service (DoS) particularly prominent due to flawed flow control implementations. Identifying potential weaknesses in the flow control across various HTTP/2 implementations has largely depended on manual inspection. However, the behavioral diversity among implementations poses significant challenges for testing.

In this study, we propose PRETT2, a stateful fuzzing framework targeting denial-of-service (DoS) vulnerabilities in HTTP/2 protocols. Utilizing automated protocol reverse engineering, PRETT2 infers state machines unique to various HTTP/2 implementations. Then it executes multiplexed fuzzing that manipulates flow control messages based on the identified state machines. Testing on servers such as Apache and Nginx revealed the capability of PRETT2 to infer multiple state machine types and uncover security vulnerabilities, including CVE-2023-43622 by Apache. This highlights the effectiveness of PRETT2 in identifying and addressing critical security vulnerabilities in HTTP/2.

1 Introduction

HTTP/2 [45] is an advanced application-level protocol that supersedes HTTP/1.1 and includes additional features. According to the Cloudflare global web traffic statistics [9], HTTP/2 traffic accounts for the majority of the HTTP protocol series (around 65%). The primary advantage of HTTP/2 is that it supports flow control for stream multiplexing [57, 58]. This addresses the limitations of HTTP/1.1, such as Head-of-Line blocking [40], and enhances the protocol's efficiency and performance.

Unfortunately, the number of vulnerability reports regarding HTTP/2 implementations has steadily increased as HTTP/2 software is developed with various security holes that may be exploited. Figure 1 illustrates the number

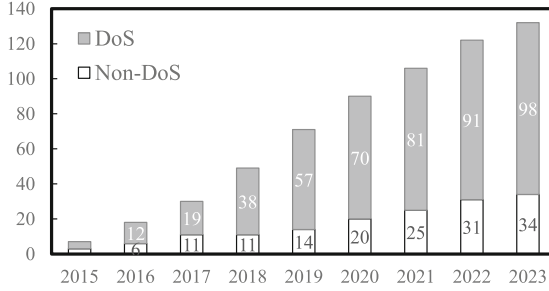


Fig. 1. The accumulated number of CVEs for HTTP/2 software per year. The majority type of CVEs is DoS. The CVEs are selected based on their descriptions that contain the keyword HTTP/2.

of Common Vulnerabilities and Exposures (CVEs) of HTTP/2 implementations since the introduction of HTTP/2 in 2015. Among these vulnerabilities, 74% are classified as denial-of-service (DoS) attacks [30] in 2023¹. Multiple studies have criticized that HTTP/2 implementations are vulnerable to DoS attacks [1, 8, 22, 24, 29, 48, 50] due to the inappropriate use of features for flow control. We highlight the point that the incomplete implementation of flow control features could potentially compromise network security.

To mitigate potential HTTP/2 DoS attacks, it is imperative to conduct a thorough analysis of HTTP/2 implementations to identify potential threats that may exhaust system resources via flow control. To do this, security analysts are required to have a deep understanding of complex communication features. Although prior endeavors using manual analysis methods have discovered many DoS vulnerabilities, they are daunting and time-consuming to apply for diverse implementations. Even worse, an HTTP/2 implementation typically consists of a significant amount of source code which makes it hard to analyze by hand. As a remedy, automated testing can significantly reduce the effort.

For the automated testing of HTTP/2, a few methods [7, 41] have been proposed. Nevertheless, it is still challenging to discover DoS vulnerabilities in HTTP/2 due to the sophisticated properties of HTTP/2. Based on our empirical analysis, we set three key challenges for automated testing: flow control, stateful testing, and scalability.

1. **Flow control:** Flow control is a vital feature of HTTP/2, enabling stream multiplexing. We examined that most CVE-registered vulnerabilities are attributed to security flaws in the flow control. Thus, it is important to take into account its multiplexing features to identify deficiencies.
2. **Stateful testing:** Stateful fuzzing is a method that enables stateful testing, which tests server under test (SUT) with test cases based on state

¹ We determined the type of a CVE as DoS if its description contains related keywords such as denial-of-service, application/server crashes, or excessive resource consumption as its effect.

information [35]. To analyze potential security flaws of protocol implementation, it is effective to use stateful testing that uses a finer-grained state machine [7]. Unlike HTTP/1.1, HTTP/2 is a stateful protocol, in which the acceptable set of messages and the observable response depend on each protocol state.

3. **Scalability:** We examined that around 100 implementations of HTTP/2 are available on the GitHub source code repository [23] and numerous versions per implementation. We examined that HTTP/2 traces for the same user behavior can vary across servers and clients, including their versions, indicating significant variations in the protocol’s state machines. The diversity can be attributed to the preferences or intentions of developers [17, 21]; they may exclude or add certain features. Hence, stateful fuzzing for HTTP/2 must be sufficiently scalable to accommodate the diverse range of state machines.

This paper introduces PRETT2, a stateful fuzzing framework designed to analyze HTTP/2 DoS via multiplexed fuzzing. Addressing the aforementioned challenges, PRETT2 infers its state machines by the use of protocol reverse engineering given network traces. Interacting with the server, PRETT2 utilizes existing work [28] for the protocol reverse engineering with a modified fuzzing algorithm that supports messages of binary protocol and checks time feedback. Based on the inferred state machines, PRETT2 traverses state transitions and performs multiplexed fuzzing, generating test cases with manipulated flow control messages. During fuzzing, PRETT2 measures another time feedback and compares it to previously measured feedback to detect extended connection durations, assessing potential DoS vulnerabilities.

The experimental results show that PRETT2 successfully inferred multiple types of state machines depending on the implementations. Through the stateful fuzzing HTTP/2 implementations, such as Apache and Nginx, PRETT2 identified three security flaws resulting in DoS, which have been reported to the developers. One of the vulnerabilities has been assigned the CVE-ID (CVE-2023-43622) by Apache.

This work presents the following contributions:

- We present a stateful fuzzing framework to automatically analyze HTTP/2 denial-of-service flaws. It uses a novel method to infer HTTP/2 state machines and performs fuzzing tests targeting its flow control features.
- Our experiments show that diverse state machines can be inferred for the same user behavior depending on the implementations. Notably, we demonstrate that the flow control of a state machine may be different from others.
- We discovered and reported multiple security flaws in the flow control of popular HTTP/2 implementations using stateful fuzzing by PRETT2.

2 Background

2.1 HTTP/2

HTTP (Hypertext Transfer Protocol) is a fundamental protocol used to request data from web servers. The first version of HTTP for common usage was

Table 1. Properties of HTTP/1.1 and HTTP/2

	<i>HTTP/1.1</i>	<i>HTTP/2</i>
Message format	plain text	binary frames
Connections	pipelining	stream multiplexing
Statefulness	stateless	stateful
Transport layer	TCP	TCP

HTTP/1 [10], followed by HTTP/1.1 [20]. One of the significant improvements in HTTP/1.1 is its ability to handle parallelized requests through pipelining. However, HTTP/1.1 still had a limitation known as head-of-line blocking (HoLB) [40] which causes delays in simultaneous data transmission and cannot cope with huge network traffic. To be specific, a response to a request has to wait until the response to the previous request is processed when requests are in parallel. To address this issue, a new protocol called SPDY was developed, forming the basis for HTTP/2 [15, 43].

Table 1 outlines the distinctions between HTTP/2 and HTTP/1.1. A key difference is the shift of HTTP/2 from plain text to binary framing for HTTP headers and messages, optimizing communication data efficiency. Unlike HTTP/1.1's reliance on multiple TCP connections, HTTP/2 employs stream multiplexing within a single TCP connection to enable parallel requests and responses. For instance, HTTP/2 allows for setting stream priorities and maintaining idle streams with the **PRIORITY** frame [57, 58]. Additionally, HTTP/2 introduces statefulness with enhanced functionalities like flow control, in contrast HTTP/1.1 has a stateless nature [45]. Such property makes the sequences of messages with appropriate frames important. The list of all HTTP/2 frames and the detailed information is described in Table 2.

Table 2. Types of HTTP/2 communication frames

Frame type	Description
SETTINGS	Configures flow control parameters for the communication in a stream.
WINDOW_UPDATE	Sets or updates a stream's data capacity for flow control.
DATA	Transmits variable-length octet sequences in a stream.
HEADERS	Opens a stream for data request or response.
PUSH_PROMISE	Allows multiple responses to a single request from the client.
PRIORITY	Specify a stream to be processed first by designating the priority.
RST_STREAM	Immediately terminates a stream with an error code.
GOAWAY	Terminates the connection using a priority error code over other frames.
CONTINUATION	Continues a prior HEADERS frame's field block sequence.
PING	Assesses idle connection functionality by measuring round-trip time.

Listing 1. HTTP/2 specification: Denial-of-Service Considerations

... The SETTINGS frame can be abused to cause a peer to expend additional processing time. This might be done by pointlessly changing SETTINGS parameters, setting multiple undefined parameters, or changing the same setting multiple times in the same frame. WINDOW_UPDATE or PRIORITY frames can be abused to cause an unnecessary waste of resources. ...

Flow Control. Flow control in HTTP/2 facilitates the multiplexing of communications within a single connection by preventing the interference of streams between a server and a client. It leverages the SETTINGS and WINDOW_UPDATE frames to manage resource allocation through DATA frames. This involves configuring the receiver’s buffering capacity, known as the flow control window, and allowing each sender to maintain distinct flow control windows for each stream and the connection as a whole [6]. Adjustments to the flow control window, such as setting initial sizes via SETTINGS frame and modifying current sizes through WINDOW_UPDATE frame, enable communication parties to signal their readiness to receive data [1].

2.2 Protocol Reverse Engineering

Protocol reverse engineering discerns the structure and sequence of protocol messages, resulting in a state machine that specifies the anticipated interactions within each state, aiding analysts in protocol behavior analysis [32]. Its key advantage lies in enabling fuzzers to conduct stateful testing by generating test cases that explore the complex state dynamics of an implementation.

Protocol reverse engineering can be done by hand, but it requires huge labor. Unfortunately, protocol state machines are often not well-defined in typical protocol specifications, according to numerous works of literature [26, 33, 36, 37]. Specifically, many protocol specifications are written in prose [26, 36] or minimally documented [32]. Further, multiple state machines may be inferred for the same protocol depending on implementation. Such diversity requires security experts to spend extra effort. As a remedy, automated protocol reverse engineering is effective in coping with the diverse state machines. Prior work has demonstrated that protocols can successfully be reverse-engineered in an automated way [11, 55], which is also possible to infer multiple state machines per implementation [28].

3 Security Analysis for HTTP/2

3.1 Prior Works

Manual Analysis. Prior works on the security analysis of HTTP/2 relied heavily on manual analysis [8, 29, 48, 50]. However, one limitation of such manual analysis is scalability. This is because HTTP/2 has multiple implementations with different features based on the developer’s interpretations [25]. For instance, one

paragraph in the **Denial-of-Service Considerations** section of the HTTP/2 specification (shown in Listing 1) does not specify the exact parameters and values (or their ranges) that correspond to the *multiple* numbers. The obscure descriptions may lead to using subjective mitigations.

Automated Analysis. As a remedy to manual analysis, proposals for automated analysis have emerged as effective means for identifying HTTP/2 vulnerabilities. However, despite these advances, they have yet to surmount all the challenges previously mentioned. The detailed description of each approach is as follows:

Table 3. Challenges addressed by automated analysis

	<i>Flow Control</i>	<i>Stateful Testing</i>	<i>Scalability</i>
http2fuzz	Yes	No	No
SGFUZZ	No	Yes	Yes
Proposed	Yes	Yes	Yes

1. http2fuzz [41] is an open-source project that conducts fuzzing on HTTP/2 messages. It uses pre-defined message formats to generate test cases and supports HTTP/2 messages that include flow control frames such as **SETTINGS** and **WINDOW_UPDATE** frames. This addresses the challenge of *flow control*. However, due to a lack of insight into the state machines implemented in different implementations, it is unable to tackle the challenges of *stateful testing* and *scalability*.
2. SGFUZZ [7] is the first to use stateful fuzzing research for HTTP/2. It is effective at addressing challenge *stateful testing* by automatically extracting the state machine of target implementation (H2O). SGFUZZ uses static analysis on source code which is the general approach that can be applied to diverse implementations and addresses challenge *scalability*. However, the state machine extracted by SGFUZZ does not include HTTP/2 messages for flow control. It only contains state transitions for data requests and responses, which makes it unable to address challenge *flow control*.

The challenges addressed by each method and our proposal have been summarized in Table 3.

3.2 Motivating Example

As the main inspiration for our study, we describe an example of an HTTP/2 DoS vulnerability, which is publically reported as CVE-2016-1546 [16]. The vulnerability is triggered by an HTTP/2 slow rate attack [48] that exploits a flaw in the flow control implementation of the Apache HTTP/2 server. The sequence diagram demonstrating the HTTP/2 slow-rate attack is shown in Fig. 2.

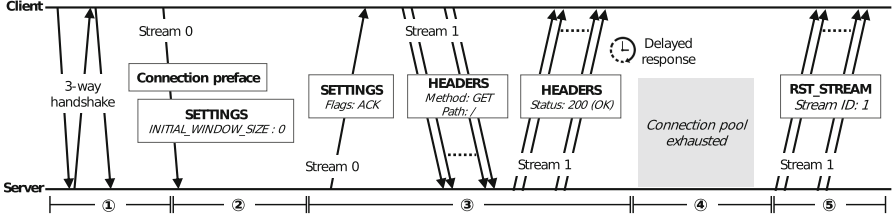


Fig. 2. Sequence diagram of HTTP/2 slow-rate attack

Slow-Rate Attack. A slow-rate attack [12] is a subcategory of DoS attacks, which is mainly known for its stealthier nature. This attack is one of the well-known HTTP/2 vulnerabilities, which is pointed out by various researches [2, 47–49, 59]. Unlike large-scale DDoS attacks that can easily be noticed, a slow-rate attack is hard to detect since it requires a small bandwidth and is similar to normal traffic. This attack proceeds according to the following steps:

- ① A TLS connection is established through a 3-way handshake, after which the client sends a connection preface on stream ID 0.
- ② Then it sends a **SETTINGS** frame that is deliberately crafted with an initial window size set to zero.
- ③ Following the receipt of another **SETTINGS** frame that includes the **ACK** flag, the attacker sends multiple **GET** requests using a **HEADERS** frame on a different stream, such as stream ID 1.
- ④ This leads to a communication deadlock, where the server is ready to send data but is artificially constrained due to the window size being set to zero, preventing any data transmission.
- ⑤ Consequently, the server keeps these connections open for an extended period, exhausting the connection pool until it terminates the stream with **RST_STREAM** frames, which could last longer than any configured timeout.

Proposed Approach. Our analysis of HTTP/2 DoS vulnerabilities emphasizes the importance of addressing flow control, stateful interactions, and scalability. Assuming an attacker can interact with the target software in parallel, they can exploit complexities in *flow control* with *stateful* messages that can strain the server. Vulnerabilities may vary across different server types and versions, necessitating a testing approach that is both flexible and *scalable*. We adopt a black box approach that realistically simulates attacks using real-world network services. The limitations of white box analysis, as demonstrated by SGFuzz [7], reinforce the strengths of the black box approach, which interactively captures complex state transitions including flow control. Our method not only overcomes the limitations of static analysis but also enhances the understanding of HTTP/2 vulnerabilities under practical attack scenarios.

4 Proposed Framework for HTTP/2 Stateful Fuzzing

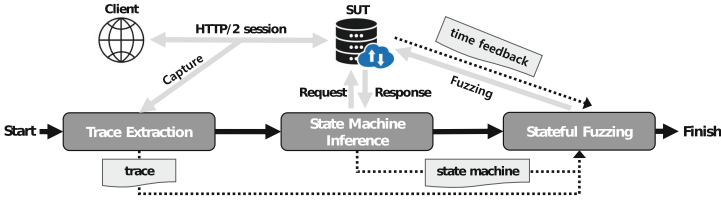


Fig. 3. PRETT2 framework

This section describes PRETT2, a stateful fuzzing framework designed for the automated detection of DoS vulnerabilities within diverse HTTP/2 implementations. Figure 3 illustrates the comprehensive methodology employed by PRETT2, which is designed to effectively identify and exploit vulnerabilities in HTTP/2 implementations through a three-phase process: trace extraction, state machine inference, and stateful fuzzing.

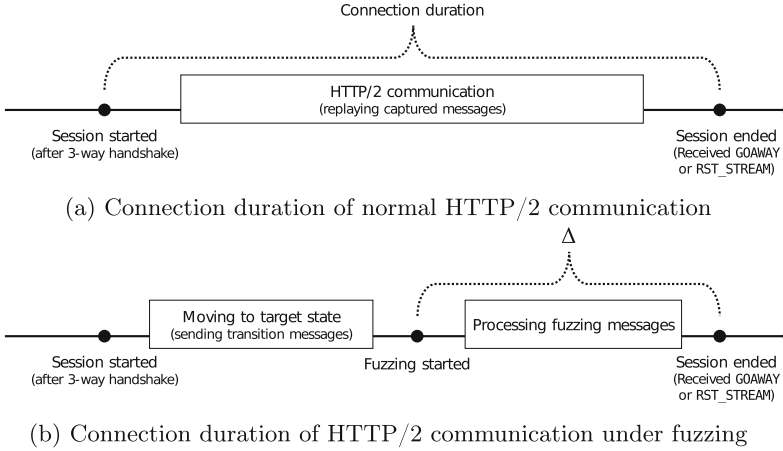


Fig. 4. Connection duration for time feedback

Improvements over PRETT. PRETT2 significantly advances beyond its predecessor, by tailoring its algorithm to address the specific challenges posed by the traits of HTTP/2. The following are several key distinctions between them:

- **Message complexity:** PRETT2 reflects both the binary frames and the order of messages in multiple streams. This differs from creating test cases by merging human-readable keywords and using a single connection.

- **Data sources:** PRETT2 analyzes protocol behavior through network traces alone, capturing both request and response times, without binary analysis.
- **Connection duration:** PRETT2 records the normal *connection duration* during state machine inference. It captures the timeframe from the start to the end of an HTTP/2 session (as shown in Fig. 4a) and uses this for its subsequent stateful fuzzing.

4.1 Trace Extraction

PRETT2 employs network traces recorded between a server under test (SUT) and a client for state machine inference and stateful fuzzing, using these traces as templates to generate fuzzing messages. By generating HTTP/2 traffic, the inherent feature of stream multiplexing is recorded, serving as a basis for the state machine inference that incorporates flow control and for the multiplexed fuzzing initiatives.

To efficiently capture these traces, an automated script can be utilized, combining the use of packet capture software like Wireshark with tshark command-line interface and a browser command for initiating traffic. An example of an automated capture script is shown in Listing 2. This script encapsulates the process of initiating the capture of HTTP/2 traffic using tshark, along with the concurrent initiation of a Chrome session directed to a specified HTTP/2-enabled server.

Listing 2. An example script for automated HTTP/2 traffic capture

```
$ tshark -i <interface> -Y "http2" -w captured_traffic.pcap &
  chrome --headless --disable-gpu --dump-dom https://<server_ip>
```

Captured traces are encrypted using the TLS protocol version 1.2 or higher for security reasons [45], limiting access to message details and complicating frame parameter manipulation for fuzzing. Thus, they need to be decrypted using a keylog file [44].

4.2 State Machine Inference

Given network traces, PRETT2 uses an interactive method of protocol reverse engineering that uses a request-and-response approach to infer state machines. PRETT2 utilizes the state machine inference algorithm presented in a previous study, PRETT [28], by repeating both state machine expansion and state machine minimization processes for each level based on a tree model.

State Machine Expansion. PRETT2 constructs a state machine based on a tree structure, in which the root node is considered an initial state. Assuming the maximum level of the tree is L , PRETT2 expands the tree to have as many nodes in the next level $L + 1$ as possible by sending candidate messages on every leaf node. The candidate messages in the traces are presented as request

messages without modification to their parameters. This prevents early rejection of malformed messages by the SUT. PRETT2 then checks the response messages with their response time and regards all the nodes expanded from the leaves as candidate nodes.

State Machine Minimization. After the state machine expansion is complete, PRETT2 eliminates duplicate candidate nodes in level $L + 1$ of the tree. To remove the duplicated nodes, PRETT2 conducts the compatibility test presented in the previous study [28]. The compatibility test finds redundant candidate nodes by determining either the compatibility or the incompatibility between nodes. In PRETT2, each candidate node is tested with other *valid* nodes based on a set of response time, request message, and response message. The valid nodes are the nodes proven to be states after the same process is conducted in the previous level.

End Condition. After all the candidate nodes are tested for compatibility with the other valid nodes, PRETT2 regards the candidate nodes that are proved to be unique (i.e., incompatible with any of the valid nodes) as a set of valid states found in the level $L + 1$. Each valid state is used as a leaf node for the state expansion of the next level. In particular, when no valid state is present as a result of state minimization in the level, no state expansion and minimization process needs to be performed and a minimized state machine that is composed of only valid states is returned.

4.3 Stateful Fuzzing

Given the state machine inferred in the previous step, PRETT2 generates test cases based on the request messages of traces. Specifically, PRETT2 mutates the variable fields of the HTTP/2 frames that exist in a message, considering the HTTP/2 stream multiplexing feature by simulating multiple interleaved streams in a single connection. Numerous test cases are transmitted to the SUT by traversing every transition in the state machine. Then PRETT2 takes time feedback from SUT in return, which is a new connection duration Δ (as shown in Fig. 4b). To determine the potential exhaustion of the connection pool, PRETT2 compares the connection duration Δ to that of normal HTTP/2 communication recorded in the previous step.

5 Evaluation

This section presents the experimental outcomes of protocol reverse engineering and stateful fuzzing carried out with PRETT2. We utilized network traces obtained from multiple web servers and browsers to ensure diversity, taking into account their popularity². PRETT2 leveraged these traces to conduct state

² For the usage share of HTTP/2 web servers and browsers, we referred to the statistics by W3Techs [52] and W3Schools [51], respectively.

machine inference and stateful fuzzing on the web servers. To ensure the reproducibility of our results, we excluded commercial implementations such as Cloudflare Server. For the reproducibility of our work, we published the source code for both the state machine inference and stateful fuzzing to the public³.

5.1 Experimental Setup

We ran a series of experiments involving protocol reverse engineering and stateful fuzzing on VMware virtual machines. Each virtual machine was configured identically, equipped with an Intel i5 CPU that has 12 logical cores running at 2.90 GHz, Ubuntu 18.04.6 LTS, 40 GB of SSD, and 4 GB of system memory. We captured network traces between a server and a client by configuring each server with the default settings for HTTP/2 and requesting an index page and a favicon from each client. Every server was set up so that resources for the index page were available, but those for the favicon were unavailable. Different HTTP/2 implementations were installed on each virtual machine to examine their behavior under identical conditions.

Table 4. Targets of HTTP/2 implementation

Servers				Browsers			
	Target	Release	Announced		Target	Release	Announced
Nginx	v1	1.14.0	06-Apr-18	Chromium	v1	62	31-Aug-17
	v2	1.21.6	25-Jan-22		v2	67	29-May-18
	latest	1.23.4	28-Mar-23		latest	111	01-Mar-23
Apache	v1	2.4.29	23-Oct-17	Firefox	v1	56	28-Sep-17
	v2	2.4.33	28-Mar-18		v2	60	09-May-18
	latest	2.4.56	07-Mar-23		latest	111	14-Mar-23
H2O	v1	2.2.4	15-Dec-17	Opera	v1	48	27-Sep-17
	v2	2.3.0-beta1	02-Jun-18		v2	53	10-May-18
	latest	2.3.0-beta2	14-Aug-19		latest	97	22-Mar-23

Target Selection. We assessed three HTTP/2 web servers - Nginx, Apache, and H2O. Nginx [19] is the most widely used web server for HTTP/2, accounting for approximately 34% of usage on millions of websites. Apache, also known as httpd [5], is the second most widely used, representing around 30% of usage. H2O [18] is used by fewer websites, but it is commonly employed by high-traffic

³ <https://github.com/choonginlee/PRETT2>.

sites. In contrast, we evaluated three HTTP/2 web browsers - Chromium [42], Firefox [31], and Opera [34]. We chose to test the versatility of state machine inference by arbitrarily selecting outdated versions (v1, v2) as well as the latest version⁴ of each implementation. We excluded extremely outdated versions that are not supported by the operating system. The selected HTTP/2 implementations are summarized in Table 4.

Control Variables. To minimize external factors that could affect the experiment’s outcome, we carefully configured the environment. First, we selected an operating system that supports all the various versions of multiple HTTP/2 implementations without switching kernels. Second, we experimented with a closed local area network (LAN) to prevent any potential interference from other network services. Third, we configured the HTTP/2 web servers to support only IPv4 and not to prefetch assets requested by a client. Finally, we ensured that the resources for the index page of each server were identical.

5.2 State Machine Inference

During our experiments, we recorded a total of 81 traces by combining nine servers and nine web browsers. Using PRETT2, we were able to successfully infer multiple state machines that effectively demonstrate the flow control process depending on three variables: server type, client type, and server version. We have demonstrated the representative state machines for Nginx and H2O in Fig. 5 and Fig. 6. For the Apache server, PRETT2 inferred different state machines for each server-browser combination. The state machines for Apache are more sophisticated than those for Nginx and H2O. Due to the page limitation, we have provided an example of an Apache state machine in the Appendix.

Results. We represent state machines from a client point of view in a directional graph. The edges specify a pair of sent-received messages (split by a slash) with one or more HTTP/2 frames, while the nodes specify the state. The dots(...) in the received messages indicate that no message has been received from a server. All states except the END state are subject to change to the END state after receiving a GOAWAY frame, which occurs when it reaches the pre-configured timeout. Numbers in square brackets indicate each stream ID on which its message is sent, and a bar(|) is used to differentiate multiple frames of a message in separate streams. To make it more readable, we shortened the name of each frame into two alphabetical letters⁵.

⁴ We determined the latest version of each implementation based on the official release at the time of the evaluation in March 2023.

⁵ ST: SETTINGS, WU: WINDOW_UPDATE, HD: HEADERS, DA: DATA, GO: GOAWAY, PR: PRIORITY, and RS: RST_STREAM.

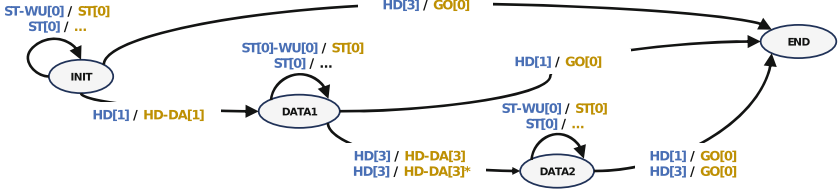


Fig. 5. State machine of Nginx and H2O (Chromium and Opera traces)

Analysis. Figure 5 illustrates a state machine that was inferred for Nginx and H2O servers. This was done using traces that recorded communication with Chromium and Opera browsers. On every state except the END state, both the client and server could configure the window size using `SETTINGS` and `WINDOW_UPDATE` frames. States DATA1 and DATA2 represent the states in which the client received and responded to a request for the first data (i.e., the index page) and second data (i.e., the favicon), respectively. It is noteworthy that data requests using different types of `HEADER` frames make different transitions on each state. For instance, servers responded quickly with a `GOAWAY` frame if the second data was requested on the INIT state. However, they responded with both `HEADER` and `DATA` frames when requested the same request on the DATA state. Interestingly, we observed slightly different responses to the request for the second data in state DATA1 when using the v1 version of H2O. This was due to the different sizes of header block fragments, which varied depending on the versions.

Figure 6 shows three different state machines inferred for either H2O or Nginx. This was done using traces that recorded communication with Firefox browsers. These state machines differ from the one shown in Fig. 5 due to the message difference in the traces. Three significant differences compared to other browsers are as follows. (1) Firefox reserved idle streams [54] for stream prioritization using `PRIORITY` frames⁶. (2) Firefox requested data using two frames, such as `HEADER` and `WINDOW_UPDATE`, in the same message. (3) Firefox sent a `GOAWAY` frame back to the server when it had received a `GOAWAY` frame.

H2O and Nginx worked differently from each other for the communication with the Firefox browser. For instance, Fig. 6a shows that H2O had a `CONFIGURED` state indicating that a server was requested for the configuration settings via `SETTINGS`, `WINDOW_UPDATE`, and `PRIORITY` frames for the first time. Additionally, H2O sent an `RST_STREAM` frame whenever a client requested data without such configuration settings. These differences were not seen in the case of Nginx (Fig. 6b). We could observe that the v1 version of H2O also worked differently (Fig. 6c) compared to other versions (Fig. 6b.) The state machine was

⁶ The `PRIORITY` frame reserved a total of six streams with stream IDs 3, 5, 7, 9, 11, and 13. This short frame is shown as `PR[(6)]` in the state machine. As a result, the first and second data requests were on stream IDs 15 and 17, respectively.

They presented a compact state machine for H2O using a state transition tree (STT), which represents the lifecycle of an HTTP/2 stream. To generate the state transition tree, SGFuzz refers to state variables that are assigned and compared to named constants using static resources of the HTTP/2 implementation. The state machine is inferred effectively because it contains specific information on the states that detail those in the HTTP/2 specification.

However, they did not successfully demonstrate flow control and multiplexed streams in practice: the state machines lack any information on configuration settings. On the other hand, PRETT2 is capable of inferring various state machines that demonstrate flow control messages on multiple streams. Specifically, the state machines contain the process of configuration settings via `SETTINGS` or `WINDOW_UPDATE`, which is done on stream ID 0, whereas requests for data are done on different stream IDs.

5.3 Security Analysis

As a result of fuzzing HTTP/2 servers using inferred state machines, PRETT2 could discover three Denial-of-Service attacks that exploit flow control flaws in both Apache HTTP servers (versions 2.4.46 and 2.4.58) and Nginx (version 1.25.3). The attacks were successful by sending multiple messages that exceeded the limit of the connection pool. We reported all the flaws to the developers and found that the attacks could be exploited when the server was configured by default without any external mitigation options.

Case Study. As a case study, we explain a vulnerability that we discovered via PRETT2. The vulnerability, known as CVE-2023-43622, was triggered by an implementation flaw in flow control for Apache httpd 2.4.58. To discover this vulnerability, we used PRETT2 to craft parameters of HTTP/2 frames with special values by fuzzing and assessing each state. Specifically, we sent a `SETTINGS` frame with a 0 `SETTINGS_INITIAL_WINDOW_SIZE` value and sent hundreds of data requests by traversing every possible state transition based on the state machine. This action fooled the server into believing that the client was not able to receive data and led to a slow-rate attack.

The attack was successful when we used a state machine based on communication traces between the server and the Chromium browser. However, we could not discover the vulnerability using the same parameters based on the communication traces with the Firefox browser. This is because Firefox sends `WINDOW_UPDATE` frames immediately after connection establishment, which convinces the web server that the client can receive the data. Instead, we were able to successfully exploit the vulnerability by manipulating the parameter `window_size_increment` of a `WINDOW_UPDATE` frame using fuzzed messages.

6 Related Work

HTTP/2 DoS. Various approaches have analyzed the security vulnerabilities of HTTP/2 DoS using manual analysis. Targeting the extended features of HTTP/2, H2DoS [29] observed that a malformed sequence of crafted frames can cause DoS. Imperva [24] tested the features of HTTP/2 and found several attack vectors for DoS. Also, HTTP/2 Distributed DoS (DDoS) attacks are presented by various literature. Adi et al. [3] presented an HTTP/2 DDoS attack model, where the server resources are exhausted with the large volume, but legitimate flash crowd traffic. Adi et al. [1] further discussed how to camouflage offending traffic, such as the traffic of the attack [3]. HTTP/2 Cannon [8] analyzed the HTTP/1.1 legacy functionalities of HTTP/2 and explored the DDoS risk posed by the upgraded functionality of the HTTP/2 protocol. HTTP/2 Tsunami [8] observed that an amplification DDoS attack is possible in a proxy environment through HPACK. The effects of a simple asymmetric DDoS attack on HTTP/1.1 and HTTP/2 servers are compared in another paper [38]. Although HTTP/2 performs better due to its performance improvements, it is found to be vulnerable to a new attack vector called the multiplexed asymmetric attack.

Slow-rate Attack. Security researchers focused on the slow-rate attack as a common attack for HTTP/2 DoS. Adi et al. [2] described how to launch several slow-rate attacks and investigated its effect. Tripathi et al. [48] proposed a feature-based statistical anomaly detection mechanism for the five types of attacks that they discovered, three of which are structurally comparable to known HTTP/1.1 attacks. Zhang et al. [59] also presented another type of slow-rate attack called the zAttack.

Protocol Reverse Engineering. Three types of protocol reverse engineering methods have been proposed for the inference of protocol communication. First, the static method clustered network traces to infer message formats and protocol states [4, 39, 46, 53]. For example, ReverX [4] generated a state machine through the process of merging and simplifying messages. Veritas [53] used statistical analysis by defining a probabilistic protocol state machine (P-PSM) which is a probabilistic generalization of the protocol state machine. Second, the dynamic method analyzed execution traces from implementations to understand communication rules and state machines [14, 56]. Xiao et al. [56] inferred an accurate state machine using the trial-and-error learning method for messages. Prospex [14] clustered protocol message sequences and analyzed their correlations. Lastly, the interactive method used a request-and-response approach using network traces to determine the correctness of messages and their sequences to

infer a state machine [13,27,60]. Cho et al. [13] and Zhang et al. [60] identified potential data message fields by sending and receiving messages as query strings.

HTTP/2 Fuzzing. Compared to the variety of methods for manual analysis, only a few automated testing methods (i.e., fuzzing) were presented. Http2fuzz [41] is the first fuzzer for testing HTTP/2 vulnerabilities presented by the Yahoo Pen-test Team. SGFuzz [7] presented an advanced fuzzing framework using stateful information. As discussed earlier, however, they are limited to addressing challenges for discovering HTTP/2 DoS due to the insufficient testing of flow control.

7 Concluding Remarks

The widespread adoption of HTTP/2 services by web servers has led to a steady increase in DoS vulnerabilities due to implementation flaws in HTTP/2 flow control. As a remedy, we proposed PRETT2, a stateful fuzzing framework designed to automatically analyze HTTP/2 DoS flaws. PRETT2 infers state machines using a novel method for automated protocol reverse engineering, successfully deriving various types depending on server type, client type, and server version. Utilizing these state machines, PRETT2 conducted stateful fuzzing on multiple HTTP/2 servers and discovered several DoS attacks exploiting flow control flaws. Despite our promising results, PRETT2 has limitations: the heavy dependence on network traces for the state machine and its focus solely on HTTP/2 flow control flaws. For future work, we aim to extend our research to include HTTP/3 testing, adapting our framework to tackle the evolving challenges in newer protocol versions. We believe that our research can significantly contribute to network security by mitigating DoS attacks on HTTP protocols as a proactive analysis method.

Acknowledgments. Special thanks to Taewoo Kim, whose contributions during the initial phase of this project helped lay the groundwork for our research. The authors thank the anonymous reviewers for their valuable comments. This work was supported by ICT Creative Consilience Program through the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. 2022-0-00277, Development of SBOM Technologies for Securing Software Supply Chains, No. 2022-0-01198, Convergence Security Core Talent Training Business (Korea University), and IITP-2024-2020-0-01819, ICT Creative Consilience program), PSC/CUNY Cycle 51 Software Security grant, and the Research Foundation of the City University of New York (RFCUNY).

Appendix

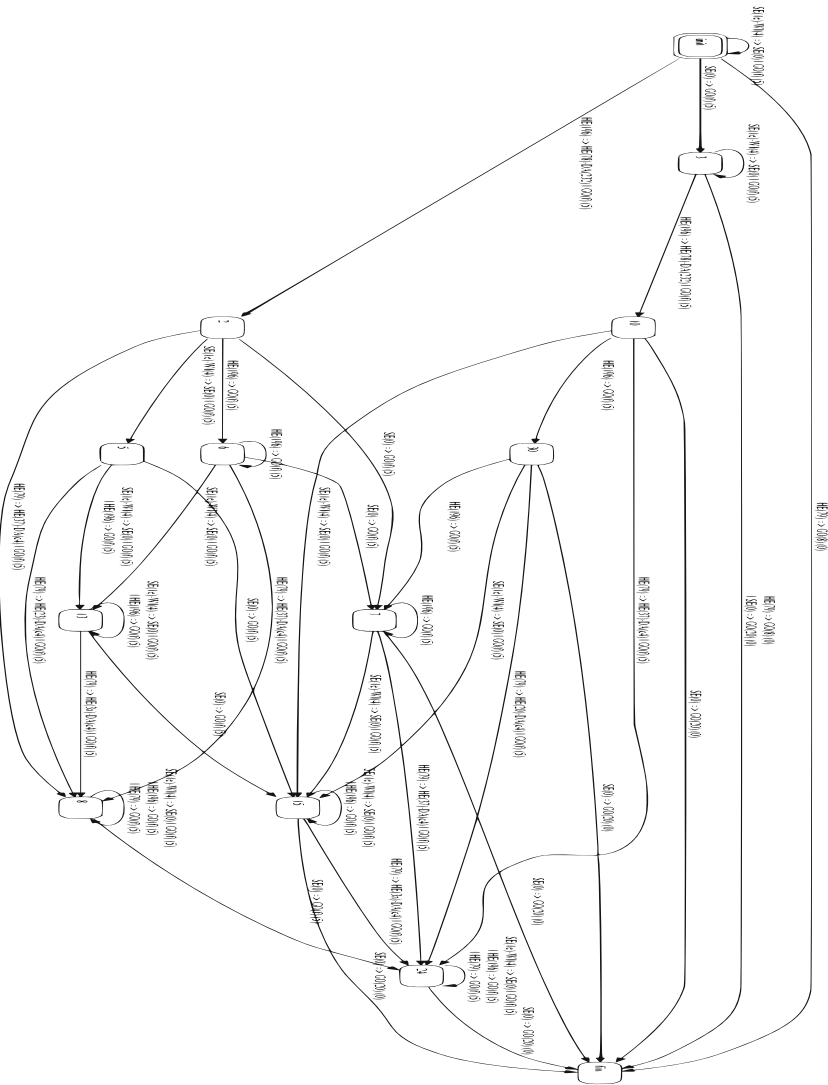


Fig. 7. The illustration of the state machine for Apache’s latest version, inferred using Chromium traces. While the detailed representation is optimized for a broad overview, it highlights the intricate complexity and extensive capabilities of Apache in contrast to the more simplified state machines of H2O and Nginx.

References

1. Adi, E., Baig, Z., Hingston, P.: Stealthy Denial of Service (DoS) attack modelling and detection for HTTP/2 services. *J. Netw. Comput. Appl.* **91**, 1–13 (2017)
2. Adi, E., Baig, Z., Lam, C.P., Hingston, P.: Low-rate denial-of-service attacks against HTTP/2 services. In: 5th IEEE International Conference on IT Convergence and Security (ICITCS), pp. 1–5 (2015)
3. Adi, E., Baig, Z.A., Hingston, P., Lam, C.P.: Distributed denial-of-service attacks against HTTP/2 services. *Clust. Comput.* **19**, 79–86 (2016)
4. Antunes, J., Neves, N., Verissimo, P.: Reverse engineering of protocols from network traces. In: 18th Working Conference on Reverse Engineering, pp. 169–178. IEEE (2011)
5. Apache Software Foundation: Apache HTTP Server. <https://httpd.apache.org/>
6. Athapathu, R.: HTTP/2 Flow Control (2019). <https://medium.com/coderscorner/http-2-flow-control-77e54f7fd518>
7. Ba, J., Böhme, M., Mirzamomen, Z., Roychoudhury, A.: Stateful Greybox Fuzzing. In: 31st USENIX Security Symposium, pp. 3255–3272 (2022)
8. Beckett, D., Sezer, S.: HTTP/2 Cannon: experimental analysis on HTTP/1 and HTTP/2 request flood DDoS attacks. In: Seventh IEEE International Conference on Emerging Security Technologies (EST), pp. 108–113 (2017)
9. Belson, D., Pardue, L.: Examining HTTP/3 usage one year on. Cloudflare (2023). <https://blog.cloudflare.com/http3-usage-one-year-on/>
10. Berners-Lee, T., Fielding, R., Frystyk, H.: RFC1945: Hypertext transfer protocol–http/1.0 (1996)
11. Caballero, J., Yin, H., Liang, Z., Song, D.: Polyglot: automatic extraction of protocol message format using dynamic binary analysis. In: 14th ACM Conference on Computer and Communications Security, pp. 317–329 (2007)
12. Cambiaso, E., Papaleo, G., Chiola, G., Aiello, M.: Slow DoS attacks: definition and categorisation. *Int. J. Trust Manage. Comput. Commun.* **1**, 300–319 (2013)
13. Cho, C.Y., Babić, D., Shin, E.C.R., Song, D.: Inference and analysis of formal models of botnet command and control protocols. In: 17th ACM conference on Computer and Communications Security, pp. 426–439 (2010)
14. Comparetti, P.M., Wondracek, G., Kruegel, C., Kirda, E.: Prospex: Protocol specification extraction. In: 30th IEEE Symposium on Security and Privacy, pp. 110–125 (2009)
15. Corbel, R., Stephan, E., Omnes, N.: HTTP/1.1 pipelining vs HTTP2 in-the-clear: performance comparison. In: 13th IEEE International Conference on New Technologies for Distributed Systems (NOTERE), pp. 1–6 (2016)
16. CVE-2016-1546. National Vulnerability Database (Jul 2016). <http://nvd.nist.gov/nvd.cfm?cvename=CVE-2016-1546>
17. De Ruiter, J., Poll, E.: Protocol state fuzzing of TLS implementations. In: 24th USENIX Security Symposium, pp. 193–206 (2015)
18. DeNA: H2O Web Server. <https://h2o.example.net/>
19. F5: Nginx. <https://nginx.org/>
20. Fielding, R., et al.: Hypertext transfer protocol–http/1.1. Tech. rep. (1999)
21. Gascon, H., Wressnegger, C., Yamaguchi, F., Arp, D., Rieck, K.: Pulsar: stateful black-box fuzzing of proprietary network protocols. In: International Conference on Security and Privacy in Communication Systems, pp. 330–347 (2015)
22. Guo, R., et al.: CDN Judo: breaking the CDN DoS protection with itself. In: Network and Distributed System Security Symposium (2020)

23. IETF HTTP Working Group: List of known HTTP/2 implementations (2020). <https://github.com/httpwg/http2-spec/wiki/Implementations>
24. Imperva: HTTP/2: In-depth analysis of the top four flaws of the next generation web protocol (2017). https://www.imperva.com/docs/Imperva_HII_HTTP2.pdf
25. Jiang, M., Luo, X., Miu, T., Hu, S., Rao, W.: Are HTTP/2 servers ready yet? In: 37th IEEE International Conference on Distributed Computing Systems (ICDCS), pp. 1661–1671 (2017)
26. Kaloper-Meršinjak, D., Mehnert, H., Madhavapeddy, A., Sewell, P.: Not-Quite-So-Broken TLS: Lessons in re-engineering a security protocol specification and implementation. In: 24th USENIX Security Symposium, pp. 223–238 (2015)
27. LaRoche, P., Burrows, A., Zincir-Heywood, A.N.: How far an evolutionary approach can go for protocol state analysis and discovery. In: IEEE Congress on Evolutionary Computation, pp. 3228–3235 (2013)
28. Lee, C., Bae, J., Lee, H.: PRETT: protocol reverse engineering using binary tokens and network traces. In: IFIP International Conference on ICT Systems Security and Privacy Protection, pp. 141–155 (2018)
29. Ling, X., Wu, C., Ji, S., Han, M.: H_2 DoS: an application-layer DoS attack towards HTTP/2 protocol. In: International Conference on Security and Privacy in Communication Systems, pp. 550–570 (2017)
30. Mirković, J., Dietrich, S., Dittrich, D., Reiher, P.: Internet denial of service: attack and defense mechanisms. Prentice Hall PTR (2004)
31. Mozilla Foundation: Mozilla Firefox. <https://www.mozilla.org/firefox/>
32. Narayan, J., Shukla, S.K., Clancy, T.C.: A survey of automatic protocol reverse engineering tools. ACM Comput. Surv. (CSUR) **48**, 1–26 (2015)
33. Nkuba, C.K., Kim, S., Dietrich, S., Lee, H.: Riding the Iot wave with vfuzz: discovering security flaws in smart homes. IEEE Access **10**, 1775–1789 (2021)
34. Opera Software: Opera Browser. <https://www.opera.com/>
35. Park, H., Nkuba, C.K., Woo, S., Lee, H.: L2fuzz: discovering bluetooth l2cap vulnerabilities using stateful fuzz testing. In: 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), pp. 343–354 (2022)
36. Pham, V.T., Böhme, M., Roychoudhury, A.: AFLNET: A greybox fuzzer for network protocols. In: IEEE 13th International Conference on Software Testing, Validation and Verification (ICST), pp. 460–465 (2020)
37. Poll, E., De Ruiter, J., Schubert, A.: Protocol state machines and session languages: specification, implementation, and security flaws. In: IEEE Security and Privacy Workshops, pp. 125–133 (2015)
38. Praseed, A., Thilagam, P.S.: Multiplexed asymmetric attacks: Next-generation DDoS on HTTP/2 servers. IEEE Trans. Inf. Forensics Secur. **15**, 1790–1800 (2019)
39. Shevertalov, M., Mancoridis, S.: A reverse engineering tool for extracting protocols of networked applications. In: 14th Working Conference on Reverse Engineering (WCRE), pp. 229–238 (2007)
40. Stenberg, D.: HTTP2 Explained. ACM SIGCOMM Comput. Commun. Rev. **44**, 120–128 (2014)
41. Stuart Larsen, J.V.: Attacking HTTP/2 Implementations. Yahoo pen-test team (2015). <https://www.slideshare.net/JohnVillamil/attacking-http2-implementations-1>
42. The Chromium Projects: Chromium. <https://www.chromium.org/>
43. The Chromium Projects: SPDY: An experimental protocol for a faster web (2009). <https://www.chromium.org/spdy/>

44. Thomson, M.: The SSLKEYLOGFILE Format for TLS. Tech. rep., Internet Engineering Task Force (Apr 2024). <https://datatracker.ietf.org/doc/draft-ietf-tls-keylogfile/01/>
45. Thomson, M., Benfield, C.: HTTP/2. RFC 9113 (Jun 2022). <https://doi.org/10.17487/RFC9113>, <https://www.rfc-editor.org/info/rfc9113>
46. Trifilò, A., Burschka, S., Biersack, E.: Traffic to protocol reverse engineering. In: IEEE Symposium on Computational Intelligence for Security and Defense Applications, pp. 1–8 (2009)
47. Tripathi, N.: Delays have Dangerous Ends: Slow HTTP/2 DoS attacks into the Wild and their Real-Time Detection using Event Sequence Analysis. IEEE Transactions on Dependable and Secure Computing (2023)
48. Tripathi, N., Hubballi, N.: Slow rate denial of service attacks against HTTP/2 and detection. Comput. Security **72**, 255–272 (2018)
49. Tripathi, N., Shaji, A.K.: Defer no time, delays have dangerous ends: slow HTTP/2 DoS attacks into the wild. In: 14th IEEE International Conference on COMmunication Systems and NETworkS (COMSNETS), pp. 194–198 (2022)
50. Beckett, D., Sezer, S.: HTTP/2 Tsunami: Investigating HTTP/2 proxy amplification DDoS attacks. In: Seventh IEEE International Conference on Emerging Security Technologies (EST). pp. 128–133 (2017)
51. W3Schools: The most popular browsers (Jan 2024). <https://www.w3schools.com/browsers>
52. W3Techs: Web servers market position report (Jan 2024). https://w3techs.com/technologies/market/web_server
53. Wang, Y., Zhang, Z., Yao, D.D., Qu, B., Guo, L.: Inferring protocol state machine from network traces: a probabilistic approach. In: International Conference on Applied Cryptography and Network Security, pp. 1–18 (2011)
54. Wijnants, M., Marx, R., Quax, P., Lamotte, W.: HTTP/2 prioritization and its impact on web performance. In: Proceedings of the 2018 World Wide Web Conference, pp. 1755–1764 (2018)
55. Wondracek, G., Comparetti, P.M., Kruegel, C., Kirda, E.: Automatic network protocol analysis. In: Network and Distributed System Security Symposium (2008)
56. Xiao, M.M., Yu, S.Z., Wang, Y.: Automatic network protocol automaton extraction. In: Third International Conference on Network and System Security, pp. 336–343 (2009)
57. Yahia, M.B., Le Louedec, Y., Simon, G., Nuaymi, L.: HTTP/2-Based streaming solutions for tiled omnidirectional videos. In: IEEE International Symposium on Multimedia (ISM), pp. 89–96 (2018)
58. Yahia, M.B., Louedec, Y.L., Simon, G., Nuaymi, L., Corbillon, X.: HTTP/2-based Frame discarding for low-latency adaptive video streaming. ACM Trans. Multimedia Comput., Commun. Appl. (TOMM) **15**, 1–23 (2019)
59. Zhang, Y., Shi, Y.: A slow rate denial-of-service attack against HTTP/2. In: IEEE 4th International Conference on Computer and Communications (ICCC), pp. 1388–1391 (2018)
60. Zhang, Z., Wen, Q.Y., Tang, W.: Mining Protocol State Machines by Interactive Grammar Inference. In: Third IEEE International Conference on Digital Manufacturing and Automation, pp. 524–527 (2012)



TurboTLS: TLS Connection Establishment with 1 Less Round Trip

Carlos Aguilar-Melchoro¹, Thomas Bailleux¹, Jason Goertzen^{1,2(✉)},
Adrien Guinet¹, David Joseph¹, and Douglas Stebila²

¹ SandboxAQ, Palo Alto, USA

{carlos.aguilar-melchoro,thomas.bailleux,jason.goertzen,
adrien.guinet,david.joseph}@sandboxaq.com

² University of Waterloo, Waterloo, Canada

{jason.goertzen,douglas.stebila}@uwaterloo.ca

Abstract. We show how to establish TLS connections using one less round trip. In our approach, which we call TurboTLS, the initial client-to-server and server-to-client flows of the TLS handshake are sent over UDP rather than TCP. At the same time, in the same flights, the three-way TCP handshake is carried out. Once the TCP connection is established, the client and server can complete the final flight of the TLS handshake over the TCP connection and continue using it for application data. No changes are made to the contents of the TLS handshake protocol, only its delivery mechanism. We avoid problems with UDP fragmentation by using *request-based fragmentation*, in which the client sends in advance enough UDP requests to provide sufficient room for the server to fit its response with one response packet per request packet. Clients can detect which servers support this without an additional round trip, if the server advertises its support in a DNS HTTPS resource record. Experiments using our software implementation show substantial latency improvements. On reliable connections, we effectively eliminate a round trip without any noticeable cost. To ensure adequate performance on unreliable connections, we use lightweight packet ordering and buffering; we can have a client wait a very small time to receive a potentially lost packet (e.g. a fraction of the RTT observed for the first fragment) before falling back to TCP without any further delay, since the TCP connection was already in the process of being established. This approach offers substantial performance improvements with low complexity, even in heterogeneous network environments with poorly configured middleboxes.

1 Introduction

The Transport Layer Security (TLS) protocol is ubiquitous and provides security services to many network applications. TLS runs over TCP. As shown in Fig. 1(a), the main flow for TLS 1.3 connection establishment [Res18] in a web browser is as follows.

First, the client makes a DNS query to translate the provided domain name into an IP address. Modern browsers simultaneously request from the DNS server

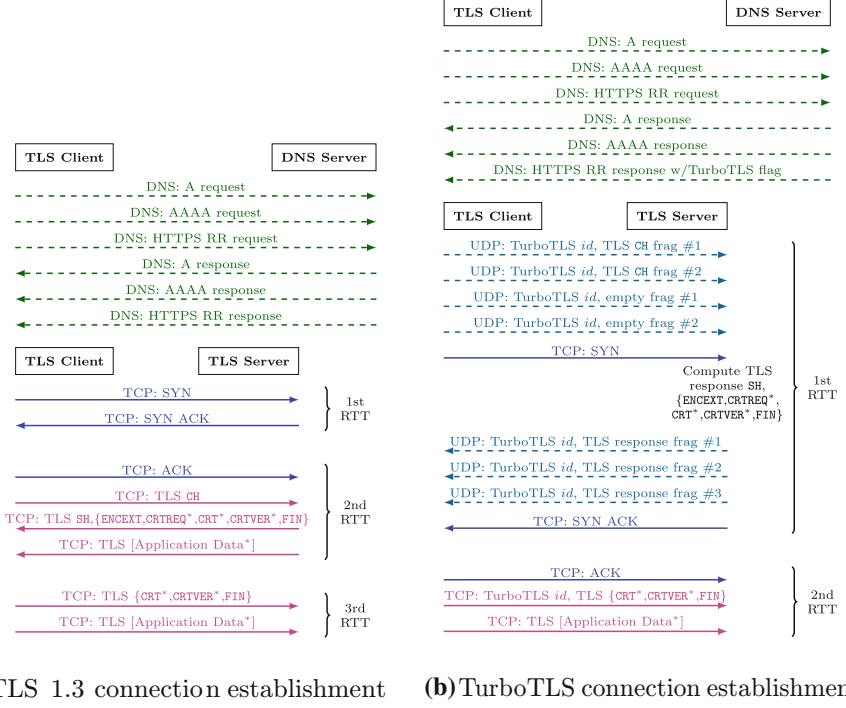


Fig. 1. Comparing message flow of TLS 1.3 and TurboTLS. Legend: dashed line - UDP; solid line - TCP. * denotes optional message. {...} denotes messages encrypted using TLS handshake traffic secret and [...] denotes messages encrypted using TLS application traffic secret.

an HTTPS resource record [SBN22] which can provide additional information about the server’s HTTPS configuration. Next, the client performs the TCP three-way handshake with the server. Once the TCP handshake has completed and a TCP connection is established, the TLS handshake can begin; it requires one client-to-server (C→S) flow and one server-to-client (S→C) flow before the client can start sending application data.

In total, excluding the DNS resolution, this results in two round trips before the client can send its first byte of application data (the TCP handshake and the first C→S and S→C flows of the TLS handshake), and another round trip before the client receives its first byte of response.

TLS does have a pre-shared key mode that allows for an abbreviated handshake permitting application data to be sent in the first C→S TLS flow, but this requires that the client and server have a pre-shared key in advance, established either through some out-of-band mechanism or saved from a previous TLS connection for session resumption.

Our Contributions. We describe a method, which we call TurboTLS, for removing one round trip of latency from TLS connection establishment by transmitting the first two flows of the TLS handshake over UDP while doing the TCP three-way handshake in parallel, then switching over to the TCP connection for the final C→S handshake flow and the transmission of application data. The message flow of TurboTLS is shown in Fig. 1(b). It allows the client to start sending its first byte of application in just one round trip (excluding the DNS resolution), without requiring any pre-shared key. TurboTLS does not require any change to the contents or state machine of the TLS protocol: it only changes the network delivery mechanism. We employ several techniques to make TurboTLS operate smoothly in a heterogeneous network environment where there may be UDP packet loss, where some servers may not support TurboTLS, and where intermediary network devices may have trouble with UDP fragmentation. We implement TurboTLS and compare its network performance with TLS 1.3 and QUIC over local, national, and intercontinental distance connections, as shown in Fig. 3 and Table 2. In general, TurboTLS achieves median latency approximately the same as QUIC, which is a 50% improvement compared to TLS 1.3 in long-distance connections, and slightly less in short-distance connections to fixed overhead.

Table 1 summarizes the characteristics of TurboTLS compared with other relevant network security protocols; see Sect. 2 and Sect. 4.1 for more details.

While our primary focus is on TLS connection establishment over TCP, many other network protocols have a similar structure. Our techniques can be generalized to an approach we call “turbo transport” which accelerates connection establishment by shifting portions of the connection establishment to UDP.

2 Background

An application wishing to establish a secure connection between a client and a server will select a protocol, or combination of protocols across network layers, depending on a number of factors. Depending on the application (performance requirements, reliability requirements, etc.) there may be a preference for a connection-oriented or connectionless protocol. Finally, if a server does not support a protocol, or an initial request is blocked due to other reasons such as firewall filtering, then the client may need to fall back to another protocol that reaches the server and is supported.

In this section, we review several existing options to set up application-level secure channels, focusing on the TLS protocol, variants of TLS, and protocols aiming to replace TLS. Our first-level categorization is whether the protocol runs over TCP or UDP.

2.1 Secure Channel Protocols over TCP

Applications requiring connection-oriented communication typically run over TCP, such as ‘vanilla’ TLS. TCP uses an initial round trip to set up the connection, using the TCP three-way handshake, then a further round trip is needed

Table 1. Characteristics of TurboTLS compared to TLS and other optimized/accelerated protocols and variants.

	Runs over	UDP 1 req. $\Rightarrow$ 1 resp.	Provides conn.	Kernel netw.	No state	TLS-based	Widely deployed	RTT to 1st byte
TLS 1.2	TCP	—	●	●	●	●	●	3
TLS 1.2 FalseStart	TCP	—	●	●	●	●	●	2
TLS 1.3	TCP	—	●	●	●	●	●	2
TLS 1.3 PSK	TCP	—	●	●	⊖	●	●	1
TLS 1.3 ECH	TCP	—	●	●	⊖	●	⊖	1
OPTLS	TCP	—	●	●	⊖	●	⊖	1
TLS 1.3 + TCP Fast Open	TCP	—	●	●	⊖	●	⊖	1
DTLS 1.3	UDP	●	⊖	●	●	●	●	2
QUIC	UDP	⊖	●	⊖	●	⊖	⊖	1
MinimalLT	UDP	●	●	⊖	●	⊖	⊖	1
MinimalLT with state	UDP	●	●	⊖	⊖	⊖	⊖	0
TurboTLS	UDP+TCP	●	●	●	●	●	—	1
TurboTLS + PSK	UDP+TCP	●	●	●	⊖	●	—	0
TurboTLS + ECH	UDP+TCP	●	●	●	⊖	●	—	0

Legend: ●: yes; ⊖, ⊖, ⊖: partial; ⊖: no; —: not applicable. **Columns:** UDP 1 req. $\Rightarrow$ 1 resp.: does each UDP request packet lead to at most one response packet? Provides conn.: does the protocol provide connection-oriented (reliable, in-order) transport to the application? Kernel netw.: are connection-oriented features generally implemented in the kernel? No state: can full optimization be achieved without pre-shared state between client and server? RTT to 1st byte: how many round trips required until the client can send its first application byte, including TCP 3-way handshake if necessary.

to complete the TLS 1.3 handshake (or two further round trips for TLS 1.2 [RD08]), during which the cryptographic parameters are negotiated, session keys are exchanged, and authentication happens.

While the number of rounds trips and the resulting inherent latency is not always a problem for clients/servers in close proximity to one another, this presents a significant inconvenience where parties are far apart or suffer high network latency. To ameliorate this issue, a series of optimizations to TLS have been proposed, using a range of approaches.

Data-Based Optimizations. Some optimizations reduce the amount of data transmitted without reducing the number of round trips. Perhaps the simplest approach is that of Compact TLS [RBTS22], which changes the format of TLS handshake messages by removing obsolete fields and defining profiles of common options. Another light-touch optimization is TLS Cached Information Extension [ST16], which allows clients and servers to indicate they already have certain sets of values, such as intermediate certificates, to avoid re-transmitting them, which can save a significant amount of bandwidth, especially in the context of post-quantum cryptography which typically results in larger intermediate certificates [SKD20].

Optimizations Using Previous State. Some optimizations are possible if the client has some prior server-dependent state, either from a previous connection or from some public directory.

TLS has a pre-shared key (PSK) mode in which a client can make use of a pre-shared symmetric key to save one round trip, allowing a client to start

sending application data in the first TLS 1.3 PSK flow (and thus on the second C→S flow including the TCP three-way handshake).

TCP Fast Open [CCRJ14] allows a client to save a cryptographic cookie from a previous TCP connection and use it in a subsequent TCP connection to immediately start sending application data without having to do a TCP three-way handshake on the subsequent connection. TLS running over TCP Fast Open would obviously then save one round trip.

OPTLS [KW16] was an alternative design for the TLS 1.3 handshake, running over TCP, which supported a so-called 0-RTT mode allowing for a client to send application data in its first C→S TLS flow (and thus on the second C→S flow including the TCP three-way handshake) provided that the client had previously cached or obtained out-of-band the server’s semi-static public key.

Encrypted Client Hello [ROSW22] is a proposed TLS extension that enables the client to encrypt more of the `ClientHello` message as well as send early application data in the first TLS C→S flight (and thus on the second C→S flow including the TCP three-way handshake) provided the client (similarly to in OPTLS) has previously cached or obtained out-of-band a public key of the server (which could be distributed in a DNS record).

There were also several other modifications to TLS 1.2 that made use of previous state, including TLS Snap Start [Lan10] and “fast-track” client-side caching [SBR04].

2.2 Secure Channel Protocols over UDP

Another branch of optimizations utilizes the connectionless properties of UDP to fast-track performance. However, since many applications need to connection-oriented channels for data transmissions, most optimizations running on top of UDP specify their own procedures for packet reordering, packet loss, and session management, although we first briefly discuss DTLS, which does not.

Protocols not Providing Connection-Oriented Features to Applications. DTLS [RTM22] runs exclusively over UDP, including for transmission of application data. Because it runs on UDP, it is left to the application to reorder packets and deal with loss. Cryptographically, DTLS is based on TLS. DTLS can be particularly useful when trying to avoid problems such as TCP meltdown, whereby applications may be trying to transport TCP traffic inside a secure tunnel which also runs on TCP, essentially stacking TCP upon TCP and thereby amplifying the occurrence of TCP timeout and other related problems. For this reason DTLS is often used for VPN applications.

Protocols Providing Connection-Oriented Features to Applications. MinimalT [PZS+13] runs exclusively over UDP and uses a completely different protocol design compared to TLS, but has not, to date, seen widespread usage.

QUIC [IT21] is another approach. Designed originally to improve performance of encrypted transport for Google’s internal services, QUIC is an ambitious and completely separate connection-oriented protocol running on top of

UDP. Like MinimaLT, QUIC fundamentally merges the transport and security layers, and provides many other protocol-specific optimizations, such as providing varying header lengths (a longer header format is used for packets establishing connections), ACK-based packet loss detection which overcomes the instability of UDP by providing a grace period to in-flight packets, packet re-ordering, and others. A further benefit of QUIC is that for re-established connections, it is possible to send encrypted application data in the first packet by re-using previously agreed cryptographic parameters and utilizing a pre-shared key setup, using a technique similar to those of OPTLS, and again at the cost of forward secrecy for the initial data sent.

3 TurboTLS Design

As described in Fig. 1(b), TurboTLS sends part of the TLS handshake over UDP, rather than TCP. Switching from TCP to UDP for handshake establishment means we cannot rely on TCP’s features, namely connection-oriented, reliable, in-order delivery. However, since the rest of the connection will still run over TCP and only part of the handshake runs over UDP, we can reproduce the required functionality in a lightweight way without adding latency and allowing for a simple implementation.

Fragmentation. One of the major problems to deal with is that of fragmentation. TLS handshake messages can be too large to fit in a single packet – especially with long certificate chains or if post-quantum algorithms are used.

Obviously the client can fragment its first C→S flow across multiple UDP packets. To allow a server to link fragments received across multiple UDP requests, we add a 12-byte connection identifier field, containing a client-selected random value *id* that is used across all TurboTLS fragments sent by the client. The connection identifier is also included in the first message on the established TLS connection to allow the server to link together data received on the UDP and TCP connections. To allow the server to reassemble fragments if they arrive out-of-order, each fragment includes the total length of the original message as well as the offset of the current fragment; this can allow the server to easily copy fragments into the right position within a buffer as they are received.

Similarly, the server can fragment its first S→C flow across multiple UDP packets. One additional problem here however is that the S→C flow is typically larger than the C→S flow (as it typically contains one or more certificates), so the server may have to send more UDP response packets than UDP request packets. As noted by [SW19] in the context of DNSSEC, many network devices do not behave well when receiving multiple UDP responses to a single UDP request, and may close the port after the first packet, dropping the request. Subsequent packets received at a closed port lead to ICMP failure alerts, which can be a nuisance.

We employ a recent method proposed by Goertzen and Stebila [GS22] for DNSSEC: request-based fragmentation. In the context of large resource records

in DNSSEC, [GS22] had the first response be a truncated response that included information about the size of the response, and then the client sent multiple additional requests, in parallel, for the remaining fragments. This ensured that there was only one UDP response for each UDP request. We adapt that method for TurboTLS: the client, in its first C→S flow, fragments its own C→S data across multiple UDP packets, and additionally sends (in parallel) enough nearly-empty UDP requests for a predicted upper bound on the number of fragments the server will need to fit its response. This preserves the model of each UDP request receiving a single UDP response, reducing the impact of misbehaving network devices and also reducing the potential for DDoS amplification attacks.

Reliability. UDP does not have reliable delivery, so packets may be lost. Since the first TurboTLS round-trip includes the TCP handshake, we can immediately fall back to TCP if a UDP packet is lost in either direction. This will induce a latency cost of however long the client decides to wait for UDP packets to arrive before giving up and assuming they were lost.

In an implementation, the client delay could be a fixed number of milliseconds, or could be variable depending on observed network conditions; this need not be fixed by a standard. We believe that in many cases a client delay of just 2ms after the TCP reply is received in the first round trip will be enough to ensure UDP responses are received a large majority of the time. In other words, by tolerating a potential 2ms of extra latency on $X\%$ of connections, we can save an entire round-trip on a large proportion ($100 - X\%$) of the connections. This mechanic was not implemented in the experimental results presented here and constitutes future work.

Advertising Support. To protect servers who do not support TurboTLS from being bombarded with unwanted UDP traffic, it would be preferable if clients only used TurboTLS with servers that they already know support it. Clients could cache this information from previous non-TurboTLS connections, but in fact we can do better. Even on the first visit to a server, we can communicate server support for TurboTLS to the client, without an extra round trip, using the HTTPS resource record in DNS [SBN22]. Today when web browsers perform the DNS lookup for the domain name in question, they typically send three requests in parallel: an A query for an IPv4 address, an AAAA query for an IPv6 address, and a query for an HTTPS resource record [SBN22]. Servers can advertise support for TurboTLS with an additional flag in the HTTPS resource record and clients can check for it without incurring any extra latency.

4 Features and Advantages

4.1 Comparison with Other Protocols

The protocols presented in Sect. 2 fall in one or more of the following categories:

- doing more than one round trip (TLS 1.2, TLS 1.2 FalseStart, TLS 1.3, DTLS 1.3, Compact TLS and TLS Cached Information Extension);

- modifying TCP/UDP directly or modify the way TCP/UDP are expected to be used (TLS 1.3 + TCP Fast Open, DTLS, MinimaLT, QUIC); or
- maintaining a state (TLS 1.3 + TCP Fast Open, TLS 1.3 PSK, OPTLS, TLS Encrypted Client Hello).

TurboTLS requires one round trip, uses TCP and UDP without modifying them, and as middleboxes would expect them to be used, and does not require a state. The rest of this section is dedicated to explaining the drawbacks of falling into one of the three categories above. We include a comparison with alternative protocols, but for a more detailed comparison with QUIC, which is gaining significant traction, we refer the reader to Sect. 7.

One Round Trip. Doing more than one round trip increases latency by at least one RTT. As already noted, global studies provided to the community by CAIDA [CAI] show that the RTT for median connections is between 50 and 200ms, mainly due to hops and distance, and if we consider the 90th percentile of connections, RTTs are beyond 500ms. This latency introduction is amplified by an integer factor for protocols in which connections occur sequentially (e.g. get a web page, get frames in the page, get images in the frames). Besides user experience, this also has an impact on usual implementations in which a server thread from a pool will not come back to the pool until it finishes dealing with a client. For connections with an RTT over a few milliseconds, when replacing TLS 1.3 by TurboTLS, we indeed observed a multiplication by two of the maximum handshakes per second that could be handled by a simple, yet usual, server implementation (thread pool with as many threads as cores that asynchronously handles connections).

Standard and Expected Usage of TCP/UDP. Some protocols modify TCP/UDP or the way they are expected to be used. Of course, TCP Fast Open modifies TCP itself by introducing cookies for the first flight (which also requires a state). This probably explains why, even if the initial proposal is from 2011, it is still an experimental RFC and not enabled by default on most browsers. Other protocols, like DTLS, MinimaLT or QUIC, just use UDP, without modifying it, but do long-term bidirectional exchanges, which is not the usual for UDP. Long-term bidirectional exchanges are in general done over TCP, and most protocols using UDP either follow the one query/one reply model (e.g. DNS) or the one query/many replies model (e.g. FTP download).

Using UDP for long-term bidirectional exchanges introduces two issues: instability and computational overhead. The main and simplest reason for UDP instability are firewall rules which often block such traffic, except for the one query/one reply model. Besides that, some middlebox functions for long term bidirectional traffic are only available for TCP and with UDP will either reduce performance or cause instability. For example, in the context of QUIC, extensive guidance [KT22] is required to deal with usual traffic management over QUIC, with the RFC addressing issues such as: “Passive Network Performance Measurement and Troubleshooting; Stateful Treatment of QUIC Traffic; Address

Rewriting to Ensure Routing Stability; Server Cooperation with Load Balancers; Filtering Behavior; UDP Blocking, Throttling, and NAT Binding; DDoS Detection and Mitigation; Quality of Service Handling and ECMP Routing;” and more. UDP bidirectional long-term exchanges that run through middleboxes that have not implemented these features (adapted to QUIC or to other protocols) will suffer from a lower quality of service and stability, sometimes with catastrophic effects [CSM+22]. On top of that, when connection-oriented features are provided (e.g. by QUIC or MinimaLT), one general drawback is that the implementation of the protocol needs to provide for packet reordering and recovery from packet loss, in user-space, whereas protocols running over TCP receive that for free from the operating system’s kernel-space TCP implementation, which has typically been highly tuned over many years, leading among other things to fewer interrupts and copies.

No State. Maintaining a state brings obvious issues (no benefit on first connection, lifetime, complexity) but most importantly, in the case of TLS, it also induces in general a loss of forward secrecy and thus of security. TLS 1.3 in pre-shared key (PSK) mode gains one round-trip only when it completely drops forward secrecy (as it relies on a pre-shared secret). This is somewhat mitigated by OPTLS and the Encrypted Client Hello TLS extension: if the client has previously obtained the server public key, then use of OPTLS and Encrypted Client Hello TLS extension are improved by one round-trip, and the loss of forward secrecy only affects the first flow of messages from the client to the server. The rest of the communication has forward secrecy.

No TLS Changes. Additionally, TurboTLS shares, with TLS 1.3 + TCP Fast Open, another nice feature: TurboTLS makes no change whatsoever to the content of a TLS handshake, only changes the delivery mechanism. As a result, all cryptographic properties of TLS are untouched. In fact, it is possible to implement TurboTLS without changing the client or server’s TLS library at all, and instead use transparent proxies on both the client and server side to change the network delivery from pure TCP in TLS to UDP+TCP in TurboTLS. Of course in such a construction the initial client or server, who does not know TurboTLS, will observe two round trip times, but if each proxy is close to its host (say on the same machine), then the two round trip times will be negligible, and the higher latency client–server distance will only be covered over one round trip.

4.2 Denial-of-Service (DoS) Considerations

We now consider the implications for TurboTLS of various types of denial-of-service and distributed denial-of-service attacks, including whether a TurboTLS server is a victim in a DoS attack or being leveraged by an attacker to direct a DDoS attack elsewhere. TurboTLS runs on top of both TCP and UDP so we have to consider attacks involving both protocols.

DoS Attacks on TurboTLS Servers. The most significant TCP DoS attack is the SYN flood attack where a target machine is overwhelmed by TCP SYN messages faster than it can process them. This is because a server, upon receiving a SYN, typically stores the source IP, TCP packet index number, and port in a ‘SYN queue’, and this represents a half-open connection. An attacker could flood the server with SYN messages thereby exhausting its memory. The server cannot just arbitrarily drop connections because then legitimate users may find themselves unable to connect. There are many protections against SYN flood attacks, one of which is allocating only very small amounts (micro blocks) of memory to half-open connections. Another is using TCP cryptographic cookies [Ber, Sim11] whereby the sequence number of the ACK encodes information about the SYN queue entry so that the server can reconstruct the entry even if it was not stored due to having a full SYN queue. TCP cookies enjoy support in the Linux kernel – this and other such mitigations are already sufficient to protect TurboTLS from SYN floods.

In general there are several vectors to consider for resource exhaustion attacks on a server running TurboTLS. The server needs to maintain a buffer of received UDP packets containing fragments of a TLS `ClientHello` message. To avoid memory exhaustion attacks, a server can safely bound the memory allocated to this buffer and flush old entries on a regular basis (e.g. after two seconds). In the worst case, a legitimate client whose UDP packets are rejected from a busy server or flushed early will be able to fall back to vanilla TLS over TCP, and will incur negligible latency loss (compared to TLS over TCP) in doing so, because TurboTLS starts the TCP handshake in parallel to the first C→S UDP flow. An attacker spoofing IP addresses and sending well-formed `ClientHello` messages could also try to exhaust a server’s CPU resources by causing a large amount of cryptographic computation. Again, a server under attack can limit the CPU resources allocated to UDP-received `ClientHello` messages, and then fall back to vanilla TLS over TCP. In the worst case, legitimate clients affected by this and having to fall back to vanilla TLS over TCP will incur negligible latency loss compared to TLS over TCP since the TCP handshake has already been started in parallel.

DDoS Attacks Leveraging TurboTLS Servers. UDP reflection attacks present another threat. Typical defenses against these are blocking unused ports, rate limiting based on expected traffic loads from peers (exorbitant traffic loads are likely to be malicious), or blocking IPs of other known vulnerable servers. However such defenses are provided by middleboxes and therefore do not affect the protocol.

It should be noted here that the redundant UDP packets sent along with `ClientHello` are part of the TurboTLS-specific technique we call request-based-fragmentation to mitigate *against* a client’s middlebox defenses incorrectly filtering TurboTLS connections, as otherwise multiple UDP responses to a single UDP request could be flagged as malicious behaviour. Furthermore, the one-

to-oneness of the UDP request/response significantly reduces the impact of any amplification attack which tries to utilize a TurboTLS server as a reflector: an attacker would have to send one UDP packet for every reflected packet generated by the server, meaning that initial requests and responses are of comparable sizes, making the amplification factor so low that it would be an ineffective use of resources. Furthermore, the UDP requests ultimately must contain a fully formed `ClientHello` before the server responds, limiting the amplification factor.

4.3 TurboTLS via Transparent Proxying

Since TurboTLS does not change the contents or computations of the TLS protocol, and only changes how packets are transmitted over the network, another benefit of TurboTLS is that it can easily be implemented as a transparent proxy. As shown in Fig. 2, a client or server (or both) could utilize a proxy to implement TurboTLS on their behalf, even for applications that only speak TLS. A proxy could be a network device or middlebox, or even a daemon running on the same machine as the software wishing to take advantage of TurboTLS. This greatly reduces the burden of deployment as operators can simply run a proxy rather than upgrading their networking stack.

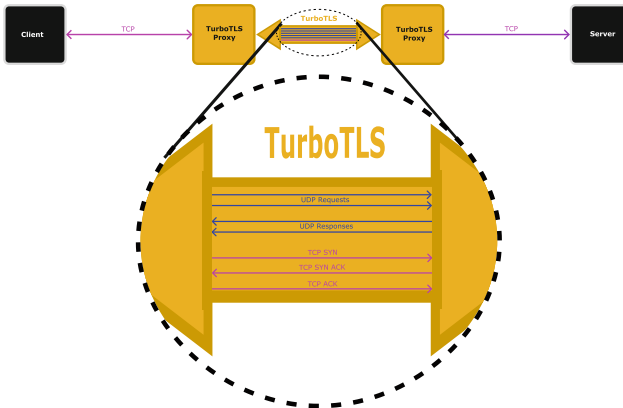


Fig. 2. A client and a server each using a proxy to use TurboTLS

A TurboTLS proxy would intercept TCP-based TLS communication and perform the client-based UDP fragmentation portion of TurboTLS while also setting up its own TCP connection. Should the UDP-based TurboTLS handshake succeed in time, the proxy would relay the `ServerHello` message received over the TurboTLS UDP socket to the client via the established (short-range) client-to-proxy TCP connection. Since the proxy will be physically close to the client, the latency of establishing the TCP connection between the client and proxy will be negligible, so the client still receives the benefit from TurboTLS of not

having had to wait for the TCP handshake across the long distance connection, while keeping the client completely oblivious to the fact of TurboTLS being used. Proxies being used for performance improvements are not a new concept, and have been used for satellite communication for years [GBK+01]. Using proxies with TLS also is not new, as several software packages offer the ability to upgrade TLS versions [CHK, Proa, prob, REB].

4.4 TurboTLS Improvements

We briefly mention a few alternative TurboTLS designs that may improve compatibility or can further reduce the latency assuming pre-shared state, and which may be interesting as future work.

TurboTLS Optimization: TurboTLS for TLS in Pre-shared Key Mode. Pre-shared key (PSK) mode of TLS 1.3 allows a client and server with a pre-shared symmetric key to eliminate parts of the handshake, and allows the client to optionally start sending encrypted application data its first C→S TLS flow, albeit without forward secrecy. The TurboTLS technique could be applied to TLS 1.3 PSK mode, running the first TLS 1.3 PSK C→S and S→C flows (including any early application data) over UDP and then switching over to TCP for the rest of the connection. This would allow for transmission of application data on the very first C→S TurboTLS flow, but comes at the cost of sacrificing forward secrecy, since PSK mode does not offer it. Early application data in both the first C→S and first S→C flows would be over UDP, with only the lightweight reliability features offered by TurboTLS compared to the more extensive reliability features offered by TCP.

TurboTLS Optimization: TurboTLS + TLS Encrypted Client Hello. Encrypted client hello (ECH) [ROSW22] is a mechanism to encrypt parts of the TLS 1.3 handshake under a semi-static server public key. This mechanism even allows for the transmission of application data one round trip earlier, but only by sacrificing forward secrecy. The TurboTLS approach combined with ECH could allow for transmission of application data on the very first C→S TurboTLS flow, at the cost of sacrificing forward secrecy. Again, early application data flows would be over UDP with TurboTLS's lightweight reliability features compared to TCP's more extensive reliability.

TurboTLS Variant: UDP First Stage + TLS 1.3 PSK Handshake. When the UDP and TCP payloads of TurboTLS are combined, they contain an unaltered TLS 1.3 handshake. However, if the TCP portion is inspected on its own, it will appear to be only a part of a TLS handshake, and there is the potential that this could cause compatibility problems for some middleboxes/firewalls/interceptors. An alternative would be for the TLS handshake to terminate after the UDP portion of TurboTLS is completed, use the TLS keying material exporter paradigm to output a shared secret between the client and the server, and then use that shared secret as a pre-shared key in a TLS 1.3 PSK mode handshake over the

TCP connection. This still maintains the RTT and latency improvements offered by TurboTLS, but ensures that the data within the TCP payloads are a fully standards-compliant TLS 1.3 PSK handshake transcript, which should further reduce the risk of incompatibilities from poorly configured middleboxes. (Note this differs from the “TurboTLS optimization: TurboTLS for TLS in pre-shared key mode” mentioned above: the earlier paragraph on optimization for TLS in PSK mode is about using the TurboTLS technique to split a non-forward secure PSK handshakes across UDP and TCP, whereas this paragraph’s TurboTLS variant does a forward-secure handshake in the UDP first stage and then uses the output of that as a PSK in a TLS 1.3 PSK handshake.)

5 Experimental Analysis

We implemented TurboTLS to compare its performance with vanilla TLS 1.3 and QUIC. Our preliminary proof-of-concept implements most of TurboTLS as described in Sect. 3, but not yet completely; see the “Limitations” paragraph below.

Libraries and Cryptographic Algorithms. Our implementation of TurboTLS is based on OpenSSL [The22], using Open Quantum Safe fork of OpenSSL to provide support for post-quantum algorithms [SM16]. We take advantage of OpenSSL’s BIO interface to have a fine control over the I/O operations, allowing us to transmit some messages over UDP instead of TCP.

In our experiments, we considered two cryptographic suites, where we varied the public key algorithms used:

- Elliptic curves: ECDSA signatures and ECDH ephemeral key exchange using the nistp256/secp256r1 curve.
- Post-quantum: Dilithium2 signatures [LDK+20], and Kyber-512 key exchange [SAB+20]. This suite results in both the C→S and S→C TLS handshake flows being fragmented.

In both cases, we used the same symmetric algorithms (AES-128 in Galois counter mode, SHA-256). We use a single self-signed certificate, in other words, a certificate chain of length 1.

Network. We used four network configurations:

- Local: The client and server are in the same data center of a cloud provider, with a ping time of 480–486 microseconds.¹
- Continental: The client was in a data centre in Paris, and the server was in a data centre in Belgium, within the same cloud provider network, joined by a network connection with an observed ping time of 4.9–5.2 milliseconds.

¹ Our TLS/TurboTLS experimental results and QUIC experiment results were collected in different data collection sessions, so there is a small difference between ping times across the two data collection sessions, which we believe is due to natural network variation.

- Intercontinental1: The client was in a data centre in Paris, and the server was in a data centre in Oregon, within the same cloud provider network, joined by a network connection with an observed ping time of 132–133 milliseconds.
- Intercontinental2: The client was in a data centre in Paris, and the server was in a data centre in Australia, within the same cloud provider network, joined by a network connection with an observed ping time of 268–270 milliseconds.

The only source of latency we introduce is distance, but there are many other reasons for latency to be in the hundreds of milliseconds: the number of network hops (when changing between providers, or going to end users), the server load (which can provoke waiting queues over each round-trip), and the technology of the intermediate networks (IoT, 3G, etc.). In practice, global studies show² that, ignoring server load or end-user delays, median connections lead to RTTs mostly between 50 ms (US west coast to US west coast) and 200 ms (west coast to Europe), and 90th percentile connections lead to RTTs going over 500 ms or even 1000 ms [HPMC02].

We addressed servers with IP addresses so we did not incur any time for DNS resolution, and the client assumed the server supported TurboTLS without making any DNS HTTPS resource record query.

Machines. In all cases, the machines used were Linux x86_64 cloud servers with 4 cores (8 vcores taking into account HyperThreading) of an Intel Xeon E5-2696V4 Processor with 16 GB of RAM.

Results. Figure 3 shows the results of the experiment across the two cryptographic suites and four network configurations, comparing the latencies of TLS 1.3, TurboTLS, and QUIC. The results reported show latencies at the 50th percentile (median) for a 10 s experiment. Table 2 in the appendix includes the same data as well as latencies at the 90th and 99th percentile.

As expected, in long distance connections (see Fig. 3c, Fig. 3d), where latency is primarily due to the time for information to travel between endpoints, saving one round-trip using TurboTLS approximately halves the latency compared to TLS 1.3, and TurboTLS latency behaves similarly to QUIC at both the median and 90th and 99th percentiles.

In low-latency connections (see Fig. 3a, Fig. 3b), the main outcome remains that TurboTLS outperforms TLS 1.3, but there is more subtlety. First, median latency of the 1 RTT protocols (TurboTLS and QUIC) is less than that of the 2-RTT TLS 1.3, but not fully reduced by 50%, which is likely due to the fixed cost of the computations. For example, in the Paris–Belgium setting (RTT of 5.2–5.6 ms) TurboTLS and QUIC have median latency about 60% that of TLS 1.3. Second, latency of TLS 1.3 and TurboTLS at the 90th and 99th percentiles scales differently than QUIC. We believe that this is not a fundamental characteristic of TLS or TurboTLS and instead is related to the cloud environment we used

² CAIDA’s Macroscopic Internet Topology Monitor <https://www.caida.org/catalog/software/walrus/rtt/>.

(Google Cloud Platform), specifically differing behaviour of TCP versus UDP in low-latency high-throughput scenarios in hypervisors.



Fig. 3. Comparing performance of TLS 1.3, TurboTLS, and QUIC in 4 network settings with 2 cryptographic suites (elliptic curves or post-quantum). Latencies reported are time in microseconds from start of connection establishment until client sends its first byte of application data.

Limitations. Our implementation does not include, as of yet, the request-based fragmentation feature (in other words, the client only sends fragments for its actual packets, and the server responds with as many UDP packets as needed). It also does not, as yet, fall back to TCP if the UDP packets are lost or delayed too much but, in our handful of trials, we never observed a failure of TurboTLS because of such issues (or any other reason). We do not expect these to substantially change performance results, as they are primarily related to compatibility.

Future Work. To properly evaluate the effect of request-based fragmentation, it is necessary to test over more complex heterogenous networks (those where pathways are likely to include poorly configured middleboxes and bad network connections) to fully ascertain the impact of the TLS fallback both compared to TurboTLS and to QUIC.

6 TLS Depth of Websites

The results of Sect. 5 are amplified when sequential TLS connections are required: the savings from TurboTLS gained during many connections in series could easily become human-noticeable, so we outline how this might happen.

Websites typically load resources from multiple domains (that eventually resolve to different hosts). More than one TLS connection is thus generally established by a browser to load and render a website. While most of these connections can be done in parallel, some of them can initiate only once some resources have been loaded, e.g. once some CSS or Javascript assets have been processed. As this can happen recursively, we call the *TLS depth* the depth of that recursion.

Figure 4 illustrates the TLS depth for two popular webpages as a directed graph of TLS initiations. Nodes are the hosts to which the browser made TLS connections, and an edge from host A to host B means that a resource downloaded from host A spawned a TLS connection to host B . The longest path in the graph is thus the TLS depth.

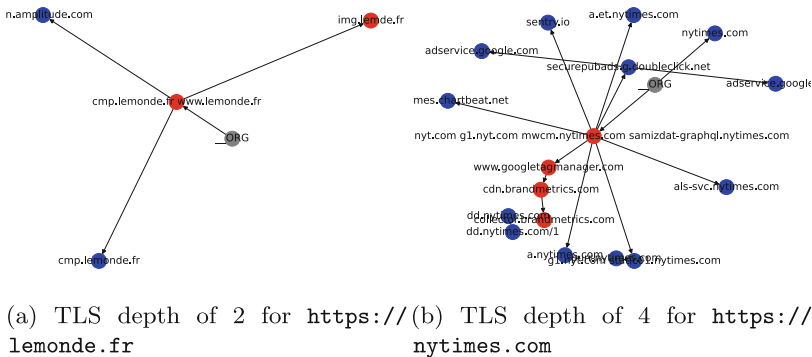


Fig. 4. Examples of TLS initiation graph for two websites, as captured on June 1, 2023 from Paris, France. Red nodes indicate one example of the longest paths (there may be more than one) leading to the TLS depth. (Color figure online)

7 Identifying Usecases

In this section, based upon TurboTLS’s strengths and weaknesses with respect to other TLS optimizations and protocols, we provide guidance to identify applications where TurboTLS might be most profitably deployed.

7.1 TurboTLS as an Alternative to QUIC

QUIC is gaining significant traction and is potentially the future of internet content delivery. QUIC solves several problems that TLS over TCP faces, including head of line blocking, mobile clients changing networks mid-connection, connection multiplexing, and improved performance for the secure connection establishment handshake. For HTTP, QUIC is undoubtedly superior to TurboTLS, if it can be deployed. Of course, this comes at the cost of a substantially different protocol and software stack. Even with all of QUIC's strengths, it will be some time before QUIC is fully adopted for web traffic, and even longer before QUIC replaces TLS for the many other applications that are layered over TLS. As TurboTLS is an optimization to TLS rather than an entirely new delivery protocol like QUIC, it may be simpler to transition to TurboTLS in order to still receive TurboTLS's handshake latency improvements.

Cybersecurity Concerns. Many standard network firewalls depend on information from legacy TCP sessions to detect threats. QUIC's encryption of transport information results in standard network sensors being unable to even detect QUIC. Combined with Google's frequent maintenance of QUIC, this can make security provision difficult for servers using QUIC, hence the recommendation by some providers [For,Net] to revert to (or remain on) HTTPS over TCP.

Legacy Applications. The issue of legacy software persists due to gnarly real-world problems of interacting applications with specific version integration requirements, subject to third party maintenance and insurance. In cases where networking upgrades such as implementing QUIC are not feasible, yet the applications running over such software are business-critical and latency sensitive, TurboTLS's proxying capabilities can be appealing due to its non-invasive implementation, and can be implemented adjacent to an application, at network gateway, or anywhere in between.

Protocol Simplicity. As previously mentioned, QUIC offers many great features. However, these features add additional complexity to networking stacks providing more opportunities for bugs to appear both in QUIC implementations as well as applications which use them. TurboTLS has a relatively simple design, thus making implementations less error-prone. TurboTLS can be used for those that desire additional scrutiny of QUIC implementations and their APIs.

A Simple Alternative: Proxying TLS. QUIC uses the TLS handshake for connection establishment, but it is integrated into the QUIC protocol and inserted into QUIC messages. Additionally, QUIC performs some cryptographic computations differently: in particular, as part of key derivation, QUIC provides different label strings to the HKDF compared to TLS, so the session key derived in QUIC will differ from that of TLS [TT21]. Consequently, without being given additional keying material, it is not possible to implement a proxy that would transparently convert a TCP-based TLS connection to a QUIC connection. As noted in

Sect. 4.3, a TurboTLS proxy, can merely pass TLS handshake and record layer messages back and forth across the appropriate UDP and TCP sockets without needing to know any additional keying material. Thus, whatever the motivation (security concerns, the complexity of migrating a given legacy application, or protocol complexity), users may want to replace TLS with TurboTLS, or just use a TurboTLS proxy instead of migrating to QUIC.

Table 2. Performance of TLS 1.3, TurboTLS, and QUIC in 4 network settings with 2 cryptographic suites (elliptic curves or post-quantum). Latencies reported are time in microseconds from start of connection establishment until client sends its first byte of application data.

Protocol Mode	Signature & Key Exchange	Ping	Throughput	Latency		
		μ s	hs/sec	μ s	P90 μ s	P99 μ s
Paris-Paris						
TLS 1.3	ECDSAp256 & ECDH nistp256	486	2,313	1,611	1,898	2,315
TurboTLS 1.3	ECDSAp256 & ECDH nistp256	486	2,288	1,337	1,639	2,396
QUIC	ECDSAp256 & ECDH nistp256	480	—	1,145	1,257	1,395
TLS 1.3	Dilithium2 & Kyber512	486	2,263	1,690	2,080	2,664
TurboTLS 1.3	Dilithium2 & Kyber512	486	2,487	1,096	1,400	2,513
QUIC	Dilithium2 & Kyber512	480	—	1,413	1,590	1,736
Paris-Belgium						
TLS 1.3	ECDSAp256 & ECDH nistp256	5,210	399	10,012	10,802	11,087
TurboTLS 1.3	ECDSAp256 & ECDH nistp256	5,210	553	6,196	10,164	11,189
QUIC	ECDSAp256 & ECDH nistp256	5,620		6,016	6,401	6,800
TLS 1.3	Dilithium2 & Kyber512	5,210	401	9,842	10,829	11,667
TurboTLS 1.3	Dilithium2 & Kyber512	5,210	562	6,239	10,264	11,071
QUIC	Dilithium2 & Kyber512	5,620	—	6,275	6,565	6,727
Paris-Oregon						
TLS 1.3	ECDSAp256 & ECDH nistp256	132,021	15	264,618	265,417	266,728
TurboTLS 1.3	ECDSAp256 & ECDH nistp256	132,021	30	132,754	133,199	134,053
QUIC	ECDSAp256 & ECDH nistp256	133,525	—	134,363	134,652	134,788
TLS 1.3	Dilithium2 & Kyber512	132,021	15	265,430	266,268	267,505
TurboTLS 1.3	Dilithium2 & Kyber512	132,021	29	132,562	132,986	133,479
QUIC	Dilithium2 & Kyber512	133,525	—	134,479	134,775	134,897
Paris-Australia						
TLS 1.3	ECDSAp256 & ECDH nistp256	268,157	7	538,426	539,390	539,787
TurboTLS 1.3	ECDSAp256 & ECDH nistp256	268,157	14	269,753	270,321	273,781
QUIC	ECDSAp256 & ECDH nistp256	270,453	—	271,018	271,553	271,839
TLS 1.3	Dilithium2 & Kyber512	268,157	7	539,279	540,387	541,679
TurboTLS 1.3	Dilithium2 & Kyber512	268,157	14	269,606	270,026	271,118
QUIC	Dilithium2 & Kyber512	270,453	—	271,262	271,793	272,056

7.2 Generic Turbo Transport Situations

In some situations, all traffic transported by TCP could benefit from the underlying ideas of TurboTLS so that for any TCP connection, one round trip is removed (by a transparent proxy). We coin such an approach TurboTCP.

Traffic Going Through an Internet Provider Core-Network. When traffic from a client regularly follows the same path: enter a core-network, use the network to get closer to the destination, and exit such a network, it would be possible to use TurboTCP proxies that significantly improve the user experience.

High-Latency Satellite Communications. Proxying TCP into TurboTCP can also be systematically done by a device/OS. For example this can be beneficial with a satellite phone, that would avoid the very significant RTTs observed in such communications.

Internet of Things. In the Internet of Things, energy is of the essence, having lower latencies can also be seen as an approach to limit the time in which communicating devices are online and using energy.

In all these settings, QUIC is clearly not an option as we are accelerating a wide span of protocols that are transported via TCP. The simplicity of proxying transparently TCP can be very attractive.

Acknowledgements. D.S. and J.G. were supported in part by Natural Sciences and Engineering Research Council of Canada (NSERC) Discovery grant RGPIN-2022-03187.

A. Computing TLS Depth

We consider a website is loaded once the Javascript `onload` event has been fired. A website that loads all its assets from the same host has a TLS depth of one. For example, take the website <https://mywebsite.com> that first loads Javascript code from <https://mycdn.com>. Next, that Javascript code fetches some images from <https://coolimages.com>. This illustrates how loading the webpage for the user may take many sequential TLS connections before being ready.

For computing the TLS depth of real-life websites, we need to capture all TLS requests that have been done by a browser from the moment the website starts loading up until the `onload` Javascript event has been fired. Most importantly, we also need to capture the initiator of these TLS request (e.g. a fetch request in Javascript code), so that we can compute the complete graph of TLS initiators.

In practice, the Google Chrome browser can be used to load a website, stop on the `onload` Javascript event, and dump all requests that have been done in the HTTP ARchive (HAR) format. The open-source project `chrome-har-capturer` [ea] does this automatically. The Chrome browser includes in its HAR dump an `initiator` field that can be used to create our TLS initiator graph. Then, using the tool, these HAR dumps can be parsed to generate TLS initiator graphs and thus compute the TLS depth of websites.

References

- [Ber] Bernstein, D.J.: SYN cookies
- [CAI] CAIDA. Round-Trip Time Internet Measurements from CAIDA’s Macroscopic Internet Topology Monitor. <https://www.caida.org/catalog/software/walrus/rtt/>. Accessed 29 Jun 2023
- [CCRJ14] Cheng, Y., Chu, J., Radhakrishnan, S., Jain, A.: TCP Fast Open. RFC 7413, December 2014
- [CHK] Cortesi, A., Hils, M., Kriechbaumer, T.: mitmproxy
- [CSM+22] Chaudhary, S., Sachdeva, P., Mondal, A., Chakraborty, S., Maity, M.: YouTube over Google’s QUIC vs internet middleboxes: A tug of war between protocol sustainability and application QoE (2022)
- [ea] Cardaci, A., et al.: Chrome HAR capturer
- [For] Fortinet. FortiGate firewall blog. <https://community.fortinet.com/t5/FortiGate/Technical-Tip-Block-QUIC-Protocol/ta-p/197661#:~:text=Go%20to%20the%20Application%20Control,port%2080%20and%20port%20443> Accessed 29 Jun 2023
- [GBK+01] Griner, J., Border, J., Kojo, M., Shelby, Z.D., Montenegro, G.: Performance Enhancing Proxies Intended to Mitigate Link-Related Degradations. RFC 3135 (2001)
- [GS22] Goertzen, J., Stebila, D.: Post-quantum signatures in DNSSEC via request-based fragmentation. arXiv (2022)
- [HPMC02] Huffaker, B., Plummer, D., Moore, D., Claffy, K.C.: Topology discovery by active probing. In: Symposium on Applications and the Internet (SAINT) 2002 Workshops, pp. 90–96. IEEE (2002)
- [IT21] Iyengar, J., Thomson, M.: QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000 (2021)
- [KT22] Kühlewind, M., Trammell, B.: Manageability of the QUIC Transport Protocol. RFC 9312 (2022)
- [KW16] Krawczyk, H., Wee, H.: The OPTLS protocol and TLS 1.3. In: IEEE European Symposium on Security and Privacy (EuroS&P) 2016, pp. 81–96. IEEE (2016)
- [Lan10] Langley, A.: Transport Layer Security (TLS) Snap Start. Internet-Draft draft-agl-tls-snapstart-00, Internet Engineering Task Force (2010)
- [LDK+20] Lyubashevsky, V., et al.: CRYSTALS-DILITHIUM. Technical report, National Institute of Standards and Technology (2020). <https://csrc.nist.gov/projects/post-quantum-cryptography/round-3-submissions>
- [Net] Alto, P.: Networks. Palo Alto Networks customer support portal. <https://knowledgebase.paloaltonetworks.com/KCSArticleDetail?id=kA10g000000ClarCAC>. Accessed 29 Jun 2023
- [Proa] Progress Software Corporation. Fiddler Everywhere
- [prob] The Squid project. Squid Web Cache
- [PZS+13] Petullo, W.M., Zhang, X., Solworth, J.A., Bernstein, D.J., Lange, T.: MinimaLT: minimal-latency networking through better security. In: Sadeghi, A.-R., Gligor, V.D., Yung, M. (eds.), ACM CCS 2013, pp. 425–438. ACM Press (2013)
- [RBTS22] Rescorla, E., Barnes, R., Tschofenig, H., Schwartz, B.M.: Compact TLS 1.3. Internet-Draft draft-ietf-tls-ctls-06, Internet Engineering Task Force (2022)

- [RD08] Rescorla, E., Dierks, T.: The Transport Layer Security (TLS) Protocol Version 1.2. RFC 5246 (2008)
- [REB] REBEX ČR s.r.o. Rebex TLS Proxy
- [Res18] Rescorla, E.: The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446 (2018)
- [ROSW22] Rescorla, E., Oku, K., Sullivan, N., Wood, C.A.: TLS Encrypted Client Hello. Internet-Draft draft-ietf-tls-esni-15, Internet Engineering Task Force (2022)
- [RTM22] Rescorla, E., Tschofenig, H., Modadugu, N.: The Datagram Transport Layer Security (DTLS) Protocol Version 1.3. RFC 9147 (2022)
- [SAB+20] Schwabe, P., et al.: CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology (2020). <https://csrc.nist.gov/projects/post-quantum-cryptography/round-3-submissions>
- [SBN22] Schwartz, B.M., Bishop, M., Nygren, E.: Service binding and parameter specification via the DNS (DNS SVCB and HTTPS RRs). Internet-Draft draft-ietf-dnsop-svcb-https-11, Internet Engineering Task Force (2022)
- [SBR04] Shacham, H., Boneh, D., Rescorla, E.: Client-side caching for TLS. *ACM Trans. Inf. Syst. Secur.* **7**(4), 553–575 (2004)
- [Sim11] Simpson, W.A.: TCP Cookie Transactions (TCPCT). RFC 6013 (2011)
- [SKD20] Sikeridis, D., Kampanakis, P., Devetsikiotis, M.: Post-quantum authentication in TLS 1.3: A performance study. In: NDSS 2020. The Internet Society (2020)
- [SM16] Stebila, D., Mosca, M.: Post-quantum key exchange for the internet and the open quantum safe project. In: Avanzi, R., Heys, H. (eds.) SAC 2016. LNCS, vol. 10532, pp. 14–37. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-69453-5_2
- [ST16] Santesson, S., Tschofenig, H.: Transport Layer Security (TLS) Cached Information Extension. RFC 7924 (2016)
- [SW19] Song, L., Wang, S.: ATR: Additional Truncation Response for Large DNS Response. Internet-Draft draft-song-atr-large-resp-03, Internet Engineering Task Force (2019)
- [The22] The OpenSSL Project. OpenSSL version 1.1.1s (2022)
- [TT21] Thomson, M., Turner, S.: Using TLS to Secure QUIC. RFC 9001 (2021)



From Fingerprint to Footprint: Characterizing the Dependencies in Encrypted DNS Infrastructures

Baiyang Li^{1,2,3}, Yujia Zhu^{1,2,3}(✉), Yong Ding^{1,3}, Yong Sun¹, Yuedong Zhang⁴,
Qingyun Liu^{1,2,3}, and Li Guo^{1,2,3}

¹ Institute of Information Engineering, Chinese Academy of Sciences,
Beijing, China
zhuyujia@iie.ac.cn

² National Engineering Research Center of Information Security,
Beijing, China

³ School of Cyber Security, University of Chinese Academy of Sciences,
Beijing, China

⁴ National Computer Network Emergency Response Technical Team/Coordination
of China, Beijing, China

Abstract. Encrypted DNS protocols—DNS-over-TLS (DoT), DNS-over-HTTPS (DoH), and DNS-over-QUIC (DoQ), have been standardized and widely embraced by the industry to enhance the security and privacy of DNS transmissions. As more software and devices support encrypted DNS protocols, adopting encrypted DNS is fast becoming the prevailing trend in domain name resolution. Despite this growing trend, the dependencies within the encrypted DNS infrastructures remain largely unexplored. Key questions arise regarding the server components and third-party DNS providers that encrypted DNS servers rely on during the resolution process. Understanding these dependencies is essential for gaining a comprehensive view of the encrypted DNS ecosystem and revealing potential vulnerabilities or points of centralization that could impact the robustness and reliability of DNS services.

This paper analyzes the dependencies from two critical aspects: the server components and the DNS resolution process. To this end, we perform large-scale measurements on encrypted DNS infrastructures and distinguish the role of servers. Based on the classification results, we extract fingerprints to identify the server components and measure the upstream DNS resolver leveraging the resolution footprints. Our findings show that (i) an encrypted DNS resolver can encompass multiple components that cooperate in the resolution process; (ii) despite the dispersed nature of encrypted DNS entrances, 75.24% of all encrypted DNS resolvers rely on the top 10 DNS providers. The concentration of query forwarding towards specific upstream resolvers signifies a growing concern related to dependencies and centralization in encrypted DNS.

Keywords: Dependencies · Centralization · DNS Measurement · Encrypted DNS

1 Introduction

The Domain Name System (DNS), a pivotal infrastructure element of the Internet that associates user-friendly names with internet resources, plays an essential role in virtually all online activities. Each Internet transaction typically commences with a DNS query. Nevertheless, the original architecture of the DNS, predicated on plain-text communication, inherently compromises its security, making it susceptible to a broad spectrum of security and privacy breaches.

To address these vulnerabilities and enhance DNS security, encrypting DNS transaction stands as a crucial countermeasure. In this context, the Internet Engineering Task Force (IETF) has standardized three protocols: DNS-over-TLS (DoT), DNS-over-HTTPS (DoH), and DNS-over-QUIC (DoQ), each designed to secure DNS queries and responses from interception and manipulation. These protocols have been widely supported and deployed by large DNS providers [1, 9, 10, 15], software [7, 11, 13], and operating systems. The escalation in the number of servers and devices offering encrypted DNS services has significantly compounded the complexity of the encrypted DNS ecosystem. Accordingly, acquiring an in-depth understanding of the encrypted DNS infrastructure is imperative.

While extant research has predominantly focused on the deployment status of encrypted DNS, there remains a gap in identifying the dependencies of encrypted DNS servers. The increased dependency and centralization within DNS infrastructures heighten the risk of single point failures and data monopolization. These issues ultimately compromise users' security and privacy. The overarching composition of encrypted DNS infrastructures, as well as the specific server components and the role of third-party DNS providers within the resolution process of resolvers, are areas that have not been sufficiently clarified. This gap presents a challenge in comprehensively understanding the encrypted DNS landscape.

In this paper, we present an up-to-date view of the dependencies inherent in open encrypted DNS infrastructures across the Internet. To this end, we first distinguish the roles of various encrypted DNS servers. Following this, our analysis focuses on understanding these dependencies from two critical aspects: the server components and the DNS resolution process. Our methodology involves the extraction of unique identifiers or “fingerprints” of encrypted DNS servers and the identification resolution “footprints”- the traces left by upstream DNS servers. By examining these aspects, we effectively characterize the dependencies of an encrypted DNS server.

In summary, the contributions of this paper can be concluded as follows.

- We execute comprehensive measurements of encrypted DNS servers across the Internet. The servers we discovered display a hybrid array of roles and functions. To discern their roles, we implement tailored queries using a variety of domain names, leading to a detailed, hierarchical classification of the encrypted DNS servers. The classification lays the groundwork for in-depth dependency analyses.

- To categorize server components, we extract fingerprints of encrypted DNS servers by leveraging distinct features from the HTTP and DNS layers. Our analysis reveals that multiple components play pivotal roles in the DNS resolution process within a single resolver. Collectively, these components form a sequential chain of operations that underpin DNS resolution, creating a chain of functional dependencies.
- We track the footprints of open encrypted resolvers and introduce a novel metric known as the Service Reliance Index (SRI), which is designed to assess the extent of dependency among providers of upstream resolvers. Our evaluation demonstrates that, although encrypted DNS entrances appear to be dispersed, the top 10 large DNS providers serve about 75.24% of all encrypted DNS resolvers.

The structure of this paper is organized as follows. Section 2 provides an overview of standardized encrypted DNS protocols and encrypted DNS infrastructures. Section 3 details our approach for discovering and distinguishing encrypted DNS servers in the wild. Section 4 analyses the dependencies of server components. Section 5 identifies the dependencies on large DNS providers. Section 6 discusses the implications of security and privacy, the limitations of our method, and the ethical considerations in our measurements. Related work is introduced in Sect. 7. Section 8 concludes our work.

2 Background

2.1 Standardized Encrypted DNS Protocols

IETF has proposed three encrypted DNS protocols as standard track protocols: *DNS-over-TLS (DoT)*, *DNS-over-HTTPS (DoH)*, and *DNS-over-QUIC (DoQ)*. Lu et al. jointly term them as DNS-over-Encryption (DoE) [29]. Figure 1 shows their protocol stacks. The commonality between these protocols is that each protocol uses the TLS (Transport Layer Security) protocol to encrypt and secure the wire-format DNS data. However, the difference lies in the protocol layer, directly impacting how clients access the encrypted DNS services.

- **DNS-over-TLS (DoT)**. DoT is the first standardized encrypted DNS protocol in 2016 [24]. The key idea behind DoT is to use TCP in conjunction with TLS to secure and encrypt DNS communication instead of UDP. Typically, the DoT client initiates the connection by performing a TLS handshake with the DoT server on port 853. Once the TLS connection is established, the client can securely transmit wire-format DNS queries using TCP.
- **DNS-over-HTTPS (DoH)**. Compared to DoT, DoH incorporates HTTP layer, encapsulating DNS packets within HTTPS transports. The DoH client can send DNS queries either by HTTP `GET` or `POST` method to the DoH server after the TLS handshake. Another difference between DoH and DoT is the presence of an additional path parameter in DoH. By default, the path parameter for DoH is set to “/dns-query” specified in RFC 8484 [23].

- **DNS-over-QUIC (DoQ)**. DoQ operates similarly to DoT regarding DNS queries but utilizes the advantages of QUIC transport protocol instead of TCP. The DoQ client transmits wire-format DNS queries over the QUIC connection after finishing a QUIC handshake with the DoQ server on port 853 [25].

2.2 Encrypted DNS Infrastructures

As delineated in Fig. 2, in the architecture of an encrypted DNS schema, recursive DNS resolvers (RDNSes) can be categorized into two distinct categories: (i) the “ingress” servers, serving as the initial interface for clients by directly receiving encrypted DNS queries from stubs, also known as open encrypted DNS resolvers (ODNSes), and (ii) the “egress” servers, tasked with communicating with Authoritative DNS Servers (ADNSes), which are typically not directly accessible by end-users. An ODNS can operate concurrently as both an ingress (RDNS_{in}) and an egress (RDNS_e) entity, embodying a dual-functional role.

Beyond the scope of encrypted ODNS, another integral element of the encrypted DNS infrastructure is encrypted ADNS. Recent research [28] has identified approximately 300 ADNSes that have commenced support for encrypted DNS queries.

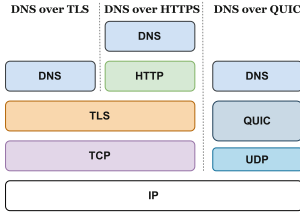


Fig. 1. DoE protocol stacks.

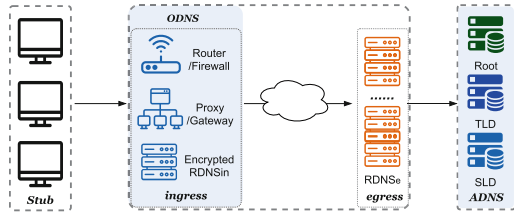


Fig. 2. Encrypted DNS infrastructures.

Open Encrypted DNS Resolver (ODNS). The ODNS serves as the entry point for DNS queries emanating from stubs. It can be further bifurcated into two categories: the encrypted DNS forwarder (FDNS) and the encrypted ingress DNS resolver (RDNS_{in}).

- **Encrypted DNS Forwarder (FDNS).** The FDNS serves as an intermediary conduit between the stub and other RDNSes. Its primary role is not to fulfill the entire DNS resolution process but to manage encrypted DNS queries received from stub resolvers. The FDNS processes DoE queries by forwarding them in encrypted manners to another encrypted RDNS or decrypting them and transmitting conventional DNS (Do53) queries to open Do53 resolvers, depending on the operational configuration of the FDNS.

- **Encrypted Ingress DNS resolver (RDNS_{in}).** The distinction between FDNS and RDNS_{in} largely hinges on the affiliation of the ingress and egress servers with the DNS provider. In the public DNS service landscape, it is generally expected for both ingress and egress servers to fall within the purview of a singular organizational entity. However, instances may arise where the FDNS, operated by service provider A , does not align in ownership with the RDNS_e managed by service provider B .

Encrypted Authoritative DNS Server (ADNS). An encrypted ADNS delivers authoritative responses for domain zones it hosts through an encrypted communication. If an encrypted ADNS encounter a query for a domain outside its zone, it can issue a REFUSED response code, indicating its limitation to respond only within its authoritative domains.

3 Encrypted DNS Servers in the Wild

This section details our strategy for discovering encrypted DNS servers and distinguishing the roles of the servers. The core steps of this process are visualized in Fig. 3. Following this, we present the results of the hierarchical classification of these servers.

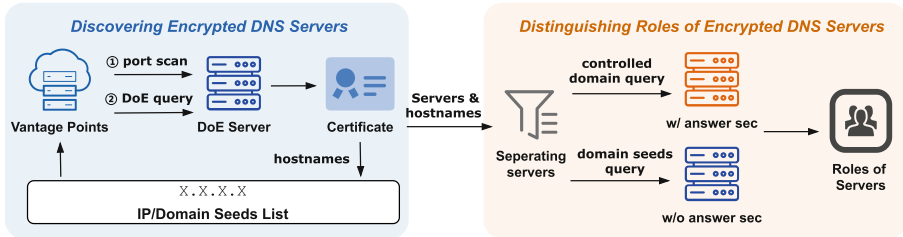


Fig. 3. Methodological overview.

3.1 Methodology

Discovering Encrypted DNS Servers. The first phase of our method involves executing extensive Internet-wide scans of IPv4 to find active targets with specific ports. This phase uses all IPv4 addresses as primary probing seeds. Following this, we direct encrypted DNS queries to these active IP addresses to evaluate their responses. If a response aligns with the DNS wire-format, we classify the respective target as an encrypted DNS server. Considering the shared usage of port 853 by both DoQ and DoT and clients' propensity to transition from DoQ to DoT in case of connection failures [25], our method specifically targets DoQ queries on the identified DoT servers. This selective approach avoids

the extensive bandwidth utilization inherent in scanning QUIC support across the entire IPv4 address range [37].

In practice, we employ the ZMap network scanner [21] to probe ports associated with DoT and DoH protocols, specifically TCP/853 and TCP/443. The DNSPython API [8] is utilized to dispatch DoT, DoH, and DoQ queries to hosts with open ports, thereby ascertaining their support for DoE services. To achieve comprehensive coverage and consider the varying impact of geographical location on Internet-wide scans [35], we initiate our measurements from five distinct vantage points hosted on cloud platforms: Germany, Hong Kong, China, Singapore, Australia, and the USA. Therefore, the data collected provides a broad and varied perspective.

Distinguishing Roles of Encrypted DNS Servers. The encrypted DNS servers we identify represent a hybrid mix of ODNsEs and ADNsEs. Within the ODNsE category, further subdivision reveals the FDNsE and the RDNsE_{in}. To distinguish the roles of the discovered servers, we deploy specialized queries using various domain names.

ADNsE. The ADNsE responds solely to DNS queries directed at domains within its designated authoritative zones. A key indication that a server operating as an ADNsE is the absence of Recursion Availability (RA) in its DNS response message, denoted by the RA flag being set to 0. Coupled with an empty answer to queries for non-authoritative domains (e.g., a REFUSED or SERVFAIL RCODE), these suggest the server’s incapacity to offer recursion. Therefore, we sift through DNS response messages, isolating those that do not contain an answer section.

The challenge in pinpointing ADNsEs lies in ascertaining the specific domain names for which they hold authority. We explore two scenarios, as delineated in Fig. 4. For *in-domain names*, an ADNsE (ns1.example.com) is responsible for its parent zone (example.com). The hostnames of a DNS server, obtained from the server’s certificate or PTR record associated with the server’s IP address, can serve as verification domain names. We send queries using these domain names to confirm whether a server is an ADNsE. Upon confirming a server as an ADNsE for a domain name, we then query the NS records of the domain to ascertain if other ADNsEs also facilitate DoE.

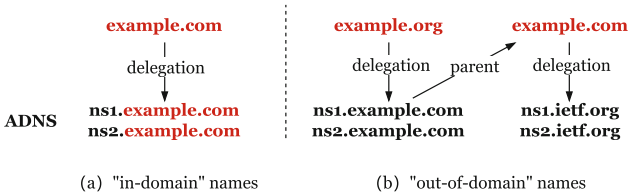


Fig. 4. Scenarios of in-domain names and out-of-domain names.

For out-of-domain names, determining the hosted domain names on an ADNS becomes difficult. We adopt the method in previous studies [20, 28], which involves testing the support of top domain names' ADNSes for DoE protocols. To broaden the collection of domain name seeds, we compile a list of 4 million unique top domain names by aggregating data from six different sources: Alexa, Majestic, Umbrella, Tranco, Cloudflare Radar, and the BuiltWith TOP-1M list [2, 3, 5, 6, 12, 14]. An ADNS is determined for the given domain name if the Authoritative Answer (AA) flag is set and the response message includes either the answer section or the authority section.

Upon identifying ADNSes in the outlined scenarios, we proceed to analyze the residual DNS servers that yield responses with empty answers. Specifically, we scrutinize servers whose certificates share the same Common Name (CN) and Organization (O) fields as the identified ADNSes. Queries are then sent to these servers using domain names over which the respective ADNSes have authority. The whole process of ADNS identification is systematically outlined in Algorithm 1.

Algorithm 1. Procedures of ADNS identification.

Input: Encrypted DNS servers $S = \{sip_1, sip_2, \dots, sip_n\}$ with an empty answer; Top domain names list $D = \{d_1, d_2, \dots, d_n\}$;

Output: Identified ADNSes S_{ADNS} ;

```

1: for  $sip \in S$  do
2:    $D_{sip} \leftarrow$  related domain names in cert and PTR record
3:   for  $d \in D_{sip}$  do
4:     send DoE query of  $d$  to  $sip$ ;
5:     if  $sip$  response with AA flag then
6:        $S_{ADNS} \leftarrow sip$ ;
7:       send DoE queries to other ADNSes of  $d$ ;
8:     end if
9:   end for
10: end for
11: for  $d \in D$  do
12:    $S_{ADNS_d} \leftarrow$  ADNS of  $d$ 
13:   for  $sip \in S_{ADNS_d}$  do
14:     send DoE query of  $d$  to  $sip$ ;
15:     if ADNS response with AA flag then
16:        $S_{ADNS} \leftarrow sip$ ;
17:     end if
18:   end for
19: end for
20: for  $sip \in S$  and  $sip \notin S_{ADNS}$  do
21:   if  $sip$  share the same O and CN fields with  $sip' \in S_{ADNS}$  then
22:     send DoE queries of  $d$  hosted on  $sip'$  to  $sip$ ;
23:   end if
24: end for

```

ODNS. The ODNS serves as the entry point for DoE service and can be divided into FDNS and RDNS_{in}. The key criterion for this categorization is the alignment of the relationship between ingress and egress resolvers. An ODNS is especially deemed to perform both ingress and egress functions when its IP address is the same as that of the RDNS_e.

Our method to pinpoint the RDNS_e associated with an ODNS is to construct destination-specific DNS queries, which encode the ODNS’s IP address as a subdomain into the query name. To mitigate the effects of DNS caching, we add a timestamp label in the sub-domain, ensuring each query is distinct and traceable. The DNS queries are then gathered at our controlled ADNS server. We extract the ODNS’s IP address embedded in the queried domain name and correlate the source IP address from these queries, which corresponds to the IP address of the RDNS_e.

3.2 Result Overview

Over a period spanning from August to December 2023, we execute a sequence of measurements at 15-day intervals. Each measurement cycle spans roughly 12 days, incorporating port scanning and DoE discovery. The port scanning phase, targeting ports 443 and 853 across all IPv4 addresses, takes three days for each port. Owing to the prevalent use of port 443, the process to identify DoH servers requires five days, whereas the discovery of DoT and DoQ is completed within a few hours.

Our findings reveal that approximately 3.4 million IPv4 addresses have open ports on 853 and 54 million on 443. While the general scale of open ports aligns with prior measurements, we noted a significant rise in encrypted DNS servers. Figure 5 illustrates the trend over time of detected DoE servers. Starting on Dec 15th, 2023, our latest measurement cycle finds 95,582 DoT servers, 21,784 DoH servers, and 4,320 DoQ servers. During the measurement period, the number of DoT servers increases by 3.6%, DoH servers by 2.7%, and DoQ servers by 3.3%. Compared to earlier studies [26, 28], the counts of DoT, DoH, and DoQ servers have surged by 340%, 97%, and 249%, respectively.

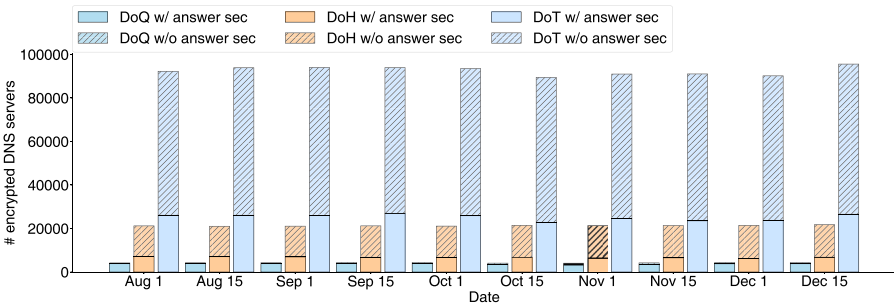


Fig. 5. The trend of discovered DoE servers over time.

Not all discovered encrypted DNS servers respond to queries with answer section. Our statistics reveal the percentages of servers capable of successfully responding to DNS queries. Specifically, the percentage for DoQ is above 90%, while the percentages for DoT and DoH hover around 30%. The remaining servers respond without an answer section. We will further analyze and differentiate these discovered servers in the following subsections. Figure 6 provides an overview of the hierarchical classification result.

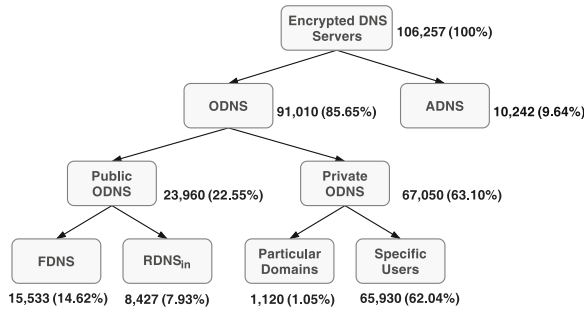


Fig. 6. Hierarchical classification result of the discovered encrypted DNS servers.

3.3 Encrypted Authoritative DNS Server

We identify encrypted ADNSes using two types of domain seeds. The first is extracted from the Common Names (CN) and Subject Alternative Names (SAN) fields of the server certificates, as well as from the PTR records associated with the servers. The second leverages domain names obtained from publicly available top domain lists.

Table 1 presents the identification results, which show a remarkable increase in the ADNS that support DoE protocols. Compared to the most recent prior measurement [28], we observe a significant increase in encrypted ADNSes. Specifically, the count of DoH ADNSes has multiplied by 148 times, DoT ADNSes by 4.35 times, and we additionally identify 15 IPv4 DoQ ADNSes. The dramatic surge in DoH ADNSes is mainly attributable to a cluster of 9,024 servers associated with Microsoft, serving the domain **.ms-acdc.office.com*. Through certificate correlation, our investigation can uncover such hidden ADNSes that might not be listed in the NS records. For DoT ADNSes, we identify 254 servers within a /24 IP prefix associated with CDN77 for *cdn77.org*, in addition to noting the adoption of DoT by the B-Root server [17] and its updated IP addresses [18]. These findings validate the effectiveness of our investigative approach.

During the ADNS identification process, we encounter several domains (like *ns2.airport.kg*, *dns.gtm.baudom.net*) that point to their NS records to public DNS servers, such as Google’s 8.8.8.8, Cloudflare’s 1.1.1.1. However, the public DNS of Google or Cloudflare does not provide authoritative DNS answers. In

this scenario, we do not categorize these servers as ADNS, as per the criterion that the AA flag is not set in the DoE response.

3.4 Open Encrypted DNS Resolver

Private ODNS. We categorize the remaining servers that provide responses with an empty answer section and have the Recursion Available (RA) flag set but are not identified as ADNSes as private ODNS, further dividing them into two subtypes:

Table 1. Statistics of encrypted DNS servers w/o an answer section.

DoE Type	ADNS			Private ODNS		
	cert&PTR seeds	top list seeds	total	particular domains	specific users	total
DoT	1,092	850	1,283	646	62,229	62,875
DoH	169	9,004	9,088	873	4,508	5,381
DoQ	15	15	15	0	491	491

1. ***Servers responding to particular domain names.*** These servers respond with an empty answer section in the encrypted DNS server discovery phase. Upon further investigation, while identifying ADNSes, they successfully responded to queries for specific domain names, albeit without the Authoritative Answer (AA) flag set. A notable example includes Apple’s DoH resolvers with the domain *doh.dns.apple.com*, configured to respond exclusively to domains owned by Apple. ControlD exhibits similar behavior, providing DoT and DoH services tailored to specific domains.
2. ***Servers responding to specific user groups.*** The remaining servers are determined to cater to specific usage scenarios and user demographics. On the one hand, about 90% of the DoT responses void of an answer section are associated with servers whose certificates’ CN field matches **.uniproxy.dnsproxifier.com*, encompassing 126 unique domain suffix prefixes. Akamai manages 1,333 DoT and 3,400 DoH resolvers under the domain *akasecure.net*, indicating dedicated use. On the other hand, open-sourced resolvers feature Access Control Lists (ACLs). For example, solutions such as Adguard Home software can be specifically configured to process requests only from assigned clients, demonstrating a targeted DNS resolution.

Public ODNS. Public ODNSes are set up to resolve any DNS queries from any users. To ascertain the providers of these servers, We group the discovered servers according to the subject attributes in their TLS certificates. We utilize the Organization (O), Common Name (CN), and Subject Alternate Name (SAN) fields from the certificates to extract subject templates, each associated with a

DoE provider. The distribution of public ODNs providers follows a long-tail pattern, with more than 77% of providers managing only a single resolver address, while larger providers hold a substantial portion of the market.

Detailed statistics of the top 5 public ODNs are provided in Table 2. Among these, major providers such as Fortinet, apple dnssd-proxy, and BlueCoat are noted for their widespread use of gateways and firewalls. NextDNS stands out by offering DoT, DoH, and DoQ services. Compared to DoT and DoH, DoQ is less prevalent and predominantly serviced by NextDNS. The other providers of DoQ collectively operate at most 53 resolvers, illustrating the varied adoption rates of different DoE protocols across the industry.

To distinguish between FDNS and RDNS_{in}, we send ten unique subdomain queries for each ODNs using timestamps as a basis for generation, then correlated the received target IP address of RDNS_e from our ADNS. Our analysis determines that 15,533 ODNs function as FDNSes, forwarding queries to other ODNs, representing 64.83% of the total public ODNs population. Meanwhile, 8,427 ODNs are identified as RDNS_{in}, affiliated with the same organization as RDNS_e, accounting for 35.17% of the public ODNs pool.

Table 2. Statistics of the Top 5 Public ODNs for DoT, DoH, and DoQ.

DoE Type	DoT		DoH		DoQ	
Provider	Fortinet	6,150	NextDNS	2,490	NextDNS	2,491
	apple.dnssd-proxy	4,796	BlueCoat	138	AliDNS	53
	Cloudflare	2,594	ControlD	126	AdguardDNS	15
	NextDNS	2,491	Google	100	CubeDNS	10
	Cleanbrowsing	1,042	AliDNS	89	adguard.ch	8

4 Characterizing the Dependencies on Server Components

This section explores the component properties of the encrypted DNS servers by examining characteristics across the DNS and HTTP protocol layers, aiming to enhance understanding of the operational aspects and component dependencies of these servers.

4.1 Methodology

DNS Server Type. The software version of a DNS server can be discerned by querying the CHAOS class TXT DNS records for the `version.bind` or `version.server` domains. We apply this method to send such queries to the discovered encrypted DNS servers.

HTTP Server Type. An HTTP response header, comprised of key-value pairs separated by colons, offers details about the associated HTTP server. We preprocess the HTTP header data to facilitate a structured analysis of DoH servers’ characteristics, extracting attributes to construct a vector representation, denoted as $s_i = \{f_1, f_2, \dots, f_n\}$. The vector of a DoH server includes nominal and binary attributes.

- *Nominal Attributes.* We selectively retain values from meaningful fields that contribute to server banners, including `access-control-allow-origin`, `Server`, and `X-Powered-By`, among others, covering a total of 10 fields. To facilitate further analysis, we apply the one-hot encoding method, which transforms these fields into vector representations.
- *Binary Attributes.* We enumerate all header fields in HTTP response headers, creating a comprehensive list that includes all possible DoH header fields. For each HTTP response header analyzed, if a specific field is present, we assign a value of 1 to the corresponding bit in our sequence. Otherwise, the bit is set to 0 if the field is absent.

By combining the two types of attributes, we create a vector representation for each DoH server, which serves for server clustering analysis.

4.2 DNS Server

We utilize the CHAOS record class along with TXT type requests to determine the software running on DoE servers. Figure 7 presents the identification results of the top 10 DNS server software responding to the specialized version queries. The most frequent response came from SK Broadband, primarily through DoT Apple dnssd-proxies. We discover that approximately 8,090 DoE servers support legacy DNS queries, although most of these servers generally refuse to respond to such requests.

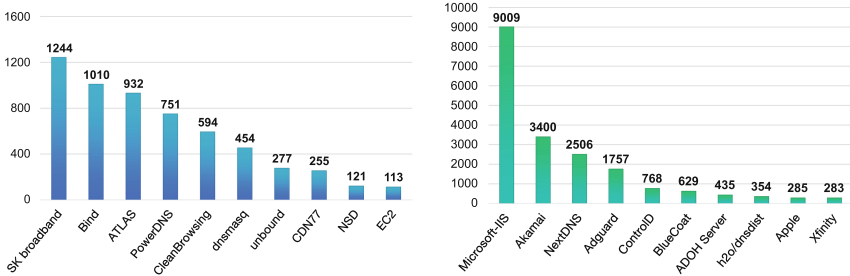


Fig. 7. The top 10 discovered DNS server software and HTTP server clusters.

4.3 DoH HTTP Server

Data Preprocessing. We first perform a frequency analysis of the occurrence of DoH HTTP response header fields. Fields that appeared fewer than 10 times across all HTTP headers are excluded, leaving us with 46 pertinent fields. To represent each nominal attribute, we extract templates using regular expressions and utilize a one-hot encoder to convert categorical data into a binary format. This process ultimately generates a 195-dimensional vector for each DoH server.

Results Analysis. We apply the K-Means algorithm to cluster the vectors derived from DoH servers' HTTP headers. To determine the optimal value of K , we use the Elbow Method, which identified 25 distinct clusters as the most efficient configuration. Figure 7 illustrates the top 10 DoH server clusters, presenting the variations in server software deployment. The categorization labels for each cluster are substantiated by information from the most frequently occurring server within each cluster. This information is sourced from the `server` field in the HTTP header or the Organization (O) and Common Name (CN) fields in the server's certificate. Such data provide a reliable foundation for the assigned labels. Our analysis not only identifies component or manufacturer clusters but also notes similarities in components across different servers and within component chains in a single ODNs.

Similarities. The clusters offer insights into the similarities of components in different providers. On the one hand, the components adopted by major DNS providers are revealed. For example, we identify QUAD9 and FortinetDNS in the h2o/dnsdist cluster, suggesting they use the same component provided by PowerDNS. On the other hand, we uncover a type of DoH forwarder that acts as an HTTP proxy forwarding query to large DNS providers (like Google). This kind of HTTP proxy exhibits similar HTTP headers to those of DNS providers but holds personal certificates. To validate the identity of these proxies, we visited their index pages and sent HTTP requests with an empty DNS parameter to trigger error page info, which showed the same page info as that of the corresponding DNS providers. Forty-eight HTTP proxy instances are observed in the Google and Cloudflare clusters.

Components Chain within an ODNs. From the user's perspective, a DoE query appears straightforward, but breaking it down from the standpoint of software components reveals a complicated process. Figure 8 provides a case illustrating the components chain we observed in clusters. From the perspective of software function realization, the process of DoE resolution can be summed up into two key stages:

- *Receiving and Decrypting DoE Queries:* The initial stage involves receiving incoming DoE queries, wherein the DoE entrance gateway or proxy decrypts the original DNS message encapsulated in the DoE query.

- *Sending Queries upstream:* Following the decryption of the original DNS message, the DoE entrance sends queries upstream, either in an encrypted or plaintext format. This step can be executed recursively.

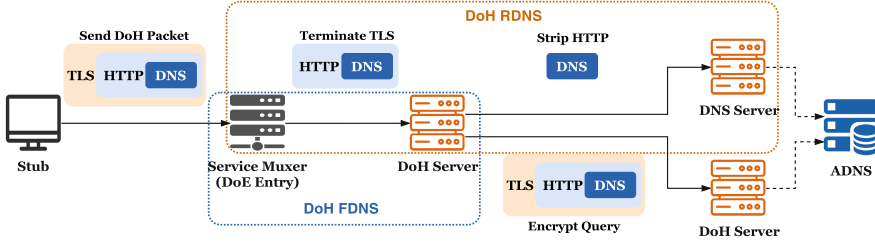


Fig. 8. The components chain within an ODNs from the perspective of software function realization.

The two essential stages can necessitate the collaboration of multiple components to achieve. In this scenario, the service muxer, like Nginx, functioning as a reverse proxy, initially receives and decrypts incoming DoH queries. Subsequently, it forwards these queries to the DoH server. The DoH server, in turn, removes the HTTP layer, exposing the original DNS query. The DNS query is finally relayed to a DNS server that performs a recursive query. Compared to Do53 transactions, the involvement of additional components introduces complexity in DoE resolutions.

5 Identify the Dependencies on Large DNS Providers

Previous research assesses concentration predominantly by examining the prevalence of providers associated with ODNs. However, these methods overlook the other crucial role of the egress resolvers ($RDNS_e$) and only capture a partial aspect of the resolver distribution. The $RDNS_e$ holds equal importance in comprehending the entire resolver ecosystem, particularly since multiple ODNs can send queries to a single $RDNS_e$.

We rely on BGP information related to ingress and egress IP addresses to determine the ownership. The prefixes of IP addresses are obtained from the RouteViews Prefix to AS mappings dataset [16], while ASN information is sourced from CIDR Report [4]. We categorize an ODN as $RDNS_{in}$ if the organization of the ODN matches the organization of the corresponding $RDNS_e$ ($Org_{ODN} = Org_{RDNS_e}$). To provide a more comprehensive evaluation of centralization with encrypted DNS, we propose the norm of a Service Reliance Index (SRI). The description of SRI is as follows.

Given a set of ODNs and $RDNS_e$ providers. We aim to calculate the SRI of $RDNS_e$ providers as follows:

$$S(p_i) = \sum_{o_j \in C(p_i)} \frac{1}{L(o_j)}, \quad (1)$$

where $p_i (0 < i \leq M)$ represents i -th RDNS_e providers and M is the number of DNS providers, $o_j (0 < j \leq N)$ represents j -th ODNS and N is the number of ODNS, $S(p_i)$ represents the SRI of i -th RDNS_e p_i , $C(p_i)$ represents the set of ODNSES that forwards queries to i -th RDNS_e provider p_i , $L(o_j)$ represents the number of RDNS_e providers that o_j forwards queries to.

The overall distribution of ODNSES and RDNS_e is depicted in Fig. 9. Notably, the top 10 RDNS_e serve for 75.24% of all ODNSES, highlighting the increasing centralization within the encrypted DNS landscape. This observation is further analyzed in Table 3, which presents the top 10 organizations according to their SRI rankings. Note that the top 10 in the SRI rank differ from the top providers of ODNSES listed in Table 3. This discrepancy arises because a provider might control a relatively small number of ODNSES but be heavily relied upon by numerous FDNSES. Specifically, the top two providers in the SRI rankings, CloudFlareNET and Google, provide DNS service for 47.9% of all FDNSES. The SRI metric offers a more comprehensive view by considering both ingress and egress resolvers.

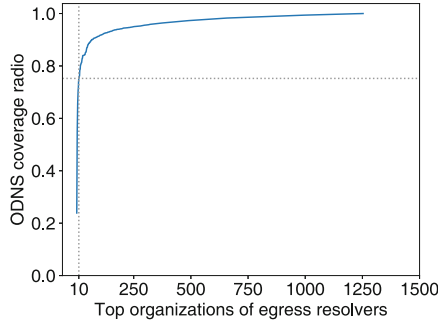


Fig. 9. Cumulative distribution of ODNSES and organizations of RDNS_ees.

Table 3. The top 10 organizations in the SRI rank.

Rank	Organization	SRI	Rank	Organization	SRI
1	CloudFlareNET	4399.28	6	SK Broadband	1078.27
2	Google	3657.24	7	AMAZON-02	587.68
3	AS-CHOOPA	2403.99	8	WOODYNET-1	516.75
4	COGENT-174	1507.50	9	KIXS-AS-KR Korea Telecom	496.00
5	FORTINET	1221.97	10	MISAKA Misaka Network	311.39

6 Discussion

6.1 Security and Privacy Implications

Our research unveils dependencies in encrypted DNS infrastructures, exposing potential vulnerabilities and risks. DoE resolvers incorporate more complex component chains than Do53, which can introduce vulnerabilities due to increased complexity. Each component, if compromised, can jeopardize the entire DNS resolution process, potentially causing downtimes or security breaches. An exploit in any single component could allow attackers to compromise other parts of the DNS resolution process. With each part of the resolver setup potentially harboring its vulnerabilities and varying in the level of security support, the component chain may raise the risk of crashes.

Moreover, the landscape of DoE resolvers is primarily dominated by a few major service providers. This concentration increases the risk of centralization, heightening the potential for single-point failures and extensive collateral damage in case of an outage or security breach. From a privacy perspective, many users remain unaware of DoE advancements [31], and providers often do not disclose details about server components and third-party involvements. Such centralization could foster data monopolization, posing risks to client privacy.

6.2 Limitations

Firstly, although we endeavor to collect as comprehensive a list of domain names as possible to determine the roles of encrypted DNS servers, approximately 5% of all servers remain unidentified due to limitations in the available features. Secondly, the characterization of server components is constrained by legacy DNS queries within encrypted environments. Due to ethical considerations, we limited our study to sending only a small number of requests, ensuring all were in the standard format. For future work, developing more types of DNS query messages could provide a more nuanced characterization of component types based on their response behaviors. Thirdly, while we have conducted extensive measurements of the upstream nodes of public ODNSES across multiple locations and times, our coverage might not be exhaustive. Nevertheless, we believe that our results provide significant insight into DNS provider dependencies, exceeding the lower bound of the metrics.

6.3 Ethical Considerations

Our study adheres to a rigorous set of ethical guidelines [21, 33]. We have ensured that our research does not involve any users, their personal information, or sensitive data. Instead, our study focuses on publicly accessible and available DoE services. To prevent any harm to infrastructure, we have taken measures to limit the rate of our requests and have refrained from making malicious requests. We conducted our experiments using domains under our control to ensure that they

do not interfere with the resolution of other domains or compromise the privacy of other users. We believe the impact of our experiments on each server is negligible. During our experiment, we received no complaints from our network provider.

7 Related Work

This section summarizes the previous research related to our work, especially measurement studies on DoE deployment and DNS infrastructures.

7.1 Measurement of DoE Deployment

Previous studies have provided various insights into the deployment of DoE. Lu et al. [29] performed the first large-scale measurement of DoT and DoH deployment by scanning the IPv4 address space for DoT servers and analyzing URL patterns for DoH servers in extensive datasets. Böttger et al. [19] evaluated a list of DoH servers maintained by the curl project, noting that DoH resolvers exhibited flexible response capabilities across different URL paths. Deccio et al. [20] conducted partial measurements to evaluate the adoption of DoH among open Do53 resolvers by active scanning. García et al. [22] developed an Nmap Lua script to locate DoH services, filtering results based on HTTP status codes and DNS answers to identify DoH resolvers, testing across both HTTP/1.1 and HTTP/2 with different methods such as HTTP GET and POST requests. Kosek et al. [26] undertook the first measurement study of DoQ, using Zmap to identify nodes supporting QUIC, then verifying DoQ support through ALPN field negotiation, covering various QUIC and DoQ protocol versions. Luo et al. [30] introduced an extensive approach for discovering encrypted open resolvers by sourcing domain names associated with potential IP addresses and validating them with valid DoE responses. Li et al. [28] measured the DNS strict privacy from multiple views. They identified encrypted DNS resolvers through the presence of the RA flag set in responses, and they pinpointed encrypted ADNSes using seeds derived from top domain lists.

Compared to previous studies, our methodology for distinguishing DoE servers hinges on whether the returned messages adhere to the DNS wire-format standard. We include an analysis of DoE servers that return empty responses, a factor often overlooked in other studies. We specify the roles of discovered DoE servers by evaluating their responses to various queried domain seeds. This approach offers a more comprehensive perspective on the adoption and functionality of encrypted DNS.

7.2 Measurement of DNS Infrastructures

Several studies have investigated the intricacies of DNS infrastructures. Schomp et al. [34] introduced a classification framework for client-side DNS servers, including open DNS, recursive DNS, and hidden DNS servers. Their work focused

on discovering complex client-side DNS infrastructures and distinguishing the behaviors of various actors. Kühner et al. [27] explored the DNS resolver landscape by analyzing empirical data collected over a year. Their research involved tracking changes in the DNS ecosystem and categorizing DNS resolvers based on device type and software version, providing a dynamic view of the DNS resolver population. Park et al. [32] identified over 3 million open resolvers in the wild, many of which operated in non-standard ways, highlighting potential risks and irregularities in DNS operations. Xu et al. [36] conducted Internet-wide measurements of both client-side and server-side DNS infrastructures to evaluate the extent of DNS centralization. Their findings revealed that over 90% of forwarding resolvers rely on approximately the top 300 DNS providers. This study underscored the centralization of DNS infrastructures.

Our research concentrates on the encrypted DNS landscape. We delve into the roles within encrypted DNS infrastructures and analyze dependencies from the perspectives of server components and large DNS providers. We aim to fill the gap concerning dependencies and centralization in encrypted DNS.

8 Conclusion

In this paper, we delve into the dependencies in encrypted DNS infrastructures, examining from the perspectives of server components and the DNS resolution process. Our methodology includes the development of a hierarchical model for classifying encrypted servers, followed by a thorough analysis of their dependencies. This research brings to light the varied roles of servers, the presence of multiple components within encrypted DNS servers that work in concert during the resolution process, and highlights their interdependent nature. Furthermore, our study reveals an increasing reliance on third-party DNS providers within the encrypted DNS ecosystem. Our findings provide a comprehensive view of the current state of encrypted DNS, contributing to informed decision-making and future research in this field. We believe that these insights are necessary for stakeholders and researchers who aim to understand the complexities and ongoing developments in the encrypted DNS ecosystem.

Acknowledgements. The authors would like to thank the anonymous reviewers for their valuable comments and helpful suggestions. This work is supported by the Scaling Program of the Institute of Information Engineering, CAS (No. E3Z0191101, No. E3Z0041101).

References

1. AdGuard DNS — ad-blocking DNS server. <https://adguard-dns.io/en/public-dns.html>
2. Alexa top 1m domains list. <https://s3.amazonaws.com/alexa-static/top-1m.csv.zip>
3. BuiltWith top 1m rank. <https://builtwith.com/top-1m>

4. CIDR report. <https://www.cidr-report.org/as2.0/>
5. Cisco umbrella top 1m domains list. https://www.trisul.org/devzone/doku.php/cisco_umbrella_top-1m_domains_list
6. Cloudflare radar. <https://radar.cloudflare.com/>
7. DNS over HTTPS (aka DoH). <https://www.chromium.org/developers/dns-over-https/>
8. dnspython. <https://github.com/rthalley/dnspython>, original-date: 2011-08-24T11:36:39Z
9. Encrypt DNS traffic · cloudflare 1.1.1 docs. <https://developers.cloudflare.com/1.1.1/encryption/>
10. Encrypted DNS. <https://quad9.net/de/support/set-up-guides/encrypted-dns>
11. Firefox DNS-over-HTTPS | Firefox Help. <https://support.mozilla.org/en-US/kb/firefox-dns-over-https>
12. Majestic million. <https://majestic.com/reports/majestic-million>
13. Microsoft Edge Privacy Whitepaper - Microsoft Edge Development | Microsoft Docs. <https://docs.microsoft.com/en-us/microsoft-edge/privacy-whitepaper/>
14. A research-oriented top sites ranking hardened against manipulation - tranco. <https://tranco-list.eu/>
15. Secure transports for DNS | public DNS | google for developers. <https://developers.google.com/speed/public-dns/docs/secure-transports>
16. Routeviews prefix to AS mappings dataset (pfx2as) for IPv4 and IPv6 (2008). <https://www.caida.org/catalog/datasets/routeviews-prefix2as/>
17. B-root offers experimental support for DNS over TLS (2023). <https://b.root-servers.org/news/2023/02/28/tls.html>
18. New addresses for b.root-servers.net (2023). <https://b.root-servers.org/news/2023/05/16/new-addresses.html>
19. Böttger, T., et al.: An empirical study of the cost of DNS-over-HTTPS. In: Proceedings of the Internet Measurement Conference, pp. 15–21. IMC '19, Association for Computing Machinery. <https://doi.org/10.1145/3355369.3355575>
20. Deccio, C., Davis, J.: DNS privacy in practice and preparation, pp. 138–143. CoNEXT '19, Association for Computing Machinery, New York, NY, USA (2019). <https://doi.org/10.1145/3359989.3365435>
21. Durumeric, Z., Wustrow, E., Halderman, J.A.: ZMap: fast internet-wide scanning and its security applications. In: 22nd USENIX Security Symposium (USENIX Security 13), pp. 605–620. USENIX Association, Washington, D.C. (2013). <https://www.usenix.org/conference/usenixsecurity13/technical-sessions/paper/durumeric>
22. García, S., Bogado, J., Hynek, K., Vekshin, D., Čejka, T., Wasicek, A.: Large scale analysis of DoH deployment on the internet. In: Atluri, V., Di Pietro, R., Jensen, C.D., Meng, W. (eds.) Computer Security – ESORICS 2022. ESORICS 2022. LNCS, vol. 13556. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-17143-7_8
23. Hoffman, P., McManus, P.: DNS queries over HTTPS (DoH). <https://doi.org/10.17487/RFC8484>, <https://www.rfc-editor.org/info/rfc8484>
24. Hu, Z., Zhu, L., Heidemann, J., Mankin, A., Wessels, D., Hoffman, P.E.: Specification for DNS over Transport Layer Security (TLS). RFC 7858 (2016). <https://doi.org/10.17487/RFC7858>, <https://www.rfc-editor.org/info/rfc7858>
25. Huitema, C., Dickinson, S., Mankin, A.: DNS over Dedicated QUIC Connections. RFC 9250 (2022). <https://doi.org/10.17487/RFC9250>, <https://www.rfc-editor.org/info/rfc9250>

26. Kosek, M., Doan, T.V., Granderath, M., Bajpai, V.: One to rule them all? A first look at DNS over QUIC. In: Hohlfeld, O., Moura, G., Pelsser, C. (eds.) PAM 2022. LNCS, vol. 13210, pp. 537–551. Springer, Cham (2022). https://doi.org/10.1007/978-3-030-98785-5_24
27. Kühner, M., Hupperich, T., Bushart, J., Rossow, C., Holz, T.: Going wild: large-scale classification of open DNS resolvers. In: Proceedings of the 2015 Internet Measurement Conference, pp. 355–368. IMC '15, Association for Computing Machinery, New York, NY, USA (2015). <https://doi.org/10.1145/2815675.2815683>
28. Li, R., et al.: A longitudinal and comprehensive measurement of DNS strict privacy. *IEEE/ACM Trans. Netw.* 1–16 (2023). <https://doi.org/10.1109/TNET.2023.3262651>
29. Li, R., et al.: A longitudinal and comprehensive measurement of DNS strict privacy. *IEEE/ACM Trans. Networking* **31**(6), 2793–2808 (2023). <https://doi.org/10.1109/TNET.2023.3262651>
30. Luo, M., Yao, Y., Xin, L., Jiang, Z., Wang, Q., Shi, W.: Measurement for encrypted open resolvers: applications and security. *Comput. Netw.* **213**, 109081 (2022). <https://doi.org/10.1016/j.comnet.2022.109081>, <https://www.sciencedirect.com/science/article/pii/S1389128622002183>
31. Nisenoff, A., Sharma, R., Feamster, N.: User Awareness and Behaviors Concerning Encrypted DNS Settings in Web Browsers, pp. 3117–3133 (2023). <https://www.usenix.org/conference/usenixsecurity23/presentation/nisenoff-awareness>
32. Park, J., Jang, R., Mohaisen, M., Mohaisen, D.: A large-scale behavioral analysis of the open DNS resolvers on the internet. *IEEE/ACM Trans. Netw.* **30**(1), 76–89 (2022). <https://doi.org/10.1109/TNET.2021.3105599>
33. Partridge, C., Allman, M.: Addressing ethical considerations in network measurement papers: Abstract. In: Proceedings of the 2015 ACM SIGCOMM Workshop on Ethics in Networked Systems Research, pp. 33. NS Ethics '15, Association for Computing Machinery, New York, NY, USA (2015). <https://doi.org/10.1145/2793013.2793014>
34. Schomp, K., Callahan, T., Rabinovich, M., Allman, M.: On measuring the client-side DNS infrastructure. In: Proceedings of the 2013 conference on Internet measurement conference, pp. 77–90. IMC '13, Association for Computing Machinery (2013). <https://doi.org/10.1145/2504730.2504734>
35. Wan, G., et al.: On the origin of scanning: the impact of location on internet-wide scans. In: Proceedings of the ACM Internet Measurement Conference, pp. 662–679. IMC '20, Association for Computing Machinery (2020). <https://doi.org/10.1145/3419394.3424214>
36. Xu, C., et al.: Measuring the centrality of DNS infrastructure in the wild **13**(9), 5739. <https://doi.org/10.3390/app13095739>
37. Zirngibl, J., Buschmann, P., Sattler, P., Jaeger, B., Aulbach, J., Carle, G.: It's over 9000: analyzing early QUIC deployments with the standardization on the horizon. In: Proceedings of the 21st ACM Internet Measurement Conference, pp. 261–275. IMC '21, Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3487552.3487826>



Netfuzzlib: Adding First-Class Fuzzing Support to Network Protocol Implementations

Jeroen Robben^(✉)  and Mathy Vanhoef 

DistriNet, KU Leuven, Belgium
`jeroen.robben@kuleuven.be`

Abstract. This paper introduces **Netfuzzlib**, a framework that emulates network input and output in user space, allowing more efficient fuzz testing and symbolic execution of Linux network programs. We first systemize common requirements and caveats for efficient fuzzing and symbolic execution of socket-based network software. Building on top of the collected requirements, we design and implement a framework that seamlessly integrates with state-of-the-art fuzzing tools, as well as symbolic execution engines, and requires no knowledge of the program's source code. Moreover, Netfuzzlib enables a range of speed optimizations, where in our evaluations it improved fuzzing speed by a factor ranging from roughly 10 to 600. By using this framework it becomes significantly easier to fuzz and symbolically analyze Linux network programs.

Keywords: Network security · Fuzzing · Dynamic testing

1 Introduction

Fuzzing is a popular dynamic testing technique that has gained significant traction due to its effectiveness in uncovering software bugs and vulnerabilities. A particularly interesting fuzzing target is network programs, since they process input received over a remote connection, making them a common target for adversaries. Recent research has introduced a plethora of approaches for generating fuzz test cases for network software, and while doing so revealed various types of errors, ranging from common issues like buffer overflows to logic bugs such as illegal protocol state transitions [16, 29, 52].

Although past research has made strong progress on how to fuzz network programs, we observed that fuzzing over network connections is still slow and inefficient in practice. Using symbolic execution to test network programs is an equally challenging task: even state-of-the-art symbolic execution engines cannot, by default, handle network-related kernel interactions [10]. The current ad hoc approach to circumvent these problems is to write custom test harnesses that test specific components of the program in isolation. However, this has several disadvantages: it requires access to the source code of the System Under Test (SUT), might not cover certain code paths and thus miss bugs, and places a substantial workload on the tester in terms of engineering effort.

In this paper, we design, implement, and evaluate a framework called Netfuzzlib that seamlessly integrates into state-of-the-art fuzzers and symbolic execution engines, with as goal to more easily and efficiently test network programs. To accomplish this, we first systemize common requirements and caveats when testing or fuzzing network software. Building on top of the collected requirements, we then design a framework for Linux network programs that emulates all network-related kernel interactions in user space with as goal to simultaneously make fuzzing easier and more efficient.

We improve fuzzing speed through several optimizations. First, by synchronizing network I/O with the fuzzer, we eliminate the need for delays, timeouts, and circumvent rate-limiting and dropped packets. Second, our framework allows one to easily replace socket I/O with a more efficient Inter-Process Communication (IPC) mechanism, such as shared memory. Third, Netfuzzlib allows the System Under Test (SUT) to notify or transfer execution from the SUT to the fuzzer on important network I/O-related events. Finally, we use the framework to provide a liveness heuristic, allowing the fuzzed program to automatically self-terminate when all fuzz input data has been processed, avoiding costly timeouts or delays that are used by existing network-based fuzzers.

We evaluate our framework on a set of network applications. This demonstrates an increased performance compared to other state-of-the-art fuzzers. Additionally, our evaluations illustrate the ability to more seamlessly symbolically analyse network services. All combined, this led to the discovery of three new vulnerabilities, which were each assigned a CVE identifier.

To summarize, our main contributions are:

- We systemize the requirements that fuzzing and symbolic execution tools must fulfill in order to seamlessly and efficiently support network software (Sect. 3).
- We propose and implement a framework to facilitate the fuzzing of Linux network programs (Sect. 4 and 5).
- We evaluate our framework with state-of-the-art fuzzing and symbolic execution tools, resulting in the discovery of three new vulnerabilities (Sect. 6).
- We publicly release our framework as well as documentation and examples.

Finally, Sect. 2 provides a background on fuzzing network software, and we compare to related work in Sect. 7 and conclude in Sect. 8.

2 Background: Fuzzing Network Software

2.1 Modern Fuzzing Approaches

Fuzzing Renaissance. Fuzzing is an umbrella term for dynamic testing methods where invalid or unexpected inputs are repeatedly evaluated on the System Under Test (SUT) to find an input that triggers a bug. In the last decade, we have seen a revival in fuzzing interest and effectiveness, initiated by the advent of feedback driven grey-box fuzzing [46, 50] and advancements in white-box fuzzing

with constraint solvers [13,22]. There is an increasing trend to integrate fuzzing into the continuous development process of software systems, which has proven to be effective in finding bugs and improving programs' robustness. For instance, Google found over 6,000 security-related bugs in its Chrome browser using coverage-guided fuzzing [2] and Microsoft announced that during the development of Windows 7, roughly 33% of all bugs identified were uncovered through the use of symbolic execution [22].

Applying Fuzzing. Unit-level, harness-based fuzzing is often regarded as the most effective and generic way to apply fuzzing [17]. The tester manually writes one or more test harness(es), which set-up the testing environment, invoke some target function with fuzz input and reset the program's state for every fuzz test. This approach allows the tester to pinpoint the entrypoint of the code to be fuzzed, allowing to test parts of the program in isolation which can otherwise be challenging to cover thoroughly.

Another advantage is that unit-level test harnesses can often be implemented to support *in-process* or *persistent mode* fuzzing [3]. Here, the same process is reused for various fuzz inputs, enabling a lightweight reset of the program state for processing subsequent inputs without the requirement of starting a new process, significantly enhancing fuzzing-throughput.

Network protocols often require an exchange of multiple messages in order to fulfill the intended operation, usually leading to implementations which maintain state over some session. While unit-level fuzzing is also effective to find bugs in network protocol implementations, this approach generally focuses on testing the handling of a single message within a particular state of the program. This forces the tester to identify and make assumptions on which states are globally reachable, and the requirements that will be met when executing certain parts of the code. Moreover, harness-based fuzzing typically requires an understanding of (and access to) the program's source code. It demands additional engineering effort and can be complex to implement correctly, especially due to the intertwining of I/O handling logic within other code. There is also a risk of missing bugs if the implemented fuzzing harnesses do not adequately cover all the program's code and states. Finally, some types of vulnerabilities for stateful systems, such as illegal state transitions, can not be detected by fuzzing each state individually.

2.2 Fuzzing of Stateful Systems

Recent research in stateful fuzzing has shown that fuzz testing stateful systems as a *whole* allows to uncover new bugs and vulnerabilities, by integrating *awareness* of the interactions/transitions between several program states into the design of the fuzzer. A categorization on stateful fuzzing techniques is available in the surveys by Daniele et al. and Zhang et al. [16,52].

The first end-to-end fuzzers for stateful systems used a generative black-box approach [5,21]. That is, these fuzzers did not use any form of feedback, but only used a model of the implemented protocol, e.g., in the form of a grammar or sample traces of valid messages, in order to generate meaningful test

cases. In contrast, more modern feedback-guided state-aware fuzzers make use of heuristics to track state transitions, for example, by extracting the semantics of received network messages [37] or by inspecting the internal memory at certain I/O related events, such as when sending or receiving data [8, 9, 31, 34]. More concretely, three notable types of state-guided network-based fuzzers are:

- In *stateful coverage-guided fuzzing* (SCGF), the combination of AFL-style edge coverage and state feedback is used in order to guide the mutation of a test corpus [9, 25, 34, 37, 49].
- *Protocol state fuzzing* uses active automata learning to fuzz the order of protocol messages to detect deviations from the protocol specification [15, 20, 42].
- PISE and MACE use symbolic execution and automata learning to infer the state machine and message formats of a network protocol, and to find possible security vulnerabilities such as memory corruption errors [14, 30].

2.3 Fuzzing Over Network I/O

Fuzzing programs over network I/O poses some unique requirements and caveats compared to testing programs which process input over some other channel, such as reading a file or via command-line arguments. In addition, grey-box feedback methods for tracking state changes require new kinds of interactions with the SUT *during* the processing of a test case, as was explained in Sect. 2.2. To provide a better understanding of these challenges, we first review the most common API used for a program to perform network I/O.

Sockets. On most general purpose operating systems such as Linux and Microsoft Windows, the kernel requires user space programs to do network I/O using an interface based on the Berkeley sockets API [27, 33, 40]. Sockets are an abstraction provided by the kernel meant to depict a *communication endpoint*. Apart from Internet connections, sockets are also used for other types of inter-process communication, e.g., UNIX or Netlink sockets. All socket state is held in the kernel and programs only make use of a *handle* to refer to a socket. Following common Unix-philosophy, on Linux, socket instances are referred to in user space using *file descriptors*. The Linux IP sockets implementation closely follows the POSIX specification [24]. Common C standard library implementations for Linux, such as glibc, aim to implement the C POSIX specification and provide a set of wrapper functions which invoke the correct syscall(s) for interacting with sockets.

3 Requirements for End-to-End Network Fuzzing

In this section, we identify, systemize, and discuss specific requirements for applying recent fuzzing techniques to network programs and how existing fuzzers try to meet these requirements. As we will explain, some of the following requirements only apply to certain fuzzing techniques, such as symbolic execution.

3.1 Requirement 1: Determinism

Feedback-guided fuzzing systems require some sense of determinism on the feedback received from the SUT on the fuzzing input to ensure that each test is consistent and not influenced by earlier inputs. For example, active automata learning algorithms such as L* and TTT, expect the same message type to be returned for the same trace of message types that were sent as input [7]. If this requirement is not met, e.g., due to the influence from a previous fuzzing cycle or an occasionally dropped packet, the learned state model could be erroneous.

Existing Approaches. The required degree of determinism in stateful fuzzing varies widely and can refer to both external observations as well as internal processes of the target depending on the used feedback method(s). Some fuzzing techniques only require a new *protocol session* to be started, e.g., by closing the underlying TCP tunnel and initiating a new connection [42]. Fuzzers that require a more profound state reset, e.g., coverage-guided fuzzers, usually restart the SUT from its *main* entrypoint for every test case. Recent research proposes to use *snapshots* to revert the target state. [4, 6, 25, 28, 43, 48]. Section 3.5 further elaborates on existing optimizations for resetting state and their impact on fuzzing speed.

3.2 Requirement 2: Data Readiness Detection

Classic network fuzzers are agnostic on the exact moment when the program is ready to receive network data. For TCP, the fuzzer typically uses a busy loop to try to connect to the SUT [34, 36, 37]. If the target is fuzzed over a connectionless network layer, such as when using UDP or raw IP packets, existing fuzzers cannot distinguish between the kernel dropping a packet because no socket was yet configured to receive the message, or because the tested program has received the packet but not (yet) transmitted a reply.

Existing Approaches. Most network fuzzers ensure that the SUT has passed the initialization step and is ready to receive messages by introducing a fixed *delay* before sending the first packet. Fine-tuning the duration of this delay is hard and often done in an ad-hoc fashion. If the delay is set too long, time is lost for the duration between the SUT being ready to receive input and the fuzzer sending a message. If the delay is too short, packets might be dropped [15, 34, 36, 37]. Recent work has proposed methods to make network I/O synchronous for effective fuzzing, i.e., by notifying the fuzzer when the SUT is ready to receive or send network data [6, 28, 39, 43].

3.3 Requirement 3: Liveness Detection

When fuzzing a specific function in isolation, control of execution typically returns to the caller once the function has finished processing the test case. On the contrary, network services usually do not terminate after processing input,

but wait indefinitely for new incoming requests. After transmitting a test case, the fuzzer cannot differentiate between the SUT still processing the submitted message(s), being stuck in an infinite loop due to a bug, or having processed the submitted data without sending a reply. For state-aware fuzzing, a premature reset can cause the failure to detect new states, for example, due to the tested program not having enough time to send a reply [15, 20].

Existing Approaches. In some protocols, the SUT explicitly terminates the session by sending a protocol-specific response message or by closing the underlying communication channel, e.g., a TCP tunnel. Most fuzzers consider the target to have completed a test case after a fixed *delay* has elapsed following the transmission of the final network message [20, 36, 37]. The processing time required by the SUT can vary widely depending on the specific fuzz input. Messages covering more code take longer to process and are likelier to exhibit interesting behavior. Therefore, the delay should be set sufficiently high to allow the target to fully process these test cases. Conversely, malformed data is often rejected early in the handling process, as is likely the case with most fuzzed messages. Therefore, a long delay can result in considerable idle time for the majority of fuzz inputs.

3.4 Requirement 4: Transfer of Execution on I/O Events

Some network-based grey-box fuzzers interact with the SUT upon certain I/O-related events, such as when receiving or sending a network message. For instance, *StateAFL* tracks the iterations of request/reply exchanges by hooking into `send()` and `receive()` calls. It utilizes the moment the SUT attempts to receive a packet to take snapshots of memory associated with a protocol session in order to track state changes [34]. As another example, *NSFuzz* adds compile-time instrumentation to raise a SIGSTOP signal upon entry into the network event loop, thereby pausing the target’s process. This pause is used to notify the fuzzer that the SUT is ready to receive a new message and to process possible changes of tracked state variables [39]. Lastly, snapshot-based fuzzers typically use the moment when the SUT attempts to receive a message to create a snapshot [6, 28, 43].

Existing Approaches. Programs typically perform network I/O using standard library functions, such as `poll()`, and `recv()`, which in turn invoke the corresponding kernel syscall(s). Various methods are employed to execute fuzzer-controlled code when calling network I/O-related methods. *StateAFL* and *NSFuzz* add instrumentation during compile-time [34, 39]. *Nyx-net* uses LD-PRELOAD to have calls to socket I/O-related functions linked to their runtime library instead of the normal standard library implementation [43]. Additionally, some fuzzers insert hooks at the system call level: *SnapFuzz* performs binary rewriting of syscall invocations in the target and its dependencies during runtime, while *FitM* patches QEMU’s syscall translation layer [6, 28].

3.5 Requirement 5: High Fuzzing Throughput

The input space that fuzzers sample test cases from is vast. A single network packet with a size of 100 bytes already encompasses more than 10^{240} combinations. In addition to finding smarter methods for generating fuzz inputs, an obvious strategy to uncover more bugs is to conduct more fuzz cycles. Beyond throwing more compute power at the problem, one can increase the number of fuzzing cycles by reducing the amount of time spent on executing each test case. By studying existing network-based fuzzers, we identified a set of key causes that contribute to low fuzzing throughput in state-aware and network fuzzers:

Expensive State Reset and Work Repetition. As was explained in Sect. 3.1, some form of state reset of the SUT needs to be performed for each fuzzing cycle. In the case where the SUT is completely restarted, a new process must be created, and the initialization step has to be repeated for every test case. Furthermore, when targeting a deeper state repeatedly, the same set of prefix messages must be exchanged to bring the SUT to the desired state. This *work repetition* means that the portion of time spent on computations dependent on the fuzz input might be limited compared to the total time used for each fuzzing cycle. Finally, when doing a reset through a complete restart, termination is usually handled by sending a termination signal such as a SIGTERM. Sending and handling such termination signals is slow, which further delays the fuzzing cycle.

Existing Approaches. AFL introduced the use of a *forkserver* for fuzzing. In this approach, instrumentation is added to suspend and create a clone of the target process just before its *main* entry point for each fuzz case. This eliminates the need to repeat the `execve()` call, linking, and libc initialization steps, but still requires the creation of a new (child) process for each test input [50].

Recent work uses various snapshot mechanisms to revert the SUT’s state to a prior execution point, either at the process level or at the system level. The goal of this is to replace work repetition with a presumably less expensive checkpoint and restore mechanism, such as by making a snapshot when having reached some deeper state. *SnapFuzz* uses an AFL-style *deferred* forkserver placed at the “*latest safe point*”, e.g., just before creating the first child thread or receiving a packet. *FitM* and *SNPSFuzzer* use CRIU user space snapshots [25, 28]. Xu et al. propose to add process-level snapshot support to the OS kernel for efficient fuzzing [48]. *Nyx-net* and *Snapchange* provide a lightweight KVM-based snapshot and restore mechanism, both utilizing copy-on-write [4, 43].

Delays. Another source of time loss in each fuzz cycle is due to the use of delays, either introduced by the fuzzer or by the application developer. Sections 3.2 and 3.3 explain the use of delays to wait for the SUT to become ready to receive input, or to give the target enough time to process a test case before performing a reset. As was already mentioned in *Snapchange*, for some targets, the fuzzing throughput is also reduced due to developer-added delays [4], for example, by adding sleep intervals while busy polling for new data.

Existing Approaches. Recent research proposed methods to make network I/O synchronous by adding a mechanism to notify the fuzzer that the SUT is ready to receive data, eliminating the need for most fuzzer-added delays [28, 39, 43]. In addition, Snapchange uses binary rewriting during runtime to remove developer-added delays [6]. Unfortunately, the implementation of these approaches is specific to each fuzzer, and currently requires substantial work to also incorporate in other network-based fuzzers.

Slow I/O. When transferring data using network sockets, the message payload is copied from user space to kernel space and vice versa for every message sent. Compared to other IPC methods, IP sockets are relatively slow [23]. Furthermore, to exchange multiple packets, various context switches are needed between the processes of the fuzzer, the SUT, and the kernel, which slows down fuzzing.

Existing Approaches. A common approach to speedup network I/O is to add hooks to replace IP with UNIX sockets [6, 44, 51]. Linux uses an efficient implementation based on remapping memory pages for transferring data over UNIX sockets, avoiding the need for byte-per-byte copying of messages. The semantics and operations of UNIX sockets differ from those of IP sockets. For instance, the auxiliary data returned by *recvmsg()* or the options that can be configured using *setsockopt()*, requiring the inserted hooks to translate between UNIX and IP sockets semantics in order to test most real-world network programs.

3.6 Requirement 6: Handling Kernel Interaction

The last requirement we identified is specific to symbolic execution. Most symbolic execution engines can only trace symbolic values in user space, which poses some challenges since operations on network sockets invoke the kernel [10].

First, the symbolic execution engine cannot deduce the relationship between the passed (possibly symbolic) input arguments and the returned data. For example, on Linux, *send()* returns the amount of bytes that were transmitted or -1 in case of an error. When attempting to send data with a (bounded) symbolic length, an appropriate value must be returned. Some symbolic execution engines handle external calls by executing the intractable code with concrete arguments, and consequently limiting the *concretized* symbolic variables to a single value, which might lead to unexplored branches or missed bugs.

Second, when handling external calls by concretizing symbolic values, the symbolic execution engine will not be able to track side effects. Branching symbolic executors, such as KLEE, do not restart the SUT when finishing a program path, but continue the exploration process from a previous branch point [13]. For example, if path A closes a socket which is shared with path B, path B will encounter an error when sending data on the (now closed) socket, making the analysis unsound.

Finally, incoming network data must be marked as *symbolic* using the API provided by the symbolic execution engine, ideally without having to modify the SUT's source code.

Existing Approaches. The usual approach to apply symbolic execution to network programs is to patch the program’s source code, remove socket-related calls and manually declare the correct variables and buffers as symbolic. Some symbolic execution engines such as KLEE and Angr ship with an environment model for certain OS calls, but these do not support network I/O [13, 45]. To the best of our knowledge, no sound environment models for Linux networking exist.

4 Motivation

In this section, we elaborate on the shortcomings of current network-based fuzzers from the perspective of both testers/developers and researchers. This provides a direct motivation for the creation of our framework, namely, to better facilitate the fuzzing and testing of network-based software.

4.1 Tester Perspective

Fuzzing has proven to be an effective and useful bug detection method, and fuzz testing a program for the first time is likely to reveal bugs, even with relatively few computing resources [12]. In order to spread its adoption and integrate fuzz testing into the day-to-day development process, ideally, application developers should not require knowledge of fuzzer internals nor have to modify the source code of the tested program. Although effective end-to-end fuzzers for certain targets, such as command-line programs exist, automated fuzzing over other input methods remains an open problem [11]. Additionally, fuzzer usability is still not sufficient [11, 38].

Setting up and configuring recent network fuzzers often requires the tester to have in-depth knowledge of the used fuzzing system. Various methods for speeding up fuzzing have been proposed, but they often lack direct portability across different fuzzers or are complex to use, such as when requiring a custom hypervisor for snapshot fuzzing. Applying symbolic execution to network software almost always requires patching source code. Consequently, in practice, testers of network programs often default to using simpler, blind, black-box, or grammar-based fuzzers due to their ease of use.

4.2 Fuzzing Researcher Perspective

Network protocol fuzzing is an active research topic, with numerous publications and advancements in recent years [16, 52]. To evaluate a new idea for improving a fuzzer building block, such as a new mutation algorithm or state tracking method, researchers typically implement a prototype, and then compare certain metrics such as code coverage or bugs found over a given time frame with other fuzzers [32, 35]. To soundly compare new fuzzing techniques with existing fuzzers, it is important that SUT-specific optimizations, such as (not) avoiding delays when doing network I/O, are identical between the compared fuzzers [19, 41]. Apart from this, the methods used to transfer execution control from the SUT

to the fuzzer on I/O events are technically complex and often reimplemented, possibly in slightly different ways, which again can make it difficult to soundly isolate and evaluate specific fuzzing improvements (see Sect. 3.4). These issues create the need for a solution that can support/optimize the fuzzing of network programs which can seamlessly integrate with different fuzzers, so that different fuzzers can more easily be compared on equal footing.

All combined, to better support the use of, and research into, network-based fuzzers, we provide a framework for fuzzing of Linux network software in user space. The framework combines existing methods and adds new ones to increase fuzzing throughput, allows symbolic execution without altering source code, and provides an API for transferring control to the fuzzer on network I/O.

5 Overview of Netfuzzlib

In this section, we first present the approach of Netfuzzlib. We then elaborate on its design, covering the fuzzing module interface, selection of kernel APIs and how we ensured consistency with the host’s system configuration, and introduce a new liveness heuristic. Finally, we describe the implementation and usage of Netfuzzlib in practice.

5.1 Approach

The Netfuzzlib framework consists of a *main library*, which implements a set of networking-related functions and syscall wrappers in the GNU C library entirely in user space. The main library is accompanied with a fuzzer-specific *fuzzing module*, which itself is another library that, just like the main library, runs in the same process(es) as the SUT and defines the outcome of certain I/O events. When a new IP packet is to be received or sent, or the target attempts to open a new TCP connection, the corresponding handler in the module is called. The fuzzing module then performs fuzzer specific tasks, such as returning the next message in a test case or inspecting the SUT’s internal memory. To communicate with the fuzzer, any IPC-mechanism of choice, e.g., shared memory, can be used.

5.2 Fuzzing Module Interface

The fuzzing module is a dynamic or static library that exports the handler functions defined in `<netfuzzlib/module.api.h>`. The functions that are defined herein, which the user must provide a (stub) implementation for, are:

- `nfl_initialize()` This function is automatically called during initialization of the framework, just before the SUT’s `main()` entrypoint.
- `nfl_tcp_connect()`: Called when the SUT attempts to initiate a new outbound TCP connection, i.e., when invoking `connect()` on a TCP connect.
- `nfl_tcp_accept()`: Used when the framework requires knowledge of a pending inbound TCP connection, for example, when the SUT calls `accept()` or `poll()` on a TCP socket in listening mode.

- `nfl_send()`: Invoked when the SUT attempts to send data, e.g., using `sendmsg()`.
- `nfl_receive()`: Called when the framework needs information about the *next* packet to be received, e.g., when using `poll()` or `recv()`. Messages received from the fuzzing module are added to a queue until consumed by the SUT.
- `nfl_end()`: Invoked when the liveness heuristic indicates that all received packets have been processed, see Sect. 5.5.

5.3 Included Kernel APIs

Methodology. To decide which kernel functionalities to include in the *main library*, a list of Linux kernel APIs related to networking was made, using the Linux man-pages project and the latest POSIX specification [24, 27]. Then, the usage of these APIs in real-world network programs was examined. Particular attention was made to allow sound symbolic execution, as discussed in Sect. 3.6.

POSIX Sockets. The socket interface provided on Linux closely follows the POSIX standard and contains well known functions operating on sockets, such as `bind()` and `recv()` [24]. Common C standard library implementations for Linux, aim to implement the C POSIX API. Other programming languages often provide runtime libraries which have strong similarities with this interface, e.g., the Python socket module, the classes under `java.net.*` and the Rust `std::net` module, which are often in their turn wrappers for the functions provided by the C standard library. All functions defined in `<sys/socket.h>` were included.

Other Operations on File Descriptors. Following common Unix-philosophy, on Linux, socket instances are referred to in user space using file descriptors. Linux provides a unified set of operations to interact with file descriptors, regardless of the associated I/O resource, e.g., `(e)poll()`, `fcntl()` and `close()`. All operations that apply to IP socket file descriptors were included in the model.

Rtnetlink. Netlink is a socket-based API used for transferring information between various kernel interfaces and user space processes. Rtnetlink, is an interface that is accessible through Netlink and is used to retrieve or change networking related configurations [26]. Rtnetlink is used in the implementation of various library functions, such as the `getifaddrs()` function. The same consideration as with Internet sockets for symbolic execution apply, see Sect. 3.6. All *getters* of Rtnetlink were included in the framework.

5.4 Sound Co-operation with Non-emulated Environment

The SUT’s behavior should remain unchanged regardless of Netfuzzlib’s use. This section outlines approaches employed to ensure the framework’s soundness. First, the handling of operations on file descriptors not owned by the framework is discussed. Then, the approach to ensure consistent behavior with the host’s network configuration is covered.

Forwarding Non-modelled File Descriptors. Many operations used on IP and Rtnetlink sockets are also applicable to other sorts of I/O resources (e.g., an ordinary file) represented by a file descriptor. For example, *close()* terminates any type of file descriptor and *poll()* can listen for events on multiple file descriptors at once.

The framework maintains a separate file descriptor table which acts as a mask over the one from the kernel. When the SUT performs an action on a file descriptor that is not managed by the framework, the operation is forwarded to the normal C standard library. For functions operating on multiple file descriptors, such as *select()* and *(e)poll()*, the given set of file descriptors is split. The kernel implementation is invoked in non-blocking mode for the file descriptors not managed by the framework. Next, the outcome from the kernel is merged with that from the framework and returned to the caller. To prevent the framework from using the same (integer) file descriptor values as the kernel, we preemptively reserve file descriptors used by the framework by creating a dummy file handle which points to */dev/null*.

Consistent Network Environment. As explained in Sect. 5.3, the framework implements certain functionality which requires knowledge of the underlying network interfaces and environment. For example, the *SIOCGIFHWADDR* ioctl returns the MAC address of a given network interface. To remain consistent with the host system, the framework makes a copy of the host system’s configuration, i.e., present network interfaces and routing table, during startup. Optionally, the fuzzing module can alter this behaviour or add new network interfaces using an API provided by the main library, defined in `<netfuzzlib/api.h>`.

5.5 Liveness Heuristic

Netfuzzlib provides an optional liveness heuristic intended to end the fuzzing cycle if the target is ready with processing a test case (recall Sect. 3.3). If the incoming data buffer is empty, the fuzzing module provides no new messages and the program performs a call that blocks on the availability of incoming network data or checks a predetermined number of times for new network data (e.g., with *poll()* or *recv()*), then the SUT is considered to have completely consumed a test case. The corresponding handler in the fuzzing module is invoked, which can then perform a fuzzer-specific action to prepare the target for the next test input or indicate that the SUT should terminate.

5.6 Netfuzzlib Usage

The functions implemented in the framework can be used as a drop-in replacement for the ones provided by the system’s C standard library, usually glibc. The *main library* can either be statically linked to the target, or the dynamic loader can be instructed to use the relevant functions in the *main library* instead of the C standard library during the runtime linking process. For GNU/Linux-based

systems, this can be done by setting the *LD_PRELOAD* environment variable to the framework’s *main library* path. In theory, the framework could also be activated by placing hooks at the syscall level, for example, to test malware that directly invokes syscalls as an obfuscation technique.

5.7 Implementation

To obtain compatibility with common symbolic execution engines, Netfuzzlib was implemented in C, with the standard library as its only dependency. The *main library* was implemented in 15511 LoC, including 10949 LoC for unit tests.

6 Evaluation and Discussion

In this section, we first explain our approach to checking the *soundness* of Netfuzzlib. We then demonstrate and discuss the practicality and benefits of the framework using three experiments. In the first one, we show the speed improvements when fuzzing over native socket I/O versus Netfuzzlib for an existing network fuzzer, AFLNet. In the second experiment, we explain how we used the framework to add network fuzzing support to AFL++, without altering its source code. In the third, we use KLEE to symbolically execute a set of network servers without creating a test harness.

6.1 Checking Soundness

General Approach. Netfuzzlib essentially re-implements a set of interfaces provided by the Linux kernel in user space. In order to do a sound analysis, a network program should behave the same regardless the use of Netfuzzlib. The standards implemented by the framework (recall Sect. 5.3) are only available in lengthy *prose* specifications, and are spread or duplicated through multiple documents and authorities, e.g., POSIX, Linux man-pages, and RFCs. No complete test-suite for the implemented APIs exists. To assure the soundness of our framework, we therefore took an engineering approach. We first developed a suite of unit tests while investigating the specifications of the APIs to implement, and used these to test the Linux kernel (v6.1) and the Netfuzzlib framework. When a specification was ambiguous or the kernel’s implementation did not exactly follow the specification, we chose to mimick the kernel’s behaviour.

Second, to ensure the correct behaviour of the network programs used in the following experiments with the framework, we *replayed* a prerecorded list of packets of a valid protocol exchange while using Netfuzzlib, and manually compared the responses to the ones received without the framework.

6.2 Experiment 1: Speeding up Fuzzing

In this experiment, the fuzzing throughput is compared of an existing network fuzzer, AFLNet, when used with or without the framework. AFLNet is a state-aware coverage-guided fuzzer with support for fuzzing over UDP or TCP [37].

Table 1. Average time to reach one million executions.

	AFLNet	AFLNet+Netfuzzlib	Speedup
Bftpd	37 h 15 m	5h 45 m	23.5x
Dcmtk	6 h 52 m	4h 34 m	1.5x
Dnmasq	41 h 58 m	3h 56 m	10.66x
LightFTP	31 h 15 m	3h 15 m	9.58x
OpenSSH	14 h 50 m	37 m	23.5x
OpenSSL	45 h 1 min	3h 54 m	11.5x
TinyDTLS	140 h 53 m	14 m	593.7x

Porting AFLNet to Netfuzzlib. AFLNet only has support for fuzzing over network I/O, hence we had to slightly alter the fuzzer’s source code to work with Netfuzzlib. We created a new fuzzing module and changed AFLNet’s *net_recv()* function, to transfer test case message(s) and response(s) between the fuzzer and the fuzzing module using a shared memory region. We configured the fuzzing module to immediately terminate the SUT when the liveness heuristic indicates that all messages received by the SUT were processed, by invoking *exit()*.

Note that in this design, the fuzzer transfers all messages and responses to and from the shared memory region at once, respectively before the SUT is started and after it is terminated. This eliminates the need for context switches between the fuzzer and the SUT when receiving/sending consequent messages.

Experimental Setup. For our tests, we selected seven real-world network programs that implement a protocol supported by AFLNet. All experiments were run on a laptop with an 8-core AMD 5850U CPU and 16GiB RAM, using Ubuntu 20.04. All stated measurements are an average of four fuzz runs.

Results. Table 1 provides an overview of the average time required to reach one million executions, with or without the framework. We find that usage of Netfuzzlib increases fuzzing throughput for every tested program, with a typical speed increase of an order of magnitude. To show the soundness of the fuzzing process, we include coverage plots in Appendix A. All coverage measurements were conducted without the use of Netfuzzlib.

6.3 Experiment 2: Adding Network Fuzzing Support

AFL++ is a continuation of the original AFL fuzzer, and aims to combine new insights and recent research in coverage-guided fuzzing in a single fuzzer implementation [18].

Fuzzing Module. The fuzzing module for AFL++ reads a test case using the fuzzer’s shared memory API, and presents it to the SUT as a single UDP packet or in a TCP stream. The module is configurable via environment variables, for example, to select the destination port.

Results. For this experiment, the same test subjects and configuration as in experiment 1 was used. It was possible to fuzz all seven network programs without creating a test harness. Edge coverage plots were included in Appendix A.

6.4 Experiment 3: Symbolic Execution

In the third experiment, we use KLEE, a state-of-the-art symbolic execution engine that can be used to detect a wide range of bugs such as memory corruption errors and assertion errors or to generate a set of test inputs for every explored program path [13]. KLEE ships with a module to simulate (symbolic) interactions with the filesystem and standard I/O, but cannot (symbolically) handle networking operations. In the first test, we examine whether it is possible to use Netfuzzlib to detect a known vulnerability in Dnsmasq with KLEE, without creating a test harness. Finally, we use symbolic execution to discover three new vulnerabilities across two publicly available network servers.

Test Setup. In order to introduce symbolic network packets, we created a Netfuzzlib module which marks memory regions containing incoming network data as symbolic using KLEE’s API (`klee_make_symbolic()`). KLEE operates on the intermediate representation used in the LLVM compiler framework, hence both Netfuzzlib’s *main library* and corresponding module were compiled to LLVM IR and added using KLEE’s `-link-llvm-lib` option.

Dnsmasq. CVE 2017-14492 and 2017-14493 are respectively a heap and stack-based buffer overflow that were present in Dnsmasq up to version 2.77, and can be triggered by a malformed ICMPv6/DHCPv6 message. While symbolically executing Dnsmasq 2.76, KLEE reported the second vulnerability when introducing a symbolic packet with a size of at least 120 bytes within 5 min. We were not able to detect the first vulnerability using KLEE within 12 h.

New Vulnerabilities. Finally, we use KLEE to detect three vulnerabilities across two publicly available network programs, with assigned CVE’s 2023-50432, 2023-50433 and 2023-50434, demonstrating the utility of the framework for symbolic execution. Two vulnerabilities are in an open-source DHCP server [1] and can be abused to perform a Denial-of-Service attack: a NULL-pointer dereference and a type confusion bug causing a failed memory allocation and subsequent crash. A third vulnerability was found in an open-source DNS server for embedded systems [47], where a stack-based buffer over-read can lead to the potential disclosure of sensitive information.

7 Related Work

Nyx-net [43]. Nyx-Net uses hypervisor-based fuzzing with incremental snapshots, combined with selective emulation of IP sockets. To reduce work repetition, it uses snapshots with an efficient copy-on-write mechanism in order to

revert the SUT to a prior state. The system first creates a normal IP socket, and emulates socket I/O when the first data is received, which can not be used when fuzzing over multiple connections. In contrast, we emulate network APIs entirely in user-space to improve usability and performance, and do not require a hypervisor. Netfuzzlib could be used in combination with Nyx-Net to further improve performance and to handle more complex scenarios.

SnapFuzz [6]. The SnapFuzz framework proposes a set of methods and optimizations for increasing the efficiency of network fuzzers. It replaces IP sockets with faster UNIX sockets, uses a *forkserver* placed at a later point in execution to reduce work repetition, and uses an in-memory filesystem to enable a faster state reset and increase file I/O throughput. SnapFuzz also adds a mechanism to eliminate fuzzer-added delays by notifying the fuzzer when the SUT is ready to receive data. Finally, it uses binary-rewriting to add various other optimization, for example, to remove developer-added delays. The concepts introduced in SnapFuzz are mainly orthogonal to our work, and could be used in combination with Netfuzzlib to further increase fuzzer performance.

LibAFL [19]. LibAFL is a modular framework for building fuzzers, which architecturally decouples the building blocks which make up a fuzzer. The authors identify that in current research, new fuzzing technologies are often implemented by forking and altering an existing fuzzer (often AFL), making it difficult to build upon or assess the combination of separate advancements in fuzzing. Netfuzzlib could be used to add (efficient) network fuzzing support to LibAFL by implementing a LibAFL *Executor* and corresponding Netfuzzlib fuzzing module.

8 Conclusion

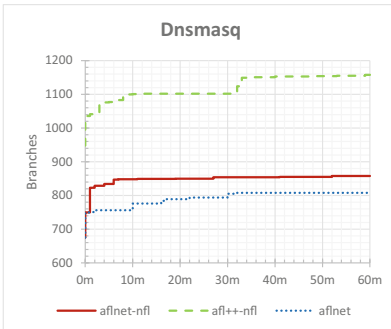
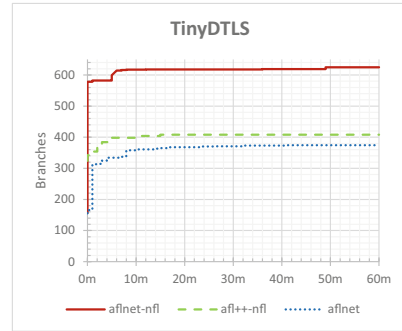
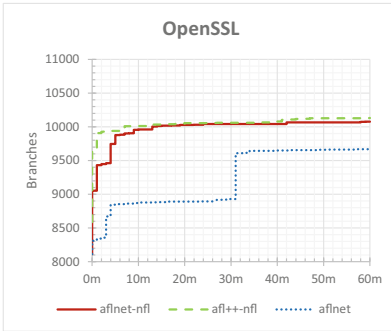
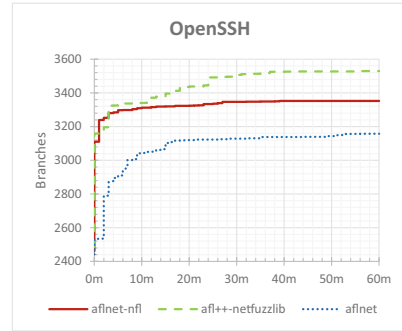
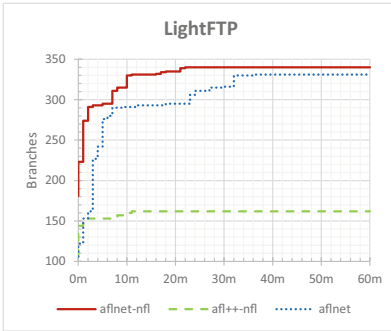
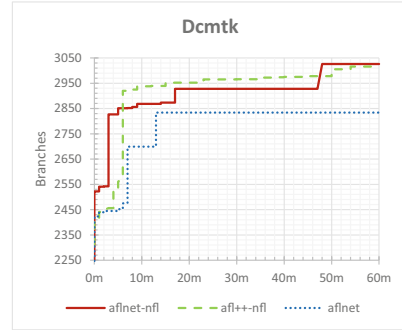
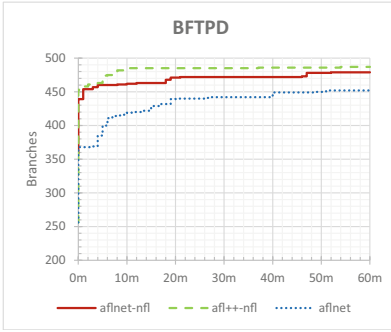
This paper identified a set of requirements and caveats for end-to-end fuzzing of network software and presented **Netfuzzlib**, a novel framework that removes the main obstacles for end-to-end fuzzing over network I/O in Linux. The framework provides a simple API to seamlessly facilitate the fuzzing and testing of network-based programs and typically results in a fuzzing throughput increase of at least 10x. This speedup is accomplished by eliminating the need for I/O delays, using a faster IPC mechanism to transfer network data, reducing the number of required context switches, automatically terminating the target when finishing a test case and various other improvements.

To enable the reproduction of our results, and encourage research into fuzzing network programs, we make Netfuzzlib open source under the Apache 2.0 and MIT license. It is available online at:

<https://netfuzzlib.com>

Acknowledgements. This research is partially funded by the Research Fund KU Leuven, and by the Cybersecurity Research Programme Flanders.

A Coverage Plots



References

1. simple-DHCP-server. <https://code.google.com/archive/p/simple-dhcp-server/>
2. ClusterFuzz (2023). <https://google.github.io/clusterfuzz/>. Accessed 19 Nov 2023
3. libFuzzer – a library for coverage-guided fuzz testing (2023). <https://llvm.org/docs/LibFuzzer.html>. Accessed 17 Nov 2023
4. Snapchange – lightweight fuzzing of a memory snapshot using KVM (2023). <https://github.com/awslabs/snapchange>
5. Sulley (2023). <https://github.com/OpenRCE/sulley>
6. Andronidis, A., Cadar, C.: SnapFuzz: high-throughput fuzzing of network applications. In: International Symposium on Software Testing and Analysis, pp. 340–351. ISSTA 2022, ACM (2022). <https://doi.org/10.1145/3533767.3534376>
7. Angluin, D.: Learning regular sets from queries and counterexamples. *Inf. Comput.* **75**(2), 87–106 (1987). [https://doi.org/10.1016/0890-5401\(87\)90052-6](https://doi.org/10.1016/0890-5401(87)90052-6)
8. Aschermann, C., Schumilo, S., Abbasi, A., Holz, T.: IJON: exploring deep state spaces via fuzzing. In: IEEE Symposium on Security and Privacy (S&P), pp. 1597–1612 (2020). <https://doi.org/10.1109/SP40000.2020.00117>
9. Ba, J., Böhme, M., Mirzamomen, Z., Roychoudhury, A.: Stateful greybox fuzzing. In: USENIX Security 22, pp. 3255–3272. USENIX Association (2022). <https://www.usenix.org/conference/usenixsecurity22/presentation/ba>
10. Baldoni, R., Coppa, E., D’elia, D.C., Demetrescu, C., Finocchi, I.: A survey of symbolic execution techniques. *ACM Comput. Surv.* **51**(3), 1–39 (2018). <https://doi.org/10.1145/3182657>
11. Boehme, M., Cadar, C., Roychoudhury, A.: Fuzzing: challenges and reflections. *IEEE Softw.* **38**(3), 79–86 (2021). <https://doi.org/10.1109/MS.2020.3016773>
12. Böhme, M., Falk, B.: Fuzzing: on the exponential cost of vulnerability discovery. In: ACM SIGSOFT Symposium on the Foundations of Software Engineering, pp. 713–724. FSE 2020, ACM (2020). <https://doi.org/10.1145/3368089.3409729>
13. Cadar, C., Dunbar, D., Engler, D.: KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In: USENIX Conference on Operating Systems Design and Implementation, pp. 209–224. OSDI’08, USENIX Association (2008). <https://doi.org/10.5555/1855741.1855756>
14. Cho, C.Y., Babić, D., Poosankam, P., Chen, K.Z., Wu, E.X., Song, D.: MACE: model-inference-Assisted concolic exploration for protocol and vulnerability discovery. In: USENIX Security 11. USENIX Association (2011). <https://www.usenix.org/conference/usenix-security-11/mace-model-inference-assisted-concolic-exploration-protocol-and>
15. Daniel, L.A., Poll, E., de Ruiter, J.: Inferring OpenVPN state machines using protocol state fuzzing. In: 2018 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), pp. 11–19 (2018). <https://doi.org/10.1109/EuroSPW.2018.00009>
16. Daniele, C., Andarzian, S.B., Poll, E.: Fuzzers for stateful systems: Survey and research directions (2024). <https://doi.org/10.1145/3648468>
17. Ding, Z.Y., Le Goues, C.: An empirical study of OSS-Fuzz bugs. In: IEEE/ACM International Conference on Mining Software Repositories (MSR), pp. 131–142. IEEE (2021). <https://doi.org/10.1109/MSR52588.2021.00026>
18. Fioraldi, A., Maier, D., Eißfeldt, H., Heuse, M.: AFL++: combining incremental steps of fuzzing research. In: Proceedings of the 14th USENIX Conference on Offensive Technologies. WOOT’20, USENIX Association (2020). <https://www.usenix.org/system/files/woot20-paper-fioraldi.pdf>

19. Fioraldi, A., Maier, D.C., Zhang, D., Balzarotti, D.: LibAFL: a framework to build modular and reusable fuzzers. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, pp. 1051–1065. CCS '22, ACM (2022). <https://doi.org/10.1145/3548606.3560602>
20. Fiterau-Brostean, P., Jonsson, B., Merget, R., de Ruiter, J., Sagonas, K., Somorovsky, J.: Analysis of DTLS implementations using protocol state fuzzing. In: USENIX Security 20, pp. 2523–2540. USENIX Association (2020). <https://www.usenix.org/conference/usenixsecurity20/presentation/fiterau-brostean>
21. Gascon, H., Wressnegger, C., Yamaguchi, F., Arp, D., Rieck, K.: PULSAR: Stateful Black-Box Fuzzing of Proprietary Network Protocols. In: Thuraisingham, B., Wang, X.F., Yegneswaran, V. (eds.) SecureComm 2015. LNICST, vol. 164, pp. 330–347. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-28865-9_18
22. Godefroid, P., Levin, M.Y., Molnar, D.: SAGE: Whitebox fuzzing for security testing. Queue **10**(1), 20–27 (2012). <https://doi.org/10.1145/2090147.2094081>
23. Goldsborough, P.: ipc-bench (2023). <https://github.com/goldsborough/ipc-bench>
24. IEEE and The Open Group: The Open Group Base Specifications Issue 7 (2018)
25. Li, J., Li, S., Sun, G., Chen, T., Yu, H.: SNPSFuzzer: a fast greybox fuzzer for stateful network protocols using snapshots. IEEE Trans. Inf. Forensics Secur. **17**, 2673–2687 (2022). <https://doi.org/10.1109/TIFS.2022.3192991>
26. Linux Programmer's Manual: rtnetlink(7) - Linux man page, 6.04 edn. (2023). <https://man7.org/linux/man-pages/man7/rtnetlink.7.html>
27. Linux Programmer's Manual: socket(7) - Linux man page, 6.04 edn. (2023). <https://man7.org/linux/man-pages/man7/socket.7.html>
28. Maier, D., Bittner, O., Munier, M., Beier, J.: FitM: binary-only coverage-guided fuzzing for stateful network protocols. In: Workshop on Binary Analysis Research (BAR), 2022 (2022). <https://doi.org/10.14722/bar.2022.23008>
29. Manès, V.J., et al.: The art, science, and engineering of fuzzing: a survey. IEEE Trans. Softw. Eng. **47**(11), 2312–2331 (2021). <https://doi.org/10.1109/TSE.2019.2946563>
30. Marcovich, R., Grumberg, O., Nakibly, G.: PISE: protocol inference using symbolic execution and automata learning. In: Workshop on Binary Analysis Research. BAR 2023 (2023). <https://doi.org/10.14722/bar.2023.23002>
31. McMahon Stone, C., Thomas, S.L., Vanhoef, M., Henderson, J., Bailluet, N., Chothia, T.: The closer you look, the more you learn: A grey-box approach to protocol state machine learning (2022). <https://doi.org/10.1145/3548606.3559365>
32. Metzman, J., Szekeres, L., Simon, L., Sprabery, R., Arya, A.: FuzzBench: an open fuzzer benchmarking platform and service. In: ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 1393–1403. ESEC/FSE 2021, ACM (2021). <https://doi.org/10.1145/3468264.3473932>
33. Microsoft: Windows Sockets 2 (2021). <https://learn.microsoft.com/en-us/windows/win32/WinSock/windows-sockets-start-page-2>. Accessed 26 Nov 2023
34. Natella, R.: StateAFL: greybox fuzzing for stateful network servers. Empirical Softw. Engg. **27**(7), 191 (2022). <https://doi.org/10.1007/s10664-022-10233-3>
35. Natella, R., Pham, V.T.: ProFuzzBench: a benchmark for stateful protocol fuzzing, pp. 662–665. ISSA 2021, ACM (2021). <https://doi.org/10.1145/3460319.3469077>
36. Pham, T.: AFLnwe (2020). <https://github.com/thuanpv/aflnwe/commits/master>
37. Pham, V.T., Böhme, M., Roychoudhury, A.: AFLNET: a greybox fuzzer for network protocols. In: IEEE International Conference on Software Testing, Validation and Verification (ICST), pp. 460–465 (2020). <https://doi.org/10.1109/ICST46399.2020.00062>

38. Plöger, S., Meier, M., Smith, M.: A usability evaluation of AFL and libFuzzer with CS students. CHI '23, ACM (2023). <https://doi.org/10.1145/3544548.3581178>
39. Qin, S., Hu, F., Ma, Z., Zhao, B., Yin, T., Zhang, C.: NSFuzz: Towards efficient and state-aware network service fuzzing. ACM Trans. Softw. Eng. Methodol. **32**(6), 1–26 (2023). <https://doi.org/10.1145/3580598>
40. Quarterman, J.S., Silberschatz, A., Peterson, J.L.: 4.2BSD and 4.3BSD as examples of the UNIX system. ACM Comput. Surv. **17**(4), 379–418 (1985). <https://doi.org/10.1145/6041.6043>
41. Rizzi, E.F., Elbaum, S., Dwyer, M.B.: On the techniques we create, the tools we build, and their misalignments: a study of KLEE. In: IEEE/ACM International Conference on Software Engineering, pp. 132–143. ICSE '16 (2016). <https://doi.org/10.1145/2884781.2884835>
42. de Ruiter, J., Poll, E.: Protocol state fuzzing of TLS implementations. In: USENIX Security 15, pp. 193–206. USENIX Association (2015). <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/de-ruiter>
43. Schumilo, S., Aschermann, C., Jemmett, A., Abbasi, A., Holz, T.: Nyx-net: network fuzzing with incremental snapshots, pp. 166–180. EuroSys '22, ACM (2022). <https://doi.org/10.1145/3492321.3519591>
44. Shoshitaishvili, Y.: preeny (2021). <https://github.com/zardus/preeny>
45. Shoshitaishvili, Y., et al.: SOK: (state of) the art of war: Offensive techniques in binary analysis. In: IEEE Symposium on Security and Privacy (S&P), pp. 138–157 (2016). <https://doi.org/10.1109/SP.2016.17>
46. Swiecki, R.: Honggfuzz (2023). <https://github.com/google/honggfuzz>
47. Tsarev, M.: emdns (2020). <https://github.com/mtsarev/emdns>
48. Xu, W., Kashyap, S., Min, C., Kim, T.: Designing new operating primitives to improve fuzzing performance, pp. 2313–2328. CCS '17, ACM (2017). <https://doi.org/10.1145/3133956.3134046>
49. Yu, Y., Chen, Z., Gan, S., Wang, X.: SGPFuzzer: a state-driven smart graybox protocol fuzzer for network protocol implementations. IEEE Access **8**, 198668–198678 (2020). <https://doi.org/10.1109/ACCESS.2020.3025037>
50. Zalewski, M.: AFL (2021). <https://github.com/google/AFL>
51. Zeng, Y., et al.: MultiFuzz: a coverage-based multiparty-protocol fuzzer for IoT publish/subscribe protocols. Sensors **20**(18), 5194 (2020). <https://doi.org/10.3390/s20185194>
52. Zhang, Z., Zhang, H., Zhao, J., Yin, Y.: A survey on the development of network protocol fuzzing techniques. Electronics **12**(13), 2904 (2023). <https://doi.org/10.3390/electronics12132904>



BOOTRIST: Detecting and Isolating Mercurial Cores at the Booting Stage

Yihao Luo^{1,2}, Yunjie Deng^{1,2}, Jingquan Ge⁴, Zhenyu Ning^{2,3},
and Fengwei Zhang^{1,2}(✉)

¹ Research Institute of Trustworthy Autonomous Systems, Southern University of Science and Technology, Shenzhen, China
{12032472,12032869}@mail.sustech.edu.cn

² Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen, China
zhangfw@sustech.edu.cn

³ Hunan University, Changsha, China
zning@hnu.edu.cn

⁴ Nanyang Technological University, Singapore, Singapore
jingquan.ge@ntu.edu.sg

Abstract. With the shrinking of transistor size and their own increasing complexity, CPUs have become more fault-prone. The mercurial cores, one type of processor failure, can randomly return silent miscomputation results and have been discovered in commercial CPUs. To address the mercurial core problem, we propose BootRIST, a software-based solution to detect and isolate mercurial cores in CPUs at the booting stage. BootRIST provides a framework to execute the testing instructions and store the result for each CPU core. Based on the execution result, BootRIST leverages the CPU hotplug mechanism to isolate mercurial cores. With BootRIST, the system and software can correctly run on CPUs with mercurial cores. We have implemented a prototype of BootRIST and deployed it on an Arm Linux server to perform an extensive evaluation. The empirical studies demonstrate that BootRIST can effectively detect and isolate mercurial cores at the booting stage.

1 Introduction

During the very-large-scale integration era, the increasing performance and the decreasing cost of CPUs have made them widely used in servers and endpoint devices. As the basic and essential component of computer systems, CPUs must handle all computation requests and logic operations correctly. Since CPU manufacturers already apply strict and sophisticated testing to CPUs, we naturally think that the CPU is reliable; the system design and software development also implicitly assume that the CPUs are fail-stop [26].

At the same time, the scale and complexity of CPUs constantly increases. Dozens of cores are integrated into a single CPU, and several extensions, such as

Y. Luo and Y. Deng—Co-first authors

Single Instruction/Multiple Data (SIMD) and hardware encryption instructions, are supported in modern CPUs. However, the shrinking hardware and complex function increase the probability of physical flaws in processors [3, 4, 6, 21, 28]. Even in a normal environment without extreme radiation, the CPUs have become more fault-prone than before. Google [11] and Facebook [7] have discovered that some server CPUs can repeatedly produce random wrong results without any error logs. Further investigation reveals that the errors typically occur on specific cores on multi-core CPUs rather than the entire CPUs. Such cores are named *mercurial cores* [11].

Although rough statistical analysis shows that the rate of mercurial cores is relatively low (a few mercurial cores per several thousand machines) [30], the large number of servers and cloud machines makes it impossible to ignore the existence of mercurial cores. Moreover, current systems lack an efficient detection and correction mechanism to handle mercurial core errors. Since the errors are produced by CPU computations, the existing fault-tolerance mechanism in the system (e.g., ECC in DRAM and RAID [5] in disk) cannot automatically correct the corruption. Even worse, the undetected corruption can be propagated to memory or files and further induce huge risks. For instance, broken configuration data can cause applications to crash, or a corrupt secret key can cause encrypted critical data to be permanently inaccessible. Consequently, there is an urgent need to address the threats of mercurial cores for server systems. A solution must not only detect mercurial cores effectively but also to keep these defective machines in use to reduce the cost to the cloud service provider.

In this paper, we propose BOOTRIST, a software-based solution to perform a **R**andom **I**nstruction **S**equences **T**est at the **B**ooting stage to detect mercurial cores and isolate them to preserve the utility of the system. The design principle of BOOTRIST is to propose a generic and automatic mercurial core detection and isolation tool that can be widely applied to the existing Linux system. Therefore, the design of BOOTRIST is based on the common features of Linux and can be integrated into the Linux kernel without any hardware-level modification. Furthermore, BOOTRIST provides an effective framework to accelerate instruction testing. We slightly modify the Linux exception handler to reduce the switching overhead when an instruction testing error occurs. The modification to the kernel only affects the system at the booting stage, while it introduces virtually no overhead to system runtime.

The execution flow of BOOTRIST contains three stages: instruction testing stage, detection stage, and isolation stage. The instruction testing stage executes a set of fault-prone instructions and records the execution result for all CPU cores. During the detection stage, BOOTRIST detects mercurial cores based on the execution result. Without requiring a correct default result to identify mercurial cores, BOOTRIST applies a voting algorithm to elect a base core. Finally, for the cores with different results from the base core, BOOTRIST isolates them from the operating system by using the CPU hotplug mechanism.

We implement a prototype BOOTRIST on openEuler 20.03 [12] with Linux kernel 4.19.90. We deploy the prototype on TaiShan Server [13], an Arm archi-

ture server with two Kunpeng 920 processors (a total of 96 cores). We first measure the efficiency of instruction testing with different instruction workloads. To further demonstrate the effectiveness of BOOTRIST, we inject the corruption results to verify the detection function and evaluate the isolation time of mercurial cores. We measure the system performance with a system-level benchmark UnixBench and an application-level benchmark SPEC2017 to demonstrate the overhead caused by BOOTRIST. The experimental results indicate that BOOTRIST achieves efficient instruction testing and can detect and isolate mercurial cores in the system. Furthermore, BOOTRIST only incurs negligible overhead at the system runtime.

We consider the contributions of our work as follows:

- We propose BOOTRIST, a software-level solution to detect and isolate mercurial cores at the booting stage. This solution does not need to modify the hardware, which can be widely deployed on the existing system.
- We design an efficient instruction testing framework to accelerate the execution of instructions. The framework improves the testing performance and reduces the testing time.
- We implement the BOOTRIST on an Arm architecture server and evaluate the efficiency and effectiveness of the BOOTRIST prototype. We also evaluate the BOOTRIST-enabled system on UnixBench and SPEC2017, and demonstrate that the BOOTRIST introduces virtually no overhead to the system runtime.

2 Preliminaries

2.1 What is Mercurial Core?

The concept of mercurial cores was initially introduced by Google [11] to describe the intermittent and undetectable miscomputations that occur on certain CPUs. Similar errors on CPUs have also been reported by Facebook [7] and Alibaba Cloud [30]. These errors manifest themselves in processor cores following seemingly innocuous changes in low-level software libraries, such as the utilization of faster but rarely-used instructions to enhance computational performance.

Further investigation shows that mercurial cores behave normally in most cases, while they return corrupt execution errors in some occasional and unpredictable conditions. Currently, the detection of mercurial cores mainly relies on manual reports, while the automatic machine check is less powerful. Without the immediate detection and correction of error results, mercurial cores may expose the computation data to large corruption risks. Researchers speculate that one fundamental reason for the occurrence of mercurial cores is the shrinking of CMOS scaling [24]. As the size of silicon decreases, it approaches physical limitations, thereby increasing the risk of latent failures. Another contributing factor is the growing complexity of processor architecture. The extensive integration of cores within a single CPU and the complex extension of instructions also elevate the risk of mercurial cores.

Solving the mercurial core problems necessitates collaborative efforts from both CPU manufacturers and server vendors. The CPU vendors are responsible for improving manufacturing testing or using the hardware-assisted way to reduce the rate of mercurial cores. As for the server vendors, they should apply the necessary fault-tolerance mechanism or isolation scheme to avoid the potential mercurial core corrupting user data. Existing solutions mainly add redundancy from both hardware and software layers to avoid its impact on the system [9,19]. In this paper, BOOTRIST provides a software-based isolation scheme for server vendors to preserve the normal function of machines with mercurial cores (Fig. 1).

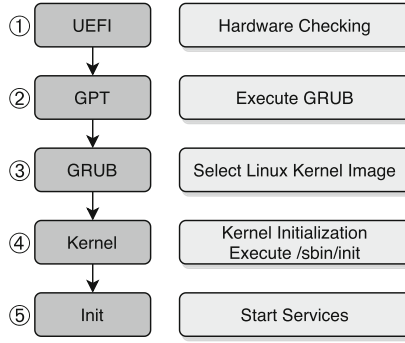


Fig. 1. The booting process of Linux.

2.2 Linux Booting Process

The entire booting process of the Linux system typically contains six steps after the power-up. First, the Unified Extensible Firmware Interface (UEFI) (①) performs the system hardware integrity check and initializes the hardware components. After the check, the BIOS executes and hands over the control to the boot loader in GUID Partition Table (GPT) (②). The MBR or GPT contains information about GRand Unified Bootloader (GRUB), which is loaded and executed in the next steps. Then the GRUB (③) allows the user to select one installed Linux kernel image as the boot image or directly use the default one according to the GRUB configuration file. In addition to the kernel image, the GRUB also loads and executes the initial RAM disk (*initrd*) images, which contain the necessary executables and system files to support the Linux system. Next, the Linux kernel image is executed to initialize the necessary kernel settings (④), such as CPU, MMU, and interrupt. Finally, according to the root file system provided by *initrd*, the Linux kernel executes the first program – */sbin/init* (⑤) to start the services. Based on the booting process, BOOTRIST is executed during the kernel initialization stage (④) to detect and isolate mercurial core.

2.3 CPU State and Hotplug in Linux Kernel

For reliability, availability, and serviceability (RAS), the Linux kernel provides the CPU hotplug mechanism to allow the system dynamically plug or unplug CPU cores. The Linux kernel maintains the following four bitmaps to manage all CPU core states.

- *possible*: The CPUs are defined in the Device Tree Source (DTS) and can be plugged into the system.
- *present*: The CPUs that are plugged.
- *online*: The CPUs that are available for scheduling.
- *active*: The CPUs that are available for receiving task migration.

The four aforementioned states are initially set to **true** by default to enable all CPU cores. However, when unplugging a CPU core from the system, the Linux kernel follows a specific process. Firstly, it migrates all running tasks and IRQs away from the core being unplugged. Subsequently, the Linux kernel sets the states of *present*, *online*, and *active* to **false** for the unplugged CPU core, preventing any new tasks from being scheduled or assigned to that core. In this paper, BOOTRIST remove mercurial cores to preserve the system functionality by leveraging the Linux CPU hotplug mechanism.

2.4 SIGILL in Linux System

SIGILL, a special signal in the Linux kernel, is typically triggered when processors execute an undefined instruction in user-mode. By default, when a program encounters the SIGILL signal, it is interrupted and crashes. However, it is possible for the program to avoid crashing by registering a SIGILL handler, which allows for handling the SIGILL signal in a customized manner. In this paper, we make use of the undefined instruction and the SIGILL handler to interrupt the test program and perform necessary operations during testing.

3 Fault Model and Assumptions

We consider a conservative fault model for transient hardware faults on CPU core. The mercurial core behaves as normal in most cases, while it can introduce silent corruption when executing specific types of instructions [11], such as floating-point instructions, vector instructions, and SIMD instructions. This silent corruption can lead to incorrect results being returned to registers or memory, thereby compromising the reliability of system.

To mitigate the impact of corruption on system runtime, BOOTRIST aims to detect and isolate the failure by testing fault-prone instructions at the booting stage. We assume that the server provider utilizes a fault-prone instruction set, allowing BOOTRIST to repeatedly execute a series of random instructions from this set during the booting stage to detect mercurial cores. Similar to existing research on processor failure detection [16, 17, 25], we do not consider faults in the memory subsystem and assume that it is protected by ECC.

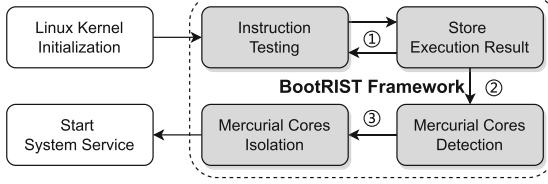


Fig. 2. BOOTRIST overview.

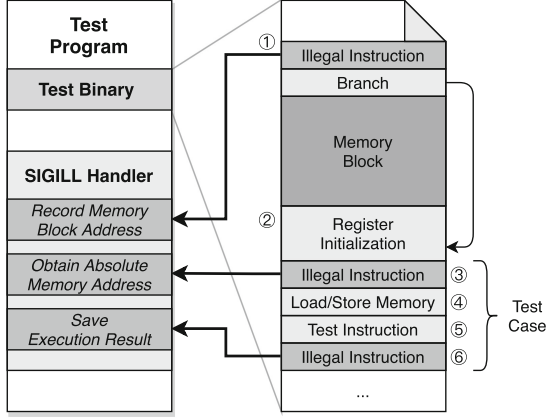


Fig. 3. The details of `risu` workflow. The test program loads and executes the test binary, whose layout is demonstrated on the right side.

4 Design

4.1 System Overview

As illustrated in Fig. 2, BOOTRIST is integrated into Linux kernel and executed during the initialization of Linux kernel. Without affecting the high-level applications, BOOTRIST finishes the detection and isolation of mercurial cores before starting system services. The execution flow of BOOTRIST is split into three steps: ① Instruction Testing, ② Mercurial Core Detection, and ③ Mercurial Core Isolation. BOOTRIST first performs Instruction Testing (①) to execute a series of fault-prone test instructions on all CPU cores, storing the execution result of each CPU core. For the Mercurial Core Detection (②), BOOTRIST compares the execution result core-by-core and utilizes a voting algorithm to find mercurial cores based on the comparison. When step ② detects the existence of mercurial cores, BOOTRIST further isolates them from normal cores to avoid unpredictable fault (i.e., ③ Mercurial Core Isolation).

4.2 Instruction Testing

The design of the instruction testing framework is based on **risu** [18], a tool used to find inconsistencies between simulators and real machines. **risu** can generate a test binary that contains a sequence of instructions that randomly selected from the instruction set. Moreover, **risu** provides a framework for executing the test binary and comparing the resulting execution outputs. Based on the design principle of **risu**, BOOTRIST leverages the **risu** to detect the mercurial core in processor. In this part, we first introduce the workflow of **risu**. Then we describe how to optimize the workflow to improve test efficiency.

4.2.1 **risu** Workflow

To achieve the test, **risu** provides a test program to collaborate with the generated test binary. Note that the test binary is not a single executable file, while it requires to be loaded by a test program and executed as a function. As illustrated in Fig. 3, we elaborate on important components in the test binary and test program. The test program contains an executable memory to execute the test binary and a **SIGILL** handler to support the dedicated operations and tackle unintentional errors. As for the test binary, it contains the following parts:

- **Memory Block:** A continuous memory region that is used to load and store memory data.
- **Register Initialization:** Initialize all general purpose registers and vector registers with pre-defined values.
- **Test Instruction:** The randomly generated arithmetic instructions according to the given instruction set.
- **Illegal Instruction:** The instructions that are intentionally inserted by **risu** to interrupt the execution and enter the **SIGILL** handler. Based on the type of illegal instruction, the handler can perform the following different operations: records the address of the memory block, obtains the load or store address in the memory block, or saves the value of registers and the memory block. After completing one of the above operations, the handler skips the illegal instructions and continues executing the binary.

In the beginning, the binary contains the first illegal instruction (①), which notifies the **SIGILL** handler to record the base address of the memory block. Following that, the binary executes a branch instruction and proceeds to the register initialization phase where all general purpose registers and vector registers are set. After the initialization, the binary proceeds to execute all test cases until reaching the end.

Each test case comprises memory load/store instructions, test instructions, and illegal instructions. Before accessing the memory, the binary first stores the offset of the memory block in **x0** register and executes a dedicated illegal instruction (③). Based on the given offset and the recorded address of the memory block, the **SIGILL** handler calculates the absolute address, which is then stored in the **x0** register as the target memory address for the subsequent load/store instructions (④). The test instructions (⑤) are executed with the

input given by the load/store instructions. After executing the test instructions, the test case reaches the final illegal instructions and notifies the SIGILL handler to save execution results in registers and the memory block (⑥).

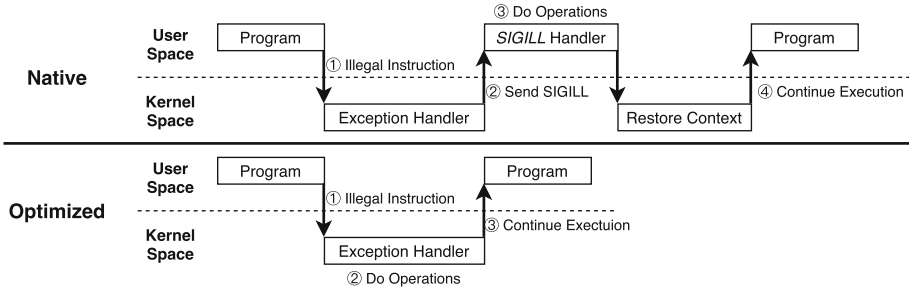


Fig. 4. The workflow comparison of the native SIGILL handler and the optimized illegal instructions handler.

4.2.2 Compress Execution Result

During the Instruction Testing, BOOTRIST requires executing a series of test instructions and storing the execution result for each test case. However, directly storing the execution result of all test cases can lead to significant memory consumption. To mitigate this issue, a scheme is devised to reduce memory usage. One possible solution is to execute the program in parallel on each CPU core and compare the results of each test case simultaneously. However, this approach requires strict parallel execution and synchronization among all CPU cores, which can introduce substantial slowdowns to the test progress and reduce the test efficiency.

To tackle these challenges, BOOTRIST utilizes a hash compression algorithm to reduce memory consumption in storing execution results. The approach involves allocating a dedicated memory region of identical size to the memory block and CPU registers for each CPU core within the Linux kernel. At the beginning of the test program, BOOTRIST initializes the hash state of the allocated memory. When the SIGILL handler receives the request to save the execution result, BOOTRIST employs a message digest algorithm to update the hash state based on the current state, as well as the values stored in the registers and memory block. After executing all test cases, the hash states in the reserved memory regions are used to identify any inconsistencies in execution. As a result, BOOTRIST effectively reduces the memory required to store the execution results while ensuring the discrimination of mercurial core.

4.2.3 Optimize the SIGILL Handler

In *risu*, the SIGILL handler is used to perform the specific operations base on the type of illegal instruction. However, the test program's frequent interactions with the SIGILL handler can introduce additional complexity and result in a slowdown during testing. To address this issue, we aim to design an efficient illegal instruction handler that improves test performance by reducing the overhead associated with entering the SIGILL handler.

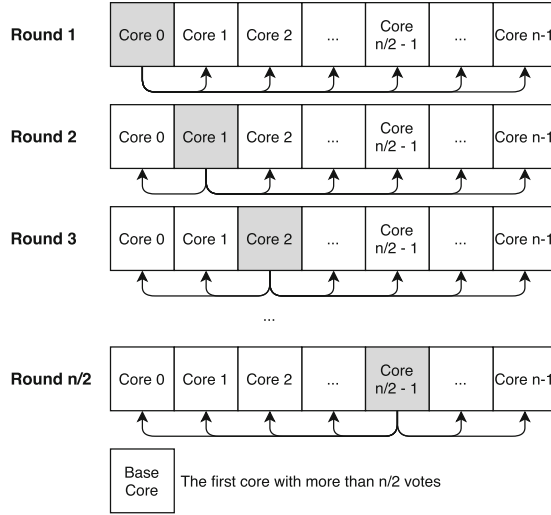


Fig. 5. Voting algorithm for mercurial core detection.

To optimize the test performance, we begin by examining the workflow that utilizes the **SIGILL** handler to handle the illegal instructions. As illustrated on the top side of Fig. 4, once the program executes an illegal instruction (①), the CPU generates an exception and traps the system into Linux exception handler. The exception handler stores the program context, and sends the **SIGILL** signal with the saved context registers to the program **SIGILL** handler (②). With the given context registers values, the **SIGILL** handler can identify the type of illegal instruction and perform corresponding operations (③). Once the **SIGILL** signal is handled appropriately, the handler returns to the kernel space. Subsequently, the kernel restores the program's state and resumes its execution (④). However, the complex Linux signal mechanism and frequent switch between the user space and kernel space can significantly slow down the test procedure. To address this, we improve the test performance by replacing the **SIGILL** handler with a efficient illegal instruction handler.

As shown in the bottom side of Fig. 4, we integrate the necessary **risu** operations into the Linux exception handler. For the illegal instruction that comes from **BOOTRIST** test, the exception handler directly performs the operations base on the type of illegal instruction. After finishing the operation, the exception handler skips the illegal instruction and resumes the program execution without sending the **SIGILL** signal to the program. Note that the modification of exception only affects the **SIGILL** raised by **BOOTRIST**, while the **SIGILL** from other programs works as normal. Consequently, **BOOTRIST** reduces the redundant context switch and eliminates the heavy signal mechanism, which improves the test performance and can find the mercurial core efficiently.

4.3 Mercurial Core Detection

To detect mercurial cores, a straightforward approach is to compare the execution results with pre-calculated correct results. However, this method requires executing tests on a fault-free machine to obtain accurate reference results, which adds extra effort. Considering that the occurrence rate of mercurial cores is typically low, it implies that the majority of cores will produce correct results. Exploiting this observation, we can employ a voting algorithm to identify the mercurial core without the need for additional pre-calculation efforts. By comparing the execution results core-by-core, the voting algorithm determines the cores that deviate from the expected outcome and identifies them as mercurial cores. This approach allows for efficient mercurial core detection without relying on pre-calculated results.

Figure 5 illustrates the process of the voting algorithm. We label the cores from 1 to n and apply the voting algorithm $\frac{n}{2}$ rounds, where n is the number of processor cores. During round i , we compare the result of core i with all other cores. The vote of core i is counted by the number of the same execution results compared with core i . For the first core with more than $\frac{n}{2}$ votes, BOOTRIST marks it as the base core and regards its execution result as reliable. Then the cores with different results from the base core are marked as mercurial cores, which will be isolated in the later stage. Note that the algorithm proceeds at most $\frac{n}{2}$ rounds since the base core cannot exist in the last $\frac{n}{2}$ cores. If no core can obtain more than $\frac{n}{2}$ votes, we regard the result is contaminated by unexpected factors, such as the unintentional modification to the memory from other program, because the rate of mercurial cores is typically low. In this case, BOOTRIST cleans all recorded results and executes the Instruction Testing and Mercurial Core Detection again to find mercurial cores.

4.4 Mercurial Core Isolation

The Mercurial Core Isolation is executed only if the detection result indicates the existence of mercurial cores. BOOTRIST leverages the CPU hotplug mechanism to isolate mercurial core. More specifically, BOOTRIST first migrates all the running tasks and IRQ in mercurial cores to other normal cores. After the migration, we change the *present*, *online*, and *active* state to **false**, disabling mercurial cores. As a result, any tasks can not be assigned to the unplugged mercurial core by scheduling or task binding.

5 Implementation

Section 4 focuses on the design principle of the BOOTRIST. In this section, we further demonstrate the necessary implementation details of our design.

5.1 Instruction Generation

To detect Mercurial cores, we need to generate error-prone instruction test cases. In AArch64 execution mode, ARMv8 runs the A64 instruction set. A64 instruction set consists of 32-bit fixed-length encoded instructions and uses 64-bit addressing mode. Each instruction encoding includes bits for Opcode, Condition Code, Immediate, Register, Shift Operation, etc. For a specific instruction, some bits, like the Opcode, are fixed, while others, such as register and immediate values, are variable. The Risu tool provides us with instruction templates for the A64 instruction set, detailing the encoding format and value constraints for the variable fields of each instruction. By parsing these templates, we can select the types and numbers of instructions we want to test and generate multiple different legal instructions based on the constraints for testing.

5.2 Parallel Execution

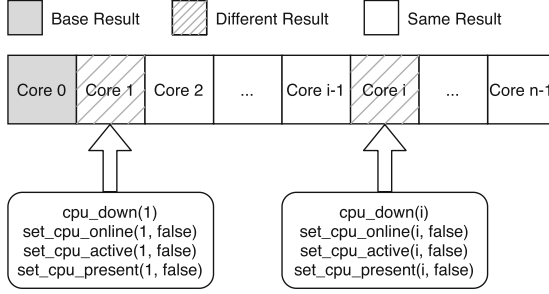
To improve the test performance, we implement the Instruction Testing with parallel execution on each CPU core. During the kernel initialization, BOOTRIST first leverages the kernel function *call_usermodehelper()* to execute the testing program. Then we duplicate the program process n times through leveraging the *fork()* function, where n is the number of CPU cores. For each forked process, we utilize the *sched_setaffinity()* function to bind it to different processor cores. Consequently, the test procedure can be parallel executed on all CPU cores.

5.3 Modification to Exception Handler

As mentioned in Sect. 4.2.3, we integrate the illegal instruction handler into Linux exception handlers to improve the test performance. Therefore, we modify the kernel exception handler *do_undefinstr()* to handle the intentionally inserted illegal instruction and process the dedicated operations, such as recording execution results and obtaining the memory block address. Note that we only migrate the illegal instruction handler part into kernel exception handlers, while the SIGILL handler is still preserved to tackle the rare unintentional execution error.

5.4 Mercurial Core Isolation

BOOTRIST achieves core isolation by leveraging the CPU hotplug mechanism in Linux kernel. Figure 6 presents an example of mercurial core detection result and illustrates how to isolate mercurial cores in detail. From the figure, we can see that core 0 is the base core, while core 1 and core i are mercurial cores as their execution results are different from the base result. BOOTRIST first utilizes the function *cpu_down()* to migrate the running task and IRQ from mercurial cores to normal cores. Then we switch the *present*, *online*, and *active* state to false to further remove the core from the scheduling list and unplug the core from the system. The switch of core states is achieved by employing Linux kernel APIs *set_cpu_online()*, *set_cpu_active()* and *set_cpu_present()*. Consequently, BOOTRIST can isolate mercurial cores from the normal cores and preserve the system's functionality.

**Fig. 6.** Mercurial core isolation.

6 Evaluation

In this section, we detail the performance evaluation of BOOTRIST. First, our evaluation environment is introduced in Sect. 6.1. In Sect. 6.2, we demonstrate the category of testing instructions and the testing performance of our framework. Furthermore, we measure the time consumption of mercurial core isolation in Sect. 6.3. Finally, we evaluate the system performance overhead caused by the deployment of the BOOTRIST in Sect. 6.4.

6.1 Evaluation Environment

We select Huawei TaiShan 200 (Model 2280) Server [13] as our experimental platform. The platform has two HiSilicon Kunpeng 920-4826 CPUs [10] with a total of 96 ARMv8-A cores. Moreover, it equips 382 GB of DDR4 memory and 2.1 TB of hard drive storage. To record the execution result, we reserve 8,192 bytes and 1,024 bytes memory to store the hash state of the memory block and register values for each CPU core (i.e., 0.75 MB and 96 KB in total), respectively. We prototype BOOTRIST based on Linux kernel 4.19.90 running in openEuler 20.03 [12].

6.2 Performance of Instruction Testing

We leverage `risu` [18] to randomly generate a various number of test instructions from the Arm instruction set [1, 2] to demonstrate the test performance. As previous work suggests that mercurial cores are more likely to be observed when executing complex instructions [11, 27] (e.g., vector operations, floating-point instructions), we use such instructions as a fault-prone instruction set to generate test binary. More specifically, the test binary contains the floating-point operations instructions, Single Instruction/Multiple Data (SIMD), and Scalable Vector Extension (SVE) instructions.

To demonstrate the test performance, we evaluate the performance of BOOTRIST with a different number of test instructions and compare the execution

Table 1. Comparison of instruction test time (seconds) and its percentage in total booting time.

Instructions	BOOTRIST		risu [18]	
	Test Time	Percentage	Test Time	Percentage
10	0.01	0.01%	0.01	0.01%
100	0.02	0.01%	0.03	0.02%
1000	0.18	0.11%	0.23	0.14%
10000	1.86	1.13%	2.31	1.40%
100000	18.78	10.34%	23.10	12.42%
500000	93.75	36.53%	115.36	41.46%

time between BOOTRIST and different experiment settings in Table 1. The column **Instructions** demonstrates the number of test instructions generated by the **risu** [18]. The column **BootRIST** and **risu** demonstrate the performance of instruction testing on BOOTRIST and native **risu**, respectively. Note that **BootRIST** leverages the exception handler to handle the illegal instruction, while **risu** uses the **SIGILL** handler. As shown in the table, the comparison between BOOTRIST and **SIGILL** demonstrates that our efficient illegal instruction handler achieves approximately 20% acceleration to the instruction testing. The experimental results show that BOOTRIST provides an efficient instruction test framework. Furthermore, we measure the time consumption of the booting process and evaluate the percentage of test time in total booting time. The native Linux system takes 162.87 (s) to boot. And the **Percentage** columns in the Table 1 show that BOOTRIST does not severely increase the booting time. Since the reboot in server systems is infrequent, the instruction testing at the booting stage would not significantly affect high-level users.

6.3 Mercurial Core Detection and Isolation

To demonstrate the effectiveness of BOOTRIST, we further evaluate the detection and isolation of mercurial cores. We manually emulate mercurial cores by injecting faults into the execution result. For the broken execution results, BOOTRIST regards the corresponding cores as mercurial cores and isolates them from other cores.

We further evaluate the time consumption caused by isolating a various number of mercurial cores. Recall from Sect. 4.3 that our voting algorithm rejects the discrimination result if the number of mercurial cores is equal to or greater than half of the total number of cores. Since our experimental platform equips with 96 cores, we evaluate the isolation time for no more than 47 mercurial cores and present the result in Fig. 7. The figure shows that the isolation time almost increases linearly with the increasing number of cores. Moreover, the isolation time is less than 2 s even if we isolate 47 cores, demonstrating that our mercurial core isolation scheme is efficient.

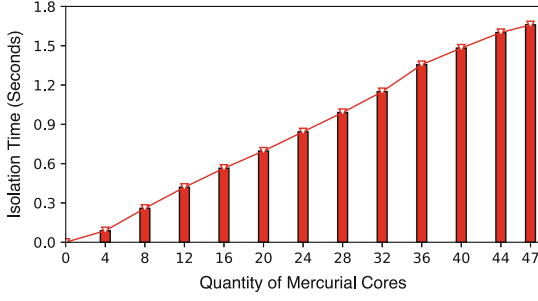


Fig. 7. Isolation time of mercurial cores.

6.4 System Performance Analysis

We further evaluate the impact of BOOTRIST with system-level and application-level benchmarks to demonstrate that the deployment of BOOTRIST incurs virtually no overhead to the system runtime.

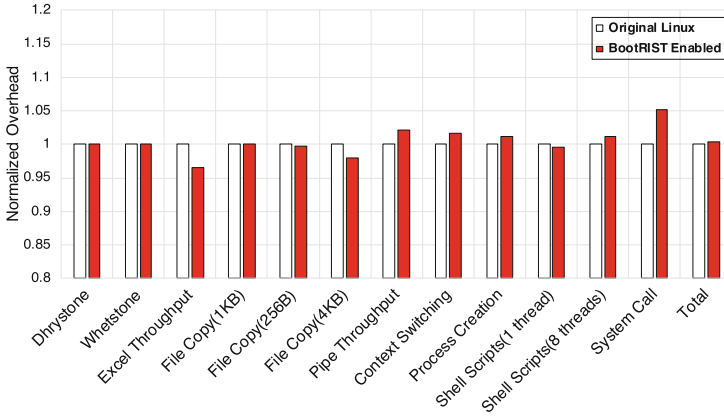


Fig. 8. The performance overhead of BOOTRIST on UnixBench.

UnixBench [15]: UnixBench is a benchmark tool to evaluate the performance of a Unix-like system. To demonstrate the performance overhead of BOOTRIST on the system, we perform UnixBench to evaluate the affection to critical system operations. Figure 8 illustrates the comparative evaluation of BOOTRIST relative to the native Linux system on UnixBench. The white column and red column represent the Linux system with native settings and the Linux system with BOOTRIST, respectively. As shown in Fig. 8, the deployment of BOOTRIST does not incur significant overhead on all test items. The largest overhead occurs in the *System Call* item because all system calls execute the `svc` instruction and enter the exception handler in Arm Linux. The modification of the exception

handler incurs such overhead, which further by optimizing the BootRIST code added in the exception handler. Nevertheless, the total overhead indicates that BOOTRIST only incurs negligible overhead. The experiment results demonstrate that BOOTRIST incurs almost no impact on the system runtime.

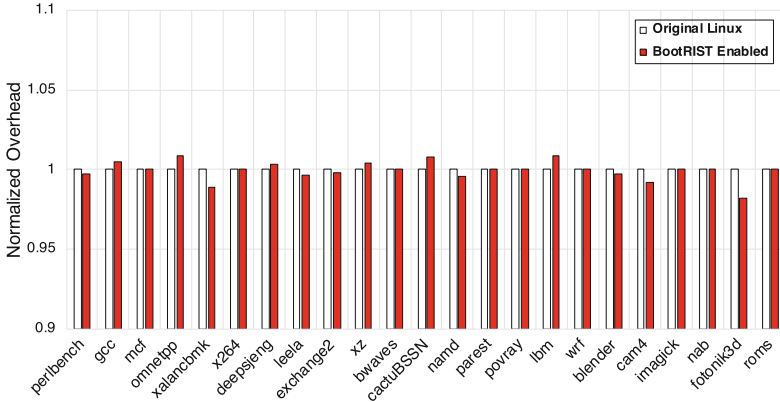


Fig. 9. The performance overhead of BOOTRIST on SPEC2017.

SPEC_CPU 2017 [29]: SPEC_CPU 2017 benchmark package provides testing suites to measure the performance of compute-intensive real user applications. Therefore, we further employ SPEC_CPU 2017 to evaluate the impact of BOOTRIST on the user application performance. We evaluate all of SPEC_CPU 2017 INT and FP applications and present the result in Fig. 9. As illustrated in Fig. 9, the performance of BOOTRIST and original Linux system are almost same, and only present some minor fluctuations on different user applications. Moreover, the average performance overhead shows that BOOTRIST introduces virtually no overhead on the user application.

7 Discussion and Future Work

The design principle of BOOTRIST is to provide an automatic mercurial core detection framework. We achieve the automatic testing and detection at the booting stage with pre-generated instruction binary, which contains a series of fault-prone instructions. However, we consider that the fault-prone instruction coverage may not be enough to discover mercurial cores. The two aspects mainly cause insufficient coverage. The first aspect is the limitation of testing time. Since BOOTRIST is running at the booting stage, the testing time cannot occupy the booting process too long to affect the normal usage of the system. The second aspect is the incomplete summary of the fault-prone instruction set. We assume that the server provider already knows the fault-prone instruction set. With such a set, the mercurial cores can be detected even if the reboot is

infrequently and the booting time is short. However, the investigation of the fault-prone instruction set relies on manual bug reports [11], which is less efficient and is hard to discover all fault-prone instructions. Therefore, we consider future work can improve the fault-prone instruction coverage by prolonging the testing time or designing an automatic way to complete the fault-prone instruction set. For instance, BOOTRIST can integrate with an online instruction generator to achieve an online test and leverage fuzzing technologies to guide the discovery of fault-prone instructions.

8 Related Work

8.1 Processor Failure

The previous studies have proposed some schemes to detect unexpected processor failure. Farron [30] is the first work to observe and characterize SDC on large-scale server clusters. Nostradamus [20] employs extra hardware to overcome the computation errors in out-of-order processor cores. It predicts the expected result of instruction, and corrects the output if a failure occurs. Besides the hardware solution, some works [16, 17, 22, 25] design a software-level redundancy scheme to overcome the soft-error. They modify the compiler to leverage the unused instruction-level resources to add redundancy. SiliFuzz [27] leverages fuzzing technology to detect potential mercurial cores. They use the known bad machines to guide the test generator and create fault-prone test cases. OpenDC-Diag [23] is an open-source framework to find defects and bugs in CPUs. OpenDC-Diag requires user manually writing test code and providing pre-calculated golden result. The CPUs failure is detected by comparing the execution result with the golden result. Intel In-Field-Scan can run circuit level test on a CPU core to detect processor failures. However, it only supports on a part of upcoming Xeon Sapphire Rapids processor, which means it can not be applied to other Intel CPUs or other manufacturers' CPUs (such as AMD and Arm).

8.2 Discover Inconsistent Execution

Several works are proposed to discover inconsistent execution to improve reliability. iScanU [8] aims to find undocumented instructions on RISC processors. It executes a single instruction and determines whether it is an undocumented instruction based on whether the SIGLL signal is triggered. EXAM-INNER [14] focuses on finding the inconsistent execution between simulators and real machines. It leverages the Arm architecture specification language (ASL) to generate representative instruction streams to find inconsistent instruction efficiently. `risu` [18] is an open-source random user-space instruction generator intended to test the implementation of simulators of Arm architecture. In BOOTRIST, we leverage the `risu` to randomly generate test instructions on a fault-prone instruction set and find the inconsistent execution between CPU cores to detect mercurial cores.

9 Conclusions

With the shrinking of transistor size and their own increasing complexity, CPUs have become more fault-prone. Recently, the mercurial cores, which are caused by permanent hardware faults and may return the corrupt result, have been discovered in modern CPUs. The existence of mercurial cores brings a huge data risk to the system and can severely affect the system's functionality. Therefore, we propose BOOTRIST, a software-based solution to detect and isolate mercurial cores at the booting stage, and can be integrated into the existing system. We design an efficient instruction testing framework to improve the testing pressure and reduce the testing time. Furthermore, without requiring the default correct execution result, we design a voting algorithm to identify the possible mercurial core. Moreover, BOOTRIST fully leverages the Linux CPU hotplug mechanism provided to isolate mercurial cores from normal cores. Based on the design, we implement a prototype BOOTRIST on a server, as well as perform the experimental studies. The experiment results demonstrate that BOOTRIST achieves efficient instruction testing and effective mercurial core detection and isolation on the system. Furthermore, our evaluation demonstrates that the deployment of BOOTRIST only incurs virtually no overhead to the system.

Acknowledgments. We would like to thank the anonymous reviewers and COMPASS members for their insightful comments. This work is partly supported by the National Natural Science Foundation of China under Grant No.62372218 and No.62102175.

References

1. Arm: Arm A64 instruction set architecture. <https://developer.arm.com/documentation/ddi0596/latest>
2. Arm: Arm architecture reference manual Armv8, for Armv8-A architecture profile. <https://developer.arm.com/documentation/ddi0487/latest/>
3. Bohr, M.T., Young, I.A.: CMOS scaling trends and beyond. *IEEE Micro* **37**(6), 20–29 (2017)
4. Borkar, S.: Designing reliable systems from unreliable components: the challenges of transistor variability and degradation. *IEEE Micro* **25**(6), 10–16 (2005)
5. Chen, P.M., Lee, E.K., Gibson, G.A., Katz, R.H., Patterson, D.A.: RAID: High-performance, reliable secondary storage. *ACM Comput. Surv. (CSUR)* **26**(2), 145–185 (1994)
6. Constantinescu, C.: Trends and challenges in VLSI circuit reliability. *IEEE Micro* **23**(4), 14–19 (2003)
7. Dixit, H.D., Pendharkar, S., Beadon, M., Mason, C., Chakravarthy, T., Muthiah, B., Sankar, S.: Silent data corruptions at scale. *arXiv preprint arXiv:2102.11245* (2021)
8. Dofferhoff, R., Göebel, M., Rietveld, K., Van Der Kouwe, E.: iScanU: a portable scanner for undocumented instructions on RISC processors. In: 2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), pp. 306–317. IEEE (2020)

9. Dong, S., Kryczka, A., Jin, Y., Stumm, M.: Evolution of development priorities in key-value stores serving large-scale applications: the {rocksdb} experience. In: 19th USENIX Conference on File and Storage Technologies (FAST 21), pp. 33–49 (2021)
10. HiSilicon: Kunpeng 920-4826 - hisilicon. <https://en.wikichip.org/wiki/hisilicon/kunpeng/920-4826>
11. Hochschild, P.H., et al.: Cores that don't count. In: Proceedings of the Workshop on Hot Topics in Operating Systems, pp. 9–16 (2021)
12. HUAWEI: openEuler. <https://www.openeuler.org/en/>
13. HUAWEI: Taishan Server. <https://e.huawei.com/en/products/servers/taishan-server>
14. Jiang, M., et al.: EXAMINER: automatically locating inconsistent instructions between real devices and CPU emulators for ARM. In: Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 846–858 (2022)
15. Kelly Lucas: byte-UnixBench. <https://github.com/kdlucas/byte-unixbench>
16. Kuvaiskii, D., Faqeh, R., Bhatotia, P., Felber, P., Fetzer, C.: HAFT: hardware-assisted fault tolerance. In: Proceedings of the Eleventh European Conference on Computer Systems, pp. 1–17 (2016)
17. Kuvaiskii, D., Oleksenko, O., Bhatotia, P., Felber, P., Fetzer, C.: ELZAR: triple modular redundancy using intel AVX (practical experience report). In: 2016 46th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), pp. 646–653. IEEE (2016)
18. Linaro: RISU. <https://git.linaro.org/people/pmaydell/risu.git/>
19. Mamone, D., Bosio, A., Savino, A., Hamdioui, S., Rebaudengo, M.: On the analysis of real-time operating system reliability in embedded systems. In: 2020 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT), pp. 1–6. IEEE (2020)
20. Nathan, R., Sorin, D.J.: Nostradamus: low-cost hardware-only error detection for processor cores. In: 2014 Design, Automation & Test in Europe Conference & Exhibition (DATE), pp. 1–6. IEEE (2014)
21. Nightingale, E.B., Douceur, J.R., Orgovan, V.: Cycles, cells and platters: an empirical analysis of hardware failures on a million consumer PCs. In: Proceedings of the Sixth Conference on Computer Systems, pp. 343–356 (2011)
22. Oh, N., Shirvani, P.P., McCluskey, E.J.: Error detection by duplicated instructions in super-scalar processors. *IEEE Trans. Reliab.* **51**(1), 63–75 (2002)
23. OpenDCDiag. <https://github.com/opendcdiag/opendcdiag>
24. Papadimitriou, G., Gizopoulos, D.: Silent data corruptions: microarchitectural perspectives. *IEEE Trans. Comput.* **72**, 3072–3085 (2023)
25. Reis, G.A., Chang, J., Vachharajani, N., Rangan, R., August, D.I.: SWIFT: software implemented fault tolerance. In: International Symposium on Code Generation and Optimization, pp. 243–254. IEEE (2005)
26. Schlichting, R.D., Schneider, F.B.: Fail-stop processors: an approach to designing fault-tolerant computing systems. *ACM Trans. Comput. Syst. (TOCS)* **1**(3), 222–238 (1983)
27. Serebryany, K., Lifantsev, M., Shtoyk, K., Kwan, D., Hochschild, P.: SiliFuzz: fuzzing CPUs by proxy. arXiv preprint [arXiv:2110.11519](https://arxiv.org/abs/2110.11519) (2021)

28. Shivakumar, P., Kistler, M., Keckler, S.W., Burger, D., Alvisi, L.: Modeling the effect of technology trends on the soft error rate of combinational logic. In: Proceedings International Conference on Dependable Systems and Networks, pp. 389–398. IEEE (2002)
29. SPEC: Spec CPU 2017. <https://www.spec.org/cpu2017/>
30. Wang, S., Zhang, G., Wei, J., Wang, Y., Wu, J., Luo, Q.: Understanding silent data corruptions in a large production CPU population. In: Proceedings of the 29th Symposium on Operating Systems Principles, pp. 216–230 (2023)



Social Control and Interactivity in Anonymous Public Events

Md Mushfekur Rahman  and Philip W. L. Fong ^(✉) 

Department of Computer Science, University of Calgary,, Calgary Alberta, Canada
{mdmushfekur.rahman,pwlfong}@ucalgary.ca

Abstract. Online event hosting platforms, such as Zoom Meetings and Twitch Streams, have revolutionized the way we socialize with one another. These platforms offer a rich set of interactive features such as live chat and gestures, enabling dynamic and engaging social events. In public events, however, participants are not well-known entities originating from the same institution, and thus traditional access control fails to provide means for maintaining order without disrupting interactivity. Zoombombing and cyberbullying in Twitch Streams are symptoms of this dilemma. The design of the aforementioned event hosting systems thus resort to social control mechanisms that allow moderators to monitor the social interactions of the participants and respond to disorderly behavior in real time. The designer of an event hosting system needs to make sure that social control mechanisms preserve interactivity expectations. In this paper, we introduce HIPE (Highly Interactive Public Event), a framework for modelling social control mechanisms, articulating interactivity expectations, as well as verifying if social control interferes with interactivity. We catalogued 4 classes of social control mechanisms that can be reused in the design of event hosting systems, including sanction, remedy, containment, and retaliation. Additionally, we formulated a 2-safety hyperproperty known as (a, p) -interactivity, for expressing the degree of interactivity expected of an event hosting system. Furthermore, we designed a model checking algorithm for verifying (a, p) -interactivity. An empirical case study has been conducted to illustrate the interplay between social control and interactivity, as well as to evaluate the performance of our model checking algorithm. To the best of our knowledge this is the first work to formally study the balancing of social control and interactivity in public events.

Keywords: Zoombombing · Anonymous Public Events · Access Control · Social Control · Interactivity · Noninterference · Hyperproperty · 2-Safety · Model Checking · Formal Verification

1 Introduction

One of the most iconic security challenges of the pandemic era is *Zoombombing* [29], in which virtual events such as PhD defences, online lectures, and

religious services hosted on video-teleconferencing platforms are hijacked or disrupted, with the attacker posting offensive images (e.g., pornographic, obscene, or violent images) or using unacceptable languages (e.g., threats, hate speech, or racial slurs). A previous study [23] found that in most Zoombombing incidents, the attackers did not hijack the meetings through leaked meeting invites or by bruteforcing the meeting IDs. Instead they were people with legitimate access to the meetings. In other words, the attacker is often an insider.

Beneath the Zoombombing phenomenon are access control challenges unique to virtually-hosted social events, challenges that cannot be fixed by better communication security [15, 19] or stricter assessment of users' trustworthiness [24]. We argue that online event hosting platforms such as Zoom [34] or Twitch [32] face a unique access control dilemma when they are used for hosting a specific kind of social events that we dub *anonymous public events*, examples of which include town hall meetings, public lectures and panel discussions, concerts and fan events, and religious gatherings. Such events share three unique requirements.

1. **Interactivity.** User participation of a highly interactive nature is what motivates the hosting of such social events. Severe restriction of privilege, as found in a webinar, does not support the required level of interactivity.
2. **Anonymous Participants.** Such events are typically "public," meaning that the participants are not well-known users within a well-defined institutional boundary. The event hosting system cannot rely on user profiles or past access history to perform a fine-grained assignment of principal or protection domain to a user. To the event organizer, there is but one generic participant profile. Inspired by Elmer *et al.* [8], we call such participants "anonymous" to mean that there is but one generic role [27] for all participants, and that the event organizer knows neither their attributes [18] nor their relationships to one another [11].
3. **Social Control.** Due to the difficulty to profile "anonymous" participants, fine-grained access control policies cannot be formulated. A unique form of access control that we call *social control* must be exercised at the time of the gathering to ensure orderly proceeding of the event. Social control relies on real-time monitoring of the social interactions among the participants, to determine responses that deter, neutralize, or contain offensive behaviours. Typically, some privileged users (e.g., hosts, co-hosts, moderators) are responsible for the monitoring and responding, although this can also be automated in some systems.

Beneath the phenomenon of Zoombombing is the challenge of balancing social control with interactivity. When there is too much social control (as in a webinar), we sacrifice interactivity for order. When there is an overly liberal provision for interactivity, order can no longer be maintained. This challenge is not unique to video teleconferencing platforms, but also other event hosting platforms such as Twitch [33].

In this work, we propose a formal framework to study the balance of social control and interactivity in a system designed for hosting anonymous public events. Specifically, we claim the following contributions:

1. We proposed a formal framework, dubbed **HIPE** (Highly Interactive Public Event), for modelling social control mechanisms and articulating interactivity expectations, so that one can verify if the model of an event hosting system satisfies the interactivity expectations (§2). To this end, we catalogued four popular types of social control (sanction, remedy, containment, and retaliation), which system modellers can use as templates in their system models (§2.1). In addition, we formalized interactivity as a noninterference-style [16, 25] service promise, to capture the idea that performing an action “facilitates” the eventual materialization of certain observation about the system (§2.2). Our notion of interactivity is a weak form of causality.
2. Exploiting the fact that interactivity is a 2-safety hyperproperty [4], we designed an efficient model checking algorithm for verifying interactivity (§3).
3. We conducted an empirical case study with a hypothetical virtual teleconferencing system (Zoom_H), firstly to illustrate how aggressive social control could disrupt interactivity, and secondly to demonstrate the efficiency of our model checking algorithm (§4).

2 The HIPE Framework

Anonymous public events are hosted on a software system, which mediates all the interactions between participants. HIPE is a methodological framework for verifying if the design of such a system satisfies well-defined interactivity expectations. The framework consists of the following components.

1. The event hosting system is modelled as a transition system.
2. Social control mechanisms are captured in the behaviour of the transition system.
3. The interactivity expectations are formally articulated.
4. Model checking is employed to verify if interactivity is preserved by the event hosting system even in the presence of social control.

This section describes two important contributions that are instrumental to the success of the above methodology. Firstly, we offer in §2.1 a catalogue of social control mechanisms that are popular in existing event hosting systems, and demonstrate how such mechanisms can be defined in a transition system. Secondly, we formalize the notion of interactivity in §2.2. Before we move to these topics, we standardize our notations.

We model an event hosting system as a transition system with a finite state space and a deterministic transition relation.

Definition 1 (Deterministic Finite Transition System).

- A **deterministic finite transition system** (or **transition system** for brevity) M is a 6-tuple $(S, Act, \rightarrow, s_0, AP, L)$, where S is a finite set of **states**, Act is a finite set of **actions**, $\rightarrow \subseteq S \times Act \times S$ is a **transition relation**, $s_0 \in S$ is an **initial state**, AP is a finite set of **atomic propositions**, and $L : S \rightarrow 2^{AP}$ is a **labelling function**. In addition, the transition relation is **deterministic**, meaning that if $(s, a, t_1) \in \rightarrow$ and $(s, a, t_2) \in \rightarrow$, then $t_1 = t_2$.
- We write $s \xrightarrow{a} t$ to mean $(s, a, t) \in \rightarrow$. We also overload the transition relation $\rightarrow$ so we can write $s \xrightarrow{\tau} t$ to mean the finite action sequence τ causes M to transition from state s to state t . More formally: (a) $s \xrightarrow{\epsilon} s$, where ϵ is the empty sequence, and (b) $s \xrightarrow{a \cdot \tau} t$ iff there exists $s' \in S$ such that $s \xrightarrow{a} s'$ and $s' \xrightarrow{\tau} t$.
- We write $traces_M(s)$ for the set $\{\tau \in Act^* \mid \exists t \in S. s \xrightarrow{\tau} t\}$. We write $traces_M$ for $traces_M(s_0)$.
- We write $enabled_M(s)$ for the set $\{a \in Act \mid \exists t \in S. s \xrightarrow{a} t\}$. If $a \in enabled_M(s)$, then we write $step_M(s, a)$ to denote the state $t \in S$ such that $s \xrightarrow{a} t$. We also write $succ_M(s)$ for the set $\{t \in S \mid \exists a \in Act. s \xrightarrow{a} t\}$. Lastly, we write $run_M(\tau)$ for the state t for which $s_0 \xrightarrow{\tau} t$.

The following is a form of “Pumping Lemma” for transition systems.

Observation 2. Suppose $M = (S, Act, \rightarrow, s_0, AP, L)$ is a transition system and $s \in S$ is a state. There exists an infinite sequence of transitions $s \xrightarrow{a_1} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} \dots$ iff there is a finite sequence of transitions $s \xrightarrow{b_1} t_1 \xrightarrow{b_2} t_2 \xrightarrow{b_3} \dots \xrightarrow{b_n} t_n$ for some $n \geq |S|$. In addition, if the infinite sequence exists, then the finite sequence can be constructed so that $\{b_1, \dots, b_n\} \subseteq \{a_1, a_2, \dots\}$; if the finite sequence exists, then the infinite sequence can be constructed so that $\{a_1, a_2, \dots\} \subseteq \{b_1, \dots, b_n\}$.

We also define the following notations.

Definition 3. Let Act be a finite alphabet. Given $\tau, \pi \in Act^*$, we write $\pi \preceq \tau$ iff π is a prefix of τ (i.e., there exists $\sigma \in Act^*$ such that $\tau = \pi\sigma$). We also write $prefixes(\tau)$ to denote $\{\pi \in Act^* . \pi \preceq \tau\}$.

Participants of the system may make observations about the system (e.g., an offensive chat message is visible to Alice). Such observations are modelled as propositions that are true in a state ($L(s)$). In the following, we use the terms “proposition” and “observation” interchangeably.

2.1 Modelling Social Control

Anonymous public events require proper control of user behavior in order to ensure the event proceeding is orderly. Anonymity, however, prevents the use of a predefined access control policy for this purpose. Social control enables privileged

users, such as hosts or moderators, to counter the disorderly behaviour in an interactive fashion. Here, we formalize four forms of social control (i.e., sanction, remedy, containment and retaliation) that are popular in actual implementations of event hosting platforms such as Zoom or Twitch.

Example 4. Consider a Zoom meeting. Bob types an offensive message in the chat regarding a polarizing issue. In response, co-host Alice suspends Bob's chat privilege (sanction). She also removes the offensive message from the chat board (remedy). Participants who have seen the short-lived message begin to react to the posting, calling each other names. Alice decides to suspend the entire chat feature for 5 minutes (containment), thereby allowing the presenter to address the issue. Bob, on the other hand, uses other means to stir up the crowd. Alice has no choice but to kick him out of the meeting (retaliation).

The first form of control is sanction control. A number of actions, known as sanctionable actions, are seen as offensive behavior to be prevented. Examples include using inappropriate language when chatting in the chat board, displaying indecent pictures when given the chance to share one's screen, and interrupting a public lecture. Initially, participants of the public event are offered full privilege to interact. Such a privilege allows them to produce a wide variety of actions, including those that are sanctionable. The assumption is that people are well-mannered, and they will exercise self-control to ensure their actions do not disrupt the event. Sanctioning occurs when that assumption is proven wrong. In particular, once a user performs a sanctionable action, the administrators have the ability to restrict the privilege of the offending user, so that the same problematic action cannot be repeated. This restriction of privilege could result in the offender losing other means of interaction (e.g., not allowed to participate in chat, not allowed to share screen, required to mute at all time, etc.).

Definition 5 (Sanction Control). *Suppose $M = (S, Act, \rightarrow, s_0, AP, L)$ is a transition system, $SA \subseteq Act$ is a set of **sanctionable actions** (for a certain user), and $AA \subseteq Act$ is a set of **administrative actions**. M offers (SA, AA) -**sanction control** iff the following holds:*

For every $\tau_0, \tau_1 \in Act^$, $a \in SA$, and $s_1 \in S$, if $s_0 \xrightarrow{\tau_0 \cdot a \cdot \tau_1} s_1$, then there exists $\tau_2 \in AA^*$ and $s_2 \in S$ such that $s_1 \xrightarrow{\tau_2} s_2$ and for every $\tau_3 \in Act^*$ and $s_3 \in S$, if $s_2 \xrightarrow{\tau_3} s_3$, then $\text{enabled}_M(s_3) \cap SA = \emptyset$.*

The definition allows a time lag (τ_1) between the offending action (a) and the implementation of sanction (τ_2). In sanction control (as in the other three forms of control), the administrator may interactively intervene (τ_2). If τ_2 is empty, it means the intervention is automatic: it is programmed into the system.

A second form of control is remedial control, which gives system administrators the ability to recover from the harmful effects of undesirable observations: e.g., pull down an obscene photo when it is shared by an attacker.

Definition 6 (Remedial Control). Suppose $M = (S, Act, \rightarrow, s_0, AP, L)$ is a transition system, $SO \subseteq AP$ is a set of **sanctionable observations**, and $AA \subseteq Act$ is a set of **administrative actions**. M offers (SO, AA) -**remedial control** iff the following holds:

For every $\tau_1 \in Act^*$ and $s_1 \in S$, if $s_0 \xrightarrow{\tau_1} s_1$ and $SO \cap L(s_1) \neq \emptyset$, then there exists $\tau_2 \in AA^*$ and $s_2 \in S$ such that $s_1 \xrightarrow{\tau_2} s_2$ and $SO \cap L(s_2) = \emptyset$.

While sanction control deters an offence by penalizing the offender, remedial control offers countermeasures to mitigate the effect of an offence.

A third form of control is containment control. When an undesirable incident occurs, the administrators might decide to turn off an entire feature for a group of users, so that the matter will not ferment and thus grow out of proportion. When the matter dies down, the administrators may choose to turn back on the feature. Studies show that we are susceptible to becoming victims of false pretences due to our intrinsic ability to process and interpret information influenced by different cognitive biases such as confirmation bias, truth bias, and naive realism [30]. Therefore, a piece of false information shared by a person with established trust and power in the social hierarchy during an ongoing session may fuel negative influence and disrupt the original goal of the social event. For this reason, various online social media also incorporate strategies to label and prevent the spreading of false news [21].

Definition 7 (Containment Control). Suppose $M = (S, Act, \rightarrow, s_0, AP, L)$ is a transition system, $SA \subseteq Act$ is a set of **sanctionable actions** for a certain user, $Priv \subseteq Act$ is a set of actions representing the **privilege** of a group of users to utilize a system feature, $AA \subseteq Act$ is a set of **administrative actions**. M offers $(SA, Priv, AA)$ -**containment control** iff the two conditions below hold:

For every $\tau_0, \tau_1 \in Act^*$, $a \in SA$, and $s_1 \in S$, if $s_0 \xrightarrow{\tau_0 \cdot a \cdot \tau_1} s_1$, then there exists $\tau_2 \in AA^*$ and $s_2 \in S$ such that $s_1 \xrightarrow{\tau_2} s_2$ and for every $\tau_3 \in Act^*$ and $s_3 \in S$, if $s_2 \xrightarrow{\tau_3} s_3$, then $Serv \cap L(s_3) = \emptyset$.

While sanction control restricts the behaviour of the offending user, containment control tentatively restricts the behaviour of an entire group of affected users. This restriction can be lifted later by discretion of the administrators.

A fourth form of control is for the system to retaliate when a user misbehaves. Retaliation involves the system refusing to deliver a certain service for the offending user. A service is modelled here as a set of observations.

Definition 8 (Retaliation Control). Suppose $M = (S, Act, \rightarrow, s_0, AP, L)$ is a transition system, $SA \subseteq Act$ is a set of **sanctionable actions** for a certain user, $AA \subseteq Act$ is a set of **administrative actions**, and $Serv \subseteq AP$ is a set of **observations** representing a service provided by the system to the above user. M offers $(SA, AA, Serv)$ -**retaliation control** iff the following holds:

For every $\tau_0, \tau_1 \in Act^*$, $a \in SA$, and $s_1 \in S$, if $s_0 \xrightarrow{\tau_0 \cdot a \cdot \tau_1} s_1$, then
 there exists $\tau_2 \in AA^*$ and $s_2 \in S$ such that $s_1 \xrightarrow{\tau_2} s_2$ and
 for every $\tau_3 \in Act^*$ and $s_3 \in S$, if $s_2 \xrightarrow{\tau_3} s_3$, then
 $Serv \cap L(s_3) = \emptyset$.

Retaliation control is formally akin to sanction control. The difference is that while sanction control penalizes by revoking user privilege, retaliation control penalizes by refusing to provide a service. This difference may cause retaliation control to disrupt interactivity (§4.3).

2.2 Formalizing Interactivity Expectations

In this subsection, we formalize the notion of interactivity. The user is expecting that the performance of a certain action a “facilitates” the eventual satisfaction of a proposition p in the system state. For example, the action a can be “raising my hand,” and the proposition p could be “my question is answered by the teacher.” This notion of interactivity is related to causality, and thus akin to the notion of noninterference [16, 25]. Yet, we do not mean to say that action a deterministically causes the observation p to materialize. Whether p materializes could still be nondeterministic, or it could depend on the actions of other participants in the system. Take voting as an example. The voter cannot deterministically cause a certain voting outcome to be observed, as there are other voters. Yet, every vote contributes to facilitating the materialization of that outcome. Therefore, what we want to be able to say is that performing action a fundamentally changes the structure of the system’s future. Two possible changes could be involved. First, some sequences of future actions by the event participants could lead the system to enter a state in which the fulfillment of p becomes impossible in the future; performing a disables some of these action sequences. Second, some sequences of future actions by event participants could lead the system to enter a state in which the fulfillment of p becomes inevitable in the future; performing a enables one or more of these action sequences. In other words, the fulfillment of p is not guaranteed by action a , but a alters the future in favour of p ’s fulfillment, by introducing inevitabilities or eliminating impossibilities (or both). This means the user who performs a is interacting with the system, steering its future course in a meaningful way. We capture this notion in the following definitions.

Definition 9. Suppose $M = (S, Act, \rightarrow, s_0, AP, L)$ is a transition system, $\tau \in traces_M$, and $p \in AP$.

- We write $future_M(\tau)$ for the set $\{\pi \in Act^* \mid \tau\pi \in traces_M\}$.
- We write $guarantee_M(\tau, p)$ to denote the set $\{\pi \in future_M(\tau) \mid \exists \sigma \in prefixes(\pi). p \in L(run_M(\tau\sigma))\}$.
- $inevitable_M(\tau, p) = \{\pi \in future_M(\tau) \mid \neg \exists \sigma \in future_M(\tau). \pi \preceq \sigma \wedge \sigma \notin guarantee_M(\tau, p) \wedge ((traces_M(\tau\sigma) = \{\epsilon\}) \vee (|\sigma| \geq |S|))\}$
- $impossible_M(\tau, p) = \{\pi \in future_M(\tau) \mid \forall \sigma \in Act^*. \pi\sigma \notin guarantee_M(\tau, p)\}$

The set $future_M(\tau)$ is the set of future executions of M after having seen the execution trace τ . The set $guarantee_M(\tau, p)$ is the set of future executions that establish p . The set $inevitable_M(\tau, p)$ is the set of future executions π that would *inevitably* lead to the fulfillment of p . More specifically, π cannot be extended to a longer future execution σ that does not fulfill p but either (a) σ is a terminated execution ($traces_M(\tau\sigma) = \{\epsilon\}$) or (b) σ is long enough ($|\sigma| \geq |S|$) to signal the existence of an infinite future execution that never fulfills p (Observation 2). The set $impossible_M(\tau, p)$ is the set of future executions π that would render the fulfillment of p impossible. In other words, it is not possible to augment π by a suffix σ such that $\pi\sigma$ is a future execution that fulfills p .

Definition 10 ((a, p)-Interactivity). *Given transition system $M = (S, Act, \rightarrow, s_0, AP, L)$, $a \in Act$ and $p \in AP$, M satisfies (a, p)-interactivity iff for every $\tau \in traces_M$ such that $\tau a \in traces_M$ as well, the following conditions hold:*

1. $future_M(\tau a) \setminus future_M(\tau) \subseteq inevitable_M(\tau a, p)$.
2. $future(\tau) \setminus future(\tau a) \subseteq impossible_M(\tau, p)$.
3. $inevitable_M(\tau, p) \cap future_M(\tau a) \subseteq inevitable_M(\tau a, p) \cap future_M(\tau)$.
4. $impossible_M(\tau a, p) \cap future_M(\tau) \subseteq impossible_M(\tau, p) \cap future_M(\tau a)$.

The four conditions above compare the differences between future executions caused by performing action a and those caused by not performing a . Condition 1 requires that the additional future executions introduced by performing a (i.e., $future_M(\tau a) \setminus future_M(\tau)$) inevitably lead to the fulfillment of p . Condition 2 demands that the additional future executions introduced by refraining from performing a (i.e., $future(\tau) \setminus future(\tau a)$) render the fulfillment of p impossible. Condition 3 and 4 are both concerning the common future executions arising from both performing a and not performing a : i.e., the executions in $future_M(\tau) \cap future_M(\tau a)$. Condition 3 is a simplified restatement of the following:

$$inevitable_M(\tau, p) \cap \left(future_M(\tau) \cap future_M(\tau a) \right) \subseteq \\ inevitable_M(\tau a, p) \cap \left(future_M(\tau) \cap future_M(\tau a) \right). \quad (1)$$

Condition 3 can be obtained from assertion (1) by noticing that $inevitable_M(\tau, p) \subseteq future_M(\tau)$ and $inevitable_M(\tau a, p) \subseteq future_M(\tau a)$. Thus Condition 3 states that among the common future executions arising from both performing a and not performing a , the inevitable future executions resulting from not performing a is a subset of the inevitable ones resulting performing a (i.e., performing a gives rise to more inevitabilities). Likewise, Condition 4 is equivalent to the following:

$$impossible_M(\tau a, p) \cap \left(future_M(\tau) \cap future_M(\tau a) \right) \subseteq \\ impossible_M(\tau, p) \cap \left(future_M(\tau) \cap future_M(\tau a) \right). \quad (2)$$

The simplification is justified by noticing that $impossible_M(\tau, p) \subseteq future_M(\tau)$ and $impossible_M(\tau a, p) \subseteq future_M(\tau a)$. Consequently, Condition 4 asserts that among the common future executions arising from both performing a and not performing a , the impossible ones resulting from performing a is a subset of the impossible ones resulting from not performing a (i.e., not performing a leads to more impossibilities).

Interactivity captures a weak form of causality. If observation p corresponds to a system service, then we can loosely interpret (a, p) -interactivity as a form of service promise: performing a contributes to the delivery of service p .

Interactivity is a hyperproperty [4]. More specifically, it is a safety hyperproperty (aka hypersafety), and even a 2-safety hyperproperty [4].¹ This enables the design of efficient model checking algorithms for verifying interactivity (§3).

Excessive social control, in some cases, could interfere with interactivity. This phenomenon will be examined in the case study of §4.

3 Verifying Interactivity

The four forms of control presented in §2 can be built into a system model (as a transition system). We would like to verify that such control mechanisms do not violate certain expectation of interactivity. We have observed that (a, p) -interactivity is a 2-safety hyperproperty, meaning that any violation to the hyperproperty can be characterized by 2 “parallel” execution traces. No extension of the two traces, or the introduction of additional traces into the system model, can cause the system model to recover from violation. Taking advantage of this observation, we present an efficient algorithm for verifying whether a transition system satisfies (a, o) -interactivity. The algorithm consists of two major steps, (a) preprocessing and (b) model checking, which we outline in §3.1 and §3.2 respectively.

3.1 Step 1: Preprocessing

Consider a reachable state s . Let’s suppose $s_0 \xrightarrow{\tau} s$ for some $\tau \in traces_M$. We classify s in the following ways.

1. State s is **inevitable** iff $traces_M(s) \subseteq inevitable_M(\tau, p)$.
2. State s is **impossible** iff $traces_M(s) \subseteq impossible_M(\tau, p)$.
3. State s is **neither inevitable nor impossible**.

These categories are mutually exclusive. Intuitively, a state s is inevitable when s satisfies the CTL formula AFp , and s is impossible when s satisfies $AG\neg p$. There are standard algorithms for labelling all states that satisfy these CTL formulas in $O(|S| + |\rightarrow|)$ time [7, §4.1]. The preprocessing step thus labels each state as one of the three categories above.

¹ See Observation 11 for a notion of violation for (a, p) -interactivity. Each violation is composed of a pair of execution traces.

Algorithm 1: CHECKINTERACTIVITY(M, a)

Input: $M = (S, Act, \rightarrow, s_0, AP, L)$ is a transition system and $a \in Act$. It is assumed that states in M have been labeled as inevitable, impossible, or neither of the two, based on some atomic proposition $p \in AP$.

Result: A boolean indicating whether M is (a, p) -interactive.

Local: ST is a stack of vertices, $visited$ is a set of states, and $visitedPairs$ is a set of state pairs.

```

1   $ST \leftarrow \{s_0\}$ ;  $visited \leftarrow \emptyset$ ;  $visitedPairs \leftarrow \emptyset$ ;
2  while  $ST$  is not empty do
3      pop  $s$  from  $ST$ ;
4      if  $s \notin visited$  then
5           $visited \leftarrow visited \cup \{s\}$ ;
6          if  $a \in enabled_M(s)$  then
7               $s' \leftarrow step_M(s, a)$ ;
8              if  $(s, s') \notin visitedPairs$  then
9                  if  $\neg CHECKDOMINANCE(M, s, s', visitedPairs)$  then
10                     return false;
11             foreach  $t \in succ_M(s)$  do push  $t$  into  $ST$ 
12 return true;

```

3.2 Step 2: Model Checking

Our model checking algorithm takes two arguments: (i) a transition system M in which states have already been labelled (i.e., inevitable, impossible, neither) based on an atomic predicate p , and (ii) an action a with which one wants to verify that M satisfies (a, p) -interactivity. The design of the algorithm is inspired by the standard nested DFS algorithm [5]. The first-level DFS is specified in the algorithm CHECKINTERACTIVITY (Algorithm 1). The algorithm enumerates every reachable state s . If $s \xrightarrow{a} s'$ for some next state s' , then the second-level DFS, CHECKDOMINANCE (Algorithm 2), is invoked to check if s is “no more favourable than” s' , meaning that s generates no more inevitable future executions and no fewer impossible future executions than s' . A loop invariant of CHECKINTERACTIVITY is that the set $visitedPairs$ contains all the pairs (s, s') that have already been checked by CHECKDOMINANCE to satisfy “ s is no more favourable than s' ”. This prevents the rechecking of the same pair.

Algorithm 2, CHECKDOMINANCE, simulates two executions from state s and s' , and checks the four conditions of Definition 10. Specifically, a violation of Condition 3 or 4 is detected in line 5. In case states s and s' are both neither inevitable nor impossible (line 8), meaning that Condition 3 and 4 are not immediately satisfied, then we have to simulate a further step of execution. In this context, a violation of Condition 1 is detected in line 10 and 11, and a violation of Condition 2 is detected in line 12 and 13. If neither Condition 1 or 2 is violated, we simulate the same step for both executions, and explores further in our second-level DFS (line 15).

Algorithm 2: CHECKDOMINANCE($M, s, s', \text{visitedPairs}$)**Local:** ST is a stack of vertex pairs.

```

1  $ST \leftarrow \{(s, s')\};$ 
2 while  $ST$  is not empty do
3   pop  $(s, s')$  from  $ST$ ;
4   if  $(s, s') \notin \text{visitedPairs}$  then
5     if  $s$  is inevitable but  $s'$  is not, or  $s'$  is impossible but  $s$  is not then
6       return false;
7      $\text{visitedPairs} \leftarrow \text{visitedPairs} \cup \{(s, s')\};$ 
8     if both  $s$  and  $s'$  are neither inevitable nor impossible then
9       foreach  $b \in \text{enabled}_M(s) \cup \text{enabled}_M(s')$  do
10        if  $b \notin \text{enabled}_M(s)$  then
11          if  $\text{step}_M(s', b)$  is not inevitable then return false
12        else if  $b \notin \text{enabled}_M(s')$  then
13          if  $\text{step}_M(s, b)$  is not impossible then return false
14        else
15           $t \leftarrow \text{step}_G(s, b); t' \leftarrow \text{step}_G(s', b);$  push  $(t, t')$  into  $ST$ ;
16 return true;

```

Complexity Analysis. Let N be $|S|$ and M be $|\rightarrow|$. As expected, the first level of DFS has a complexity of $O(N + M)$ and invokes the second-level DFS $O(N)$ times. We claim that across all $O(N)$ invocations, the second-level DFS has a time complexity of $O(N^2 + NM)$. To see this, observe that the second-level DFS visits state pairs of the form (s, s') . There are $O(N^2)$ such pairs. The total number of transitions traversed by the second-level DFS is $\sum_{s, s' \in S} (|\text{enabled}_M(s)| + |\text{enabled}_M(s')|) = \sum_{s' \in S} \sum_{s \in S} |\text{enabled}_M(s)| + \sum_{s \in S} \sum_{s' \in S} |\text{enabled}_M(s')|$. The sum above simplifies to $O(NM)$. This confirms our claim that the second-level DFS runs in $O(N^2 + NM)$ time. In conclusion, the overall running time of our nested DFS algorithm is $O(N + M + N^2 + NM) = O(N(N + M))$.

4 An Empirical Case Study

This section reports an empirical case study we conducted using a hypothetical version of the popular virtual teleconferencing platform Zoom, which we dub Zoom_H . Our goals are (a) to illustrate the interplay between social control and interactivity guarantees (§4.3), and (b) to evaluate the efficiency of our model checking algorithm (§4.4).

4.1 Model Description

Zoom_H is an online lecturing platform. It features concurrent hosting of lectures in multiple (virtual) classrooms. In each classroom is a teacher and a number of

students. Within one classroom, a student can post a question on the classroom-specific chat board. The teacher may, from time to time, nondeterministically turn to the students and answer questions posted on the chat board. This, however, makes the service quality unpredictable: if the teacher is too engaged with presentation, no question will get answered. A student may improve the predictability of question answering by raising his hand. The system features a clock, and in fixed intervals, the teacher will be prompted by the system to turn and answer the questions asked by students with raised hands.

When a student is not actively participating in a lecture (meaning no question asked and hand not raised), she can leave the classroom and move to an “adjacent” classroom. Classrooms are connected to one another via virtual corridors. Thus the classrooms and corridors form an undirected graph.

Model Variants. We created a number of models for the aforementioned domain, each fashioned with a different social control mechanism.

- **Z-BASIC:** This baseline model, with no social control mechanism.
- **Z-SAN:** In this and the following models, there is a distinguished sanctionable action that each student may perform. When the student perform the sanctionable action in Z-SAN, the teacher responds by permanently disabling the sanctionable action for the offending student.
- **Z-REM:** The sanctionable action will produce a distinguished sanctionable observation, which the teacher will neutralize in response.
- **Z-CON:** In response to the performance of a sanctionable action, the teacher will restrict all students’ privilege so that the sanctionable action cannot be performed for a short period of time.
- **Z-ALL:** This is a combination of Z-SAN, Z-REM, and Z-CON.
- **Z-RET:** The teacher will refuse to answer the questions posted by a student who has performed the sanctionable action.

Interactivity Expectations. We want to make sure that asking questions and raising hands contribute to questions being answered. In other words, we want to ensure that the aforementioned models satisfy (a_x, p_x) -interactivity and (b_x, p_x) -interactivity, where a_x is the action “student x asks a question,” b_x is “student x raises her hand.” p_x is “student x ’s question is answered.”

Note that the causal relationship between b_x (raising hand) and p_x (question answered) is deterministic. On the contrary, there is no causal relationship between a_x and p_x . Even when a_x is performed, the teacher may nondeterministically choose to answer or not answer the question. If x chooses to raise his hand, then p_x is guaranteed. But hand raising (a_x) is a nondeterministic choice of x . Yet, we expect that the act of asking a question fundamentally reshapes the structure of the system’s future, so that doing so “facilitates” the answering of questions.

In all the following experiments, we focus on checking for (a_x, p_x) -interactivity.

4.2 Experimental Setup

We implemented the preprocessing and model checking algorithms of §3, and measure their performance on instances of the 6 models described in §4.1.

Model Specification and Generation. The models were encoded in a subset of the Planning Domain Description Language (PDDL) [2, 17], which allows one to specify transition systems based on either STRIPS or ADL (Action Description Language). We developed a Java tool that compiles a PDDL specification to a graph, which forms the basis of model checking.

We generated multiple instances of transition systems for each of the 6 models described in §4.1. The difference in the instances depends on (a) number of classrooms, (b) their connectivity (aka corridors), and (c) number of students initially residing in each classrooms. We generated the classrooms and their connections as Erdős-Rényi random graphs $G(n, p)$, consisting of n vertices, and each undirected edge materializes with probability p [9]. We set p to 0.5. We used the `GnpRandomGraphGenerator < V, E >` class of the *JGraphT* Java library to generate such a random graph [20]. In the initial states, we assigned each student randomly to a classroom. Finally, we randomly select a student x as the basis to formulate the action a_x and observation p_x in verifying (a_x, p_x) -interactivity.

We ran the preprocessing and model checking algorithms of §3 against the generated model instances, and measured the running time. We exclude the time spent on generating the transition systems from the PDDL specification.

System Configuration. All the experiments were conducted on the *Advanced Research Computing (ARC)* cluster at the University of Calgary. Specifically, they ran in the *cpu2022* partition of the ARC cluster equipped with 52 cores, 2x Intel(R) Xeon(R) Gold 5320 CPU @ 2.20GHz (Ice Lake), 256 GB memory, 40 Gbit/s InfiniBand network, and 15 TB of storage. Our experiments do not rely on the volume of the input data as it takes the model specification, generates transition systems, and then model checks; therefore, the experiments are primarily CPU and memory-intensive. The experiments ran with variable sizes of allocated memory, all managed via the Slurm configurations deployed by ARC. The experimental code, including the preprocessing, model checking, and model generation algorithms, were developed on the Java Development Kit (JDK) version 17.

4.3 Interplay Between Social Control and Interactivity

We checked if the 6 models of §4.1 satisfy (a_x, p_x) -interactivity, with the number of students and rooms set to 2 and 2 respectively. The goal was to understand how different social control mechanisms affect service promises (namely, interactivity) and to gauge the performance of our model checking algorithm in small models. The results are shown in Table 1.

Without any social control (Z-BASIC), (a, p) -interactivity is satisfied as expected. The introduction of sanction control (Z-SAN), remedial control (Z-REM), containment control (Z-CON), and their combination (Z-ALL) preserve

Table 1. Interplay Between Social Control and Interactivity.

Model	Students	Rooms	States	Transitions	Time (ms)	Interactive?
Z-BASIC	2	2	768	5,888	23	Yes
Z-SAN	2	2	27,648	294,912	317	Yes
Z-REM	2	2	6,912	80,640	129	Yes
Z-CON	2	2	12,096	132,480	171	Yes
Z-ALL	2	2	55,296	562,176	578	Yes
Z-RET	2	2	10,800	116,640	147	No

interactivity as well. This is because in these 4 models, offences are penalized by revoking the privilege to perform the sanctionable action. Thus interactivity, which takes the form of service promises (“questions are answered when hands are raised”), is not affected. The exception is retaliation control (Z-RET), which violates interactivity. In Z-RET, the teacher retaliates by categorically refusing to answer questions posted by the offending student, thereby breaking service promises, leading to the violation of interactivity.

Due to the small model sizes, the model checking algorithm terminates within microseconds.

4.4 Scaling Up

We conducted further experiments to assess the performance of the model checking algorithm. Specifically, we measured the runtime of the preprocessing and model checking algorithm against 4 instances of the Z-ALL model (i.e., with sanction, remedial and containment control). We created only one classroom, but varied the number of students from 2 to 5 to observe how the verification algorithm scaled. One can prove a small model property (Theorem 12 in Appendix A) to show that it is not necessary to verify interactivity for models bigger than one classroom and one student. We nevertheless varied the number of students only to assess the performance of the model checking algorithm.

The results are plotted in Fig. 1, which is in log-log scale. With 5 students, the number of states exceeds 47 million and the number of transitions surpasses 832 million. The runtime of the preprocessing and model checking algorithm is 20 min and 46 s, which is quite acceptable. We deem the design of our model checking algorithm adequate for practical use.

5 Related Works

This work is an extension to a line of work aimed at better understanding how novel access control schemes can improve the security and privacy of online social interactions [10–13]. While previous works focus on the security and privacy of online social network systems, the present work attends to the access control challenges of social event hosting platforms.

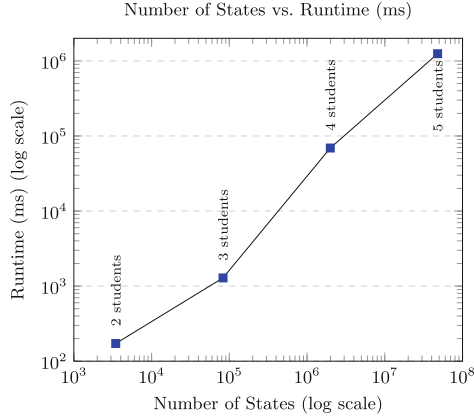


Fig. 1. Log-log relationship between the number of states and runtime across increasingly large state spaces of the Z-ALL model.

Recent research in the security of Zoom has predominantly focused on communication security, with particular emphasis on cryptographic methods to safeguard communications. Gauthier *et al.* performed a security analysis on Zoom, Google Meet, and Microsoft Teams [15]. It is discovered that such platforms fail to provide end-to-end encryption for protecting communications. Isobe *et al.* explore the implementation of end-to-end encryption in Zoom, highlighting the critical role of cryptographic security in protecting user data [19]. Also relevant is the work of Metallidou *et al.*, who devise a sophisticated model for assessing the trustworthiness of users in an online learning system via direct or indirect observations [24]. These measures, however, do not address the unique access control challenges of anonymous public events. To the best of our knowledge, our work is the first to confront such challenges, and to study the balance of social control and interactivity in a formal setting.

Noninterference, as conceptualized by Goguen and Meseguer [16], uses causality to model information flow: if there is a causal relationship between a secure input and the output of an insecure channel, then there is information leak. Many variants of noninterference has been proposed [25]. A major contribution of the aforementioned line of work is to allow the system designer to separate policy (i.e., noninterference security) from mechanism (e.g., the Bell-LaPadula model [22]). Inspired by Goguen and Meseguer, the approach of this paper is to separate social control mechanisms from interactivity expectations. Our notion of interactivity is a noninterference-style notion of causality. Compared to noninterference, our interactivity is a weaker form of causality: (a, p) -interactivity does not guarantee a deterministic causal relationship between action a and observation p . Instead, it only asserts that a facilitates the future materialization of p , by reshaping the future behavior of the system to introduce inevitabilities and eliminate impossibilities. A second point of comparison is that in informa-

tion flow control, the goal is to eliminate causality, while in our work, the goal is to preserve causality.

A property, such as safety or liveness, is a predicate over execution traces [1, 28]. A hyperproperty, on the other hand, is a predicate over sets of execution traces. Such a set of traces represents all the possible behaviours of a system. Thus a hyperproperty is a predicate over systems. Hyperproperties can capture the correlation of traces, as found in noninterference [16]. Clarkson and Schneider pioneered the study of hyperproperty, in which they propose safety hyperproperty as a generalization of safety property [4]. A safety property can be characterized by a set of violations, each being a finite trace representing an irrecoverable failure. Likewise, a safety hyperproperty can be characterized by a set of violations, each being *a set of* finite traces, representing problematic correlation of executions from which no system extension can fix. As a special case, a 2-safety hyperproperty is one for which every violation contains only two finite traces. It has been demonstrated that secure information flow is in fact a 2-safety hyperproperty [31], and thus its verification can be conducted by the simultaneous examination of two execution traces [3, 6, 26]. In our work, (a, p) -interactivity is a 2-safety hyperproperty, thereby allowing the same reasoning technique (simultaneously examining two execution traces) to be applied for verification.

The nested-DFS algorithm, first proposed by Courcoubetis *et al.* [5], is an efficient algorithm for verifying temporal properties in finite-state systems. Our design of model checking algorithm is inspired by the aforementioned algorithm. We exploit the fact that (a, p) -interactivity is a 2-safety hyperproperty, and have the second-level DFS explores two execution traces simultaneously. As we demonstrated empirically, this design leads to an efficient running time.

6 Conclusion and Future Work

Anonymous public events call for the design of social control mechanisms to maintain order. Excessive control, however, could disrupt interactivity, which is the very motivation for hosting such events. We employed the HIPE framework for formalizing both social control and interactivity, so that one can use an efficient model checking algorithm to verify that interactivity expectations are preserved by social control mechanisms.

We envision a number of future directions for this research. (1) Four popular forms of social control are catalogued in §2, but we recognize the need for the further development of a comprehensive theory of social control to facilitate formal modelling and verification of virtual event hosting systems. (2) The notion of (a, p) -interactivity was the first form of interactivity that caught our attention. We want to study other notions of interactivity, their compatibility with social control, and their verification procedures. Examples include (a) collaborative influence, (b) competition, and (c) group decision making [14]. (3) While social control in anonymous public events does not rely on user profiles, a fruitful research direction is to study social control mechanisms that consider past

engagement history, especially in an event series (e.g., concert tour, lecture series, etc.).

Acknowledgement. This work is supported in part by an NSERC Discovery Grant (grant number RGPIN-2020-05238).

A Small Model Property for Zoom_H

As interactivity is a 2-safety hyperproperty, there is a notion of violation for a system if the latter does not satisfy interactivity.

Observation 11 (Violation). *A violation of (a, p) -interactivity by transition system M consists of a pair of execution traces: $\tau\pi$ and $\tau a\pi$, such that $\tau, \tau a \in \text{traces}_M$ and $\pi \in \text{future}_M(\tau) \cup \text{future}_M(\tau a)$. In addition, for the pair to be a violation, one of the following must hold:*

- $\pi \in \text{future}_M(\tau a) \setminus \text{future}_M(\tau)$ but $\pi \notin \text{inevitable}_M(\tau a, p)$.
- $\pi \in \text{future}_M(\tau) \setminus \text{future}_M(\tau a)$ but $\pi \notin \text{impossible}_M(\tau, p)$.
- $\pi \in \text{future}_M(\tau) \cap \text{future}_M(\tau a)$ and either (a) π belongs to $\text{inevitable}_M(\tau)$ but not to $\text{inevitable}_M(\tau a)$, or (b) π belongs to $\text{impossible}_M(\tau a, p)$ but not to $\text{impossible}_M(\tau, p)$.

The small model property for Zoom_H can be established based on this notion of violation.

Theorem 12. *An instance of the model Z-ALL satisfies (a_x, p_x) -interactivity if and only if an instance of Z-ALL containing one classroom and one student satisfies (a_x, p_x) -interactivity. (Here, a_x and p_x are defined in §4.1.) The same is true of the other models: Z-BASIC, Z-SAN, Z-REM, Z-CON, and Z-RET.*

Proof. We sketch a proof of the statement for Z-ALL. The proof for the other 5 models are similar.

Let M_{sm} be the instance of Z-ALL with one classroom and one student, and let M_{bg} be another instance of Z-ALL with either a different number of classrooms or a different number of students. We thus prove that M_{sm} has a violation if and only if M_{bg} has a violation.

Suppose M_{sm} has a violation of (a_x, p_x) -interactivity, which takes the form of two execution traces $\tau\pi$ and $\tau a\pi$ as specified above, the the same sequences $\tau\pi$ and $\tau a\pi$ are legitimate traces in M_{bg} (just assuming the rest of the students do not perform any action). As the students' actions are independent of one another, whether the trace π is inevitable or impossible is the same whether we are looking at M_{sm} or M_{bg} . The violation is therefore preserved by M_{bg} .

Suppose now M_{bg} has a violation made up of sequences $\tau\pi$ and $\tau a\pi$. Again, as students' actions are independent of one another, the violation can be attributed to the student x . We can remove from τ and π the actions by other students, to obtain traces τ' and π' , and the pair $\tau'\pi'$ and $\tau'a\pi'$ will still form a violation. In addition, the violation must occur in a particular classroom. We can therefore

remove from τ' all the actions of x prior to her last entering that classroom, and obtain the trace τ'' . The pair $\tau''\pi'$ and $\tau''a\pi'$ form a violation, one that involves only one classroom and one student. This pair can be replicated in M_{sm} and form a violation for that small model. $\square$

References

1. Alpern, B., Schneider, F.B.: Defining liveness. *Inf. Process. Lett.* **21**(4), 181–185 (1985)
2. Bacchus, F.: Subset of PDDL for the AIPS2000 planning competition (draft 1). <https://engineering.purdue.edu/~relation/surf/papers/pddl-subset.pdf>
3. Barthe, G., D’Argenio, P.R., Rezk, T.: Secure information flow by self-composition. *Math. Struct. Comput. Sci.* **21**(6), 1207–1252 (2011)
4. Clarkson, M.R., Schneider, F.B.: Hyperproperties. *J. Comput. Secur.* **18**(6), 1157–1210 (2010)
5. Courcoubetis, C., Vardi, M., Wolper, P., Yannakakis, M.: Memory-efficient algorithms for the verification of temporal properties. *Formal Methods Syst. Des.* **1**(2–3), 275–288 (1992)
6. Darvas, Á., Hähnle, R., Sands, D.: A theorem proving approach to analysis of secure information flow. In: Hutter, D., Ullmann, M. (eds.) *SPC 2005*. LNCS, vol. 3450, pp. 193–209. Springer, Heidelberg (2005). https://doi.org/10.1007/978-3-540-32004-3_20
7. Clarke, E.M., Gumberg, O., Peled, D.A.: *Model Checking*. MIT Press (1999)
8. Elmer, G., Neville, S.J., Burton, A., Ward-Kimola, S.: Zoombombing during a global pandemic. *Soc. Media + Soc.* **7**, 205630512110353 (2021)
9. Erdős, P., Rényi, A.: On random graphs i. *Publ. Math. Debrecen* **6**, 290–297 (1959)
10. Fong, P.W.L.: Preventing Sybil attacks by privilege attenuation: a design principle for social network systems. In: *Proceedings of the 2011 IEEE Symposium on Security and Privacy (S&P 2011)*, pp. 263–278 (2011)
11. Fong, P.W.L.: Relationship-based access control: protection model and policy language. In: *Proceedings of the First ACM Conference on Data and Application Security and Privacy (CODASPY 2011)*, pp. 191–202 (2011)
12. Fong, P.W.L., Anwar, M., Zhao, Z.: A privacy preservation model for Facebook-style social network systems. In: Backes, M., Ning, P. (eds.) *ESORICS 2009*. LNCS, vol. 5789, pp. 303–320. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-642-04444-1_19
13. Fong, P.W.L., Mehregan, P., Krishnan, R.: Relational abstraction in community-based secure collaboration. In: *Proceedings of the 20th ACM Conference on Computer and Communications Security (CCS 2013)*, pp. 585–598 (2013)
14. Gaertner, W.: *A Primer in Social Choice Theory*. Oxford Academic (2006)
15. Gauthier, N.H., Husain, M.I.: Dynamic security analysis of zoom, google meet and Microsoft teams. In: Park, Y., Jadav, D., Austin, T. (eds.) *SVCC 2020*. CCIS, vol. 1383, pp. 3–24. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-72725-3_1
16. Goguen, J.A., Meseguer, J.: Security policies and security models. In: *1982 IEEE Symposium on Security and Privacy*, pp. 11–11. IEEE (1982)
17. Haslum, P., Lipovetzky, N., Magazzeni, D., Muise, C.: *An Introduction to the Planning Domain Definition Language*. Springer, Cham (2019). <https://doi.org/10.1007/978-3-031-01584-7>

18. Hu, V.C., Kuhn, D.R., Ferraiolo, D.F.: Attribute-based access control. *Computer* **48**(2), 85–88 (2015)
19. Isobe, T., Ito, R.: Security analysis of end-to-end encryption for zoom meetings. In: Baek, J., Ruj, S. (eds.) *ACISP 2021*. LNCS, vol. 13083, pp. 234–253. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-90567-5_12
20. JGraphT Team: Create a random graph based on the $g(n,p)$ erdős-rényi
21. Khan, T., Michalas, A., Akhunzada, A.: Fake news outbreak 2021: Can we stop the viral spread? *J. Netw. Comput. Appl.* **190**, 103112 (2021)
22. LaPadula, L.J., Bell, D.E.: MITRE technical report 2547, volume II: secure computer systems. *J. Comput. Secur.* **4**(2–3), 239–263 (1996)
23. Ling, C., Balci, U., Blackburn, J., Stringhini, G.: A first look at Zoombombing. In: *2021 IEEE Symposium on Security and Privacy (S&P)*, pp. 1452–1467 (2021)
24. Metallidou, C., Psannis, K.E., Goudos, S., Sarigiannidis, P., Ishibashi, Y.: Communication and security issues in online learning during the COVID-19 pandemic. In: *2021 IEEE 9th International Conference on Information, Communication and Networks (ICICN)*, pp. 538–544 (2021)
25. van der Meyden, R., Zhang, C.: A comparison of semantic models for noninterference. *Theoret. Comput. Sci.* **411**(47), 4123–4147 (2010)
26. Pottier, F., Simonet, V.: Information flow inference for ML. *ACM Trans. Program. Lang. Syst.* **25**(1), 117–158 (2003)
27. Sandhu, R.S., Coyne, E.J., Feinstein, H.L., Youman, C.E.: Role-based access control models. *Computer* **29**(2), 38–47 (1996)
28. Schneider, F.: Enforceable security policies. *ACM Trans. Inf. Syst. Secur.* **3**(1), 30–50 (2000)
29. Setera, K.: FBI warns of teleconferencing and online classroom hijacking during COVID-19 pandemic. FBI Boston (2020). <https://www.fbi.gov/contact-us/field-offices/boston/news/press-releases/fbi-warns-of-teleconferencing-and-online-classroom-hijacking-during-covid-19-pandemic>
30. Stahl, K.: Fake news detection in social media. *California State Univ. Stanislaus* **6**, 4–15 (2018)
31. Terauchi, T., Aiken, A.: Secure information flow as a safety problem. In: Hankin, C., Siveroni, I. (eds.) *SAS 2005*. LNCS, vol. 3672, pp. 352–367. Springer, Heidelberg (2005). https://doi.org/10.1007/11547662_24
32. Twitch. <https://www.twitch.tv/>
33. Zhou, Y., Farzan, R.: Designing to stop live streaming cyberbullying: a case study of twitch live streaming platform. In: *Proceedings of the 10th International Conference on Communities & Technologies-Wicked Problems in the Age of Tech*, pp. 138–150 (2021)
34. Zoom video communications. <https://zoom.us/>



Faster Post-quantum TLS 1.3 Based on ML-KEM: Implementation and Assessment

Jieyu Zheng, Haoliang Zhu, Yifan Dong, Zhenyu Song, Zhenhao Zhang,
Yafang Yang, and Yunlei Zhao^(✉)

School of Computer Science, Fudan University, Shanghai, China
{jyzyheng23, hlzhu22, yfdong22, songzy23, zhenhaozhang23}@m.fudan.edu.cn,
{18110240046, ylzhaol}@fudan.edu.cn

Abstract. TLS is extensively utilized for secure data transmission over networks. However, with the advent of quantum computers, the security of TLS based on traditional public-key cryptography is under threat. To counter quantum threats, it is imperative to integrate Post-Quantum Cryptography (PQC) into TLS (PQ-TLS, for short). Most PQ-TLS research focuses on integration and evaluation, but few studies address the improvement of PQ-TLS performance by optimizing PQC implementation. As a critical part of PQ-TLS, post-quantum key encapsulation Mechanisms (PQ-KEMs) directly impact PQ-TLS handshake performance. ML-KEM is a NIST-standardized PQ-KEM. In this work, we explore how to improve ML-KEM performance using the latest Intel's Advanced Vector Extensions instruction set AVX-512. We give our implementation details of parallelizing polynomial multiplication, modular reduction, and other computationally intensive modules strategically within ML-KEM. Our optimized ML-KEM implementation achieves up to $1.64\times$ speedup compared to the latest AVX2 implementation. Based on our ML-KEM AVX-512 implementation, we introduce a novel batch key generation method for ML-KEM that can seamlessly integrate into the TLS protocols. The batch method accelerates the key generation procedure by $3.5\times$ to $4.9\times$. Finally, we integrate the optimized AVX-512 implementation of ML-KEM into TLS 1.3 and assess handshake performance under both PQ-only and PQ-hybrid modes. The assessment demonstrates that our faster ML-KEM implementation results in a higher number of TLS 1.3 handshakes per second under both modes. To further investigate ML-KEM performance improvement, we revisit two IND-1-CCA KEM constructions proposed at Eurocrypt'22 and Asiacrypt'23, implement them based on ML-KEM and integrate the one of better performance into TLS 1.3 with benchmarks.

Keywords: Post-Quantum Cryptography · TLS 1.3 · ML-KEM · AVX-512

1 Introduction

Digital communications are ubiquitous worldwide, with most Internet connections relying on Transport Layer Security (TLS) to secure data transmission. However, the current TLS protocol remains vulnerable to quantum attacks. TLS employs public-key cryptography algorithms, including Elliptic Curve Diffie-Hellman (ECDH), Elliptic Curve Digital Signature Algorithm (ECDSA), and RSA. However, these algorithms are susceptible to quantum computing threats, as demonstrated by Shor’s algorithm [47]. To address this vulnerability, the National Institute of Standards and Technology (NIST) launched the PQC competition in 2016. After three rounds, NIST announced the selection of the first algorithms to be standardized, including Kyber, Dilithium, Falcon, and SPHINCS+ [36]. In August 2023, NIST designated three standard drafts: ML-KEM [52], ML-DSA [51], and ML-SLH [53], renamed from Kyber, Dilithium and SPHINCS+ respectively.

Amidst the continuous progress in NIST’s PQC algorithm standardization, significant endeavors have been devoted to the development of PQ-TLS over the past decade. Beginning in 2014, Chang et al. [16] introduced a post-quantum SSL/TLS library for embedded systems. Subsequently, Bos et al. [14] developed PQC ciphersuites for TLS based on the ring learning with errors (R-LWE) problem. In July 2016, Chrome introduced the `newhope1024` post-quantum option [13], signaling an initial step towards integrating PQC into mainstream browsers. However, due to patent issues, Chrome removed the `newhope1024` option in November 2016. Subsequent efforts by industry giants such as Google and Cloudflare focused on evaluating the performance of PQC candidates within TLS. In 2018, Google and Cloudflare conducted experiments with NIST PQC candidate HRSS and X25519 in TLS 1.3 Chrome, while in 2019 they utilized `ntuhrss701` and `sntrup761` in their TLS experiments [32].

Recent research on PQ-TLS has predominantly concentrated on three pivotal domains: (i) Integration of PQC into the TLS Protocol: Exploring methodologies to seamlessly incorporate PQC into the TLS framework [20, 25, 38, 40, 41, 44, 45]. (ii) Performance Evaluation and Communication Overheads: Assessing the computational efficiency and communication overheads incurred by PQC primitives within the TLS ecosystem [7, 15, 23, 27, 28, 39, 48–50, 54]. (iii) Optimized Implementations for Enhanced PQ-TLS Efficiency: Enhancing the computational efficiency of PQC within TLS through optimized implementations [11]. Most research on PQ-TLS primarily focuses on exploring how to integrate various PQC cryptographic primitives into TLS and evaluating their performance. However, there is a noticeable scarcity of work improving PQ-TLS performance. Performance stands as a critical factor in TLS applications. As an integral component of PQ-TLS, PQC algorithms directly impact the handshake time of PQ-TLS. Therefore, optimizing the implementation of PQC algorithms and integrating them into TLS may contribute to reducing the handshake time of PQ-TLS.

Motivations. Recent work in USENIX Security 2024 presented an accelerated Ed25519 and X25519 AVX-512 engine tailored for TLS 1.3, offering

significant performance improvement [56]. Previous work explored optimized PQC implementations using the latest Intel Single Instruction Multiple Data (SIMD) instruction AVX-512 (e.g. [17, 18, 33]). However, the integration of PQC AVX-512 optimized implementations into TLS 1.3 remains unrealized, and there is currently no AVX-512 implementation available for ML-KEM. This gap prompts our focus on optimizing ML-KEM using AVX-512 and seamlessly integrating the optimized ML-KEM implementation into TLS 1.3. Besides, in USENIX Security 2022, Bernstein proposed a batch key generation tailored for `sntrup761` [11]. We also consider if there are other batch key generation methods for ML-KEM and other PQC algorithms.

As of now, existing research on PQ-TLS migration relies on OQS-OpenSSL, which lacks support for OpenSSL3 and remains outdated. However, the latest OQS provider [9] not only supports OpenSSL3 but also facilitates a clear separation between OpenSSL code and PQ-KEM code. Our comprehensive integration of ML-KEM using the OQS provider could provide valuable guidance for researchers seeking to migrate to PQ-TLS.

In addition, recent studies [29, 31, 44, 45] have affirmed the sufficiency of IND-1-CCA KEMs for TLS 1.3 handshake to be secure. Intriguingly, IND-1-CCA KEMs can be obtained from any OW-CPA/IND-CPA KEMs without re-encryption and de-randomization [29, 31] that are required by IND-CCA KEMs used in PQ-TLS previously. An idea can be easily deduced that TLS 1.3 handshake might demonstrate improved efficiency by applying such IND-1-CCA KEMs. However, there remains a notable absence of experiments focusing on IND-1-CCA-security KEM TLS 1.3. This gap in research catalyzes us to conduct experiments and evaluations on IND-1-CCA-secure PQ-TLS handshake protocols.

Contributions. In this work, we aim to bridge the gap between PQC engineering implementations and TLS protocol applications. We approach this task from both an optimization engineering perspective and a TLS system perspective.

- We present an optimized implementation of ML-KEM using AVX-512. We accelerate bottlenecks in ML-KEM by implementing 32-way parallel polynomial multiplication and 8-way hash function. Besides, we enhance polynomial sampling through the new features of AVX-512 such as masked registers and compressive store instructions. Our implementation successfully passes NIST’s KAT tests [6], achieving a $1.64\times$ speedup compared to the state-of-the-art AVX2 implementation of ML-KEM.
- We propose a novel batch key generation method for ML-KEM to batch 8 independent key pairs. Our batch key generation method achieves a speedup of $3.5\times$ to $4.9\times$ compared to key generation without batching. Compared to batch key generation in [11] which is only suitable for `sntrup761`, our batch generation approach can also be applied to batch not only ML-KEM but other PQC key generation involving hash function calls.
- We integrate the AVX-512 optimized implementation of ML-KEM into TLS 1.3, assess its impact on TLS 1.3 handshake time and evaluate the influence

of different KEM constructions on TLS handshake efficiency. Our evaluation reveals that an efficient implementation of ML-KEM utilizing AVX-512 can yield a higher number of handshakes per second compared to the latest AVX2 implementation.

- We revisit two IND-1-CCA KEM constructions discussed in Eurocrypt’22 [29] and Asiacrypt’23 [31], and implement them with the underlying CPA-secure public key encryption (PKE) of ML-KEM. We then evaluate the performance of IND-1-CCA KEMs and integrate the better one into TLS 1.3. The benchmark results indicate that IND-1-CCA KEMs improve the performance of the TLS 1.3 handshake compared to IND-CCA KEMs.

2 Preliminaries

2.1 Notation

The notation in this paper is the same as the FIPS 203 draft [52]. We denote $\mathcal{R}_q$ as the cyclic polynomial ring $\mathbb{Z}_q[x]/(x^n + 1)$. We define $r' = r \bmod^{\pm} \alpha$ (resp. $r' = r \bmod \alpha$) to be the unique element r' in the range $-\lfloor \frac{\alpha}{2} \rfloor < r' \leq \lfloor \frac{\alpha}{2} \rfloor$ (resp. $0 \leq r' < \alpha$) such that $\alpha | (r - r')$. By default, regular font letters denote elements in $\mathcal{R}_q$, bold lower-case letters are column vectors and bold upper-case letters are matrices.

2.2 ML-KEM

ML-KEM is a NIST-standardized lattice-based KEM. Its security is based on the Module Learning With Errors (M-LWE) problem. ML-KEM is derived from NIST Round 3 version of Kyber [8]. The polynomial multiplication over $\mathbb{Z}_{3329}[x]/(x^{256} + 1)$ is a fundamental operation in ML-KEM. Utilizing the property $n|(q - 1)$, ML-KEM employs an incomplete Number Theoretic Transform (NTT) to accelerate this operation. ML-KEM uses four SHA-3 hash functions: SHA3-256, SHA3-512, SHAKE128, and SHAKE256. For more details, readers can refer to FIPS 203 draft [52].

Number Theoretic Transform. NTT is a variant of Fast Fourier Transform (FFT) in finite fields. Its essence is to use the point-value representation of polynomials to perform efficient polynomial multiplication operations. We denote the forward NTT transform as NTT, and the inverse NTT as INTT. The symbol “ $\cdot$ ” denotes point-wise multiplication. Polynomial multiplication $h(x) = f(x) \times g(x) \in \mathbb{Z}_q[x]/(x^n + 1)$ can be computed as follows:

$$h(x) = f(x) \times g(x) = \text{INTT}(\text{NTT}(f) \cdot \text{NTT}(g)).$$

For cyclic polynomial ring $\mathbb{Z}_q[x]/(x^n + 1)$, complete NTT requires $2n|(q - 1)$, which is not satisfied in ML-KEM parameters. Instead, ML-KEM uses a variant of NTT, which deletes the last layer of the NTT. For this type of NTT that follows the “bottom cropping” method, we call it T-NTT (truncated NTT) for

short and let β be the number of truncated layers. ML-KEM uses the case of $\beta = 1$ for T-NTT. Given $\mathbb{Z}_q[x]/(x^n + 1)$, for any integer $\beta \geq 0$, q is a prime satisfies $\frac{n}{2^{\beta-1}} \mid (q-1)$, T-NTT(f) has the general form:

$$\mathbb{Z}_q[x]/(x^n + 1) \cong \prod_{i=0}^{\frac{n}{2^\beta}-1} \mathbb{Z}_q[x]/\left(x^{2^\beta} - \omega_{\frac{2n}{2^\beta}}^{2 \cdot br_{n/2^\beta}(i)+1}\right)$$

where $\omega_{2n/2^\beta}$ denotes the $\frac{2n}{2^\beta}$ -th root of unity, $br_{n/2^\beta}(i)$ denotes the bit-reversal permutation of $\{0, 1, \dots, \frac{n}{2^\beta} - 1\}$.

Through the T-NTT transformation, the n -dimensional polynomials f, g are separately decomposed into $\frac{n}{2}$ linear polynomials. Then the original multiplication $f \times g$ is transformed into $\frac{n}{2}$ multiplications of linear polynomials.

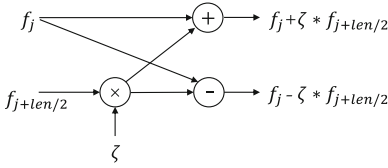


Fig. 1. Cooley-Tukey Butterfly.

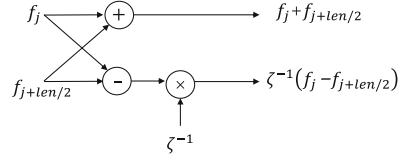


Fig. 2. Gentleman-Sande Butterfly.

Cooley-Tukey transform [19] and Gentleman-Sande transform [26] are used in NTT and INTT respectively, shown in Fig. 1 and Fig. 2.

2.3 AVX-512 Instruction Set

The AVX-512 instruction set was introduced by Intel in 2013 and initially supported in the 2016 Xeon Phi series processors [2], AVX-512 has since become an important feature for both major CPU manufacturers. AVX-512 offers several key functionalities: it introduces 32 512-bit **zmm** registers, enabling simultaneous processing of multiple data elements and vectorized computations. AVX-512 covers a wide range of floating-point and integer operations for diverse computational requirements. Additionally, AVX-512 provides 8 mask registers (**k0-k7**) for conditional operations. These mask registers enhance the flexibility and efficiency of data processing by enabling advanced operations such as compression/-expansion and masking.

2.4 TLS 1.3 and PQ-TLS

TLS is a standard developed by the Internet Engineering Task Force (IETF) in 1999. Its primary role is to encrypt communication between web applications and servers, with the latest version being TLS 1.3 [42]. When initiating a TLS

connection, the client and server use key exchange to negotiate a shared key and use the signature algorithm to authenticate the identity of the entities. Currently, the signature and key exchange algorithms used by TLS are not resistant to quantum computer attacks. In the quantum era, we need to replace the traditional algorithms in TLS with PQC, that is PQ-TLS. PQ-TLS operates in two modes: PQ-hybrid mode and PQ-only mode. The PQ-hybrid mode, as standardized by the IETF in TLS 1.3 [1], supports simultaneous usage of ECDH and PQ-KEMs, albeit at the cost of increased data transmission size and computational resources. Conversely, the PQ-only mode exclusively employs PQ-KEMs for key exchange (KEX) and PQC signature algorithms for authentication.

3 Revisiting PQC Security of KEM in TLS 1.3

Recent works [22, 29, 31, 44] state that the IND-CCA KEM used in TLS 1.3 handshake can be replaced by an IND-1-CCA KEM, providing improved efficiency and sufficient security. This idea inspires us to construct IND-1-CCA KEMs with the underlying PKE of ML-KEM, and evaluate their performance in TLS 1.3. We revisit the theoretical analysis of using IND-1-CCA KEM in TLS 1.3, as well as the detailed description of the IND-1-CCA constructions in the following sections.

3.1 An Efficient Choice of PQ-Secure KEX: IND-1-CCA KEM

In existing PQ-TLS implementations, the ephemeral KEX is implemented with IND-CCA KEMs. IND-CCA KEMs are usually constructed by applying Fujisaki-Okamoto (FO) transform or its variants (e.g. ML-KEM [52]) on an OW/IND-CPA PKE scheme, while FO transform requires re-encrypting the plaintext during decapsulation, significantly reducing the efficiency of KEMs and increasing the cost of side-channel protection [34].

Recent protocols (e.g. TLS 1.3 [22] and KEMTLS [44]) are designed to achieve forward security. In such protocols, each pair of ephemeral public/private keys is discarded immediately after being used once, and a new key pair will be generated for new messages. This means that an adversary will be able to request a decryption only once for a given key pair. Informally, IND-1-CCA security states that an adversary needs to distinguish an honestly generated key from a randomly generated key with at most one decapsulation query. Thus, the IND-1-CCA security of KEMs is sufficient to replace the Diffie-Hellman (DH) key exchange, ensuring the security of such protocols. In the security proof of TLS 1.3 handshake under the multi-stage model given in [22], the DH key exchange could be replaced by an IND-1-CCA KEM and the proof would still hold. This idea inspired a series of work, see [29, 31, 44, 45] for details.

[29] proposed that an IND- q -CCA-secure KEM could be obtained from any passively secure PKE (OW-CPA/IND-CPA) without re-encryption. Specifically, [29] presented two constructions named T_{CH} and T_H , as well as their security proofs. Both T_{CH} and T_H do not require re-encryption and de-randomization,

and such IND-1-CCA KEMs could be used in TLS 1.3 handshake, improving the efficiency while ensuring security. However, T_{CH} leads to ciphertext expansion and the security of T_H was only proved in ROM, the QROM proof was not provided. Based on [29,31] provided both ROM and QROM proofs of T_H and T_{RH} (an implicit variant of T_H), with much tighter reductions than [29].

3.2 IND-1-CCA KEM Constructions

In this work, we apply T_{CH} and T_H/T_{RH} to the underlying PKE of ML-KEM respectively, obtain different IND-1-CCA KEMs, and evaluate their performance. Here we review the definition of T_{CH} and T_H/T_{RH} , as well as their reduction tightness in ROM/QROM.

Constructions. The idea of T_{CH} is to send an additional hash value along with ciphertext, which will be used for key confirmation in decapsulation (see Fig. 3). T_H works without additional hash value, the key is derived by $H(m, c)$ directly (Fig. 4). T_{RH} is an implicit variant of T_H , the only difference between T_{RH} and T_H is the return value for invalid decryption.

<u>Gen()</u>	<u>Encaps(pk)</u>	<u>Decaps(sk, ct)</u>
01 $(pk, sk) \leftarrow \mathbf{Gen}^{\text{pke}}()$	01 $m \xleftarrow{\$} \mathcal{M}$	01 $(c', tag') \leftarrow ct$
02 return (pk, sk)	02 $c \leftarrow \mathbf{Enc}^{\text{pke}}(pk, m)$	02 $m' := \mathbf{Dec}^{\text{pke}}(sk, c')$
	03 $tag := H'(m, c)$	03 if $H'(m', c') \neq tag'$
	04 $K := H(m)$	04 return $\perp$
	05 return $(K, ct := (c, tag))$	05 else return $K := H(m')$

Fig. 3. T_{CH} transform.

<u>Gen()</u>	<u>Encaps(pk)</u>	<u>Decaps(sk, c)</u>
01 $(pk, sk) \leftarrow \mathbf{Gen}^{\text{pke}}()$	01 $m \xleftarrow{\$} \mathcal{M}$	01 $m' := \mathbf{Dec}^{\text{pke}}(sk, c)$
02 return (pk, sk)	02 $c \leftarrow \mathbf{Enc}^{\text{pke}}(pk, m)$	02 if $m' = \perp$
	03 $K := H(m, c)$	03 return $\perp$ // T_H
	04 return (K, c)	04 return $K := H(*, c)$ // T_{RH}
		05 else return $K := H(m', c)$

Fig. 4. T_H/T_{RH} transform.

Reduction Tightness. The security of T_{CH} was proved in ROM and QROM by Huguenin-Dumittan and Vaudenay [29], with tightness $\epsilon_R \approx O(1/q_H)\epsilon_A$ and $\epsilon_R \approx O(1/q_H^3)\epsilon_A^2$ respectively, where ϵ_R (resp. ϵ_A) is the advantage of the

reduction R (resp. adversary $\mathcal{A}$) against the underlying PKE (resp. the IND-1-CCA KEM) and q_H denotes the number of $\mathcal{A}$'s queries to H . Later, they updated an ePrint version of [29] and gave a tighter QROM proof of T_{CH} with bound $\epsilon_R \approx O(1/q_H^2)\epsilon_{\mathcal{A}}^2 - O(q_H^3/2^n) - O(q_H/\sqrt{2^n})$. The ROM and QROM proof of T_H and T_{RH} were presented in [31], with tightness $\epsilon_R \approx O(1/q_H)\epsilon_{\mathcal{A}}$ and $\epsilon_R \approx O(1/q_H^2)\epsilon_{\mathcal{A}}^2$ respectively. The comparison between FO transform and $T_{CH}/T_H/T_{RH}$ is summarized in Table 1.

Table 1. Comparison between FO transform and $T_{CH}/T_H/T_{RH}$.

Construction	Security	ϵ -Expansion	Re-Enc	ROM tightness	QROM tightness
FO	IND-CPA $\Rightarrow$ IND-CCA	$\ominus$	$\bullet$	$\epsilon_R \approx \epsilon_{\mathcal{A}}$	$\epsilon_R \approx O(1/q_H)\epsilon_{\mathcal{A}}^2$
T_{CH}	OW-CPA $\Rightarrow$ IND-1-CCA	$\bullet$	$\ominus$	$\epsilon_R \approx O(1/q_H)\epsilon_{\mathcal{A}}$	$\epsilon_R \approx O(1/q_H^2)\epsilon_{\mathcal{A}}^2$
T_H/T_{RH}	IND-CPA $\Rightarrow$ IND-1-CCA	$\ominus$	$\ominus$	$\epsilon_R \approx O(1/q_H)\epsilon_{\mathcal{A}}$	$\epsilon_R \approx O(1/q_H^2)\epsilon_{\mathcal{A}}^2$

4 ML-KEM AVX-512 Implementation

The most time-consuming operations in ML-KEM include modular reduction, polynomial sampling, polynomial multiplication, and hash functions. In this section, we will outline our rationale behind the AVX-512 implementation design for these computationally intensive components. Our AVX-512 optimization techniques for ML-KEM mainly contain reasonable register allocation, layer merging, coefficients permutation, and mask register utilization.

4.1 Modular Reduction Implementation

Modular reduction plays a crucial role in ML-KEM polynomial arithmetic, where the arithmetic operates over the ring $\mathbb{Z}_q$ with $q = 3329$. However, AVX-512 does not have constant-time instructions that can directly perform modulo operations. To prevent the leakage of secret information, we need to use constant-time modular reduction algorithms to make the coefficients within the $\mathbb{Z}_q$ range. Commonly used modular reduction algorithms include the Montgomery reduction [35] and the Barrett reduction [10]. The core idea of implementing modular reduction using AVX-512 is to optimize parallelism and minimize instruction overhead.

Reduction Algorithms. The Signed Montgomery reduction computes the Hensel remainder of a signed integer within the range $[-\beta q/2, \beta q/2)$, specifically employed in ML-KEM to reduce the product of two 16-bit coefficients. The output remainder integer resides in the Montgomery domain which is further multiplied by β^{-1} . Barrett reduction operates within a smaller input range compared to Montgomery reduction. Typically, the input to Barrett reduction does not exceed a 16-bit signed integer range. Therefore, Barrett reduction is commonly utilized to reduce coefficients that surpass the range $[-q, q]$ after addition

and subtraction operations. The resulting output from Barrett reduction remains within $\mathbb{Z}_q$.

AVX-512 Implementations. We give our AVX-512 implementations of signed Montgomery reduction and Barrett reduction. For the modulus $q = 3329$, we set $\beta = 2^{16}$. We store coefficients in a 16-bit data lane for a 512-bit vector register to achieve 32-way parallelism. Given that v remains constant in Barrett reduction [46, Sec 3.3], we can precompute v and store it in the vector register. Regarding the computation of $av/2^\beta$, it involves two operations: multiplication and division. These two operations can be executed using one AVX-512 instruction `vpmulhw`, which computes the product of every two 16-bit data lanes in vector registers and retains only the higher 16 bits. Additionally, the division by $2^{\lfloor \log(q) \rfloor - 1}$ can be efficiently implemented using AVX-512 shift instructions. In Algorithm 1, we introduce a macro `red16` designed to compute Barrett reduction using AVX-512 instructions. Within this macro, the `zmm\r` register holds the output of the Barrett reduction, while `zmm\rv` stores the constant used in Barrett reduction, represented by $v = \left\lfloor \frac{2^{\lfloor \log(3329) \rfloor - 1} 2^{16}}{3329} \right\rfloor = 20159$. Additionally, the `zmm\r1` register stores an immediate value. For Montgomery reduction, we implement it with butterfly operation in Sect. 4.2.

Algorithm 1. Signed Barrett reduction AVX-512 code

```

1: .macro red16 r, rv, r1
2: vpmulhw %zmm\rv, %zmm\r, %zmm\r1
3: vpsraw $10, %zmm\r1, %zmm\r1
4: vpsubw %zmm\r1, %zmm\r, %zmm\r
5: .endm

```

4.2 NTT Implementation

NTT is one of the most challenging components in the ML-KEM AVX-512 implementation. Both NTT and INTT operations in ML-KEM require 7 layers of butterfly operations. In NTT implementation, our main consideration is how to merge more layers and how to allocate registers properly.

Register Allocation. A 512-bit vector register `zmm` can accommodate a maximum of 32 16-bit integers, requiring only 8 vector registers to store all 256 polynomial coefficients. Therefore, we allocate 8 vector registers for storing coefficients, 2 for intermediate registers, and 2 for constant registers, leaving 20 vector registers unused.

Algorithm 2. Cooley-Tukey butterfly AVX-512 code

```

1: .macro Cooley-Tukey butterfly l,r,zl,zh
2: vpmullw %zmm\zl,%zmm\r,%zmm11
3: vpmulhw %zmm\zh,%zmm\r,%zmm\r
4: vpmulhw %zmm0,%zmm11,%zmm11
5: vpsubw %zmm11,%zmm\l,%zmm\r
6: vpaddw %zmm11,%zmm\l,%zmm\l
7: .endm

```

Algorithm 3. Gentleman-Sande butterfly AVX-512 code

```

1: .macro Gentleman-Sande butterfly l,r,zl,zh
2: vpsubw %zmm\l,%zmm\r,%zmm21
3: vpaddw %zmm\r,%zmm\l,%zmm\l
4: vpmulhw %zmm\zh,%zmm21,%zmm22
5: vpmulhw %zmm\zh,%zmm21,%zmm22
6: vpmulhw %zmm0,%zmm21,%zmm21
7: vpsubw %zmm21,%zmm22,%zmm\r
8: .endm

```

Butterfly Unit. As mentioned in Sect. 2.2, Cooley-Tukey butterfly operation computes the product of coefficients and twiddle factors, then performs addition and subtraction. Montgomery reduction is involved during product computation. Our Cooley-Tukey butterfly code is outlined in Algorithm 2. Registers `zmm\l` and `zmm\r` respectively store coefficients f_i and $f_{j+len/2}$, while `zmm0` holds the modulus q . We precompute $q^{-1} \bmod^{\pm\beta}$ into twiddle factors to reduce one multiplication operation in Montgomery reduction. To obtain the low and high bits of the 16-bit integer multiplication product, we utilize vector integer multiplication instructions `vpmullw` and `vpmulhw`. These instructions eliminate the necessity of extending coefficients to 32 bits after multiplication. Our Gentleman-Sande Butterfly code is presented in Algorithm 3. Similar to the Cooley-Tukey butterfly, registers `zmm\l` and `zmm\r` store coefficients f_i and $f_{j+len/2}$, respectively. Additionally, registers `zmm\zl` and `zmm\zh` hold precomputed $\zeta \cdot q^{-1}$ and ζ values, respectively. The order of twiddle factors in the NTT differs from that in the INTT, thus the same twiddle factors can be used. However, employing the same twiddle factors table requires additional permutation. In our implementation, we opt to utilize two distinct twiddle factors tables for the NTT and INTT.

In the implementations of NTT and INTT, we employ layer merging and coefficient permutation methods to reduce the memory access overhead. Specifically, we merge the 7 layers of NTT. We load all coefficients into 8 vector registers only at the first layer and store them back in memory after completing the final layer computation. Achieving such layer merging incurs some additional overhead for the coefficient permutation. Since each vector register accommodates 32 coefficients, in the initial three layers of the NTT, the coefficients stored in vector registers satisfy the correct distance and can directly perform the Cooley-Tukey butterfly. Starting from the fourth layer, as the butterfly distance becomes 16,

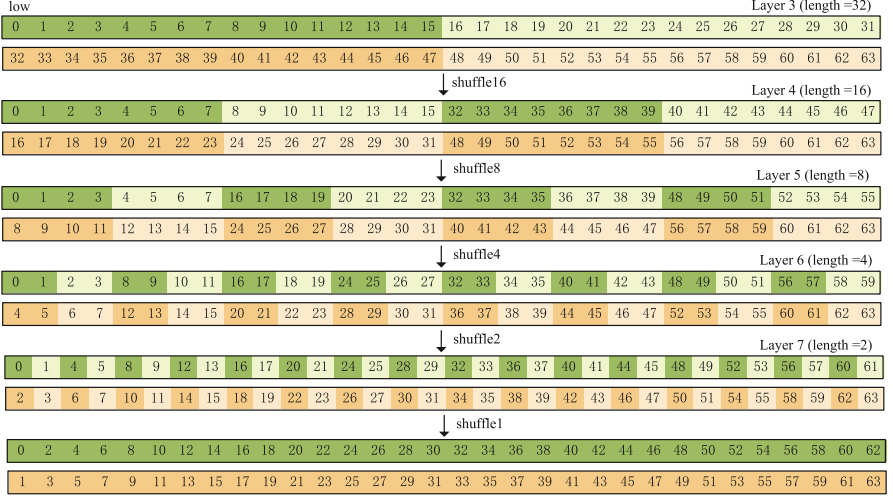


Fig. 5. Coefficients permutation.

the coefficient pairs requiring butterfly operations are housed within the same vector register. Thus, it's necessary to separate the coefficient pairs at corresponding distances into different vector registers.

As depicted in Fig. 5, we can observe the arrangement of coefficients within a vector register during the third to seventh layers of the NTT. The functions `Shuffle16`, `Shuffle8`, `Shuffle4`, `Shuffle2`, and `Shuffle1` are utilized for permuting pairs of 16, 8, 4, 2, and 1 coefficients, respectively. We perform an extra `Shuffle1` to bring convenience for polynomial point-wise multiplication. After completing all NTT layers, the coefficients are constrained within the range of signed 16-bit integers, eliminating the need for Barrett reduction in NTT. However, this differs in the case of INTT. We adopt lazy reduction in INTT as outlined in [55].

4.3 SHA3 Keccak

ML-KEM utilizes SHA3-256, SHA3-512, SHAKE128, and SHAKE256 as hash functions, pseudorandom functions (PRFs), and eXtendable-output functions (XOFs). These algorithms belong to the SHA-3 family [24] developed by NIST and are based on the Sponge Construction [12]. The round function employed in the SHA-3 algorithm is the Keccak-p[1600,24] permutation function, which operates on a 64-bit state. In the previous AVX2 implementation of ML-KEM by the Crystals team, Keccak-p [24] achieved 4-way parallelism due to the 256-bit vector register width. However, with AVX-512 vector registers, we can compute 8 Keccak permutation results in parallel, as a 512-bit vector register can process 8 Keccak states concurrently.

4.4 Other Modules Implementation

We also implement Compress and Decompress, pkDecode, skDecode, skEncode and pkEncode and several polynomial arithmetic functions. The implementation idea is similar to the AVX2 implementation. For simplicity, we don't discuss details here. One notable difference is that, in the implementation of pkDecode, skDecode, and Decompress, we utilize the AVX-512 instruction `vpermb`, which operates on bytes, unlike AVX2 instructions. This instruction allows us to conveniently adjust the order of byte streams. We implement rejection sampling using AVX-512 according to the method presented in [57].

5 ML-KEM TLS 1.3 Integration Design Consideration

5.1 Batch Key Generation Using Parallel Keccak

We explore the potential applicability of batch key generation in other PQC algorithms. Drawing inspiration from [43], we propose a novel batch key generation approach for ML-KEM, as outlined in Algorithm 4. The SHA3-256x8 function is constructed based on the 8-way Keccak function discussed in Sect. 4.3. Our 8-way ML-KEM key generation AVX-512 implementation function is designed to generate 8 independent (pk, sk) pairs simultaneously.

Differences From Previous Batch Methods. There are two batch methods proposed in previous research [11, 56]. Our main difference from [11] is that there is no polynomial inversion in ML-KEM, so using Montgomery's trick to batch key is unfeasible, instead we try to batch key generation using AVX-512 parallelism. Our main idea is similar to [56] that uses AVX-512 parallelism to batch key. But the specific deployment method is different, we use parallel 8-way hash functions while they use 8-way finite field arithmetic.

Algorithm 4. Batch key generation for ML-KEM.KEM

Ensure: Encapsulation keys array $ek[8] \in \mathbb{B}^{384k+32}$.

Ensure: Decapsulation keys array $dk[8] \in \mathbb{B}^{768k+96}$.

```

1:  $z \xleftarrow{\$} \mathbb{B}^{32}$ 
2: for  $i = 0$  to 7 do
3:    $(ek_{PKE}[i], dk_{PKE}[i]) \leftarrow K\text{-}PKE.\text{KeyGen}()$ 
4:    $ek[i] \leftarrow ek_{PKE}[i]$ 
5: end for
6: output array  $output[8] \leftarrow \text{SHA3-256}\times 8(ek[0], ek[1], ek[2], ek[3], ek[4], ek[5], ek[6], ek[7])$ 
7: for  $i = 0$  to 7 do
8:    $(dk[i] \leftarrow (dk_{PKE} || ek || output[i]) || z)$ 
9: end for
10: Return  $(ek[8], dk[8])$ 
```

5.2 ML-KEM AVX-512 TLS 1.3 Migration Implementation

Integrating the PQC algorithm into TLS 1.3 presents several challenges: Firstly, the PQC message format must align with TLS 1.3 standards while maintaining forward secrecy. Secondly, PQ-TLS must support PQC algorithms while remaining compatible with traditional TLS protocols. Finally, it is crucial to ensure that the handshake performance of PQ-TLS is comparable to or better than traditional TLS algorithms. Notably, there are mature open-source projects such as those from the Open Quantum Safe (OQS) team [37]. We utilize the OQS provider OpenSSL 3.3.0-dev for migrating the ML-KEM AVX-512 implementation. OpenSSL is an open-source software library implementing the SSL and TLS protocols. Most PQ-TLS research works use OQS-OpenSSL to migrate PQC algorithms into TLS 1.3. However, OQS-OpenSSL ceased updates in July 2023 and does not support the latest OpenSSL 3.0 version. The latest OQS provider separates the integration of PQ algorithms into TLS 1.3 from the main logic of OpenSSL, without altering the core cryptographic algorithms. This separation isolates the embedding of post-quantum algorithms from the extensive OpenSSL codebase. For ML-KEM AVX-512 code migration, we choose liboqs 0.10.1 [21]. Liboqs is an open-source C library for PQC algorithms. The newest liboqs 0.10.1 version adds ML-DSA and ML-KEM C reference and AVX2 codes. We add our ML-KEM AVX-512 code in the `ml_kem` directory of liboqs. Besides, we implement the corresponding ML-KEM AVX-512 Keygen, Encaps, and Decaps APIs in liboqs. We define the macro `OQS_ENABLE_KEM_ml_kem_512_avx512` in the OQS configuration file. By configuring this macro, users can run ML-KEM-512 AVX-512 code within the liboqs library.

6 Discussions and Results

6.1 Experimental Setup

We used an emulation experiment setup in one machine. The benchmark platform is an Intel Xeon Platinum 8475B with 4 vCPUs and 8 GiB memory¹. The OpenSSL version we used is OpenSSL 3.3.0-dev. We used GCC 9.4.0 to compile all programs. We disabled AVX-512 instructions when benchmarking ML-KEM AVX2 implementation for better comparison. We didn't disable Turbo Boost and hyper-threading to simulate real-world conditions during TLS handshake.

6.2 Speed of ML-KEM AVX-512 Implementation

We conducted a performance evaluation of our ML-KEM AVX-512 implementations. Our approach is built upon the Crystals team's open-source Code [5],

¹ Our testing machine is provided by Alibaba Cloud, belonging to the compute-optimized instance family c8i. It is powered by an Intel Xeon Sapphire Rapids processor, with the specific specification `ecs.c8i.xlarge`, featuring 4 vCPUs and 8 GiB of memory.

which was implemented for the FIPS 203 draft. To assess performance, we executed the speed testing program 100,000 times and recorded the median CPU cycle count. The results are summarized in Table 2, which compares various ML-KEM implementations across three security levels. Our implementation achieved a speedup ratio ranging from approximately $1.30\times$ to $1.64\times$ compared to the state-of-the-art AVX2 implementation. This performance enhancement can be attributed to the intricate design of our NTT AVX-512 implementation and the vectorization benefits offered by AVX-512 instructions. As illustrated in Table 2, adopting batch method in Sect. 5.1 can accelerate the key generation process by $3.5\times$ to $4.9\times$.

Table 2. ML-KEM implementation performance comparison. (CPU Cycles)

	ML-KEM-512			ML-KEM-768			ML-KEM-1024			ISA/ISE
	Keygen	Encaps	Decaps	Keygen	Encaps	Decaps	Keygen	Encaps	Decaps	
[5]	81750	90,922	118,790	130,048	142,688	178,206	217,174	223,780	274,588	x86-64
Speed-up	$6.21\times$	$6.68\times$	$7.80\times$	$6.61\times$	$7.61\times$	$8.36\times$	$8.61\times$	$8.93\times$	$9.52\times$	
[5]	17,750	18,708	19,838	28,996	28,566	30,476	39,996	41,162	44,538	AVX2
Speed-up	$1.35\times$	$1.37\times$	$1.30\times$	$1.47\times$	$1.52\times$	$1.43\times$	$1.59\times$	$1.64\times$	$1.54\times$	
This work	13,170	13,620	15,230	19,684	18,760	21,308	25,216	25,062	28,834	AVX-512
Batch Keygen	2,804 ($4.9\times$)	13,620	15,230	4,866 ($4.2\times$)	18,760	21,308	7,394 ($3.5\times$)	25,062	28,834	

6.3 ML-KEM AVX-512 TLS 1.3 Benchmark

We evaluated the performance of TLS 1.3 handshakes integrated with the ML-KEM AVX-512 implementation in a simulated network environment. Initially, we utilized OpenSSL commands to generate key pairs and server certificates. Subsequently, utilizing the previously generated keys and certificates, we launched a TLS server via the `openssl s_server` tool [3]. We assessed the handshake connections per second using the `openssl s_time` tool [4]. Both the server and client were compiled against OpenSSL3.3.0-dev with the OQS provider, thus facilitating support for both PQ-hybrid mode and PQ-only mode.

PQ-only ML-KEM PQ-TLS 1.3 Benchmark. In TLS, we mainly observe the ML-KEM algorithm for KEX. However, there are multiple signature algorithms available, each with different security levels. Therefore, observing the combination of different signature algorithms with ML-KEM is crucial and worth considering. For our experiments, we selected three signature algorithms: the standardized PQC signature algorithms Dilithium and Falcon, as well as the traditional public key cipher RSA2048.

Table 3. PQ-only ML-KEM Handshake performance comparison. (connections/sec)

KEM/SIG	ML-KEM-512		ML-KEM-768		ML-KEM-1024
	Dilithium2	Falcon512	RSA2048	Dilithium3	Dilithium5
AVX2	3416.02	3191.46	2582.38	2710.85	2392.01
AVX-512	3589.22	3364.61	2650.45	2856.71	2451.08

As shown in Table 3, under PQ-only setting, AVX-512 implementation of ML-KEM brings more handshakes per second compared to AVX2 implementation. This indicates that using AVX-512 to optimize ML-KEM performance can bring improvement to the PQ-TLS handshake. Besides, the handshake time decreases as the security level increases. Compared to RSA2048, using the Dilithium and Falcon post-quantum signature schemes for authentication does not impose significant performance overhead.

Hybrid ML-KEM PQ-TLS 1.3 Benchmark. OQS provider facilitates PQ-hybrid key exchange for TLS 1.3 by the IETF draft on post-quantum traditional hybrid Schemes [30]. Consequently, we also evaluated the performance of ML-KEM and other ECDH curves hybrid schemes. The data from Table 4 indicates a significant decrease in handshake performance in PQ-hybrid mode compared to the PQ-only mode. Furthermore, the enhancement in handshake performance achieved through the AVX-512 implementation is minimal.

Table 4. Hybrid ML-KEM handshake performance using RSA2048 for authentication. (connections/sec)

Hybrid KEM	ML-KEM-512		ML-KEM-768		ML-KEM-1024	
	p256	x25519	p384	x448	p521	p384
AVX2	893.98	1965.58	375.19	948.04	200.45	369.84
AVX-512	944.23	2031.37	377.72	999.85	201.66	371.39

Discussion of PQ-TLS Performance. In PQ-only setting, the improvement brought by ML-KEM AVX-512 implementation is subtle. As mentioned in Sect. 2.4, handshake time is not only determined by KEX time but also influenced by factors such as transmission time and authentication time. We cannot quantify the real improvement optimized ML-KEM implementation will bring to the PQ-TLS handshake performance in advance, but since ML-KEM is one of the core parts of the entire PQ-TLS, it is still necessary to optimize ML-KEM, and the optimized implementation of ML-KEM can also be used in other protocols and applications involving KEM. In PQ-hybrid mode, the handshake time is primarily composed of two components: ECDH and ML-KEM. Therefore, solely improving the performance of ML-KEM in PQ-hybrid mode is insufficient; there is also a need to enhance the performance of the ECDH scheme.

6.4 TLS 1.3 Handshake with IND-1-CCA KEM

In this section, we implemented IND-1-CCA KEMs based on T_{CH} and T_{RH} constructions proposed in [29,31], integrated the better one into TLS 1.3 and analyzing its performance.

Comparison of KEM Constructions. We constructed two IND-1-CCA KEMs based on the underlying PKE of ML-KEM, employing T_{CH} and T_{RH} respectively. Both T_{CH} and T_{RH} utilized SHA3-256 to instantiate the hash function H , consistent with the original FO transform in ML-KEM. Additionally, we fixed the length of the *tag* in T_{CH} to 32 bytes. The results are summarized in Table 5, which compare the execution speed and communication overhead across three constructions (ML-KEM’s original FO construction, T_{CH} and T_{RH}). From the experiment results, the following facts are concluded.

For key-generation and encapsulation, T_{CH} and T_{RH} show rational improvement in efficiency, mainly because they omit the hash of pk in key-generation, as well as the de-randomization in encapsulation. For decapsulation, T_{CH} and T_{RH} achieve a speed ratio of at least $3.04\times$ compared to FO implementation, which can be attributed to removing re-encryption. The improvement becomes more significant as the security level increases because higher security levels result in slower encryption. T_{CH} requires extra 32 bytes in ciphertext storage, and T_{CH} ’s decapsulation is slightly slower than T_{RH} due to the key confirmation. Overall, the KEM based on T_{RH} shows better performance in both efficiency and communication overhead.

Table 5. Performance comparison of different FO transform. (CPU cycles)

	ML-KEM-512				ML-KEM-768				ML-KEM-1024			
	Keygen	Encaps	Decaps	Ct	Keygen	Encaps	Decaps	Ct	Keygen	Encaps	Decaps	Ct
<i>FO</i>	17,750	18,708	19,838	768	28,996	28,566	30,476	1,088	39,996	41,162	44,538	1,568
<i>T_{CH}</i>	12,932	18,434	6,520	800	21,942	28,316	8,974	1,120	30,292	42,450	12,172	1,600
<i>T_{RH}</i>	12,896	17,676	5,666	768	21,942	27,580	8,440	1,088	30,336	40,096	11,240	1,568

Note : Ct represents Ciphertext.

IND-1-CCA KEM TLS 1.3 Benchmark. The T_{RH} -based IND-1-CCA KEM is integrated into TLS 1.3 in both PQ-only mode and PQ-hybrid mode. We measured the number of handshakes per second and compared the results with the original FO implementation, shown in Table 6.

The results show that T_{RH} -based IND-1-CCA KEMs increase the number of TLS 1.3 handshakes per second while maintaining communication overhead. Especially in the PQ-only mode with a higher security level, the removal of re-encryption brings more significant improvement, which is in line with our expectations.

Our experiments confirm the advantages of applying IND-1-CCA KEMs in TLS 1.3 at a practical level. Specifically, in application scenarios that place more

Table 6. ML-KEM handshake performance under different security. (connections/sec)

SIG	KEM	FO	T_{RH}
Dilithium2	ML-KEM-512	3416.02	3476.17
	p256_ML-KEM-512	924.61	978.12
	x25519_ML-KEM-512	2014.98	2038.19
Dilithium3	ML-KEM-768	2710.85	2803.24
	x25519_ML-KEM-768	1942.31	2036.58
	p256_ML-KEM-768	902.83	931.99

emphasis on handshake efficiency, IND-1-CCA KEMs have better adaptability than FO-based IND-CCA KEMs. These results inspire further research on IND-1-CCA constructions (e.g. tighter reduction) and their TLS 1.3 implementation.

7 Conclusion

In this paper, we present our implementation of ML-KEM using AVX-512 and introduce a novel batch method for ML-KEM key generation. With the support of OQS provider, we integrate our optimized ML-KEM AVX-512 implementation into TLS 1.3 and evaluate the performance in both PQ-only and PQ-hybrid modes. The vast majority of server-level processors support the AVX-512 instruction set, so our implementation can be used to accelerate the ML-KEM performance on server-level processors, as well as the corresponding PQ-TLS performance. Furthermore, we revisit two IND-1-CCA KEMs and analyze their impact on PQ-TLS handshake performance. Through our optimized implementation, integration, and assessment efforts, we provide valuable insights for future work aimed at enhancing PQ-TLS handshake performance.

References

1. Hybrid key exchange in TLS 1.3. <https://www.ietf.org/archive/id/draft-ietf-tls-hybrid-design-04.html>
2. Intel Xeon phi processor 7250 specifications. <https://www.intel.com/content/www/us/en/products/sku/94035/intel-xeon-phi-processor-7250-16gb-1-40-ghz-68-core/specifications.html>
3. OpenSSL: s_server- TLS/SSL server program. OpenSSL documentation (2022). https://www.openssl.org/docs/man3.3/man1/s_server.html
4. OpenSSL: s_time- SSL/TLS performance timing program. OpenSSL documentation (2022). https://www.openssl.org/docs/man3.3/man1/s_time.html
5. Kyber standard code (2024). <https://github.com/pq-crystals/kyber/tree/standard>. Accessed 27 March 2024
6. PQC - known answer tests and test vectors (2024). <http://csrc.nist.gov/groups/ST/post-quantum-crypto/example-files.html>. Accessed 21 June 2024

7. Abiega-L'Eglise, A.F.D., Delgado-Vargas, K.A., Valencia-Rodriguez, F.Q., Quiroga, V.G., Gallegos-García, G., Nakano-Miyatake, M.: Performance of new hope and crystals-dilithium postquantum schemes in the transport layer security protocol. *IEEE Access* **8**, 213968–213980 (2020). <https://doi.org/10.1109/ACCESS.2020.3040324>
8. Avanzi, R., et al.: Crystals-Kyber algorithm specifications and supporting documentation. NIST PQC Round **2**(4), 1–43 (2019)
9. Baentsch, M., Paquin, C., Levitte, R., Hess, B., Segeth, J.: oqsprovider - open quantum safe provider for openssl. Website (2023). <https://github.com/open-quantum-safe/oqs-provider>
10. Barrett, P.: Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor. In: Odlyzko, A.M. (ed.) *CRYPTO 1986*. LNCS, vol. 263, pp. 311–323. Springer, Heidelberg (1987). https://doi.org/10.1007/3-540-47721-7_24
11. Bernstein, D.J., Brumley, B.B., Chen, M., Tuveri, N.: OpenSSLNTRU: Faster post-quantum TLS key exchange. In: Butler, K.R.B., Thomas, K. (eds.) *31st USENIX Security Symposium, USENIX Security 2022*, Boston, MA, USA, 10–12 August 2022, pp. 845–862. USENIX Association (2022). <https://www.usenix.org/conference/usenixsecurity22/presentation/bernstein>
12. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: Sponge functions. In: *ECRYPT hash workshop*, vol. 2007 (2007)
13. Blog, G.S.: Experimenting with post-quantum cryptography (2016). <https://security.googleblog.com/2016/07/experimenting-with-post-quantum.html>. Accessed 18 June 2024
14. Bos, J.W., Costello, C., Naehrig, M., Stebila, D.: Post-quantum key exchange for the TLS protocol from the ring learning with errors problem. In: *2015 IEEE Symposium on Security and Privacy*, pp. 553–570. IEEE (2015)
15. Bozhko, J., Hanna, Y., Harrilal-Parchment, R., Tonyali, S., Akkaya, K.: Performance evaluation of quantum-resistant TLS for consumer IoT devices. In: *2023 IEEE 20th Consumer Communications & Networking Conference (CCNC)*, pp. 230–235. IEEE (2023)
16. Chang, Y.A., Chen, M.S., Wu, J.S., Yang, B.Y.: Postquantum SSL/TLS for embedded systems. In: *2014 IEEE 7th International Conference on Service-Oriented Computing and Applications*, pp. 266–270. IEEE (2014)
17. Cheng, H., Fotiadis, G., Groszschädl, J., Ryan, P.Y.: Highly vectorized SIKE for AVX-512. *IACR Trans. Cryptographic Hardware Embed. Syst. (TCHES)* **2022**(2), 41–68 (2022)
18. Cheng, H., Fotiadis, G., Groszschädl, J., Ryan, P.Y., Roenne, P.: Batching CSIDH group actions using AVX-512. *IACR Trans. Cryptographic Hardware Embed. Syst. (TCHES)* **2021**(4), 618–649 (2021)
19. Cooley, J.W., Tukey, J.W.: An algorithm for the machine calculation of complex fourier series. *Math. Comput.* **19**(90), 297–301 (1965)
20. Crockett, E., Paquin, C., Stebila, D.: Prototyping post-quantum and hybrid key exchange and authentication in TLS and SSH. In: *NIST 2nd PQC Standardization Conference*. Santa Barbara, California (2019). published elsewhere
21. Douglas Stebila, M.M.: liboqs project (2024). <https://github.com/open-quantum-safe/liboqs>
22. Dowling, B., Fischlin, M., Günther, F., Stebila, D.: A cryptographic analysis of the TLS 1.3 handshake protocol. *J. Cryptol.* **34**(4), 37 (2021)
23. Fan, J., et al.: Impact of post-quantum hybrid certificates on PKI, common libraries, and protocols. *Int. J. Secure. Network.* **16**(3), 200–211 (2021)

24. National Institute for Standards and Technology (NIST): Secure hash algorithm-3 (SHA-3) standard: Permutation-based hash and extendable-output functions. FIPS 202 (2014)
25. Garcia, C.R., Aguilar, A.C., Olmos, J.J.V., Monroy, I.T., Rommel, S.: Quantum-resistant TLS 1.3: A hybrid solution combining classical, quantum and post-quantum cryptography. In: 2023 IEEE 28th International Workshop on Computer Aided Modeling and Design of Communication Links and Networks (CAMAD), pp. 246–251. IEEE (2023)
26. Gentleman, W.M., Sande, G.: Fast fourier transforms: for fun and profit. In: Proceedings of the 7-10 November 1966, fall joint computer conference, pp. 563–578 (1966)
27. Gonzalez, R., Wiggers, T.: Kemtls vs. post-quantum TLS: performance on embedded systems. In: Batina, L., Picek, S., Mondal, M. (eds.) International Conference on Security, Privacy, and Applied Cryptography Engineering, vol. 13783, pp. 99–117. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-22829-2_6
28. Henrich, J., Heinemann, A., Wiesmaier, A., Schmitt, N.: Performance impact of PQC KEMs on TLS 1.3 under varying network characteristics. In: Athanasopoulos, E., Mennink, B. (eds.) International Conference on Information Security, vol. 14411, pp. 267–287. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-49187-0_14
29. Huguenin-Dumittan, L., Vaudenay, S.: On IND-qCCA security in the ROM and its applications - CPA security is sufficient for TLS 1.3. In: Dunkelman, O., Dziembowski, S. (eds.) Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part III. Lecture Notes in Computer Science, vol. 13277, pp. 613–642. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-07082-2_22
30. Force, I.E.T.: Hybrid Terminology for Post-Quantum Key Establishment. Tech. rep, Internet Engineering Task Force (2023)
31. Jiang, H., Ma, Z., Zhang, Z.: Post-quantum security of key encapsulation mechanism against CCA attacks with a single decapsulation query. In: Guo, J., Steinfeld, R. (eds.) International Conference on the Theory and Application of Cryptology and Information Security, vol. 14441, pp. 434–468. Springer, Cham (2023). https://doi.org/10.1007/978-981-99-8730-6_14
32. Kris Kwiatkowski, L.V.: The TLS post-quantum experiment (2019). <https://blog.cloudflare.com/the-tls-post-quantum-experiment>
33. Lei, D., He, D., Peng, C., Luo, M., Liu, Z., Huang, X.: Faster implementation of ideal lattice-based cryptography using avx512. ACM Trans. Embed. Comput. Syst. **22**(5), 1–18 (2023)
34. Melissa Azouaoui, J.W.B.: Surviving the Fo-Calypse: securing PQC implementations in practice (2022). <https://iacr.org/submit/files/slides/2022/rwc/rwc2022/48/slides.pdf>
35. Montgomery, P.L.: Modular multiplication without trial division. Math. Comput. **44**(170), 519–521 (1985)
36. National Institute of Standards and Technology: post-quantum cryptography standardization: Selected algorithms (2022). <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>
37. Norm Ashley, M.B.: Open quantum safe: software for the transition to quantum-resistant cryptography (2024). <https://openquantumsafe.org/>

38. Pablos, J.I.E., Marriaga, M.E., del Pozo, Á.L.P.: Design and implementation of a post-quantum group authenticated key exchange protocol with the libOQS library: a comparative performance analysis from classic McEliece, Dowling, NTRU, and saber. *IEEE Access* **10**, 120951–120983 (2022)
39. Paquin, C., Stebila, D., Tamvada, G.: Benchmarking post-quantum cryptography in TLS. In: Ding, J., Tillich, J.-P. (eds.) *PQCrypto 2020*. LNCS, vol. 12100, pp. 72–91. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-44223-1_5
40. Paul, S., Kuzovkova, Y., Lahr, N., Niederhagen, R.: Mixed certificate chains for the transition to post-quantum authentication in TLS 1.3. In: *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, pp. 727–740 (2022)
41. Paul, S., Schick, F., Seedorf, J.: TPM-based post-quantum cryptography: A case study on quantum-resistant and mutually authenticated TLS for IoT environments. In: *Proceedings of the 16th International Conference on Availability, Reliability and Security*, pp. 1–10 (2021)
42. Rescorla, E.: The transport layer security TLS protocol version 1.3. Tech. Rep. RFC 8446, RFC Editor (2018). <https://doi.org/10.17487/RFC8446>
43. Roy, S.S.: SaberX4: high-throughput software implementation of saber key encapsulation mechanism. In: *2019 IEEE 37th International Conference on Computer Design (ICCD)*, pp. 321–324. IEEE (2019)
44. Schwabe, P., Stebila, D., Wiggers, T.: Post-quantum TLS without handshake signatures. In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1461–1480 (2020)
45. Schwabe, P., Stebila, D., Wiggers, T.: More efficient post-quantum KEMTLS with pre-distributed public keys. In: Bertino, E., Shulman, H., Waidner, M. (eds.) *ESORICS 2021*. LNCS, vol. 12972, pp. 3–22. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-88418-5_1
46. Seiler, G.: Faster AVX2 optimized NTT multiplication for Ring-LWE lattice cryptography. *IACR Cryptol. ePrint Arch*, p. 39 (2018). <http://eprint.iacr.org/2018/039>
47. Shor, P.W.: Algorithms for quantum computation: discrete logarithms and factoring. In: *Proceedings 35th annual Symposium on Foundations of Computer Science*, pp. 124–134. IEEE (1994)
48. Sikeridis, D., Kampanakis, P., Devetsikiotis, M.: Assessing the overhead of post-quantum cryptography in TLS 1.3 and SSH. In: Han, D., Feldmann, A. (eds.) *CoNEXT 2020: The 16th International Conference on emerging Networking EXperiments and Technologies*, Barcelona, Spain, December, 2020, pp. 149–156. ACM (2020). <https://doi.org/10.1145/3386367.3431305>
49. Sikeridis, D., Kampanakis, P., Devetsikiotis, M.: Post-quantum authentication in TLS 1.3: A performance study. In: *27th Annual Network and Distributed System Security Symposium, NDSS 2020*, San Diego, California, USA, 23–26 February 2020. The Internet Society (2020). <https://www.ndss-symposium.org/ndss-paper/post-quantum-authentication-in-tls-1-3-a-performance-study/>
50. Sosnowski, M., et al.: The performance of post-quantum TLS 1.3. In: *Companion of the 19th International Conference on emerging Networking EXperiments and Technologies*, pp. 19–27 (2023)
51. National Institute of Standards and Technology: Module-lattice-based digital signature standard. Federal Information Processing Standards Publication (FIPS) NIST FIPS 204 IPD, Department of Commerce, Washington, D.C. (2023)

52. National Institute of Standards and Technology: Module-lattice-based key-encapsulation mechanism standard. Federal Information Processing Standards Publication (FIPS) NIST FIPS 203 IPD, Department of Commerce, Washington, D.C. (2023)
53. National Institute of Standards and Technology: Stateless hash-based digital signature standard. Federal Information Processing Standards Publication (FIPS) NIST FIPS 205 IPD, Department of Commerce, Washington, D.C. (2023)
54. Tasopoulos, G., Li, J., Fournaris, A.P., Zhao, R.K., Sakzad, A., Steinfeld, R.: Performance evaluation of post-quantum TLS 1.3 on resource-constrained embedded systems. In: Su, C., Gritzalis, D., Piuri, V. (eds.) International Conference on Information Security Practice and Experience, vol. 13620, pp. 432–451. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-21280-2_24
55. Westerbaan, B.: When to Barrett reduce in the inverse NTT. Cryptology ePrint Archive (2020)
56. Zhang, J., Huang, J., et al.: ENG25519: faster TLS 1.3 handshake using optimized X25519 and Ed25519. In: Usenix Security (2024)
57. Zheng, J., Zhu, H., Song, Z., Wang, Z., Zhao, Y.: Optimized vectorization implementation of crystals-dilithium. arXiv preprint [arXiv:2306.01989](https://arxiv.org/abs/2306.01989) (2023)



CRATOR a CRAWler for TOR: Turning Dark Web Pages into Open Source INTelligence

Daniel De Pascale¹(✉), Giuseppe Cascavilla², Damian A. Tamburri²,
and Willem Jan Van Den Heuvel¹

¹ Tilburg University - JADS, 's-Hertogenbosch, Netherlands
{d.de.pascale,w.j.a.m.v.d.heuvel}@jads.nl

² Eindhoven University of Technology - JADS, Eindhoven, Netherlands
{g.cascavilla,d.a.tamburri}@jads.nl

Abstract. Dark web crawling is a complex process that involves specific methodologies and techniques to navigate the Tor network and extract data from hidden services. This study proposes a dark web crawler designed to extract pages handling security protocols, such as CAPTCHAs, efficiently. Our approach uses a combination of seed URL lists, link analysis, and scanning to discover new content. We also incorporate methods for user-agent rotation and proxy usage to maintain anonymity and avoid detection. We evaluate the effectiveness of our crawler using metrics such as coverage, performance, and robustness. Our results demonstrate that our crawler effectively extracts pages handling security protocols while preserving anonymity and avoiding detection. Our proposed dark web crawler can be used for several applications, including threat intelligence, cybersecurity, and online investigations.

Keywords: Law Enforcement Agency · TOR · Dark Web · crawler · Open Source Intelligence

1 Introduction

The dark web is a part of the internet not indexed by search engines and requires special software, such as the Tor browser, to access [10,29]. It is often used for illegal activities, such as selling drugs, weapons, and stolen information. The dark web is often associated with illicit activities and is known to host various criminal markets, including illegal drug trading, weapons sales, and stolen data [8,12,19,26].

Dark web crawling, the act of systematically accessing and collecting data from the dark web, is a complex and challenging task. Unlike the surface web, the dark web lacks structure, making it difficult to discover and navigate its content. Additionally, the dark web's anonymity makes it challenging to verify the credibility and authenticity of the collected information.

One common objective of a dark web crawler is to gather information. Because the dark web is largely unregulated and anonymous, it can be a source of valuable data that is not easily accessible through other means. For example, a dark web crawler might be used to search for and collect information such as stolen user credentials, sensitive documents, or other confidential information that has been posted or traded on dark web marketplaces. Moreover, due to the illegal activities on the dark web, we can collect information to monitor illicit behaviors by looking for trends on specific drugs, armies, or criminal schemes. Law enforcement agencies (LEAs) may use dark web crawlers to monitor and gather intelligence on criminal activities such as drug trafficking, human trafficking, illegal weapon trade, and other forms of illicit behavior [7]. Law enforcement officials can anticipate and mitigate illicit phenomena or identify and seize criminal gangs by collecting data on these activities. Cybersecurity professionals and companies may also use dark web crawlers to investigate and predict potential cyber threats. For example, a dark web crawler could track the activities of hacking groups, monitor the spread of malware or ransomware, or detect data breaches posted online and specifically on the dark web. This information can then be used to strengthen cybersecurity defenses and protect against future attacks. An additional application of a dark web crawler involves examining the user behavior and the user activities or analyzing the criminal content available. This can furnish valuable insights into the underground market and the methodologies used for illicit activities on the internet.

The implementation of a dark web crawler includes several challenges. The dark web is designed to provide anonymity to users, making it challenging to track and identify the source of information [28]. A dark web crawler must be able to navigate this anonymity while still collecting relevant data. Moreover, differently from the surface web, the dark web is unstructured and lacks organization, making it challenging to discover and retrieve specific data.

We aim to address the challenges associated with crawling the dark web and provide researchers and practitioners with a reliable tool for exploring and analyzing dark web content. Hence, in this study, we propose the development of a crawler designed to navigate the complex structure of the dark web. Our crawler offers the capability to bypass basic security layers, such as login form pages with simple CAPTCHAs. It allows for the integration of multiple CAPTCHAs with real-time manual intervention by the user. The contribution of our work is twofold. Firstly, we propose an architecture specifically tailored for web crawling on the dark web, showcasing the integration of cookie rotation and user-driven manual intervention. Secondly, we have developed a general-purpose crawler for dark web crawling, making it publicly available for use.

To evaluate the performance and effectiveness of our crawler, we compared its performance against ACHE [2], a widely used dark web crawler. The comparison was based on various metrics used to evaluate the crawlers' capabilities [14, 27]. The results clearly indicate that our crawler outperformed ACHE in all the evaluated metrics.

The main contribution of our research is to provide a reliable, efficient, and robust crawler for the dark web that can handle pages with or without security controls, such as CAPTCHA, by using cookies, rotating them to avoid being detected, and identifying CAPTCHA page redirection. This capability allows the crawler to understand when it has been detected as a bot and to adapt its behavior accordingly to continue crawling. The contribution of our work is twofold. Firstly, we propose an architecture specifically tailored for web crawling on the dark web, showcasing the integration of cookie rotation and user-driven manual intervention. Secondly, we have developed a general-purpose crawler for dark web crawling, making it publicly available for use.

The structure of the paper is organized as follows. In Sect. 2, we discuss the related works on dark web analysis and dark web crawlers, providing insights into the existing research and its limitations. Section 3 presents the architecture of our crawler in detail, discussing each layer and its functionalities. Section 4 describes the evaluation process, including the metrics used, the dataset employed, and the intended experimentation. Section 5 presents our work’s results, highlighting our crawler’s performance and effectiveness compared to other approaches. In Sect. 7, we conclude the paper by summarizing the findings, discussing their implications, and outlining potential avenues for future research.

2 Related Work

Dark web crawlers are specialized software tools designed to index and collect information from hidden websites and services not accessible through conventional search engines. As the demand for more comprehensive and reliable intelligence on the dark web has increased, several researchers have attempted to develop effective dark web crawlers. In this section, we discuss some of the previous work done in the field of dark web crawling, with a particular focus on the findings of a systematic literature review made by Bergman et al. [3].

Bergman et al. conducted a systematic literature review to identify existing dark web crawlers and evaluate their effectiveness. During the review, they identified 34 potential dark web crawlers but found that only four of them had publicly available code repositories. The authors conclude the study by highlighting the absence of open-source crawlers, and that few researchers mention which open-source crawler was used in their study. Moreover, the experiments showed that the Tor crawler managed to scrape 251 pages in 20 min and eight minutes to scrape The Guardian’s clear website of 223 pages with an average of circa 6.4 pages.

Narayanan et al. present TorBot [25], an open-source intelligence tool for the dark web that enables users to gather and analyze information from hidden websites and services. TorBot is a Python-based tool that uses the Tor network to crawl the dark web and collect information such as website content, metadata, and links. It has several features, such as keyword-based searching, content analysis, and automated data extraction. The paper discusses the design and implementation of TorBot, including its architecture and components, and provides a detailed evaluation of its performance and capabilities.

Barbosa et al. introduce ACHE [2], an adaptive crawler designed to identify entry points to hidden web resources. The hidden web refers to resources that are not accessible through conventional search engines and require specific queries to access. The proposed crawler adapts to the structure of the hidden web by leveraging machine learning techniques to identify entry points and improve its crawling efficiency automatically. The algorithm uses a combination of content-based and link-based analysis to identify hidden web entry points. It continuously updates its search strategy based on the results of previous crawls. The paper describes the design and implementation of the adaptive crawler and evaluates its performance in locating hidden web entry points.

Kalpakis et al. [20] explores an interactive search engine for Home Made Explosives recipes on the surface and dark web. The hybrid architecture combines a dedicated crawler and domain-specific queries, adapting the widely used web crawler, Apache Nutch [21], to operate on the dark web. Similarly, Celestini et al. [9] address the need for effective web search and retrieval. It presents a flexible toolkit for structure and content mining on the Web, including the Tor dark Web. The toolkit incorporates web crawling, extraction, indexing, and mining modules. They have customized the well-established web crawler, bUbiNG [5], to effectively operate on the dark web. Their adaptation enables the crawler to navigate and retrieve information from the hidden layers of the internet. A more recent approach is available in Boshmaf et al. [6] that introduced Dizzy, a large-scale crawler for onion services designed to analyze and index dark web content. While Dizzy made significant contributions to the field, we did not consider it for our analysis due to its limitation in handling websites with login access, primarily because of the lack of cookie management. Furthermore, Dizzy’s repository was not shared publicly, preventing us from replicating and integrating their approach into our work. In contrast to the aforementioned works, our contribution addresses the limitations of availability and replicability in dark web crawling tools. We propose *CRATOR*, a robust and adaptive dark web crawler designed specifically for dark web marketplaces. Unlike ACHE and other existing tools, *CRATOR* integrates advanced techniques such as proxy usage, IP rotation, and cookie rotation to enhance crawling efficiency and minimize detection risks. These features ensure a higher success rate in downloading pages and maintaining consistent performance across various levels of depth, as demonstrated in our evaluation.

To conduct our comparative analysis, we have opted to re-run all the projects cited before. However, we faced limitations regarding the availability and suitability of certain crawlers. We excluded the TorBot crawler from our analysis due to outdated external dependencies, making it unusable for our purposes. Furthermore, the custom crawlers developed by Kalpakis and Celestini based on Apache Nutch and bUbiNG, respectively, were not publicly released, making their replication challenging. Moreover, neither the crawlers Nutch nor bUbiNG were specifically designed to work with the dark web. Therefore, for our experimentation, we used ACHE (GitHub repository here [1]) as a comparison tool

because it is currently the only available and replicable tool suitable for dark web crawling.

3 Design

3.1 Architecture

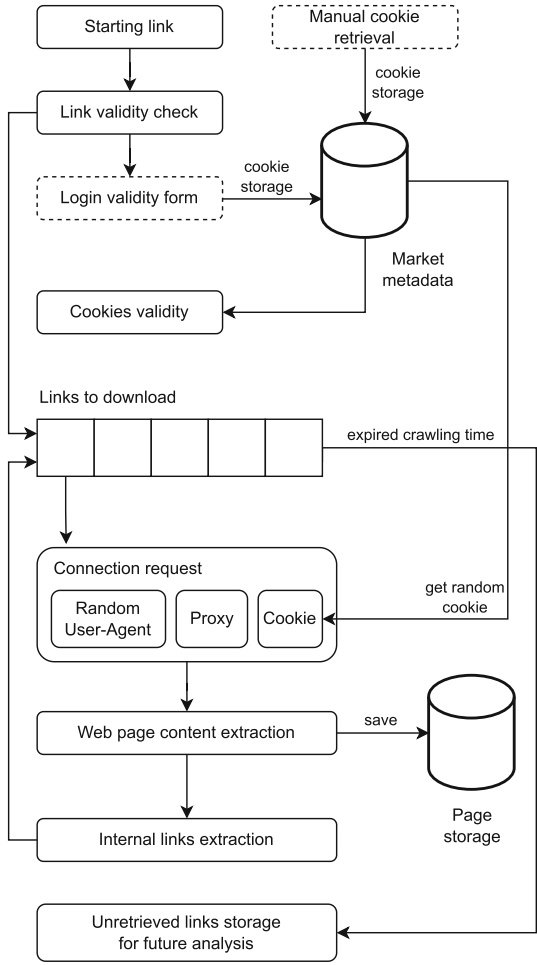


Fig. 1. Overview of CRATOR Architecture: steps for downloading pages from the dark web, including auto login, cookie management, and download queue, with the use of cookie rotation and random user agents.

A dark web crawler architecture typically consists of several components that work together to discover hidden web content. Figure 1 shows our dark

web crawler architecture, providing an overview of the entire crawling process, from the starting link until the content page storage process. The following is a general description of the key components of the architecture. Initially, the crawler receives a list of potential links, namely *starting links*, which the system can obtain from public dark web directories or directly as input from the user. Subsequently, the crawler performs a validity check to ensure that one of these input links is accessible. If a valid link is found, it becomes the starting point of the crawling process.

If a website employs a security check layer, such as a login page or CAPTCHA security check, the user must provide session cookies to log in correctly and open a user session. The architecture offers two methods for providing cookies to the crawler. The first method involves human intervention, where the user manually writes one or more cookies for the crawler to use during the crawling process. The second method involves the implementation of a login validity script, which tries to automatically log in and bypass the CAPTCHA on a dark web link by obtaining login credentials from a Market metadata file. In addition, the architecture incorporates a layer for verifying the validity of cookies, in which all collected cookies undergo validation by inspecting for any unexpected redirects. An unexpected redirect is defined as a redirection to a page other than the intended page, such as a response header containing a login link for a homepage link.

After the verification of link and cookie validity, the crawling process begins by downloading the first link along with all internal links, which are links sharing the same domain as the initial link, such as <https://en.wikipedia.org/wiki/IOT> and <https://en.wikipedia.org/wiki/Crawler>, both having the domain *en.wikipedia.org*. For each link, the crawler extracts and saves the webpage content locally and checks if there are other internal links to put in the download queue. The connection module makes use of proxy settings to establish a connection with .onion links, cookies to bypass security checks, and random user agents to avoid being identified as a bot. This process keeps running until it reaches a certain exit condition, such as a preset crawling time or a maximum number of links crawled.

3.2 Breadth-First Crawling Approach

Various strategies exist for web crawling, and choosing the right strategy is crucial for a crawler's success.

We decided to adopt a breadth-first crawling strategy because it provides more high-quality pages than other approaches, as proven by Cho et al. [11]. They use several metrics to define the concept of the “quality” of a page, like the number of links that appear during the crawling process or PageRank, a metric used to measure the importance or popularity of web pages based on their incoming links. With the breadth-first strategy, the crawler starts with a specific link and then visits all the links on that page before moving to the next page.

3.3 Link Validity Check

URL reachability check is an important process that ensures that the crawler can access and retrieve the content of a web page before attempting to extract information from it. This check is necessary because not all URLs that are identified for crawling are necessarily valid or accessible. URLs can become broken due to various reasons such as server errors, domain expiration, or page deletion, making them unreachable and causing errors in the crawling process [23, 24, 30].

URL reachability check involves sending a request to the URL or .onion link using the HTTP or HTTPS protocol and checking the response status code returned by the server. A response status code of 200 indicates that the URL is reachable and the server has successfully returned the page's content. On the other hand, a status code of 404 indicates that the page is not found, while a status code of 503 indicates that the server is temporarily unavailable.

If the crawler determines that a URL is not reachable, it marks it as broken or invalid and removes it from the crawling queue. This helps prevent unnecessary requests to unreachable URLs, which can slow down the crawling process and waste system resources.

In addition, the crawler may also check if the URL is a duplicate or a mirror of another page and avoid crawling such pages to prevent content duplication.

3.4 Login Forms and Cookies Rotation

In many cases, web crawlers need to login to a website to access restricted content. Automating the login process can save time and effort, especially when crawling multiple websites.

To perform an automatic login, a web crawler needs to submit login credentials to the website login page. This can be done using HTTP POST requests with the appropriate form data, such as the username and password. The response from the server indicates whether the login was successful or not.

Once the crawler has successfully logged in, it needs to maintain the session opened to access restricted content. This is usually done using cookies, which are small pieces of data stored on the client side and sent with each request to the server. The server uses the cookie to identify the user and maintain the session.

To avoid being detected as a crawler, it is important to rotate the cookies periodically. This can be done by logging out and logging back in again to obtain a new cookie or by using multiple cookies chosen with a fisher-yates shuffle algorithm. This helps to prevent the website from blocking the crawler or treating it as suspicious activity.

It is worth noting that some websites may have additional security measures in place, such as CAPTCHAs or two-factor authentication. These can make the automatic login process more challenging and may require additional coding or human intervention.

Our architecture offers the flexibility of managing cookies manually or automatically. This is achieved through an automated login script that logs into the starting link pages that require a username and password or by including a list of cookies in the market metadata file.

3.5 CAPTCHA Detection

CAPTCHA detection is an important aspect of web crawling, as many websites use CAPTCHAs to prevent automated bots from accessing their content. CAPTCHAs are usually presented as a challenge-response test [15,22], where the user is asked to perform a task that is easy for humans but difficult for machines, such as identifying distorted text, selecting images that match certain criteria, or solving a mathematical problem.

Detecting CAPTCHAs in a web crawler can be challenging, as they can appear in different forms and locations on the page and can be triggered by various factors, such as the number of requests, the frequency of requests, or the crawler's IP address. To detect CAPTCHAs, web crawlers need to analyze the page structure and content and look for patterns that indicate the presence of a CAPTCHA.

Crawlers employ several techniques to identify CAPTCHAs effectively. It examines the HTML structure and embedded JavaScript to detect common elements and scripts associated with CAPTCHA challenges, such as specific tags or script functions known to generate CAPTCHA challenges. The tool can also use image recognition algorithms to identify visual patterns typical of CAPTCHA images, such as distorted text or image grids. By combining these techniques, crawlers can robustly identify CAPTCHAs on web pages, allowing for more effective web crawling and data extraction.

3.6 Connection Setup

On the surface web, connection requests are made using the standard HTTP or HTTPS protocols. This involves sending a request to a web server and receiving a response that includes the requested content. The server may also set cookies or other tracking data that the client can store and use in future requests.

In contrast, the dark web operates on a different protocol called Onion Routing [16,18], which uses several layers of encryption to anonymize communication between clients and servers. To make connection requests on the dark web, the crawler must be configured to use the Tor network, which provides a layer of encryption and routing that obscures the origin and destination of requests, and a proxy server. A proxy server acts as an intermediary between the client and the server, forwarding requests and responses and providing an additional layer of anonymity. The proxy server can be either a public or a private one.

A further technique useful for helping crawlers avoid detection and blocking by servers is user agent rotation. A user agent is a string that identifies the browser type and operating system used to make the request. By rotating the user agent, a crawler can simulate human-like behavior and avoid being detected as an automated bot.

One of the main differences between surface web and dark web requests is the domain resolution process. On the surface web, domain names are translated into IP addresses using a centralized DNS resolution system. However, the dark web uses a decentralized naming system called Tor Hidden Services, which allows

websites to have .onion domain names that are not registered with standard DNS servers. Python scripts that are designed to make connection requests on the dark web must be configured to use Tor Hidden Services to resolve domain names and connect to the appropriate servers.

3.7 Stop Criteria

A crawler uses exit conditions to determine when the crawling process should stop. There are several exit conditions that can be used depending on the goals of the crawler and the resources available. Our algorithm employs the following stop criteria:

- **Max depth:** the crawler stops if it reaches a certain level of depth. This criteria is useful when you want to limit the amount of resources used during the crawling process.
- **Max number of links downloaded:** one common exit condition is to stop crawling after a certain number of links have been visited. This approach is useful when you want to limit the amount of data collected or when you are crawling a large website and want to ensure that the process does not continue indefinitely.
- **Time limit:** it is important to stop crawling after a certain amount of time has passed. Considering the dynamic nature of the dark web, where lots of websites are frequently updated, having a stop criteria condition ensures that the data collected is up-to-date.
- **Data crawled:** the last stop criteria is determined by whether or not the crawler has collected all the targeted links.

4 Evaluation

In this section, we describe the evaluation criteria and the case scenario used to evaluate a dark web crawler.

4.1 Case Scenario

To evaluate our dark web crawling methodology, we chose to analyze a prominent dark web marketplace, *Cocorico Market*. This marketplace is known for hosting a wide range of illicit activities, including the sale of drugs, weapons, counterfeit items, stolen data, and various other illegal goods and services.

Notably, Cocorico Market does not have a mandatory login page; access to the marketplace is available without requiring login credentials. This means we did not need to extract user/password combinations from the market metadata for this evaluation. Instead, we only need cookies that can be manually saved by the user in a metadata file, associating the seed URL with the extracted cookie.

4.2 Metrics

Coverage: A crawler’s coverage refers to the extent to which it is able to successfully and comprehensively extract all of the desired data from a given set of sources. Measuring coverage is important for evaluating the crawler’s effectiveness and ensuring that it is meeting its intended goals. One way to measure the coverage of a crawler is by calculating the *ratio of the number of downloaded pages* against the *total number of web pages* of a website [13, 17]. Since the total number of pages of a website is not known in advance, one approach to estimating the coverage of a crawler is to compare the number of pages downloaded by the crawler to the total number of pages downloaded by other crawlers. Using this approach, we compare which crawler is capable of downloading the higher number of .onion pages.

Performance: Performance is a critical factor in evaluating a web crawler’s effectiveness, and several metrics can be used to measure its performance. Two common metrics used to evaluate the performance of a crawler are the *execution time* of each crawler and the *number of pages downloaded per minute* [4, 13].

In evaluating the performance of a dark web crawler, the *execution time* is a fundamental metric due to the unique challenges posed by the dark web environment. The TOR network adds complexity to the crawling task, making it more demanding from a time perspective. Therefore, minimizing execution time is crucial for enhancing the agility and responsiveness of the crawler in navigating the dark web.

The *number of pages downloaded per minute* is a measure of how quickly a crawler can navigate the web and retrieve content. This metric is essential for ensuring that the crawler can keep up with the constant stream of new content being added to the web. A high rate of page downloads per minute can be an indication of a well-performing crawler, while a low rate may suggest that the crawler is struggling to keep up with the volume of content. However, while it can be a good performance indicator, it cannot solely determine its effectiveness because the downloaded pages may include bad content, such as empty or error pages, which do not contribute to the quality of the crawled data. Therefore, in the final results, it is necessary to evaluate these metrics jointly with other metrics to estimate the goodness of the tool.

Robustness: In the context of web crawling, robustness refers to the ability of a crawler to continue operating effectively and efficiently even when faced with unexpected or challenging conditions. A robust crawler is able to handle a variety of situations, including network connectivity issues, site changes, and other potential obstacles, without experiencing significant disruptions or failures. Robustness is an important characteristic of a successful crawler, as it ensures that the crawler is able to continue operating effectively over time, even in the face of changing conditions and challenges. We measured the robustness by counting the *number of errors* that happen during the crawling process [17, 31].

4.3 Experimentation

Table 1. Comparison between ACHE and CRATOR based on the components of a dark web crawler outlined in Sect. 3.

	Search Strategy	Cookies	Cookie Rotation	CAPTCHA Detection
ACHE	BF	✓	✗	✗
CRATOR	BF	✓	✓	✓

To evaluate the effectiveness of our dark web crawler, we conducted experiments comparing it with the ACHE crawler. We selected this crawler based on its prominence in the field and its ability to navigate the dark web.

The goal of the experimentation is to assess the performance of our crawler in terms of **coverage**, **performance**, and **robustness**. We aim to determine how well our crawler collects data from the dark web, enlightening the strengths and weaknesses of our proposed approach to capturing data from Cocorico Market. The evaluation metrics provided a comprehensive framework for assessing the crawlers’ performance and determining our approach’s suitability in the context of dark web crawling.

Additionally, our comparison table (see Table 1) highlights significant differences in key components between ACHE and CRATOR. CRATOR incorporates advanced features such as cookie rotation and CAPTCHA detection, which are crucial for navigating sites with stringent security measures. These enhancements were expected to contribute to improved performance and robustness in challenging dark web environments.

The experimental setup involved running each crawler separately on identical hardware infrastructure, a desktop PC with an Intel(R) Core(TM) i7-9700 CPU @ 3.00GHz CPU, 16GB 2666MT/s DDR4 RAM, and Ubuntu 22.04 OS, using the same configurations and parameters. To ensure a fair comparison among the crawlers, we established a specific time frame of 10 h for the crawling process. The experimentation was repeated 10 times for each level of depth, ranging from depth 1 to depth 4. Subsequently, we calculated the mean and standard deviation of each metric involved in the analysis.

5 Results

In this section, we present the results of the experimentation conducted on both dark web crawlers *ACHE* and *CRATOR*.

5.1 Coverage

In the analysis of the dark web marketplace *Cocorico Market* to validate the coverage, we considered two coverage metrics as explained in Sect. 4.2. Coverage

Table 2. Results of the coverage, as a number of success links downloaded by each approach, and relative coverage (rate) of CREATOR and ACHE. The analysis was conducted on the Cocorico market with 10 iterations and 4 levels of depth. Mean values represent the average of 10 iterations, while standard deviation indicates variability for each level of depth.

		DEPTH 1			DEPTH 2			DEPTH 3			DEPTH 4		
		identified	success	rate	identified	success	rate	identified	success	rate	identified	success	rate
CRATOR	mean	115.9	115.9	1	622.2	622.2	1	2266.5	2266.5	1	4125.7	4125.7	1
	std	0.316	0.316	0	12.2	12.2	0	79.271	79.271	0	193.82	193.82	0
ACHE	mean	117.0	110.5	0.944	539.3	516.1	0.829	1757.2	1395	0.763	2784.1	2210.5	0.705
	std	0.0	19.506	0.167	214.925	246.84	0.36	619.048	616.559	0.154	1148.065	1110.328	0.287

is defined as the *number of pages downloaded*, while relative coverage is the success rate of a crawler calculated as *pages downloaded* divided by *pages found*.

Our tool, *CRATOR*, demonstrated remarkable effectiveness in downloading all the pages it found. On the other hand, *ACHE* achieved a maximum coverage of 0.944 with a level of depth 1 (see results in Table 2). When considering relative coverage, it seems that *CRATOR* outperforms *ACHE* consistently.

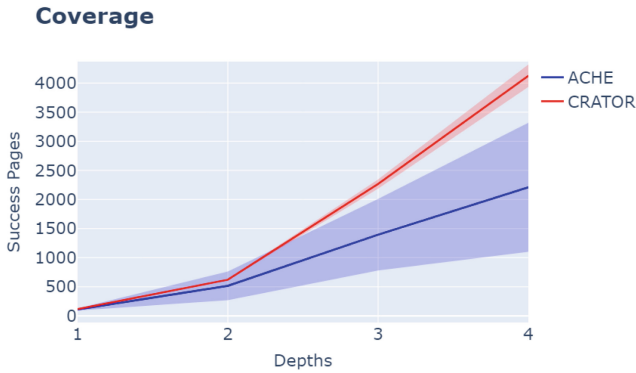
This metric provides insights into the internal coverage of the tool, as it is based on the links discovered during crawling. Hence, it lacks knowledge of the real size of the marketplace. By utilizing coverage metrics, we gain a better understanding of the capabilities of both tools in downloading all the pages within the marketplace.

From the results presented in Table 2 and the visual representation in Fig. 2a, it is evident that *CRATOR* consistently outperforms *ACHE* across all scenarios (levels of depth 1, 2, 3, 4). Furthermore, the standard deviation outlined in Fig. 2a indicates that *CRATOR* exhibits higher reliability across different iterations. The lower standard deviation of *CRATOR* suggests that the results achieved with different executions are more consistent compared to *ACHE*.

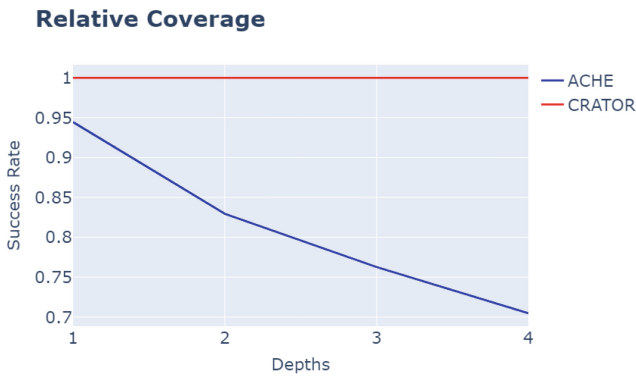
5.2 Performance

As outlined in Sect. 4.2, we evaluate our tool regarding the performance metric using two key metrics: execution time and the number of pages downloaded per minute. Figure 3a illustrates the execution time of both approaches. It is evident from the figure that *CRATOR* exhibits lower execution times compared to *ACHE* for the first three levels of depth. However, *ACHE* outperforms *CRATOR* with the depth 4 (see Table 3).

To understand this discrepancy, we analyze the second performance metric. Figure 3b presents the pages downloaded per minute by each tool for each level of depth. It is evident that *CRATOR* consistently outperforms *ACHE* in every scenario (see Table 4). Thus, despite the higher execution time of *CRATOR* at level of depth 4, it is clear that *CRATOR* downloaded more pages, as confirmed by the results presented in Table 2.



(a) Coverage of the two approaches, expressed as the number of pages downloaded by each approach.

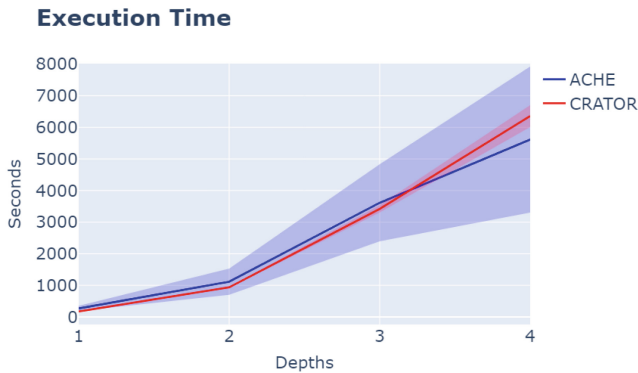


(b) Relative coverage of CRATOR and ACHE, expressed as the number of pages successfully downloaded over the total pages found.

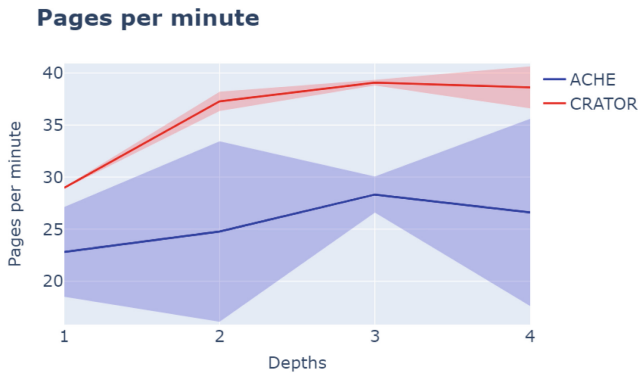
Fig. 2. Coverage of CRATOR and ACHE. Figure (a) shows the absolute coverage, highlighting how CRATOR performs better with every level of depth, while fig. (b) shows how CRATOR successfully downloaded all the pages that have been found.

Table 3. Execution time (seconds).

		DEPTH 1	DEPTH 2	DEPTH 3	DEPTH 4
ACHE	mean	268.373	1114.383	3612.991	5615.148
	std	85.021	416.752	1220.657	2311.664
CRATOR	mean	173.3	939.2	3420.6	6356.2
	std	1.418	18.103	119.295	350.985



(a) Execution time (seconds).



(b) Pages per minute.

Fig. 3. Performance metrics of CRATOR and ACHE. Figure (a) illustrates the execution time, where CRATOR outperforms ACHE in the first three levels of depth. However, at depth 4, ACHE spends less time crawling pages. Figure (b) displays the pages per minute ratio of both approaches. CRATOR outperforms ACHE at all levels of depth, exhibiting a higher ratio than ACHE.

Table 4. Pages per minute.

		DEPTH 1	DEPTH 2	DEPTH 3	DEPTH 4
ACHE	mean	22.815	24.767	28.322	26.602
	std	4.317	8.666	1.742	8.998
CRATOR	mean	28.975	37.276	39.076	38.623
	std	0.079	0.929	0.280	2.015

5.3 Robustness

As discussed in Sect. 4.2, we evaluate robustness using the failure rate metric, which is expressed as the number of pages not downloaded over the total number of pages found. Table 5 presents the number of pages failed, with the failure rate expressed as a value between 0 and 1.

Table 5. Failure rate of CRATOR and ACHE by analyzing Cocorico market with 10 iterations and 4 level of depth.

		DEPTH 1			DEPTH 2			DEPTH 3			DEPTH 4		
CRATOR	mean	115.9	0	0	622.2	0	0	2266.5	0	0	4125.7	0	0
	std	0	0	0	12.2	0	0	79.271	0	0	193.82	0	0
ACHE	mean	117.0	6.5	0.056	539.3	23.2	0.171	1757.2	362.2	0.237	2784.1	573.6	0.295
	std	0.0	19.506	0.167	214.925	66.344	0.36	619.048	219.622	0.154	1148.065	475.628	0.287

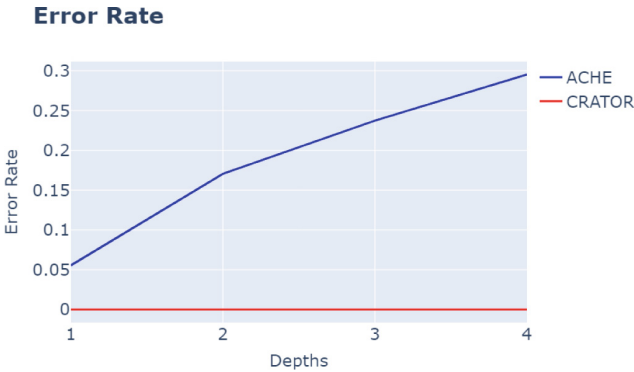


Fig. 4. Error rate of CRATOR and ACHE.

In Fig. 4, we observe that our tool, *CRATOR*, successfully downloaded all the pages found, demonstrating a failure rate of 0. This is in contrast to *ACHE*, which exhibits a failure rate ranging between 0.05 and 0.29. These results underscore the effectiveness of the techniques employed to conceal our crawler’s activities on the dark web, as outlined in Sect. 3. Specifically, techniques such as proxy usage, IP rotation, and cookie rotation have proven to be efficient in crawling the dark web while minimizing the risk of detection and failure.

6 Discussion

The results obtained from the evaluation highlight the strengths of *CRATOR* in terms of coverage, performance, and robustness.

In terms of coverage, *CRATOR* consistently outperformed *ACHE* across all levels of depth, as shown in Table 2 and Fig. 2. This suggests that *CRATOR* is more effective in discovering and downloading pages from the dark web marketplace despite the challenges posed by its unstructured nature.

Regarding performance, *CRATOR* exhibited lower execution times for the first three levels of depth and a higher number of pages downloaded per minute across all depths, as depicted in Fig. 3. This indicates that *CRATOR* is more efficient in crawling and downloading pages, even though it took longer at depth 4. The higher efficiency can be attributed to *CRATOR*'s architecture, which integrates cookie rotation and user-driven manual intervention, allowing it to handle more complex crawling tasks more effectively.

The robustness analysis revealed that *CRATOR* had a failure rate of 0, in stark contrast to *ACHE*, which had failure rates between 0.05 and 0.29 (see Table 5). This demonstrates the reliability of *CRATOR* in maintaining continuous access and successfully downloading content without detection or failure, which is critical for effective dark web crawling.

7 Conclusion and Future Work

In this study, we presented *CRATOR*, a dark web crawler that offers an efficient and robust approach for capturing data from dark web marketplaces. We outlined the architecture of *CRATOR*, emphasizing its security features, such as an automatic login form, cookies rotation, and random user agent implementation, which enhance its ability to handle security checks prevalent in the dark web.

To validate the effectiveness of our proposed crawler, we compared *CRATOR* with an established dark web crawler, *ACHE*. The comparison was based on metrics such as coverage, performance, and robustness. The results demonstrated that *CRATOR* outperformed *ACHE* in terms of coverage, downloading a greater number of pages within the specified timeframe. It also exhibited a higher page-per-minute rate, indicating its efficiency in navigating and retrieving information. Additionally, *CRATOR* raised fewer errors, highlighting its robustness and ability to overcome common obstacles encountered in the dark web.

Our study demonstrates the potential of *CRATOR* as an effective tool for dark web crawling. With its improved coverage, performance, and robustness compared to existing approaches, *CRATOR* offers a significant contribution to the exploration and understanding of the dark web ecosystem.

As a future work, we plan to conduct more exhaustive experimentation, focusing on daily analysis of dumps and incorporating different dark web marketplaces with varying security layers. This expanded evaluation will allow us to assess the usability and performance of *CRATOR* in a broader range of scenarios, further validating its capabilities as a reliable dark web crawler.

Furthermore, to encourage reproducibility and facilitate research analysis, we have made the *CRATOR* source code available on GitHub¹.

¹ <https://github.com/danielp92/crator>.

Acknowledgements. The work is supported by the EU H2020 framework programme, grant “ONCOSCREEN” under Grant No.: 101097036.

References

1. Santos, A., Pham, K.: Ache focused crawler (2023). <https://github.com/VIDA-NYU/ache>
2. Barbosa, L., Freire, J.: An adaptive crawler for locating hidden-web entry points. In: Proceedings of the 16th international conference on World Wide Web, pp. 441–450 (2007)
3. Bergman, J., Popov, O.B.: Exploring dark web crawlers: a systematic literature review of dark web crawlers and their implementation. *IEEE Access* **11**, 35914–35933 (2023)
4. Bharati, K., Premchand, P., Govardhan, A.: HIGWGET-a model for crawling secure hidden webpages. *Int. J. Data Min. Knowl. Manage. Process* **3**(2), 23 (2013)
5. Boldi, P., Marino, A., Santini, M., Vigna, S.: BUBiNG: massive crawling for the masses. *ACM Trans. Web (TWEB)* **12**(2), 1–26 (2018)
6. Boshmaf, Y., Perera, I., Kumarasinghe, U., Liyanage, S., Al Jawaheri, H.: Dizzy: large-scale crawling and analysis of onion services. In: Proceedings of the 18th International Conference on Availability, Reliability and Security, pp. 1–11 (2023)
7. Cascavilla, G.: The rise of cybercrime and cyber-threat intelligence: perspectives and challenges from law enforcement. *IEEE Secur. Priv.* (01), 2–11 (2024). <https://doi.org/10.1109/MSEC.2024.3407859>
8. Cascavilla, G., Tamburri, D.A., Van Den Heuvel, W.J.: Cybercrime threat intelligence: a systematic multi-vocal literature review. *Comput. Secur.* **105**, 102258 (2021)
9. Celestini, A., Guarino, S.: Design, implementation and test of a flexible tor-oriented web mining toolkit. In: Proceedings of the 7th International Conference on Web Intelligence, Mining and Semantics, pp. 1–10 (2017)
10. Chen, H.: Dark web: exploring and mining the dark side of the web. In: Domenach, F., Ignatov, D.I., Poelmans, J. (eds.) *ICFCA 2012. LNCS (LNAI)*, vol. 7278, pp. 1–1. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-29892-9_1
11. Cho, J., Garcia-Molina, H., Page, L.: Efficient crawling through URL ordering. *Comput. Netw. ISDN Syst.* **30**(1–7), 161–172 (1998)
12. De Pascale, D., Cascavilla, G., Tamburri, D.A., Van Den Heuvel, W.J.: SENSEI: scraper for enhanced analysis to evaluate illicit trends. In: Troya, J., et al. (eds.) *International Conference on Service-Oriented Computing*, pp. 374–383. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-26507-5_36
13. Dhabliya, D., Alawadi, A.H., Rai, V.K., Velusudha, N., Sarathy, K.: Evaluation of focused web crawlers in real-world scenarios. In: 2023 3rd International Conference on Technological Advancements in Computational Sciences (ICTACS), pp. 711–715 (2023). <https://doi.org/10.1109/ICTACS59847.2023.10389832>
14. Dikaiaikos, M.D., Stassopoulou, A., Papageorgiou, L.: An investigation of web crawler behavior: characterization and metrics. *Comput. Commun.* **28**(8), 880–897 (2005)
15. Dinh, N.T., Hoang, V.T.: Recent advances of captcha security analysis: a short literature review. *Procedia Comput. Sci.* **218**, 2550–2562 (2023). <https://doi.org/10.1016/j.procs.2023.01.229>, International Conference on Machine Learning and Data Engineering

16. Dutta, N., Jadav, N., Tanwar, S., Sarma, H.K.D., Pricop, E.: TOR—the onion router. In: *Cyber Security: Issues and Current Trends*. SCI, vol. 995, pp. 37–55. Springer, Singapore (2022). https://doi.org/10.1007/978-981-16-6597-4_3
17. Girardi, C., Ricca, F., Tonella, P.: Web crawlers compared. *Int. J. Web Inf. Syst.* **285–94** (2006)
18. Goldschlag, D.M., Reed, M.G., Syverson, P.F.: Hiding routing information. In: Anderson, R. (ed.) *IH 1996*. LNCS, vol. 1174, pp. 137–150. Springer, Heidelberg (1996). https://doi.org/10.1007/3-540-61996-8_37
19. Greenberg, A.: Hacker lexicon: what is the dark web? *Wired*. <http://www.wired.com/2014/11/hacker-lexicon-whats-dark-web> [dostkep 6.02. 2017] (2014)
20. Kalpakis, G., et al.: Interactive discovery and retrieval of web resources containing home made explosive recipes. In: Tryfonas, T. (ed.) *HAS 2016*. LNCS, vol. 9750, pp. 221–233. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-39381-0_20
21. Khare, R., Cutting, D., Sitaker, K., Rifkin, A.: Nutch: a flexible and scalable open-source web search engine. *Oregon State Univ.* **1**, 32–32 (2004)
22. Kumar, M., Jindal, M.K., Kumar, M.: A systematic survey on CAPTCHA recognition: types, creation and breaking techniques. *Arch. Comput. Methods Eng.* **29**(2), 1107–1136 (2022)
23. Liu, B., Liu, B., Menczer, F.: Web crawling. In: *Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data*, pp. 311–362 (2011). <https://doi.org/10.1007/978-3-642-19460-3>
24. Moshe, N., Brizinov, S., Onitza-Klugman, R., Efimov, K.: Exploiting URL parsers: The good, bad, and inconsistent. Tech. rep., Technical report. 2021 (2022). <https://security.claroty.com/URLparserconfusion>
25. Narayanan, P.S., Ani, R., King, A.T.L.: TorBot: open source intelligence tool for Dark Web. In: Ranganathan, G., Chen, J., Rocha, Á. (eds.) *Inventive Communication and Computational Technologies*. LNNS, vol. 89, pp. 187–195. Springer, Singapore (2020). https://doi.org/10.1007/978-981-15-0146-3_19
26. Ngo, F.T., Marcum, C., Belshaw, S.: The Dark web: what is it, how to access it, and why we need to study it. *J. Contemp. Crim. Justice* **39**(2), 160–166 (2023). <https://doi.org/10.1177/10439862231159774>
27. Pant, G., Srinivasan, P., Menczer, F.: Crawling the web. *Web Dyn.* **2004**, 153–178 (2004)
28. Prabha, C., Mittal, A.: Dark web: a review on the deeper side of the web. In: *2022 OPJU International Technology Conference on Emerging Technologies for Sustainable Development (OTCON)*, pp. 1–6 (2023). <https://doi.org/10.1109/OTCON56053.2023.10113989>
29. Raman, R., Kumar Nair, V., Nedungadi, P., Ray, I., Achuthan, K.: Darkweb research: past, present, and future trends and mapping to sustainable development goals. *Heliyon* **9**(11), e22269 (2023)
30. Ristoski, P.: Web mining. In: Rokach, L., Maimon, O., Shmueli, E. (eds.) *Machine Learning for Data Science Handbook*, pp. 447–467. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-24628-9_20
31. Yang, D., Thiengburanathum, P.: Scalability and robustness testing for open source web crawlers. In: *2021 Joint International Conference on Digital Arts, Media and Technology with ECTI Northern Section Conference on Electrical, Electronics, Computer and Telecommunication Engineering*, pp. 197–201 (2021). <https://doi.org/10.1109/ECTIDAMTNCN51128.2021.9425701>



ProxyKiller: An Anonymous Proxy Traffic Attack Model Based on Traffic Behavior Graphs

Hongbo Xu¹, Zhenyu Cheng^{1(✉)}, Shuhao Li³, Chenxu Wang^{1,2},
Peishuai Sun^{1,2}, Jiang Xie³, and Qingyun Liu¹

¹ Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
{xuhongbo, chengzhenyu, wangchenxu, sunpeishuai, liuqingyun}@iie.ac.cn

² School of Cyber Security, University of Chinese Academy of Sciences,
Beijing, China

³ Zhongguancun Laboratory, Beijing, China
{lishuhao, xiejiang}@zgclab.edu.cn

Abstract. As common tools for evading censorship and protecting privacy, anonymous proxies are widely favored for their lightweight and easy deployment. In recent years, anonymous proxies have continually evolved regarding traffic obfuscation and masking strategies. Due to the lack of design targeting the connection behavior features of different proxy flows, current methods for proxy traffic identification often suffer from high false positive rates. Consequently, these methods are insufficient for real-world applications. This masks the deficiencies of existing anonymous proxy technologies in combating detection models based on flow relationship analysis.

To address the issues mentioned, we introduce a novel proxy traffic attack model called ProxyKiller based on multi-flow connection behavior features. After obtaining the original PCAP traffic files, ProxyKiller constructs traffic behavior graphs using extracted temporal, spatial, content, and specially designed byte features. Subsequently, ProxyKiller extracts latent topological structure features through graph neural networks, integrates them into a unified graph-level representation vector, and obtains prediction results for each traffic behavior graph through a random forest model. Finally, ProxyKiller implements a global decision-making mechanism from a traffic graph correlation perspective to correct prediction results for related traffic behavior graphs. Experimental results on three real-world datasets demonstrate that ProxyKiller outperforms several state-of-the-art traffic classification methods and shows robustness in various proxy traffic classification tasks. These results indicate that current anonymous proxies exhibit certain vulnerabilities when countering traffic analysis methods based on traffic behavior graph structure.

Keywords: anonymous proxy · proxy traffic classification · graph neural network · traffic analysis

1 Introduction

Network traffic classification is one of the essential tasks in network management and network security [4]. With the increasing demand for privacy protection, encryption communication technologies such as Transport Layer Security (TLS), Virtual Private Network (VPN), and anonymous proxies [25] are gradually becoming widely used. Among them, *anonymous proxies* are lightweight proxy services that utilize proxy protocols such as socks5. They employ techniques for disguising and obfuscating traffic to bypass censorship. This technology can be used for various purposes, including evading online tracking, bypassing censorship, and circumventing geographical restrictions on streaming media. The advantages of anonymous proxies include being lightweight, highly adversarial, easy to deploy, and low cost. Therefore, compared to other technologies such as VPN, their scope of use is broader.

Existing proxy traffic classification techniques can be categorized into active and passive traffic identification. Active probing obtains server response features by sending data packets to complete classification. Although this method often yields high accuracy, it is susceptible to circumvention by obfuscation techniques. In contrast, passive traffic identification extracts spatio-temporal features from captured traffic and trains machine learning or deep learning models for traffic classification. Passive methods are generally less vulnerable to evasion than active methods. However, existing methods have several problems.

(1) Existing techniques are insufficient for mining proprietary features specific to proxy traffic. They generally use spatio-temporal statistical features, which are inadequate to precisely capture proxy traffic’s behavioral features.

(2) Features across different types of traffic are heterogeneous. Essentially, these static features are tabular data [11]. For instance, temporal and spatial features are not highly correlated. Therefore, simply transferring deep learning models from other domains may be less effective [17].

(3) There is an Only One Flow Problem [6] in proxy traffic, which significantly reduces the availability of flow features within the proxy traffic, thereby decreasing the accuracy of existing traffic classification methods. Existing methods usually classify traffic at a singular level, such as packet or flow level, lacking relational feature learning between different flow samples.

The shortcomings of the above passive traffic classification methods have obscured the ongoing privacy and security issues inherent in current anonymous proxies. Developers of existing anonymous proxies often overlook the need to disguise connection behavior features between different flows during protocol design. Proxy traffic exhibits similar client and server properties compared to the complex changes in attack targets in malicious traffic classification. This makes it more suitable for correlation through a graph structure, which is our primary research motivation.

To validate the security of existing anonymous proxies and demonstrate the powerful applicability of methods based on multi-flow correlation behavior, we propose an attack model based on graph neural networks called ProxyKiller. This method comprehensively considers traffic features from different dimensions and

the connection behavior features of different flows to detect proxy traffic. Our main contributions are summarized as follows:

- Firstly, on top of general temporal and spatial features, we propose byte distribution-related features for proxy traffic. These features include byte content distribution, payload length distribution, plaintext features, and entropy features of bytes.
- Secondly, based on the connection behavior features of proxy traffic, we designed a graph construction method that represents the relationships between different flow samples and integrates heterogeneous traffic features. Graph representation learning of traffic behavior graphs can capture the distinct behavioral pattern differences between proxy and regular traffic. Then, ProxyKiller uses a random forest model to predict the labels of different node feature vectors in the graph and makes a category label decision for the entire graph through ensemble decision-making. Through enhanced graph neural networks, various types of traffic features are adaptively aggregated, solving the problem of heterogeneous feature representation learning and improving the robustness of the model.
- Furthermore, we have constructed a global decision-making mechanism from a traffic graph correlation perspective. This mechanism employs a unified decision-making process for labeling traffic graphs with interconnected relationships, minimizing false positives among individual graph labels.
- We manually constructed a proxy traffic dataset, AnonProxy2023, containing four popular anonymous proxy protocols, which can simulate the distribution of proxy traffic in the real world to the greatest extent. Experimental results on three types of anonymous proxy traffic datasets, including AnonProxy2023, show that our model can precisely detect anonymous proxy traffic, maintaining a precision rate exceeding 97% across various tasks. These research findings indicate that popular anonymous proxy protocols risk being monitored and identified by censors.

2 Related Work

Current techniques for identifying anonymous proxy traffic can be divided into active probing and passive traffic identification.

2.1 Active Probing

Frolov et al. [10] first disclosed that the GFW detects Shadowsocks servers through traffic replay. Since the connections of legitimate clients contain identity information, the server usually responds. Subsequently, Frolov et al. [9] conducted studies using active probing technologies to detect anti-censorship proxies. The authors identified five anti-detection proxy protocols, including obfs4, Shadowsocks, Lampshade, etc. These protocols exhibit unique TCP behaviors during censorship circumvention, allowing censors to confirm suspected proxy servers with few probe connections effectively. Utilizing these features,

the authors constructed rule models. Through testing hundreds of thousands of servers collected at a 10 Gbps traffic observation point at a university ISP, they found that their rule models could efficiently identify proxy servers with a very low false positive rate. Beznazwy et al. [2] found that the Great Firewall (GFW) employs a two-step method to detect Shadowsocks [7] servers: the first step involves filtering proxy traffic by relying on the length and entropy features of passively collected traffic packets. In the second step, GFW employs multiple strategies to actively probe servers through traffic replay, detecting proxy traffic based on server responses.

2.2 Passive Traffic Identification

Currently, there are limited studies on passive traffic identification for anonymous proxies. We have summarized the most representative work as follows.

Wang et al. [25] proposed a traffic classification model called SSAPPIDENTIFY, designed explicitly for the Shadowsocks protocol. This model builds upon traditional statistical and distributional features based on packet length and timestamps by adding sliding window JS divergence features. It utilizes a one-vs-all model combined with machine learning algorithms such as Random Forest for multi-class application classification. The sliding window JS divergence feature effectively reduces the impact of different devices on application traffic identification, significantly enhancing the robustness of the proxy traffic classification model. Xue et al. [29] have achieved low false-positive identification of proxy traffic with nested encryption configurations using fingerprints generated from nested TLS handshakes. However, due to the detection principles' limitations, this method only applies to the detection of anonymous proxies with nested encryption configurations. Zhao et al. [30] focused on identifying single-user network behavior within tunnels and proposed a TMT-RF framework, which uses the Random Forest algorithm to recognize mixed application behavior within tunnels. In real scenarios where a single user uses a tunnel, the traffic over a period may contain multiple applications. The authors effectively segmented the mixed network behavior within the tunnel by using methods such as Burst segmentation and head-tail segmentation. However, this work only studied network behavior identification in single-user scenarios, and its effectiveness in more complex scenarios remains to be verified.

2.3 Summary

From the above work, it is evident that although existing methods are relatively accurate, the current efforts to identify hidden proxy traffic primarily focus on analyzing traffic features for specific protocols and then using active probing and passive traffic identification to exploit protocol vulnerabilities. However, in real-world environments, the rapid iteration of anonymous proxy technologies means that methods based on specific protocol vulnerabilities can easily be circumvented by patching. However, in real-world environments, the rapid iteration

of anonymous proxy technologies means that methods based on particular protocol vulnerabilities can easily be circumvented by patching. For example, the Shadowsocks-2022 protocol [22] patched the security vulnerabilities of AEAD encryption and included comprehensive replay protection.

3 Motivation and Threat Model

3.1 Motivation

We highlight two of the most common anonymous proxy disguise techniques to understand how proxy traffic differs from other traffic classification tasks.

WebSocket Disguise. WebSocket is a communication protocol that provides the capability for full-duplex (bi-directional) communication over a single persistent connection. It is designed to establish a long-term connection between a client and a server, allowing data to be transmitted with minimal delay. Disguising proxy traffic as WebSocket communication effectively makes proxy traffic appear as regular web browsing traffic, thus bypassing censorship and blocking systems based on network traffic pattern recognition. Additionally, WebSocket’s long-connection attribute increases the difficulty of distinguishing between proxy traffic and benign traffic.

OBFS Plugin. The OBFS plugin, short for “obfuscation plugin”, is an extension plugin for Shadowsocks designed to enhance its traffic disguise capabilities. This plugin can obfuscate Shadowsocks’ traffic features to make them appear similar to regular network traffic, making it difficult for network censorship systems to recognize and block. The plugin has multiple functions, including modifying the header structure of the traffic packets to disguise them as HTTP or HTTPS headers. In addition, it can randomize packet size and sending intervals to conduct traffic feature adversarial actions effectively.

These measures significantly enhance network traffic concealment, greatly increasing the difficulty of proxy detection. To address these challenges, we leverage graph neural networks to analyze the relationships between different proxy flows, thereby enhancing the detection capabilities of our model. Specifically, we first extract traffic packets into different flows. As shown in Fig. 1, each flow is split into two nodes. Subsequently, based on the correlation relationships between different flows, we link different nodes to each other to construct a traffic behavior graph.

3.2 Threat Model

This paper aims to design a traffic attack model that identifies anonymous proxies based on side-channel features observed during the encrypted proxy traffic transmission process. The design goal of the model is to detect users within a small local area network (LAN) who exhibit censorship evasion behaviors by capturing network traffic. As shown in Fig. 2, by mirroring the router traffic on a bypass, attackers can monitor the outbound traffic of LAN users without affecting normal traffic.

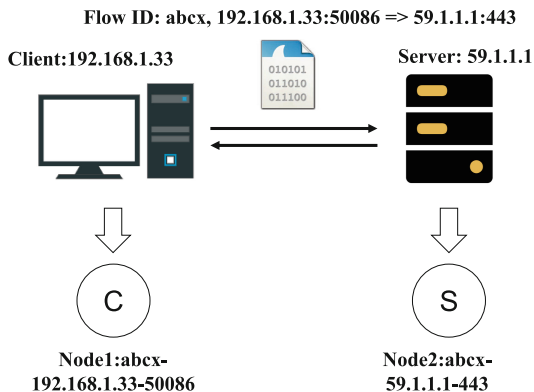


Fig. 1. Example of Node Construction

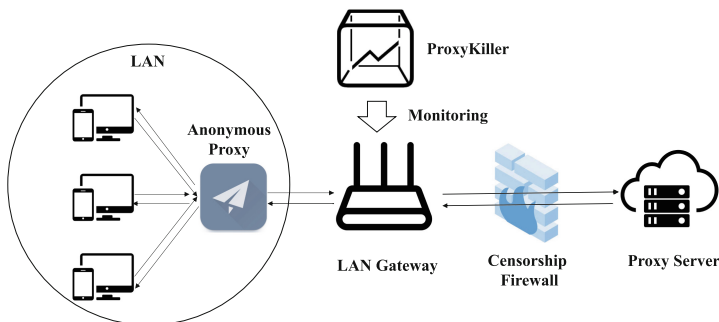


Fig. 2. Example of Attack Scenario.

4 Methodology

ProxyKiller’s main implementation motivations are as follows. First, ProxyKiller assigns heterogeneous features to nodes as initial features. Then, we integrate these heterogeneous traffic features from different categories through graph learning into a graph-level representation vector. So, it overcomes the limitations of traditional deep learning models that struggle with handling heterogeneous traffic features. ProxyKiller enhances the representation learning capability and robustness of the proxy traffic classification model by incorporating statistical traffic features and implicit topological structure features.

4.1 Overview

The static statistical features of traffic data are fundamentally tabular data [13]. We utilize a graph autoencoder for representation learning to capitalize on the abundant data in traffic classification tasks fully. Unlike typical graph classification tasks with fewer node features, traffic behavior graphs often contain

features numbering in the hundreds. Traditional graph encoders primarily aim to reconstruct structural features. In contrast, GraphMAE [15] adopts a masking approach for self-supervised learning that emphasizes reconstructing node features. Consequently, we choose GraphMAE to facilitate feature learning in traffic behavior graphs.

Figure 3 illustrates ProxyKiller’s model structure. The detailed training and prediction process of the model is explained as follows.

(1) Initially, flow data is extracted from raw network traffic. Timing features, packet length spatial features, traffic content features, and byte features are extracted from flow data as initial node features. Subsequently, traffic behavior graphs are constructed by correlating different flow samples. As shown in Fig. 1, the nodes labeled ‘S’ and ‘C’ represent the server and client nodes, respectively.

(2) ProxyKiller employs a graph feature encoder to extract implicit topological features. For a constructed flow connection behavior graph G , we first randomly select a proportion of nodes and replace their features with Mask tokens. Let C be ProxyKiller’s graph encoder and D be its graph decoder. The encoder C encodes the entire graph into a feature vector, which the decoder D then decodes into a new graph feature vector. By comparing the scaled cosine loss between the output of the decoder D and the original feature vector, we reconstruct the feature vectors of the masked nodes. The goal of graph structure learning is to train a graph encoder to reconstruct hidden features within the nodes, thereby obtaining node feature vectors that include structural features. During the prediction phase, we eliminate the graph decoder and feed the traffic behavior graph data into the graph encoder to generate node feature vectors that include structural features.

(3) Subsequently, we train a random forest model using all node embedding vectors in the graph as inputs, allowing us to complete the label prediction for each node. We determine the label for each flow connection behavior graph by selecting the most frequent label category. Finally, we use a global decision-making mechanism to correct the labels of multiple traffic behavior graphs with correlational relationships, thereby reducing the false positive rate.

4.2 Traffic Behavior Graph Construction

In this paper, we present the concept of the traffic behavior graph. For any given proxy traffic file P , a traffic behavior graph can be represented as a tuple $G(P) = (N, E)$, where N is a set of vertices and E is a set of undirected edges.

General Node Traffic Feature Extraction. We utilize the NFStream [3] and DPKT libraries for extracting traffic features, where the flow slicing parameters maintain the NFStream default settings. We uniquely identify a node with a triple (Flow ID, IP address, Port number). As shown in Fig. 1, we divide the flow with Flow ID ‘abcx’ into two nodes, Node1 and Node2, following this identification method. Simultaneously, flow features are assigned to different nodes based on the direction of packet transmission. By combining the Flow ID, we

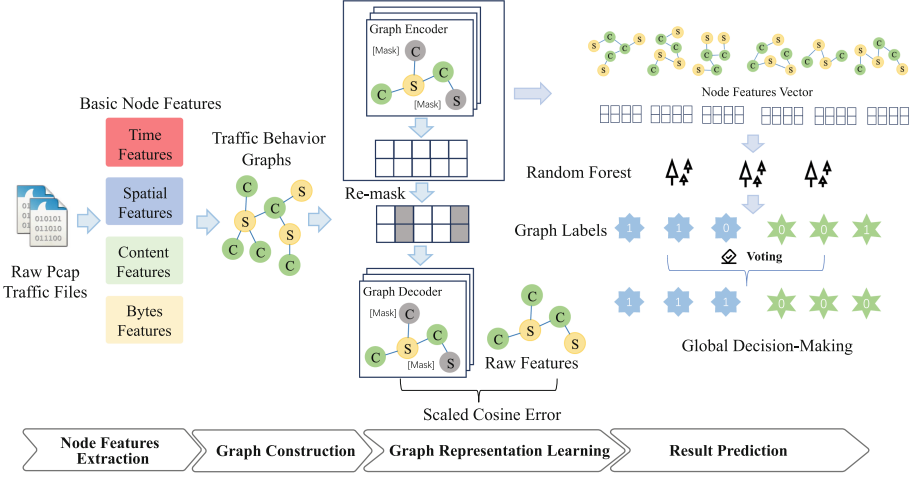


Fig. 3. The structure of ProxyKiller.

can resolve the issue of port multiplexing, which causes node conflicts within a given timeframe.

Detailed node features are shown in Table 1. We convert categorical features into integers using LabelEncoder and normalize the continuous numerical features. The features of each node have two directions: one is the single transmission feature, and the other is the bidirectional transmission feature. Statistical traffic features are calculated based on basic traffic features. Due to space limitations, they are not displayed in Table 1. For example, the statistical features of the packet field include Origin, Minimum, Maximum, Mean, and Standard Deviation. The statistical features of the packet interval field include Minimum, Maximum, Mean, and Standard Deviation. For each node, we extract initial node features in four parts.

The first three parts comprise temporal, spatial, and content statistical features. These features are commonly used as foundational features in traffic classification [27]. They are calculated based on a series of traffic statistical features grounded in the traffic packets' timestamps, packet lengths, and content fields. In the next part, we introduce our proposed byte features for anonymous proxy traffic.

Extraction of Byte Features for Proxy Traffic. Inspired by [26], we propose specific byte distribution features for anonymous proxy traffic. The features proposed by Wu et al. [26] are fixed. Consequently, they can be easily bypassed with targeted modifications. ProxyKiller extracts features such as byte content distribution, payload length distribution, and printable characters, allowing for automatic representation learning of these features through neural networks. To accelerate processing, we extract only the first six bytes from the payload of each encrypted packet and categorize these byte features into three distinct groups.

Table 1. Detailed Features of Network Traffic Analysis

ID	Feature Name	Directions	Category
1	Protocol	-	Content Features
2	IP Version	-	Content Features
3-4	Duration (ms)	Bidirectional, Single	Temporal Features
5-14	Packet Count	Bidirectional, Single	Spatial Features
15-16	Bytes Size	Bidirectional, Single	Spatial Features
17-24	Packet Interval	Bidirectional, Single	Temporal Features
25-26	SYN Packet Count	Bidirectional, Single	Content Features
27-28	CWR Packet Count	Bidirectional, Single	Content Features
29-30	ECE Packet Count	Bidirectional, Single	Content Features
31-32	URG Packet Count	Bidirectional, Single	Content Features
33-34	ACK Packet Count	Bidirectional, Single	Content Features
35-36	PSH Packet Count	Bidirectional, Single	Content Features
37-38	RST Packet Count	Bidirectional, Single	Content Features
39-40	FIN Packet Count	Bidirectional, Single	Content Features
41	Application Name	-	Content Features
42	Application Category Name	-	Content Features
43	Application is Guessed	-	Content Features
44	Payload Size in Bytes	-	Byte Features
45-300	Payload Bytes Content Distribution	-	Byte Features
301-379	Payload Length Distribution	-	Byte Features
380	Bytes Average Popcount	-	Byte Features
381	Bytes Average Printable Count	-	Byte Features

The first part consists of 256-dimensional byte content distribution features, representing the cumulative counts of individual byte values ranging from 0 to 255. The second part focuses on payload length distribution. To accommodate different MTU configurations across various network environments, we set the maximum length at 1560 and compress it at 1:20 to reduce feature dimensionality. After compression, this part has 79 dimensions, including zero payload length. Neural networks could extract latent traffic features from the first two parts.

The third part consists of the payload size in bytes, the average popcount values, and the average printable character count. Printable characters refer to characters with corresponding plaintext values in the ASCII table. Even though all traffic is encrypted, the first few bytes of non-proxy traffic using standard TLS protocols often remain in plaintext. In contrast, traffic packets encrypted through anonymous proxy protocols generally exhibit completely random characteristics and do not display plaintext features, making them distinct from regular traffic.

The popcount refers to counting the number of set bits (bits that are 1). Average popcount values can be considered a measure of payload content entropy. This could be used to identify whether traffic content exhibits random characteristics. These features will significantly help identify proxy traffic.

Edge Construction. We employ two mechanisms for constructing edges to better characterize the relationships between different flows. The first type is the actual edge. If there is a flow from node A to node B , it indicates an actual interaction, and thus an undirected edge will be established between node A and node B .

The second type is the correlation edge, connecting nodes with similar or identical attributes. Based on our observation, proxy servers often belong to the same Class C subnet. Specifically, we first determine server and client nodes based on the magnitude of their port numbers. Then, we connect nodes with the same server IP address via an undirected edge, and nodes with server IP addresses in the same Class C network are interconnected. Moreover, since the proxy software used by the client remains constant, the JA3 fingerprint [21], which characterizes the client certificate information, can be used to link all proxy traffic from the same client. **These association fields are crucial for identifying anonymous proxy traffic through traffic behavior graphs.** Finally, we also link flows that contain the same domain names, including the query name field in the DNS protocol, the server name indication in the TLS protocol, and the hostname field in the HTTP protocol. Using plaintext hostname, we can correlate traffic masquerading as HTTP protocol, thereby supporting differentiation between plaintext and encrypted traffic. To reduce the complexity of graph connections, we connect different nodes in sequential order for multiple nodes with similar or identical attributes based on the timeline instead of fully connecting each pair.

To ensure a balanced number of nodes in the graph and maintain inference speed, we set an upper limit, L , on the number of nodes in a graph. That is, when the number of nodes with correlated relationships exceeds L , we perform a split, dividing them into different traffic behavior graphs. In this paper, L is set to 500.

4.3 Graph-Level Traffic Feature Representation Learning

GNN Encoder. The role of the GNN encoder is to learn the connection behavior features between different nodes based on the initial node embeddings through gradient descent. This process results in a comprehensive feature vector that integrates both traffic nodes' dynamic behavior features and their static statistical features. Specifically, we employ a uniform random sampling strategy to obtain a subset of masked nodes, V , from the traffic behavior graph G . In our implementation, we use the GIN model [28], which has shown the best experimental performance, as the encoder for ProxyKiller. The GNN encoder represents the entire graph's node vectors as a feature vector H .

GNN Decoder. The GNN decoder maps the node representations H obtained by the encoder back to the new node features, and it is used only during the training process. Unlike the simple decoders used in NLP tasks (such as MLP), the reconstruction targets in graph learning tasks contain information at a higher semantic level due to their dimensional richness. Therefore, ProxyKiller employs a single-layer graph neural network for vector decoding. Through experimental verification, we have chosen the best-performing GIN model [28] as the decoder for ProxyKiller. We re-mask the encoder nodes based on the masked node indices. The GIN model can reconstruct the feature vectors of the masked nodes by leveraging the distribution of unmasked features among the neighbors of the nodes in the flow connection behavior. This ability helps the graph encoder learn more advanced feature information.

Loss Function Settings. To adapt to graph autoencoder learning, ProxyKiller employs Scaled Cosine Error (SCE) [15] to evaluate reconstruction effects. This error adjustment can be considered a form of adaptive weight modification. The model adjusts its weights based on the reconstruction error of each sample.

Random Forest-Based Graph Pooling. Traditional graph classification models often use simple graph pooling functions to pool all nodes in a graph, such as graph max pooling and graph average pooling. To more comprehensively consider the status of each node within the graph, we employ a random forest model to learn the node vectors within the graph, assigning a category label to each node. Subsequently, an ensemble decision method is used to select the category with the most labels as the predicted label for the entire graph. Please note that in the prediction tasks during the inference stage, the graph encoder is only used to generate node embedding vectors and performs no masking.

Global Decision-Making Mechanism. To enhance the model’s accuracy in real-world environments and reduce false positives, we have designed a global decision-making mechanism based on the perspective of traffic graph association after the classification model. Initially, we aggregate the traffic behavior graphs that are to be predicted, utilizing features including JA3 fingerprints as outlined in the edge construction section of Sect. 4.2. This process results in the formation of grouped traffic behavior graphs. Then, an ensemble decision-making mechanism is applied to traffic behavior graphs within the same group, using the majority class label as the label for the entire group of traffic graphs. This holistic prediction strategy is intended to facilitate the practical implementation of the model and reduce false positives.

5 Evaluation

5.1 Dataset Setup

We employed three distinct types of datasets to test our model comprehensively. Firstly, we selected traffic data generated by six types of anonymous proxy tools

from the dataset Datacon2021 [8]. These six proxy tools are V2Ray (Version 4.31.0), Clash (Version 0.18.1), Lantern (Version 6.9.4), Shadowsocks (Version 4.4.0.0), Netch (Version 1.9.2), and Firefox (Version 93.0).

The second dataset, AnonProxy2023¹, is a self-collected classification dataset of anonymous proxy traffic protocols, including the four most popular types of anonymous proxy protocols (Shadowsocks [7], ShadowsocksR [5], VMess [20], and Trojan [12]) and non-proxy traffic. We manage access to various websites by utilizing MonkeyRunner on three mobile devices. We configure the Clash software on these devices to facilitate anonymous proxies and employ IPtables to suppress system background traffic, thereby ensuring the accuracy of the dataset’s annotations. The configuration of the Clash software is set to global proxy to prevent issues like DNS leakage and maximize the privacy protection of the anonymous proxies. We capture traffic on the router side using Tshark and filter it by IP address to obtain the specific traffic from the tested mobile devices. The detailed configurations of the four protocols are as follows: both the Shadowsocks and Trojan protocols adhere to their standard original specifications; the ShadowsocksR protocol utilizes the OBFS plugin in HTTP simple mode; and the Vmess protocol uses WebSocket for obfuscation.

The third dataset is the LFETT2021 dataset [14]. It contains proxy traffic data for various application behaviors using the ShadowsocksR and VMess protocols, covering 137 categories under various proxy settings. Specifically, DataCon2021 comprises 0.95 GB of data and 62,931 flows, AnonProxy2023 contains 17.41 GB and 290,969 flows, and LFETT2021 comprises 6.03 GB and 121,268 flows.

5.2 Ethical Concerns

During the data collection process, we considered ethical issues. The AnonProxy2023 dataset was generated by test machines and contains no personal information. The Datacon2021 and LFETT2021 datasets are public datasets. Thus, they do not involve any issues of personal information leakage or infringement. To support the reproducibility of our paper, we have made the AnonProxy2023 dataset publicly available. Due to privacy concerns and ethical considerations, we have decided not to release our code.

5.3 Baselines and Metrics

To comprehensively evaluate ProxyKiller’s performance, we used three categories of baseline models. The first category includes machine learning-based models such as CUMUL [19] and BIND [1], which have demonstrated superior performance in various encrypted traffic classification tasks. The second category comprises deep learning models, notably GraphDapp [23] and FS-Net [18]. GraphDapp utilizes graph neural networks to classify traffic at the single flow level.

¹ The AnonProxy2023 dataset can be found at <https://github.com/MrRobotsAA/AnonProxy2023-Dataset>. Researchers who use the dataset should indicate the data source by citing this paper.

FS-Net employs a multi-layer encoder-decoder architecture to mine the latent sequential features of flows deeply and incorporates a reconstruction mechanism to enhance the effectiveness of the features. The third category includes proprietary models specifically developed for proxy traffic, such as SSAPPIDENTIFY [25]. This model is specifically designed to classify anonymous proxy (Shadowsocks) traffic using statistical traffic features.

We use four commonly used classification metrics to evaluate ProxyKiller: Accuracy (Acc), Recall (Re), Precision (Pr), and F1 Score (F1). In multi-class calculations, we calculate the mean based on macro averages. We set a maximum training epoch to 300 in the training phase and used five-fold cross-validation for every model. The optimizer is chosen as Adam, with a learning rate of 0.001. To ensure a fair comparison, we expand the results of the traffic behavior graphs to the flow level to align with other flow-level classification models. This means we label all the flow results in the same graph with the overall graph's label.

5.4 Analysis of Experimental Results

Analysis of Comparison Experiment Results. We compare ProxyKiller with five state-of-the-art traffic classification methods on three datasets. As shown in Table 2 and Table 3, ProxyKiller achieves the best results in all metrics on the three datasets. The F1 score improvements range from 2.77% to 34.37% (Datacon2021), 1.76% to 10.82% (AnonProxy2023), and 24.07% to 53.1% (LFETT2021). Most models perform poorly in the LFETT2021 dataset. This dataset is primarily used for more challenging fine-grained anonymous proxy behavior classification, such as sending messages using Facebook. Other models do not consider the correlation between flow samples, which results in them failing to extract higher-order topological structure features.

Notably, based on the results, ProxyKiller's improvement appears to be marginal compared to other models on partial datasets. However, given the vast scale of traffic data in the real world, even a 1% decrease in precision can significantly increase the volume of false positives. Therefore, the potential for deploying ProxyKiller in real-world applications is substantially greater than other models.

Analysis of Ablation Experiment Results. Based on the LFETT2021 dataset, we conducted ablation studies on the key modules in ProxyKiller, including the newly added byte features, the enhanced graph representation learning model, and the global decision-making mechanism (GDMM) from the perspective of traffic behavior graph association.

As shown in Fig. 4, ProxyKiller's evaluation metrics decrease by 3.72%-4.6% after removing byte features. By replacing different GNN models (GCN [16], GAT [24]), we find that our model improved all metrics by 3.84%-8.88% compared to other GNN models. By ablating the global decision-making mechanism, we found that applying this mechanism improved ProxyKiller's evaluation metrics by 1.23%-1.71%. These findings indicate that the proposed byte features, the

Table 2. Performances of ProxyKiller and other models on Datacon2021 and AnonProxy2023 datasets.

Methods	Datacon2021				AnonProxy2023			
	Acc	Pr	Re	F1	Acc	Pr	Re	F1
CUMUL [19]	0.6742	0.6488	0.6654	0.6527	0.9125	0.8959	0.9072	0.9013
BIND [1]	0.7892	0.8156	0.8093	0.8107	0.8842	0.8891	0.8816	0.8853
FS-Net [18]	0.9679	0.9686	0.9603	0.9642	0.9677	0.9643	0.9682	0.9661
GraphDapp [23]	0.7436	0.6994	0.6845	0.6858	0.9175	0.9052	0.9134	0.9087
SSAPPIDENTIFY [25]	0.9783	0.9752	0.9645	0.9687	0.9783	0.9764	0.9731	0.9759
ProxyKiller	0.9924	0.9967	0.9962	0.9964	0.9949	0.9938	0.9912	0.9935

Table 3. Performances of ProxyKiller and other models on LFETT2021 dataset.

Methods	LFETT2021			
	Acc	Pr	Re	F1
CUMUL [19]	0.5472	0.4731	0.4386	0.4451
BIND [1]	0.5764	0.5639	0.5547	0.5582
FS-Net [18]	0.5026	0.5178	0.5142	0.5149
GraphDapp [23]	0.5204	0.4925	0.4874	0.4896
SSAPPIDENTIFY [25]	0.7675	0.7212	0.7387	0.7354
ProxyKiller	0.9764	0.9763	0.9760	0.9761

improved graph representation learning model, and the global decision-making mechanism all contribute to ProxyKiller’s ability to represent proxy traffic.

Analysis of Robustness Experiment Results. We conduct this experiment on the LFETT2021 dataset to test our model’s robustness. In this experiment, the training set proportion is gradually decreased to test the model against a large data volume in a real-world environment.

Figure 5 shows that ProxyKiller exhibits the most minor performance degradation as the number of test sets gradually reduces. When the test set ratio reaches 95%, its F1 score remains 90.36%. These results indicate that ProxyKiller is more robust than other models and can potentially be applied to real-world environments. Compared to different models, ProxyKiller poses a more significant threat to anonymous proxies.

5.5 Evaluation of Basic Node Feature Importance

To better understand the impact of basic node features in the traffic graph on the model’s performance, we evaluated the importance of node features in addition to the proxy behavior connection features using the interpretability of the Random Forest model. The assessment results are shown in Table 4.

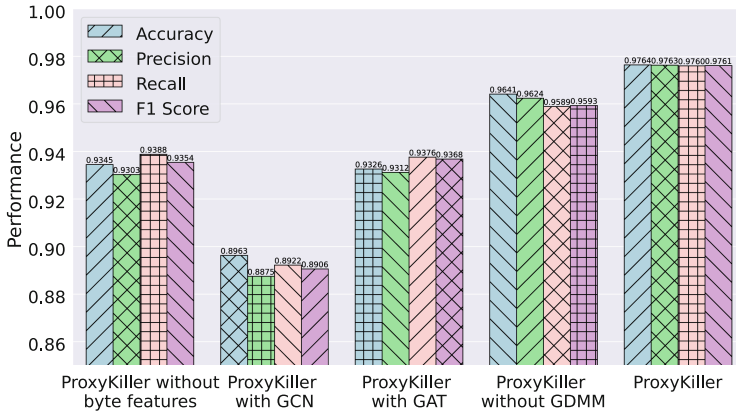


Fig. 4. Ablation results of ProxyKiller.

Table 4. Top 20 Features of Each Dataset.

Rank	AnonProxy2023	Datacon2021	LFETT2021
1	Payload Bytes Content Distribution (130)	Packet Interval (max)	Duration
2	Bytes Average Popcount	Duration	Packet Interval (max)
3	Payload Bytes Content Distribution (84)	Packet Interval (mean)	Packet Interval (stddev)
4	Payload Bytes Content Distribution (32)	Packet Interval (min)	Packet Interval (mean)
5	Payload Bytes Content Distribution (47)	Packet Count (max)	Packet Interval (max)
6	Bytes Average Printable Count	Packet Count (stddev)	Packet Count (max)
7	Payload Bytes Content Distribution (1)	Packet Count (mean)	Duration
8	Payload Bytes Content Distribution (3)	Bytes Size	Packet Count (mean)
9	Packet Count (mean)	Packet Count (min)	Packet Count (stddev)
10	Payload Bytes Content Distribution (0)	Bytes Average Popcount	Packet Interval (mean)
11	Payload Bytes Content Distribution (37)	ACK Packet Count	Packet Interval (stddev)
12	Payload Bytes Content Distribution (69)	Packet Count (max)	Bytes Size
13	Packet Count (max)	Payload Bytes Content Distribution (3)	Payload Size in Bytes
14	Payload Bytes Content Distribution (254)	Payload Bytes Content Distribution (0)	Bytes Average Popcount
15	Payload Bytes Content Distribution (23)	Bytes Size	Packet Count (max)
16	Packet Count (stddev)	Packet Interval (stddev)	Packet Count
17	Packet Count (max)	Payload Bytes Content Distribution (23)	Packet Count (min)
18	Payload Bytes Content Distribution (2)	ACK Packet Count	ACK Packet Count
19	Packet Count (min)	PSH Packet Count	Packet Count (mean)
20	Payload Bytes Content Distribution (10)	Payload Size in Bytes	Packet Interval (min)

The data analysis in the table above indicates that byte distribution features significantly impact the classification of the proxy protocol. This is because different proxy protocols may use obfuscation padding strategies, leading to an imbalance in the proportion of specific bytes. Additionally, the importance rankings of the printable characteristics of leading bytes (Bytes Average Printable Count) and entropy distribution (Bytes Average Popcount) are relatively high. This suggests that the non-plaintext nature and high entropy values in the headers of proxy traffic packets are crucial attributes for distinguishing different proxy protocols. Furthermore, statistical features related to the duration of bidirectional connections (Mean Packet Count, Max Packet Count) are highly significant in both proxy behavior and proxy tool classification tasks. Proxy traffic often has

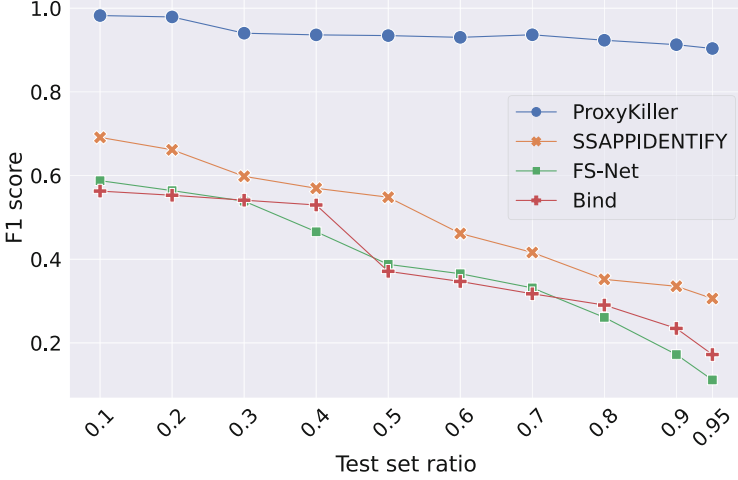


Fig. 5. Robustness experiment results of ProxyKiller.

long connection attributes compared to regular non-proxy traffic, and different application behaviors result in substantial variations in communication connection times, making the connection duration of different behaviors under proxies distinctly distinguishable. The significant differences in feature importance rankings across different tasks demonstrate that ProxyKiller can adaptively select key classification features based on the characteristics of each task.

6 Discussion

This paper primarily discusses the feasibility of attacking based on the connection behavior of proxy traffic and designs a model based on graph neural networks for detection. Although this model requires more time than traditional models, experiments have proven its high detection accuracy, making it feasible for deployment in real-world environments. Extensive testing has shown that mainstream anonymous proxy protocols’ privacy protection level needs to be revised to evade more refined censorship. **We hope this paper will draw the attention of the proxy tool development community and encourage adding obfuscation features related to connection behavior features in the design of proxy protocols.**

6.1 Time Overhead

According to our experimental measurements, ProxyKiller processes data at a speed of 2.59 GB/min using the NFStream feature extraction tool. Our Linux server experimental environment consists of 188 GB RAM (Ubuntu 16.04.6 LTS), and during the experiments, the model’s memory usage remained within 50 GB.

The performance evaluation results indicate that the feature extraction part is the key factor affecting the model's prediction performance, with the model evaluation inference speed being 1.036 s per GB. The time overhead ratio between the feature extraction and model inference parts is approximately 22:1. Currently, for development efficiency, we primarily use Python for feature extraction. However, implementing feature extraction in a compiled language like C++ would significantly enhance the model's runtime performance.

Moreover, compared to tasks such as malicious traffic detection, proxy traffic detection has the uniqueness of long-duration connections. For instance, the work of Xue et al. [29] pointed out that some censorship systems only need to detect about one-quarter of all traffic. For example, suppose a user generates 2000 flows while accessing an anonymous proxy. In that case, the attacker only needs to detect one of these flows to discover the user's censorship evasion behavior by IP address. Therefore, in practical deployment, attackers can adopt a data-dropping strategy to reduce the volume of data to be detected. Attackers can also maintain a domain allowlist, allowing traffic within the allowlist to pass directly.

6.2 Possible Resistance Measures

To inspire the design of anti-censorship protocols, we briefly discuss possible defense measures for defenders.

Alternating Multi-server Transmission. The effectiveness of ProxyKiller fundamentally relies on correlating different flow samples. Therefore, weakening the correlation between different flow samples can significantly impact the model's effectiveness. Nevertheless, this task is challenging, as proxy traffic can generally be well-correlated simply through the server IP. If proxy tools could implement a rotation among multiple proxy servers, with each server configured differently, then the differences between various flows from different servers would greatly vary. This strategy will increase the difficulty of multi-flow correlation. Specifically, dynamic IP address changes are a good way to implement this.

Mimicry Defense. Current proxy tools often maintain a single configuration across multiple clients. It is possible to implement a client traffic awareness module and combine it with the WebSocket to mimic client-whitelisted traffic. In that case, it will significantly threaten the detection model. Firstly, current traffic-based detection models usually maintain an allowlist, which could allow anonymous proxy traffic to escape detection. Secondly, in such cases, ProxyKiller's correlation mechanism would build both regular and anonymous proxy traffic into the same graph, fundamentally undermining the accuracy of our model. Simulating the byte distribution of normal traffic is also a more effective obfuscation method from the point of view of the importance of the traffic features that affect ProxyKiller.

Using Advanced Encryption Protocols. The best disguise is not to use obfuscation. From the development of anonymous protocols, when proxy developers create new protocols, various vulnerabilities often emerge. For instance, Wu et al. [26] pointed out that the GFW (Great Firewall) identifies fully encrypted anonymous proxy traffic through global high-entropy characteristics. Proxy developers can directly use well-designed advanced encryption protocols instead of complex traffic obfuscation methods. TLS 1.3 is a protocol with a higher level of encryption. Not only does TLS 1.3 enhance security and improve performance and privacy protection by simplifying the handshake process, enforcing forward secrecy, removing insecure algorithms, and enhancing privacy protections. With the TLS 1.3 protocol configured, even SNI (Server Name Indication) information will be encrypted, making association methods based on plaintext domain information ineffective, thereby further increasing the difficulty of multi-flow association. Thus, using TLS 1.3 is an effective way to enhance communication security and privacy protection.

Overall, proxy developers should follow the above suggestions and adopt more sophisticated evasion strategies, such as dynamic IP address changes, using more advanced encryption protocols, and implementing more complex traffic obfuscation methods based on byte distribution features. In that case, ProxyKiller’s performance will be significantly impacted. These strategies would undermine the model’s fundamental implementation motivation (classification based on traffic association behavior features). However, implementing the above evasion strategies will also pose considerable challenges for anonymous proxy development.

6.3 Incremental Learning Mechanism

Anonymous proxy technologies evolve rapidly, often rendering trained models obsolete. However, ProxyKiller employs a self-supervised graph autoencoder model, which enables it to perform incremental learning. To classify a new proxy protocol X , we need first to gather annotated proxy traffic data for the corresponding protocol. Then, the pre-trained model parameters of the graph encoder are loaded to obtain the embedding features of the proxy traffic data. These features are then combined with data from other proxy protocols to retrain the Random Forest model. This process results in a new ProxyKiller model capable of classifying the new proxy protocol X .

7 Conclusion

To validate the security of anonymous proxies, we propose an attack model called ProxyKiller, which targets anonymous proxy traffic. This model is based on the fusion of temporal, spatial, and byte traffic features and the correlation mining of multi-flow behavior features. ProxyKiller utilizes graph neural networks to learn and integrate heterogeneous traffic features and correlations between different flows. Experimental results from three real datasets demonstrate that

ProxyKiller exhibits excellent predictive performance and robustness across various anonymous proxy traffic classification tasks.

These results indicate that existing implementations of anonymous proxies still pose significant risks. Attackers can even identify specific user behaviors when using anonymous proxies through traffic behavior graph analysis. **We hope the publication of this paper will attract the attention of the proxy development community, promote the development of related protocol designs, and better protect user privacy.**

Acknowledgments. This work is supported by the Scaling Program of Institute of Information Engineering, CAS (Grant No. E3Z0041101).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Al-Naami, K., et al.: Adaptive encrypted traffic fingerprinting with bi-directional dependence. In: Proceedings of the 32nd Annual Conference on Computer Security Applications (2016)
2. Alice, Bob, Carol, Beznazwy, J., Houmansadr, A.: How China Detects and Blocks Shadowsocks. In: Proceedings of the ACM Internet Measurement Conference, pp. 111–124 (2020)
3. Aouini, Z., Pekár, A.: NFStream: a flexible network data analysis framework. *Comput. Networks* **204**, 108719 (2022)
4. Azab, A., Khasawneh, M.T., Alrabaa, S., Choo, K.K.R., Sarsour, M.: Network traffic classification: techniques, datasets, and challenges. *Digit. Commun. Networks* (2022)
5. Breakwall: ShadowsocksR Project. <https://github.com/shadowsocksrr>. Accessed 12 Sept 2023
6. Chen, H.Y., Lin, T.N.: The challenge of only one flow problem for traffic classification in identity obfuscation. *Environments* **9**, 84110–84121 (2021)
7. Clowindy: Shadowsocks Project. <https://github.com/shadowsocks> (2012). Accessed 12 Sept 2023
8. Community, D.: DataCon Open Dataset - DataCon2021 - Encrypted Proxy Traffic Dataset. <https://datacon.qianxin.com/opendata/openpage?resourcesId=10> (2021). Accessed 12 Sept 2023
9. Frolov, S., Wampler, J., Wustrow, E.: Detecting Probe-resistant Proxies. In: Network and Distributed System Security Symposium (2020)
10. Frolov, S., Wustrow, E.: HTTPPT: a probe-resistant proxy. In: 10th USENIX Workshop on Free and Open Communications on the Internet (FOCI 20) (2020)
11. Gorishniy, Y., Rubachev, I., Khrulkov, V., Babenko, A.: Revisiting Deep Learning Models for Tabular Data. vol. abs/2106.11959, pp. 18932–18943 (2021)
12. GreaterFire: Trojan-GFW Project. <https://github.com/trojan-gfw>. Accessed 12 Sept 2023
13. Grinsztajn, L., Oyallon, E., Varoquaux, G.: Why do tree-based models still outperform deep learning on typical tabular data? *Adv. Neural. Inf. Process. Syst.* **35**, 507–520 (2022)

14. Gu, Z., Gou, G., Hou, C., Xiong, G., Li, Z.: LFETT2021: A large-scale fine-grained encrypted tunnel traffic dataset. In: 2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), pp. 240–249 (2021)
15. Hou, Z., Liu, X., Cen, Y., Dong, Y., Yang, H., Wang, C., Tang, J.: GraphMAE: self-supervised masked graph autoencoders. In: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp. 594–604 (2022)
16. Kipf, T., Welling, M.: Semi-supervised classification with graph convolutional networks. ArXiv **abs/1609.02907** (2016)
17. Lichy, A., Bader, O., Dubin, R., Dvir, A., Hajaj, C.: When a RF Beats a CNN and GRU, together - a comparison of deep learning and classical machine learning approaches for encrypted malware traffic classification. *Comput. Secur.* **124**, 103000 (2022)
18. Liu, C., He, L., Xiong, G., Cao, Z., Li, Z.: FS-Net: a flow sequence network for encrypted traffic classification. In: IEEE INFOCOM 2019 - IEEE Conference on Computer Communications, pp. 1171–1179 (2019)
19. Panchenko, A., et al.: Website fingerprinting at internet scale. In: Network and Distributed System Security Symposium (2016)
20. Raymond, V.: Project V. <https://github.com/v2ray>. Accessed 12 Sept 2023
21. Salesforce: JA3: A method for fingerprinting SSL clients (2024). <https://github.com/salesforce/ja3>. Accessed 17 Apr 2024
22. Shadowsocks-NET: Shadowsocks 2022 Edition. <https://github.com/Shadowsocks-NET/shadowsocks-specs/blob/main/2022-1-shadowsocks-2022-edition.md> (2022). Accessed 17 Apr 2024
23. Shen, M., Zhang, J., Zhu, L., Xu, K., Du, X.: Accurate decentralized application identification via encrypted traffic analysis using graph neural networks. *IEEE Trans. Inf. Forensics Secur.* **16**, 2367–2380 (2021)
24. Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y.: Graph attention networks. ArXiv **abs/1710.10903** (2017)
25. Wang, S., Yang, C., Guo, G., Chen, M., Ma, J.: SSAPPIDENTIFY: a robust system identifies application over Shadowsocks’s traffic. *Comput. Networks* **203**, 108659 (2021)
26. Wu, M.L., et al.: How the great firewall of china detects and blocks fully encrypted traffic. In: USENIX Security Symposium (2023)
27. Xie, J., Li, S., chun Yun, X., Zhang, Y., Chang, P.: HSTF-Model: an HTTP-based Trojan detection model via the Hierarchical Spatio-temporal Features of Traffics. *Comput. Secur.* **96**, 101923 (2020)
28. Xu, K., Hu, W., Leskovec, J., Jegelka, S.: How powerful are graph neural networks? *Abs/1810.00826* (2019)
29. Xue, D., Kallitsis, M., Houmansadr, A., Ensafi, R.: Fingerprinting Obfuscated Proxy Traffic with Encapsulated TLS Handshakes (2024)
30. Zhao, P., Gou, G., Liu, C., Guan, Y., Cui, M., Xiong, G.: TMT-RF: tunnel mixed traffic classification based on random forest. In: Security and Privacy in Communication Networks (2021)



Confidential and Verifiable Machine Learning Delegations on the Cloud

Wenxuan Wu^{1(✉)}, Soamar Homs², and Yupeng Zhang³

¹ Texas A&M University, College Station, USA
ww6726@tamu.edu

² Air Force Research Laboratory / Information Directorate,
Rome, NY, USA
soamar.homs@us.af.mil

³ University of Illinois Urbana-Champaign, Champaign, USA

Abstract. With the growing adoption of cloud computing, the ability to store data and delegate computations to powerful and affordable cloud servers have become advantageous for both companies and individual users. However, the security of cloud computing has emerged as a significant concern. Particularly, Cloud Service Providers (CSPs) cannot assure data confidentiality and computations integrity in mission-critical applications. In this paper, we propose a confidential and verifiable delegation scheme that advances and overcomes major performance limitations of existing Secure Multiparty Computation (MPC) and Zero Knowledge Proof (ZKP). Secret-shared Data and delegated computations to multiple cloud servers remain completely confidential as long as there is at least one honest MPC server. Moreover, results are guaranteed to be valid even if all the participating servers are malicious. Specifically, we design an efficient protocol based on interactive proofs, such that most of the computations generating the proof can be done locally on each server. In addition, we propose a special protocol for matrix multiplication where the overhead of generating the proof is asymptotically smaller than the time to evaluate the result in MPC. Experimental evaluation demonstrates that our scheme significantly outperforms prior work, with the online prover time being 1–2 orders of magnitude faster. Notably, in the matrix multiplication protocol, only a minimal 2% of the total time is spent on the proof generation. Furthermore, we conducted tests on machine learning inference tasks. We executed the protocol for a fully-connected neural network with 3 layers on the MNIST dataset and it takes 2.6 s to compute the inference in MPC and generate the proof, 88× faster than prior work. We also tested the convolutional neural network of Lenet with 2 convolution layers and 3 dense layers and the running time is less than 300 s across three servers.

Keywords: Zero-Knowledge Proof · Privacy-preserving Machine Learning · Secure Multiparty Computations

1 Introduction

Cloud computing has revolutionized the way organizations, businesses and individuals store, access, and process data. Rather than relying on traditional data centers that are centralized, expensive to setup, and costly to maintain and run, cloud computing offers a remote access to unlimited resources available on demand over the internet. Although cloud computing has numerous advantages, including elasticity, scalability, and cost savings, it comes with major cybersecurity risks and concerns. Specifically, they require end-to-end confidentiality of the data, meaning that the data should remain confidential from Cloud Services Providers (CSPs) and the integrity of the data and computations, meaning that the data and results should be valid even if the cloud servers are compromised.

We thus seek to address the limitations of the state of the art applied cryptographic methods and to build the necessary building blocks that enable the development of secure and efficient applications such as Machine Learning (ML) evaluation. We primarily focus on confidential and verifiable ML prediction outsourced to commercial CSPs by combining primitives from secure multiparty computations (MPC) with zero-knowledge proofs (ZKP). Figure 1 illustrates an example of a user, with limited resources and energy, on a task that needs to utilize a ML model. To ease the burden of performing local data storing and processing, our scheme can delegate ML inference to multiple CSPs via methods, such as additive secret-sharing [32], to keep the model and data confidential. After the user secret-shares the data, the cloud servers compute the ML inference using an MPC protocol. In order to verify the correctness of the ML prediction, the servers generate a ZKP via an MPC protocol. Finally, the user reconstructs and verifies the results.

Attack Model and Security Guarantees. In our setting, we consider attackers that can compromise any number of cloud servers, and which behave arbitrarily once compromised. First, we aim to guarantee the confidentiality of both the data and ML model in the presence of malicious attackers compromising all but one server under MPC protocols [23]. Second, we want to ensure the integrity of the ML inference results even if all servers are compromised. This property is not satisfied when only using malicious MPC protocols. We will make the

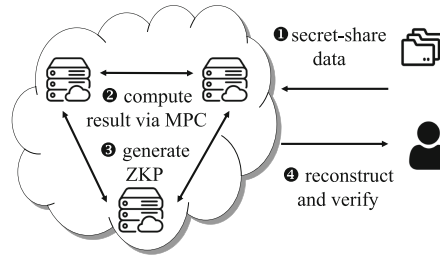


Fig. 1. Confidential and Verifiable Delegation.

MPC servers generate a ZKP to convince the verifier about the correctness of the result. Consequently, our scheme guarantees both the confidentiality of data and ML model, as long as there is at least one honest server, and the integrity of the inference results even if all servers are malicious.

Applications. One concrete use case of the proposed scheme is on the ZKP generation of ML inferences for blockchain applications. ZKP plays an important role to perform computations off-chain and validate proofs on-chain, and has lead to massive adoptions of zkRollups and active developments of zkEVMs. Recently, the focus has shifted to ML functionalities allowing smart contracts to interact with ML models and use the result of ML inferences, see [1] and [3]. As generating ZKPs is very resource intensive, our scheme can be used to delegate the generation of ZKP for ML to cloud servers, while preserving the confidentiality of both the ML model and the users' data. A client with limited computing resource can obtain a ZKP from the cloud servers and post it on the blockchain, without revealing the sensitive information to the servers.

In this paper, we have made the following contributions:

- We proposed a confidential and verifiable outsourcing scheme for secure computations that are modeled as arithmetic circuits. The scheme is based on generic MPC protocols, such as SPDZ [23], and a ZKP scheme based on interactive proofs (IP) [19]. We designed a special MPC protocol to compute the messages of the IPs, such that most of the computations are done locally on each prover. This can greatly reduce the time to generate proof in a low bandwidth setting.
- We introduced a novel ZKP protocol for efficiently verifying matrix multiplication operations in ML. Given the result of the matrix multiplication of two $m \times m$ matrices computed via MPC, the additional computation and communication complexities to generate the proof are only $O(m^2)$ and $O(m)$, respectively. To the best of our knowledge, this is the first method that can generate the ZKP with less overhead and response time than those incurred when computing matrix multiplication over MPC.
- We implemented the system of confidential and verifiable ML delegation and evaluated its performance. Experiments show that our scheme improves the running time (the online prover time of MPC) of existing collaborative ZKP [28] by one to two orders of magnitude. Furthermore, for matrix multiplications, the running time of our scheme is completely dominated by the MPC evaluation. The additional time to ensure the integrity of the result using ZKP is only 2% of the total time. On a fully-connected neural network inference task with 3 layers, it only takes 2.6s to run the full protocol, which is $88\times$ faster than the prior work of [28]. On the convolutional neural network of Lenet [24], it takes 288s for one inference, which is $74.8\times$ faster than the prior work of [28].¹

¹ The full version of the paper can be found at <https://eprint.iacr.org/2024/537>.

Table 1. Comparison to existing schemes. C denotes the circuit size and d is the depth of the circuit. The matrices are of size $m \times m$, and a circuit computing the matrix multiplication naively is of size $C = O(m^3)$.

Scheme	Prover Time	Prover Communication	Proof Size	Verifier Time	Setup Type
zkSNARK [28]	$O(C \log C)$	$O(C)$	$O(1)$	$O(1)$	per-circuit
Plonk [28]	$O(C \log C)$	$O(C)$	$O(1)$	$O(1)$	universal
Marlin [10]	$O(C \log C)$	$O(C)$	$O(1)$	$O(1)$	universal
[21]	$O(C)$	$O(C)$	$O(\log C)$	$O(C)$	transparent
[11]	$O(C \log C)$	$O(C \log C)$	$O(\log^2 C)$	$O(\log^2 C)$	transparent
Ours (General)	$O(C)$	$O(C)$	$O(d \log C)$	$O(d \log C)$	universal
Ours (Matrix)	$O(m^2)$	$O(m)$	$O(\log m)$	$O(\log m)$	universal

1.1 Related Work

Papers that are mostly related to our work include [10, 11, 16, 21, 28, 31]. Schoenmakers et al. proposed a privacy-preserving verifiable computation scheme based on the SNARK [30] and an MPC protocol named Trinocchio [31]. Trinocchio assumes a circuit-specific trusted setup, while our scheme only requires a universal trusted setup. Another work, Kanjalkar et al. [21], introduced the concept of auditable MPC which proves to a third party that the results of an MPC protocol is correctly computed even if all the parties are malicious by generating the proof of the zkSNARK [9] using MPC. Ozdemir and Boneh [28] presented the notion of collaborative ZK based on Groth16 [20] and Plonk [15]. Later, Dayama et al. [11] considered the same setting and developed schemes based on interactive oracle proofs (IOP). More recently, Chiesa et al. [10] designed efficient schemes for zkSNARKs based on polynomial IOPs. Garg et al. [16] further improved the total running time utilizing multiple provers and the packed secret sharing techniques. None of the above mentioned papers have studied schemes based on interactive proofs. Moreover, all of them assume that the extended witness (i.e., all values in the circuit) has been computed, secret-shared among parties, and only consider the step of proof generation. By contrast, we take the circuit evaluation using MPC into account and based on which we develop an end-to-end IP scheme. Table 1 shows the comparison among existing schemes excluding the overhead of MPC. Our scheme has the best prover time which is linear in the size of the circuit. Moreover, the special protocol for matrix multiplication improves both the prover time and the communication by $O(m)$.

Concurrent work. In [26], Liu et al. proposed a collaborative ZK scheme based on the GKR protocol. The main protocol is similar to our approach for general circuits. They used packed secret sharing in order to achieve a faster running time, with a lower threshold on the number of malicious provers, while we use the standard additive sharing. Our main focus is on the machine learning delegations, and we propose a special protocol for matrix multiplications where the additional

overhead to generate ZKP is small. Similar to [16, 28], the experiments in [26] does not consider the phase of circuit evaluation in MPC either.

Verifiable computation on encrypted data. An alternative solution to ensure both confidentiality and integrity is via verifiable computation on encrypted data by combining ZKPs with homomorphic encryptions [7, 13, 14, 17]. However, due to the heavy mechanism, they mainly remain theoretical and cannot scale to the ML computations considered in this paper.

2 Preliminaries

We use capital letters, e.g., A , to represent matrices and bold lower-case letters, such as $\mathbf{a}$ to represent vectors. PPT stands for probabilistic polynomial time.

2.1 Interactive Proofs

Our scheme is built upon ZKPs based on interactive proofs and polynomial commitments, and we describe the protocols in this section.

The Sumcheck Protocol. The sumcheck protocol is a fundamental interactive proof protocol in which the prover $\mathcal{P}$ tries to convince the verifier $\mathcal{V}$ that the sum H of a polynomial $f : \mathbb{F}^\ell \rightarrow \mathbb{F}$ on a binary hypercube is

$$H = \sum_{b_1, b_2, \dots, b_\ell \in \{0, 1\}} f(b_1, b_2, \dots, b_\ell)$$

Notice that computing this sum directly by $\mathcal{V}$ will take 2^ℓ work. The sumcheck protocol delegates this work to $\mathcal{P}$ and allows $\mathcal{P}$ to generate the proof in ℓ rounds as shown in Protocol 2 in Appendix A.

The GKR Protocol. The GKR protocol [19] is an interactive proof protocol for layered circuits, which uses the sumcheck protocol as a building block. Let C be an arithmetic circuit with depth d over a finite field $\mathbb{F}$. The circuit is layered in such a way that the output wire from a gate at layer i can only be an input wire to a gate at layer $i - 1$. The GKR protocol starts from the layer 0 (output layer) and proves the correctness of the circuit computation one layer at a time, until it reaches layer d (input layer). Specifically, $\mathcal{P}$ first sends a claim of the output to $\mathcal{V}$. Then, $\mathcal{P}$ reduces the claim of the output layer to a claim of wire value at the layer below using sumcheck protocol. In the i -th round, $\mathcal{P}$ will generally reduce the claim about the values of wires at the i th layer to a claim about wire values at the $(i + 1)$ -th layer.

Formally, we use S_i to denote the number of gates at layer i , and $s_i = \lceil \log(S_i) \rceil$ denotes the number of bits that can represent all the gates in layer i . We also define a function $V_i : \{0, 1\}^{s_i} \rightarrow \mathbb{F}$ that maps an input gate's index to its wire value. We also define two functions $add_i, mult_i : \{0, 1\}^{s_{i-1} + 2s_i} \rightarrow \{0, 1\}$

that are called wiring predicates. For example, add_i takes three gates (a, b, c) such that a is from layer $i - 1$ and b, c are from layer i . add_i outputs 1 if and only if a is an addition gate whose 2 inputs are outputs from gate b and c ($mult_i$ is defined similarly). With all these definitions, V_i can be written as:

$$V_i(z) = \sum_{x,y \in \{0,1\}^{s_{i+1}}} (add_{i+1}(g, x, y)(V_{i+1}(x) + V_{i+1}(y)) + mult_{i+1}(g, x, y)(V_{i+1}(x)V_{i+1}(y))) \quad (1)$$

where $\tilde{V}_i$, $\tilde{add}_i$, and $\tilde{mult}_i$ are multilinear extensions of V_i , add_i , and $mult_i$. With Eq. 1, the prover $\mathcal{P}$ initially sends the claimed output of the circuit to $\mathcal{V}$. Specially, $\mathcal{P}$ defines a polynomial $\tilde{V}_0(x)$ based on the output wires' value. Then $\mathcal{V}$ takes $\tilde{V}_0(x)$ and computes $\tilde{V}_0(g)$ for $g \in \mathbb{F}^{s_0}$. Then, $\mathcal{P}$ and $\mathcal{V}$ engage in a sumcheck protocol to check this equation. The sumcheck in the i th layer takes $s_i + 2s_{i+1}$ rounds of interaction due to the number of variables in the polynomial. Recall at the end of each sumcheck, $\mathcal{V}$ needs to evaluate the polynomial $f_i(x, y)$ with randomness $w_1, w_2 \in \mathbb{F}^{2s_{i+1}}$, where w_1, w_2 are randomness selected in the sumcheck protocol. $\mathcal{V}$ can compute the wire predicate $\tilde{add}_i$, and $\tilde{mult}_i$ given they are public. $\mathcal{V}$ will need oracle access to values $\tilde{V}_{i+1}(w_1)$ and $\tilde{V}_{i+1}(w_2)$, which are reduced to a sumcheck protocol for layer $i + 1$ using a random linear combination. Due to space limit, please refer to [18, 33] for more details.

As shown in prior work [34, 35, 38], one can build a ZKP scheme by combining the GKR protocol with a polynomial commitment scheme. The prover commits to the witness of the circuit, executes the GKR protocol with the verifier, and finally opens the polynomial at a random point to check the correctness of the last claim of the GKR protocol. In this paper, we use the KZG polynomial commitment [22] and we present the protocol in Sect. 3.5. We denote this scheme as $(ZKP.\mathcal{P}, ZKP.\mathcal{V})$.

2.2 Secure Multiparty Computation

Secure multiparty computation [27, 36] allows a set of parties $\{P_1, \dots, P_n\}$ to jointly compute a function f over their inputs, and reveals nothing other than the output. In this paper, we use MPC protocols based on a linear secret sharing scheme and primarily focus on additive sharing. In an additive secret sharing scheme, a value a is split into $(\langle a \rangle_1, \langle a \rangle_2, \dots, \langle a \rangle_n)$ and distributed among all n parties such that $a = \langle a \rangle_1 + \langle a \rangle_2 + \dots + \langle a \rangle_n \pmod p$ in a field $\mathbb{F}_p$.

- **MPC.Add** $(\langle a \rangle, \langle b \rangle)$: Given the shares of two values $\langle a \rangle$ and $\langle b \rangle$, their addition $c = a + b$ can be computed locally by the parties as $\langle c \rangle = \langle a \rangle + \langle b \rangle$ without any interaction because of the linearity of the secret sharing scheme. Similarly, subtraction and multiplication by a public value can also be computed locally.
- **MPC.Mult** $(\langle a \rangle, \langle b \rangle)$: To compute the multiplication of two shared values, one can use Beaver's multiplication triplets [6]. In the offline phase, the parties precompute a multiplication triplet $\langle u \rangle, \langle v \rangle, \langle w \rangle$ such that $u \cdot v = w$. Then in the online phase, the parties exchange $\langle a \rangle + \langle u \rangle$ and $\langle b \rangle + \langle v \rangle$ and reconstruct

$e = a + u, f = b + v$. Finally, each party sets $\langle c \rangle = e \cdot \langle b \rangle - f \cdot \langle u \rangle + \langle w \rangle$ locally, and it can be shown that $c = a \cdot b$.

- **MPC.InnerProduct**($\langle \mathbf{a} \rangle, \langle \mathbf{b} \rangle$): The inner product of two shared vectors can be computed by applying **MPC.Mult** element-wise and adding the shares locally.
- **MPC.ScalarProduct**($\langle \mathbf{a} \rangle, \langle b \rangle$): The protocol multiplies every element of a shared vector by a shared value. It is realized using **MPC.Mult** element-wise.

3 Confidential and Verifiable Delegation

In this section, we present our confidential and verifiable delegation scheme. Section 3.1 gives the security definition of the scheme, and Sect. 3.2 gives an overview of our scheme combining MPC and ZKP. We present our scheme for general circuits in Sect. 3.3, and for matrix multiplications in Sect. 3.4.

3.1 Security Definitions

We first propose the security definition of a confidential and verifiable delegation scheme, which is adapted from collaborative zk-SNARKs proposed in [28].

Definition 1. *A confidential and verifiable delegation scheme for n provers, a verifier and a computation modeled as an arithmetic circuit C consists of 4 algorithms (Setup, Share, Prove, Verify):*

- **Setup**($1^\lambda, C$) $\rightarrow$ **pp**: *This algorithm outputs the public parameters.*
- **Share**(w) $\rightarrow$ $\langle w \rangle$: *This algorithm is called by the verifier to share the witness with the provers.*
- **Prove**($\langle w \rangle, C, x, \mathbf{pp}$) $\rightarrow$ (y, π) : *This algorithm allows the provers to jointly evaluate the circuit C and output the result y and a proof π .*
- **Verify**($y, x, \pi, \mathbf{pp}$) $\rightarrow$ $\{0, 1\}$ *This algorithm enables the verifier to verify the result and outputs 1 or 0.*

Completeness: *For all provers,*

$$\Pr \left[\begin{array}{l} \text{Setup}(1^\lambda, C) \rightarrow \mathbf{pp} \\ \text{Share}(w) \rightarrow \langle w \rangle : \text{Verify}(y, x, \pi, \mathbf{pp}) \rightarrow 1 \\ \text{Prove}(\langle w \rangle, C, x, \mathbf{pp}) \rightarrow (y, \pi) \end{array} \right] = 1$$

Knowledge Soundness: *For all x , for all sets of efficient algorithms $\mathbf{P} = \{P_1, \dots, P_n\}$, there exists an efficient extractor ε :*

$$\Pr \left[\begin{array}{l} \text{Setup}(1^\lambda, C) \rightarrow \mathbf{pp} \\ \mathbf{P}(C, x, \mathbf{pp}) \rightarrow (y, \pi) : \text{Verify}(y, x, \pi, \mathbf{pp}) \rightarrow 1 \\ \varepsilon(x, \pi, \mathbf{pp}) \rightarrow w \quad \wedge y \neq C(x, w) \end{array} \right] \leq \text{negl}(\lambda)$$

Protocol 1 (Confidential and Verifiable Delegation) Let C be a layered arithmetic circuit. Let w be the witness and x be the public input of C . The protocol is between a verifier $\mathcal{V}$ and n provers $P_1, \dots, P_n$.

- **Setup**(1^λ): run the **KeyGen** algorithm of the polynomial commitment and output pp .
- **Share**(w): $\mathcal{V}$ shares w to all the provers $P_1, \dots, P_n$ using a linear secret sharing.
- **Commit**($\langle w \rangle, x$): each prover defines the multilinear extension of $\langle w \rangle$ concatenated with $\langle x \rangle$ as $\langle \tilde{V}_d \rangle$. Each prover runs **Commit**($\langle \tilde{V}_d \rangle, \text{pp}$). Output the combined commitment com .
- **Prove**($\langle w \rangle, C, x, \text{pp}$)
 1. The provers execute an MPC protocol to evaluate the circuit to obtain $y = C(x, w)$ and the shares of each wire value in the circuit.
 2. The provers run the GKR protocol on shares to generate the proof.
 3. For the input layer, the provers run **Open**($\langle \tilde{V}_d \rangle, r_d, \text{pp}$), where r_d is the last random challenge for the input layer of the GKR protocol.
 4. The provers output y and all the proofs generated during the protocol.
- **Verify**($\text{com}, y, \pi, \text{pp}$): $\mathcal{V}$ verifies the proof of the GKR protocol and the proof of the polynomial commitment protocol. If all checks pass, output 1; otherwise, output 0.

***t*-Zero-Knowledge:** For all efficient adversary $\mathcal{A}$ corrupting $k \leq t$ provers $P_{i_1}, \dots, P_{i_k}$, there exists a simulator Sim such that for all x, w and C and all efficient distinguisher D

$$\Pr \left[\begin{array}{l} \text{Setup}(1^\lambda, C) \rightarrow \text{pp} \\ \text{Sim}(x, y, \langle w \rangle_{i_1}, \dots, \langle w \rangle_{i_k}, \text{pp}) \rightarrow \text{tr} : D(\text{tr}) = 1 \end{array} \right] \\ - \Pr \left[\begin{array}{l} \text{Setup}(1^\lambda, C) \rightarrow \text{pp} \\ \text{View}_{\mathcal{A}}(x, w) \rightarrow \text{tr} : D(\text{tr}) = 1 \end{array} \right] \leq \text{negl}(\lambda)$$

where tr is a transcript, $\text{View}_{\mathcal{A}}(x, w)$ denotes the view of $\mathcal{A}$ when provers $P_{i_1}, \dots, P_{i_k}$ interact with input x and witnesses w .

In Definition 1 above, the knowledge soundness guarantees that as long as the provers can generate a valid proof passing the verification, there exists an extractor that extracts the witness w such that $C(w, x) = y$ with an overwhelming probability. In other words, the protocol guarantees the integrity of the result y even if all provers are malicious, which is beyond the capability of malicious MPC. The t -zero-knowledge guarantees that the proof leaks no information about the secret witness w even if the adversary corrupts up to t provers. In the application of ML inference, w includes both the ML model and the data.

3.2 Overview of Our Scheme

In order to build a confidential and verifiable delegation scheme, we combine the cryptographic primitives of MPC and ZKP. As shown in Protocol 1, the verifier

first secret-shares the witness to all the provers. The provers then commit to the witness using a polynomial commitment on shared values. This step is necessary to derive randomness using the Fiat-Shamir heuristic [12] to make the protocol non-interactive, and also to allow the provers to have additional witness. Later, to perform the computation and generate a proof, the provers execute an MPC protocol to compute y without revealing w , and generate the proof using the shares of wire values in the circuit. As we are using the ZKP based on GKR and polynomial commitments, this step is divided into generating GKR proofs and polynomial commitment proofs on shared values in MPC.

The framework is similar to the constructions in [16, 28], where they use MPC to generate ZKPs based on zkSNARKs and Plonk. However, we are the first to consider interactive proofs and achieve better prover efficiency. Moreover, none of prior work considers the MPC step to evaluate the circuit. In the following, we present our protocols for GKR and polynomial commitments on shared values.

3.3 GKR in MPC for General Circuits

In this section, we present our protocol for the provers to compute the proofs of GKR when the values are secret-shared. We follow the two-phase approach in [35]. Due to space limit, we defer the original algorithms to the full version of the paper, and present our GKR protocol in MPC in this section. The key observation is that most of the steps in the algorithms are linear. For example, as the randomness is either sent by the verifier in the interactive setting or computed using Fiat-Shamir [12] in the non-interactive setting, it is known by all the provers. Therefore, Procedure `Precompute( $g$ )` in Algorithm 1 can be computed locally by each server. Moreover, in the initialization procedure, e.g., line 11 of Algorithm 1, the equation is a linear function of $\tilde{V}_{i+1}(y)$ as $\mathbf{g}$ is a public vector. Similarly, the equations to update the vectors in line 20 and 21 are linear as well. The only place that requires an MPC protocol is line 19 multiplying a value computed from $\mathbf{h}$ and a value computed from $\mathbf{v}$, which are both secret-shared. We use an MPC protocol for inner product to perform this step. We present the protocol of Phase 1 for each server in Algorithm 1. The shared values are indicated using $\langle \rangle$ and we omit the subscript of the server in the algorithm. As shown in the algorithm, only step 15 and 16 are executed using an MPC protocol and all other steps are computed locally. At the end of this phase, the provers hold the shares of $\langle \tilde{V}_{i+1}(u) \rangle = \langle \mathbf{v} \rangle[0]$.

After Phase 1, the provers execute Phase 2 on shared values and the algorithm is shown in Algorithm 3 in Appendix B. Surprisingly, all the steps can be done locally after a scalar multiplication in MPC. This is because in Phase 2, the initialization only depends on $\tilde{mult}_{i+1}$, which is defined by the circuit and is known to each prover. Thus the vector $\mathbf{m}$ is publicly known by all the provers. After the scalar multiplication in Step 8, the remaining `Sumcheckproof` procedure is executed on a shared vector and a public vector, and thus no MPC protocol is involved at all.

Combining Two Points Randomly. At the end of the sumcheck, $\mathcal{V}$ needs to combine two evaluations of $\tilde{V}_{i+1}$ via a random linear combination. Upon

Algorithm 1. Phase 1 in MPC

Input: Shared multilinear extension $\langle \tilde{V}_{i+1} \rangle$ and randomness

$g = g_1, \dots, g_{s_{i+1}}, u = u_1, \dots, u_{s_{i+1}};$

Output: Proofs of the sumcheck protocol $a_1, \dots, a_{s_{i+1}}$, where each a_i contains 3 evaluations uniquely representing a degree 2 polynomial;

```

1: procedure  $\mathbf{g} \leftarrow \text{Precompute}(g)$   $\triangleright \mathbf{g}$  is an array of size  $2^{s_{i+1}}$ .
2:   Set  $\mathbf{g}[0] = 1$ 
3:   for  $i = 0, \dots, s_{i+1} - 1$  do
4:     for  $b \in \{0, 1\}^i$  do
5:        $\mathbf{g}[b, 0] = \mathbf{g}[b] \cdot (1 - g_{i+1})$ 
6:        $\mathbf{g}[b, 1] = \mathbf{g}[b] \cdot g_{i+1}$ 
7:   return  $\mathbf{g}$ ;
8: procedure  $\langle \mathbf{h} \rangle \leftarrow \text{Initialize\_Phase1}(\langle \tilde{V}_{i+1} \rangle, \mathbf{g})$ 
9:    $\forall x \in \{0, 1\}^{s_{i+1}}$ , set  $\langle \mathbf{h} \rangle[x] = 0$ 
10:  for every multiplication gate  $(z, x, y)$  such that  $\tilde{mult}_{i+1}(z, x, y) = 1$  do
11:     $\langle \mathbf{h} \rangle[x] = \langle \mathbf{h} \rangle[x] + \mathbf{g}[z] \cdot \langle \tilde{V}_{i+1}(y) \rangle$ 
12:  return  $\langle \mathbf{h} \rangle$ ;
13: procedure  $(a_1, \dots, a_{s_{i+1}}) \leftarrow \text{Sumcheckproof}(\langle \mathbf{h} \rangle, \langle \tilde{V}_{i+1} \rangle, u)$ 
14:   Set vector  $\langle \mathbf{v} \rangle[b] = \langle \tilde{V}_{i+1}(b) \rangle$  for all  $b \in \{0, 1\}^{s_{i+1}}$ 
15:   for each round  $i = 1, \dots, s_{i+1}$  do
16:     for  $b \in \{0, 1\}^{s_{i+1}-i}$  do //  $b$  denotes both the number and its binary string
17:       for  $t = 0, 1, 2$  do
18:         Compute  $\langle \mathbf{h} \rangle[b](1 - t) + \langle \mathbf{h} \rangle[b + 2^{s_{i+1}-i}]t$  and
19:          $\langle \mathbf{v} \rangle[b](1 - t) + \langle \mathbf{v} \rangle[b + 2^{s_{i+1}-i}]t$ . Store them in vectors  $\langle \mathbf{h} \rangle_t$  and  $\langle \mathbf{v} \rangle_t$ .
20:          $\langle \mathbf{h} \rangle[b] = \langle \mathbf{h} \rangle[b](1 - u_i) + \langle \mathbf{h} \rangle[b + 2^{s_{i+1}-i}]u_i$ 
21:          $\langle \mathbf{v} \rangle[b] = \langle \mathbf{v} \rangle[b](1 - u_i) + \langle \mathbf{v} \rangle[b + 2^{s_{i+1}-i}]u_i$ 
22:       for  $t = 0, 1, 2$  do
23:          $a_{t,i} = \text{MPC.InnerProduct}(\langle \mathbf{h} \rangle_t, \langle \mathbf{v} \rangle_t)$ 
24:   return  $a_1, \dots, a_{s_{i+1}}$ .

```

receiving the two claims $\tilde{V}_i(u)$ and $\tilde{V}_i(v)$, $\mathcal{V}$ selects $\alpha_i, \beta_i \in \mathbb{F}$ randomly and computes $\alpha_i \tilde{V}_i(u) + \beta_i \tilde{V}_i(v)$, which can be written as

$$\begin{aligned}
& \alpha_i \tilde{V}_i(u) + \beta_i \tilde{V}_i(v) \\
&= \alpha_i \sum_{x, y \in \{0, 1\}^{s_{i+1}}} (\tilde{add}_{i+1}(u, x, y)(\langle \tilde{V}_{i+1} \rangle(x) + \langle \tilde{V} \rangle_{i+1}(y)) \\
&\quad + \tilde{mult}_{i+1}(u, x, y)(\langle \tilde{V}_{i+1} \rangle(x) \langle \tilde{V}_{i+1} \rangle(y))) \\
&+ \beta_i \sum_{x, y \in \{0, 1\}^{s_{i+1}}} (\tilde{add}_{i+1}(v, x, y)(\langle \tilde{V}_{i+1} \rangle(x) + \langle \tilde{V}_{i+1} \rangle(y)) \\
&\quad + \tilde{mult}_{i+1}(v, x, y)(\langle \tilde{V}_{i+1} \rangle(x) \langle \tilde{V}_{i+1} \rangle(y)))
\end{aligned} \tag{2}$$

$$\begin{aligned}
&= \sum_{x,y \in \{0,1\}^{s_{i+1}}} ((\alpha_i \tilde{add}_{i+1}(u, x, y) + \beta_i \tilde{add}_{i+1}(v, x, y))) \\
&\quad (\langle \tilde{V}_{i+1} \rangle(x) + \langle \tilde{V}_{i+1} \rangle(y)) \\
&\quad + ((\alpha_i \tilde{mult}_{i+1}(u, x, y) + \beta_i \tilde{mult}_{i+1}(v, x, y))) (\langle \tilde{V}_{i+1} \rangle(x) \langle \tilde{V}_{i+1} \rangle(y))
\end{aligned}$$

The polynomials $\tilde{add}$ and $\tilde{mult}$ are defined by the circuit and known by all provers. Hence, we can rewrite the Equation as:

$$\begin{aligned}
&\sum_{x,y \in \{0,1\}^{s_{i+1}}} (A\tilde{D}D_{i+1}(v, x, y) (\langle \tilde{V}_{i+1} \rangle(x) + \langle \tilde{V}_{i+1} \rangle(y)) \\
&\quad + M\tilde{U}LT_{i+1}(v, x, y) \langle \tilde{V}_{i+1} \rangle(x) \langle \tilde{V}_{i+1} \rangle(y))
\end{aligned} \tag{3}$$

Note that Eq. 3 is of the same form as Eq. 2 on shared values of $\langle \tilde{V}_{i+1} \rangle$ and our new algorithms can be applied with the new public $A\tilde{D}D_{i+1}$ and $M\tilde{U}LT_{i+1}$.

Complexity Analysis. As most of the steps are computed locally, the prover time of both Phase 1 and Phase 2 remains $O(C)$, linear in the size of the circuit. The additional MPC protocols in Step 15–16 of Algorithm 1 and Step 8 of Algorithm 3 do not introduce any asymptotic overhead on the running time. The communication complexity between the provers in Phase 1 is $O(C)$ per prover if these MPC protocols are implemented naively, and can be further reduced to $O(1)$ per prover using Shamir’s packed secret sharing. Excluding the scalar multiplication, there is no communication in Phase 2 other than sending the proofs and receiving the randomness. The round complexity remains $O(d \log C)$ as in the original GKR protocol.

The protocol can be made non-interactive via the Fiat-Shamir heuristic [12]. Namely, in each round in Algorithm 1 (i.e., line 13 and 14), instead of receiving the randomness u_i from the verifier, it is computed non-interactively by hashing all the previous messages computed by the provers. It can also be further modified to achieve zero-knowledge by adding masking polynomials as proposed in [8, 37]. These modifications do not introduce any overhead asymptotically to the computation and communication of the provers, and we instantiate Step (2) in Protocol 1 with the non-interactive and zero-knowledge GKR protocol on shared values.

3.4 GKR in MPC for Matrix Multiplications

In this section, we further propose a confidential and verifiable delegation scheme tailored for matrix multiplications. For simplicity, we assume that A, B and D are all $m \times m$ matrices such that $A \times B = D$. A and B are secret-shared by $\mathcal{V}$ to the provers, or intermediate shared values during a larger computation consisting of the matrix multiplication as one step. The provers first execute an MPC protocol to compute the shares of D . Using the special MPC protocol [23], the computation complexity on each server is $O(m^3)$, and the communication complexity is $O(m^2)$.

Algorithm 2. Confidential and Verifiable Matrix Multiplication

Input: $\langle A \rangle, \langle B \rangle$, randomness $g = g_1, \dots, g_{\log m}, u = u_1, \dots, u_{\log m}, v = v_1, \dots, v_{\log m}$;
Output: Proofs of the sumcheck protocol $a_1, \dots, a_{\log m}$;

```

1:  $\mathbf{g} \leftarrow \text{Precompute}(g)$ 
2:  $\mathbf{u} \leftarrow \text{Precompute}(u)$ 
3: procedure  $(\langle \mathbf{A} \rangle, \langle \mathbf{B} \rangle) \leftarrow \text{Initialize\_MM}(\langle A \rangle, \langle B \rangle)$ 
4:   Initialize  $\langle \mathbf{A} \rangle, \langle \mathbf{B} \rangle$  as all 0s
5:   for  $\ell \in [m]$  do
6:     for  $k \in [m]$  do
7:        $\langle \mathbf{A} \rangle[\ell] = \langle \mathbf{A} \rangle[\ell] + \langle A \rangle_{k,\ell} \cdot \mathbf{g}[k]$ 
8:        $\langle \mathbf{B} \rangle[\ell] = \langle \mathbf{B} \rangle[\ell] + \langle B \rangle_{\ell,k} \cdot \mathbf{u}[k]$ 
9: procedure  $(a_1, \dots, a_{\log m}) \leftarrow \text{Sumcheckproof}(\langle \mathbf{A} \rangle, \langle \mathbf{B} \rangle, v)$ 
10:  for each round  $i = 1, \dots, \log m$  do
11:    for  $b \in \{0, 1\}^{\log m - i}$  do ▷ we use  $b$  as both a number and its binary representation
12:      for  $t = 0, 1, 2$  do
13:        Compute  $\langle \mathbf{A} \rangle[b](1 - t) + \langle \mathbf{A} \rangle[b + 2^{s_{i+1}-i}]t$  and
         $\langle \mathbf{B} \rangle[b](1 - t) + \langle \mathbf{B} \rangle[b + 2^{s_{i+1}-i}]t$ . Store them in vectors  $\langle \mathbf{A} \rangle_t$  and  $\langle \mathbf{B} \rangle_t$ .
14:         $\langle \mathbf{A} \rangle[b] = \langle \mathbf{A} \rangle[b](1 - v_i) + \langle \mathbf{A} \rangle[b + 2^{s_{i+1}-i}]v_i$ 
15:         $\langle \mathbf{B} \rangle[b] = \langle \mathbf{B} \rangle[b](1 - v_i) + \langle \mathbf{B} \rangle[b + 2^{s_{i+1}-i}]v_i$ 
16:      for  $t = 0, 1, 2$  do
17:         $a_{t,i} = \text{MPC.InnerProduct}(\langle \mathbf{A} \rangle_t, \langle \mathbf{B} \rangle_t)$ 
18:  return  $a_1, \dots, a_{\log m}$ .
```

After computing D , as observed by Justin Thaler in [33], the matrix multiplication relationship can be modeled as a sumcheck equation:

$$\tilde{D}(x, y) = \sum_{z \in \{0,1\}^{\log m}} \tilde{A}(x, z) \tilde{B}(z, y), \quad (4)$$

where $\tilde{A}, \tilde{B}, \tilde{D}$ are multilinear extensions $\{0, 1\}^{\log m} \times \{0, 1\}^{\log m} \rightarrow \mathbb{F}$ defined by the matrices A, B, D . In particular, $\tilde{A}(x, y) = A_{\bar{x}, \bar{y}}$, $\tilde{B}(x, y) = B_{\bar{x}, \bar{y}}$ and $\tilde{D}(x, y) = D_{\bar{x}, \bar{y}}$ for all $x, y \in \{0, 1\}^{\log m}$ and $\bar{x}, \bar{y}$ denote their corresponding value in $[m]$. It is not hard to see that Eq. 4 holds for all $x, y \in \{0, 1\}^{\log m}$ by the definition of the polynomials, and thus the equation holds for the multilinear extensions.

To verify the correctness of the matrix multiplication, the verifier picks $g, u \in \mathbb{F}^{\log m}$ and evaluates $\tilde{D}(g, u)$. Again this value can also be sent by the prover during the GKR protocol in a larger computation. The verifier then runs a sumcheck protocol with the provers to check:

$$\tilde{D}(g, u) = \sum_{z \in \{0,1\}^{\log m}} \tilde{A}(g, z) \tilde{B}(z, u). \quad (5)$$

We propose an efficient MPC protocol to execute this sumcheck when the matrices A and B are secret-shared between n provers. Observe that Eq. 5 is again a sumcheck protocol on the product of two polynomials, and in the first step, we would like to evaluate both polynomials on the Boolean hypercube, i.e., $\tilde{A}(g, z)$ and $\tilde{B}(z, u)$ for all $z \in \{0, 1\}^{\log m}$. For each z , we have

$$\tilde{A}(g, z) = \sum_{t \in \{0, 1\}^{\log m}} \tilde{I}(g, t) A_{t, z}, \quad (6)$$

where $\tilde{I}(g, t)$ is the identity polynomial defined in Sect. 2. The equation above holds for all $g \in \{0, 1\}^{\log m}$ by the definition of $\tilde{A}$, and thus holds for all $g \in \mathbb{F}^{\log m}$ because both sides are multilinear extensions. This equation suggests that we can precompute $\tilde{I}(g, t)$ for all $t \in \{0, 1\}^{\log m}$, and then sum up $\tilde{I}(g, t) A_{t, z}$ for each z , as shown in Steps 1–8 in Algorithm 2. Crucially, as g is known by all the provers, this initialization is a linear function and can be computed locally on each server. At the end of this step, each prover ends up with a shared vector $\langle \mathbf{A} \rangle$ of size m . The same algorithm applies to matrix B .

After the initialization, each prover holds the shares of two vectors storing the evaluations of $\tilde{A}(g, z)$ and $\tilde{B}(z, u)$ on the Boolean hypercube. We then apply the dynamic programming technique to compute the proofs of the sumcheck protocol. The algorithm is presented in Steps 9–18 in Algorithm 2 and is similar to the protocol in Algorithm 1. Only Steps 16 and 17 involve an MPC protocol to compute the inner product.

Complexity Analysis. The main advantage of our scheme is that the overhead to generate the proof is minimal compared to the time of computing the matrix multiplication in MPC. The computation complexity on each prover is $O(m^2)$, dominated by Procedure `Initialize_MM`. The communication complexity is only $O(m)$ to compute the inner products. These are smaller than the MPC protocol by a factor of $O(m)$. The proof size and the number of rounds are both $O(\log m)$.

3.5 Polynomial Commitments in MPC

In the last round of the GKR protocol, $\mathcal{V}$ receives a claim about the evaluation of a multilinear extension defined by the witness at a random point from the provers. In order to complete the protocol, the provers further prove that the evaluation is correct using a polynomial commitment scheme. We use multivariate polynomial commitment proposed by Papamanthou et al. [29], which is a generalization of the KZG polynomial commitment [22] to the multivariate case. Similar to [28], we also observe that all computations in the multivariate polynomial commitment are linear and can be computed locally by each prover when the witness is secret-shared. The protocol of each prover is exactly the same as the original polynomial commitment as long as we set the field of the arithmetic secret sharing the same as the order of the base group of the bilinear map. We defer the protocol to the full version of the paper.

Theorem 1. *Protocol 1 is a confidential and verifiable delegation scheme under Definition 1.*

The proof can be found in the full version of the paper.

4 Experiments

We have fully implemented our confidential and verifiable delegation scheme for both general circuits and matrix multiplications, and we present the experimental results in this section.

Implementation and Setting. The system is implemented in C++. The sum-check protocol and the GKR protocol are based on the open-source code of Libra [2]. For MPC computations, we adopt the MP-SPDZ library [4] to implement an additive secret-sharing protocol with malicious security and all but one honest party. We ran all experiments on an AWS c5.9xlarge instance with an Intel Xeon Platinum 8000 processor and 72 GB of RAM. We simulated multiple provers in a LAN setting with 10 Gbps network bandwidth using the `tc` command. We report the average running time over 10 executions.

4.1 Performance of Our System

We first benchmark our confidential and verifiable delegation system.

General circuits. We executed our system with 2 provers on circuits ranging from 2^{17} to 2^{20} gates. We break down prover’s runtime and communication (column Comm.) and show each component in Table 2. For a circuit size of 2^{18} , our confidential and verifiable delegation protocol’s end-to-end prover time is 163s. Approximately 11% of this time is spent on evaluating the circuit in MPC, while the majority (89%) is dedicated to the MPC protocol for ZKP generation. This is because although most of the steps in Algorithm 1 and 3 are performed locally, the size of the MPC computation is still linear in the size of the circuit. Note that the time reported in the table includes both the offline phase and the online phase of the MPC protocol. The time of the local computation to generate the proofs is very small compared to the other two parts.

The proof size is from 13.8KB for $C = 2^{17}$ to 14.9KB for $C = 2^{20}$, which grows logarithmically with the size of the circuit. Similarly, the verifier ranges from 13 ms to 15 ms.

Table 2. Breakdown of prover time in seconds and communication in megabytes for general circuits.

# of gates	Prover	Circ Eval	ZKP (MPC)	ZKP (local)	Comm.
2^{17}	78.5	8.61	69.8	0.02	30.34
2^{18}	162.71	17.25	145.43	0.035	60.41
2^{19}	314.5	34.27	281.73	0.074	120.53
2^{20}	631.21	68.48	562.59	0.15	240.76

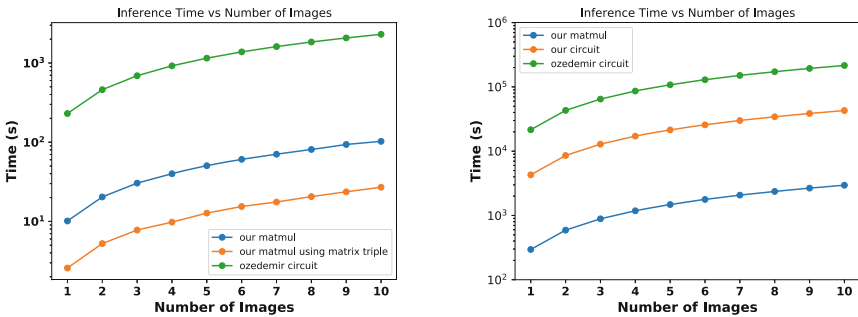
Table 3. Breakdown of prover time in seconds and communication in megabytes for matrix multiplications.

Matrix Size	Prover	Matrix Mult	ZKP (MPC)	ZKP (Local)	Comm.
64×64	0.139	0.101	0.0359	0.00017	1.605
128×128	0.254	0.18	0.074	0.0006	4.617
256×256	0.601	0.591	0.15	0.003	18.398

Matrix Multiplications. In Table 3, we demonstrate the performance of our specialized matrix multiplication protocol by varying the matrix size from 64×64 to 256×256 . As shown in the table, the breakdown is completely different from the case of general circuits. In particular, for $m = 256$, 98.3% of the prover time is spent on computing the matrix multiplication in MPC. The additional overhead to generate the ZKP is minimal, thanks to our special protocol that improves both the computation and the communication by $O(m)$. Moreover, the special protocol is generally faster than the protocol for general circuits in all aspects. For example, implementing a 64×64 matrix multiplication would require a circuit of size more than 2^{18} , and the prover time of the special protocol is $1170\times$ faster than the protocol in Table 2 for $C = 2^{18}$, and the communication is $37\times$ smaller. Finally, the proof size ranges from 0.15 KB to 0.2 KB, and the verifier time is from 0.02 ms to 0.036 ms, both are succinct.

4.2 Comparison to Prior Work

In this section, we further compare the performance of our system to the prior work of [28]. We use their open-source implementation in the Github repository at [5]. We generate circuits of the same size and compare the performance the scheme in [28] with our scheme. As [28] only reports the online time of MPC assuming the multiplication triplets have been generated, we also only show the online time of our scheme to get a fair comparison. Figure 3b in Appendix C



(a) Neural network inference. The time is total time ranging from 1 to 10 images. (b) Lenet inference. The time is total time ranging from 1 to 10 images.

Fig. 2. Machine learning experiment

shows the comparison of the prover time to the three schemes in [28]. As shown in the figure, our scheme improves the prover time significantly. For a circuit with 2^{20} gates², our online prover time is $10\times$ faster than the scheme based on Groth16, $80\times$ faster than the one based on Marlin and $133\times$ faster than the one based on Plonk.

However, the proof size of our scheme is admittedly larger than these schemes because of the underlying cryptographic techniques. The proof size of these schemes is only several hundreds of bytes, while it is tens of KBs in our scheme and it further grows with the depth of the circuit.

4.3 Machine Learning Inference

We implemented confidential and verifiable machine learning inferences by secret-sharing a pre-trained neural network model to 3 provers via additive sharing. The experiment utilized the MNIST dataset [25] comprising hand-written digit images with dimensions 28×28 . The images, flattened to a vector of size 784, are secret-shared to the servers. The neural network model, trained using PyTorch, consists of three fully connected layers (128, 84, and 10 neurons) with ReLU activation functions.

In Fig. 2a, a single inference and its proof generation on three provers take only 2.6s. Comparing our protocol for general circuits with the new matrix multiplication protocol, our scheme is $12.7\times$ faster. Additionally, it is $88\times$ faster than the prior work [28] (Groth16). Our implementation supports multiple image inferences, taking 26.9 s for 10 images.

We also tested our scheme on the convolutional neural network of Lenet [24]. It consists of 2 convolution layers with 5×5 kernels and 2 padding, 2 average pooling layers, and 3 fully connected layers of size 120,84, and 10 respectively.

As shown in Fig. 2b, the running time for 1 inference is 288s on three provers. Likewise, we include the performance of implementing the lenet inference as a circuit. A single convolution on a 28×28 image with kernel size 5 will result in a 1024×1024 square matrix multiplication, which translate into 2^{20} gates. The running time of our scheme with the new protocol for matrix multiplication is $14.8 \times$ faster than the protocol for general circuit, and $74.8 \times$ faster than the prior work of [28]. Our implementation takes 2888s to inference 10 images.

² Groth16 and Marlin use the rank-1-constraint-systems (R1CS) instead of circuits, and we execute them on R1CS with the same number of constraints.

5 Conclusions

This paper introduces a confidential and verifiable delegation scheme by integrating MPC and ZKP. We demonstrate efficient proof generation for interactive-proof-based protocols within MPC, emphasizing a specialized protocol for matrix multiplications with asymptotically smaller proof generation time than MPC evaluation. Since we observe that MPC is typically the dominant cost, future work could explore enhancing ZKP generation in MPC by utilizing multiple provers for improved running time.

Acknowledgments. This work is funded by the AFRL/RI, Rome, NY, USA under Contract Number FA8750-22-2-0267. The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of the Air. Approved for Public Release on 16 Mar 2024. Distribution is Unlimited. Case Number: AFRL-2024-1294.

A Sumcheck Protocol

Protocol 2 (Sumcheck protocol) *The protocol proceeds in ℓ rounds.*

- In the first round, $\mathcal{P}$ sends $\mathcal{V}$ a univariate polynomial

$$g_1(x_1) \stackrel{\text{def}}{=} \sum_{b_2, \dots, b_\ell \in \{0,1\}} g(x_1, b_2, \dots, b_\ell),$$

$\mathcal{V}$ checks $H = g_1(0) + g_1(1)$. Then $\mathcal{V}$ sends $\mathcal{P}$ a random challenge $r_1 \in \mathbb{F}$.

- In the i th round, $\mathcal{P}$ sends $\mathcal{V}$ a univariate polynomial

$$g_i(x_i) \stackrel{\text{def}}{=} \sum_{b_{i+1}, \dots, b_\ell \in \{0,1\}} g(r_1, \dots, r_{i-1}, x_i, b_{i+1}, \dots, b_\ell),$$

$\mathcal{V}$ checks $g_{i-1}(r_{i-1}) = g_i(0) + g_i(1)$, and sends $\mathcal{P}$ a random challenge $r_i \in \mathbb{F}$.

- In the ℓ th round, $\mathcal{P}$ sends a univariate polynomial

$$g_\ell(x_\ell) \stackrel{\text{def}}{=} g(r_1, r_2, \dots, r_{\ell-1}, x_\ell),$$

$\mathcal{V}$ checks $g_{\ell-1}(r_{\ell-1}) = g_\ell(0) + g_\ell(1)$.

- In the final round, $\mathcal{V}$ generates a random challenge $r_\ell \in \mathbb{F}$. and accept if and only if $g_\ell(r_\ell) = g(r_1, r_2, \dots, r_\ell)$.

B Phase 2 of the GKR Protocol in MPC

Algorithm 3. Phase 2 in MPC

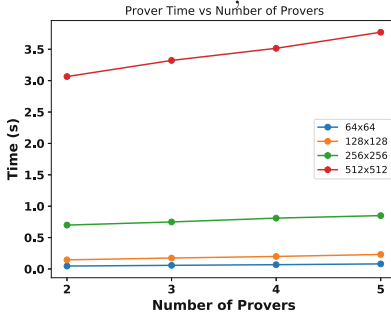
Input: Shared multilinear extension $\langle \tilde{V}_{i+1} \rangle$, shared evaluation $\langle \tilde{V}_{i+1}(u) \rangle$ from Phase 1, and randomness $g = g_1, \dots, g_{s_{i+1}}, u = u_1, \dots, u_{s_{i+1}}, v = v_1, \dots, v_{s_{i+1}}$;
Output: Proofs of the sumcheck protocol $a'_1, \dots, a'_{s_{i+1}}$;

```

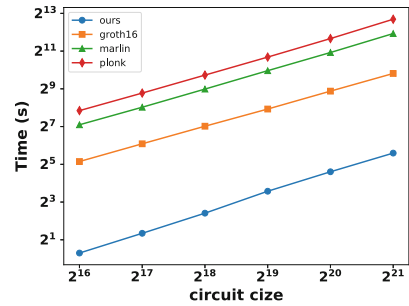
1:  $\mathbf{g} \leftarrow \text{Precompute}(g)$ 
2:  $\mathbf{u} \leftarrow \text{Precompute}(u)$ 
3: procedure  $\mathbf{m} \leftarrow \text{Initialize\_Phase2}(\mathbf{g}, \mathbf{u})$ 
4:    $\forall y \in \{0, 1\}^{s_{i+1}}$ , set  $\mathbf{m}[y] = 0$ 
5:   for every multiplication gate  $(z, x, y)$  such that  $\tilde{m}_{t_{i+1}}(z, x, y) = 1$  do
6:      $\mathbf{m}[y] = \mathbf{m}[y] + \mathbf{g}[z] \cdot \mathbf{u}[x]$ 
7:   return  $\mathbf{m}$ ;
8: Compute  $\text{MPC.ScalarProduct}(\langle \tilde{V}_{i+1}(u) \rangle, \langle V_{i+1} \rangle(y))$  for all  $y \in \{0, 1\}^{s_{i+1}}$ . Denote the result as  $\langle \mathbf{v}' \rangle$ .
9: procedure  $(a'_1, \dots, a'_{s_{i+1}}) \leftarrow \text{Sumcheckproof}(\langle \mathbf{v}' \rangle, \mathbf{m}, v)$ 
10:  Set  $\langle a_{t,i} \rangle = 0$  for  $i = 1, \dots, s_{i+1}$  and  $t = 0, 1, 2$ 
11:  for each round  $i = 1, \dots, s_{i+1}$  do
12:    for  $b \in \{0, 1\}^{s_{i+1}-i}$  do ▷ we use  $b$  as both a number and its binary representation
13:      for  $t = 0, 1, 2$  do
14:         $\langle a'_{t,i} \rangle = \langle a_{t,i} \rangle + (\langle \mathbf{v}' \rangle[b](1-t) + \langle \mathbf{v}' \rangle[b + 2^{s_{i+1}-i}t])(\mathbf{m}[b](1-t) + \mathbf{m}[b + 2^{s_{i+1}-i}t])$ 
15:         $\langle \mathbf{v}' \rangle[b] = \langle \mathbf{v}' \rangle[b](1-v_i) + \langle \mathbf{v}' \rangle[b + 2^{s_{i+1}-i}v_i]v_i$ 
16:         $\mathbf{m}[b] = \mathbf{m}[b](1-v_i) + \mathbf{m}[b + 2^{s_{i+1}-i}v_i]v_i$ 
17:  Reconstruct  $\langle a'_1 \rangle, \dots, \langle a'_{s_{i+1}} \rangle$ 
18:  return  $a'_1, \dots, a'_{s_{i+1}}$ .
```

C Additional Experiments

Scaling to Multiple Servers. We demonstrate the performance of the matrix multiplication scheme with varying numbers of provers (2 to 5) in Fig. 3(a). The running time scales linearly with the number of provers, remaining practical. For a 512×512 matrix multiplication on 5 provers, even with all but one potentially malicious, it takes only 3.77 s.



(a) Prover time with various number of provers



(b) Comparison to schemes in [28]. The time is for the online phase of 2 provers.

Fig. 3. Prover runtime experiment

References

1. EZKL. <https://cli.gizatech.xyz/frameworks/ezkl>
2. Libra. <https://github.com/sunblaze-ucb/Libra>
3. Modulus. <https://medium.com/@ModulusLabs>
4. SPDZ. <https://github.com/data61/MP-SPDZ>
5. collaborative-zksnark. <https://github.com/alex-ozdemir/collaborative-zksnark> (2021)
6. Beaver, D.: Efficient multiparty protocols using circuit randomization. In: CRYPTO (1991)
7. Bois, A., Cascudo, I., Fiore, D., Kim, D.: Flexible and efficient verifiable computation on encrypted data. In: Garay, J.A. (ed.) Public-Key Cryptography – PKC 2021: 24th IACR International Conference on Practice and Theory of Public Key Cryptography, Virtual Event, May 10–13, 2021, Proceedings, Part II, pp. 528–558. Springer International Publishing, Cham (2021). https://doi.org/10.1007/978-3-030-75248-4_19
8. Chiesa, A., Forbes, M.A., Spooner, N.: A zero knowledge sumcheck and its applications. CoRR, abs/1704.0208 (2017)
9. Chiesa, A., Hu, Y., Maller, M., Mishra, P., Vesely, N., Ward, N.: Marlin: preprocessing zkSNARKs with universal and updatable SRS. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020. LNCS, vol. 12105, pp. 738–768. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-45721-1_26
10. Chiesa, A., Lehmkuhl, R., Mishra, P., Zhang, Y.: EOS: efficient private delegation of zkSNARK provers. In: 32st USENIX Security Symposium (USENIX Security 23), (2023)
11. Dayama, P., Patra, A., Paul, P., Singh, N., Vinayagamurthy, D.: How to prove any np statement jointly? Efficient distributed-prover zero-knowledge protocols. In: Proceedings on Privacy Enhancing Technologies (2022)
12. Fiat, A., Shamir, A.: How to prove yourself: practical solutions to identification and signature problems. In: Crypto (1986)
13. Fiore, D., Gennaro, R., Pastro, V.: Efficiently verifiable computation on encrypted data. In: Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, pp. 844–855 (2014)
14. Fiore, D., Nitulescu, A., Pointcheval, D.: Boosting verifiable computation on encrypted data. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) Public-Key Cryptography – PKC 2020: 23rd IACR International Conference on Practice and Theory of Public-Key Cryptography, Edinburgh, UK, May 4–7, 2020, Proceedings, Part II, pp. 124–154. Springer International Publishing, Cham (2020). https://doi.org/10.1007/978-3-030-45388-6_5
15. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: Plonk: permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive (2019)
16. Garg, S., Goel, A., Jain, A., Policharla, G.-V., Sekar, S.: zkSaaS: zero-knowledge SNARKs as a service. In: 32nd USENIX Security Symposium (USENIX Security 23), pp. 4427–4444, Anaheim, CA (2023)
17. Gennaro, R., Gentry, C., Parno, B.: Non-interactive verifiable computing: outsourcing computation to untrusted workers. In: Rabin, T. (ed.) CRYPTO 2010. LNCS, vol. 6223, pp. 465–482. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-14623-7_25
18. Goldwasser, S., Kalai, Y.T., Rothblum, G.: Delegating computation: interactive proofs for muggles. In: STOC, vol. 2008, pp. 113–122 (2008)

19. Goldwasser, S., Kalai, Y.T., Rothblum, G.N.: Delegating Computation: interactive proofs for muggles. *J. ACM*, **62**(4), 27:1–27:64 (2015)
20. Groth, J.: On the size of pairing-based non-interactive arguments. In: Fischlin, M., Coron, J.-S. (eds.) *EUROCRYPT 2016*. LNCS, vol. 9666, pp. 305–326. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-662-49896-5_11
21. Kanjalkar, S., Zhang, Y., Gandlur, S., Miller, A.: Publicly auditable MPC-as-a-service with succinct verification and universal setup. In: 2021 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), pp. 386–411. IEEE (2021)
22. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: *ASIACRYPT*, pp. 177–194 (2010)
23. Keller, M.: MP-SPDZ: a versatile framework for multi-party computation (2020)
24. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. *Proc. IEEE* **86**(11), 2278–2324 (1998)
25. LeCun, Y., Cortes, C., Burges, C.J.: Mnist handwritten digit database. *ATT Labs*, 2 (2010). <http://yann.lecun.com/exdb/mnist>
26. Liu, X., Zhou, Z., Wang, Y., Zhang, B., Yang, X.: Scalable collaborative zk-snark: fully distributed proof generation and malicious security. *Cryptology ePrint Archive*, Paper 2024/143 (2024). <https://eprint.iacr.org/2024/143>
27. Micali, S., Goldreich, O., Wigderson, A.: How to play any mental game. In: *Proceedings of the Nineteenth ACM Symposium on Theory of Computing, STOC*, pp. 218–229. ACM New York, NY, USA (1987)
28. Ozdemir, A., Boneh, D.: Experimenting with collaborative {zk-SNARKs}: {Zero-Knowledge} proofs for distributed secrets. In: 31st *USENIX Security Symposium (USENIX Security 22)*, pp. 4291–4308 (2022)
29. Papamanthou, C., Shi, E., Tamassia, R.: Signatures of correct computation. In: Sahai, A. (ed.) *TCC 2013*. LNCS, vol. 7785, pp. 222–242. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-36594-2_13
30. Parno, B., Howell, J., Gentry, C., Raykova, M.: Pinocchio: nearly practical verifiable computation. In: *S&P 2013*, pp. 238–252 (2013)
31. Schoenmakers, B., Veeningen, M., de Vreede, N.: Trinocchio: privacy-preserving outsourcing by distributed verifiable computation. In: Manulis, M., Sadeghi, A.-R., Schneider, S. (eds.) *ACNS 2016*. LNCS, vol. 9696, pp. 346–366. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-39555-5_19
32. Shamir, A.: How to share a secret. *Commun. ACM* **22**(11), 612–613 (1979)
33. Thaler, J.: Time-optimal interactive proofs for circuit evaluation (2013)
34. Wahby, R.S., Tzialla, I., Shelat, A., Thaler, J., Walfish, M.: Doubly-efficient zkSNARKs without trusted setup. In: 2018 IEEE Symposium on Security and Privacy (SP), pp. 926–943. IEEE (2018)
35. Xie, T., Zhang, J., Zhang, Y., Papamanthou, C., Song, D.: Libra: succinct zero-knowledge proofs with optimal prover computation. In: *Advances in Cryptology (CRYPTO)* (2019)
36. Yao, A.C.: Protocols for secure computations. In: 23rd Annual Symposium on Foundations of Computer Science (SFCS 1982), pp. 160–164. IEEE (1982)
37. Zhang, J., Xie, T., Zhang, Y., Song, D.: Transparent polynomial delegation and its applications to zero knowledge proof. In: 2020 IEEE Symposium on Security and Privacy (SP), pp. 859–876 (2020)
38. Zhang, Y., Genkin, D., Katz, J., Papadopoulos, D., Papamanthou, C.: vSQL: verifying arbitrary SQL queries over dynamic outsourced databases. In *IEEE Symposium on Security and Privacy (S&P)* (2017)



Process Identity-Based Firewalling

Radu Mantu, Mihai Chiroiu^(✉), and Costin Raiciu

University Politehnica of Bucharest, Bucharest, Romania
{radu.mantu,mihai.chiroiu,costin.raiciu}@upb.ro

Abstract. In this paper we introduce **DAF**, a distributed application firewall capable of filtering traffic based on process identity. The process identity is inferred from local-system level knowledge of processes and their access to specific sockets, without needing to examine the user data. In order to ensure its reliable operation in microservice-rich environments, we extend the process identification capabilities to function across namespaces, thus allowing seamless runtime container inspection. Additionally, individual instances of **DAF** are able attach proof of compliance to the locally configured firewall ruleset by annotating packets with HMACs as IP options. In an effort to reduce the difficulty of integrating our solution with existing software stacks, we offer **iptables** and **snort3** extensions that are capable of ascertaining the compliance of intercepted traffic with said ruleset.

Keywords: Distributed firewall · Process identity · Network security

1 Introduction

During the early days of the Internet, external network threats were easy to counteract due to the unambiguous nature of clients, as well as the limited availability of web applications. The association between server port numbers and the listening applications could be drawn more reliably. In turn, this simplified assessing the purpose behind each new connection. In 2003 however, Gartner Research defined what they considered to be four sound approaches to gateway security [20], thus coining the term “Next Generation Firewall” (NGFW). With the purpose of opposing emerging network security threats, NGFWs integrated Intrusion Detection and Prevention Systems (IDS/IPS) and application awareness [12] into conventional firewalls. Even at that time, it could be stated that endpoint applications were by and large detached from the meaning that the port they were bound to conferred in years prior. Thus, techniques such as JA3S or Markov Chain Fingerprinting [8] have since been developed for the purpose of active threat mitigation. However, their efficacy (as well as that of IDS/IPS) is significantly diminished when applied to encrypted traffic. Despite ongoing efforts towards encrypted traffic classification [1,31,34], TLS traffic decryption [21] unequivocally remains the most preponderant solution to this problem, levying a compromise between user privacy and general network security.

In spite of this shortcoming, NGFWs are more relevant than ever due to the ongoing adoption of Zero Trust [25] network architectures. As enterprises seek to implement fine-grained, least-privilege resource access policies, the utilization of micro-segmentation [2] has become increasingly common, as shown in a recent report [10] on Alphabet’s transition to the Zero Trust paradigm, i.e. Google BeyondCorp [18,30]. We argue that current access decision systems which are commonly based on user authentication are insufficient for preventing attacks that are facilitated by vulnerable versions of common tools or shared libraries, e.g. the recent `liblzma` backdoor or software supply chain attacks [17]. This problem is further exacerbated by the increasing fluidization of network demarcations, a phenomenon determined by the necessity of free movement between enterprise-owned and public networks. We believe that meeting these challenges should be aided by end-user devices. Based on this, we consider the following research questions:

RQ1: Reliable Endpoint Data Source for NGFWs? Micro-segmentation is a foundational component of zero-trust architectures. In contrast to traditional methods such as VLANs, micro-segmentation ensures network compartmentalization based on application layer visibility. However, most solutions today rely on the application classification features of NGFWs. We propose that one method of alleviating the over-reliance on TLS traffic decryption is by providing a pertinent, alternative data source that can uniquely identify endpoint applications and correctly associate this information with intercepted packets.

RQ2: Compatibility with Loosely Defined Networks? In keeping with initiatives such as BeyondCorp, we take into account the fact that users could not only transition between internal networks, but also access resources from beyond the conventional network perimeter. As a result, we adopt a stateless Layer 3 traffic annotation mechanism and discuss possible improvements with regard to public network traversal.

RQ3: Impact on Network Integrity and Performance? Ideally, a solution to the previous two questions would have minimal impact on both the network and individual hosts. Although certain concessions can oftentimes be made in favor of better security guarantees, we acknowledge the necessity for unobtrusive integration with pre-existing software stacks, as well as marginal downgrades in throughput. To this end, we aim to provide a prototype that is easily tunable to fit the unique requirements of each user.

In order to address these questions we propose DAF, a distributed application firewall [13] prototype capable of filtering traffic based on application identity. Its main characteristics are as follows:

Originating Process Identification: In contrast to existing solutions, we adopt a more discerning stance on how we distinguish applications from one another. We do not simply rely on the content extracted from processed packets to infer the identity of the originator. Instead, we determine all processes that had access to the socket that was used to emit a certain packet (a common occurrence in highly complex applications). Moreover, we are able to account for any

object that was mapped in the virtual address space of said processes and use them as rule matching criteria. This approach is motivated by recent advances in attaining reproducible builds [3] for both open and closed source software. One application of this feature is the ability to filter traffic that originated from processes that use a known vulnerable version of a certain library (e.g. `libssl` for traffic encryption). Alternatively, whitelists can be defined based on sets of binaries that comprise specific applications.

Packet Annotation: In order to communicate that a packet had passed through DAF on the originating endpoint, we attach a Message Authentication Code (MAC). The MAC is embedded into the packet as an IP option, making our system connectionless. This allows the destination endpoint or any middle-boxes that are aware of this annotation scheme to verify the authenticity of the received traffic. Alternatively, we attach an aggregate hash of all memory-mapped objects that comprised the originating process. This latter approach has a number of practical uses, e.g. allowing a system administrator to dictate the exact application and version one process is allowed to communicate with. Another function that could be served is that of a more general and reliable replacement of user-agent identification.

Compatibility with Container-Based Isolation: Due to the widespread adoption of microservice architectures, we deemed it necessary to ensure that DAF could function across namespace boundaries. As a result, we demonstrate that our firewall can apply fine-grained filtration rules in Docker-based environments. This approach has the added benefit of isolating the firewall process from potentially malicious processes, to a certain degree.

We claim the following contributions:

- The implementation of DAF¹, a network firewall capable of filtering and authenticating traffic based on the identity of the endpoint processes, regardless of namespace isolation.
- The integration of DAF with pre-existing firewalls (`iptables`) and Intrusion Prevention Systems (`snort3`), illustrating its potential for interoperability with existing traffic filtering technologies.
- An analysis of the inherent limitations of `NetfilterQueue` and the optimization techniques necessary for achieving satisfying throughputs for both consumer grade hardware as well as datacenter environments.

The remainder of this paper is organized as follows. In Sect. 2 we describe the architecture of our firewalling system. In Sect. 3 we present implementation details. Section 4 illustrates the performance impact of DAF on both workstations and servers with 1Gbps and 10Gbps NICs. Section 5 contains a comparison of DAF with similar existing solutions. In Sect. 6 we discuss future work. Section 7 concludes the paper.

¹ Code available at <https://github.com/RaduMantu/DAF>.

2 Architecture

In this section we describe the architecture of DAF. A high-level overview of our solution can be seen in Fig. 1. In order to operate our firewall, the first step would be for an administrator to create a whitelist of all the applications and their particular type of traffic that is allowed to be emitted. DAF allows the creation of classic firewall rules, such as permitting communication to/from specific IPs or ports, but extends this capability in one significant way: allowing applications to only accept traffic from other *specific* applications. Moreover, DAF can prevent attacks such as DNS rebinding by limiting the pool of accessible networked resources on a per-application basis. Once started, DAF begins to collect system-level event data, create a local system-state model, and process packets on behalf of the built-in firewall, all while adding custom tags to the outgoing traffic. The tags can be used by either middle-boxes or endpoint devices to filter incoming traffic. Depending on the requirements of each user, chain policies can be implemented to either accept or drop untagged traffic by default.

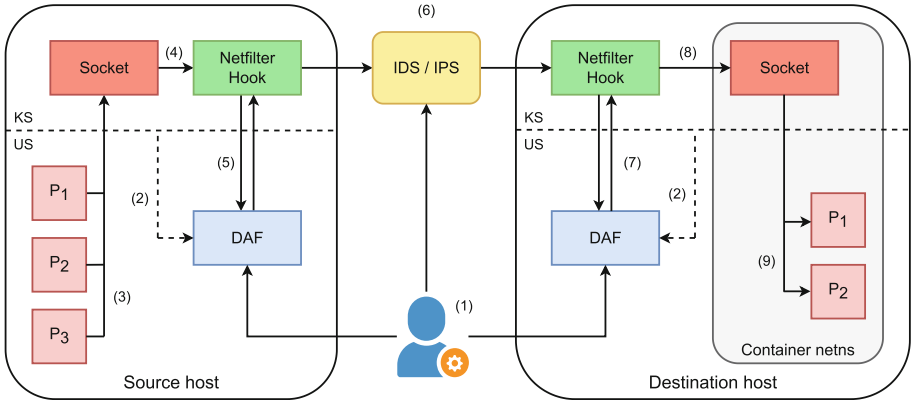


Fig. 1. Tagged packet data path. Both endpoints utilize DAF.

In implementing DAF we address two different types of problems: enabling fast and verifiable correlation between process and network traffic on an end-host (see Subsect. 2.1) and finding a method of transparently annotating traffic (see Subsect. 2.2).

2.1 Endpoint Processes Identification

In order to correlate a process with its specific network traffic, we have identified multiple possibilities:

- When considering a threat model where the attacker has limited user-level privileges, DAF can be implemented as an extension to the current firewall mechanisms (e.g., `iptables`), with no intervention on the kernel. This is our main implementation detailed in this paper on Linux.

- If we assume an attacker with elevated privileges, DAF requires a kernel level implementation, similar with existing antivirus solutions. We have implemented this model on Windows as a proof of concept.
- Alternatively, if we regard the attacker as capable of gaining control of his immediate environment but unable to break container isolation, DAF can safely run in userspace, in a superior namespace.
- If we assume that the attacker has kernel-level privileges and there is no support for virtualization-based solutions, DAF can either be implemented on an external hardware token or in a Trusted OS. The former implies that network traffic can no longer be attributed to a certain process due to the lack to OS-level information, but can still be attributed to a certain host and prevent spoofing. Meanwhile, the latter would require hardware support for separating trusted and untrusted protection domains (e.g., ARM TrustZone). These use-cases are outside the scope of this paper.

2.2 Network Integration

After identifying the correct process-to-traffic mapping, we then need to manifest this correlation as a tag, annotated to the packet in question. For this tag, we decided to use a SHA256 Hash-based Message Authentication Code (HMAC-SHA256) with a pre-shared secret in order to provide endpoint authentication and if needed, payload integrity guarantees (limited to the payload due to the complexities of NAT traversal [4]). We discarded digital signatures as an alternative early on due to having higher computational requirements and occupying more space than generally available for protocol options.

We note that in this version of DAF, the HMAC secrets are assumed to be shared between parties of an organization a priori. Any annotations sent to third parties that are not aware of our system should be silently filtered by the TCP/IP stack of the endhosts, as long as they *correctly* implement the IP protocol. Key management, including secret sharing and revocation for compromised hosts are outside the scope of this paper and are the subject of future work.

In an effort to minimize overhead, we avoided solutions that would render the Internet Protocol no longer stateless (e.g., spanning the signature over multiple packets). However, doing so would have to be transparent to systems that are not compliant with our implementation. For example, appending the signature as part of the payload would indubitably break communication at the application layer. Although the 40-byte **IP Options** section is an adequate conduit for our 32-byte HMAC and can painlessly cover multiple Layer 4 protocols, we acquiesce that it raises concerns with regard to non-ICMP traffic traversal over the Internet [7, 11, 15, 22]. In order to overcome this limitation, we examined the possibility of extending said annotation scheme to the transport layer:

TCP Options: Although almost universally accepted due to their impact on WAN performance (e.g., Selective Acknowledgement, Window Scaling, etc.) the limited 40-byte space [14] has made it essentially impossible to introduce new experimental options. If we followed the guideline to shared use of experimental

option codepoints [26], our annotation would total 36 bytes, prohibiting the inclusion of other options. Although the TCP Extended Data Offset Option [28] addresses this issue, it has been pending standardization since 2014. Existing studies [29] are also insufficient for assessing its viability.

UDP Options: In contrast to IP and TCP options, UDP options are a recent creation [27] that stemmed from a redundancy between the IP Total Length and UDP Length fields. While pending standardization since 2015, concerns have been raised regarding inconsistent behaviour observed in network equipment when calculating the L4 checksum [35]. Although the Checksum Compensation Option [6] has been proposed as a solution, there is still lacking evidence that this new protocol extension will be accepted in the wider Internet.

For these reasons, we considered it prudent to select **IP Options** over any Layer 4 alternative. We may revisit this decision once TCP EDO is standardized and UDP options receive sufficient testing over public networks.

3 Implementation

3.1 Packet Interception and Modification

There are multiple methods of intercepting, filtering and modifying network traffic. Our implementation is based on the `NetfilterQueue` extension to the `Xtables` system. `NetfilterQueue` takes advantage of the `nfnetlink_queue` kernel subsystem to deviate packets that match `iptables` rules through a userspace process. This delegates the selection of built-in targets (i.e., `ACCEPT`, `DROP`, etc.) that are to be applied, to said process. Once the userspace process subscribes to a certain queue and starts receiving packets, these can be analyzed in full (starting with the Layer 3 header), further deferred for judgement to a different process and optionally, be arbitrarily modified before their eventual reintroduction in kernel space.

This approach permits us to implement `iptables` extensions without requiring the user to load any additional kernel modules. Considering that this system was initially introduced in August 2005 as part of Linux 2.6.14, it is safe to assume that most current-day distributions support this feature. Loading kernel modules on the other hand, while providing better performance due to the reduced number of context switches (exacerbated in `NetfilterQueue` by the lack of zero-copy), may be difficult to achieve on some virtual cloud environments due to the kernel being in lockdown. Our decision to extend `iptables` and not other firewall alternatives is motivated primarily by its prominence in the industry. For example, LinkedIn recently implemented a distributed firewall [5] that manages host-bound filtering rules based on `iptables` and its predecessor, `ipset`. Aside from the performance disadvantage, the main flaw that applies to `iptables` in general, and translates to our solution as well, is the possibility to bypass the Netfilter kernel hooks by using the `PF_PACKET` low-level socket interface. However, this operation necessitates root privileges on the part of the attacker. Consequently, this particular scenario falls outside the purview of our threat model.

3.2 Inferring Process Identity

Correlating the information available in the headers of a packet with the process that emitted it is not a trivial task. The reason for this is the fact that identifying said process can only be done by instrumenting `sendmsg()` and other related system calls. By the time the payload effectively enters the network stack, the sought information would have already been lost. We decided to forgo this option for a simple reason: attaching proof of compliance to the legitimate traffic would require significant changes to the networking system.

An alternative solution that we considered and ultimately decided to utilize involves correlating a selection of fields (e.g., source and destination IPs and ports) with the originating socket of the packet. We would subsequently use the inode of the socket to identify the processes that *could* have sent the packet. Despite not being able to pinpoint the exact origin, this method gives a satisfactory approximation. All the information required for obtaining this mapping would normally be readily available to any `Xtables` rule match module that operates on the OUTPUT or POSTROUTING chains. However, it is not available on ingress. Since the destination socket is decided only after passing the PREROUTING and INPUT chain hooks and investigating the Layer 4 header, implementing this inference method as an `Xtables` extension would require a partial redesign of the Netfilter hook system.

We identified a different means to the same goal in the Netlink Socket Diagnostics subsystem. Originally designed as an alternative to `ioctl()` and an improvement over BSD routing sockets, Netlink sockets [23] are used for Inter Process Communication (IPC) and facilitate interaction between userspace and certain kernel subsystems (e.g., sensor communication, packet routing/filtering rule manipulation, etc.) The Socket Diagnostics subsystem allows querying the kernel for specific socket-related information, including state and inode.

Nonetheless, using the inode to identify the originating process of a packet raises a new challenge: socket file descriptors can easily be transferred between processes, and not only through inheritance on `fork()` or `clone()`. File descriptors can also be duplicated across processes using the ancillary `SCM_RIGHTS` socket type. In early 2020, this procedure had been further simplified with the addition of a new system call capable of copying file descriptors from other processes, namely `pidfd_getfd()`. Consequently, it would be ill-advised to regard only the original owner of the socket when deciding the identity of the sender. Instead, we must account for all open socket file descriptors, and their associated processes. Consequently, when evaluating an exclusionary match rule (i.e., DROP on object match), one process is sufficient to reach a verdict. Similarly, when we need to ensure that a certain version of a dependency is utilized, all potential originating processes must conform to the match criteria.

To this end, we trace a subset of system calls that are relevant for maintaining an accurate record of every socket that is accessible by one or more processes, as well as what those are. For example, `socket()` and `close()` may be relevant in following the activity of a single process. Meanwhile `fork()` or `clone()` compel us to copy the open file descriptors over to the child process in our model, by

virtue of inheritance. This solution does not account for the system state prior to starting the firewall. As a result, we are prepared to perform a `procfs` walk and inspect the symbolic links in `/proc/<pid>/fd/` when the need arises (i.e., when a packet is matched to a socket of which we are unaware). Nonetheless, we do not expect this operation to be issued more than once. In order to perform this system call trace, we use two complementary mechanisms: eBPF tracepoints and the Netlink Process Event Connector subsystem. The former consists of analysis functions that are compiled to an eBPF target and uploaded into the kernel at runtime. Both the passed arguments and the return value are communicated to our firewall via a BPF ring buffer. Note that this mechanism is specific to `libbpf` v1.0, introduced with the 6.0 kernel release.

The latter tracing mechanism is yet another Netlink subsystem. The Process Event Connector is similar in function with eBPF probes, in that it reports system calls, including arguments, return values and the PID of the caller. We initially chose this tracing method due to encountering inconsistent behaviour in eBPF-based tracing of certain syscalls. The reason why we still use eBPF is that the Process Event Connector is only able to report on a very limited set of system calls, many falling outside its ambit.

3.3 Filtering Mechanism

By using `NetfilterQueue`, the network traffic passes through two filtering stages (see Fig. 2). The first stage consists of the enforcement of the regular `iptables` rules. The second stage is entered only when one packet matches a rule with the `NFQUEUE` target. As a result of jumping to this target, the packet is passed to our userspace firewall instance for detailed

analysis. The analysis is heavily reliant on the network socket utilization model that we continuously update. Specifically, the firewall needs to guarantee the availability of a list of processes that could have generated the packet (on egress) or are ready to receive the packet (on ingress). Similarly to `iptables`, DAF implements three chains: `INPUT`, `OUTPUT` and `FORWARD`. A DAF rule also consists of match criteria and a verdict. While the verdict can only be `ACCEPT` or `DROP`, the match criteria can be more complex, in comparison.

The method we use the identity of a process to filter packets utilizes the objects mapped in its address space. The user can generate a rule requiring that a specific object (represented by its SHA256 digest) be either present or absent from all processes that are capable of generating or receiving the packets. Alternatively, the user can employ an aggregate hash of all file-backed objects

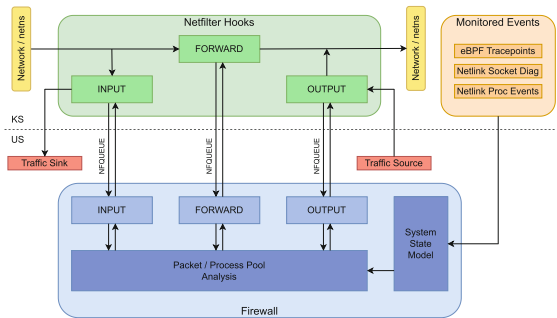


Fig. 2. Firewall architecture

with an executable section (sorted in alphabetical order on computation) to represent a specific version of a specific program, in a single rule. If the user wishes to issue a whitelist, then all processes must comply to this rule. Otherwise, if the user desires to enforce a blacklist, then a single process is sufficient to contravene the match criteria in order for the packet to be dropped.

The SHA256 digest itself is calculated based on the backing file of a memory mapped object, not on the in-process memory itself. The reason is twofold: monitoring memory access for every process without using static instrumentation is extremely costly, and excluding the writable sections to maintain the consistency of the digest gives leeway to an attacker. Although we assume that file permissions and ACLs are sufficient to protect the stored object files from tampering, one could extend DAF to monitor changes to hashed files via the inotify system.

Alongside the process identity matching mechanism, DAF also implements filtering based on IPs in CIDR notation, ports and Layer 4 protocol. These can all be configured by the DAF companion app, analogous to `iptables` itself. In addition to listing and managing rules, DAF is also capable of computing hashes and aggregate hashes for binaries and shared objects. This functionality was added to simplify the rule construction procedure.

Listing 1.1. DAF rule configuration with companion app

```
$ ctl-fw \
  -A          '# append rule' \
  -c OUTPUT  '# chain' \
  -v DROP    '# verdict' \
  -d 192.168.100.0/24 '# destination IP' \
  -! --dport 80 '# (not) destination port' \
  -p tcp      '# L4 protocol' \
  -n /proc/$$/ns/net '# network namespace' \
  --sng-hash $(sha256sum /usr/bin/curl \
                | awk '{print $1}')
```

Listing 1.1 is an example of DAF rule that targets all local processes that run in the current network namespace and make use of the `curl` base binary. The rule prevents these processes from generating TCP traffic to ports other than 80 in a given subnet. For performing an exact match on `curl` in such way as to include its shared objects dependencies, the `--agg-hash` flag can be used instead of `--sng-hash`.

3.4 Support for Docker Containers

The isolation guarantees that a Docker container offers its user depends on a number of Linux primitives that can be employed individually to lesser effects, namely cgroups, namespaces and OverlayFS (supplanting `aufs`). When implementing container support in DAF, we tested our solution on a Dockerized proof of concept for CVE-2022-3602 (an `OpenSSL` buffer overflow that occurs during the X.509 certificate verification). We identified two distinct challenges:

Resolving the Absolute Path of an Object: Because containerized processes utilize the mount namespace to an effect similar to that of `chroot`, the memory mapped objects present file paths that are relative to the mountpoint of the runtime OverlayFS layer. This mount is also called a *merge directory* and offers access to the union of read-only *lower* layers and the writable, instance-specific *upper directory*. Assuming that DAF can access this mountpoint via its own filesystem (which is always the case if ran in the top-level namespace), resolving the absolute path becomes trivial. However, identifying the mountpoint can be achieved only by obtaining the mount options of the root filesystem that is used by the container. For OverlayFS, these options contain the paths to the lower, upper and merge directories and can be extracted from `/proc/<pid>/mountinfo`. Whenever we need to ascertain the identity of a process believed to have generated a matched packet, we first check whether it belongs to a different mount namespace. If so, we extract the path of the mountpoint (i.e., the *merge directory*) and construct the absolute path of its objects.

Contending with Constrained Socket Diagnostics: Although the Netlink Socket Diagnostics subsystem is the preferred method of identifying sockets based on Layer 3 and Layer 4 information, it has a severe limitation. The scope of any query is restricted to the network namespace that the calling thread is bound to. Because in this scenario the target process essentially belongs to another (virtual) subnetwork, the routing table must be consulted in order to first determine if the process in question even resides on the same system as the firewall and if so, what network namespace it is bound to. In this situation, we opted for a simpler solution: adding a required argument to each rule, explicitly stating the applicable network namespace. We deem this requirement to be reasonable, considering that this information should be readily accessible to a user with sufficient permissions and capabilities to configure DAF. Because the events that drive the continuous update of our system state model are not constricted by namespaces, once the socket inode is identified, the associated (real) PIDs can also be retrieved. This in turn provides the solution to the previous problem.

3.5 Windows Integration

Although the Linux implementation of our firewall is closer to being feature complete, we also tested a simplified variant² based on the Windows Filtering Platform (WFP). Because the `Winsock` structure has the same problem of only identifying one process as the owner of the socket, this alternative implementation fulfills the same process identification criteria. In contrast to DAF however, we decided to implement it as a kernel module. Moreover, we attach the tag as a TCP option (while erasing pre-existing ones during the initial handshake) instead of an IP option. This decision was made in the early stages of development on DAF, when we did not yet have a clear understanding of the implications of annotating transport layer protocols. The main goal of the Windows imple-

² <https://github.com/david-gherghita/label-based-firewall-for-windows>.

mentation was to show that DAF can be adopted by it and not interfere with other network tools such as the build-in firewall of Windows.

4 Evaluation

4.1 Experimental Setup

The experiments described in this section have been carried out on the following hardware:

- **Intel NUC:** Intel Core i7-7567U CPU @3.50 GHz, Intel I219-V 1-Gigabit Ethernet Controller, 8 GB DDR4 memory @2400MT/s.
- **IBM System x3550 M4:** Intel Xeon E5-2650Lv2 CPU @1.70 GHz, Intel 82599ES 10-Gigabit SFP+ Ethernet Controller, 32GB DDR3 memory @1600MT/s.

The reported throughput was derived from the `iperf3` output and was based on the application data transfer, excluding any headers. Both systems are running a minimal installation of Arch Linux with kernel version 6.7.0.

4.2 Optimizations and Trade-Offs

During the development of DAF, in an effort to achieve a satisfying performance, we were confronted with a number of potential trade-offs. Although some had the capacity to improve throughput by up to an order of magnitude, each would incur a reduction in our security guarantees. Consequently, we decided to implement these optimizations as either build-time or run-time user options. Following is a list of said optimizations, as well as their associated CLI invocation flag.

- **Rescan prevention (R):** During the analysis of each packet, avoid scanning the virtual address space of each potential originator process, except during its initial identification. Subsequent queries will utilize the process state as it was ascertained during its first packet emission or reception. Dynamically loaded objects will not be accounted for.
- **Skip namespace switches (S):** Prevent switching network namespaces on consecutive rule evaluations that reference the same netns symlink. The symlink in question belonging to *procfs* would not be an issue in and of itself. However, external references to said symlink are often used in order to prevent the kernel from deleting the namespace after all processes belonging to it have terminated. This principle can be found implemented in the `netns` subcommand of `iproute2`.
- **Disable cached object ordering (build-time; U):** In order to match packets using an aggregate hash that is representative for a certain program, the comprising objects must be arranged in a deterministic manner. To this end, we decided to sort them alphabetically based on their absolute paths. In our implementation, we cache the hashes of all objects belonging to a process

as an unordered set, where operations have an average constant-time complexity. When retrieved by the Process Pool Analysis component, an ordered set (logarithmic complexity) is initialized from the contents of the unordered set, relying on its key comparison implementation to order the elements. During our experiments we discovered that we have initially underestimated the cost of this operation. As a result, we offer the choice of working directly with the unordered set in exchange for the ability to match whole programs, not just individual objects.

- **Batch verdict communication (b, B):** During our experiments we concluded that the packet data transfer between kernel space and user space is not a costly operation in and of itself. On the other hand, reporting the verdict for a given packet back to the `NetfilterQueue` module would block the process for what would at times amount to approx. 30% of the total execution time. As a result, we decided to implement a verdict batching mechanism. Although capable of significantly improving the performance of DAF, we note that this optimization is not as straightforward to utilize as the previously described ones, its performance being contingent on the type of processed traffic. Thus, we define user-adjustable upper limits for both the number of batched packets, and the amount of time that the verdict of any packet is allowed to be delayed (see Subsect. 4.3). Albeit difficult to suggest generic values that would guarantee an adequate performance increase regardless of the type of workload, we found it prudent to carry out forced verdict transmissions whenever a TCP SYN or TCP PSH is encountered. The reason behind the former is to avoid delaying the establishment of new TCP connections. As for the latter, postponing the transfer of what is expected to be a complete message to the endpoint process could provoke unwanted transmission delays at the application level.

4.3 Netfilter Queue Tuning

When selecting the parameters for the **batched verdict communication optimization (b, B)**, we first determine the lower bounds for the maximum batch size and transmission timeout. Choosing the values for either of these parameters while falling short of this threshold can detract from the performance improvement of the optimization, to the point of completely negating it (e.g. size = 1).

In order to empirically determine the aforementioned thresholds, we performed six rounds of experiments, varying the maximum batch size by 1 packet and the timeout by 1 μ s in the 1–100 range (thus obtaining a dataset of 60,000 throughput values). Figure 3 illustrates the impact of each of the two tuning parameters via the average throughput on a 10Gbps link with constant MTU. In order to exemplify the effect of one tuning parameter independently from the other, the dimensions corresponding to the second parameter as well as the experiment round had to be collapsed. Consequently, each average throughput data point was calculated over a subset of 600 values representing the splice obtained from fixing the value on the abscissa.

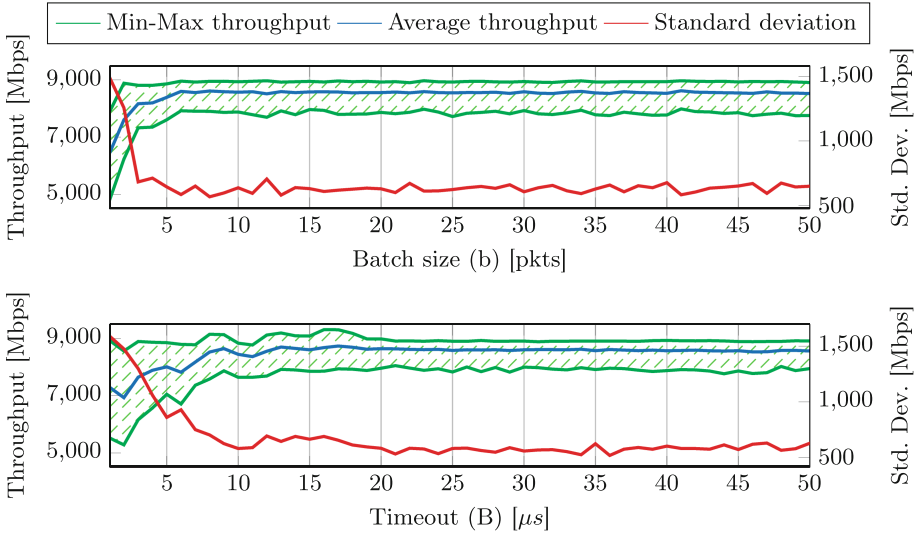


Fig. 3. Influence of verdict batch size & timeout on throughput.

For the minimum-maximum intervals we decided to fix both the experiment round and one of the tuning parameters, in order to then average the per-experiment minimums and maximums. The motive behind this was to de-emphasize outliers. To compensate for this and correctly represent the throughput variations, we included the standard deviation obtained from the same data slice that was used to calculate the average throughput. Here, we mention that the value range for both plots was truncated to 1–50 in order to better illustrate the impact of suboptimal parameter choices. The subsets used in the calculation of the average throughputs have not been proportionately truncated.

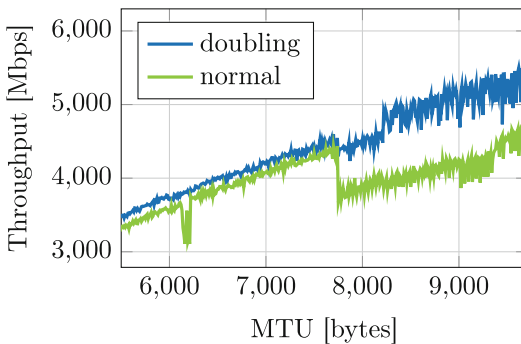


Fig. 4. Effect of introducing dynamic resizing of Netlink socket buffer.

One other value that we concluded needed to be fine tuned is the **Netlink socket receive buffer size**. While this socket is used for communication between userspace and the **Netfilter Queue** kernel driver, its receive buffer is also used to store packets that are pending a verdict, only to later be reintroduced in the network stack. In the eventuality that this buffer fills (a common occurrence when the userspace

application is slow to reach a verdict) new packets are discarded and the following `read()` operation fails with the `ENOBUFFS` error code, as an informative

measure. Ignoring this problem can lead to aliasing issues that normally manifest as illustrated in Fig. 4. Insufficient TCP socket buffer sizes may also lead to similar performance downgrades. In response to these events, we assess the current Netlink socket buffer size and double it. This action is performed using the `SO_RCVBUFFORCE` socket option as to override the system maximum socket size. We note that after doubling the buffer size from userspace, the kernel doubles it once more in order to allow for bookkeeping overhead.

4.4 Performance Analysis

Our performance analysis of DAF is primarily based on `iperf3` throughput tests between a client running our firewall and a server on another identical host. The two hosts are directly linked, with no additional equipment serving as middleboxes. We note that the values we report represent the average throughput over 10s sessions, averaged once more over 10 and 5 rounds of experiments on the IBM server and NUC system respectively. On the 10-Gigabit servers, we decided to manually increase the default and maximum TCP socket buffer sizes, a common practice in such cases. Although in our final reports these are set to 32MB, we did not perceive any noticeable performance gain for values higher than 8 MB. The NUC desktop computers utilize the default maximums of 4 MB for the send buffer and 6 MB for the receive buffer.

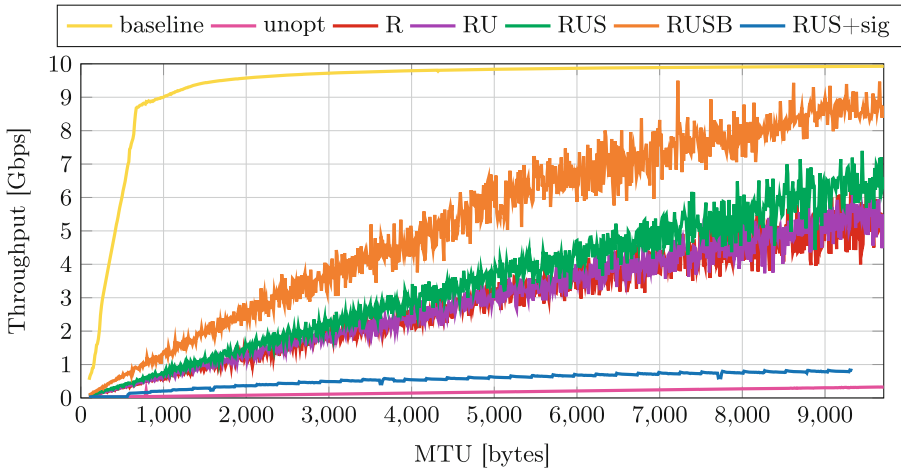


Fig. 5. DAF throughput on IBM system.

Figure 5 illustrates the performance gain of each optimization. Here, we note the exceedingly high variations. We attribute these to the use of the `io_uring` interface for asynchronous I/O operations. In our implementation, we decided to launch a kernel thread on a different CPU core that continuously polls the system call submission queue in DAF. Prior to this revision, we employed an `epoll` hierarchization system to permit us the prioritization of certain events (e.g.,

system state model updates) over other. However, profiling sessions revealed that our firewall spent over 50% of the time waiting for `epoll` events, limiting the average throughput with full optimizations to approx. 5.4 Gbps. By not only offloading the majority of I/O operations (consisting mostly of packet reads from the netlink socket) to a secondary core but also drastically limiting the amount of context switches between protection domains, we have increased the average throughput to approx. 8.8 Gbps. We believe that the gap between this and the baseline throughput can be bridged by extending `libnetfilter_queue` to allow asynchronous batched verdict transmission.

In the same figure we also depict the throughput of DAF on the 10-Gigabit server while also calculating and attaching SHA256-HMACs of the Layer 4 payload as IP options. Notice how we forgo the use of verdict batching since it would not permit us to re-inject the modified packet into the kernel, strictly due to a `NetfilterQueue` API limitation. Once again, we believe that this limitation could be alleviated by adding a simultaneous packet override through an `iovec` interface. However, this would also require changes not only to `libnetfilter_queue` but also the `NetfilterQueue` driver, both of which we consider outside the scope of this paper. During this experiment we manually enforced a smaller Maximum Segment Size (MSS) for `iperf3` in order to ensure that the newly introduced 36-byte IP options section would not exceed the MTU at any given time. Note however, that `iperf3` clamps the MSS values within the 88–9216 range, thus leading to severe performance downgrades due to fragmentation. Under these conditions, we register a peak average throughput of approx. 835 Mbps.

Observing the measurements on the Intel NUC mini-PC from Fig. 6, we conclude that it is significantly easier to obtain near-linear throughput both when annotating traffic and when only performing packet filtering. These results are in part due to the single CPU core speed, motivated by the fact that DAF is not multi-threaded. We note that the Intel driver for gigabit network adapters (i.e., `e1000e`) imposes a minimum MTU value of 604 bytes. Any attempt to set a lower value is met with a reset of the network interface, coupled with a silent override of the MTU, using this lower limit.

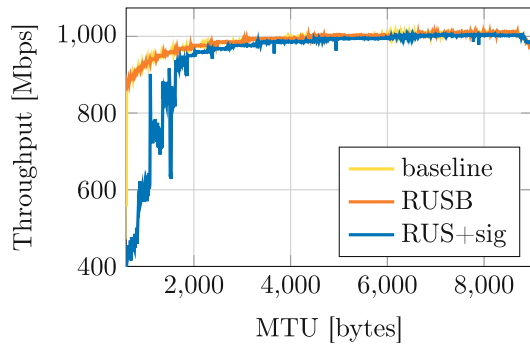


Fig. 6. DAF throughput on Intel NUC.

4.5 Integration with Xtables and snort3

In order to leverage the benefits of DAF even on end hosts where it is not possible, or at least not practical to be deployed, we have implemented two out-of-tree extensions for `iptables` and `snort3`. The former consists of a shared object that implements the module-specific `iptables` frontend, and a `Xtables` kernel module that accomplishes the packet match evaluation. Because the latter operates in userspace and the packet retrieval is deferred to `libDAQ` (i.e., its data acquisition library), both functions are accomplished in the same resulting shared object (i.e., the IPS module). We note that neither extension requires its associated tool to be recompiled, both being dynamically loaded at runtime. In both cases, we consider a packet match to occur if the computed SHA256-HMAC over the Layer 4 payload (with a user-provided secret) corresponds to the contents of an experimental IP option found in the same packet.

5 Related Work

In 2008, Srivastava et al. proposed `VMwall` [24], a Xen-based application-level firewall running in `dom0`. By leveraging VM introspection to identify the owner process of the socket bound on the source port of the initial packet, the decision of allowing the connection to be established is made based on the name of the underlying executable. Although `VMwall` presents a smaller attack surface, DAF provides a stronger assurance against attackers capable of hijacking the originating process at arbitrary moments in time. Furthermore, our solution permits finer grained application filtering, based on the hashes of its memory mapped objects, also taking into account the realities of socket sharing. Finally, DAF attaches HMACs to the packets, serving both as proof of origin, and as proof that a certain packet did not bypass the filtering mechanism.

`DStar` [33] is a Distributed Information Flow Control (DIFC) system that extends local implementations of specialized operating systems such as `HiStar` [32] and `Flume` [16] over a network. In `DStar`, all network communication is tagged with labels that either restrict it or grant other processes privileges over the data they receive. This solution is similar to DAF in that it requires the traffic passing through a component that enforces these constraints (i.e. exporter daemon). One notable difference however, is that under `DStar` each process is responsible for appending its label to the generated traffic as part of the payload whereas with DAF, this is transparent to the process.

In 2000, Ioannidis et al. [13] brought to attention the limitation of perimeter firewalls and their inadequacy in isolating untrusted hosts within the internal network. As a result, they introduced a distributed firewall scheme that transferred the responsibility of traffic validation to individual hosts. While they used IPsec for attesting the authenticity of either endpoint, we contend that using encrypted channels is not always desirable. In our view, a distributed firewall should ensure authentication and integrity, but not confidentiality, as to not interfere with any IDS in an internal network. This approach has the added benefit of allowing middleboxes to enforce their own filtering policies.

Assayer [19] is a solution dating from 2012 by Parno et al. that leveraged a minimal hypervisor (MiniVisor) and modular counters that would track application-specific information, to later be appended to network traffic. This information would aid in certain endeavors (e.g. spam email detection). The main difference between **Assayer** and our solution is that neither MiniVisor nor the kernel enforce the traffic annotation scheme. As a result, each application must be aware of this service and opt into using it. Additionally, their annotation scheme does not conform to existing protocol extension standards, meaning its utilization should be predicated by a negotiation phase between endpoint applications lest it broke the application-level communication.

Dymo [9] is a system that implements a code identity primitive that can generate application-specific labels based on the memory mapped objects of a process. While the authors acknowledge the possibility of encountering JIT-ed code, they exclude it from the label calculation on the basis that code generated by a trusted binary is itself trusted. Additionally, Gilbert et al. provide an extension which allows the addition of the generated labels to network traffic as IP options. At a glance, the most significant difference between **Dymo** and **DAF** is that the former computes the cryptographic hashes over the virtual memory of the target process while the latter does so based on the backing files (i.e. loaded from the file system). However, **Dymo** also proposes a *relaxed matching policy* where only the executable sections of a subset of binaries are taken into consideration. Another distinction is that **Dymo** is implemented as a Windows XP kernel module that hooks the page fault handler, so a direct port to Linux would not be possible without modifying the kernel.

BorderPatrol [36] is a more recent solution aimed at enforcing BYOD policies in Android devices. When new applications are introduced, an Offline Analyzer indexes all methods found in class metadata. Meanwhile, a Context Manager process running on the users' devices monitors all socket operations and attaches information regarding the calling function as IP options. While **BorderPatrol** is used to block specific features that rely on network communication, these features are identified only by the method name and class path. This approach may have some merit against an unknowing assailant (e.g. mistakenly uploading company files to their personal cloud). However, an attacker may circumvent the underlying Xposed Framework by either targeting an ELF binary (which is not forked from Zygote) or exploiting a vulnerability in the Dalvik VM.

6 Future Work

The current implementation of **DAF** suffers from excessive complexity in many areas. This is a result of an early decision to constrict ourselves to userspace (with the exception of eBPF probes). As a result, further improvements to **DAF** will become increasingly harder to integrate.

In order to address this problem, we decided to start working on a Linux Kernel Module (LKM) version of **DAF**. This new iteration will utilize kprobes

to monitor certain key execution paths throughout the kernel and extract the information required to generate the process identity model. The utilization of kprobes will allow us to interact with an unmodified kernel: the LKM could reside in the initial ramdisk and be loaded before pivoting the root filesystem and replacing the `init` process image, requiring only that the user adds the initramdisk to the TCB of his system.

As we plan to explore other uses of a more efficient process identity measurement module (e.g., remote attestation of processes), we decided to separate this component from the `Xtables` match module (i.e., the firewall unit). Additionally, we intend to implement a key management system (that is currently missing from DAF) based on the Kernel Key Retention Service.

7 Conclusion

In this paper we explored an alternative approach to application awareness that does not rely on deep packet inspection. By leveraging the ability to gain local system-level knowledge of the constituent executable objects of a given process, and disseminating this information in the context of a distributed firewall, we proved that it is possible to more accurately ascertain application identity without compromising the confidentiality of user data.

In demonstrating this we implemented DAF, a prototype distributed firewall of approx. 3500 LoC that can discern potential endpoint processes, filter traffic based on specific versions of memory mapped objects, and annotate outgoing traffic. As we acknowledge the effort that is oftentimes required to integrate emerging technologies with pre-existing software stacks, we implemented support for cross-namespace process identification. As a result, our solution can also be deployed in microservice-rich environments. Additionally, we offer both `iptables` and `snort3` plugins (470 LoC total) capable of matching DAF packet annotations.

Finally, we offer an evaluation of the performance of DAF on both consumer-grade hardware that would usually be found in workplace environments, as well as servers connected to 10-Gigabit networks. Additionally, we provide an analysis of the expected impact of each firewall optimization that may be enabled with the intent of prioritizing throughput over security guarantees.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Akbari, I., et al.: A look behind the curtain: traffic classification in an increasingly encrypted web. *Proc. ACM Meas. Anal. Comput. Syst.* **5**(1), 1–26 (2021)
2. Basta, N., Ikram, M., Kaafar, M.A., Walker, A.: Towards a zero-trust micro-segmentation network security strategy: an evaluation framework. In: *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, pp. 1–7. IEEE (2022)

3. Butler, S., et al.: On business adoption and use of reproducible builds for open and closed source software. *Softw. Qual. J.* **31**(3), 687–719 (2023)
4. Camarillo, G., Audet, F., Rosenberg, J., Boulton, C.: NAT traversal practices for client-server SIP. RFC 6314 (2011). <https://doi.org/10.17487/RFC6314>, <https://rfc-editor.org/rfc/rfc6314.txt>
5. Davidson, M., Marchuk, W., Zaugg, A., Garate, M., Buenzle, W.: Distributed firewall (DFW): network security at the host level at LinkedIn (2021). <https://engineering.linkedin.com/blog/2021/distributed-firewall-network-security>
6. Fairhurst, G., Jones, T., Zullo, R.: Checksum compensation options for UDP options. Internet-Draft draft-fairhurst-udp-options-cco-00, Internet Engineering Task Force (2018). <https://datatracker.ietf.org/doc/draft-fairhurst-udp-options-cco/00/>, work in Progress
7. Fonseca, R., Porter, G., Katz, R., Shenker, S., Stoica, I.: IP options are not an option. Technical report, EECS Department, University of California, Berkeley (2005)
8. Gancheva, Z., Sattler, P., Wüstrich, L.: TLS fingerprinting techniques. *Network* **15** (2020)
9. Gilbert, B., Kemmerer, R., Kruegel, C., Vigna, G.: DYMO: tracking dynamic code identity. In: Sommer, R., Balzarotti, D., Maier, G. (eds.) RAID 2011. LNCS, vol. 6961, pp. 21–40. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-23644-0_2
10. Gonçalves, G., O'Malley, K., Saltonstall, M., et al.: BeyondCorp and the long tail of zero trust (2023)
11. Goodchild, B.J., et al.: The record route option is an option! In: Proceedings of the 2017 Internet Measurement Conference, pp. 311–317 (2017)
12. Heino, J., Hakkala, A., Virtanen, S.: Study of methods for endpoint aware inspection in a next generation firewall. *Cybersecurity* **5**(1), 25 (2022)
13. Ioannidis, S., Keromytis, A.D., Bellovin, S.M., Smith, J.M.: Implementing a distributed firewall. In: Proceedings of the 7th ACM Conference on Computer and Communications Security, pp. 190–199 (2000)
14. John, P.: Transmission control protocol. RFC 793 (1981)
15. Kazemian, P., Varghese, G., McKeown, N.: Header space analysis: static checking for networks. In: 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2012), pp. 113–126 (2012)
16. Krohn, M., et al.: Information flow control for standard OS abstractions. *ACM SIGOPS Oper. Syst. Rev.* **41**(6), 321–334 (2007)
17. Ohm, M., Plate, H., Sykosch, A., Meier, M.: Backstabber's knife collection: a review of open source software supply chain attacks. In: Maurice, C., Bilge, L., Stringhini, G., Neves, N. (eds.) DIMVA 2020. LNCS, vol. 12223, pp. 23–43. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-52683-2_2
18. Osborn, B., McWilliams, J., Beyer, B., Saltonstall, M.: BeyondCorp: design to deployment at Google (2016)
19. Parno, B., Zhou, Z., Perrig, A.: Using trustworthy host-based information in the network. In: Proceedings of the Seventh ACM Workshop on Scalable Trusted Computing, pp. 33–44 (2012)
20. Richard, S.: Four paths to true network security. Technical report, Commentary COM-20-0571, Gartner Research (2003)
21. Radivilova, T., Kirichenko, L., Ageyev, D., Tawalbeh, M., Bulakh, V.: Decrypting SSL/TLS traffic for hidden threats detection. In: 2018 IEEE 9th International Conference on Dependable Systems, Services and Technologies (DESSERT), pp. 143–146. IEEE (2018)

22. Mantu, R., Chiroiu, M., Tăpus, N.: Framework for evaluating TCP/IP extensions in communication protocols. *Int. J. Comput. Commun. Control* (April) **19** (2024)
23. Salim, J., Khosravi, H., Kleen, A., Kuznetsov, A.: Linux netlink as an IP services protocol. RFC 3549, RFC Editor (2003)
24. Srivastava, A., Giffin, J.: Tamper-resistant, application-aware blocking of malicious network connections. In: Lippmann, R., Kirda, E., Trachtenberg, A. (eds.) RAID 2008. LNCS, vol. 5230, pp. 39–58. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-87403-4_3
25. Stafford, V.: Zero trust architecture. NIST Spec. Publ. **800**, 207 (2020)
26. Touch, D.J.D.: Shared use of experimental TCP options. RFC 6994 (2013). <https://doi.org/10.17487/RFC6994>, <https://rfc-editor.org/rfc/rfc6994.txt>
27. Touch, D.J.D.: Transport options for UDP. Internet-Draft draft-ietf-tsvwg-udp-options-23, Internet Engineering Task Force (2023). <https://datatracker.ietf.org/doc/draft-ietf-tsvwg-udp-options/23/>. Work in Progress
28. Touch, D.J.D., Eddy, W.: TCP extended data offset option. Internet-Draft draft-ietf-tcpm-tcp-edo-13, Internet Engineering Task Force (2022). <https://datatracker.ietf.org/doc/draft-ietf-tcpm-tcp-edo/13/>. Work in Progress
29. Trieu, H., Touch, J., Faber, T.: Implementation of the TCP extended data offset option. USC Information Sciences Institute, Marina Del Rey, CA, USA, Technical report, ISI-TR-696 (2015)
30. Ward, R., Beyer, B.: BeyondCorp: a new approach to enterprise security (2014)
31. Yun, X., Wang, Y., Zhang, Y., Zhao, C., Zhao, Z.: Encrypted TLS traffic classification on cloud platforms. *IEEE/ACM Trans. Netw.* (2022)
32. Zeldovich, N., Boyd-Wickizer, S., Kohler, E., Mazieres, D.: Making information flow explicit in HiStar. *Commun. ACM* **54**(11), 93–101 (2011)
33. Zeldovich, N., Boyd-Wickizer, S., Mazieres, D.: Securing distributed systems with information flow control. In: NSDI, vol. 8, pp. 293–308 (2008)
34. Zhang, Y., Zhao, S., Zhang, J., Ma, X., Huang, F.: STNN: a novel TLS/SSL encrypted traffic classification system based on stereo transform neural network. In: 2019 IEEE 25th International Conference on Parallel and Distributed Systems (ICPADS), pp. 907–910. IEEE (2019)
35. Zullo, R., Jones, T., Fairhurst, G.: Overcoming the sorrows of the young UDP options. In: TMA (2020)
36. Zungur, O., Suarez-Tangil, G., Stringhini, G., Egele, M.: BorderPatrol: securing BYOD using fine-grained contextual information. In: 2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), pp. 460–472. IEEE (2019)

Privacy and Personal Data Protection



Privacy-Preserving Regular Expression Matching Using TNFA

Ning Luo^{1(✉)}, Chenkai Weng², Jaspal Singh³, Gefei Tan^{2(✉)},
Mariana Raykova⁴, and Ruzica Piskac⁵

¹ University of Illinois Urbana-Champaign, Champaign, IL, USA
nl27@illinois.edu

² Northwestern University, Evanston, IL, USA
{ckweng, gefeitan}@u.northwestern.edu

³ Purdue University, West Lafayette, IN, USA
singjasp@oregonstate.edu

⁴ Google, Mountain View, CA, USA
marianar@google.com

⁵ Yale University, New Haven, CT, USA
ruzica.piskac@yale.edu

Abstract. Regular expression (*regex*) is a powerful tool to efficiently search patterns in text strings. However, it is common when the regex and string are privately held by different parties who are unwilling to reveal sensitive information. In this work, we provide solutions for private regex pattern match.

By exploiting the structure of Thompson nondeterministic automata, we design a regex match algorithm with $O(mn)$ complexity where m and n are the length of the string and regex. It is oblivious to the input string and, hence, is friendly to private evaluation. We develop two 2-party regex matching protocols that take input a private string from one party and a private regex from another. The first protocol uses oblivious stack as a building block and reduces the communication complexity of the state-of-the-art secure regex protocols from $O(mn^2)$ to $O(mn \log n)$. The second protocol is based on the oblivious transfer and shows better empirical results for short regex in low-latency networks.

We also evaluated our protocols in the context of joint intrusion detection and DNA sequence search. The results indicate one more step toward the practicality of private regex pattern matching in real-world applications.

1 Introduction

A pattern match algorithm takes as input a string and a pattern and checks whether the given pattern appears in the string. It has been widely used in many areas, including bioinformatics [5], database search engines [9], intrusion detection systems [36], text processing [22], and digital forensics [4]. Matching patterns are commonly represented as regular expressions (regex), as regex are

Table 1. Comparisons with existing secure-regex protocols. κ , m , n and Σ are the computational security parameter, length of the string, the size of the regex, and the alphabet, respectively. Protocols [14, 18, 25] exhibit complexity exponential to the size of the regex because they are DFA-based.

Protocol	Comm. Complexity	Round Complexity
[18]	$O(m2^n \Sigma)$	$O(m)$
[14]	$O(m(2^n + \Sigma))$	$O(1)$
[25]	$O(m2^n\kappa)$	$O(1)$
[23]	$O(\kappa mn(n \Sigma))$	$O(m)$
[30]	$O(\kappa n^2(m + \Sigma))$	$O(mn)$
Ours (from OS)	$O(\kappa mn(\log n + \log \Sigma))$	$O(1)$
Ours (from OT)	$O(mn(n + \kappa \log \Sigma))$	$O(mn)$

able to match text using a structured pattern and are commonly used in search engines, word processors, and programming languages.

In this paper we focus on the regex matching problem, which emerges from the needs of specific privacy-preserving applications: *two-party pattern match*. In this setting, a pattern holder and a string holder want to verify whether the pattern matches the string. Both parties should not learn anything beyond the match result and the length of inputs. We refer to this as *secure-regex* and realize it by secure two-party computation (2PC) protocols [41]. A typical application is privacy-preserving network intrusion detection where a third-party intrusion detection system (IDS) evaluates sensitive network packets without knowing their contents. The evaluation is done by secret regex developed by IDS, which is viewed as intellectual property and thus is not open for access [32]. For the example of DNA sequence search, one party holds a regex that describes genetic mutation and another party holds patient DNA sequences. Such pattern match facilitates clinical diagnoses and neither party is able to reveal its input.

Challenges. Generic 2PC techniques are able to securely process an algorithm if it is “oblivious” to its input. Here an algorithm is termed oblivious if the control flow and memory access pattern are independent of the inputs except for their length. However, regex pattern match in plaintext relies on finite automata simulation, and the access pattern of known algorithms and data structures are highly dependent on the input regex and string. Hence additional oblivious data structures or algorithms should be used to prevent information leakage. Existing work [23, 30] utilize matrices to represent finite automata and perform linear scans over the matrices for simulation. These approaches introduce overhead that is quadratic to the size of the finite automata and regex, which makes the protocol relatively impractical.

Our Main Observation. We develop a novel algorithm for pattern match that employs more efficient linear scans over finite automata for simulation. Specifically, our approach utilizes Thompson nondeterministic finite automata (TNFA), a type of finite automata commonly employed in regex pattern match. Our key insight is that a TNFA can be viewed as a special sparse graph wherein all nodes can be ordered, such that all accessible nodes can be reached via a path with at most one “backward” edge. This observation enables our algorithm to simulate TNFA while maintaining an access pattern independent of the input string, with only a small constant overhead. Nonetheless, the access pattern still relies on the TNFA graph, which reveals partial information about the underlying regex. To address this issue, we propose protocols for secure-regex in which the TNFA graph’s information is protected using cryptographic building blocks such as oblivious stack or oblivious transfer. They lead to the most practical performance yet known.

1.1 Our Contribution

The pattern match in plaintext consists of character transition and epsilon transition (details in Sect. 3). In privacy-preserving settings, the character transition can be executed directly through linear scans and equality checks, which are easily handled by 2PC protocols. Thus, the main challenge of this work is to design data-oblivious epsilon transitions. Epsilon transitions involve searching for all reachable states in the TNFA. In the plaintext execution, this search can be performed using a depth-first search (DFS). However, the best-known oblivious DFS for general graphs requires a circuit size of $O(n^2)$, leading to a total complexity of $O(mn^2)$ to process a length- n regex and a length- m string [6]. Additionally, the evaluation results demonstrate that the concrete complexity is considerable due to the substantial constant factor involved. In this paper, we present protocols that achieve the best-known asymptotic complexity and improved concrete complexity. We also provide evaluations over real-world benchmarks.

TNFA Simulation via Two Linear Scans. We first propose a pattern match algorithm that uses two linear scans for epsilon transitions, and takes advantage of the fact that the TNFA can be modeled as a sparse graph. Specifically, we observed that all reachable nodes in the TNFA can be accessed via a path with at most one backward edge in the epsilon transition. This insight enables us to derive the elements of the reachable set in each round by performing just two linear scans instead of $O(n)$. Importantly, the access pattern of the epsilon transition is independent of the input string.

Secure-Regex. To minimize the need for additional data-oblivious operations to protect the graph structure, we further investigate the properties of TNFAs and prove a bound on the maximum degree of each node. By limiting the number of predecessors and successors of each node, we develop two efficient two-party protocols for secure-regex. Define κ to be the computational security parameter.

The first protocol, based on the oblivious stack (OS), is a constant-round protocol with a communication complexity of $O(\kappa mn(\log n + \log |\Sigma|))$, outperforming the previous best-known result of $O(\kappa n^2(m + |\Sigma|))$ [30]. The second protocol, based on 1-out-of- $n+1$ oblivious transfer (OT), achieves a communication overhead of $O(mn(n + \kappa \log |\Sigma|))$ with a linear number of round-trips while incurring a higher asymptotic complexity. However, it outperforms the first protocol in scenarios with low network latency and short input regex. Comparisons with related work on both asymptotic complexity (Table 1) and empirical performance (Fig. 7b) demonstrate that our approach achieves both asymptotic and empirical optimality for secure-regex.

Implementation and Evaluation. To empirically evaluate our proposed protocols, we implemented both the OT-based and OS-based secure-regex protocols and measured their performance. Our secure-regex protocols attain the best-known results for two-party regex match, and their practicality is demonstrated in real-world DNS sequence match and intrusion detection based on SNORT PCRE [11]. The OT-based protocol performs a match on a 512-byte string and a 63-byte pattern in 4s in a LAN with 100Mbps bandwidth. In a WAN with 60ms latency, our OS-based protocol performs a match on a 512-byte string and a 15-byte pattern in 4s. We conducted a comparison of our secure-regex implementations that utilize OS-based and OT-based protocols, testing them under varying network and input conditions. Additionally, we compared our OT-based protocol secure-regex against the prior state-of-the-art [30]. Our findings indicate that our OT-based protocol reduces communication costs by up to $164.5\times$ for 2^{10} -byte regex.

1.2 Related Work

Previous secure two-party pattern matching protocols [2, 12, 17, 20, 29, 37, 42] support string operations such as exact matching, substring matching, approximate matching, and wildcard matching. In contrast, our methods enable the use of more expressive languages, i.e., regular expressions, to specify patterns. A comparison of the communication and round-trip complexity of regex-based pattern matching protocols can be found in Table 1.

Secure protocols for general NFA evaluation have been proposed in [23, 30]. They either employ computationally intensive homomorphic encryption or have a communication complexity of $\Omega(mn^2\kappa)$ (where κ is the security parameter), which makes them less practical than our protocols. Protocols in [14, 18, 23, 25, 35] involve the private evaluation of DFA, whose size also suffers from the exponential blow-up compared to the NFA size. We refer to the empirical evaluation and comparison presented in [30] (Sect. 6.1) on the size of DFA and NFA for the same regex. Moreover, the utilization of data-oblivious graph algorithms as presented in [6, 39] can be extended to simulate TNFAs in a secure two-party computation setting, but this approach results in quadratic complexity relative to the length of the regex.

2 Preliminaries

2.1 Regular Expressions and Thompson NFAs

In this section, we introduce formal definitions for regular expressions and for NFAs. We present Thompson's construction [34], which shows how any regular expression can be converted into an equivalent NFA that accepts the same set of strings as the regular expression does.

Regular Expression. A regular expression (regex) on the alphabet Σ consists of characters in Σ and meta characters $\{ |, (,), * \}$. Let ε and ϕ denote the empty string and the empty set, respectively. A regular expression re describes a set of strings (or language) $L(\text{re})$ over Σ recursively:

1. basic case: $L(\phi) = \phi$, $L(\varepsilon) = \{\varepsilon\}$ and $L(a) = \{a\}$ for $a \in \Sigma$;
2. concatenation: $L(\text{re re}') = \{xx' | x \in L(\text{re}), x' \in L(\text{re}')\}$;
3. union: $L((\text{re}|\text{re}')) = L(\text{re}) \cup L(\text{re}')$;
4. loop: $L((\text{re})^*) = \bigcup_{k \in \mathbf{N}} \{x_0 x_1 \cdots x_k | x_i \in L(\text{re})\} \cup \{\varepsilon\}$.

Nondeterministic Finite Automata. A nondeterministic finite automata (NFA) N is defined as a 5-tuple $(Q, \Sigma, \delta, S_0, Q_A)$. Q is a set of states and Σ is the alphabet set. $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow 2^S$ is a transition function that maps a state along with character to a set of states. For states S, S' we write $S \xrightarrow{\varepsilon} S'$ if either $S = S'$ or there exists a sequence of states $S_1, \dots, S_n$ with $S = S_1$ and $S_n = S'$ such that for all $1 \leq i < n$, $S_{i+1} \in \delta(S_i, \varepsilon)$. For $x \in \Sigma$ and states S, S' , we write $S \xrightarrow{x} S'$ when there are states S_1 and S_2 such that $S \xrightarrow{\varepsilon} S_1$ and $S_2 \in \delta(S_1, x)$ and $S_2 \xrightarrow{\varepsilon} S'$. An NFA N accepts a string $x_0 \cdots x_{m-1}$ on Σ if there exists a sequence of states $S_0, S_1, S_2, \dots, S_m$, such that 1) S_0 is the starting state; 2) for $0 \leq i \leq m-1$, $S_i \xrightarrow{x_i} S_{i+1}$; and 3) $S_m \in Q_A$ is an accepting state.

Thompson NFA Construction. An NFA, N , is equivalent to a regular expression, re , if $L(N) = L(\text{re})$. It is well known that any regular expression can be converted to an equivalent NFA using Thompson's construction, which is called a Thompson NFA (TNFA). We adopt a specific variant [31] of the TNFA construction. A characteristic of a TNFA denoted by such G is that all edges within the alphabet Σ , denoted by E_Σ , are of the form $(j-1, j)$, where j is a specific integer.

Given a regular expression, re , the algorithm processes each character in re in order and outputs a directed graph, $G = (V, E = E_\Sigma \cup E_\varepsilon)$ where $V = \{0, \dots, n\}$. G can be interpreted as a TNFA as follows:

- $Q = V$, $S_0 = 0$ and $Q_A = \{n\}$;
- $j \in \delta(i, \varepsilon)$ if $(i, j) \in E_\varepsilon$;
- $j \in \delta(i, c)$ if $(i, j) \in E_\Sigma$ is labeled by $c \in \Sigma$;
- $\delta(i, c) = \phi$ for the rest of the undefined $(i, c) \in V \times (\Sigma \cup \varepsilon)$.

By this way, the graph $G = (V, E = E_\Sigma \cup E_\varepsilon)$ defines a TNFA, $N = (V, \Sigma \cup \varepsilon, \delta, 0, \{n\})$.

As the set of nodes in the directed graph is the set of states, we do not differentiate states and nodes without raising confusion in the rest of this paper. The output TNFA is equivalent to the language defined by re [31].

TNFA Simulation. Taking as input TNFA G for re and a string $x = x_0 \dots x_{m-1}$, the TNFA simulation decides if x is an element of $L(\text{re})$ by checking whether or not G accepts x . The TNFA simulation algorithm is listed in Algorithm 1. It simulates the operation of an NFA by computing Q_i , which includes all reachable states after processing the first i characters of x . The algorithm finally outputs if S_n is in Q_m .

Computing Q_i from Q_{i-1} involves finding all states that can be accessed through a path $e_0, \dots, e_\ell$, where $e_0 \in E_\Sigma$ is labeled x_{i-1} and $e_i \in E_\varepsilon$ for $i > 0$. The process is divided into two steps. First, the algorithm sets Q_i to all states that can be reached from Q_{i-1} through $e_0 \in E_\Sigma$ (lines 5–7 in Algorithm 1). As all edges in E_Σ have the form $(j-1, j)$, Q_i can be set up through a linear scan. This step is referred to as the *character transition*. Second, the algorithm extends Q_i to all states reachable from Q_i through any number of epsilon transitions (line 8 in Algorithm 1). This is done by finding all reachable nodes through a depth-first search (DFS). This step is referred to as the *epsilon transition*.

For simplicity of presentation of our protocols, from here onward we will represent a regular expression of size n by a graph of epsilon transitions $G = (V, E_\varepsilon)$ and an array re of size $n+1$, such that $\text{re}[j]$ equals the character transition for the edge $(j, j+1)$ if it exists, else it is set to $\perp$.

Functionality $\mathcal{F}_{\text{OT}}$

- Upon receiving $(\text{ot}, N, \ell, \{m_i\}_{i \in [0, n)})$ from P_0 and (ot, N, ℓ, b) from P_1 such that $m_i \in \{0, 1\}^\ell$ and $b \in [0, N)$, send m_b to P_1 .
- Upon receiving (rot, N, ℓ) from P_0 and P_1 , uniformly sample $(m_0, \dots, m_{N-1}) \leftarrow \{0, 1\}^{n \times \ell}$ and $b \in [0, N)$. Send $\{m_i\}_{i \in [0, N)}$ to P_0 and (b, m_b) to P_1 .

Fig. 1. The functionality of oblivious transfer.

Functionality $\mathcal{F}_{2PC}$

For $i \in [0, 1]$, upon receiving $(\mathcal{C}, x_i)$ from P_i , compute $(y_0, y_1) \leftarrow \mathcal{C}(x_0, x_1)$. Send y_i to P_i .

Fig. 2. The functionality of two-party computation.

2.2 Cryptographic Preliminaries

We define the following cryptographic building blocks and defer the protocol instantiation to Sect. 5.

Oblivious Transfer (OT). It takes N messages $(m_0, \dots, m_{N-1})$ from a sender and a choice index b from a receiver. The receiver receives m_b without knowing $\{m_i\}_{i \neq b}$ while the sender has no information on b . The random oblivious transfer (ROT) differs from OT such that both $\{m_i\}_i$ and b are uniformly sampled.

Two-Party Computation (2PC). It allows two mutually untrusted parties to jointly compute a public function over their private inputs without leaking anything beyond the output. Our protocols are proven secure against semi-honest adversaries in simulation-based paradigm [10]. We employ Yao's garbled circuits [41] (GC) to securely realize the functionality $\mathcal{F}_{2PC}$. The function to evaluate is represented as a Boolean circuit consisting of XOR and AND gates. A binary value computed in GC is denoted as $([b]^Y)$.

Secret Shares and Their Conversion. A binary value $x \in \{0, 1\}$ can be additively shared by two parties such that the sum of shares is equal to x . Denote two shares as $[b]_0^B$ and $[b]_1^B$. We consider mixed computation that connects Yao's garbled circuits and the additive secret shares [15] by the conversion shown in [13].

- Y2B $([b]^Y \rightarrow [b]^B)$: Yao's secret to additive shares (local operation [13]).
- B2Y $([b]^B \rightarrow [b]^Y)$: Additive shares to Yao's secret.

Oblivious Stack. An oblivious stack data structure can be realized by the garbled circuits protocol. It allows conditional push and pop that take a secret Boolean value dictating whether the operation should be performed or disguised by a dummy execution [39]. We rely on the following operations.

- $\text{ObStack} \leftarrow \text{stack}()$: initialize a stack and a tag $\perp$;
- $(\cdot) \leftarrow \text{ObStack.CondPush}([b], [x])$: push the element x to the oblivious stack if $b = 1$, else skip.
- $[x] \leftarrow \text{ObStack.CondPop}([b])$: pop and return the top element x if $b = 1$, otherwise return $\perp$.

Algorithm 2: TNFA simulation via two linear scans

```

1 Function 2ScanTNFAEval( $x, N$ ):
2    $s \leftarrow (1, 0, \dots, 0) \in \{0, 1\}^{n+1}$ 
3   EpsilonTransitions( $G(V, E_\epsilon), s$ )
4   for  $i = 0$  to  $(m - 1)$  do
5     CharacterTransitions( $x_i, \text{re}, s$ )
6     EpsilonTransitions( $G(V, E_\epsilon), s$ )
7   return  $s_n$ 
8 Function CharacterTransitions( $x_i, \text{re}, s$ ):
9   for  $j = n$  to  $1$  do
10     $s_j \leftarrow (s_{j-1} \wedge (\text{re}_j \stackrel{?}{=} x_i)) \vee (\text{re}_j \stackrel{?}{=} '?')$ 
11     $s_0 \leftarrow 0$ 
12 Function EpsilonTransitions( $G(V, E_\epsilon), s$ ):
13   for  $t = 0$  to  $1$  do
14     for  $j = 0$  to  $n$  do
15        $s_j \leftarrow s_j \vee (\vee_{(\alpha, j) \in E_\epsilon} s_\alpha)$ 

```

Definition 2. A pair of edges $(u_0, u_1), (v_0, v_1) \in E_\epsilon$ are called **nested edges**, if $u_0 < v_0 < v_1 < u_1$ or $v_0 < u_0 < u_1 < v_1$, otherwise they are called a pair of **cross edges**.

Lemma 1. For $u, v \in V$, if v is reachable from u in $G(V, E_\epsilon)$, then there exists a path from u to v that contains a maximum of one backward edge.

Theorem 1. (Correctness of Algorithm 2) For any $x \in \Sigma^*$ and TNFA N , the Algorithm 2 outputs $N(x)$.

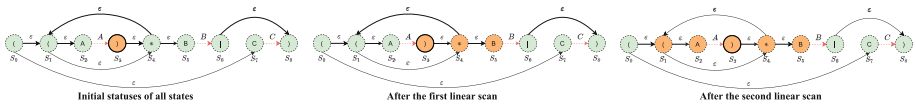


Fig. 3. This figure demonstrates the activation of reachable states from a currently active state S_3 . During the first linear scan, states S_1 and S_2 are not activated as they are reachable from state S_3 through a backward edge between S_4 and S_1 . At the end of the first linear scan, state S_4 is activated and causes states S_1 and S_2 to become activated in the second linear scan.

Optimization for Wildcard. The wildcard metacharacter “?” can be used to represent any single character in a string, making it a powerful tool for pattern match. For example, the wildcard pattern “a?b” would match any string that starts with “a” and ends with “b” with one letter in between, such as “acb” and “alb”. Classical TNFA simulation can perform pattern match with wildcards using union operations for a finite alphabet, but this can be inefficient when

the alphabet is large. Algorithm 2 avoids this issue by treating the wildcard metacharacter as a character that matches any character in character transition (See Line 10 in Algorithm 2) so that the cost of wildcard match is independent of the size of the alphabet.

Complexity. Each invocation of the epsilon transition requires scanning nodes sequentially and updating their active bit twice. This depends on whether any of its predecessors are active and each node has at maximum 2 predecessors (Lemma 2). Thus, the epsilon transition requires $O(n)$ time, which makes the TNFA evaluation run in $O(nm)$ time in total since it invokes the epsilon transition procedure m times.

4 Secure Regular Expression Matching

In this section, we introduce our protocols for secure two-party regular expression matching (secure-regex). Its functionality is shown in Fig. 4. Based on the TNFA simulation detailed in Algorithm 2, secure-regex consists of two components: character transition and epsilon transition. The character transition involves comparing each character of a length- n regular expression to a single character of the input string and updating the states based on the comparison results. This process is represented by n comparison circuits of size $\log |\Sigma| - 1$ and $2n$ logical AND gates for state updates. These character transitions can be easily expressed as a Boolean circuit and securely evaluated through 2PC. The communication complexity for privately performing all character transitions in a pattern matching task is $O(\kappa mn \log |\Sigma|)$ when 2PC is implemented through the Garbled Circuit (GC) method.

Functionality $\mathcal{F}_{\text{RegexMatch}}$

Receive a regex and its TNFA ($G = (V, E_\epsilon), \text{re}$) from the pattern holder and a string x from the string holder, return 1 to two parties if x can be matched by the regex, and 0 otherwise.

Fig. 4. The functionality of regex pattern matching.

Implementing epsilon transitions in Algorithm 2 directly through GC is difficult due to the need to keep the interconnection of the graph $G(V, E_\epsilon)$ secret from the string holder. The state of a node in the graph is propagated from its predecessors, and the 2PC protocol must perform an oblivious retrieval of a state from a list of secretly shared states. This type of oblivious table lookup operation is often challenging. While oblivious RAM could solve the problem generically, it does not provide concrete efficiency [16, 33]. We aim to handle the problem with more lightweight solutions.

The two methods for efficient handling of the epsilon transition are presented in the subsequent sections: one based on the oblivious stack (OS) and another based on the oblivious transfer (OT). The OT-based approach incurs lower communication overhead for short input regular expressions, but its cost increases faster than OS-based protocol, and thus is less efficient for long regex. Also, the OS-based approach incurs only a constant number of round-trip communications and has a significant advantage in the high-latency network. We defer our detailed performance evaluation and comparison to Sect. 5.

4.1 Epsilon Transition via Oblivious Stack

This section presents a constant-round two-party computation protocol that employs an oblivious stack for epsilon transitions. The protocol requires $O(n)$ stack operations with an average communication complexity of $O(\log n)$ per operation. First, we introduce the following observations over the TNFA graph.

Definition 3. An edge $(u_0, u_1) \in E_\epsilon$ is termed a **long edge** if $|u_1 - u_0| > 1$. In addition, (u_0, u_1) is called u_0 's outgoing edge or u_1 's incoming edge.

Lemma 2. Every node in the TNFA graph G has at most one incoming long-forward edge and at most one outgoing long-forward edge in E_ϵ .

Lemma 3. Every pair of backward edges is nested.

Lemma 4. A pair of long-forward edges (u_0, u_1) and (v_0, v_1) with $v_0 < u_0$ are cross edges if and only if $v_1 = u_0 + 1$ and $re[u_0] = ' | '$.

Lemma 3 and Lemma 4 illustrate that all backward edges are pairwise nested, and forward edges are almost nested with the exception described in Lemma 4. We will show the proof of the above lemmas in the full version.

Graph Representation. By Lemma 2, the number of incoming and outgoing edges for each node in $G = (V, E_\epsilon)$ is limited to 2. In addition, each node has at most one incoming and outgoing long edge. Therefore, G can be represented by six binary vectors `hasSFIE`, `hasLFIE`, `hasBIE`, `hasLFOE`, `hasBOE`, `isOR`, each of dimension $n + 1$. The i -th bit of them represents whether the i -th node has short forward incoming, long forward incoming, backward incoming, long forward outgoing, backward outgoing edges and whether the i -th character in regex is `'|'`, respectively.

With the oblivious graph representation of the TNFA, we present the formal outline of our oblivious stack-based circuit in Algorithm 5. It consists of two sub-protocols: the *forward scan* (Algorithm 3) and the *backward scan* (Algorithm 4).

In the backward scan subprotocol, nodes are processed from n to 0 and activated (by setting their s entry to `true`) if they have an incoming backward edge from an active node. When reaching an outgoing backward edge from state j , s_j is pushed into the stack. When the current state s_j has an incoming backward edge, a state bit is popped from the stack and its status is propagated to s_j .

Algorithm 3: ForwardScan

```

1 Input:  $G = (V, E_\varepsilon)$  presented by hasSFIE, hasLFIE, hasLFOE, isOR and states
    $\{s_0 \dots, s_n\}$ 
2 ostack  $\leftarrow$  stack()
3 cross  $\leftarrow$  false, tmp  $\leftarrow$  false
4 for  $j = 1$  to  $n$  do
5   popElmt  $\leftarrow$  ostack.pop( $(\text{hasLFIE}[j] \wedge \neg \text{cross}) \vee \text{isOR}[j]$ )
6    $s_\alpha \leftarrow \text{Ite}(\text{cross}, \text{tmp}, \text{popElmt})$  // If-then-else
7    $s_\alpha \leftarrow \text{Ite}(\text{hasLFIE}[j], s_\alpha, \text{false})$ 
8    $s'_\alpha \leftarrow \text{Ite}(\text{hasSFIE}[j], s_{j-1}, \text{false})$ 
9    $s_j \leftarrow s_j \vee s_\alpha \vee s_{\alpha'}$ 
10  ostack.push( $\text{hasLFOE}[j], s_j$ )
11  cross  $\leftarrow$  isOR[j]
12  tmp  $\leftarrow \text{Ite}(\text{cross}, \text{popElmt}, \text{false})$ 

```

Due to the nested property and stack data structure, the backward scan ensures correctness, meaning any node reachable from an active node via a backward edge is activated.

Algorithm 4: BackwardScan

```

1 Input:  $G = (V, E_\varepsilon)$  presented by hasBIE, hasBOE and states  $\{s_0 \dots, s_n\}$ 
2 ostack  $\leftarrow$  stack()
3 for  $j = n$  to  $0$  do
4    $\alpha \leftarrow$  ostack.pop( $\text{hasBIE}[j]$ )
5    $s_j \leftarrow s_j \vee \alpha$ 
6   ostack.push( $\text{hasBOE}[j], s_j$ )

```

The objective of the forward scan is to activate a node if a path to it from an active node consists only of forward edges. Unlike backward edges, forward edges can cross. However, according to Lemma 4, these crossings can only occur between adjacent nodes and can be identified by examining if the character is ' $\prime$ ' in re .

To handle these special crossings, the stack-based approach used for backward edges can be adjusted. When the stack contains two nodes with crossed forward edges, such as s_0, s_6 in the processing of S_6 , the top two elements can be swapped to s_6, s_0 . This allows s_0 to be popped from the stack and used for updating the next state with a long forward incoming edge (LFIE), S_7 in this case. However, swapping the elements causes three operations on the stack. To reduce the number of operations, the variable **tmp** is introduced to store the top bit of the stack instead of swapping. After processing S_6 , the stack will have s_6 and **tmp** = s_0 , which will be used immediately for the next state with an LFIE. This approach is valid due to the structure of the TNFA presented in Lemma 4.

Algorithm 5: Epsilon Transition via Oblivious Stack

-
- 1 **Input:** $N = (G = (V, E_\varepsilon), \text{re})$ and states $\{s_0, \dots, s_n\}$ that are shared by two parties.
 - 2 ForwardScan($G = (V, E_\varepsilon), \{s_0, \dots, s_n\}$)
 - 3 BackwardScan($G = (V, E_\varepsilon), \{s_0, \dots, s_n\}$)
 - 4 ForwardScan($G = (V, E_\varepsilon), \{s_0, \dots, s_n\}$)
- Output:** Updated shared states' statuses $\{s_0, \dots, s_n\}$
-

Security. All components in Algorithm 5 are realized in $\mathcal{F}_{2\text{PC}}$. Thus the security follows the instantiation of the underlying generic 2PC. We discuss more about the realization of the protocol components in Sect. 5. We provide the following theorem and defer the proof to Appendix A.2.

Theorem 2. *Assume the epsilon transition in Fig. 2 is defined by the algorithm shown in Fig. 5. The evaluation of Algorithm 2 represented by a Boolean circuit in $\mathcal{F}_{2\text{PC}}$ securely realizes Functionality 4 against semi-honest adversaries.*

Complexity. An epsilon transition requires a total of $6n + 2$ stack accesses. The efficient construction of oblivious stacks [43] for a stack of size $O(n)$ incurs $O(\log n)$ amortized cost for each oblivious access. Thus the net circuit size for oblivious stack is $O(n \log n)$. Additionally, both forward and backward scans take $O(n)$ AND gates for non-stack operations. Hence, the epsilon transition has $O(n \log n)$ circuit complexity (in the number of AND gates). If the epsilon transition is realized by this stack-based protocol and the functionality $\mathcal{F}_{2\text{PC}}$ is instantiated by the garbled circuits protocol, the communication complexity of the regular expression matching task (shown in Algorithm 2) is $O(\kappa mn(\log n + \log |\Sigma|))$.

4.2 Epsilon Transition via 1-Out-n+1 OT

The predecessors of each state in a TNFA are solely determined by the input regular expression. As a result, the pattern holder knows the exact states (i.e., the value of α at line 15 in Algorithm 2) that need to be retrieved when updating a state during epsilon transitions. Based on this, we propose an alternative OT-based approach for secure epsilon transition (Fig. 5) that has a total communication overhead of $O(mn(n + \kappa \log |\Sigma|))$. Its asymptotic communication complexity is worse than the OS-based approach, but with a smaller hidden constant. Hence it is more practical when matching with short regular expressions or in low-latency networks.

The protocol utilizes the functionalities $\mathcal{F}_{2\text{PC}}$, $\mathcal{F}_{\text{OT}}$, and the conversion functions Y2B and B2Y discussed in Sect. 2.2. It is based on the bound of the number of predecessors for each state established in Lemma 2, which states that a state can have at most 2 predecessors. The protocol starts by having the two parties additively share the states $(s_0, \dots, s_n) \in \{0, 1\}^{n+1}$. The goal is to update a state

s_j based on its two predecessors s_α and $s_{\alpha'}$ if they exist. To do this, the parties use 1-out-of- $n+1$ OT twice to retrieve the secret shares of α -th and α' -th state (steps 1–6). Then they employ $\mathcal{F}_{2PC}$ to update the secret shared state s_j (line 7). Y2B and B2Y are utilized to convert between the forms of the secret shares when transforming between $\mathcal{F}_{OT}$ and $\mathcal{F}_{2PC}$ (steps 7–8).

Protocol $\Pi_{\epsilon trans}$

Input: String holder P_s inputs shares of states $\{[s_0]_0^B, \dots, [s_n]_0^B\}$. Pattern holder P_p inputs $G(V, E_\epsilon)$, and her shares of states $\{[s_0]_1^B, \dots, [s_n]_1^B\}$

For $t = 0$ to 1; $j = 0$ to n :

1. P_s samples a random bit r_j , and sends $(ot, n+1, 1, \{[s_i]_0^B \oplus r_j\}_{i \in [0, n]})$ to $\mathcal{F}_{OT}$.
2. Denote α as the index of j -th state's predecessor. P_p sends $(ot, n+1, 1, \alpha)$ to $\mathcal{F}_{OT}$.
3. $\mathcal{F}_{OT}$ returns $m = [s_\alpha]_0^B \oplus r_j$ to P_p .
4. P_p sets $t_j \leftarrow m \oplus [s_\alpha]_1^B$.
5. For another predecessor of j -th state α' , two parties repeat the above to get a new r'_j and t'_j respectively.
6. Invoke $\mathcal{F}_{2PC}$ with B2Y to compute $[s_j]_0^Y, [s_j]_1^Y \leftarrow ([s_j]_0^B \oplus [s_j]_1^B) \vee (r_j \oplus t_j) \vee (r'_j \oplus t'_j)$.
7. Translate Yao's share to additive share as $[s_j]_0^B, [s_j]_1^B \leftarrow \text{Y2B}([s_j]_0^Y, [s_j]_1^Y)$.

Output: P_s (resp. P_p) holds shares of updated states $\{[s_0]_0^B, \dots, [s_n]_0^B\}$ (resp. $\{[s_0]_1^B, \dots, [s_n]_1^B\}$).

Fig. 5. Epsilon Transition via 1-out-of- $n+1$ OTs

The security of the protocol in Protocol 5 is stated below and its formal proof is provided in Appendix A.1.

Theorem 3. *Assume that in Algorithm 2, the epsilon transition is realized by Protocol 5 and other components are naively evaluated by $\mathcal{F}_{2PC}$. The combination of them securely realizes Functionality 4 against semi-honest adversaries in the $(\mathcal{F}_{2PC}, \mathcal{F}_{OT})$ -hybrid mode.*

Complexity. Algorithm 5 makes $4(n+1)$ invocations of 1-out-of- $n+1$ OT. The corresponding garbled circuit(GC) contains $4(n+1)$ inputs from both the garbler and evaluator and $4(n+1)$ AND gates. The $4(n+1)$ invocations of 1-out-of- $n+1$ OT only require the string holder to send $4(n+1)^2$ bits to the pattern holder during the online phase. We will detail how we achieve $(n+1)$ -bit per 1-out-of- $n+1$ OT in the online phase with a sublinear preprocessing in the full version. Assume that the string holder is the garbler and the pattern holder is the evaluator of GC. Among $2(n+1)$ rounds, the garbler sends 4κ bits and the evaluator sends 2 bits per round. This epsilon transition protocol incurs a total of $4(n+1)^2 + 8(n+1)\kappa + 4(n+1)$ bits communication overhead and $2(n+1)$ round trips. When applying this OT-based epsilon transition to

the TNFA simulation described in Algorithm 2, it costs $O(mn(n + \kappa \log |\Sigma|))$ communication and $O(mn)$ round-trips.

5 Performance Evaluation

In this section, we provide the instantiation of our cryptographic building blocks and performance evaluation of both OS and OT-based protocols.

Instantiate Cryptographic Building Blocks. Both OS and OT-based protocols require the invocation of OT, and we employ OT based on PCG-based constructions [7, 8, 40] in all scenarios. The 1-out-of-N OT required by OT-based protocol is converted from 1-out-of-N random OT [19, 26]. We use the GGM PPRF construction to generate 1-out-of-N random OT from $\log_2 N$ 1-out-of-2 OTs [26]. We will show the details of construction, optimizations and security proofs of OT construction in the full version. The garbled circuits that realize $\mathcal{F}_{2PC}$ is instantiated by the half-gates protocol [44], with optimizations including free-XOR [21], point-and-permute [3] and garbled row reduction [27]. Furthermore, We use the oblivious stack protocol proposed in [43]. Each conditional push or conditional pop operation incurs $O(\kappa \log n)$ amortized communication complexity when instantiated in garbled circuits.

5.1 Performance Evaluation of Secure-Regex

We implement our privacy-preserving pattern match protocols and demonstrate their performance through a series of experiments. Our use of garbled circuits protocol is built on top of the emp-toolkit [38]. We use the oblivious stack proposed in [43] and open sourced from [24]. We use two Amazon EC2 m5.xlarge instances located in the same region to act as the string holder and the pattern holder. The instances are equipped with 4 vCPUs with clock speed up to 3.1 GHz, 16 GiB RAM and up to 10 Gbps network bandwidth. We use only a single thread. Meanwhile, we use the Linux traffic control tool `tc` to control the network bandwidth and create latency to simulate the wide area network.

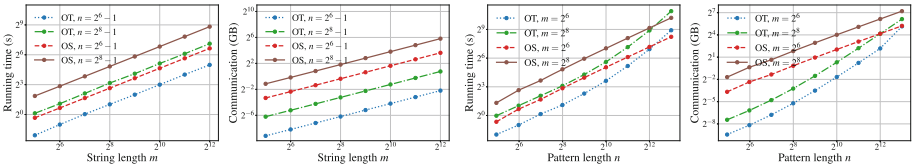


Fig. 6. The running time (in seconds) and communication overhead (in gigabytes) of our OT-based and OS-based protocol with subject to the change of string length (m) and pattern length (n). The network bandwidth is fixed to 1 Gbps.

Benchmark the Protocols. We first fix the network bandwidth to 1 Gbps and show the performance with regard to variables n (pattern length) and m (string length). We report the running time and communication overhead for each execution for both approaches (OT-based or OS-based). Recall that asymptotically, the communication complexity of the OT-based approach is $O(mn(n + \kappa \log |\Sigma|))$ and the OS-based approach is $O(\kappa mn(\log n + \log |\Sigma|))$. Figure 6 (first and second) demonstrates how the protocols scale with the increasing of the input string length m . We choose the parameters ($n = 2^i - 1, m = 2^j$) where $i \in \{6, 8\}$ and $j \in [5, 12]$. Both the running time and the communication overhead for the two protocols are linear to m , which aligns with our analysis. Figure 6 (third and fourth), shows the performance with the increased pattern length n . We choose the parameters ($n = 2^i - 1, m = 2^j$) where $i \in [5, 13]$ and $j \in \{6, 8\}$. For short patterns, the communication overhead of the OT-based approach is dominated by the character transition and thus is still linear to n . As n increases, the cost of epsilon transition dominates thus the overhead becomes quadratic to n . The performance of the OS-based approach complies with our analysis. Overall the OT-based approach is more efficient for short patterns, but is outperformed by OS-based when $\log n > 12$.

Table 2. The running time (seconds) of the protocol with subject to the bandwidth. The pattern length $n = 255$ and the string length $m = 256$.

Scheme	Bandwidth(Mbps)					Latency(ms)				
	50	100	500	1K	5K	LAN	2	20	40	60
OT	18.2	9.1	8.9	8.8	8.4	0.5	17.2	167.3	333.75	500.2
OS	568.4	284.2	56.8	28.4	11.8	1.5	1.6	1.7	2.8	14.2

Next, we fix the pattern length $n = 255$ and string length $m = 256$. To demonstrate how our protocols react to the change in the network bandwidth, we report the running time under different network settings. In Table 2 (left), we report the running time of two protocols while increasing the network bandwidth. The performance of the OT-based protocol does not change when the bandwidth is higher than 100 Mbps due to its low communication overhead. The running time of the OS-based protocol decreases as the bandwidth is raised from 50 Mbps to 5000 Mbps, which shows that communication is its bottleneck.

The influence of the network latency on our protocol is demonstrated in Table 2 (right). We fix the pattern length $n = 63$ and string length $m = 64$. We choose the maximum latency to be 60 ms, which is the round-trip delay between the US west and east coast. This is a proper simulation of real-world WAN. The OT-based protocol has a number of round-trips linear to mn , thus it deteriorates quickly when the latency increases. The OS-based protocol is less affected by the latency due to its constant round of communication and performs better than the OT-based approach in all delayed network settings.

Overall, the OT-based approach shows better performance in local networks and favors short patterns, while the OS-based protocol is more efficient in delayed networks, and favors long patterns. Although the OT-based protocol suffers from $O(mn)$ round-trips in WAN setting, it is suitable for batch matching where the same regex is applied to match s number of strings of the same length m . The communication rounds will be irrelevant to s since each round of message across s parallel executions can be sent in the pipeline.

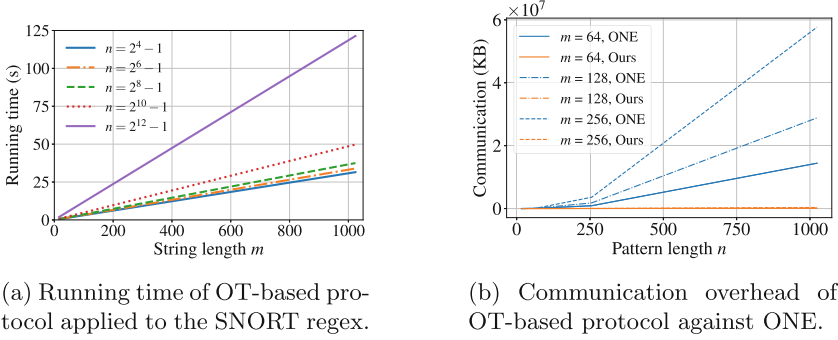


Fig. 7. (a) Running time of OT-based protocol applied to the SNORT regex. (b) Communication overhead of OT-based protocol against ONE.

Benchmarks Against SNORT. We demonstrate the use of secure-regex in resolving privacy concerns in real-world intrusion detection. A pattern holder holds rule-based regex and can identify potential threats from incoming network packets, while a string holder holds confidential network packets from clients. To protect both the rules and packet contents, two parties use secure-regex protocols to perform secure pattern matching.

We use the open-source SNORT PCRE (Perl Compatible Regular Expressions) to showcase the use of regex in intrusion detection [11]. SNORT PCRE is a sophisticated form of regex with advanced features like backreferences and recursive patterns. Our focus is on classical regex, which typically supports only disjunction, concatenation, and loop operations. We rewrite the SNORT PCRE into our format described in Sect. 2.1. This results in 416 SNORT PCRE that are suitable for use in our secure-regex protocol, with 356 (85.5%) of them having a length of $\leq 2^{12}$ bytes. We benchmark our secure-regex protocol against regex of SNORT, ignoring the regex that has length $> 2^{12}$. We show in Fig. 7a the running time of secure-regex (OT-based approach) with partial SNORT PCRE against input string of varied length. Note that our secure-regex implementation requires the length of regex padded to almost a power of 2. For clarity we only benchmark for $n = 2^j - 1, j \in \{4, 6, 8, 10, 12\}$. The bandwidth is set to 1 Gbps.

Table 3. The running time (seconds) of the protocol with all three DNA matching experiments. The bandwidth is fixed to 100 Mbps. We refer to [1] (Fig. 18) for the complete regex evaluated in the experiment.

Regex index	r18	r19	r20
OT	5.3	147.2	806.2
OS	17.7	387.7	1560.2

Real-World Application: DNA Matching We now present benchmark results of secure-regex in privacy-preserving DNA matching. In this setting, a user wishes to check if their DNA data contains a certain sequence (expressed as regex) that might indicate the presence of some particular mutations. The user wants to keep their sensitive DNA data private, while the party that holds the DNA sequence might also want to keep it private for proprietary reasons. Secure-regex allows DNA matching without revealing either the DNA data or that particular sequence.

For our experiment, we use the same set of regex and DNA data provided by [1] (detailed in Fig. 14 and Fig. 18). We report the running time of both our OS-based and OT-based secure-regex protocol. To mimic a realistic online setting, we fixed our bandwidth to 100 Mbps. Table 3 shows the experiment result of all three regexes.

Comparison with ONE [30]. ONE is a state-of-the-art solution for secure regex pattern matching. It proposes an oblivious TNFA evaluation algorithm and implements it using homomorphic encryption [28]. Although the implementation of ONE is not presently accessible, we compare its communication overhead with our OT-based protocol. The data related to ONE relies on the performance metrics reported in [30]. As shown in Fig. 7b, the OT-based secure-regex achieves $4.5\times$ to $164.5\times$ improvement in communication compared to ONE.

6 Conclusion

In this paper, we present two protocols for two-party private regular expression pattern matching. Our OT-based protocol shows high efficiency in the LAN setting and for short regexes, while the OS-based protocol achieves the best asymptotic complexity and performs well in the WAN setting and for long regexes. These techniques can be potentially applied to network intrusion detection and DNA sequence matching to protect the privacy of input strings and regexes.

A Secure-Regex: Security Proofs

A.1 Security Proof of OT-Based Secure-Regex

Theorem 3. Assume that in Algorithm 2, the epsilon transition is realized by Protocol 5 and other components are naively evaluated by $\mathcal{F}_{2PC}$. The combina-

tion of them securely realizes Functionality 4 against semi-honest adversaries in the $(\mathcal{F}_{2PC}, \mathcal{F}_{OT})$ -hybrid mode.

Proof: We follow the simulation-based paradigm to prove the Theorem 3. We first consider a corrupted string holder $\mathcal{A}$, who acts as the sender in $\mathcal{F}_{OT}$ and a garbler in $\mathcal{F}_{2PC}$. A simulator $\mathcal{S}$ emulates the functionality $\mathcal{F}_{OT}$ and $\mathcal{F}_{2PC}$. Initially, $\mathcal{S}$ is given the input and output of $\mathcal{A}$. When emulating OT, it receives and stores the OT input messages from $\mathcal{A}$. When emulating $\mathcal{F}_{2PC}$ in the epsilon transition, it receives the inputs from $\mathcal{A}$, samples a random bit $b \in \{0, 1\}$ and returns it to $\mathcal{A}$. When emulating $\mathcal{F}_{2PC}$ in the character transition, it receives the inputs from $\mathcal{A}$. The view of corrupted string holder is perfectly simulated. Firstly, the security relies on secure instantiation of $\mathcal{F}_{OT}$ and $\mathcal{F}_{2PC}$. Second, $\mathcal{A}$ does not receive any output from OT. During the $Y2B$ conversion, the output is uniformly distributed from the view of $\mathcal{A}$. This is guaranteed by the point-and-permute in Yao's garbling scheme.

We consider a corrupted pattern holder $\mathcal{A}$. A simulator $\mathcal{S}$ emulates the functionality $\mathcal{F}_{OT}$ and $\mathcal{F}_{2PC}$. Initially, $\mathcal{S}$ is given the input and output of $\mathcal{A}$. When emulating OT, it receives and stores the choice indices from $\mathcal{A}$ and returns a random bit c as the output for OT receiver. When emulating $\mathcal{F}_{2PC}$ in the epsilon transition, it receives the inputs from $\mathcal{A}$, samples a random bit $d \in \{0, 1\}$ and returns it to $\mathcal{A}$. When emulating $\mathcal{F}_{2PC}$ in the character transition, it receives the inputs from $\mathcal{A}$. The view of the corrupted corrupted pattern holder is perfectly simulated. Firstly, the security relies on secure instantiation of $\mathcal{F}_{OT}$ and $\mathcal{F}_{2PC}$. Second, the output that a pattern holder receives from OT is masked by random bits, thus uniformly distributed. Thus their distribution is the same as the simulated random bits. The output of $Y2B$ conversions also has the same distribution in the real and the ideal world for the above reason.

A.2 Security Proof of OS-Based Secure-Regex

Theorem 2. Assume the epsilon transition in Fig. 2 is defined by the algorithm shown in Fig. 5. The evaluation of Algorithm 2 represented by a Boolean circuit in $\mathcal{F}_{2PC}$ securely realizes Functionality 4 against semi-honest adversaries.

Proof: In the OS-based protocol, all components are executed in a garbled circuits protocol so the security fully relies on the underlying half-gates construction instantiating $\mathcal{F}_{2PC}$. Define the computation of the Algorithm 2 to be a binary circuit $\mathcal{C}$ which consists of character transitions and epsilon transition (described in Algorithm 5). Define a PPT adversary $\mathcal{A}$ and construct a simulator $\mathcal{S}$ that simulates views for $\mathcal{A}$ when $\mathcal{A}$ corrupts one of the parties. In both cases, $\mathcal{S}$ receives the input and output of $\mathcal{A}$, and emulates $\mathcal{F}_{2PC}$ to evaluate the circuit $\mathcal{C}$. The view of $\mathcal{A}$ is perfectly simulated and the security is reduced to $\mathcal{F}_{2PC}$.

References

1. Angel, S., Ioannidis, E., Margolin, E., Setty, S., Woods, J.: Reef: fast succinct non-interactive zero-knowledge regex proofs. *Cryptology ePrint Archive* (2023)
2. Baron, J., El Defrawy, K., Minkovich, K., Ostrovsky, R., Tressler, E.: 5pm: secure pattern matching. *J. Comput. Secur.* **21**(5), 601–625 (2013)
3. Beaver, D., Micali, S., Rogaway, P.: The round complexity of secure protocols. In: *Proceedings of the Twenty-Second Annual ACM Symposium on Theory of Computing*, pp. 503–513 (1990)
4. Beebe, N.: Digital forensic research: the good, the bad and the unaddressed. In: Peterson, G., Sheno, S. (eds.) *DigitalForensics 2009*. *IAICT*, vol. 306, pp. 17–36. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-642-04155-6_2
5. Benson, G.: Tandem repeats finder: a program to analyze DNA sequences. *Nucleic Acids Res.* **27**(2), 573–580 (1999)
6. Blanton, M., Steele, A., Alisagari, M.: Data-oblivious graph algorithms for secure computation and outsourcing. In: *Proceedings of the 8th ACM SIGSAC Symposium on Information, Computer and Communications Security*, pp. 207–218 (2013)
7. Boyle, E., et al.: Efficient two-round ot extension and silent non-interactive secure computation. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pp. 291–308 (2019)
8. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Scholl, P.: Efficient pseudo-random correlation generators: silent OT extension and more. In: Boldyreva, A., Micciancio, D. (eds.) *CRYPTO 2019*. *LNCS*, vol. 11694, pp. 489–518. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-26954-8_16
9. Bruno, N., Koudas, N., Srivastava, D.: Holistic twig joins: optimal xml pattern matching. In: *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data*, pp. 310–321 (2002)
10. Canetti, R.: Universally composable security. *J. ACM* **67**(5) (2020). <https://doi.org/10.1145/3402457>
11. CISCO: Snort intrusion prevention system (2014). <https://www.snort.org>
12. Darivandpour, J., Atallah, M.J.: Efficient and secure pattern matching with wild-cards using lightweight cryptography. *Comput. Secur.* **77**, 666–674 (2018)
13. Demmler, D., Schneider, T., Zohner, M.: Aby-a framework for efficient mixed-protocol secure two-party computation. In: *NDSS* (2015)
14. Frikken, K.B.: Practical private DNA string searching and matching through efficient oblivious automata evaluation. In: Gudes, E., Vaidya, J. (eds.) *DBSec 2009*. *LNCS*, vol. 5645, pp. 81–94. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-642-03007-9_6
15. Goldreich, O., Micali, S., Wigderson, A.: How to play any mental game, or a completeness theorem for protocols with honest majority. In: *Providing Sound Foundations for Cryptography: On the Work of Shafi Goldwasser and Silvio Micali*, pp. 307–328 (2019)
16. Goldreich, O., Ostrovsky, R.: Software protection and simulation on oblivious rams. *J. ACM (JACM)* **43**(3), 431–473 (1996)
17. Hazay, C., Lindell, Y.: Efficient protocols for set intersection and pattern matching with security against malicious and covert adversaries. In: Canetti, R. (ed.) *TCC 2008*. *LNCS*, vol. 4948, pp. 155–175. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-78524-8_10
18. Kerschbaum, F.: Practical private regular expression matching. In: *Security and Privacy in Dynamic Environments: Proceedings of the IFIP TC-11 21st International Information Security Conference*, pp. 461–470 (2006)

19. Kolesnikov, V., Kumaresan, R., Rosulek, M., Trieu, N.: Efficient batched oblivious prf with applications to private set intersection. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp. 818–829 (2016)
20. Kolesnikov, V., Rosulek, M., Trieu, N.: SWiM: secure wildcard pattern matching from OT extension. In: Meiklejohn, S., Sako, K. (eds.) FC 2018. LNCS, vol. 10957, pp. 222–240. Springer, Heidelberg (2018). https://doi.org/10.1007/978-3-662-58387-6_12
21. Kolesnikov, V., Schneider, T.: Improved garbled circuit: free XOR gates and applications. In: Aceto, L., Damgård, I., Goldberg, L.A., Halldórsson, M.M., Ingólfssdóttir, A., Walukiewicz, I. (eds.) ICALP 2008. LNCS, vol. 5126, pp. 486–498. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-70583-3_40
22. Kukich, K.: Techniques for automatically correcting words in text. *ACM Comput. Surv.* (CSUR) **24**(4), 377–439 (1992)
23. Laud, P., Willemson, J.: Universally composable privacy preserving finite automata execution with low online and offline complexity. *Cryptology ePrint Archive* (2013)
24. Luo, N., Judson, S., Antonopoulos, T., Piskac, R., Wang, X.: ppsat: Towards two-party private sat solving. In: 31st USENIX Security Symposium, pp. 2983–3000 (2022)
25. Mohassel, P., Niksefat, S., Sadeghian, S., Sadeghiyan, B.: An efficient protocol for oblivious DFA evaluation and applications. In: Dunkelman, O. (ed.) CT-RSA 2012. LNCS, vol. 7178, pp. 398–415. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-27954-6_25
26. Naor, M., Pinkas, B.: Oblivious transfer and polynomial evaluation. In: Proceedings of the Thirty-First Annual ACM Symposium on Theory of Computing, pp. 245–254 (1999)
27. Naor, M., Pinkas, B., Sumner, R.: Privacy preserving auctions and mechanism design. In: Proceedings of the 1st ACM Conference on Electronic Commerce, pp. 129–139 (1999)
28. Paillier, P.: Public-key cryptosystems based on composite degree residuosity classes. In: Stern, J. (ed.) EUROCRYPT 1999. LNCS, vol. 11592, pp. 223–238. Springer, Heidelberg (1999). https://doi.org/10.1007/3-540-48910-X_16
29. Samadani, M.H., Berenjkoo, M., Blanton, M.: Secure pattern matching based on bit parallelism. *Int. J. Inf. Secur.* **18**(3), 371–391 (2019)
30. Sasakawa, H., Harada, H., duVerle, D., Arimura, H., Tsuda, K., Sakuma, J.: Oblivious evaluation of non-deterministic finite automata with application to privacy-preserving virus genome detection. In: Proceedings of the 13th Workshop on Privacy in the Electronic Society, pp. 21–30 (2014)
31. Sedgewick, R., Wayne, K.: Algorithms, 4th edn. Addison-Wesley, Boston (2011)
32. Sherry, J., Lan, C., Popa, R.A., Ratnasamy, S.: Blindbox: deep packet inspection over encrypted traffic. In: Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication, pp. 213–226 (2015)
33. Stefanov, E., et al.: Path oram: an extremely simple oblivious ram protocol. *J. ACM (JACM)* **65**(4), 1–26 (2018)
34. Thompson, K.: Programming techniques: regular expression search algorithm. *Commun. ACM* **11**(6), 419–422 (1968)
35. Troncoso-Pastoriza, J.R., Katzenbeisser, S., Celik, M.: Privacy preserving error resilient DNA searching through oblivious automata. In: Proceedings of the 14th ACM Conference on Computer and Communications Security, pp. 519–528 (2007)

36. Van Lunteren, J.: High-performance pattern-matching for intrusion detection. In: Proceedings IEEE INFOCOM 2006. 25th IEEE International Conference on Computer Communications, pp. 1–13. Citeseer (2006)
37. Vergnaud, D.: Efficient and secure generalized pattern matching via fast fourier transform. In: Nitaj, A., Pointcheval, D. (eds.) AFRICACRYPT 2011. LNCS, vol. 6737, pp. 41–58. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-21969-6_3
38. Wang, X., Malozemoff, A.J., Katz, J.: EMP-toolkit: Efficient MultiParty computation toolkit (2016). <https://github.com/emp-toolkit>
39. Wang, X.S., et al.: Oblivious data structures. In: Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, pp. 215–226 (2014)
40. Yang, K., Weng, C., Lan, X., Zhang, J., Wang, X.: Ferret: fast extension for correlated ot with small communication. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, pp. 1607–1626 (2020)
41. Yao, A.C.: Protocols for secure computations. In: 23rd Annual Symposium on Foundations of Computer Science (SFCS 1982), pp. 160–164. IEEE (1982)
42. Yasuda, M., Shimoyama, T., Kogure, J., Yokoyama, K., Koshihara, T.: Secure pattern matching using somewhat homomorphic encryption. In: Proceedings of the 2013 ACM Workshop on Cloud Computing Security Workshop, pp. 65–76 (2013)
43. Zahur, S., Evans, D.: Circuit structures for improving efficiency of security and privacy tools. In: 2013 IEEE Symposium on Security and Privacy, pp. 493–507 (2013). <https://doi.org/10.1109/SP.2013.40>
44. Zahur, S., Rosulek, M., Evans, D.: Two halves make a whole: reducing data transfer in garbled circuits using half gates. In: Oswald, E., Fischlin, M. (eds.) EUROCRYPT 2015. LNCS, vol. 9057, pp. 220–250. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-46803-6_8



Nemesis: Combating Abusive Information in Encrypted Messaging with Private Reporting

Rui Lian¹, Yulong Ming¹, Chengjun Cai², Yifeng Zheng³, Cong Wang^{1(✉)},
and Xiaohua Jia¹

¹ City University of Hong Kong, Hong Kong, China
congwang@cityu.edu.hk

² City University of Hong Kong (Dongguan), Dongguan, China

³ Harbin Institute of Technology, Shenzhen, Shenzhen, China

Abstract. Abusive messages spread rapidly within popular end-to-end encrypted messaging services (EEMSs) due to the inability to conduct content moderation on plaintext messages. This issue has turned EEMSs into breeding grounds for fake news, misinformation, and other viral abusive content. However, enabling the identification of abusive information without careful consideration can compromise the cherished confidentiality feature of EEMSs. To address this issue, we propose Nemesis, a privacy-friendly, user-reporting-based content moderation system for EEMSs. In Nemesis, users can privately report abusive messages, and the system traces the source if the report count exceeds a predefined threshold. Compared to existing systems, Nemesis offers three key advantages: 1) it supports flexible and adaptable thresholds to meet different content moderation needs, 2) it prevents dishonest reporters from disrupting the system, and 3) it better ensures the privacy of all users during the moderation process. Nemesis utilizes lightweight cryptographic techniques to achieve these goals, and evaluation results demonstrate its low overhead in securely identifying abusive messages and tracing their sources in EEMSs.

Keywords: Content moderation · End-to-end encryption · Threshold aggregation

1 Introduction

There is a growing viral spread of abusive information (e.g., fake news, misinformation, and child sexual abuse material) among online systems for political purposes, self-interests, and so on, leading to serious consequences for society. In an effort to curb the dissemination of abusive information, major sites like Facebook, Twitter, and Weibo are struggling to moderate online content. They monitor online information sent through their networks and intervene when necessary, e.g., by blocking accounts that generate harmful information.

However, content moderation poses a significant challenge in widely used end-to-end encrypted messaging services (EEMSs) such as WhatsApp, Signal, Telegram, and Facebook Messenger. These systems employ end-to-end encryption to

conceal plaintext messages from the platforms, ensuring that only the intended recipients can access the plaintext content. Despite the fact that the privacy assurances provided by EEMSs, laws, and social organizations have increasingly required EEMSs to be capable of content moderation [30]. Hence, a problem naturally arises: *how to support content moderation without compromising user privacy in EEMS?*

Today's efforts to address this problem can be categorized into two main approaches: *blocklist-based* and *user-reporting-based* methods. Several research works [1, 21, 28, 31] have proposed protocols for encrypted content checks against a pre-defined blocklist (e.g., the perceptual hash values of harmful contents like child sexual abuse material [17], URLs [20]). An EEMS holding such a blocklist runs a private set intersection protocol with a user to proactively scan encrypted messages and check for matches with the blocklist. However, the pre-defined blocklist may not capture all forms of harmful content timely, allowing newly emerging harmful content to bypass the blocklist in practical deployments.

An alternative line of work (which we follow in this paper) relies on user reporting with various privacy enhancements, where the EEMS tracks down the users related to reported abusive messages. While there exist many privacy-aware user-reporting protocols for EEMSs [18, 19, 22, 24, 32, 33], they are not yet satisfactory to be adopted in realistic EEMSs due to the following shortcomings:

(1) Rigid constraint on pre-fixed threshold. Most works [18, 19, 24, 32, 33] only support a single report to trigger traceback, which can lead to burdensome moderation workloads for EEMSs in practice [25]. An improved approach is to enable threshold-triggered reporting-then-tracing, where the EEMS can set a threshold and moderate messages once their report counts reach that threshold. Abusive messages often spread rapidly and widely, making it necessary to adjust the threshold over time [14]. For example, Meta has been lowering the threshold of its moderation system in response to a significant increase in hateful content on Instagram [29]. However, the recent work FACTS [22], proposing a threshold-based oblivious report counting approach, fails to support lowering the threshold below 100 for accurate counting due to the use of probabilistic data structures. It is imperative to support flexible and adaptable threshold settings to address the diverse needs of EEMSs.

(2) Lack of considering reporter trustworthiness. Although FACTS [22] supports a threshold setting that reduces the impact of biased reporters, reporters are trusted to increment and check if the report count of the reported message reaches a threshold. But in reality, dishonest reporters might exploit the privacy-preserving reporting process to submit malformed reports to disrupt the moderation service. They may affect the report count of a received message by submitting a non-one value or faking the report of a non-received message. Thus, it is necessary to enable verification of reports.

(3) Insufficient user privacy protection. Given the objective of minimizing the dissemination of viral misinformation while prioritizing user privacy, traceback systems do not need to identify extensive information beyond the originator

Table 1. The comparison of security properties with Signal and prior works on trace-back systems.

	Confi.	Repo.	Trac.	Unli.	Hone.	Rob.
Signal [10]	★	☆	☆	☆	☆	☆
Tyagi et al. [33]	★	☆	★	★	☆	☆
Peale et al. [24]	★	☆	★	★	☆	☆
Kenney et al. [19]	★	☆	★	★	☆	☆
Hecate [18]	★	☆	★	★	☆	☆
FACTS [22]	★	☆	★	★	☆	☆
Nemesis	★	★	★	★	★	★

★: fully provided,

☆: not provided, ☆: not fully provided.

Confi.: Message confidentiality, **Repo.:** Private and accurate report tally, **Trac.:** Source traceability, **Unli.:** Unlinkability, **Hone.:** Honest users anonymity, **Rob.:** Robustness against dishonest reporters.

of the reported message. However, existing works would typically sacrifice the complete or partial privacy of intermediate users related to a reported message in the EEMS, i.e., EEMS can readily learn the links between a reported message and reporters, receivers, and/or forwarders, which may discourage users (e.g., whistleblowers) from actively using EEMSs and reporting abusive information [1]. Therefore, it is still not sound for user privacy protection in practice.

In light of the above, in this paper, we present Nemesis, a new privacy-friendly user-reporting-based content moderation system for EEMSs. Nemesis mainly departs from prior works by supporting private and accurate report tally with a flexible and adaptable threshold to meet the custom demands of EEMSs, robustness against dishonest reporters to prevent malformed reports, and secure source tracing to identify the source of a reported abusive message (a comprehensive comparison of security properties with Signal and prior works is shown in Table 1). Our evaluations show that Nemesis introduces low overheads for the private reporting feature.

At a high level, Nemesis utilizes a custom combination of lightweight cryptographic technologies to achieve its advantages. *Firstly*, to enable oblivious counting of reported messages, we employ distributed point function (DPF) [5] to hide the connection between reporters and their reported messages. Despite being workable, prior approaches [3, 12, 34] that aim to preserve user privacy during threshold aggregation remain hard to extend such design to support secure comparison against a threshold, as either significant costs for users or leakage of user inputs are incurred when adjusting the threshold. To achieve this, we then enhance the design by incorporating secret-shared shuffle [15] and distributed comparison function (DCF) [3, 6] to efficiently compare against a threshold without compromising user privacy and requiring heavy reset overheads of users when

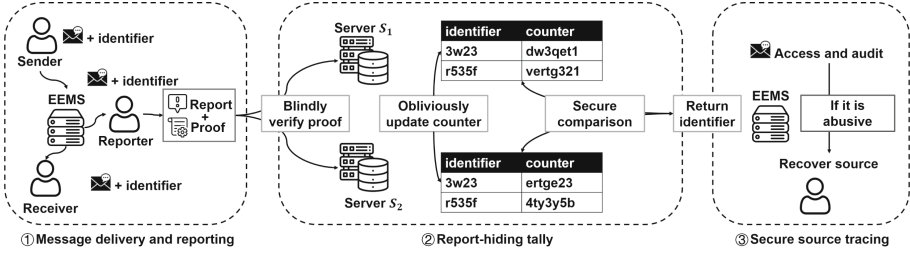


Fig. 1. The overview of Nemesis.

adjusting the threshold. *Secondly*, although existing solutions [3, 6] have proposed DPF schemes to detect non-one values, they fail to check whether a reporter has received a reported message. In contrast, we introduce a MAC-based proof construction to validate DPF-based reports by combining information-theoretic MACs with DPF schemes, ensuring reporters increment the report count for a received message by one. *Finally*, to trace the source while preserving user identities, we design a new identifier tuple that secretly links the source and its messages. This identifier tuple is passed along with the message and used as a unique index for counting and comparison. The EEMS can only access the identifier tuple and trace the source when the report count exceeds the threshold, ensuring privacy for all intermediate users.

2 Problem Formulation

2.1 Overview of Nemesis

Motivated by the surge of harmful information that is being circulated in existing EEMSs, we propose Nemesis, which provides a new third-party private aggregation service to help combat abusive messages in an EEMS in a privacy-friendly and flexible manner. Nemesis consists of the following entities: an *EEMS*, *EEMS users*, and a *private aggregation service (PAS)* (which is uniquely provided in Nemesis). As depicted in Fig. 1, users conduct private communications with other users, where each message and an identifier tuple are delivered to the intended recipients through the end-to-end encryption channel provided by the EEMS. If a user receives a message they deem abusive, they issue a report to the PAS. Utilizing the submitted reports, the PAS offers a report-hiding tally service that securely counts the number of reports on any given message. If the count exceeds a predefined threshold, the PAS provides the EEMS with flagged identifier tuples of the corresponding messages. The EEMS then traces back to identify the sources responsible for the flagged abusive messages.

Similar to existing works, while the underlying EEMS enables the delivery of encrypted and authenticated messages between point-to-point users, it reveals who communicates with whom and retains the forwarding utility, complying with WhatsApp, Signal, and academic proposals [18, 22, 24, 33].

2.2 Threat Model

Firstly, Nemesis follows a trending practice [9,23,34] to consider two non-colluding servers (denoted by $S_{\{1,2\}}$) from different trust domains (which could be taken up by independent content moderation platforms like Hive Moderation, Amazon Rekognition, Alibaba Cloud in practice), for efficiently instantiating the PAS. Furthermore, we assume they are semi-honest for retaining their reputations, i.e., each server will honestly follow the protocols but may independently attempt to infer the secrets of users (e.g., the counts or the senders/receivers of each message) due to various interests.

For users, we consider dishonest behaviors in both the send and report operations. When sending messages, a dishonest source or forwarder may want to send a message that cannot be traced back later (e.g., by forging an honest user as the message source while he/she is not). For another, a dishonest reporter may attempt to disrupt the private message counting process by sending a malformed report (e.g., adding a positive integer that is more than one or a negative integer to the report count of a message) to accelerate/delay the report counts so that the target message in the EEMS will be more likely/unlikely to be flagged by the PAS.

For the EEMS that aims to use our service to combat abusive information, we consider that they will faithfully work as requested in Nemesis so that a robust moderation service can be assured, similar to existing works in the field [10,13,27]. In addition, basic security properties for messages promised by the EEMS are assumed, e.g., original and forwarded messages are confidential against the EEMS.

We assume that there exist secure and authenticated channels (e.g., through trusted public key infrastructure (PKI)) for all parties (i.e., users, the PAS, and the EEMS) to communicate with each other safely. Side-channel and Sybil attacks are out of the scope of this paper.

2.3 Security Goals

In our system, we aim to achieve the following security goals:

- **Message confidentiality.** Before the report count of a message reaches the specified threshold, the message remains confidential and inaccessible to non-receiving users, the EEMS, and the PAS. Even after hitting the threshold, only the EEMS is able to access and audit the message.
- **Private and accurate report tally.** Private and accurate counting of reported messages under a flexible and adaptable threshold setting should be supported. The PAS never learns the specific count of any reported message and the threshold beyond the fact that the report counts of messages have surpassed the threshold.
- **Source traceability.** A user cannot transmit a message without the possibility of being tracked down the source identity and cannot accuse an innocent user as the source of a message he never sent.

- **Unlinkability.** No user is associated with a specific message until it is deemed abusive upon audit.
- **Honest users anonymity.** After a message is audited as abusive, only the source identity is learned by the EEMS, while receivers, forwarders, and reporters are not disclosed.
- **Robustness against dishonest reporters.** A reporter is unaware of the count of any message or threshold and cannot report a non-received message or report a message more/less than once.

3 Design Overview

Since messages are continuously generated and delivered, Nemesis runs in epochs like [22], and each epoch consists of three phases: *Message delivery and reporting*, *Report-hiding tally*, and *Secure source tracing*. To secure these phases, we identify the necessary algorithms as follows: **GenId**, **VrfId**, **RepoMsg**, **GenPrf**, **VrfPrf**, **CountRepo**, **TalCom**, and **TraSrc**, and build on secure computation techniques to appropriately realize these algorithms. These designed algorithms are decoupled from the end-to-end encryption algorithms used by an EEMS. Thus, each EEMS can create an instance in Nemesis and work independently.

We assume that the underlying EEMS offers end-to-end encryption send-receive scheme (**send**($m; U_s, U_r$), **receive**($m; U_s, U_r$)) for delivering m from a sender U_s to a receiver U_r and consider there are M original messages and a set $\mathcal{N}$ of reporters in an epoch, each reporter $U_i \in \mathcal{N}$ issues a report for a message and the EEMS holds a threshold t . The high-level workflow is as follows:

- *Phase 1: Message delivery and reporting.* To deliver a new/forwarded message m , a sender U_s interacts with the EEMS to produce a custom identifier tuple $\mathcal{I}$ by running **GenId**. In the case of a forward, the newly produced $\mathcal{I}$ is dropped and m is sent along with its original $\mathcal{I}$ from a prior sender by calling **send**($m, \mathcal{I}; U_s, U_r$). In the case of a new m , beyond the delivery of m and the new $\mathcal{I}$, $\mathcal{I}$ is split into two secret shares, which are sent to $S_{\{1,2\}}$. The receiver U_r runs **receive**($m, \mathcal{I}; U_s, U_r$) and calls **VrfId** to verify $\mathcal{I}$ to make sure that $\mathcal{I}$ corresponds to m and the source identity has not been modified. It is noted that the verification would not reveal the source identity to U_r in a forward state. Once receiving an abusive message, a reporter U_i then encodes its $\mathcal{I}$ as a report¹ by executing **RepoMsg**. To prevent a malformed report from adding a non-one value to the counter for a message or reporting a non-received message, the reporter is required to produce a proof for the report by running **GenPrf**. The report and proof pair are sent to $S_{\{1,2\}}$.
- *Phase 2: Report-hiding tally.* Upon receiving report and proof pairs for $\mathcal{N}$, $S_{\{1,2\}}$ first blindly verify that these reports are valid by running **VrfPrf** and then obviously updates the counter for the M messages by running **CountRepo**. Then, $S_{\{1,2\}}$ privately compare the counts with the threshold

¹ Note that Nemesis limits the number of reports per reporter per epoch to avoid a reporter overwhelming the system.

t (which can be flexibly defined by each EEMS and is easily adaptable) and return the flagged identifier tuples $\tilde{\mathcal{I}}$ of the corresponding messages (i.e., the ones that hit t) to the EEMS by calling **TalCom**.

- *Phase 3: Secure source tracing.* Given $\tilde{\mathcal{I}}$, the EEMS executes **TraSrc** that firstly accesses the corresponding message by decrypting the stored ciphertext (for auditing whether it is truly abusive information) with the key extracted from the identifier tuple $\mathcal{I} \in \tilde{\mathcal{I}}$. Then, if the message is audited as abusive, EEMS will recover the true identity of the source that originates this abusive message.

4 Constructing Nemesis

In this section, we show the cryptographic tools and design rationale we employ to realize the three phases and describe the corresponding algorithms. We start with introducing the preliminaries.

4.1 Preliminaries

Function Secret Sharing. Two-party function secret sharing (FSS) [5, 16] splits a function f into two compact function shares. Each function share reveals nothing about the function f , and only the sum of the evaluation outputs at a given point indexed by x is the corresponding result $f(x)$. More formally, a two-party FSS scheme is a pair of algorithms (FSS.Gen, FSS.Eval): 1) FSS.Gen($1^\lambda, f$) $\rightarrow (k_1, k_2)$: Given a security parameter λ and function description f , produce two FSS keys k_1, k_2 that define two function shares of f ; 2) FSS.Eval(k_b, x) $\rightarrow y_b$: Given an input x and the FSS key k_b for $b \in \{1, 2\}$, output y_b , corresponding to the party b 's share of $f(x)$.

Distributed point functions (DPF) and distributed comparison functions (DCF) are two special schemes of FSS. A DPF [3, 6] scheme is defined to secretly share and evaluate a point function $f_{\alpha, \beta} : \mathbb{Z}_{2^n} \rightarrow \mathbb{Z}_{2^p}$ where $f(\alpha) = \beta$ and $f(\cdot) = 0$, i.e., $\text{DPF.Eval}(k_1, x) + \text{DPF.Eval}(k_2, x) = \beta$ only if $x = \alpha$. DCF [4, 8] are FSS schemes for comparison function $g_{\mu, \nu} : \mathbb{Z}_{2^p} \rightarrow \mathbb{Z}_{2^p}$ where $g(j) = \nu$ if $j > \mu$ and $g(\cdot) = 0$ otherwise, i.e., $\text{DCF.Eval}(k_1, j) + \text{DCF.Eval}(k_2, j) = \nu$ only if $j > \mu$.

Information-Theoretic MAC. Information-theoretic MAC from $\text{SPD}\mathbb{Z}_{2^k}$ [11] defines as follows. For a secret value $\beta \in \mathbb{Z}_{2^n}$ and a key $mk \in \mathbb{Z}_{2^s}$, $\text{MAC}_{(mk)}(\beta) = mk \cdot \beta \in \mathbb{Z}_{2^{n+s}}$. $\text{MAC}_{(mk)}(\beta)$ is a linear function and additively homomorphic, i.e., $mk \cdot \beta = mk \cdot [\beta]_1 + mk \cdot [\beta]_2$ where $\beta = [\beta]_1 + [\beta]_2$. The probability that an attacker can forge a MAC is $1/2^s$.

Additive Secret Sharing. Let $\mathbb{Z}_{2^n}$ denote the ring of integers modulo 2^n . Given a secret value x in the ring $\mathbb{Z}_{2^n}$, an additive secret sharing (ASS) scheme [2] generates two additive shares ($[x]_1$ and $[x]_2$) that are random values such that $x = [x]_1 + [x]_2$. Each share is uniformly distributed in a ring and thus alone reveals no information about x .

4.2 Message Delivery and Reporting

During this phase, we need to consider 1) identifier design, i.e., supporting threshold-triggered reporting-then-tracing, 2) private reporting, i.e., hiding the reported message of a reporter, and 3) authentication needs for the report, i.e., preventing malformed reports from interfering with the oblivious counting process. We will start with identifier design.

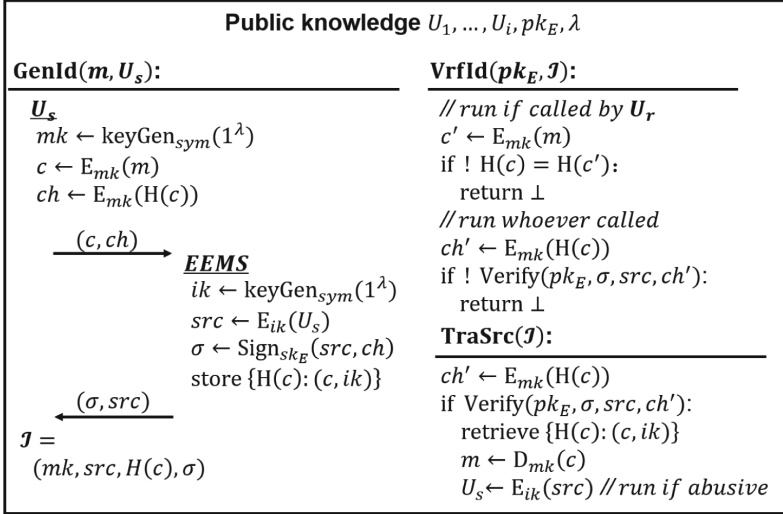


Fig. 2. Constructions of **GenId**, **VrfId**, and **TraSrc**.

Identifier Design with Enhanced Privacy. Existing traceback systems [18, 19, 22, 24] have designed custom tuples that bind a message to its source, where a reporter issues a report by sending it with the plaintext message to the EEMS for auditing and source tracing. However, integrating the constructions of these tuples into Nemesis is challenging because revealing the reported message would undermine the reporter’s privacy and contradict the goal of oblivious counting.

We take the basic structure of a tuple and rework it to function in Nemesis. In brief, we design an identifier tuple $\mathcal{I}$, which is sent along with the message among users through the end-to-end encryption channel of the EEMS like prior works [18, 19, 22, 24]. An $\mathcal{I}$ is also obtained by $S_{\{1,2\}}$ to derive a unique index for oblivious counting, which is returned from $S_{\{1,2\}}$ to the EEMS if the report count of the corresponding message hits the threshold (see details in Sect. 4.3). However, a malicious user may deliver a fake $\mathcal{I}$ that either fails to link a message to its correct source identity or corresponds to a non-existent message. Following the trust model of prior works [18, 19, 22, 24], we pass the responsibility to verify that the $\mathcal{I}$ is valid to the receiving parties. We craft two algorithms, **GenId** and

VrfId, where a sender calls **GenId** to generate an $\mathcal{I}$ and **VrfId** is used by a received party to verify the validity of $\mathcal{I}$.

Notations. Let λ denote a security parameter and $\perp$ indicate a failure. We use a symmetric encryption scheme $(k \leftarrow \text{keyGen}_{\text{sym}}(1^\lambda), c \leftarrow \text{E}_k(m), m \leftarrow \text{D}_k(c))$, a signature scheme $((pk, sk) \leftarrow \text{keyGen}_{\text{sig}}(1^\lambda), \sigma \leftarrow \text{Sign}_{sk}(m), \{0, 1\} \leftarrow \text{Verify}(pk, \sigma, m))$, and a collision-resistant hash function $\text{H} : \{0, 1\}^* \rightarrow \mathbb{Z}_{2^{n+s}}$. We consider that the EEMS has a long-term pair of keys (pk_E, sk_E) , and each user has an identification number U_i .

Now, assume a source U_s has a new plaintext message m , we show how to build $\text{GenId}(m, U_s) \rightarrow \mathcal{I}$, $\text{VrfId}(pk_E, \mathcal{I}) \rightarrow 1/0$, as shown in Fig. 2. We follow the design of prior works where the EEMS does a signature on the message and its source identity. To ensure message confidentiality, the initial step is to let U_s generate the message ciphertext c and the hash ch of the message ciphertext using a random encryption key mk via $\text{E}_{mk}(m)$ and $\text{E}_{mk}(\text{H}(c))$. Then, the EEMS does a signature σ on the encrypted source identity $src \leftarrow \text{E}_{ik}(U_s)$ generated by itself where ik is a random encryption key known only to itself and ch received from U_s , i.e., $\sigma \leftarrow \text{Sign}_{sk_E}(src, ch)$ and stores key-value pair $\{\text{H}(c) : (c, ik)\}$. Due to encryption, σ reveals nothing about U_s and m while preventing a malicious user from binding the wrong identity to m . Finally, an $\mathcal{I}$ consists of $mk, src, \text{H}(c), \sigma$. Note that prior works [22, 24] adopt a commitment scheme where the source produces the commitment cm for the message plaintext using a commitment randomness r_m and the EEMS does the signature on cm and src . A tuple includes cm, r_m, src , and the signature so that the receiver can check whether the tuple matches the received plaintext message. In contrast, in Nemesis, in addition to allowing a receiver to do verification, an identifier tuple is required to allow the EEMS to recover the plaintext message m for auditing and $S_{\{1,2\}}$ to do verification. To support the first requirement, we adopt a symmetric encryption scheme here to generate the message ciphertext c using a random encryption key mk that will be stored in the EEMS. In this context, we cannot directly use c to replace the role of cm for verification to avoid $S_{\{1,2\}}$ recovering the message plaintext once receiving an identifier tuple, violating the security guarantees of Nemesis. Thus, we additionally generate ch that binds c and mk to replace the prior cm for supporting the verification of receivers and $S_{\{1,2\}}$. To verify an $\mathcal{I}$, a receiver U_r receiving m and $\mathcal{I}$ can check σ and the hash value $\text{H}(c)$ match the provided plaintext message m , encryption key mk , and encrypted source identity src , while $S_{\{1,2\}}$ can check σ match mk, src , and $\text{H}(c)$.

DPF-Based Report Generation. We then elaborate on how to enable a reporter to generate a report, which is then used by the PAS ($S_{\{1,2\}}$) for oblivious counting of reported messages while hiding which message is being reported by the reporter. Nemesis’s starting point is to leverage lightweight two-party distributed point functions (DPF) [3, 6] where a reporter U_i encodes the reported

message's identifier tuple $\mathcal{I}^2$ as the secret index α and 1 as β for a point function $f_{\alpha,\beta}$ to generate a pair of DPF keys k_1^i, k_2^i that are considered as a report and delivered to $S_{\{1,2\}}$. Formally, given a cryptographic hash function $\tilde{H} : \{0,1\}^* \rightarrow \mathbb{Z}_{2^n}, n < k$, $\mathbf{RepoMsg}(\mathcal{I}) \rightarrow (k_1^i, k_2^i) : (k_1^i, k_2^i) \leftarrow \mathbf{DPF.Gen}(\alpha = \tilde{H}(\mathcal{I}), \beta = 1)$. Within such a report representation, $S_{\{1,2\}}$ can conclude the secret shares $[y^x]_1, [y^x]_2$ of the report count $y^x \in \mathbb{Z}_{2^n}$ of the message indexed by x by evaluating all DPF keys from $\mathcal{N}$ on the index $x \in \mathbb{Z}_{2^n}$ since the sum of evaluation on the DPF keys is 1 only at position indexed by α and is zero at any other positions.

Verifiable Report for Robustness. Up to this point, we have considered a semi-honest reporter, but to defend against a dishonest reporter who may report a non-received message by assigning a random value to α or accelerate or delay the counts for a received message reaching the threshold by setting $\beta \neq 1$ for a point function $f_{\alpha,\beta}$, $S_{\{1,2\}}$ need to check that the report is well-formed. Previous works [3, 6] have proposed techniques to detect malformed DPF keys that affect the sum of evaluation values by assigning $\beta \neq 1$. However, they do not address the issue of verifying whether the reported message is received by the reporter. To overcome this limitation, we introduce a solution that detects both dishonest behaviors. We leverage information-theoretic MAC [11] and show how to generate efficient proof for verification. Our insight is to have a reporter U_i holding a secret key $mk_i \in \mathbb{Z}_{2^s}$ associated with the reported message's identifier tuple $\mathcal{I}$ compute a MAC value τ as the proof, which is to “hash” $\beta \in \mathbb{Z}_{2^n}$ using mk_i , i.e., $\tau \leftarrow mk_i \cdot \beta \bmod \mathbb{Z}_{2^{n+s}}$. The proof and the pair of DPF keys k_1^i, k_2^i for the point function $f_{\alpha,\beta}$ are sent to $S_{\{1,2\}}$, which locally “hash” the evaluation values on k_1^i, k_2^i at position indexed by each index $x \in \mathbf{id}x$ ($\mathbf{id}x$ is the index vector held by $S_{\{1,2\}}$, and we describe the details in Sect. 4.3) using its associated private key mk_x , i.e., $mk_x \cdot \mathbf{DPF.Eval}(k_b^i, x)$ for $b \in \{1, 2\}$. Thanks to the correctness of DPF, the sum of these evaluation values equals $mk_i \cdot \beta$ if the reporter only reports a received message once. As β keeps a constant number one, directly using this method will reveal the reported message according to the unique private key mk_x associated with a unique identifier tuple. Inspired by the work [23], we sample a random key to hide the private key, where τ is to “hash” a value with a private key and then add a random key $\gamma \in \mathbb{Z}_{2^{n+s}}$. The algorithms are as follows.

- **GenPrf**($\mathcal{I}$) $\rightarrow ([\tau]_1, [\tau]_2, \gamma)$: U_i generates $\tau \leftarrow \mathcal{I}.mk_i \cdot 1 + \gamma \bmod 2^{n+s}$ and $[\tau]_1, [\tau]_2 \in \mathbb{Z}_{2^{n+s}}$ where $\tau = [\tau]_1 + [\tau]_2$, and sends $[\tau]_b, \gamma$ to S_b where $b \in \{1, 2\}$.
- **VrfPrf**($\mathbf{id}x, [\tau]_1, [\tau]_2, \gamma, k_1^i, k_2^i$) $\rightarrow 1/0$: S_b computes $[\omega]_b \leftarrow [\tau]_b - \sum mk_x \cdot \mathbf{DPF.Eval}(k_b^i, x)$ for $x \in \mathbf{id}x$, exchanges $[\omega]_1$ and $[\omega]_2$, and checks whether or not $[\omega]_1 + [\omega]_2 = \gamma$. If equals, the report is valid and outputs 1 and 0 otherwise.

² It is noted that an identifier tuple $\mathcal{I} \in \mathbb{Z}_{2^k}$ is appended to the message, delivered from senders to receivers through the EEMS. We describe how to generate it in Sect. 4.2.

4.3 Report-Hiding Tally

The above basic design for private reporting can already be used to achieve better privacy protections, but it is still far from practical in two critical aspects. In particular, we need to consider 1) optimization of oblivious counting, i.e., enhancing the performance of DPF evaluations, and 2) secure comparison against threshold, i.e., enabling the EEMS to adjust threshold with trivial overheads for the system. Now, we show how to optimize oblivious counting.

Optimized Oblivious Counts Obtainment. Instead of evaluating DPF keys on all indexes $x \in \mathbb{Z}_{2^n}$, $S_{\{1,2\}}$ is supposed to use the identifier tuples corresponding to M original messages in an epoch to derive indexes for evaluation since M is much smaller than 2^n .

The key challenge lies in enabling $S_{\{1,2\}}$ to acquire these identifier tuples from sources in a way without revealing the association between a source and a message. To tackle this issue, inspired by the work [15], our main idea is to have $S_{\{1,2\}}$ obtain shuffled identifier tuples based on two random permutations unknown to each other. Specifically, we represent the M identifier tuples as a matrix $\mathcal{I}^M$ based on their generation time. Each source sends secret shares of its identifier tuple to $S_{\{1,2\}}$ such that at the end of the phase of *Message delivery and reporting*, two secret shares of a length- M matrix $\mathcal{I}^M$ are held by $S_{\{1,2\}}$, which then run a secret-shared shuffle protocol³ for obtaining an identical shuffled matrix $\tilde{\mathcal{I}}^M$. Within that, $S_{\{1,2\}}$ derive the same length- M index vector $\mathbf{id}\mathbf{x}$ where each index $x \leftarrow \tilde{\mathcal{H}}(\mathcal{I})$, $\mathcal{I} \in \tilde{\mathcal{I}}^M$ and initiate two length- M vectors $[\mathbf{y}]_1, [\mathbf{y}]_2$ as report counters where each entry is the secret shares $[y^x]_1, [y^x]_2$ of the report count y^x of the message indexed by $x \in \mathbf{id}\mathbf{x}$. Meanwhile, $S_{\{1,2\}}$ have cached the reports from $\mathcal{N}$ and run **VrfPrf** to verify whether they are well-formed (details are provided in Sect. 4.2), then carry out **CountRepo**($\mathbf{id}\mathbf{x}, \{k_1^i, k_2^i \mid U_i \in \mathcal{N}\}) \rightarrow ([\mathbf{y}]_1, [\mathbf{y}]_2)$: For each $x \in \mathbf{id}\mathbf{x}$, $S_{\{1,2\}}$ update $[y^x]_b = [y^x]_b + \text{DPF.Eval}(k_b^i, x)$ where $b \in \{1, 2\}$ for each pair of DPF keys from $\mathcal{N}$.

Secure Comparison Against Flexible and Adaptable Threshold. So far $S_{\{1,2\}}$ have updated report counters $[\mathbf{y}]_1, [\mathbf{y}]_2$. To facilitate secure comparison between the counts and a threshold $t \in \mathbb{Z}_{2^n}$, it is crucial to not only preserve the privacy of the report counts of messages until they reach the threshold but also enable the EEMS to flexibly adjust t without requiring a complete system reset. Our solution is inspired by the works [4, 7], where $S_{\{1,2\}}$ hold identical masked report counts via additive secret sharing [2] and then compare them with the masked threshold by leveraging distributed comparison functions (DCF) [3, 6].

To let $S_{\{1,2\}}$ obtain identical masked report counts, our idea is to have the EEMS generate a random mask $r \in \mathbb{Z}_{2^n}$ that is split into additive secret shares $[r]_1$ and $[r]_2$, assigned to S_1 and S_2 . $S_b, b \in \{1, 2\}$ compute $[y^x]_b + [r]_b$, exchange them to get an identical masked report count $y^x + r$ for the message indexed by x . Within that, the problem is translated to compare $y^x + r$ held by $S_{\{1,2\}}$ and

³ We use this protocol [15] as a black box. Thus, we omit the detailed description for simplicity and show it in Appendix A.

the masked threshold $t + r$ held by the EEMS privately⁴. To handle this, the EEMS can derive a pair of DCF keys for the comparison function $g_{\mu,\nu}$ where $\mu = t + r \in \mathbb{Z}_{2^{n+s}}$ and $\nu = 1$ and deliver them to $S_{\{1,2\}}$, which then evaluate the DCF keys on $y^x + r$, exchange the evaluation values, and check whether or not the sum equals one. If it outputs one, the report count y^x of the message indexed by x is greater than the threshold t . The count and threshold will not be revealed to $S_{\{1,2\}}$ during the comparison process, fully respecting the privacy requirements of the EEMS. Furthermore, such a solution can achieve flexible and adaptable threshold settings with invariant and light communication costs. Formally, let $\gamma \xleftarrow{R} \mathbb{Z}$ denote random sampling of γ from $\mathbb{Z}$, $\mathbf{TalCom}([y]_1, [y]_2, t) \rightarrow \tilde{\mathcal{I}}$:

1. For $l \in [M]$, the EEMS samples a mask $r_l \in \mathbb{Z}_{2^n}$ and $[r_l]_1, [r_l]_2 \xleftarrow{R} \mathbb{Z}_{n+s}$ such that $r_l = [r_l]_1 + [r_l]_2$, generates DCF keys $\kappa_1^l, \kappa_2^l \in \mathbb{Z}_{2^{n+s}}$ for the comparison function $g_{\mu=t+r_l, \nu=1}$, and delivers $[r_l]_b, \kappa_b^l$ to S_b where $b \in \{1, 2\}$.
2. $S_{\{1,2\}}$ initiates a matrix $\tilde{\mathcal{I}}$.
3. For $x \in \mathbf{id}\mathbf{x}, \mathcal{I} \in \tilde{\mathcal{I}}^M, b \in \{1, 2\}, l \in [M], [y^x]_b \in [y]_b$:
 - (a) S_b computes $[y^x + r_l]_b \leftarrow [y^x]_b + [r_l]_b \bmod \mathbb{Z}_{2^{n+s}}$.
 - (b) $S_{\{1,2\}}$ swap $[y^x + r_l]_1$ and $[y^x + r_l]_2$ to get the masked count $y^x + r_l \leftarrow [y^x + r_l]_1 + [y^x + r_l]_2 \bmod \mathbb{Z}_{2^{n+s}}$.
 - (c) S_b computes $tp_b \leftarrow \text{DCF.Eval}(\kappa_b^l, y^x + r_l)$.
 - (d) $S_{\{1,2\}}$ exchange tp_1 and tp_2 and get the output $1/0 \leftarrow tp_1 + tp_2$.
 - (e) If the output equals 1, $S_{\{1,2\}}$ add the corresponding $\mathcal{I}$ into $\tilde{\mathcal{I}}$.
4. $S_{\{1,2\}}$ deliver $\tilde{\mathcal{I}}$ to the EEMS.

4.4 Secure Source Tracing

Nemesis supports secure source tracing for messages with report counts surpassing the threshold by designing the algorithm **TraSrc**, as described in Fig. 2. In particular, the EEMS uses $H(c)$ as an index to retrieve the stored c, ik and decrypt c using the key mk to get m . If m is audited as abusive, the EEMS decrypts src using the key ik to recover U_s for taking further actions.

5 Security of Nemesis

We present security definitions capturing the security goals provided by Nemesis. Due to space constraints, we refer readers to Appendix B for full security definitions written as indistinguishable games.

Definition 1 (Security vs. EEMS). *We say that the scheme is secure against a semi-honest EEMS if the advantage of PPT $\mathcal{A}$ is negligible in $\text{Game}_{\text{Nemesis}}^{\text{EEMS}}(\mathcal{A})$, i.e., $\text{Adv}_{\text{Nemesis}}^{\text{EEMS}}(\mathcal{A}) \leq \text{negl}(\lambda)$.*

⁴ It is noted that two numbers are added in the ring $\mathbb{Z}_{2^n}$, the sum may overflow leading to incorrect comparison results due to the modular operation. To prevent that, we sample shares in $\mathbb{Z}_{2^{n+s}}$ to represent a number in $\mathbb{Z}_{2^n}$ and p equals $n + s$ where s is a statistical security parameter.

Theorem 1. *Using Definition 1, Nemesis is secure against a semi-honest EEMS.*

Proof Sketch for Theorem 1. The EEMS can view c and ch on a call of `GenId` and the leakage from the underlying `send`. The challenger views the same leakage. c and ch are indistinguishable from random values since mk is randomly and uniformly selected and both $E(\cdot)$ and $H(\cdot)$ are computationally indistinguishable schemes, for any PPT $\mathcal{A}$, the probability of distinguish two ciphertexts is at most $1/2 + \text{negl}(\lambda)$. The EEMS can view a matrix $\tilde{\mathcal{I}}$ on a call of `TraSrc` to learn the plaintext message and its source while the view of all other messages is indistinguishable from uniformly random strings so that it cannot obtain anything about other messages.

Definition 2 (Security vs. PAS). *We say that the scheme is secure against two semi-honest servers if the advantage of PPT $\mathcal{A}$ is negligible in $\text{Game}_{\text{Nemesis}}^{\text{PAS}}(\mathcal{A})$, i.e., $\text{Adv}_{\text{Nemesis}}^{\text{PAS}_1}(\mathcal{A}) + \text{Adv}_{\text{Nemesis}}^{\text{PAS}_2}(\mathcal{A}) + \text{Adv}_{\text{Nemesis}}^{\text{PAS}_3}(\mathcal{A}) \leq \text{negl}(\lambda)$.*

Theorem 2. *Using Definition 2, Nemesis is secure against two semi-honest servers.*

Proof Sketch for Theorem 2. Each server can view shares and random values on a call of `secret-shared shuffle`, `VrfPrf`, `CountRepo`, and `TalCom`. The challenger views the same values. They follow the protocol honestly and use input from each other. All values they view are uniformly random by defining a shuffle correlation during shuffling. All the inputs and outputs in `VrfPrf`, `CountRepo`, and `TalCom` are exactly indistinguishable from the values that would be sent and received in the real protocol. Hence, the two versions are indistinguishable for any PPT $\mathcal{A}$.

Definition 3 (Security vs. Users) *We say that the scheme is secure against malicious users if for any PPT $\mathcal{A}$ wins $\text{Game}_{\text{Nemesis}}^{\text{U}}(\mathcal{A})$ with negligible probability, i.e., $\text{Adv}_{\text{Nemesis}}^{\text{U}}(\mathcal{A}) \leq \text{negl}(\lambda)$.*

Theorem 3 *Using Definition 3, Nemesis is secure against malicious users.*

Proof Sketch for Theorem 3. The view of $\mathcal{A}$ on a call of `GenId` and `VrfId` involves pairs of m and $\mathcal{I}$. src and σ are encryption of the identity and signature generated from a secure encryption scheme $E(\cdot)$ and a secure signature scheme $\text{Sign}(\cdot)$, which have been proved that the outputs are indistinguishable for any PPT $\mathcal{A}$. mk is a uniformly random value and thus, any PPT $\mathcal{A}$ cannot distinguish between two uniformly random values. $H(\cdot)$ is a random oracle. There is a negligible probability of outputting the same values with different inputs for all PPT $\mathcal{A}$. Hence, the probability of passing all verification with a fake $\mathcal{I}$ is negligible.

6 Implementation and Evaluation

In this section, we show the implementation of Nemesis to facilitate practical deployment in real scenarios and we perform two sets of experiments for evaluating Nemesis’s performance: 1) we assess the costs incurred by each entity during the three phases; 2) we compare our Rust implementation with the open-source prototype of the tree-linkable scheme of [24] which maintains the same EEMS setting. To ensure a fair comparison, we run the benchmarks of [24] on the same machine. While FACTS [22] provides fuzzy threshold-triggered reporting-then-tracing, its non-open-source nature prevents us from conducting experimental comparisons with Nemesis. Therefore, we present FACTS’ evaluation results solely for intuition purposes.

Implementation. We provide a prototype implementation in Rust, consisting of roughly 2K LOC, and we use Signal’s Double Ratchet protocol⁵ for the baseline EEMS send-receive scheme. We use RustCrypto library’s implementation of AES-128 GCM for symmetric encryption scheme, SHA-256 for collision-resistant hash function, MD5 for cryptographic hash function, and ed25519 for signature scheme. We set the security parameters λ and s to 128 so that we sample shares and compute MACs using a ring $\mathbb{Z}_{2^{256}}$. We implement distributed point function and distributed comparison function with SIMD and Hirose optimizations as libraries. Our optimized libraries are available as back-ends for other systems. Nemesis uses rocksdb⁶ for the underlying key-value store. Our implementation is open-source and available at <https://github.com/myl7/nemesis>.

Evaluation Environment. We run the benchmarks for users, the EEMS, and the PAS on a machine running Arch Linux with AMD Ryzen 7 5800H as the CPU and 16 GB of RAM. During the evaluation, we establish 16 threads on 8 physical cores and ensure the charger is plugged in to avoid CPU under clocking. Each experimental result is performed using Criterion and the average of 10 trials, with outliers removed.

Experimental Parameters. Message size limits are 2,000 B in Signal and 4,096 B in WhatsApp and Telegram. Thus, message sizes in our experiments vary from 16 B to 4,096 B. We explore different epoch settings, with the number M of messages per epoch ranging from 10^4 to 10^8 .

6.1 Benchmarks: Costs of Each Entity

Message Delivery and Reporting. Overall, the additional costs for users are low, requiring computation overheads at the microsecond level, which closely resemble the performance of an EEMS without reporting-then-tracing capabilities. For a user, the computation times for generating and verifying an identifier tuple increase linearly with the message size, which takes about $7.993\mu s$ and approximately $43.659\mu s$ for the maximum message size 4,096 B, respectively.

⁵ <https://github.com/sebastianv89/double-ratchet>.

⁶ <https://rocksdb.org/>.

Table 2. Computation times for the operations of the PAS and the offline operations of the EEMS in the phase of report-hiding tally with different numbers of messages.

Number of messages	10^4	10^5	10^6	10^7
The PAS's costs				
Secret shuffle (s)	0.034	0.377	3.686	35.835
Blind verification (ms)	0.508	5.435	54.338	527.24
Oblivious counting (ms)	4.614	46.697	450.56	4537.8
Secure comparison (s)	0.050	0.500	4.984	48.777
The EEMS's offline cost				
Generate masks and DCF keys (s)	0.135	1.334	14.862	153.27

The additional communication overhead is from delivering the identifier tuple, whose size is independent of message size and only 164 B. The computation time of splitting it into two shares is negligible and around $0.055\mu s$. For the EEMS, the computation time to produce items of an identifier tuple is independent of message size and takes about $27.091\mu s$. Subsequently, the EEMS performs a key-value append operation. We exclude the evaluation of this operation as it does not involve any cryptographic computations, and the time required for this operation is low in most industrial key-value stores. Baseline EEMS does not store message ciphertexts while reporting-then-tracing EEMSs [18, 19, 22, 24, 33] require additional storage costs. In Nemesis, the size of the stored key-value data per message is small and no more than 28 B. If the purpose is to address ongoing viral abusive messages, the EEMS can be designed to selectively store data from specific epochs within the current week or month to optimize storage costs. A reporter takes around $13.968\mu s$ and $0.238\mu s$ to generate a report and proof. Overall, the data sent to each server is 2.957 KB.

Report-Hiding Tally. We measure the computation times for the operations of the PAS and the offline operations of the EEMS⁷ with different numbers of messages, as shown in Table 2. As expected in Nemesis, the computation times for all operations of the PAS and the EEMS grow with M . The results show that the cost is low even for handling reports over millions of messages. Note that for the PAS, we compute the time of a single server since the operations of the two servers can be done in parallel. While we do not measure the cost of network communication for this procedure, it is important to note that we do achieve it within our system, and we anticipate that it would not substantially increase costs. For the EEMS, the total size of masks and DCF keys sent to each server is linear with M per epoch and 5.143M KB.

Secure Source Tracing. The running time that the EEMS takes is only $41.327\mu s$ for accessing and tracing a message source. Note that we do not include

⁷ We do not include the preprocessing costs of generating correlation quantities and deliver them to the PAS since the EEMS can do the preprocessing during off-peak times.

the moderation time for a message that the moderator of the EEMS takes, and we assume the time is moderate.

Table 3. A comparison of the additional computation times and communication overheads of the tree-linkable scheme of [24] and Nemesis to deliver and report a 1 KB message.

	Sender		Receiver		Reporter		EEMS	E2E latency
[24]	148.01 μ s	256 B	131.71 μ s	320 B	45.710 μ s	160 B	20 μ s	7.3901 ms
Nemesis	2.148 μ s	164 B	27.091 μ s	164 B	43.659 μ s	2.957 KB	27.091 μ s	1.0052 ms

6.2 Comparison with Prior Work

Here, we compare with the tree-linkable scheme of [24] and FACTS [22]. Table 3 presents a comparison of the tree-linkable scheme of [24] and Nemesis for delivering and reporting a 1 KB message⁸. Both the additional computation times and communication overheads are independent of message size. The tree-linkable scheme [24] incurs higher user computation overheads than Nemesis. Meanwhile, the additional communication overheads of a sender and receiver are also higher than Nemesis. Although it requires a smaller communication overhead for reporting a message than Nemesis, Nemesis achieves report-hiding protection for a reporter and only requires a single flow of communication, which we think is affordable for both PC and mobile users.

While both Nemesis and FACTS [22] are threshold-triggered reporting-then-tracing systems for EEMS, there are differences in terms of: (1) *security*: Nemesis protects the count of any message and the threshold against users, while reporters in FACTS can learn this information; (2) *accuracy of report counting*: Nemesis ensures exact report tally while FACTS provides approximate report tally; (3) *performance*: Nemesis allows issuing a report with low running time (around 14.206 μ s), while the time of issuing a report in FACTS grows with the threshold decrease. Based on their evaluations, it needs around 375ms for a reporter when the threshold is 200.

7 Conclusion and Discussion

We design, implement, and evaluate Nemesis, a new privacy-friendly user-reporting-based content moderation system for EEMSs. Nemesis enables messaging users to report abusive messages while ensuring that the EEMS can trace

⁸ Note that in our evaluation, we quantify the end-to-end (E2E) latency of delivering a 1 KB message, which encompasses the entire process that begins with a source generating an identifier tuple and delivering it with the message to a receiver via the EEMS and ends when the receiver receives the message and verifies the identifier tuple, including the simulated network overhead of the EEMS server and users in our instances.

the message source only when the report count surpasses a pre-defined threshold, with flexible and adaptable threshold support, enhanced user privacy protection, and robustness against dishonest users over prior art. Extensive experimental results demonstrate that Nemesis achieves practically affordable performance for combating abusive information in existing EEMSs.

Deployment Issues. Like prior works [18, 19, 22, 24, 33], we allow forwarding feature in EEMS. Nevertheless, when a user retypes a received message and sends it to others, the message is treated as new, and the user becomes the perceived source whose malicious behavior remains unchanged, thus making it deemed acceptable. As a result, traceback systems in EEMS typically trace the initial user who sends the message rather than the actual originator. We emphasize that it is possible to integrate client-side applications to detect and avoid such retying behavior.

Future Work. Here we name a few opportunities for future work. Firstly, EEMSs have their own policies for defining abusive content. To avoid unjust censorship, moderation functions should align with policies targeting severe harm developed with authoritative organizations. Secondly, transparency in auditing is essential to verify that in-house moderators adhere to claimed policies [26]. Thirdly, the system should prevent further dissemination of abusive messages without accessing users' plaintext messages. Finally, it is desirable to develop effective incentive mechanisms that encourage users to report with highly accurate judgments.

Acknowledgments. We sincerely thank all anonymous reviewers for their useful comments and instructions. This work was supported in part by CityU of HK under Grants 9678146 and 9678126, by HK RGC under Grants CityU 11218521, 11218322, R6021-20F, R1012-21, RFS2122-1S04, C2004-21G, C1029-22G, and N_CityU139/21, by Guangdong Basic and Applied Basic Research Foundation under Grant 2023A1515010714 and 2023A1515140137, by Guangdong-Hong Kong Joint Laboratory for Data Security and Privacy Preserving under Grant 2023B1212120007, by the Shenzhen Science and Technology Program under Grants JCYJ20220531095416037 and JCYJ20230807094411024, by National Nature Science Foundation of China under Grant 62202398, and by InnoHK initiative, the Government of the HKSAR, and Laboratory for AI-Powered Financial Technologies.

A Protocol for Secret-Shared Shuffle and Verification

The process starts with S_1 holding $[\mathcal{I}^M]_1, \pi_1, \mathbf{a}_1$, and S_2 holding $[\mathcal{I}^M]_2, \pi_2, \mathbf{a}_2, \mathbf{a}_3, \Delta$. Here, $\pi_1, \pi_2 : \mathbb{Z}_M \rightarrow \mathbb{Z}_M$ are two random permutations and applied in a row-wise manner, $\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3$ are three random matrices of the same size as $\mathcal{I}^M$, and $\Delta \leftarrow \pi_2(\pi_1(\mathbf{a}_3) + \mathbf{a}_1) - \mathbf{a}_2$. Note that input-independent correlation quantities $\pi_1, \pi_2, \mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3, \Delta$ can be prepared by a third party (e.g., the EEMS) in advance and distributed offline. Then, S_2 computes and sends $z_2 \leftarrow [\mathcal{I}^M]_2 - \mathbf{a}_3$ to S_1 , which computes and sends $z_1 \leftarrow \pi_1(z_2 + [\mathcal{I}^M]_1) - \mathbf{a}_1$ to S_2 . Afterwards, S_2 computes $\tilde{\mathcal{I}}^M \leftarrow \pi_2(z_1) + \Delta + \mathbf{a}_2$ and delivers it to S_1 .

Finally, $S_{\{1,2\}}$ execute **VrfId** for each entry in the matrix to get a verifiable shuffle identical matrix $\tilde{\mathcal{I}}^M$ where $\tilde{\mathcal{I}}^M = \pi_2(\pi_1(\mathcal{I}^M))$. Each server learns nothing else about $\mathcal{X}^M$ and the correlation quantities held by another server.

B Security Analysis

We define indistinguishable games $\text{Game}_b^{\mathcal{A}}$ where $b \in \{0, 1\}$ is a secret variable that decides the version of the game the probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ sees, and the advantage of the PPT $\mathcal{A}$ is defined as

$$\text{Adv}_{\text{Nemesis}}^{\text{game}}(\mathcal{A}) = \left| \Pr \left[\text{Game}_1^{\mathcal{A}} = 1 \right] - \Pr \left[\text{Game}_0^{\mathcal{A}} = 1 \right] \right|.$$

Security vs. EEMS. We define the game $\text{Game}_{\text{Nemesis}}^{\text{EEMS}}(\mathcal{A})$ between a PPT adversary $\mathcal{A}$ who controls the EEMS and some users and a challenger:

1. $\mathcal{A}$ is given access to some oracles and obtains some sessions and key-value pairs, i.e., $\text{send}(m_i, \mathcal{I}_i; U_{s_i}, U_{r_i})$ and $\{\text{H}(c_i), (c_i, ik_i)\}$ where $i \in$ a user set and no send-receive users (U_{s_i}, U_{r_i}) are controlled by $\mathcal{A}$.
2. The challenger chooses $b \leftarrow \{0, 1\}$: if $b = 0$, runs the **GenId** and **TraSrc** with the inputs from $\mathcal{A}$ and returns $\mathcal{A}$ the resulting EEMS view; if $b = 1$, chooses and sends random values with lengths $|c|$ and $|ch|$ to the EEMS, chooses random values with lengths $|m_i|$ and $|\mathcal{I}_i|$ and sends them from U_{s_i} to U_{r_i} using **send** for each i .
3. $\mathcal{A}$ outputs a bit b' and has the advantage $\text{Adv}_{\text{Nemesis}}^{\text{EEMS}}(\mathcal{A})$.

Security vs. PAS. We define the game $\text{Game}_{\text{Nemesis}}^{\text{PAS}}(\mathcal{A})$ between a PPT adversary $\mathcal{A}$ who controls a semi-honest server (e.g., S_1 or S_2) and some users and a challenger:

1. $\mathcal{A}$ is given access to some oracles and obtains some random values and shares, i.e., z_2 and the shares of identifier tuples from sources and the EEMS.
2. The challenger chooses $b \leftarrow \{0, 1\}$: if $b = 0$, runs the secret-shared shuffle with the inputs from $\mathcal{A}$ and returns $\mathcal{A}$ the resulting S_1 view; if $b = 1$, chooses and sends a random value with length $|z_2|$ to the S_1 , which computes and returns z_1 to the S_2 . Based on z_1 , the S_2 computes and returns the shuffled matrix to the S_1 .
3. $\mathcal{A}$ outputs a bit b' and has the advantage $\text{Adv}_{\text{Nemesis}}^{\text{PAS}_1}(\mathcal{A})$.
4. Once outputting a shuffled matrix, $\mathcal{A}$ views random value γ and shares $(k_1, [\tau]_1, \omega_1)$ from a reporter.
5. The challenger chooses $b \leftarrow \{0, 1\}$: if $b = 0$, runs the **VrfPrf** and **CountRepo** with the inputs from $\mathcal{A}$ and returns $\mathcal{A}$ the resulting S_1 view; if $b = 1$, chooses and sends a random value with length $|\omega_2|$ to the S_1 , which computes γ .
6. $\mathcal{A}$ outputs a bit b' and has the advantage $\text{Adv}_{\text{Nemesis}}^{\text{PAS}_2}(\mathcal{A})$.
7. After obtaining report count shares of messages, $\mathcal{A}$ views $\kappa_1, [r]_1$, the report count shares of messages.

8. The challenger chooses $b \leftarrow \{0, 1\}$: if $b = 0$, runs the **TalCom** with the inputs from $\mathcal{A}$ and returns $\mathcal{A}$ the resulting S_1 view; if $b = 1$, chooses and sends random values with lengths $|\text{DCF.Eval}(\kappa_2, y^x + r_l)|$ to the S_1 , which computes and checks whether the sum of the evaluation values equals one.
9. $\mathcal{A}$ outputs a bit b' and has the advantage $\text{Adv}_{\text{Nemesis}}^{\text{PAS}_3}(\mathcal{A})$.

Security vs. Users. We define the game $\text{Game}_{\text{Nemesis}}^{\text{U}}(\mathcal{A})$ between a PPT adversary $\mathcal{A}$ who controls some malicious users and a challenger:

1. $\mathcal{A}$ selects two messages m and m' with same length, who controls a user U_c . Honest users (U_a and U_b) are not controlled by $\mathcal{A}$.
2. The challenger chooses $b \leftarrow \{0, 1\}$: if $b = 0$, runs the **Genld** with m from $\mathcal{A}$ and sends the m and random values with length $[\mathcal{I}]$ from U_a to U_b via the underlying send operation, runs the **Genld** with m' from $\mathcal{A}$ and sends the m' and random values with length $[\mathcal{I}]$ from U_b to U_c via the underlying send operation, and returns $\mathcal{A}$ the resulting user view; else, runs the **Genld** with m from $\mathcal{A}$ and sends the m and $\mathcal{I}$ from U_a to U_b via the underlying send operation, runs the **Genld** with m from $\mathcal{A}$ and sends the m and $\mathcal{I}$ from U_b to U_c via the underlying send operation.
3. $\mathcal{A}$ outputs a bit b' and has the advantage $\text{Adv}_{\text{Nemesis}}^{\text{U}_1}(\mathcal{A})$.
4. $\mathcal{A}$ calls **Genld** with the input m and obtains $\mathcal{I}$ with the source U_c . $\mathcal{A}$ generates m' and $\mathcal{I}$ and sends them to U_a .
5. The challenger runs **Vrflld** with the inputs m' and $\mathcal{I}$ where $\mathcal{I}$ represents a message source who is not controlled by $\mathcal{A}$. $\mathcal{A}$ wins if **Vrflld** successes.
6. The challenger calls secret-shared shuffle with the inputs from $\mathcal{A}$ and returns the resulting PAS view. $\mathcal{A}$ wins if the inputs are not dropped.
7. $\mathcal{A}$ calls **RepoMsg** with input $\mathcal{I}$ and sends random values to the PAS. $\mathcal{A}$ wins if **RepoMsg** returns success.

References

1. Bartusek, J., Garg, S., Jain, A., Policharla, G.V.: End-to-end secure messaging with traceability only for illegal content. In: Hazay, C., Stam, M. (eds.) **EUROCRYPT** 2023. LNCS, vol. 14008, pp. 35–66. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-30589-4_2
2. Benhamouda, F., Degwekar, A., Ishai, Y., Rabin, T.: On the local leakage resilience of linear secret sharing schemes. *J. Cryptol.* **34**, 1–65 (2021)
3. Boneh, D., Boyle, E., Corrigan-Gibbs, H., Gilboa, N., Ishai, Y.: Lightweight techniques for private heavy hitters. In: *Proceedings of IEEE SP* (2021)
4. Boyle, E., et al.: Function secret sharing for mixed-mode and fixed-point secure computation. In: Canteaut, A., Standaert, F.X. (eds.) **EUROCRYPT** 2021. LNCS, vol. 12697, pp. 871–900. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-77886-6_30
5. Boyle, E., Gilboa, N., Ishai, Y.: Function secret sharing. In: Oswald, E., Fischlin, M. (eds.) **EUROCRYPT** 2015. LNCS, vol. 9057, pp. 337–367. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-46803-6_12

6. Boyle, E., Gilboa, N., Ishai, Y.: Function secret sharing: improvements and extensions. In: Proceedings of ACM CCS (2016)
7. Boyle, E., Gilboa, N., Ishai, Y.: Secure computation with preprocessing via function secret sharing. In: Hofheinz, D., Rosen, A. (eds.) TCC 2019. LNCS, vol. 11891, pp. 341–371. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-36030-6_14
8. de Castro, L., Polychroniadou, A.: Lightweight, maliciously secure verifiable function secret sharing. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022. LNCS, vol. 13275, pp. 150–179. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-06944-4_6
9. Chen, W., Popa, R.A.: Metal: a metadata-hiding file-sharing system. In: Proceedings of NDSS (2020)
10. Cohn-Gordon, K., Cremers, C., Dowling, B., Garratt, L., Stebila, D.: A formal security analysis of the signal messaging protocol. *J. Cryptol.* **33**, 1914–1983 (2020)
11. Cramer, R., Damgård, I., Escudero, D., Scholl, P., Xing, C.: SpdZ_{2^k} : efficient MPC mod 2^k for dishonest majority. In: Malkin, T., Peikert, C. (eds.) CRYPTO 2021. LNCS, vol. 12827, pp. 656–686. Springer, Cham (2018). https://doi.org/10.1007/978-3-030-84252-9_22
12. Davidson, A., Snyder, P., Quirk, E., Genereux, J., Livshits, B., Haddadi, H.: Star: secret sharing for private threshold aggregation reporting. In: Proceedings of ACM CCS (2022)
13. Dechand, S., Naiakshina, A., Danilova, A., Smith, M.: In encryption we don't trust: the effect of end-to-end encryption to the masses on user perception. In: Proceedings of EuroS&P (2019)
14. Dizikes, P.: Study: On Twitter, false news travels faster than true stories (2018). <https://news.mit.edu/2018/study-twitter-false-news-travels-faster-true-stories-0308>
15. Eskandarian, S., Boneh, D.: Clarion: anonymous communication from multiparty shuffling protocols. In: Proceedings of NDSS (2022)
16. Gilboa, N., Ishai, Y.: Distributed point functions and their applications. In: Nguyen, P.Q., Oswald, E. (eds.) EUROCRYPT 2014. LNCS, vol. 8441, pp. 640–658. Springer, Heidelberg (2014). https://doi.org/10.1007/978-3-642-55220-5_35
17. Hua, Y., Namavari, A., Cheng, K., Naaman, M., Ristenpart, T.: Increasing adversarial uncertainty to scale private similarity testing. In: Proceedings of USENIX Security (2022)
18. Issa, R., Alhaddad, N., Varia, M.: Hecate: abuse reporting in secure messengers with sealed sender. In: Proceedings of USENIX Security (2022)
19. Kenney, E., Tang, Q., Wu, C.: Anonymous traceback for end-to-end encryption. In: Proceedings of ESORICS (2022)
20. Kogan, D., Corrigan-Gibbs, H.: Private blocklist lookups with checklist. In: Proceedings of USENIX Security (2021)
21. Kulshrestha, A., Mayer, J.: Identifying harmful media in {End-to-End} encrypted communication: Efficient private membership computation. In: Proceedings of USENIX Security 21 (2021)
22. Liu, L., Roche, D.S., Theriault, A., Yerukhimovich, A.: Fighting fake news in encrypted messaging with the fuzzy anonymous complaint tally system (facts). In: Proceedings of NDSS (2022)
23. Newman, Z., Servan-Schreiber, S., Devadas, S.: Spectrum: high-bandwidth anonymous broadcast. In: Proceedings of USENIX NSDI (2022)
24. Peale, C., Eskandarian, S., Boneh, D.: Secure complaint-enabled source-tracking for encrypted messaging. In: Proceedings of ACM CCS (2021)

25. Peter Elkind, J.G., Craig Silverman, P.: How Facebook undermines privacy protections for its 2 billion WhatsApp users (2021). <https://thewire.in/tech/facebook-undermines-privacy-protections-whatsapp-users>
26. Rahalkar, C., Virgaonkar, A.: Sok: content moderation schemes in end-to-end encrypted systems. arXiv preprint [arXiv:2208.11147](https://arxiv.org/abs/2208.11147) (2022)
27. Rösler, P., Mainka, C., Schwenk, J.: More is less: on the end-to-end security of group chats in signal, WhatsApp, and Threema. In: Proceedings of EuroS&P (2018)
28. Scheffler, S., Kulshrestha, A., Mayer, J.: Public verification for private hash matching. In: Proceedings of IEEE SP (2023)
29. Shtaya, M.: How Meta’s platforms normalize anti-Palestinian racism (2023). <https://mei.edu/publications/how-metas-platforms-normalize-anti-palestinian-racism>
30. Thomas, K., et al.: Sok: hate, harassment, and the changing landscape of online abuse. In: Proceedings of IEEE SP (2021)
31. Thomas, K., et al.: Robust, privacy-preserving, transparent, and auditable on-device blocklisting. arXiv preprint [arXiv:2304.02810](https://arxiv.org/abs/2304.02810) (2023)
32. Tyagi, N., Grubbs, P., Len, J., Miers, I., Ristenpart, T.: Asymmetric message franking: content moderation for metadata-private end-to-end encryption. In: Boldyreva, A., Micciancio, D. (eds.) CRYPTO 2019. LNCS, vol. 11694, pp. 222–250. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-26954-8_8
33. Tyagi, N., Miers, I., Ristenpart, T.: Traceback for end-to-end encrypted messaging. In: Proceedings of ACM CCS (2019)
34. Zhong, K., Ma, Y., Angel, S.: Ibex: privacy-preserving ad conversion tracking and bidding. In: Proceedings of ACM CCS (2022)



ARPSO: An OIDC-Compatible Privacy-Preserving SSO Scheme Based on RP Anonymization

Junlin He¹, Lingguang Lei^{2,3}, Yuewu Wang^{4(✉)}, Pingjian Wang^{2,3},
and Jiwu Jing⁴

¹ School of Computer Science and Technology, University of Chinese Academy of Sciences, Beijing 100043, China

² Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100089, China

³ School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100043, China

⁴ School of Cryptology, University of Chinese Academy of Sciences, Beijing 100043, China
`wangyuewu@ucas.ac.cn`

Abstract. OpenID Connect (OIDC) is one of the most widely used single sign-on (SSO) protocols today. However, like all other popular SSO protocols, it does not take user privacy into account, which means that Identity Providers (IdPs) and colluding Relying Parties (RPs) may carve a user's behavior through protocol interaction easily. Many studies have attempted to address these privacy issues, but none of them support the most commonly adopted OIDC code flow very well because of their insufficient consideration of the IdP-to-RP authentication, which is the key mechanism to enhance the security of OIDC. In this paper, we propose a privacy-preserving SSO scheme named ARPSO that may be adapted to OIDC code flow very well. We first realize RP anonymous authentication by issuing anonymous credentials to the RP, thus preventing the user's login behavior from being tracked by the IdP. We also design a two-party secure computation scheme based on anonymous credentials, which implements user identity mix-up and thus prevents the user identity from being linked by colluding RPs. In addition, we introduce a user-imperceptible trusted in-browser data forwarding mechanism to ensure that the entire SSO process is transparent to the user, allowing users to seamlessly use conventional authentication mechanisms and standard browsers for a great user experience. The security analysis shows that ARPSO achieves the privacy goals without compromising the intrinsic security properties of the OIDC protocol. Performance evaluation on the prototype implementation shows that ARPSO may work with acceptable overhead compared to original SSO systems.

Keywords: Single Sign-On · OpenID Connect · Code Flow · Privacy

1 Introduction

OpenID Connect (OIDC) [38] is the most widely adopted single sign-on (SSO) protocol for public online services. It is currently used by over 450,000 websites, representing 61% of the total login provider market [4]. OIDC provides a seamless user experience by enabling users to access multiple services, known as relying party (RP), using a single identity maintained by the identity provider (IdP). Specifically, the IdP authenticates the user and issues an ID token, which the RP uses to extract and verify the user's identity. There are two types of ID token forwarding mechanisms in OIDC: a risky implicit flow and a more secure code flow (as shown in Fig. 1). The former forwards the token via the risky user agent (e.g., browser), with risks such as token leakage [34]. The latter only forwards an authorization code through the user agent, which the token is transferred directly between the RP's and the IdP's back-end servers, and is only sent to the designated RP after its identity has been authenticated. OIDC code flow is a widely used protocol today and is recommended by industry (e.g., Yahoo [3] and Microsoft [39]) and Technical Standards Organizations [35].

Recent studies [24–26, 33, 44, 45] have paid much attention to two serious privacy issues with OIDC, namely traceability and linkability. The former arises from the need of IdP to generate an ID token containing both the user identifier (*uid*) and the RP identifier (*cid*) representing a specific user on a specific RP, which allows the IdP (usually assumed as honest-but-curious) to track the user's login events. The latter is because the IdP typically uses the user's unique identifier to generate their login credentials on different RPs to ensure a consistent identity for multiple login sessions on the same RP. This allows colluding RPs to link a user's login events across different RP services.

A number of privacy-preserving SSO protocols [14, 18, 19, 23–26, 31, 33, 44, 45] have been proposed to address the traceability and linkability issues, but none of them may be adopted directly for the OIDC code flow. These solutions can be divided into two categories, namely, user anonymization and RP identifier randomization. The user anonymization schemes [14, 31] anonymize the user identity at the user side, which requires the user to maintain/generate multiple unlinkable identities for different RPs and authenticated by the IdP through anonymization mechanisms such as anonymous credentials [31], blind signature [14], etc. Therefore, a customized client is usually required, leading to poor usability and little adoption in practice. The RP identifier randomization solutions [19, 24–26, 33] allow users to be authenticated by the IdP using conventional mechanisms such as username-password, two-factor authentication (2FA) and FIDO2, etc., thus providing a better user experience. However, these solutions are tailored for OIDC implicit flow that does not involve RP authentication, enabling RPs to provide only randomized RP identifiers to IdPs for untraceability.

In this paper, we provide an RP-anonymization-based privacy-preserving SSO scheme called ARPSO, which can be adapted to OIDC code flow well. This scheme is transparent to the users (i.e., conventional authentication mechanisms and standard browsers can be used) and solves both traceability and linkability problems while retaining the intrinsic security properties of the OIDC

protocol [17]. We first design an RP anonymous credential system to implement IdP-to-RP anonymous authentication, which requires the RP to generate an RP anonymous identifier in each SSO session randomly. Because of the introduced IdP-to-RP anonymous authentication, generating a consistent identity for the same user across multiple login sessions on the same RP becomes difficult. To address this problem, we introduce a two-party secure computation scheme that supports IdP-to-RP anonymous authentication, where the RP provides the RP anonymous identifier (i.e., the anonymous *cid*), the IdP provides the user identifier (i.e., *uid*), and they cooperate in generating the ID token. The RP then derives a unique, unlinkable, and consistent identifier for each user on this RP through its own identifier (i.e., *cid*) and the ID token.

Moreover, we also introduce a user-transparent forwarding mechanism to securely forward the confidential data via the user agent, i.e., the ID token in implicit flow or the authorization code in code flow. Existing solutions achieve secure forwarding by introducing a trusted component (e.g., a trusted plug-in or a trusted execution environment) in the user agent, which is less user-friendly due to the need for each user to participate. In ARPSSO, we enable secure in-browser data forwarding via a trusted script. The trusted script is publicly available with fixed content, so RPs, IdPs, and users can verify its integrity and security easily.

We prove the security of ARPSSO through protocol analysis and formal methods, including whether it guarantees two privacy properties (i.e., user sign-on untraceability and user identity unlinkability) as well as the intrinsic security properties of the OIDC protocol. We also evaluate the performance of ARPSSO through a prototype system. The results show that the overhead introduced is acceptable compared to conventional SSO systems.

The main contributions of this paper are:

- We propose an OIDC-compatible privacy-preserving SSO scheme named ARPSSO that enables user sign-on untraceability and user identity unlinkability. To the best of our knowledge, it is the first scheme to support the code flow of the OIDC protocol.
- We introduce a user-imperceptible in-browser data forwarding mechanism. The entire SSO process is transparent to the user and works seamlessly with conventional authentication mechanisms and standard browsers, making it user-friendly.
- We analyze the security of ARPSSO and show that it balances both privacy goals and the intrinsic security properties of the OIDC protocol. We also evaluate the performance of the protocol through a prototype implementation and show that the overhead is acceptable compared to conventional SSO systems.

2 Background

2.1 OpenID Connect

OpenID Connect provides an identity layer on top of the OAuth 2.0 framework [29], tailored for SSO. There are two primary OIDC protocol flows, the code flow and the implicit flow, as illustrated in Fig. 1. The Code flow may be viewed as a full version of the implicit flow, with additional RP authentication (i.e., steps ⑨) added to ensure secure ID token forwarding. It consists of 13 steps: ① The user accesses the RP service and initiates a login request. ②–③ The RP redirects the login request to the IdP. ④–⑤ The IdP prompts the user to perform user authentication. ⑥–⑧ Upon successful authentication, the IdP generates an authorization *code* and redirects it to the RP via the user agent. ⑨–⑪ The RP backend authenticates to the IdP and exchanges the *code* for the *id_token*. ⑫–⑬ Finally, the RP verifies the *id_token* and notifies the user of the login result.

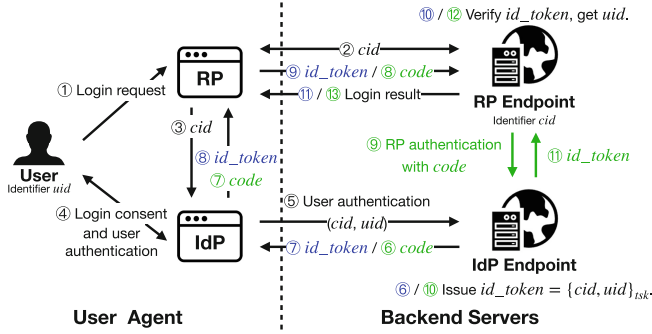


Fig. 1. OIDC Protocol Flow. It includes two working flows: a risky implicit flow and a more secure code flow, marked in blue and green. (Color figure online)

2.2 Proof of Knowledge

Proof of Knowledge (PoK) is realized by using zero-knowledge proofs. It allows the prover to prove to the verifier that they know a piece of information without revealing it. The PoK scheme in this paper is non-interactive, which is based on the Sigma protocol [13] and the Fiat-Shamir heuristic [21]. For instance, the notation $PoK \{(\alpha, \beta) : y = g^\alpha h^\beta \wedge \tilde{y} = \tilde{g}^\alpha \wedge \phi(\alpha, \beta) = true\}$ represents a Non-Interactive Zero-Knowledge (NIZK) proof of knowledge of integers α, β such that $y = g^\alpha h^\beta$ and $\tilde{y} = \tilde{g}^\alpha$ where α, β satisfy relation ϕ .

2.3 Anonymous Credential Based on PS Signature

An anonymous credential system typically involves three roles: the issuer, the prover, and the verifier. The issuer issues anonymous credentials containing a

set of identity attributes for the prover. These credentials enable the prover to authenticate itself to the verifier without revealing personal information (e.g., age, gender), a process called anonymous authentication. Our scheme uses the Pointcheval-Sanders (PS) signature algorithm [41] to build the anonymous credential system, described as follows.

Setup(1^ℓ) $\rightarrow$ (pp): Choose a type-3 bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T, \mathbb{G}_1 \neq \mathbb{G}_2$ [22] with generators g_1 and g_2 for groups $\mathbb{G}_1$ and $\mathbb{G}_2$ to define system parameters $pp \leftarrow (e, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2)$.

KeyGen(pp, n) $\rightarrow$ (sk, vk): For n attributes, randomly select $x, y_1, \dots, y_n \xleftarrow{\$} \mathbb{Z}_p$. Construct $X = g_1^x, Y_i = g_1^{y_i}$ and $\hat{X} = g_2^x, \hat{Y}_i = g_2^{y_i}$ for $i = 1 \dots n$. The keypair is (sk, vk) where $sk = (x, y_1, \dots, y_n, X)$ and $vk = (Y_1, \dots, Y_n, \hat{X}, \hat{Y}_1, \dots, \hat{Y}_n)$.

PSSign(A, h, sk) $\rightarrow \sigma$: With a random generator $h \xleftarrow{\$} \mathbb{G}_1$, produce signature $\sigma = (\sigma_1, \sigma_2) = (h, h^{x+A_1 \cdot y_1 + \dots + A_n \cdot y_n})$ on attributes $A = (A_1, \dots, A_n)$.

PSVerify(σ, A, vk) $\rightarrow b$: Verify the signature $\sigma \leftarrow (\sigma_1, \sigma_2)$ on attributes A using $e(\sigma_2, g_2) = e(\sigma_1, \hat{X}) \prod_{i=1}^n e(\sigma_1, \hat{Y}_i)^{A_i}$.

It is essential to highlight that the PS signature possesses a randomizable feature. Specifically, **PSVerify**($(\sigma_1^k, \sigma_2^k), A, vk$) = $true, k \xleftarrow{\$} \mathbb{Z}_p$.

Leveraging the PS signature, we can build an anonymous credential system containing two distinct phases. In the initial phase, the prover obtains the validation of a set of attributes $A = (A_1, \dots, A_n)$ from the issuer, i.e., it obtains a PS signature on attributes A . One approach is to sign directly using **PSSign**, requiring the prover to disclose all attributes. Another approach is to use a blind issue protocol **CredIssue**, where the prover separates attributes into hidden attributes $A_h = (A_1, \dots, A_j)$ and public attributes $A_p = (A_{j+1}, \dots, A_n)$. When the issuer signs these attributes A , the hidden attributes A_h remain unknown.

CredIssue(pp, sk, A_h, A_p, ϕ) $\rightarrow (\sigma)$: The issuer and the prover run the following interactive protocol for anonymous credential issuance.

- **PrepareBlindSign**(vk, A_h, ϕ) $\rightarrow (d, \lambda, \phi)$: The prover chooses $d \xleftarrow{\$} \mathbb{Z}_p$ and computes commitment $C = g_1^d \prod_{i=1}^j Y_i^{A_i}$, then generates a NIZK proof $\lambda = PoK \left\{ (d, A_h) : C = g_1^d \prod_{i=1}^j Y_i^{A_i} \wedge \phi(A_h) = true \right\}$, which validates C and a relation ϕ . This λ is then sent to the issuer.
- **BlindSign**(sk, A_p, λ, ϕ) $\rightarrow (A_p, \hat{\sigma})$: The issuer verifies the NIZK proof λ , and extracts C from it. Then the issuer chooses $r \xleftarrow{\$} \mathbb{Z}_p$, and computes blind signature $\hat{\sigma} = (g_1^r, (XC \prod_{i=j+1}^n Y_i^{A_i})^r)$ with public attributes A_p . Finally, the issuer sends $A_p, \hat{\sigma}$ to the prover.
- **Unblind**($d, A_h, A_p, \hat{\sigma}$) $\rightarrow (A, \sigma)$: The prover receives $(\hat{\sigma}_1, \hat{\sigma}_2) \leftarrow \hat{\sigma}$, then unblinds it by computing $\sigma = (\hat{\sigma}_1, \hat{\sigma}_2 / \hat{\sigma}_1^d)$ to obtain the signature $\sigma = (\sigma_1, \sigma_2) = (h, h^{x+A_1 \cdot y_1 + \dots + A_n \cdot y_n})$, $h = g_1^r$ on the attributes $A = A_h \cup A_p$.

In the second phase, the prover demonstrates to the verifier the possession of certain attributes $A_p' = (A_{j'+1}, \dots, A_n)$, previously authenticated by the issuer, and hides other attributes $A_h' = (A_1, \dots, A_{j'})$. These hidden attributes can be

verified to satisfy a specific relation ϕ' (e.g., age over 18) without the verifier being explicitly aware of them. This involves the following two algorithms.

CredPresent(vk, A, σ, ϕ') $\rightarrow (A_h', A_p', \pi, \phi')$: The prover first selects $k, t \xleftarrow{\$} \mathbb{Z}_p$ and randomizes the signature by computing $\sigma' = (\sigma'_1, \sigma'_2) = (\sigma_1^k, (\sigma_2 \sigma_1^t)^k)$. Then the prover chooses a set of hidden attributes $A_h' \in A$ and public attributes $A_p' = A \setminus A_h'$. Finally, the prover generates the credential presentation containing a NIZK proof $\pi = PoK\{(t, A_h') : PSVerify((\sigma'_1, \sigma'_2/\sigma_1^t), A, vk) = true \wedge \phi'(A_h') = true\}$ to prove the correctness of the signature and sends it to the verifier.

CredPresentVerify(vk, A_p', π, ϕ') $\rightarrow (true/false)$: The verifier verifies the credential presentation with the NIZK proof π to check whether the hidden attributes satisfy the statement ϕ' or not.

3 Adversary Model, Assumptions and Target Properties

In this section, we describe the adversary model, the security assumptions, and the target properties that we aim to achieve.

3.1 Adversary Model

Honest-but-Curious IdP: The IdP in ARPSSO is non-malicious and follows the protocol honestly. However, it is interested in tracking the user's login events across multiple protocol sessions.

Colluding RPs: A set of RPs that are controlled by an adversary. Each RP has limited knowledge of the user identity, while the adversary attempts to link the user based on its knowledge.

Malicious Users and/or Malicious RPs: An adversary can control users, RPs, or a combination of both to participate in the SSO process. These malicious entities aim to impersonate a victim user while logging into honest RPs.

3.2 Assumptions

We assume that appropriate protection measures are implemented at the network level to prevent IdPs from tracking RP identities (e.g., the IP address) across the network, such as Tor [15] or the more efficient Oblivious HTTP (OHTTP) [42]. In addition, we assume that users do not grant their personally identifiable information (PII), such as phone numbers, e-mail addresses, etc., to RPs via ID tokens.

3.3 Target Properties

We aim to achieve the following properties in the ARPSSO.

The first is the privacy properties, including user login untraceability and user identity unlinkability. The former requires RPs to remain anonymous to IdPs so that curious IdPs cannot determine which RP the user is requesting to log into. The latter requires that user identifiers in different RPs are unique and unlinkable, preventing colluding RPs from identifying the target user.

The second is the security properties, including RP anonymous authentication unforgeability, i.e., the attackers can not impersonate an honest RP to pass the IdP-to-RP authentication and obtain the ID token; user identity uniqueness and consistency, i.e., each user has a unique identity in the RP which remains consistent across multiple logins to the same RP; in-browser data forwarding trustworthiness, i.e., the *code* can be securely forwarded to the specified RP via the user agent.

The last is a transparent user experience. Users can complete the privacy-preserving SSO process using conventional authentication mechanisms in standard browsers, without installing additional plug-ins or software. They can also provide consent based on the verified RP's identity, ensuring a consistent experience similar to their daily activities.

4 Design

ARPSSO is a privacy-preserving SSO scheme based on OIDC code flow, which implements the properties described in Sect. 3.3. Figure 2(a) shows the overview of ARPSSO. It accomplishes privacy preservation through three mechanisms, namely the RP anonymous authentication, the user identity mix-up, and the trusted in-browser data forwarding, highlighted in red, blue, and green, respectively. First, to achieve user login untraceability, the IdP-to-RP authentication in step ⑨ needs to be anonymous. Therefore, we propose an RP anonymous authentication mechanism, which generates an anonymous identifier (*acid*) for the RP in each SSO session and achieves RP authentication based on *acid*. Second, in order to achieve untraceability and unlinkability, it needs to ensure that the IdP does not know the RP's real identity for this login (i.e., *cid*) and the RP does not know the user's real identity (i.e., *uid*) throughout the SSO process. This poses a challenge to generate a unique, consistent and unlinkable identity for each user in the RP. To address this problem, we propose a user identity mix-up mechanism based on two-party secure computation. Finally, we provide a user-transparent trusted in-browser data forwarding mechanism. It completes the user agent operations in steps ③, ④ and ⑦ (i.e., presenting the RP's real identity for user consent after verifying the *acid* and forwarding the IdP-generated *code* securely to the specified RP) without disclosing the RP's real identity to the IdP via a trusted third-party script called data verify forwarder (DVF). Details are as follows.

RP Anonymous Authentication. We achieve RP anonymous authentication based on the anonymous credential scheme described in Sect. 2.3. In ARPSSO,

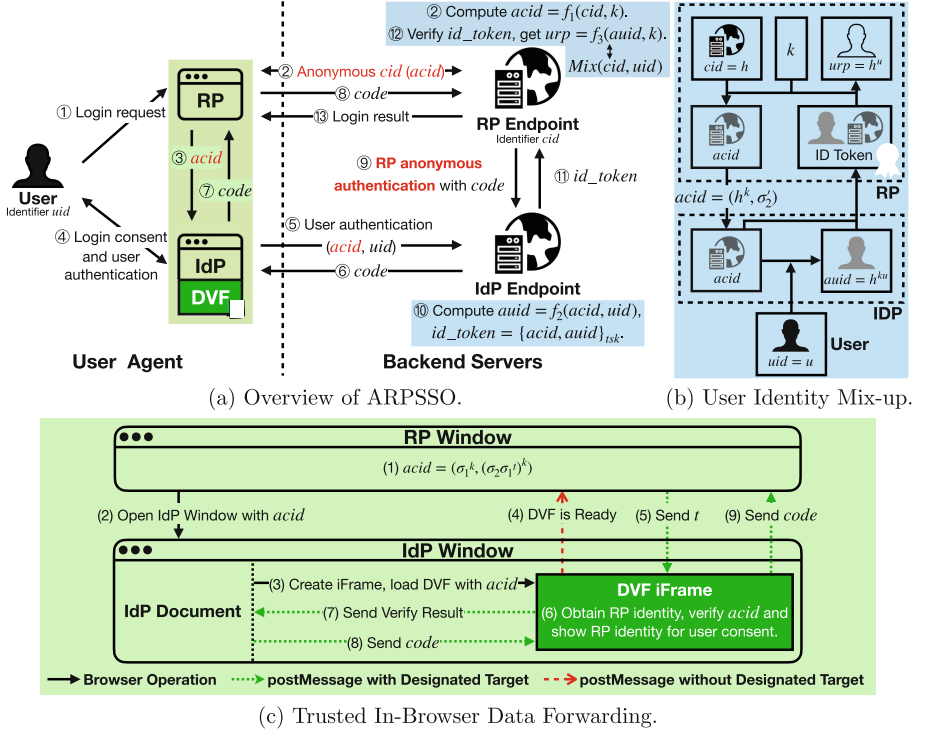


Fig. 2. The workflow of ARPSO. (a) shows the overview of ARPSO, where the user identity mix-up marked in blue is detailed in (b), and the trusted in-browser data forwarding marked in green is detailed in (c). (Color figure online)

the IdP acts as both the issuer and the verifier, while the RP acts as the prover. Before the SSO process shown in Fig. 2(a), the RP requests an anonymous credential from the IdP by submitting a secret $\mathcal{S}$ (similar to a private key) and the RP domain $\mathcal{D}$ for issuance. This process is implemented through the blind issue protocol **CredIssue** to keep the $\mathcal{S}$ known only to the RP. In the **BlindSign** algorithm, the IdP chooses a unique random number $r \xleftarrow{\$} \mathbb{G}_1$ to compute $h = g^r$, which serves as the unique RP identifier (i.e., $cid = h$) and is encoded in the first part of the RP anonymous credential signature $\sigma = (\sigma_1, \sigma_2)$, i.e., $\sigma_1 = h = cid$. During the SSO process, the RP generates random number $k, t \xleftarrow{\$} \mathbb{Z}_p$ and computes a randomized signature $\sigma' = (\sigma_1^k, (\sigma_2 \sigma_1^t)^k)$ based on the signature σ . It then uses this randomized signature as the RP anonymous identifier $acid = \sigma'$ for this SSO session and sends it to the IdP in step ②. After receiving the *code*, the RP generates a credential presentation using the **CredPresent** algorithm and sends it to the IdP in step ⑨. This credential presentation contains a NIZK proof to prove the correctness of the randomized signature σ' without revealing the $\mathcal{S}$ which is only known to the RP, and $\mathcal{D}$ which is supposed to be

unknown to the IdP. Then, the IdP anonymously authenticates the RP using the **CredPresentVerify** algorithm.

User Identity Mix-Up. There is a unique RP identifier in the IdP (i.e., $cid = h$). Suppose the unique user identifier in the IdP is $uid = u, u \leftarrow \mathbb{Z}_p$, we can construct the user identifier in the RP as $urp = cid^{uid} = h^u$, which is unique for each user in each different RP and unlinkable according to the Decision Diffie-Hellman (DDH) [11] assumption. To allow the RP to compute the identifier urp without disclosing to the IdP the RP's real identity for this login (i.e., cid) and without knowing the user identifier obtained by the IdP (i.e., uid), we introduce a user identity mix-up mechanism as shown in Fig. 2(b). During the SSO process, the RP provides an anonymous identifier to the IdP, i.e., $acid = (\sigma'_1, \sigma'_2) = (h^k, (\sigma_2 h^t)^k)$ in step ②. In step ⑤, the IdP authenticates the user and obtains the user identifier $uid = u$. In step ⑩, the IdP computes the anonymous user identifier $auid = acid^{uid} = h^{ku}$ and encloses it in the ID token. It then sends the token to the RP in step ⑪. When the RP receives the ID token, it obtains $auid$ and calculates $urp = auid^{1/k} = h^u$ in step ⑫. Note that the RP chooses the random factor k , making it easy to compute $1/k$ in $\mathbb{Z}_p$.

Trusted In-Browser Data Forwarding. We introduce the DVF to perform the user agent operations in steps ③, ④ and ⑦ of Fig. 2(a) without revealing the RP identity to the IdP. The DVF is a trusted third-party script with the workflow shown in Fig. 2(c), which consists of 9 steps: (1) When a user accesses the RP service, the RP front-end window obtains the RP anonymous identifier (i.e., $acid$) for this session from the RP back-end server. (2) The RP window opens a new IdP window to perform user authentication and passes $acid$ to the IdP window. (3) In the IdP window, an **iframe** is created to load the DVF script, and $acid$ is further passed to DVF. We use an **iframe** instead of loading the script directly in the IdP document to ensure that the DVF and IdP documents work in two different origins. (4)–(5) The DVF interacts with the RP window via **postMessage** to get the random number t in $acid$. The interaction is initiated by the DVF. Since the RP origin (i.e., domain $\mathcal{D}$) is not known to the DVF, the “ready” **postMessage** in step (4) does not specify the designated target (i.e., broadcast). Upon receipt of the “ready” message from the DVF, the RP verifies the origin of the message sender and responds t only to the trusted DVF script domain (specifying the DVF origin as the designated target in the **postMessage** function). This message event will also inform DVF of the RP domain $\mathcal{D}$. (6) DVF verifies the correspondence between $acid$ and $\mathcal{D}$, and presents the RP identity to the user for consent if the correspondence holds. (7) The DVF notifies the IdP of the RP verification result. The IdP performs user authentication and generates the *code*. (8)–(9) The DVF forwards the IdP-generated *code* to the verified RP. During this process, the DVF acts as an intermediate agent to verify the RP's identity and securely forward the data between the IdP and the RP.

In step (6), the DVF needs to verify the correspondence between $acid$ and the RP domain $\mathcal{D}$. Directly using the anonymous credential will involve creating and verifying NIZK proofs because the credential contains the secret $\mathcal{S}$, known only to the RP. This process will increase system complexity and performance

overhead. However, we can avoid verifying NIZK proofs in the user agent via a simple optimization. Specifically, we split the RP anonymous credential into a pair of credentials, i.e., an identity credential $\mathcal{IC}$ containing the RP domain $\mathcal{D}$ and the RP identifier cid , but excluding the secret $\mathcal{S}$; and an authentication credential $\mathcal{ATC}$ containing the RP domain $\mathcal{D}$ and the secret $\mathcal{S}$. $\mathcal{ATC}$ and $\mathcal{IC}$ are associated through the RP domain $\mathcal{D}$. The random number of the signature in $\mathcal{IC}$ is used as the RP identifier cid . The RP authentication in step ⑨ can be done cooperatively by $\mathcal{IC}$ and $\mathcal{ATC}$, while the RP verification in the user agent can be done independently through $\mathcal{IC}$. Since $\mathcal{IC}$ does not contain the secret $\mathcal{S}$, this design allows DVF to verify the authenticity of $acid$ without any NIZK proof. Specifically, the RP's anonymous identifier $acid = (\sigma_1^k, (\sigma_2\sigma_1^t)^k)$ is the randomization of the signature $\sigma = (\sigma_1, \sigma_2)$ in $\mathcal{IC}$. The RP obtains $acid$ in step (3), and the domain $\mathcal{D}$ and the random number t in step (5). It can then extract σ_1^k from $acid$ and compute $(\sigma_1^k)^{-t}$, and verify $acid$ using equation $\mathbf{PSVerify}((\sigma_1^k, (\sigma_2\sigma_1^t)^k)/(\sigma_1^k)^t, \mathcal{D}, vk) \rightarrow b$.

5 ARPSO

The main workflow of ARPSO consists of three stages.

5.1 Initialization

In this stage, the IdP generates the global parameters pp and the signing key pairs (tsk, tvk, csk, cvk) , and publishes pp and public keys (cvk, tvk) . (tsk, tvk) is the token key pair used to issue and verify ID tokens. It is generated based on JSON Web Key (JWK) [32] since the standard ID token is used in ARPSO. (csk, cvk) is the credential key pair used to issue and verify RP anonymous credentials $\mathcal{IC}$ and $\mathcal{ATC}$, as shown below.

The IdP chooses a secure parameter ℓ and an type-3 bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, where g_1, g_2 are the generators of group $\mathbb{G}_1, \mathbb{G}_2$ respectively. The public parameters are denoted as $pp \leftarrow (e, \mathbb{G}_1, \mathbb{G}_2, g_1, g_2)$. With the pp , the IdP chooses $x, y, \bar{x}, \bar{y}_1, \bar{y}_2 \xleftarrow{\$} \mathbb{Z}_p$, and computes $X = g_1^x, Y = g_1^y, \hat{X} = g_2^x, \hat{Y} = g_2^y, \bar{X} = g_1^{\bar{x}}, \bar{Y}_1 = g_1^{\bar{y}_1}, \bar{Y}_2 = g_1^{\bar{y}_2}, \hat{\bar{X}} = g_2^{\bar{x}}, \hat{\bar{Y}}_1 = g_2^{\bar{y}_1}$ and $\hat{\bar{Y}}_2 = g_2^{\bar{y}_2}$. The credential key pair (csk, cvk) is set as $csk \leftarrow (x, y, X, \bar{x}, \bar{y}_1, \bar{y}_2, \bar{X})$, $cvk \leftarrow (Y, \hat{X}, \hat{Y}, \bar{Y}_1, \bar{Y}_2, \hat{\bar{X}}, \hat{\bar{Y}}_1, \hat{\bar{Y}}_2)$. The values of $x, y, X, Y, \hat{X}, \hat{Y}$ are used for $\mathcal{IC}$ and others for $\mathcal{ATC}$.

5.2 Registration

In this stage, users and RPs register with the IdP, which assigns unique identifiers to the users and RPs and issues a pair of anonymous credentials to each RP. The IdP verifies the user's identity and provides a unique identifier $uid = u, u \leftarrow \mathbb{Z}_p$ to the user. The RP registration is accomplished via the interactive protocol shown in Fig. 3, through which the IdP issues two credentials

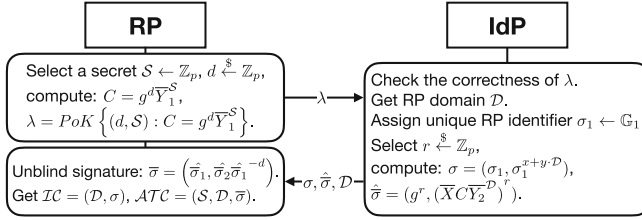


Fig. 3. RP Registration Protocol.

to the RP, namely $\mathcal{IC}$ and $\mathcal{ATC}$. Briefly, each RP has two attributes, i.e., the RP secret S and the RP domain D . The secret S is generated randomly and stored by the RP secretly. The domain D has two states: a public state used for implementing RP verification in the user agent through **PSVerify**, and a hidden state used for achieving RP anonymous authentication in the IdP backend through **CredPresentVerify**. The $\mathcal{IC}$ contains D and a signature σ generated via **PSSign**. The signature σ contains a unique RP identifier assigned by the IdP, i.e., $\sigma_1 = cid$. $\mathcal{ATC}$ contains S , D and a signature $\bar{\sigma}$ generated via **CredIssue**.

5.3 Single Sign-on

In this stage, users perform untraceable and unlinkable SSO based on OIDC code flow. The SSO flow is illustrated in Fig. 4, which contains three parts. Steps ① to ⑦, the RP backend randomizes the signature of $\mathcal{IC}$ to generate an anonymous identifier $acid$ for the SSO session, and sends it to the IdP backend. Steps ⑧ to ⑱, the IdP document creates and loads the DVF document to perform the trust in-browser data forwarding, which displays the RP identity corresponding to $acid$ to the user, obtains the user's consent, and forward the *code* generate by the IdP backend after the user authentication. Steps ⑲ to ㉑, the RP backend exchanges the ID token from the IdP backend with *code* and a NIZK proof π , which is used for RP anonymous authentication to prove the RP's identity authenticity corresponding to the $acid$. Then, the RP extracts the unlinkable user identifier from the ID token through the user-identity mixup mechanism.

Referer Header Privacy. Instead of opening IdPdoc directly in RPdoc, we introduce an intermediate document (i.e., RPRedirdoc) after the step ②. This is because the browsers add a referer header [5] to tell the backend server where the request came from, when opening a new window or redirecting to a new document. Therefore, opening IdPdoc directly in RPdoc would expose the RP's domain to the IdP. We can solve this problem by setting the “norereferrer” attribute. However, directly setting “norereferrer” in step ② will cause the popup window to lose the RP window handle, making it impossible to communicate between the two windows (e.g., step ⑩ and ⑪). For this reason, we introduce an intermediate document RPRedirdoc and set “norereferrer” for redirecting to IdPdoc in the popup window (i.e., step ⑤).

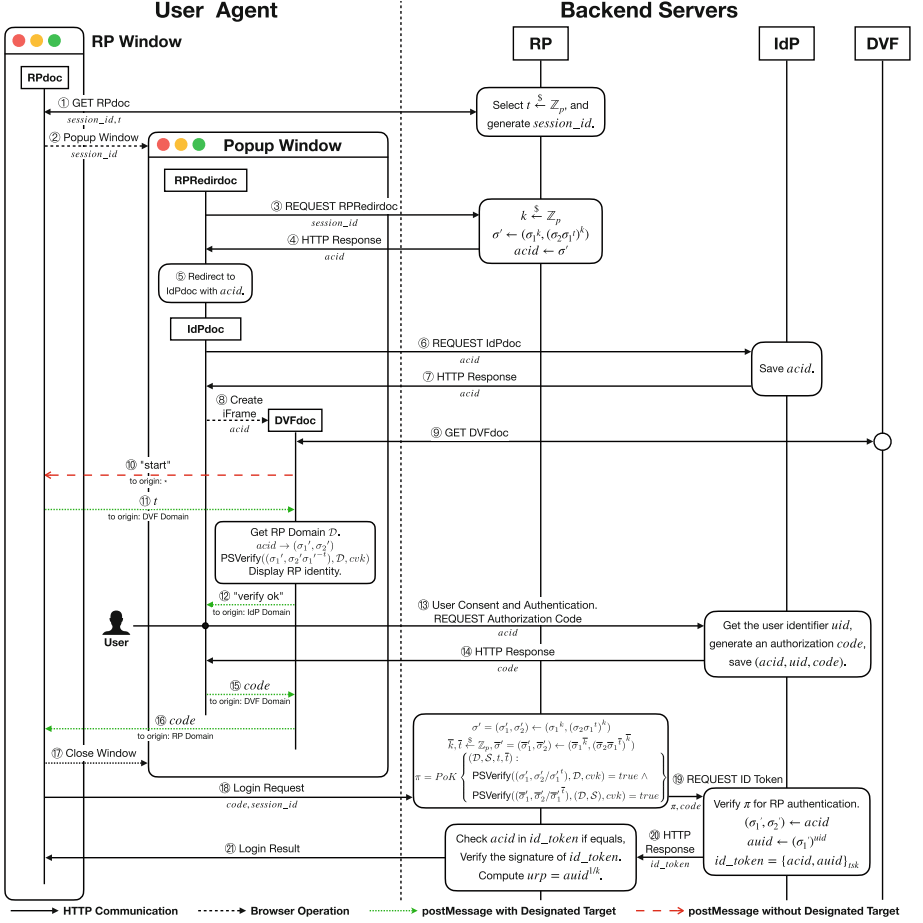


Fig. 4. SSO Flow of ARPSO.

DVF Document Trustworthiness. Since DVFdoc is a fixed piece of code that implements a well-defined function (i.e., verifying the RP's identity and securely forwarding the IdP-generated *code* to the verified RP), its trustworthiness is critical to both the RP and the IdP. For example, the RP needs to ensure that it does not expose the RP's identity to the IdP, and the IdP needs to ensure that it only sends *code* to the verified RP. We provide multiple DVF deployment methods to ensure the DVF is trustworthy, as discussed in Sect. 9.

6 Analysis

6.1 Security of ARPSO

The security properties of the standard OIDC protocol have been defined and proven in the Dolev-Yao (DY) model [17, 20, 26], and they fall into two main

categories, i.e., authentication property and session integrity [17]. The ARPSSO aims to introduce privacy preservation to the standard OIDC protocol while maintaining these security properties. Overall, we introduce three mechanisms, i.e., RP anonymous authentication, user identity mix-up, and trusted in-browser data forwarding, whose main impact lies in the authentication property, while the other security properties are consistent with the standard OIDC framework. Specifically, ARPSSO is able to employ the same countermeasures against attacks such as relay attack and CSRF as OIDC [38], ensuring authentication session integrity. In addition, ARPSSO uses the standard ID token, which provides integrity and authenticity. Next, we specifically analyze the potential impact of the introduced mechanisms on the authentication property.

To begin, we examine the implementation of the authentication property by the standard OIDC code flow protocol. This property ensures that each user can log in to the intended RP and operate under their unique identity within that RP. The following mechanisms have been provided to fulfill this requirement.

- **RP Verification in User Agent:** In OIDC, the `id.token` in the implicit flow and the `code` in the code flow should only be forwarded to the corresponding RP through the user agent (usually the browser). To achieve this, the standard protocols enforce real-name RP verification before forwarding. Specifically, the IdP maps RP identifiers to RP domains and requires user agents to redirect the ID token or `code` to the RP endpoint. The RP endpoint URL is provided in HTTPS form to enable the user agent to verify the correspondence between the RP domain and the RP via a digital certificate.
- **RP Authentication:** To address token leakage in the implicit flow, the code flow passes the ID token directly and only between the IdP and the RP backend server, and the IdP authenticates the RP before sending the token. This ensures that confidential ID tokens are only passed to the RP with which the user intends to log in.
- **Designated ID Token:** The ID token in the OIDC is an attestation of an authenticated user identity in a specific RP. It should contain the user identity and the RP identity and be signed by the IdP. This ensures that the ID token can only be used to log in as the specified user to the specified RP. In addition, the user identity at the RP should be unique and consistent, ensuring that each user has a unique identity and remains consistent across multiple logins to the same RP.

Definition 1 formalized the authentication property for ARPSSO, similar to [17]. In short, it means the adversary should be unable to log in under a victim user's account at an honest RP using an honest IdP.

Definition 1 (Authentication Property of ARPSSO). *The authentication property in ARPSSO is satisfied when the following condition is met: For every honest RP rp , honest IdP idp , honest DVF dvf , honest user u , and their browser b . If u initiates the protocol on b to log in to rp via idp , and gives consent to do so with both idp and dvf , then at the end of the SSO process, only rp receives an*

ID token from idp (with attestations for u and rp) and returns the login status to u via b .

We introduce three mechanisms in ARPSSO to address privacy issues in SSO. To achieve the above authentication property, they need to fulfill the following three essential requirements. First, the RP anonymous authentication should be unforgeable such that no adversary can successfully pass the authentication, ensuring that the ID token only reaches its designated RP (RP anonymous authentication unforgeability). Second, through the user identity mix-up algorithm, the designated RP can extract from the ID token a specific user identity that is the user previously authenticated to the IdP (user identity uniqueness and consistency). Third, since the *code* generated by the IdP is intrinsically tied to the authenticated user, it must be kept confidential from all entities outside this SSO session. Consequently, the *code* must be securely forwarded to the target RP within the browser through the DVF (in-browser data forwarding trustworthiness).

Theorem 1 (RP Anonymous Authentication Unforgeability). *For a given honest RP's anonymous identifier $acid$, it is infeasible for any adversary to create a valid anonymous credential presentation to pass the authentication.*

Theorem 2 (User Identity Uniqueness and Consistency). *For any user who initiated the SSO flow to log in to the RP multiple times, the RP can extract a unique and consistent user identifier urp from the ID token, and the urp represents the user in the RP.*

Theorem 3 (In-Browser Data Forwarding Trustworthiness). *For every honest RP rp , honest IdP idp , honest DVF dvf , honest user u , and user-agent b . Whenever u logs into rp with idp and dvf on b , as a result of running the protocol, dvf guarantees that the data is sent only to rp , i.e., any malicious entity other than rp , idp , dvf , or u cannot know the transmitted data.*

6.2 Privacy of ARPSSO

Theorem 4 (User Sign-On Untraceability). *For a probabilistic polynomial time (PPT) adversary $\mathcal{A}$ controlling an IdP, it is infeasible to distinguish whether a user who has initiated two different SSO sessions is trying to sign on to the same RP or not.*

Theorem 5 (User Identity Unlinkability). *For a PPT adversary $\mathcal{A}$ controlling two colluding RPs, it is infeasible to distinguish whether two users in two different RPs correspond to the same user within the IdP.*

Due to space limitations, detailed proofs of Theorem 1, 2, 3, 4 and 5 are available in our full version [1].

7 Evaluation

We implement a prototype system [1] for ARPSSO and evaluate the performance. The BLS12-381 curve [12] is used for the PS signature due to its strong security and computational efficiency [9]. We develop RP and IdP backend in Go, and implement the PS signature and NIZK protocol for RP anonymous credential using Cloudflare’s CIRCL library [16]. The front-end DVF script is implemented in Javascript using the Nobel-bls12-381 package [37]. We deploy the RP and IdP backend on a virtual machine (with a 2.7GHz single-core virtual CPU, 50GB SSD, and 8GB RAM), and use Chrome on a Google Pixel 8 and a MacBook Pro (with an Intel Core i7-9750H CPU, 512GB SSD, and 16GB of RAM) to execute client-side codes including the DVF script. Table 1 shows the overhead of ARPSSO compared to standard OIDC, and all values in the table are averaged over 1000 repetitions. The overall overhead is about 200ms. Client-side operations introduce the most overhead due to the inefficiency of executing JavaScript code in the browser, which could be optimized with WebAssembly (WASM) [45].

Table 1. Overhead of ARPSSO Compared to Standard OIDC

Protocol Stage	Operation	Time (in ms)
Initialization	Key generation	6.6
RP Registration	RP generates commitment	2.6
	IdP generates blind signature	2.6
	RP unblinds signature	7.9
SSO	RP generates anonymous id <i>acid</i>	1.3
	DVF verifies <i>acid</i> in browser	107.1/123.1 *
	RP generates a proof for <i>acid</i>	68.8
	IdP verifies the proof	24.3

*Overhead on two client devices, Google Pixel 8 and MacBook Pro.

8 Related Work

In this section, we describe existing privacy-preserving SSO schemes, which can be categorized into two types, i.e., asynchronous authentication privacy-preserving SSO and synchronous authentication privacy-preserving SSO.

Asynchronous Authentication Privacy-Preserving SSO. Based on anonymous credentials, many works [7, 8, 27, 28, 36, 40, 43, 45] achieve untraceability by constructing an asynchronous authentication model [45]. In this model, users obtain anonymous credentials from the IdP and use these anonymous credentials to authenticate directly to the RP during the SSO process. Since the IdP is not required to participate in the SSO process, there is no traceability issue. In addition, users are allowed to generate unlinkable identifiers for different RPs

based on the anonymous credential to achieve unlinkability. However, managing the credentials issued by the IdP can be a burden on users, resulting in a poor user experience. In addition, this model increases the burden on the RP to authenticate the user, e.g., to verify whether the anonymous credentials have been revoked.

Synchronous Authentication Privacy-Preserving SSO. In most public SSO systems, the IdP performs user authentication centrally and synchronously when users access RP services. This is the standard SSO model and can reduce

Table 2. Comparison of Synchronous Authentication Privacy-preserving SSO solutions.

Solution	SSO Feature			Privacy-Preserving Feature		User-Friendly Feature	
	Unique and Consistent User ID at RP	User Authentication Only to IdP	Compatible with OIDC Code Flow	User Sign-on Untraceability	User Identity Unlinkability	No Special Demands for User Agent	No Special Demands for User Authn
BrowserID [18]	✓	⦿	✗	✓	✗	✓	✓
SPRESSO [19]	✓	✓	✗	✓	✗	✓	✓
PRIMA [10]	✓	✗	✗	✓	✗	✗	✗
PPID [23]	✓	✓	✓	✗	✓	✓	✓
Apple [30]	✓	✓	✓	✗	✓	✓	✓
PseudoID [14]	✓	✗	✗	⦿	✓	✓	✗
UnlimitID [31]	✓	✗	✓	⦿	✓	✗	✗
POIDC [26]	✓	✓	✗	✓	✓	⦿	✓
UPPRESSO [25]	✓	✓	✗	✓	✓	⦿	✓
AIF [33]	✓	✓	✗	✓	✗	⦿	✓
UP-SSO [24]	✓	✓	✗	✓	✓	✗	✓
MISO [44]	✓	✓	⦿	✓	✓	✓	✓
ARPSO	✓	✓	✓	✓	✓	✓	✓

- ⦿ denotes the feature is partially supported.
- 1 The user can use standard browsers as the user agent without installing a specialized client or browser plug-ins.
 - 2 The user can authenticate to the IdP through conventional authentication methods (e.g., username-password) or enhanced authentication methods (e.g., 2FA, FIDO2).
 - 3 In BrowserID, the user generates a private key to sign an identity attestation, and the RP must verify the signature.
 - 4 MISO is not a standard SSO model. It introduces the Mixer as a bridge between the IdP and the RP. When exchanging the ID token using the *code*, the IdP authenticates the Mixer instead of the RP to achieve untraceability. Therefore, it is not a strict OIDC code flow mode.
 - 5 In PseudoID and UnlimitID, the IdP can obtain the user’s identifier in the RP, so it can track the user’s login behaviors in the same RP.
 - 6 AIF and PIODC do not provide implementations. According to our analysis, they both require a browser plug-in or a dedicated user agent software to perform data forwarding between the RP and the IdP. UPPRESSO provides two implementations, one requiring a browser plug-in and the other not. However, in the non-plug-in version, the script responsible for data forwarding is provided and controlled by the IdP, which would violate the “IdP is curious” model.

the cost of identity management for RP services. There are also a range of SSO solutions [10, 14, 18, 19, 23–26, 30, 31, 33, 44] that achieve privacy-preserving while retaining centralized synchronous authentication. ARPSSO belongs to this category and is the first privacy-preserving SSO solution compatible with the OIDC code flow. Table 2 provides a detailed comparison of these solutions.

BrowserID is the first privacy-preserving SSO solution to implement user sign-on untraceability, and a privacy vulnerability has been discovered in its implementation [18]. SPRESSO [19] and PRIMA [10] achieve untraceability by randomizing RP identifiers to IdP. They use self-designed SSO processes that do not include RP authentication and are, therefore, incompatible with the OIDC code flow. In addition, none of these solutions provide user identity unlinkability. The main idea to achieve unlinkability is user identifier anonymization, i.e., ensuring that a user's identifier is distinct in different RPs. NIST's Pairwise Pseudonymous Identifiers (PPIDs) [23] and Sign in with Apple [30] anonymize user identifiers through the IdP, and therefore suffer from the traceability issue. PseudoID [14] and UnlimitID [31] anonymize user identifiers at the user side, enabling users to authenticate to the IdP anonymously using blind signature [14] or anonymous credentials [31]. However, these schemes are restricted to protocol-specific user authentication credentials, limiting their practical usability.

POIDC [26] and UPPRESSO [25] achieve untraceability and unlinkability through RP identifier randomization. However, both POIDC and UPPRESSO are designed based on the OIDC implicit flow and lack a method for RP authentication based on the randomized RP identifier. A recent paper [33] points out the importance of RP authentication and constructs an Authenticated Implicit Flow (AIF) that supports untraceability, but it does not support unlinkability and is not compatible with OIDC code flow.

UP-SSO [24] and MISO [44] are privacy-preserving SSO schemes based on Trusted Execution Environment (TEE). They use a TEE component to mediate communication between the RP and the IdP, including randomizing the RP identifier to the IdP and generating a consistent, unlinkable user identifier in the RP. UP-SSO deploys the TEE component at the user side, imposing specific requirements on the user's device. MISO deploys the TEE component (called Mixer) in the cloud as an intermediary between the RP and the IdP. This solution deviates from the standard SSO model, doubling the protocol latency, introducing a single point of failure risk, and increasing deployment costs.

9 Discussion

Compatibility and Extendability. Since the implicit flow is a simplified version of the code flow, ARPSSO naturally supports untraceability and unlinkability for the OIDC implicit flow, except that the DVF shall securely forward the ID token instead of the *code* in the user agent. In addition, the RP anonymous credential and user identity mix-up we introduce can be used as building blocks to achieve untraceability and unlinkability simultaneously. Further research could build on this foundation to create customized IdP-to-RP Identity and Access

Management (IAM), e.g., to support revocation, tracking, and de-anonymization of RP anonymous credential to meet regulatory requirements.

DVF Deployment. Since the DVF script is static, it can be easily deployed by hosting it on a file server. To establish a trustworthy DVF, as mentioned in Sect. 5.3, deploying it on a trusted third-party server is an option, although it would increase the deployment costs. Another option is to deploy it through a static site hosting service and verify its integrity during the SSO process. GitHub Pages [2] offers this service at no cost and supports custom domain configuration (providing a unique domain name for the `postMessage` mechanism). It also allows for public inspection of the files stored on it. We can verify the integrity of the DVF script based on the Subresource Integrity (SRI) Web technology currently being developed by the W3C [6]. Specifically, SRI allows a document to create an `iframe` or `link` with an attribute integrity that takes a hash value. Initially, the RP and the IdP review the script file and record its hash value. During SSO, the RP loads the DVF script as a `link` and uses SRI to verify the script's integrity before redirecting the user to the IdP. Similarly, the IdP verifies the script's integrity before loading the DVF `iframe`.

10 Conclusion

In this paper, we introduce a privacy-preserving SSO scheme based on OIDC code flow. It achieves user sign-on untraceability by designing an RP anonymous credential that allows RP to generate anonymous identifiers for each SSO session. These RP anonymous identifiers are used in a two-party secure computation to ensure user identity unlinkability and enable RP anonymous authentication to comply with OIDC code flow. We also introduce the DVF for secure data forwarding between the IdP and the RP, accompanied by user login consent. Our security and privacy analysis shows that our system preserves the security of the conventional OIDC system while satisfying two essential privacy goals. Also, the prototype of ARPSO shows that the overhead it introduces is reasonable.

Acknowledgments. This work is supported by the National Key R&D Program of China (Grant No. 2022YFB3103303).

References

1. ARPSO/ARPSO. <https://github.com/ARPSO/ARPSO>
2. GitHub Pages. <https://pages.github.com/>
3. Implicit Grant Flow for Client-Side Apps - Yahoo Developer Network. https://developer.yahoo.com/oauth2/guide/flows_implicitgrant/
4. Login Provider Technologies Market Share and Web Usage Statistics. <https://www.similartech.com/categories/login-provider>
5. Referrer Policy. <https://www.w3.org/TR/referrer-policy/>
6. Subresource Integrity (2016). <https://www.w3.org/TR/SRI/#verification-of-html-document-subresources>

7. Alpár, G., Van Den Broek, F., Hampiholi, B., Jacobs, B., Lueks, W., Ringers, S.: IRMA: practical, decentralized and privacy-friendly identity management using smartphones. In: 10th Workshop on Hot Topics in Privacy Enhancing Technologies (HotPETs 2017), pp. 1–2 (2017)
8. Androulaki, E., et al.: Hyperledger fabric: a distributed operating system for permissioned blockchains. In: Proceedings of the Thirteenth EuroSys Conference, p. 30. ACM (2018)
9. Aranha, D.F., Fuentes-Castañeda, L., Knapp, E., Menezes, A., Rodríguez-Henríquez, F.: Implementing pairings at the 192-bit security level. In: Abdalla, M., Lange, T. (eds.) Pairing 2012. LNCS, vol. 7708, pp. 177–195. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-36334-4_11
10. Asghar, M.R., Backes, M., Simeonovski, M.: PRIMA: privacy-preserving identity and access management at internet-scale. In: 2018 IEEE International Conference on Communications (ICC), pp. 1–6. IEEE (2018)
11. Boneh, D.: The decision Diffie-Hellman problem. In: Buhler, J.P. (ed.) ANTS 1998. LNCS, vol. 1423, pp. 48–63. Springer, Heidelberg (1998). <https://doi.org/10.1007/BFb0054851>
12. Bowe, S.: BLS12-381: new zk-SNARK elliptic curve construction. <https://electriccoin.co/blog/new-snark-curve/>
13. Damgård, I.: On Σ -protocols. Lecture Notes, University of Aarhus, Department for Computer Science, p. 84 (2002)
14. Dey, A., Weis, S.: PseudoID: enhancing privacy in federated login (2010)
15. Dingledine, R., Mathewson, N., Syverson, P.: Tor: the second-generation onion router. Technical report, Naval Research Lab Washington DC (2004)
16. Faz-Hernández, A., Kwiatkowski, K.: Introducing CIRCL: an advanced cryptographic library. <https://github.com/cloudflare/circl/>
17. Fett, D., Küsters, R., Schmitz, G.: The web SSO standard OpenID connect: in-depth formal security analysis and security guidelines. In: 2017 IEEE 30th Computer Security Foundations Symposium (CSF), pp. 189–202. IEEE (2017)
18. Fett, D., Küsters, R., Schmitz, G.: Analyzing the BrowserID SSO system with primary identity providers using an expressive model of the web. In: Pernul, G., Ryan, P.Y.A., Weippl, E. (eds.) ESORICS 2015. LNCS, vol. 9326, pp. 43–65. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-24174-6_3
19. Fett, D., Küsters, R., Schmitz, G.: SPRESSO: a secure, privacy-respecting single sign-on system for the web. In: Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, CCS 2015, pp. 1358–1369. Association for Computing Machinery (2015)
20. Fett, D., Küsters, R., Schmitz, G.: A comprehensive formal security analysis of OAuth 2.0. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, CCS 2016, pp. 1204–1215. Association for Computing Machinery (2016)
21. Fiat, A., Shamir, A.: How to prove yourself: practical solutions to identification and signature problems. In: Odlyzko, A.M. (ed.) CRYPTO 1986. LNCS, vol. 263, pp. 186–194. Springer, Heidelberg (1987). https://doi.org/10.1007/3-540-47721-7_12
22. Galbraith, S.D., Paterson, K.G., Smart, N.P.: Pairings for cryptographers. Discret. Appl. Math. **156**(16), 3113–3121 (2008)
23. Grassi, P., et al.: Digital identity guidelines: federation and assertions. Technical report, NIST Special Publication (SP) 800-63C, National Institute of Standards and Technology (2020)

24. Guo, C., Lang, F., Wang, Q., Lin, J.: UP-SSO: enhancing the user privacy of SSO by integrating PPID and SGX. In: 2021 International Conference on Advanced Computing and Endogenous Security, pp. 01–05 (2022)
25. Guo, C., et al.: UPPRESSO: untraceable and unlinkable privacy-PREserving single sign-on services. arXiv preprint [arXiv:2110.10396](https://arxiv.org/abs/2110.10396) (2021)
26. Hammann, S., Sasse, R., Basin, D.: Privacy-preserving OpenID connect. In: Proceedings of the 15th ACM Asia Conference on Computer and Communications Security, ASIA CCS 2020, pp. 277–289. Association for Computing Machinery (2020)
27. Han, J., Chen, L., Schneider, S., Treharne, H., Wesemeyer, S.: Anonymous single-sign-on for n designated services with traceability. In: Lopez, J., Zhou, J., Soriano, M. (eds.) ESORICS 2018. LNCS, vol. 11098, pp. 470–490. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-99073-6_23
28. Han, J., Chen, L., Schneider, S., Treharne, H., Wesemeyer, S., Wilson, N.: Anonymous single sign-on with proxy re-verification. IEEE Trans. Inf. Forensics Secur. **15**, 223–236 (2020)
29. Hardt, D.: The OAuth 2.0 authorization framework. RFC 6749 (2012)
30. Inc, A.: Sign in with Apple. <https://developer.apple.com/sign-in-with-apple/>
31. Isaakidis, M., Halpin, H., Danezis, G.: UnlimitID: privacy-preserving federated identity management using algebraic MACs. In: Proceedings of the 2016 ACM on Workshop on Privacy in the Electronic Society, WPES 2016, pp. 139–142. Association for Computing Machinery (2016)
32. Jones, M.B.: JSON web key (JWK). RFC 7517 (2015)
33. Kroschewski, M., Lehmann, A.: Save the implicit flow? Enabling Privacy-preserving RP authentication in OpenID connect. Proc. Priv. Enhanc. Technol. (2023)
34. Lodderstedt, T., McGloin, M., Hunt, P.: RFC 6819: OAuth 2.0 threat model and security considerations. Internet Eng. Task Force (IETF) 1–71 (2013)
35. Lodderstedt, T., Bradley, J., Labunets, A., Fett, D.: OAuth 2.0 security best current practice. Internet-Draft draft-ietf-oauth-security-topics-23, Internet Engineering Task Force/Internet Engineering Task Force (2023)
36. Maganis, G., Shi, E., Chen, H., Song, D.: Opaak: using mobile phones to limit anonymous identities online. In: Proceedings of the 10th International Conference on Mobile Systems, Applications, and Services, MobiSys 2012, pp. 295–308. Association for Computing Machinery (2012)
37. Miller, P.: Noble-BLS12-381. <https://github.com/paulmillr/noble-bls12-381>
38. Nat, S., John, B., Mike, J., de Medeiros, B., Mortimore, C.: OpenID connect core 1.0 incorporating errata set 1. The OpenID Foundation, specification (2014)
39. nickludwig: OAuth 2.0 implicit grant flow - The Microsoft identity platform - Microsoft Entra (2022). <https://learn.microsoft.com/en-us/azure/active-directory/develop/v2-oauth2-implicit-grant-flow>
40. Paquin, C.: U-prove technology overview v1. 1. Microsoft Corp. Draft Revision 1 (2011)
41. Pointcheval, D., Sanders, O.: Short randomizable signatures. In: Sako, K. (ed.) CT-RSA 2016. LNCS, vol. 9610, pp. 111–126. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-29485-8_7
42. Thomson, M., Wood, C.A.: Oblivious HTTP. Internet Draft draft-thomson-http-oblivious-01, Internet Engineering Task Force (2021)
43. Wang, J., Wang, G., Susilo, W.: Anonymous single sign-on schemes transformed from group signatures. In: 2013 5th International Conference on Intelligent Networking and Collaborative Systems, pp. 560–567. IEEE (2013)

44. Xu, R., Yang, S., Zhang, F., Fang, Z.: MISO: legacy-compatible privacy-preserving single sign-on using trusted execution environments. In: 2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P), pp. 352–372. IEEE Computer Society (2023)
45. Zhang, Z., Król, M., Sonnino, A., Zhang, L., Rivière, E.: EL PASSO: privacy-preserving, asynchronous single sign-on (2021)



DPC: Filtering Out Patch-Based Poisoned Samples with Differential Privacy

Yukun Yan¹, Peng Tang², Rui Chen^{1(✉)}, Qilong Han^{1(✉)}, and Ruochen Du¹

¹ Harbin Engineering University, Harbin, China
{yanyukun, ruichen, hanqilong, drc96}@hrbeu.edu.cn

² Shandong University, Qingdao, China
tangpeng@sdu.edu.cn

Abstract. Patch-based data poisoning backdoor attacks have been exposed the vulnerability of deep neural networks (DNNs). While the differentially private training algorithm is a promising defense method, it faces significant challenges: 1) simultaneously limiting the fitting of clean and poisoned samples induces the degradation of clean accuracy, and 2) maintaining model stability struggles when poisoned samples dominate the target class. To address these challenges, we propose the *Bi-optimization Training Strategy*, which integrates robust training with poisoned sample filtering techniques and conducts asynchronous optimization to complete the defense. Next, to implement this strategy, we combine the Differentially Private training algorithm with the Confusion training method to unveil a practical defense framework (DPC). This approach focuses on filtering out poisoned samples and retraining the model with the rest. To take full advantage of inherent stability of the differentially private training algorithm, even the poisoned samples dominate the target class, we adopt self-supervised pre-training to treat poisoned samples as outliers in the latent space. Then, the supervised fine-tuning algorithm enhanced with differential privacy can effectively limit the fitting of these poisoned samples. Additionally, we adaptively adjust the strength of differential privacy protection based on insights from filtered samples, improving clean sample fitting and further strengthening poison samples detection. Finally, our extensive experiments demonstrate that DPC (Our code is publicly available at <https://github.com/yyk1997/DPC>) preserves clean accuracy effectively while providing robust backdoor protection.

Keywords: Differential privacy · Data poisoning · Backdoor attacks

1 Introduction

One of the biggest concerns in the DNNs' security is a sneaky type of attack known as patch-based data poisoning backdoor attacks [5, 15, 20, 22, 24]. In these attacks, attackers embed a special patch-based *trigger* onto a few images. These altered samples, with labels chosen by the attacker, are mixed with the regular training data for the DNNs. The insidious aspect of these attacks is that the network performs well on normal data but starts executing the attacker's commands upon detecting this specific *trigger*.

Various kinds of research have emerged in recent years to combat these harmful attacks [3, 4, 7, 9, 12, 13, 21–23, 25]. Among the diverse strategies developed, the differentially private training algorithm, which has always been a method for protecting the privacy of the training data [1, 19], has emerged as a critical defense method [3, 7, 11, 14, 26]. The core principle is its inherent stability; it can maintain the model’s performance unaffected even if several data points in the training set are inserted, modified, or removed [3, 16]. Therefore, the differentially private training algorithm naturally becomes one of the robust enhancement techniques to resist data poisoning attacks.

Despite its advantages, applying the differential privacy training algorithm as a countermeasure to data poisoning backdoor attacks presents certain challenges. Firstly, without distinguishing between clean and poisoned samples, clipping gradients and adding noise in each training iteration to achieve differential privacy limits the fitting of both samples. This restricts the formation of backdoors but also inevitably leads to a noticeable degradation of clean accuracy. Furthermore, differentially private training algorithms struggle when poisoned samples dominate the target class. Under such conditions, the algorithm might mistakenly treat the *trigger* as the main feature of the target class, maintaining the model’s stability by fitting this feature and leading to defense failure.

To overcome these challenges and defend against patch-based data poisoning backdoor attacks, we first propose the *Bi-optimization Training Strategy*, which seamlessly integrates robust training methods with poisoned sample filtering techniques. The core idea is to leverage the filtered poisoned samples to improve the fitting of clean samples within robust training methods. Concurrently, robust training methods help the identification of clean candidates, eliminating the need for reserved clean samples in the poisoned sample filtering process. Furthermore, we propose DPC, a practical implementation of above strategy, which combines the **D**ifferentially **P**rivate training algorithm [1] with the **C**onfusion training [21] method. DPC employs asynchronous updates to solve the above *Bi-optimization Training Strategy* and effectively filter out poisoned samples. In the robust training module of DPC, we employ a blend of self-supervised pre-training with differentially private fine-tuning. This combination limits the fitting of poisoned samples even these samples dominate the target class and provides the foundation for segregating clean candidates from the poisoned dataset. Specifically, self-supervised pre-training can treat the poisoned samples originally from non-target classes as outliers in the latent space, which will present larger loss values in the fine-tuning. In this case, the differentially private fine-tuning effectively limit the fitting of these poisoned samples, and simultaneously take inherent stability to restrict the fitting of other poisoned samples originally from the target class. After fine-tuning, clean candidates can be identified by evaluating the loss value associated with each sample. In the poisoned sample filtering module of DPC, we embrace the strategy of *fighting poison with poison* by implementing the confusion training method [21], which utilizes clean candidates with randomized labels to decouple the fitting of clean samples and identify those fitted as poisoned. To further refine the fitting of clean samples and improve the *precision rate* of clean candidates, we introduce an

adjustment mechanism which uses the identified poisoned samples to adaptively adjust the clipping norm and noise level in the differentially private fine-tuning. After several rounds of asynchronous optimizations in the DPC, we can effectively isolate the poisoned samples and retrain the model with the rest. This process not only minimizes the degradation of clean accuracy but also enhance the model’s robustness against the backdoor attacks.

We summarize our key contributions as follows:

- We explore the challenges of using the differentially private training algorithm to mitigate data poisoning backdoor attacks, particularly highlighting its vulnerabilities when poisoned samples dominate the target class.
- We introduce a novel defense against patch-based data poisoning backdoor attacks, named DPC, which employ the *Bi-optimization Training Strategy*, seamlessly integrating the differentially private training algorithm with techniques for filtering out poisoned samples. The implementation of self-supervised pre-training with differentially private fine-tuning can limit the fitting of poisoned samples, even these samples dominate the target class.
- To assess the efficacy of the DPC, we conduct comprehensive evaluations using real-world datasets across various attack scenarios. These evaluations demonstrate DPC’s effectiveness in defending against patch-based data poisoning backdoor attacks. Notably, it reduces the deploying requirement while maintaining both the model’s robustness and performance.

2 Related Works

2.1 Patch-Based Data Poisoning Backdoor Attacks

BadNets [5] pioneers patch-based data poisoning backdoor attacks by embedding triggers in randomly selected images from a dataset and changing their labels as per the attacker’s choice. Trojan [15] inverse DNNs to generate a general patch trigger and retrain the model with reversed engineering training data to inject malicious behaviors into the model. CL [24] proposed the clean-label attacks, where the attacker only selects samples from the target class and constructs the poisoned data only by tampering with the images. TaCT [22] constructs the poisoned samples from source classes and the cover samples which are also stamped patch-based triggers but still have correct labels. Adap_patch [20] challenges the notion that poisoned and clean samples are separable in latent space, which uses multiple triggers and complex strategies to diversify poisoned sample’s latent representations.

2.2 Patch-Based Data Poisoning Backdoor Defenses

In general, existing defenses can be divided into two main categories, as follows:

Robust Training Methods. These defenses limit the fitting of poisoned samples during the training process to prevent the creation of hidden backdoors.

Du *et al.* [3] first explore noisy SGD to learn differentially private DNNs for the defense. Hong *et al.* [7] adopt differentially private stochastic gradient descent (*DP-SGD*) to clip and perturb individual gradients during the training process. Nguyen *et al.* [18] uses differential privacy techniques to first propose the theory of *backdoor-free*. Mohammad *et al.* [17] proposes that differential privacy can effectively defend against backdoor attacks under federated learning. Bolun *et al.* [25] proposes a technique for reverse engineering hidden triggers embedded inside DNNs. Li *et al.* [12] proposes the ABL to apply intervention during model training to suppress the influence of poisoned samples. Li *et al.* [13] propose the NAD, which attempts to eliminate the backdoor via fine-tuning the model on a small clean dataset. Li *et al.* [9] propose the DBD, which decouples the original end-to-end training process into three stages to get the backdoor robust model. Gao *et al.* [4] proposed an adaptively splitting dataset defense with a small reserved clean dataset. However, these defense methods often fail to ensure clean accuracy while enhancing the robustness of the model.

Poisoned Samples Filtering Methods. These defenses aim at filtering out poisoned samples from the poisoned training dataset. After the filtering process, only benign samples will be used in the training process, which eliminates the creation of backdoors. Tran *et al.* [23] first demonstrates that poisoned samples tend to leave behind a detectable trace in the spectrum of the covariance of feature representations. Tang *et al.* [22] use the reserved data distribution to separate the clean and poisoned samples into two clusters in the latent space. Qi *et al.* [21] applies an additional poisoning attack to the already poisoned dataset, actively exposing backdoor patterns to detection. However, these methods often rely heavily on the predefined *poisoned rate* or the reserved clean dataset.

3 Preliminaries

3.1 Deep Learning with Differential Privacy

Abadi *et al.* [1] proposed a differentially private stochastic gradient descent (*DP-SGD*) algorithm as a general mechanism of applying differential privacy in the deep learning model training phase, which restricts the privacy loss in each iteration by updating the model in two steps: 1) clipping the L_2 norm of the gradient for each example with bound C to limit the sensitivity of gradient; 2) summing up the clipped per-sample gradients and add Gaussian noise $\mathcal{N}(0, C^2\sigma^2)$ before updating the model parameters.

3.2 Threat Model

In this paper, we consider the DNNs as common classification models, which is represented as $f(\cdot; \theta) : \mathcal{X} \rightarrow [\Delta]$, where $\mathcal{X}$ is the input space, Δ is the number of classes, and θ is the model parameters. We also define the hard-label classifier $F(\cdot; \theta) := \operatorname{argmax}_{[\Delta]} f(\cdot; \theta)$ that gives the hard prediction. In general, a deep neural network $f(\cdot; \theta)$ can be decomposed as $l(\cdot; w_l) \circ h(\cdot; w_h)$, where l is the linear prediction layer, h is the backbone feature extractor before the prediction layer, w_l and w_h are the corresponding parameters. We use $D = \{(x_i, y_i)\}_{i=1}^n$ as a clean

dataset and use $\mathcal{A}$ to denote the learning algorithm. Accordingly, $\theta := \mathcal{A}(D)$ is the benign model trained on the clean dataset D . We call $\mathbb{P}_{(x,y) \sim D}\{F(x; \theta) = y\}$ the *clean accuracy* (ACC) of the classifier $F(\cdot; \theta)$.

Attacker’s Capabilities. Like the majority of current backdoor poisoning strategies, the attacker: **1)** can alter a small fraction of the training data (typically under 10%); **2)** accessing the victim’s model architecture, training process, and any countermeasures against backdoors. We use $\mathcal{T} : \mathcal{X} \rightarrow \mathcal{X}$ to denote the trigger planting strategy that plants the patch-based backdoor trigger to a data sample. And we use $\mathcal{R} : \mathcal{X} \times \mathcal{Y} \rightarrow t$ to denote the relabeling function on poison samples, where $\mathcal{Y}$ denotes the label space and t denotes the target class. The attacker can manipulate k samples in the clean dataset D , and we use $\mathcal{J} := \{j_1, \dots, j_k\}$ to denote these sample’s indices. Then, the poisoned dataset is denoted as $\tilde{D} = \{(\tilde{x}_i, \tilde{y}_i) | i = 1, \dots, n\}$, where $(\tilde{x}_i, \tilde{y}_i) = (\mathcal{T}(x_i), \mathcal{R}(x_i, y_i)) | i \in \mathcal{J}$, else $(\tilde{x}_i, \tilde{y}_i) = (x_i, y_i)$. We decompose $\tilde{D} := D_{clean} \cup D_{poison}$, where $D_{poison} := \{(\tilde{x}_i, \tilde{y}_i) | i \in \mathcal{J}\}$ and $D_{clean} := \tilde{D} \setminus D_{poison}$. We set the *poisoned rate* as the $\frac{|D_{poison}|}{|\tilde{D}|} \times 100\%$. The model $\tilde{\theta} = \mathcal{A}(\tilde{D})$ is the backdoor model which is trained on the poisoned dataset $\tilde{D}$. The quantity $\mathbb{P}_{(x,y) \sim D} |_{y \neq t} \{F(\mathcal{T}(x), \tilde{\theta}) = t\}$ is the *attack success rate* (ASR) on the backdoor model.

Attack Goals. The attacker aims to construct a poisoned dataset $\tilde{D}$ such that the trained model $\tilde{\theta} := \mathcal{A}(\tilde{D})$ will be backdoored. In general, a desired backdoor attack satisfies maintaining a level of *clean accuracy* (ACC) that is comparable to that of a benign model. On the other hand, the backdoored model should have a high *attack success rate* (ASR).

3.3 Filtering Backdoor Poisoned Samples

We investigate methods for filtering poisoned samples as a means of defense against backdoor poisoning attacks.

Defender’s Capabilities. The defender has full control over the poisoned dataset, including the ability to access, scrutinize, and process the dataset, as well as the liberty to independently train models on it.

Defender’s Goal. The defender aims to identify and isolate a collection of suspect samples $D_{suspect}$ from the poisoned dataset $\tilde{D}$. We denote $\frac{|D_{suspect} \cap D_{poison}|}{|D_{poison}|}$ as the *true positive rate* (TPR) and $\frac{|D_{suspect} \setminus D_{poison}|}{|D_{poison}|}$ as the *false positive rate* (FPR) of the defense. The goals of the defense are two-fold: 1) High TPR : filter out as many backdoor poison samples as possible; 2) Low FPR : the quantity of mistakenly eliminated clean samples should be controlled at a low level.

4 Differential Privacy for Backdoor Defenses

In this section, we examine the effectiveness of the DP - SGD within the context of patch-based poisoned datasets. Our analysis highlights the challenges of using

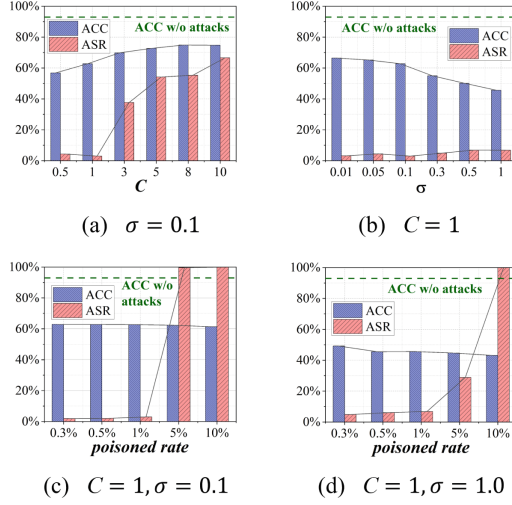


Fig. 1. The ACC and ASR obtained from *DP-SGD* training under different settings. (a)–(b): DNNs trained under poisoned rate as 1% with fixing $\sigma = 0.1$ or $C = 1.0$. (c)–(d): DNNs trained under $C = 1.0$ with $\sigma = 0.1$ or $\sigma = 1.0$.

the differentially private training algorithm as a defense method against data poisoning backdoor attacks.

Settings. We employ the BadNets [5] attack on the CIFAR10 dataset [10], setting the default *poisoned rate* to 1%. We conduct *DP-SGD* training with the fixed C or σ to evaluate the *clean accuracy* (*ACC*) and the *attack success rate* (*ASR*), respectively. Subsequently, we maintain both C and σ constant while varying the *poisoned rate* in additional experiments.

Results. As shown in Fig. 1(a)–(b), setting C less than 3 with $\sigma = 0.1$, or fixing $C = 1.0$ with σ range from 0.01 to 1, effectively keep the ASR below to 10%. Notably, ASR is more sensitive to changes in the clipping norm C than σ ; it gradually increases from under 10% to over 60% as C increased in Fig. 1(a) while remaining stable in Fig. 1(b). This finding is consistent with the expectation in literature [7, 17, 18], where the clipping norm C primarily mitigates the impact of malicious gradients and Gaussian noise (parameterized by $C\sigma$) merges the gradient variations between poisoned samples and clean samples. However, regardless of the C and σ settings, there is always a significant drop in the ACC compared to scenarios w/o attacks. As shown in Fig. 1(c)–(d), the *DP-SGD* training is effective only in limited scenarios of *poisoned rate*. Specifically, Fig. 1(c) shows that the ASR can be controlled below 10% only when the *poisoned rate* is under 5%. After adjusting σ to 1.0, as depicted in Fig. 1(d), a 70% reduction in ASR at *poisoned rate* = 5% occurs alongside a steep decline in ACC across all *poisoned rate* configurations. However, in the *poisoned rate* = 10%, the ASR still achieves 100%. A key factor behind this phenomenon is the predominance of poisoned samples within the target class: with a 10% *poisoned rate*, BadNets introduced 5,000 poisoned samples to the target class, overshadowing the 4,500

benign samples. Hence, despite the differential privacy can improve the stability of deep learning models and help to learn the main features of the dataset, a strong correlation between backdoor triggers and the target label forms when poisoned samples predominate the target class, leading to defense failure.

Challenges. Based on these results, we identify two main challenges for the differentially private training algorithm in addressing backdoor attacks:

- Differentially private training simultaneously limits the fitting of both clean and poisoned samples while defending against backdoor attacks, making it difficult to balance model robustness and clean accuracy.
- The characteristic of differentially private training algorithms maintaining model stability is hard to cope with the scenarios when poisoned samples dominate the target class.

5 The *Bi-optimization Training Strategy*

To address the challenges discussed earlier, this paper introduces the *Bi-optimization Training Strategy*. To lay the groundwork for our innovative approach, we initially establish the definitions of the sample characteristic function and the identification of the clean or poisoned samples.

Definition 1 (Sample Characteristic Function). A sample characteristic function is defined as $\phi(\cdot; \theta) : \mathcal{X} \times \mathcal{Y} \rightarrow \mathcal{Z}$, which maps data point (x, y) to characteristic vectors $\phi(x, y; \theta) \in \mathcal{Z}$, where $\mathcal{Z}$ is a characteristic space that facilitates discrimination between clean and poisoned samples.

Definition 2 (Identification of Clean or Poisoned Samples). A clean or poisoned samples detector is the composition of a sample characteristic function $\phi(\cdot; \theta)$ and a decision function $\xi : \mathcal{Z} \rightarrow \{0, 1\}$. Given a data point (x, y) , the decision function ξ takes $\phi(x, y; \theta)$ as its input and predicts whether (x, y) is a clean or poisoned sample, where $\xi(\phi(x, y; \theta)) = 1$ indicates a positive prediction and $\xi(\phi(x, y; \theta)) = 0$ otherwise.

Our strategy leverages the strengths of various defense mechanisms. Specifically, it is founded on the principle that the identification of poisoned samples enhances the robust training model to accurately fit clean samples. Simultaneously, the robust training model captures features of clean samples, aiding in the identification of poisoned ones. By optimizing both aspects, we can effectively filter out poisoned samples and retrain the model with the rest, thus maintaining the model's robustness and performance. The following sections detail this strategy:

Definition 3 (The *Bi-optimization Training Strategy*). A defender identifies the suspect poisoned portion of a contaminated dataset $\tilde{D}$ by solving the following problem:

$$\begin{aligned} D_{\text{suspect}}^* &= \max_{\phi_{\text{poison}}, \xi_{\text{poison}}} TPR(\phi_{\text{poison}}, \xi_{\text{poison}}, \theta_{\text{robust}}^*) \\ \text{s.t. } FPR(\phi_{\text{poison}}, \xi_{\text{poison}}, \theta_{\text{robust}}^*) &\leq \gamma, \gamma \in [0, 1] \end{aligned} \quad (1)$$

$$\theta_{robust}^* = \mathcal{A}_{robust}^*(\tilde{D}, D_{suspect}^*) \quad (2)$$

where $\mathcal{A}_{robust}^*$ denote the robust training method which can contain a series of operations. We use $TPR(\phi_{poison}, \xi_{poison}, \theta)$ and $FPR(\phi_{poison}, \xi_{poison}, \theta)$ to denote the True Positive Rate and False Positive Rate of the poisoned sample identification.

Practical Insight: Considering the stability of models enhanced by differential privacy and the fact that the majority of the poisoned dataset is clean, these robust training models can effectively capture the dataset’s clean features and help the identification of clean candidates from the poisoned dataset. Additionally, recent poisoned sample filtering methods heavily rely on a reserved clean dataset [13, 21, 22, 25]. Therefore, we introduce the variable D_{clean}^* which represents the clean candidates within the poisoned dataset, and address the Eqs. (1–2). Specifically, We use D_{clean}^* to tackle the bilevel optimization problem via a separate yet asynchronous optimization solution. The robust training module can be written as follows:

$$\begin{aligned} D_{clean}^* = \max_{\phi_{clean}, \xi_{clean}} \quad & Precision(\phi_{clean}, \xi_{clean}, \theta_{robust}^*) \\ \text{s.t. } \quad & Recall(\theta_{robust}^*, \phi_{clean}, \xi_{clean}) \geq \beta, \beta \in [0, 1] \end{aligned} \quad (3)$$

$$\theta_{robust}^* = \mathcal{A}_{robust}^*(\tilde{D}, D_{suspect}^*) \quad (4)$$

where we denote $\frac{|D_{clean}^* \cap D_{clean}|}{|D_{clean}^*|}$ as the *precision rate* (*Precision*) and $\frac{|D_{clean}^* \cap D_{clean}|}{|D_{clean}|}$ as the *recall rate* (*Recall*) of the D_{clean}^* . The poisoned sample filtering module can be written as follows:

$$\begin{aligned} D_{suspect}^* = \max_{\phi_{poison}, \xi_{poison}} \quad & TPR(\phi_{poison}, \xi_{poison}, \theta_{poison}^*) \\ \text{s.t. } \quad & FPR(\theta_{poison}^*, \phi_{poison}, \xi_{poison}) \leq \gamma, \gamma \in [0, 1] \end{aligned} \quad (5)$$

$$\theta_{poison}^* = \mathcal{A}_{filter}^*(\tilde{D}, D_{clean}^*) \quad (6)$$

where $\mathcal{A}_{filter}^*$ denotes the poisoned sample filtering method which can also contain a series of operations.

6 DPC

In this section, we integrate the *DP-SGD* training with the confusion training to present DPC, which takes advantage of the *Bi-optimization Training Strategy* to empower patch-based data poisoning backdoor defenses. We start with an overview of DPC in Sect. 6.1, followed by technical details in Sect. 6.2–Sect. 6.4.

6.1 Overview

As shown in Fig. 2, DPC consists of two primary modules: the robust training module and the poisoned sample filtering module. Two modules execute in succession to respectively identify clean candidates and suspect poisoned samples

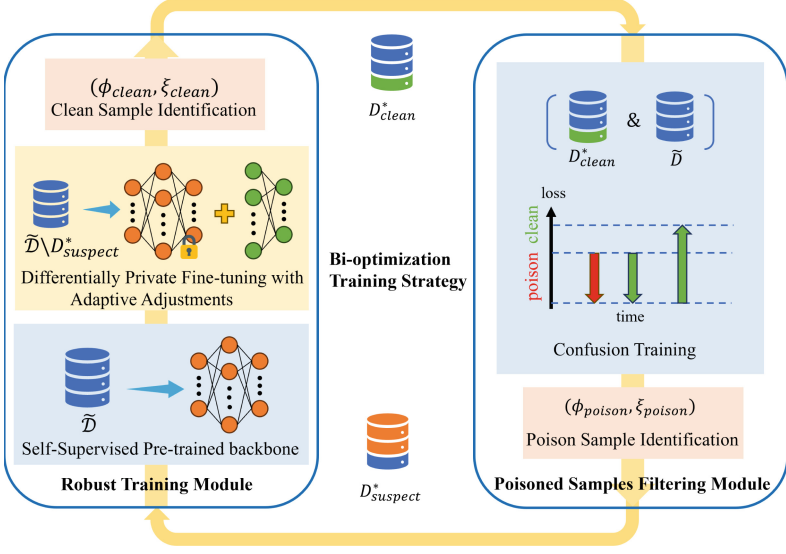


Fig. 2. The DPC defense framework.

within the poisoned dataset. After the final identification of the poisoned sample, we filter out these suspects and retrain with the rest to complete defenses.

We sketch the pipeline in the Algorithm 1. Initially, we initialize parameters (line 1). Next, we implement Eq. 4 using a self-supervised pre-trained backbone coupled with the DP -SGD fine-tuning linear predication layer (line 3). Whenever the DP -SGD training proceeds for v epochs, we conduct the data identification. Utilizing Eq. 3 for clean sample identification, we obtain D_{clean}^* which represent the clean candidates of the poisoned dataset (line 5). In the poisoned sample filtering module, we apply the confusion training method to implement Eq. 6. This method involves taking D_{clean}^* , constructing a confused dataset with randomized labels to decouple the fitting of clean samples in $\tilde{D}$, and obtaining the model $\hat{\theta}_{poison}$, which is tailored to fit only the poisoned samples in $\tilde{D}$ (line 6). Coupled with the implementing Eq. 5 for poisoned sample identification, we find out the suspect poisoned dataset $D_{suspect}^*$ which fits in $\hat{\theta}_{poison}$ (line 7). Then, we remove $D_{suspect}^*$ from $\tilde{D}$ to get the cleansed dataset D^* (line 8). After a series of data identification operations, we use the current cleansed dataset D^* to dynamically adjust C' and σ' in DP -SGD fine-tuning. This adjustment further mitigates the limitation imposed by the differential private training algorithm on the fitting of clean samples (lines 9–11). After m training epochs, we finalize the cleansed dataset D^* and retrain the model with it to develop a model that is immune to backdoor attacks. The details of confusion training and poisoned sample identification are provided in Appendix A.1.

Algorithm 1. DPC

Input: Poisoned dataset: $\tilde{D}$; Self-supervised pre-trained backbone model parameters: w_h^* ; Linear-layer parameters: w_l ; Clipping norm: C ; Gaussian noise parameter: σ ; Clean samples filter rate list: $[\alpha_1, \dots, \alpha_n]$; number of training epochs: m ; Number of epochs between filtering operation: v

Output: The cleansed dataset D^*

```

1: Initialization:  $D^* = \tilde{D}$ ,  $C' = C$ ,  $\sigma' = \sigma$ 
2: for  $i = 1 \rightarrow m$  do
3:    $\hat{\theta}_{robust} = DP\text{-}SGD(D^*, w_h^*, w_l, C', \sigma')$  ▷ Robust training
4:   if  $i \% v = 0$  then
5:      $D_{clean}^* = \text{Clean Sample Identification}(D^*, \hat{\theta}_{robust}, \alpha_i)$ 
6:      $\hat{\theta}_{poison} = \text{Confusion Training}(\tilde{D}, D_{clean}^*)$  ▷ Fitting poisoned samples
7:      $D_{suspect}^* = \text{Poisoned Sample Identification}(\tilde{D}, \hat{\theta}_{poison})$ 
8:      $D^* = \tilde{D} \setminus D_{suspect}^*$ 
9:      $\eta = \frac{|D_{suspect}^*|}{|\tilde{D}|}$ 
10:     $C' = \frac{C}{\eta}$ 
11:     $\sigma' = \eta \cdot \sigma$ 

```

6.2 Self-supervised Pre-training with DP-SGD Fine-Tuning

Observations from the Sect. 4 reveal that when poisoned samples dominate the target class, supervised training with differential privacy inevitably leads to a strong correlation between *triggers* and the target label. To overcome this challenge, an intuitive approach is to discard the labels from the dataset, extract each sample’s representation, and train with the assumption that all poisoned samples are outliers. This approach allows the differentially private training algorithm to naturally limit the fitting of poisoned samples. To implement this idea, we resort to the training paradigm of self-supervised pre-training with supervised fine-tuning. Specifically, the self-supervised pre-training groups the high-dimensional representations of both clean and poisoned samples originally from the same source class. If we continue with supervised fine-tuning of the linear prediction layer using poisoned labels, the representations of poisoned samples from non-target classes will significantly mismatch with the target label. These poisoned samples can then be identified as outliers during fine-tuning, resulting in high loss values. Additionally, employing a differentially private fine-tuning algorithm can further limit the fitting of poisoned samples from all source classes. For a better understanding, please refer to Fig. 4 in Appendix A.2.

In practice, we use contrastive learning to pre-train the representation backbone on poisoned dataset $\tilde{D}$. Specifically, we adopt the SimCLR [2] which leverages various data augmentation techniques to provide a more expressive representation of each sample. We optimize the backbone parameters w_h based on the unlabeled version of $\tilde{D}$ as follows:

$$w_h^* = \underset{w_h}{\operatorname{argmin}} \sum_{(x,y) \in \tilde{D}} NT\text{-}Xent \text{ Loss}(x, w_h) \quad (7)$$

where *NT-Xent Loss* [2] uses cosine similarity to narrow the distance of similar samples and push the distance of dissimilar samples in high-dimensional space.

Algorithm 2. Clean Sample Identification

Input: Candidate dataset: D^* ; Robust model $\hat{\theta}_{robust} = [w_h^*, w_l^*]$; Clean samples detector: ϕ_{clean} , ξ_{clean} ; Clean samples filter rate list: $\alpha = [\alpha_1, \dots, \alpha_r]$; the number of filtering operations: r ; Fine-grained filter rate list: $\lambda = [\lambda_1, \dots, \lambda_{r-1}]$

Output: The clean portion of dataset $\tilde{D} : D_{clean}^*$

- 1: $\theta' = \mathcal{A}(\tilde{D})$ ▷ train with several epochs
- 2: **if** $i \in (1, r - 1)$ **then**
- 3: $D'_{clean} = \xi_{clean}(\phi_{clean}(D^*; \hat{\theta}_{robust}); \alpha_i)$
- 4: $D_{clean}^* = \xi_{clean}(\phi_{clean}(D'_{clean}; \theta'); \lambda_i)$
- 5: **else**
- 6: $D_{clean}^* = \text{random sampling}(D^*, \alpha_i)$

Once w_h^* is obtained, we can freeze it and adopt $\tilde{D}$ with $DP\text{-}SGD$ to further optimize remaining linear prediction layer parameters w_l , *i.e.*,

$$w_l^* = \underset{w_l}{\operatorname{argmin}} \sum_{(x,y) \in \tilde{D}} \mathcal{L}(f_{[w_h^*, w_l]}(x), y) \quad (8)$$

where $\mathcal{L}(\cdot)$ indicates the supervised loss.

6.3 Clean Sample Identification

After obtaining the model $\hat{\theta}_{robust} = [w_h^*, w_l^*]$, we use it to evaluate all samples to identify D_{clean}^* . We sketch the clean sample identification in Algorithm 2. Since poisoned samples are limited from fitting during $DP\text{-}SGD$ fine-tuning, it is intuitive to evaluate each sample using the loss function and select samples with the lowest loss value as the clean candidates. Then, we implement the ϕ_{clean} to evaluate each sample's loss value with model $\hat{\theta}_{robust}$ as Eq. 9, and implement the ξ_{clean} to identify D'_{clean} with the number of $\alpha_i \cdot |D^*|$ samples which have the lowest loss values as Eq. 10 in line 3.

$$\phi_{clean}(x, y; \theta_{robust}^*) = \mathcal{L}(f_{\theta_{robust}^*}(x), y) \quad (9)$$

$$\xi_{clean}(\phi_{clean}(D^*; \theta_{robust}^*); \alpha_i) = \underset{\alpha_i \cdot |D^*|}{\operatorname{argmin}}_{(x,y) \in D^*} \mathcal{L}(f_{\theta_{robust}^*}(x), y) \quad (10)$$

$$\phi_{clean}(x, y; \theta') = \mathcal{L}(f_{\theta'}(x), y) \quad (11)$$

$$\xi_{clean}(\phi_{clean}(D'_{clean}; \theta'); \lambda_i) = \underset{\lambda_i \cdot |D'_{clean}|}{\operatorname{argmax}}_{(x,y) \in D'_{clean}} \mathcal{L}(f_{\theta'}(x), y) \quad (12)$$

To further reduce the number of poisoned samples in the clean candidate dataset D'_{clean} , we leverage the observation that poisoned samples fit more quickly than clean samples during the initial epochs of training on the poisoned dataset $\tilde{D}$ [12]. Employing the poisoned model θ' from line 1, we utilize a fine-grained identification as described by Eqs. 11–12. This method identifies the $\lambda_i \cdot |D'_{clean}|$ samples exhibiting the largest loss values within D'_{clean} , thereby refining a purer clean candidate dataset D_{clean}^* . To keep the sample diversity in the clean candidates, we adopted the method of random sampling from D^* in the final filtering operation (line 6). It's worth noting that, in all clean sample identifications, we select samples from the dataset in a class-aware way.

6.4 Adaptive Adjustment in *DP-SGD* Fine-tuning

To further improve the fitting of clean samples in the robust training module with $D_{suspect}^*$, we take adaptive adjustments on the clipping norm C' and the noise level σ' in Algorithm 1 (lines 9–11). Specifically, we establish a relationship between the adjustment ratio η with the size of $D_{suspect}^*$. Since $D_{suspect}^*$ is a subset of $\tilde{D}$ and $\eta = \frac{|D_{suspect}^*|}{|\tilde{D}|}$ falls within the range $[0, 1]$, the adjusted C' will always be greater than its initial value while the adjusted σ' will remain below its initial value throughout the training process.

In the confusion training [21], the model θ_{poison} initially fits all samples in the poisoned dataset adequately. The number of input clean candidates significantly influences the ability of decoupling the fitting of clean samples within the poisoned dataset: more clean candidates input results in more clean samples unfitting, which gradually reduce the size of $D_{suspect}^*$. In practical DPC deployment, we incrementally increase the number of the clean candidates $|D_{clean}^*|$, which gradually decreases both the size of $D_{suspect}^*$ and the adjustment ratio η . Consequently, this adjustment leads to a monotone increasing in C' and a monotone decreasing in σ' , relaxing the gradient restrictions and enhancing the model’s ability to accurately fit clean samples under effectively filtering out poisoned ones.

7 Evaluation

7.1 Evaluation Setup

Datasets and the DNN Model. Our experimental analysis centers on two widely recognized image classification datasets: CIFAR10 [10] and GTSRB [8]. Utilizing these datasets enables us to conduct a comprehensive comparison with cutting-edge methods. To ensure uniformity in our evaluation, ResNet-18 [6] serves as the default model and we take the standard cross-entropy loss function to compute the supervised loss $\mathcal{L}$ across all experiments.

Evaluation Metrics. Our evaluation is presented using two sets of metrics: 1) for poisoned samples filtering defense, we detail the detection’s *true positive rate* (*TPR*) and *false positive rate* (*FPR*); 2) for end-to-end backdoor defense, we measure the final model’s *clean accuracy* (*ACC*) and *attack success rate* (*ASR*) post-defense implementation. Moreover, to compare all different types of defenses, we also evaluate the ACC and ASR for poison sample detection defenses which retrain the model on the cleansed dataset. To ensure the thoroughness of our evaluation, all principal experiments are conducted independently three times, with the average of these outcomes reported.

Attacks. To validate the performance of the DPC, we evaluate it against 5 patch-based data poisoning backdoor attacks in the literature, including 1) classical dirty-label attacks (BadNet [5], Trojan [15]), 2) clean-label attacks (CL [24]), 3) latent space adaptive attacks (TaCT [22], Adap_patch [20]). We strictly followed the protocols and suggested configurations of their original papers. On

CIFAR10, we implement and evaluate all these attacks. On GTSRB, we omit CL since its original paper only released poisoning settings for CIFAR10. In our main evaluation, we set the default *poisoned rate* to 0.5% and 1% in CIFAR10 and GTSRB, and set the default *cover rate* as 3% and 5% in CIFAR10 and GTSRB respectively under TaCT and Adap_patch attacks.

Baseline Defenses. We compare DPC with 8 patch-based backdoor defenses in the literature, including 1) poisoned samples filtering methods (SS [23], SCAn [22], CT [21]), 2) robust training methods (NC [25], ABL [12], NAD [13], DBD [9], ASD [4]). Besides, SCAn, CT, NC, and NAD are assumed to have access to 2000 reserved clean samples, ASD is assumed to have access to 10 clean training data in each class.

DPC Setups. For our DPC, we initially set the clipping norm C to 1.0 and the noise level σ to 0.5, set the number of iterations m to 30 and the number of iterations between filtering operation v to 10. The backbone model is trained by 100 epochs under self-supervised training. In the clean sample filtering method, we set the filter rate list α to $[0.1, 0.5, \frac{5000}{|D^*|}]$, set fine-grained filter rate list λ to $[0.1, \frac{2000}{|D^*|}]$. In the poisoned sample filtering module, we adopt the same setting of CT [21] from its original paper. In the last poisoned sample filtering operation, we take the strategy of *Iterative Poison Distillation* from CT [21].

7.2 Main Results

We report our main results in Table 1 (for poisoned samples filtering defenses) and Table 2 (for all defenses). Same as [21], we consider a poison detector is **successful** against an attack if the TPR is more than 80%; otherwise, we say the detector is **unsuccessful**. Stricter than [21], we consider a defense is **successful** against only if the ASR is reduced below 5%, otherwise, it is **unsuccessful**.

TPR and FPR. According to Table 1, as a backdoor poison samples detector, DPC **successfully** detects poisoned samples across all attacks and datasets. In contrast, SCAn(2000) performs well only on the GTSRB dataset, while CT(2000) performs a little bit poorly in response to Adap_patch attacks. Different from CT(2000), DPC, which also utilizes the confusion training method, is not limited by the number of reserved clean samples and can identify more than 2000 clean candidates, thereby improving the *true positive rate* (TPR). However, in terms of the *false positive rate* (FPR), DPC’s performance is acceptable but slightly inferior to that of CT(2000) and SCAn(2000). This is primarily because CT(2000) and SCAn(2000) can uniformly select 2000 clean samples from all classes as the reserved clean samples, whereas DPC can only identify clean candidates through iterative processes. The diminishing diversity in D_{clean}^* compared to the reserved clean samples results in less effectiveness of the clean samples unfitting during the confusion training, leading to a slightly worse FPR performance.

ACC and ASR. According to the Table 2, DPC **succeeds** in reducing the ASR below 5% in all settings, demonstrating consistent effectiveness. Other defense

Table 1. Comparison of DPC with existing backdoor poisoned sample detectors.

	(%)	mean		SS		SCAn(2000)		CT(2000)		DPC(ours)	
		TPR	FPR	TPR	FPR	TPR	FPR	TPR	FPR	TPR	FPR
CIFAR10	No Poison	-	15	-	2.4	-	2.2	-	1.2	-	-
	BadNet	100	7.03	100	0	100	0.2	100	0.2	100	1.02
	Trojan	3.2	7.52	0	3.85	100	0.01	100	0.43	100	0.43
	CL	7.6	7.49	100	0	100	0.1	100	0.89	100	0.89
	TaCT	39.2	7.34	0	1.2	100	1.2	100	0.93	100	0.93
	Adap_Patch	1.2	7.53	0	2.3	70.8	2.22	84.0	2.33	100	0.93
	Average	30.24	8.65	40	1.63	94.16	0.95	96.8	1.13	100	0.93
GTSRB	No Poison	-	39.1	-	2.1	-	3.3	-	2.9	-	-
	BadNet	98.12	38.35	98.12	0.92	100	0.77	100	0.58	100	0.58
	Trojan	90.22	38.43	99.24	0.56	100	1.58	100	2.77	100	0.56
	TaCT	80.82	38.65	96.99	0.86	98.49	0.16	100	0.56	100	0.56
	Adap_Patch	52.55	38.81	82.85	0.97	74.43	1.16	81.57	2.1	100	0.56
	Average	80.42	38.66	94.3	1.082	93.23	1.394	95.39	1.782	100	0.56

methods, except for CT(2000), are **unsuccessful** to defend one or several types of attacks. DPC maintains an *clean accuracy* (ACC) always above 93% on CIFAR10 and GTSRB. Compared with the CT(2000) with a similarly excellent performance among the settings, DPC avoids the reserved clean dataset requirement which makes it more practical for deployment.

7.3 Defensive Results on Different Poisoned Rates

According to Fig. 3, we consider the set of *poisoned rate* = {0.5%, 1%, 5%, 10%} which covers both the high and low *poisoned rate* regions in CIFAR10 and GTSRB. For each setting, DPC consistently maintains the *clean accuracy* (ACC) above 92.5% and controls the *attack success rate* (ASR) lower than 4%. With the increasing of the *poisoned rate* under TaCT and Adap_patch, the ASR slightly rises in both CIFAR10 and GTSRB. The primary reason for this phenomenon is that DPC struggles to completely filter out all poisoned samples under these latent space adaptive attacks. Nevertheless, these results are considered acceptable since the ASR remains below 4%. Therefore, these findings underscore the effectiveness of DPC in preserving model performance while effectively mitigating the impact of patch-based backdoor attacks across varying *poisoned rates*.

7.4 Ablation Studies

Next, we present additional ablation studies on three factors: 1) the setting in decoupled robust training; 2) the setting of fine-grained clean samples filtering; and 3) adaptive adjustments in *DP-SGD* fine-tuning.

Table 2. Evaluation of DPC’s effectiveness as a backdoor defense.

(%)	mean		No Defense	SS	SCAn	NC	ABL	NAD	DBD	ASD	CT	DPC(ours)
C I F A R 10	BadNet	ACC	93.35	92.82	93.76	89.95	89.83	87.14	90.1	93.01	93.82	93.83
		ASR	100	0.62	0.58	1.5	9.62	2.55	2.33	100	0.61	0.48
	Trojan	ACC	93.93	93.56	93.77	90.7	91.2	65.01	92.75	93.2	93.53	94.15
		ASR	99.97	99.77	100	99.76	1.38	8.86	2.07	1.44	2.28	1.48
	CL	ACC	92.23	93.83	93.87	90.08	86.11	76.79	90.4	93.21	93.46	93.33
		ASR	99.62	99.62	1.68	1.05	96.34	1.48	99.98	0.76	1.41	1.3
	TaCT	ACC	93.72	93.46	93.88	92.53	91.69	86.1	91.78	93.22	93.46	93.41
		ASR	99.62	99.74	99.74	2.25	99.4	16.05	0.37	0.25	1.25	1.38
	Adap_patch	ACC	92.21	93.55	93.67	91.37	82.2	78.67	91.45	92.85	93.96	93.17
		ASR	99.98	99.97	100	0.52	100	1.77	2.91	89.61	1.37	0.86
G T S R B	BadNet	ACC	93.08	93.60	93.79	90.93	88.21	78.74	91.29	93.09	93.64	93.58
		ASR	99.83	79.94	60.4	21.01	61.35	6.14	21.53	38.41	1.38	1.1
	Trojan	ACC	93.65	90.76	94.20	91.14	85.78	94.39	83.26	94.08	92.77	93.75
		ASR	99.71	1.06	0.3	15.73	0.1	4.3	0.01	100	0.62	0.27
	TaCT	ACC	93.71	91.60	95.07	92.96	95.8	94.13	80.57	96.09	93.91	93.36
		ASR	98.25	30.22	0.19	32.85	99.93	1.78	0.07	0.26	0.52	0.61
	Adap_patch	ACC	93.54	93.54	93.68	93.54	81.75	94.15	86.08	96.29	93.97	93.59
		ASR	99.02	99.02	3.9	99.02	26.99	1.56	0.04	6.64	1.46	0.58
	Average	ACC	93.03	90.93	93.3	92.44	76.47	94.06	84.94	93.56	92.7	93.25
		ASR	84.55	53.82	0.29	54.83	95.18	19.17	0.01	99.93	2.77	1.06

The Setting in Robust Training. To verify the effectiveness of the self-supervised pre-training plus DP -SGD fine-tuning, we denote our method as SS+DP and compare it with two variants, including 1) SS, which denote only self-supervised pre-train backbone and frozen parameters followed by a supervised fine-tuning of the linear prediction layer; 2) DP, which only employs DP -SGD to supervised train the complete model (backbone with linear prediction layer).

As shown in the Table 3, we evaluate using the number of poisoned samples in the 10% dataset with the lowest loss value in $\tilde{D}$. SS+DP demonstrates superior inhibition of poisoned sample fitting in most scenarios compared to other variants. At the *poisoned rate* of 0.5% or 1%, the number of poisoned samples for SS+DP and SS are roughly equivalent, illustrating that self-supervised pre-training significantly reduces the fitting of poisoned samples. However, as the poisoned rate increases to 5% and 10%, the number of poisoned samples in SS+DP is substantially lower than that in the other variants, primarily because the DP -SGD training effectively prevents the fitting of poisoned samples from both target and non-target classes. It’s worth noting that SS and SS+DP perform better against TaCT compared to other attacks. This advantage is due to the fact that all poisoned samples in TaCT attacks originate from non-target classes. The reason for this phenomenon is that, after self-supervised pre-training, poisoned samples are re-clustered back to their original classes. Subsequently, DP -SGD

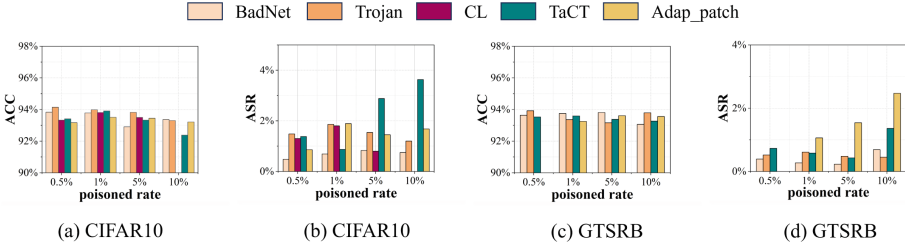


Fig. 3. All defensive results on different poison rates. When the poisoned rate hits 10% under CL attacks on CIFAR10, it results in a target class full of poisoned samples. This leaves no clean samples in the target class and contradicts the attacker’s goal of maintaining ACC, so we omit its experiment. When GTSRB faces Adap_patch attacks, a poisoned rate of 0.5% cannot lead the attack to success, so this is also omitted.

Table 3. The number of poisoned samples in D_{clean}^* .

			0.5%	1%	5%	10%				0.5%	1%	5%	10%
CIFAR10	BadNet	SS	3	6	28	156	GTSRB	BadNet	SS	2	0	23	47
		DP	1	4	316	355			DP	2	1	48	399
		SS+DP	2	7	28	102			SS+DP	2	0	23	38
	Trojan	SS	3	5	31	103		Trojan	SS	2	4	20	60
		DP	0	2	470	952			DP	3	4	274	339
		SS+DP	3	1	23	72			SS+DP	2	3	16	48
	CL	SS	1	9	233	—							
		DP	12	46	500	—							
		SS+DP	0	4	157	—							
	TaCT	SS	0	0	5	8		TaCT	SS	0	0	6	8
		DP	0	0	213	950			DP	1	8	283	416
		SS+DP	0	0	1	2			SS+DP	0	0	1	3
	Adap_patch	SS	3	5	20	69		Adap_patch	SS	2	2	15	45
		DP	0	2	42	117			DP	0	1	23	206
		SS+DP	0	3	20	64			SS+DP	1	1	12	40

fine-tuning sufficiently inhibits these poisoned samples’ fitting, enhancing overall model robustness. In contrast, in the DP variant, the number of poisoned samples substantially increases as the *poisoned rate* rising, particularly at 10%, highlighting the challenges faced by the differentially private training algorithm in coping with large numbers of poisoned samples, as discussed in Sect. 4.

The Setting of Fine-Grained Clean Sample Filtering. To verify the effectiveness of the fine-grained clean sample identification, we report the results of each step in the DPC with a fixed scenario. As shown in Table 4, the columns labeled D'_{clean} and D^*_{clean} display the count of poisoned samples relative to the number of corresponding clean candidates. In the first two identifications, D^*_{clean} shows marked reductions in the number of poisoned samples compared

to D'_{clean} . This demonstrates that fine-grained clean sample identification can further distill a purer set of clean candidates.

The Adaptive Adjustment in DP -SGD Fine-Tuning. We report the outcomes for each step in the DPC and DPC without adaptive adjustments (ad^2) in Table 4. Compared with the DPC w/o ad^2 , C' gradually increases, and σ' gradually decreases in DPC with adjustments. During the second identification, DPC has fewer poisoned samples in D'_{clean} compared to DPC w/o ad^2 in clean candidates, which lead to fewer poisoned samples in D^*_{clean} and further bring out a higher *true positive rate*. In the third identification, DPC achieves fewer poisoned samples in D^*_{clean} and ensures the defense’s success. This result underscores the benefit of adaptive adjustments during iterations, promoting more clean samples gradually fitted with lower loss values. In contrast, DPC w/o ad^2 struggles to maintain a high *precision rate* in D^*_{clean} , resulting in a lower *true positive rate* and defense failure.

Table 4. The results of each step in the DPC or DPC w/o ad^2 when defending Adap_patch attack with the poisoned rate of 10% on the GTSRB.

	Filtering operations	C'	σ'	D'_{clean}	D^*_{clean}	TPR(%)	FPR(%)	ACC(%)	ASR(%)
DPC	1	1.0	0.5	40/2664	0/244	81.79	42.33	92.56	2.47
	2	2.16	0.23	10/7155	0/1035	87.46	21.26		
	3	3.58	0.14	-	5/4973	97.14	1.44		
DPC w/o ad^2	1	1.0	0.5	43/2664	0/244	81.93	43.38	92.57	66.44
	2	1.0	0.5	19/6960	2/1034	82.95	20.69		
	3	1.0	0.5	-	37/5020	90.61	1.5		

8 Conclusion

In this paper, we propose DPC as a defense against patch-based data poisoning backdoor attacks. It utilizes the *Bi-optimization Training Strategy* which seamlessly combines the differentially private robust training with the poisoned sample filtering method. Our evaluations demonstrate that DPC enhances the model’s robustness and successfully maintains its performance. In the future, we will research to defend against other data poisoning attacks and explore the possibility of applying DPC to other data poisoning backdoor attacks.

Acknowledgments. This work was supported by the Heilongjiang Key R&D Program of China under Grant No. GA23A915, the National Natural Science Foundation of China under Grant No. 62072136 & No. 62002203, Shandong Provincial Natural Science Foundation No. ZR2020QF045 and Young Scholars Program of Shandong University.

A Appendix

A.1 Confusion Training with Poisoned Sample Identification

We present the confusion training with poisoned sample identification in the Algorithm 3. Initially, we train the model adequately on the poisoned dataset (line 1), ensuring it fully captures the features of both clean and poisoned samples. Subsequently, we mix clean candidates with randomized labels to decouple the fitting of clean samples within the poisoned dataset (lines 2–7). The number and *precision rate* of clean candidates are crucial for effectively decoupling the fitting of clean samples. After confusion training, we use the model with parameters θ_{ct} to identify the suspect poisoned samples (lines 9–12). The condition is that as long as the model fits a sample, then we consider this sample is suspected poisoned.

Algorithm 3. Confusion Training with Poisoned Sample Identification

Input: Dataset: $\tilde{D} = \{(\tilde{x}_i, \tilde{y}_i)\}_{i=1}^n$; candidate clean dataset: D_{clean}^* ; loss function: $\mathcal{L}$, confusion factor: λ , number of confusion iterations: m

Output: The suspect poisoned dataset $D_{suspect}^*$

```

1:  $\theta_{ct} = \mathcal{A}(\tilde{D})$ 
2: for  $i = 1, \dots, m$  do ▷ Confusion training
3:    $(\tilde{X}_i, \tilde{Y}_i) \leftarrow$  a random batch from  $\tilde{D}$ 
4:    $(X'_i, Y'_i) \leftarrow$  a random batch from  $D_{clean}^*$ 
5:    $Y^* \leftarrow$  random mislabels other than  $Y'_i$ 
6:    $\mathcal{L}_{ct} \leftarrow \frac{\mathcal{L}(f(\tilde{X}_i; \theta_{ct}), \tilde{Y}_i) + (\lambda - 1)\mathcal{L}(f(X'_i; \theta_{ct}), Y^*)}{\lambda}$ 
7:    $\theta_{ct} \leftarrow$  one step gradient descent on  $\mathcal{L}_{ct}$ 
8:  $D_{suspect}^* \leftarrow \tilde{D}$  ▷ Poisoned sample identification
9: for  $i=1, \dots, n$  do
10:   if  $\tilde{y}_i \neq F(\tilde{x}_i; \theta_{ct})$  then
11:      $D_{suspect}^* \leftarrow$  remove  $(\tilde{x}_i, \tilde{y}_i)$  from  $D_{suspect}^*$ 
12: return  $D_{suspect}^*$ 

```

A.2 The Self-supervised Pretrain Within the Robust Training

As shown in Fig. 4(a), under supervised training, all the poisoned samples are clustered together in the latent space. A strong correlation between the *trigger* and the target label (airplane) is constructed in the attacked model to implement the backdoor attack. Conversely, as shown in Fig. 4(b), under self-supervised pre-training, all poisoned data reverts to their original classes in the latent space. Within each cluster, clean samples and poisoned samples are intermixed and the number of poisoned samples in each cluster is significantly lower than that of clean ones. In this case, if we continue with supervised fine-tuning with labels, poisoned samples originally from non-target classes will exhibit inconsistency between their high-dimensional presentations and their labels. Therefore, these poisoned samples can be considered outliers in the poisoned dataset. On this basis, the robust training method enhanced by differential privacy can suppress

the fit of these poisoned samples and guide the defender to distinguish clean samples from the poisoned dataset.

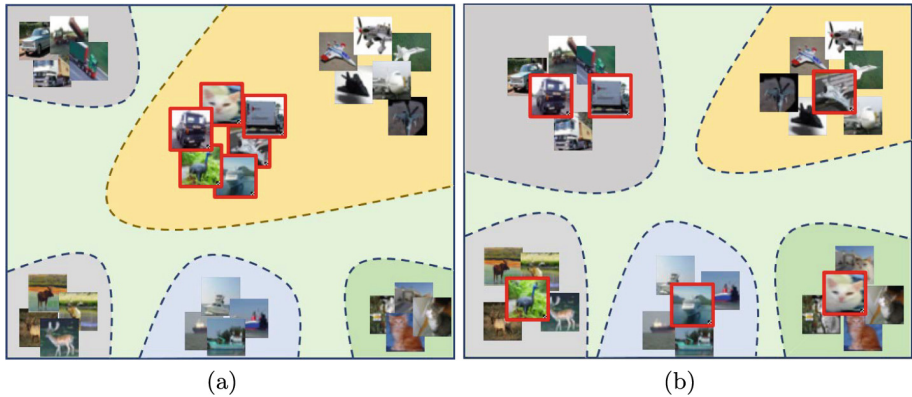


Fig. 4. In the CIFAR10 dataset, part of the samples are presented in a latent space under a BadNets attack. (a) shows the representation of the poisoned samples and clean samples under supervised training with the target class (airplane), (b) shows the corresponding representations of these samples under the self-supervised pre-training.

References

1. Abadi, M., et al.: Deep learning with differential privacy. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, 24–28 October 2016, pp. 308–318. ACM (2016)
2. Chen, T., Kornblith, S., Norouzi, M., Hinton, G.: A simple framework for contrastive learning of visual representations. In: International Conference on Machine Learning, pp. 1597–1607. PMLR (2020)
3. Du, M., Jia, R., Song, D.: Robust anomaly detection and backdoor attack detection via differential privacy. In: 8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, 26–30 April 2020. OpenReview.net (2020)
4. Gao, K., Bai, Y., Gu, J., Yang, Y., Xia, S.: Backdoor defense via adaptively splitting poisoned dataset. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, 17–24 June 2023, pp. 4005–4014. IEEE (2023)
5. Gu, T., Dolan-Gavitt, B., Garg, S.: Badnets: identifying vulnerabilities in the machine learning model supply chain. CoRR abs/1708.06733 (2017). <http://arxiv.org/abs/1708.06733>
6. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778 (2016)
7. Hong, S., Chandrasekaran, V., Kaya, Y., Dumitras, T., Papernot, N.: On the effectiveness of mitigating data poisoning attacks with gradient shaping. CoRR abs/2002.11497 (2020). <https://arxiv.org/abs/2002.11497>
8. Houben, S., Stallkamp, J., Salmen, J., Schlipsing, M., Igel, C.: Detection of traffic signs in real-world images: the German traffic sign detection benchmark. In: International Joint Conference on Neural Networks. No. 1288 (2013)

9. Huang, K., Li, Y., Wu, B., Qin, Z., Ren, K.: Backdoor defense via decoupling the training process. In: The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, 25–29 April 2022. OpenReview.net (2022)
10. Krizhevsky, A.: Learning multiple layers of features from tiny images. Technical report (2009)
11. Lecuyer, M., Atlidakis, V., Geambasu, R., Hsu, D., Jana, S.: Certified robustness to adversarial examples with differential privacy. In: 2019 IEEE Symposium on Security and Privacy (SP), pp. 656–672. IEEE (2019)
12. Li, Y., Lyu, X., Koren, N., Lyu, L., Li, B., Ma, X.: Anti-backdoor learning: training clean models on poisoned data. In: Ranzato, M., Beygelzimer, A., Dauphin, Y.N., Liang, P., Vaughan, J.W. (eds.) *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021*, NeurIPS 2021, 6–14 December 2021, virtual, pp. 14900–14912 (2021)
13. Li, Y., Lyu, X., Koren, N., Lyu, L., Li, B., Ma, X.: Neural attention distillation: Erasing backdoor triggers from deep neural networks. In: 9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, 3–7 May 2021. OpenReview.net (2021)
14. Liu, S., Cullen, A.C., Montague, P., Erfani, S.M., Rubinstein, B.I.: Enhancing the antidote: improved pointwise certifications against poisoning attacks. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, pp. 8861–8869 (2023)
15. Liu, Y., et al.: Trojaning attack on neural networks. In: 25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, 18–21 February 2018. The Internet Society (2018)
16. Ma, Y., Zhu, X., Hsu, J.: Data poisoning against differentially-private learners: attacks and defenses. In: *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, 10–16 August 2019*, pp. 4732–4738. ijcai.org (2019)
17. Naseri, M., Hayes, J., Cristofaro, E.D.: Local and central differential privacy for robustness and privacy in federated learning. In: 29th Annual Network and Distributed System Security Symposium, NDSS 2022, San Diego, California, USA, 24–28 April 2022. The Internet Society (2022)
18. Nguyen, T.D., et al.: {FLAME}: taming backdoors in federated learning. In: 31st USENIX Security Symposium (USENIX Security 2022), pp. 1415–1432 (2022)
19. Papernot, N., Song, S., Mironov, I., Raghunathan, A., Talwar, K., Erlingsson, Ú.: Scalable private learning with PATE. In: 6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, 30 April–3 May 2018, Conference Track Proceedings. OpenReview.net (2018)
20. Qi, X., Xie, T., Li, Y., Mahlouljifar, S., Mittal, P.: Revisiting the assumption of latent separability for backdoor defenses. In: *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, 1–5 May 2023*. OpenReview.net (2023)
21. Qi, X., Xie, T., Wang, J.T., Wu, T., Mahlouljifar, S., Mittal, P.: Towards a proactive ML approach for detecting backdoor poison samples. In: 32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, 9–11 August 2023, pp. 1685–1702. USENIX Association (2023)
22. Tang, D., Wang, X., Tang, H., Zhang, K.: Demon in the variant: statistical analysis of DNNs for robust backdoor contamination detection. In: Bailey, M.D., Greenstadt, R. (eds.) *30th USENIX Security Symposium, USENIX Security 2021*, 11–13 August 2021, pp. 1541–1558. USENIX Association (2021)

23. Tran, B., Li, J., Madry, A.: Spectral signatures in backdoor attacks. In: Bengio, S., Wallach, H.M., Larochelle, H., Grauman, K., Cesa-Bianchi, N., Garnett, R. (eds.) *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, 3–8 December 2018, Montréal, Canada*, pp. 8011–8021 (2018)
24. Turner, A., Tsipras, D., Madry, A.: Label-consistent backdoor attacks. CoRR abs/1912.02771 (2019). <http://arxiv.org/abs/1912.02771>
25. Wang, B., et al.: Neural cleanse: identifying and mitigating backdoor attacks in neural networks. In: *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, 19–23 May 2019*, pp. 707–723. IEEE (2019)
26. Yan, Y., Ye, Q., Hu, H., Chen, R., Han, Q., Wang, L.: Towards defending against byzantine LDP amplified gain attacks. In: Wang, X., et al. (eds.) *DASF AA 2023. LNCS*, vol. 13943, pp. 627–643. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-30637-2_42



Private Decision Tree Evaluation with Malicious Security via Function Secret Sharing

Jiaxuan Fu, Ke Cheng^(✉), Yuheng Xia, Anxiao Song, Qianxing Li,
and Yulong Shen

School of Computer Science and Technology, Xidian University, Xi'an, China
{jiaxuanfu,yuhengxia,songanxiao,qianxing.li}@stu.xidian.edu.cn,
chengke@xidian.edu.cn, ylshen@mail.xidian.edu.cn

Abstract. Private Decision Tree Evaluation (PDTE) allows the client to use a decision tree classification model on the server to classify their private data, without revealing the data or the classification results to the server. Recent advancements in PDTE have greatly enhanced its effectiveness in scenarios involving semi-honest security, offering a viable and secure alternative to traditional, less secure approaches for decision tree evaluation. However, this model of semi-honest security may not always align well with real-world problems. In this work, we present FSSTree, a malicious-secure three-party PDTE protocol using function secret sharing (FSS). FSSTree achieves its high performance against malicious adversaries via several innovative cryptographic designs. Especially, 1) we transform a comparison operation into a prefix parity query problem, allowing us to implement malicious-secure comparisons rapidly using lightweight and verifiable FSS. 2) Building upon this, we further propose a constant-round protocol for securely evaluating Conditional Oblivious Selection (COS). 3) We utilize these optimized protocols to enhance the PDTE processes, achieving a considerable decrease in both communication costs and the number of rounds. The experimental results show that FSSTree reduces the runtime in a WAN environment by up to $22.3\times$ times and saves up to $4.1\times$ the communication cost compared to the state-of-the-art work.

Keywords: Private Decision Tree Evaluation · Function Secret Sharing · Malicious Model

1 Introduction

Decision trees are popular in machine learning for applications like e-commerce recommendations, spam filtering, and medical diagnostics. In a cloud-assisted service, a model provider can deploy a trained decision tree model in the cloud for remote client evaluation. This offers benefits like scalability, ubiquitous access, and cost efficiency. However, outsourcing decision tree evaluation to the cloud

Table 1. Comparison of PDTE Protocols

Protocol	Parties	Security	Feature Selection	Conditional Selection	Communication Costs*	Rounds*
Liu et al. [20]	2PC	○	-	HE, MT	$O(2^d \ell)$	$O(d)$
Zheng et al. [29]	2PC	○	MT	MT, SS	$O(2^d n \ell)$	$O(\ell)$
Ma et al. [21]	2PC	●	OT	GC, OT	$O(dn\ell)$	$O(d)$
Ji et al. [16]	3PC	○	FSS	FSS	$O(d(\log m + \log n + 2\ell))$	$O(d)$
Bai et al. [3]	3PC	●	RSS	RSS, FSS	$O(dn\ell)$	$O(d\ell)$
Ours	3PC	●	RSS	FSS	$O(dn\ell)$	$O(d)$

* Communications and Rounds only statistics online evaluation phase.

○: semi-honest, ●: one-side malicious, ●: malicious, m : the number of tree nodes, n : the number of features, d : the longest depth of a tree, ℓ : the bit-length of feature values. HE: homomorphic encryption, MT: triple multiplication, SS: secret sharing, GC: garbled circuit, OT: oblivious transfer, RSS: replicated secret sharing, FSS: function secret sharing.

raises privacy concerns for both the model and input data. The model must remain confidential, and the model provider should not know the client’s input. This requires Private Decision Tree Evaluation (PDTE) to ensure no information leakage about the client’s input or the model provider’s decision tree.

Recently, many PDTE protocols [3, 16, 20, 21, 29] have been proposed with different security and efficiency trade-offs. As shown in Table 1, most existing protocols [16, 20, 29] assume a semi-honest adversary who follows the protocol honestly. Since this assumption is hard to meet in reality, Bai et al. [3] recently propose Mostree to achieve security against malicious adversaries with sublinear communication, forming a three-party architecture where a malicious adversary can compromise one party, which could be the model provider, client, or assistant computing party. However, as an initial attempt, Mostree is not entirely satisfactory and needs performance optimization. The number of communication rounds in the online phase is linearly related to the number of bits ℓ for value representation, causing significant latency. This is because node feature selection in PDTE involves a secure comparison protocol under a malicious model. Mostree implements these comparisons bit-wise on additive shared data using MSB extraction techniques, requiring $O(\ell)$ communication rounds. In a LAN setting, over 85% of the online phase time overhead in Mostree is due to this malicious secure comparison protocol. This overhead is even more pronounced in a WAN setting, reducing the scheme’s practicality. To address this, a new PDTE scheme under a malicious model with fewer communication rounds is needed to enhance efficiency and practicality.

1.1 Related Works

Semi-honest PDTE Schemes. Most previous works focus on PDTE against semi-honest adversaries. Existing works [4, 15, 20, 27] use homomorphic encryption (HE) for decision tree evaluation, but the high computational and communication costs make it impractical. Tai et al. [25] introduce the concept of path cost to reduce communication overhead without traversing the entire tree.

Subsequent works [11, 17, 29] improved computational efficiency further. Ji et al. [16] achieve efficient decision tree evaluation using Function Secret Sharing (FSS), based on distributed point functions (DPF) and distributed comparison functions (DCF), implementing oblivious transfer (OT) and conditional oblivious transfer (COT) protocols with optimal communication complexity. However, this work only defends against semi-honest adversaries.

Malicious PDTE Schemes. Recently, PDTE schemes against malicious adversaries have gained attention. The Works [21, 25, 27] focus on protecting models from malicious clients. Some works [25, 27] use Zero-Knowledge Proofs (ZKPs) to detect malicious input, while Ma et al. [21] use the malicious-secure Garbled Circuits (GC) protocol [2]. Damgaard et al. [10] propose a PDTE protocol using SPDZ in a dishonest majority setting. The work most closely related to ours is Mostree [3], which is the first PDTE protocol to achieve sublinear communication and malicious security in a three-party honest-majority setting. Mostree uses DPF and replicated secret sharing (RSS) for oblivious selection protocols with sublinear communication costs, combined with lightweight consistency checks for a secure PDTE protocol. However, Mostree mainly optimizes oblivious selection under the malicious model, without improving the time-consuming comparison operations in decision trees, leaving room for performance improvement.

1.2 Our Contributions

This paper proposes FSSTree, a malicious-secure PDTE protocol using function secret sharing (FSS). Our design follows the same architecture of Mostree [3] and also works in the three-party honest-majority setting [1, 13, 19, 22], yet with significant optimizations to achieve largely boosted performance compared to the state-of-the-art work. Our contributions are summarized as follows:

- First, we transform a comparison operation into a prefix parity query problem, and based on this, propose a verifiable comparison protocol (Π_{VCMP}) with plaintext input. The protocol implements malicious-secure comparisons rapidly using lightweight and verifiable FSS, which do not make use of any public key operations or arithmetic MPC.
- Second, by providing the lightweight Π_{VCMP} protocol, we further propose a secure conditional oblivious selection protocol (Π_{COS}) that achieves malicious security with constant communication rounds. We carefully combine the proposed Π_{COS} protocol and consistency check mechanism of replicated secret sharing (RSS), to design FSSTree, achieving a considerable decrease in both communication costs and the number of rounds.
- Third, we formally prove the security of all our protocols using the proof-by-simulation technique [18]. We implement FSSTree and evaluate its performance over various public datasets. Experiments demonstrate that FSSTree can obtain better performance in time and communication overhead. Compared with Mostree, the online runtime over a WAN environment is up to $22.3\times$ better. In the meantime, FSSTree offers up to $4.1\times$ savings in communication costs.

2 Preliminaries

In this section, we initially introduce decision tree evaluation, followed by an introduction to various secret-sharing primitives and their foundational computation protocols. Table 2 lists the main notations utilized throughout our work.

Table 2. Notation Descriptions

Notations	Definitions
$\llbracket x \rrbracket$	$\binom{n}{n}$ -additive secret shares of the value x
$\llbracket x \rrbracket_b$	the $\binom{n}{n}$ -shares of x stored in party P_b
$\langle x \rangle$	$\binom{3}{2}$ -additive secret shares of the value x
$\langle x \rangle_b = (\llbracket x \rrbracket_b, \llbracket x \rrbracket_{b+1})$	the $\binom{3}{2}$ -shares of x stored in party P_b
ℓ	the bit length of the value x
λ	the security parameter of FSS

2.1 Decision Tree Evaluation

Decision tree evaluation utilizes a binary tree structure for classification, integrating decision nodes and leaf nodes. Each decision node contains a feature attribute and a threshold, guiding the evaluation process. Starting from the root, the feature vector is compared to these thresholds at each node, directing the path to the left child for values that surpass the threshold and to the right for others. This traversal continues until a leaf node is reached, which bears a classification label. This label, generated in the leaf, signifies the classification result of the input feature vector.

2.2 Additive Secret Sharing

Sharing Semantics. Our scheme works on a three-party computation (3PC) model where parties $P = (P_0, P_1, P_2)$ are connected by bidirectional synchronous channels. To further elucidate, we use the following sharing semantics:

$\binom{n}{n}$ -sharing $\llbracket x \rrbracket^n$. A secret value $x \in \mathbb{Z}_{2^\ell}$ is said to be $\llbracket \cdot \rrbracket$ -shared among n parties, if party P_b for $b \in \mathbb{Z}_n$ holds $\llbracket x \rrbracket_b^n = x_b$ such that $x = \sum_{i=0}^{n-1} x_i$. For the brevity of the description, we omit the corner n when there is no ambiguity. We use $n = 2$ and 3 in this paper.

$\binom{3}{2}$ -sharing $\langle x \rangle$ (a.k.a. Replicated Secret Sharing, RSS). A secret value $x \in \mathbb{Z}_{2^\ell}$ is said to be $\langle \cdot \rangle$ -shared among P , if party P_b for $b \in \mathbb{Z}_3$ holds $\langle x \rangle_b = (\llbracket x \rrbracket_b, \llbracket x \rrbracket_{b+1})$.

We extend the above definitions to vectors, where $\vec{x} \in \mathbb{Z}^m$ denotes an m -dimensional vector. Accordingly, we use $\llbracket \vec{x} \rrbracket$ and $\langle \vec{x} \rangle$ to denote $\binom{n}{n}$ -sharing and

$(\frac{3}{2})$ -sharing of a vector $\vec{x}$, respectively. Without special declaration, all computations are performed in $\mathbb{Z}_{2^\ell}$. For brevity, we omit the notation $(\text{mod } 2^\ell)$. Note that the additive sharing schemes discussed above can be seamlessly expanded to handle Boolean-type data, which we refer to as Boolean sharing.

Semi-honest Protocols. $(\frac{3}{2})$ -sharing mainly includes the following (semi-honest) computation protocols:

- **Reshare:** Given $(\frac{3}{2})$ -shared values $\llbracket x \rrbracket$, to obtain $\langle x \rangle$, P_b for $b \in \mathbb{Z}_3$ locally computes $x'_b = x_b + r_b$, where $r_0 + r_1 + r_2 = 0$ and the auxiliary parameter r_b can be generated offline as [26]. Then, P_b sends x'_b to P_{b-1} and sets $\langle x \rangle_b = (x'_b, x'_{b+1})$.
- **Add:** Given secret shares $\langle x \rangle$ and $\langle y \rangle$, to obtain $\langle x + y \rangle$, P_b for $b \in \mathbb{Z}_3$ locally computes $\langle x + y \rangle_b = (x_b + y_b, x_{b+1} + y_{b+1})$. For a public constant c , $\langle x \rangle + c$ can be computed as $(x_0 + c, x_1, x_2)$ where P_b can compute its share locally.
- **Mul:** Given secret shares $\langle x \rangle$ and $\langle y \rangle$, to obtain $\langle xy \rangle$, P_b for $b \in \mathbb{Z}_3$ locally computes $z_b = x_b y_b + x_{b+1} y_b + x_b y_{b+1}$ so that $z = z_0 + z_1 + z_2 = xy$ and z is $(\frac{3}{2})$ -shared. Then, parties invoke **Reshare** with inputting $\llbracket z \rrbracket$ to get $\langle z \rangle = \langle xy \rangle$. Additionally, for a public constant c , $c \cdot \langle x \rangle$ can be computed as $(c \cdot x_0, c \cdot x_1, c \cdot x_2)$ where P_b can compute its share locally.

Malicious Protocols. Our work uses the following assumed ideal functionalities to achieve malicious security:

- $\mathcal{F}_{\text{mul}}^{\llbracket \cdot \rrbracket}$: Given $(\frac{3}{2})$ -shared $\llbracket x \rrbracket, \llbracket y \rrbracket$, share $\llbracket x \cdot y \rrbracket$ between two parties.
- $\mathcal{F}_{\text{mul}}^{\langle \cdot \rangle}$: Given secret shares $\langle x \rangle, \langle y \rangle$, share $\langle x \cdot y \rangle$ between three parties.
- $\mathcal{F}_{\text{rand}}$: Sample a random value $r \leftarrow \mathbb{Z}$ and share $\langle r \rangle$ between three parties.
- $\mathcal{F}_{\text{share}}$: Inputting a secret x held by P_b , share $\langle x \rangle$ between three parties.
- $\mathcal{F}_{\text{recon}}$: Given a $(\frac{3}{2})$ -shared $\langle x \rangle$ and a party index b , reveal x to P_b .
- $\mathcal{F}_{\text{open}}$: Given a $(\frac{3}{2})$ -shared $\langle x \rangle$, reveal x to all three parties.
- $\mathcal{F}_{\text{os}}$: Given a $(\frac{3}{2})$ -shared index $\langle i \rangle$ and a $(\frac{3}{2})$ -shared list $\langle L \rangle$, share $\langle L[i] \rangle$ between three parties.

All these functionalities can be securely computed with malicious security using well-established protocols [3, 9, 13, 19, 28]. Furthermore, whenever a three-party RSS-based secure computation (referred to as 3PC), is used in a black-box manner, we simplify it to the 3PC ideal function $\mathcal{F}_{3\text{PC}}^{\mathbb{Z}}$.

2.3 Function Secret Sharing

A Function Secret Sharing (FSS) scheme [5, 6] for a function family $\mathcal{F}$ splits a function $f(x) \in \mathcal{F}$ into two additive shares $f_0(x)$ and $f_1(x)$, such that each share hides f and ensures that $\forall x \in \mathbb{G}^{\text{in}}, f_0(x) + f_1(x) = f(x)$.

Definition 1 (FSS: Syntax [5, 6]). A (two-party) FSS scheme is a pair of probabilistic polynomial-time (PPT) algorithms $\{\text{Gen}(\cdot), \text{Eval}(\cdot)\}$ such that:

- $\text{Gen}(1^\lambda, f)$ is a key generation algorithm that, given the security parameter λ and the description of a function f , outputs a pair of functional keys (k_0, k_1) . Each key is capable of efficiently describing f_0 and f_1 without revealing f .
- $\text{Eval}(b, k_b, x)$ is a polynomial-time evaluation algorithm that, given the party index $b \in \{0, 1\}$, the key k_b , and the input x , it outputs an additive share $f_b(x)$, such that $f_0(x) + f_1(x) = f(x)$.

(k_0, k_1) are called FSS keys. The number of bits required to store an FSS key is called the *key size*. In this work, we focus on the Distributed Point Function (DPF), an FSS scheme for point functions that evaluate a non-zero value at exactly one index in their domain. Formally, such a function is defined as follows:

$$f_{\alpha, \beta}^\bullet(x) = \begin{cases} \beta, & x = \alpha \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

2.4 Verifiable Distributed Point Function (VDPF)

Since the party responsible for generating and distributing DPF keys may be malicious, it may generate incorrect keys. To prevent this, the Verifiable DPF (VDPF) [8] provides an additional verifiable attribute defined as follows:

Definition 2 (*Verifiable DPF*). A (two-party) verifiable distributed point function (VDPF) scheme is a set of probabilistic polynomial-time (PPT) algorithms $\{\text{Gen}(\cdot), \text{BVEval}(\cdot), \text{Verify}(\cdot)\}$ such that:

- $\text{VDPF.Gen}(1^\lambda, f_{\alpha, \beta}^\bullet) \rightarrow (k_0, k_1)$ is a key generation algorithm that, given the security parameter λ and the description of a point function $f_{\alpha, \beta}^\bullet$, outputs a pair of functional keys (k_0, k_1) .
- $\text{VDPF.BVEval}(b, k_b, \{x_i\}_{i \in \{1, \dots, L\}}) \rightarrow (\llbracket y_b^{(x_i)} \rrbracket_{i \in \{1, \dots, L\}}, \pi_b)$ is a polynomial-time evaluation algorithm that, given a batch of L inputs $\{x_i\}_{i \in \{1, \dots, L\}}$, outputs a tuple of values. The first set of values are the FSS outputs, where $\llbracket y_b^{(x_i)} \rrbracket = f_{\alpha, \beta}^\bullet(x_i)$. The second output is a proof π_b that is used to verify the well-formedness of the output.
- $\text{VDPF.Verify}(\pi_0, \pi_1) \rightarrow (\text{Accept/Reject})$ is an algorithm that, given a pair of proofs π_0 and π_1 , outputs either *Accept* or *Reject*. The output should only be *Accept* if $y_0 + y_1$ defines the truth table of some point function, which occurs if it is non-zero in at most one location.

3 Secure Conditional Selection

In the PDTE, the most crucial step is conditional selection based on feature value f and threshold t . For each decision node, the formal definition of conditional selection is as follows:

$$\text{next} = \begin{cases} l, & f \geq t \\ r, & \text{otherwise} \end{cases} \quad (2)$$

where l and r represent its left and right child index.

In most previous schemes, comparing feature values and thresholds usually requires a loop related to the bit length ℓ with multiple rounds of communication. This approach is significantly affected by network latency, causing performance degradation. Therefore, our design aims to reduce the number of communication rounds during protocol execution. In this section, we first propose a verifiable comparison protocol with plaintext input that implements the comparison of feature values and thresholds. Based on this, we further design a constant-round secure verifiable conditional oblivious selection protocol that hides the access patterns when selecting the child index.

3.1 Verifiable Comparison Protocol with Plaintext Input

We first propose a verifiable comparison protocol (Π_{VCMP}) in the 2PC setting with plaintext input, which will be a key component of the subsequent algorithms. Our strategy for verifiable comparison is to convert the comparison into a prefix parity query and then use the VDPF scheme [8] for verification.

Given a bitstring $x = x_0x_1 \cdots x_{n-1}$ of length n , assume there is a substring $x_ax_{a+1} \cdots x_{b-1}$ of x , denoted $x[a, b)$, where $0 \leq a < b \leq n$. The *parity query* for $x[a, b)$ is defined as:

$$\text{parity}(x[a, b)) := \bigoplus_{i=a}^{b-1} x_i$$

When comparing a public value x with a public threshold t , if $x < t$, x always appears to the left of t on the number line. For a one-hot encoding of x in $\mathbb{Z}_{2^\ell}$, denoted $\hat{\alpha} = \alpha_0\alpha_1 \cdots \alpha_{\ell-1}$, where $\hat{\alpha}$ is all 0s except for a 1 at α_x , the equation $\alpha_0 \oplus \alpha_1 \oplus \cdots \oplus \alpha_{t-1} = 1$ holds. Thus, we can transform a comparison of plaintext values into a prefix parity query on $\hat{\alpha}$. A recent work [24] proposes a *parity-segment tree* data structure that answers parity queries with complexity $O(\ell)$ and extends this to the DPF scheme, allowing comparison functionality through the DPF scheme.

Inspired by the parity-segment tree and VDPF [8], we propose the verifiable comparison protocol with plaintext input to compute the following function:

$$f_{y,1}^>(x) = \begin{cases} 1, & x > y \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

We abbreviate the above equation as $1\{x > y\}$. The details of our protocol are shown in Algorithm 1. Note that the variables CW , s , and t in the algorithm follow the original definitions in [6], representing correcting words, output seeds, and disagreeing bits, respectively.

Our protocol requires a pair of verified DPF keys $k_b, b \in \mathbb{Z}_2$, which we generate by invoking VDPF.Gen and running the DPF key verification protocol in [8]. In the key evaluation phase, our protocol's execution logic is similar to the DPF.Eval algorithm, with the addition of res_b and d , representing an updated parity value and the current tree traversal direction. The protocol evaluates the verified key following the traversal method in [24] and outputs the value of $f_{y,1}^>(x)$

in Boolean-shared form. Using the VDPF key generation and verification process ensures the protocol can detect malformed shared values when the evaluator is semi-honest, as defined below:

Algorithm 1. Verifiable Comparison Protocol with Plaintext Input (Π_{VCMP})

Input: P_b for $b \in \{0, 1\}$ holds a verified DPF key k_b for the point function $f_{y,1}^\bullet$ and a public index x . Let $\text{Convert}_G : \{0, 1\}^\lambda \rightarrow \mathbb{G}$ be a map converting a random λ -bit string to a pseudorandom group element of $\mathbb{G}$.

Output: P_b outputs res_b where $\text{res}_0 \oplus \text{res}_1 = f_{y,1}^\bullet(x)$

P_b for $b \in \{0, 1\}$ does:

Let $x = (x_1, \dots, x_\ell) \in \{0, 1\}^\ell$ be the bit decomposition

Parse $k_b = s^{(0)} || t^{(0)} || CW^{(1)} || \dots || CW^{(\ell+1)}$

$\text{res}_b \leftarrow 0, d \leftarrow 0$

for $i \in \{1, \dots, \ell\}$ **do**

Parse $CW^{(i)} = s_{CW} || t_{CW}^L || t_{CW}^R$

$\tau^{(i)} \leftarrow G(s^{(i-1)}) \oplus (t^{(i-1)}) \cdot [s_{CW} || t_{CW}^L || s_{CW} || t_{CW}^R]$

Parse $\tau^{(i)} = s^L || t^L || s^R || t^R \in \{0, 1\}^{2(\lambda+1)}$

if $x_i == 0$ **then**

$s^{(i)} \leftarrow s^L, t^{(i)} \leftarrow t^L$

else

$s^{(i)} \leftarrow s^R, t^{(i)} \leftarrow t^R$

end if

if $d \neq x_i$ **then**

$d = x_i$

$\text{res}_b = \text{res}_b \oplus t^{(i-1)}$

end if

end for

if $d == 1$ **then**

$\text{res}_b = \text{res}_b \oplus t^{(\ell)}$

end if

return res_b

Definition 3 (*Verifiable Comparison Share Integrity, or Security Against Malicious Share Generation*). Let k_b be the (possibly maliciously generated) share received by the server P_b . For an adversarially chosen set of inputs $\{x_i\}_{i=1}^\eta$, let $(\{y_b^{(x_i)}\}_{i=1}^\eta, \pi_b) \leftarrow \text{VDPF.BVEval}(b, k_b, \{x_i\}_{i=1}^\eta)$ and $\{z_b^{(x_i)}\}_{i=1}^\eta \leftarrow \Pi_{\text{VCMP}}(b, k_b, \{x_i\}_{i=1}^\eta)$. We say that Π_{VCMP} is secure against malicious share generation if the following holds with all but negligible probability over the adversary's choice of randomness. If $\text{VDPF.Verify}(\pi_0, \pi_1)$ outputs *Accept*, then the values $z_0^{(x_i)} + z_1^{(x_i)}$ must satisfy:

$$z_0^{(x_i)} + z_1^{(x_i)} = \begin{cases} 1, & x_i > p \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

It should be noted that the result of the above protocol is a bit of $(\frac{2}{2})$ -Boolean sharing, and further, we need to execute a bit injection $\mathcal{F}_{\text{B2A}}$ to convert it to

($\frac{2}{2}$)-Additive sharing. Due to space limitations, please refer to the full version [12] for details of the Π_{B2A} protocol.

3.2 Secure Verifiable Conditional Oblivious Selection Protocol

Next, we present a secure verifiable conditional oblivious selection protocol (Π_{cos}) in the 3PC setting, which consists of two parts: the comparison of feature values and thresholds, and the oblivious selection.

Since the comparison protocol given in the previous section requires public input, we first address how to hide the input to preserve privacy, which allows us to use Π_{VCOMP} in combination with an additive secret sharing scheme. In general, we can add an offset to the public input x , that is, $x' = x + r$. However, this may lead to an over-ring problem, resulting in incorrect results for Π_{VCOMP} . To address this problem, we use the technique from [14, 23] to perform the following transformation:

$$\text{MSB}_\ell(x) = \text{MSB}_\ell(x_0) \oplus \text{MSB}_\ell(x_1) \oplus 1\{y_0 + y_1 \geq 2^{\ell-1}\}$$

where $x = x_0 + x_1$, $y_0 = x_0 \bmod 2^{\ell-1}$, and $y_1 = x_1 \bmod 2^{\ell-1}$, and $\text{MSB}_\ell(x)$ represents the most significant bit extraction for x . The above equation converts $\text{MSB}_\ell(x)$ into extracting the most significant bit of two shared values and a comparison. Note that both y_0 and y_1 are less than $2^{\ell-1}$, so that $y_0 + y_1$ does not lead to any over-ring problems. Using the above equation, we get:

$$\begin{aligned} 1\{f \geq t\} &= \text{MSB}_\ell(\delta = f - t + r) \\ &= \text{MSB}_\ell(\delta) \oplus \text{MSB}_\ell(2^\ell - r) \oplus 1\{y_0 > 2^{\ell-1} - y_1 - 1\} \oplus 1 \end{aligned}$$

where $y_0 = \delta \bmod 2^{\ell-1}$ and $y_1 = (2^\ell - r) \bmod 2^{\ell-1}$. Based on the above equation, we only need to invoke Π_{VCOMP} to compute $1\{y_0 > 2^{\ell-1} - y_1 - 1\}$.

Recall that for the $\langle \cdot \rangle$ -shared values $\langle l \rangle$ and $\langle r \rangle$ in the 3PC setting. P_b and P_{b+1} hold common share values l_{b+1} and r_{b+1} .

This implies that P_b and P_{b+1} can initially perform a secure comparison of f with t to obtain the result $\llbracket \theta \rrbracket$ in ($\frac{2}{2}$)-sharing form. Next, the parties jointly compute $\llbracket next \rrbracket_b^2 = l_{b+1} \cdot \llbracket \theta \rrbracket + r_{b+1} \cdot (1 - \llbracket \theta \rrbracket)$ in an oblivious choice of l_{b+1} or r_{b+1} . This process is repeated for two out of three parties. At the end of the three-round execution, each party holds two ($\frac{2}{2}$)-shares (since each party executes oblivious selection twice with the other two parties). After that, the parties locally sum up the two ($\frac{2}{2}$)-shares. In this manner, three parties jointly produce a ($\frac{3}{3}$)-sharing of $next$, which can be reshared back to an RSS-sharing $\langle next \rangle$. Since the result of the comparison of f and t is the same for each round of execution, the parties will always choose both l or both r , which makes it easy to see that $next$ satisfies Eq. 2. Figure 1 sketches the above process. The details of the Π_{cos} protocol are shown in Algorithm 2.

Next, we show how our design resists malicious attacks. Recall that in the three-party honest-majority setting, there is at most one malicious party, which may add errors in the execution of the protocol or not execute the protocol as

Algorithm 2. Verifiable Conditional Oblivious Selection Protocol (Π_{cos})**[Offline phase]:**

Upon initialization, the party $P_b, b \in \mathbb{Z}_3$ does:

- 1: Randomly sample $\alpha_b \leftarrow \mathbb{Z}_{2^\ell}$ and $\alpha_b^0 \leftarrow \mathbb{Z}_{2^\ell}$
- 2: $\alpha_b^1 = \alpha_b - \alpha_b^0$
- 3: $x_b = 2^\ell - \alpha_b$
- 4: $y_b = x_b \bmod 2^{\ell-1}$
- 5: $(k_0^\bullet, k_1^\bullet) \leftarrow \text{VDPF.Gen}(1^\lambda, 2^{\ell-1} - y_b - 1, 1, \{0, 1\})$
- 6: $c_b = \text{MSB}_n(x_b) \oplus 1$
- 7: Randomly sample c_0 from $\mathbb{Z}_2$
- 8: $c_b^1 = c_b \oplus c_0^0$
- 9: $k_i = c_i || k_i^\bullet$, for $i \in \{0, 1\}$.
- 10: P_b sends $(\alpha_b^0, k_b^0, c_b^0)$ to P_{b+2} and sends $(\alpha_b^1, k_b^1, c_b^1)$ to P_{b+1} .
- 11: P_{b+1} and P_{b+2} jointly run the DPF key verification protocol from [8] to check the well-formness of DPF keys. If the check fails, abort.
- 12: P_{b+1} and P_{b+2} expand its DPF key on domain $\mathbb{Z}_{2^\ell}$ to produce a shared vector $\llbracket \vec{v} \rrbracket$ in the $(\frac{3}{2})$ -sharing. Then, P_{b+1} and P_{b+2} jointly compute:
- 13: $\llbracket t \rrbracket = \sum_{i \in \mathbb{Z}_{2^\ell}} \llbracket \vec{v}[i] \rrbracket$
- 14: $\llbracket s \rrbracket = \llbracket y \rrbracket_b - \sum_{i \in \mathbb{Z}_{2^\ell}} (i \cdot \llbracket \vec{v}[i] \rrbracket)$
- 15: P_{b+1} and P_{b+2} open $\llbracket t \rrbracket, \llbracket s \rrbracket$ then check if $t = 1$ and $s = 0$. If the check fails, abort.

[Online phase]:

Upon receiving the shared messages $\langle l \rangle, \langle r \rangle$ and the shared conditions $(\langle f \rangle, \langle t \rangle)$, the party $P_b, b \in \mathbb{Z}_3$ does:

- 16: $\langle \delta \rangle = \langle f \rangle - \langle t \rangle$
- 17: Set $\langle \alpha_b \rangle = (\alpha_b^0, \alpha_b^1)$, $\langle \alpha_{b+1} \rangle = (0, \alpha_{b+1}^0)$, and $\langle \alpha_{b+2} \rangle = (\alpha_{b+2}^1, 0)$
- 18: **for** $i \in \{0, 1, 2\}$ **do**
- 19: $\langle \delta_i \rangle = \langle \delta \rangle + \langle \alpha_i \rangle$.
- 20: Party P_{i+1} obtains δ_i by invoking $\mathcal{F}_{\text{recon}}(\langle \delta_i \rangle, i + 1)$
- 21: Party P_{i+2} obtains δ_i by invoking $\mathcal{F}_{\text{recon}}(\langle \delta_i \rangle, i + 2)$
- 22: **end for**
- 23: **for** $i \in \{0, 1\}$ **do**
- 24: Parse k_{b+i+1}^i as $c_i || k_i^\bullet$
- 25: $\hat{y}_i^b = \delta_{b+i+1} \bmod 2^{\ell-1}$
- 26: $\text{res}_i^b \leftarrow \Pi_{\text{VCMF}}(k_i^\bullet, \hat{y}_i^b)$
- 27: $\theta_i^b = i \cdot \text{MSB}_n(\delta_{b+i+1}) \oplus c_i \oplus \text{res}_i^b$
- 28: **end for**

The party P_b and $P_{b+1}, b \in \mathbb{Z}_3$ do:

- 29: Obtain $\llbracket \theta_0^b \rrbracket$ by invoking Π_{B2A} with P_b input θ_0^b and P_{b+1} input θ_1^{b+1} .
- 30: $\llbracket \text{next}_0 \rrbracket = \llbracket l \rrbracket_{b+1} \cdot \llbracket \theta_0^b \rrbracket + \llbracket r \rrbracket_{b+1} \cdot (1 - \llbracket \theta_0^b \rrbracket)$
The party P_b and $P_{b+2}, b \in \mathbb{Z}_3$ do:
- 31: Obtain $\llbracket \theta_1^b \rrbracket$ by invoking Π_{B2A} with P_b inputting θ_1^b and P_{b+2} inputting θ_0^{b+1} .
- 32: $\llbracket \text{next}_1 \rrbracket = \llbracket l \rrbracket_b \cdot \llbracket \theta_1^b \rrbracket + \llbracket r \rrbracket_b \cdot (1 - \llbracket \theta_1^b \rrbracket)$
The party $P_b, b \in \mathbb{Z}_3$ does:
- 33: $\llbracket \text{next} \rrbracket = \llbracket \text{next}_0 \rrbracket + \llbracket \text{next}_1 \rrbracket$
- 34: $\langle \text{next} \rangle \leftarrow \text{reshare}(\llbracket \text{next} \rrbracket)$
- 35: **return** $\langle \text{next} \rangle$

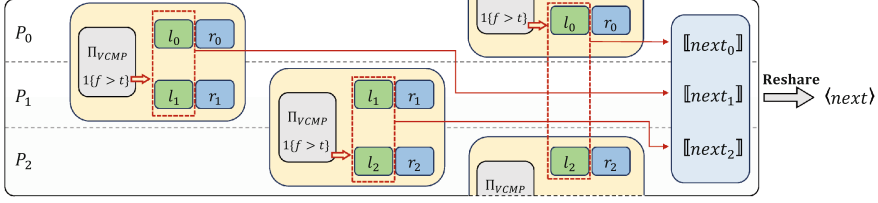


Fig. 1. Our design of conditional oblivious selection.

agreed. For our protocol, multiple malicious attacks can occur in different interactive parts, including corrupted DPF keys, incorrect sharing reconstructions, and additive errors present in the resharing phase.

First, when generating and distributing DPF keys, the key provider P_{b+2} can be corrupted and launch two types of attacks. The corrupted key generator may create an incorrect DPF key that doesn't conform to the protocol, i.e., generate a key for the point function $f_{\alpha, \beta}^\bullet$ with $\beta \neq 1$, which disrupts the Π_{VCMF} protocol as it only works correctly with $\beta = 1$. Additionally, the corrupted party may generate the correct DPF key but share an incorrect index $\alpha^* \neq \alpha$ between P_b and P_{b+1} , leading to an incorrect opening of $\delta^{in} = \delta + \alpha + e$ where $e = \alpha^* - \alpha$, causing incorrect conditional selection. Our protocol uses the VDPF scheme [8] to prevent the generation of incorrect DPF keys. This scheme allows key executors to run a verification protocol to check key correctness, ensuring the DPF keys are well-formed and executed accurately. We use the lightweight verification method from [3] to ensure $\beta = 1$. Specifically, during DPF key full-domain verification, a $\binom{2}{3}$ -shared vector $\llbracket v \rrbracket$ is obtained. By checking if the sum of all elements in $\llbracket v \rrbracket$ equals 1, we confirm $\beta = 1$. Additionally, by computing $\llbracket s \rrbracket = \llbracket \alpha \rrbracket_b - \sum_{i \in \mathbb{Z}_{2^\ell}} (i \cdot \llbracket v[i] \rrbracket)$ and opening $\llbracket s \rrbracket$, then verifying if $s = 0$, we ensure the correctness of α corresponding to the point function.

Second, we notice that even if the DPF keys are correctly generated and α is correctly shared, how to correctly reconstruct the public input δ_{b+2} to P_b and P_{b+1} remains a question. If only P_b and P_{b+1} are involved in the reconstruction of δ_{b+2} , it is impossible to ensure correctness because an adversary who corrupts either P_b or P_{b+1} can always add errors during the reconstruction process. Fortunately, for the reconstruction of δ_{b+2} , the parties can define a consistent $\langle \cdot \rangle$ -sharing of $\langle \alpha \rangle$ from $\llbracket \alpha \rrbracket = (\llbracket \alpha \rrbracket_0, \llbracket \alpha \rrbracket_1)$ as:

$$\langle \alpha \rangle = ((\llbracket \alpha \rrbracket_0, \llbracket \alpha \rrbracket_1), (\llbracket \alpha \rrbracket_1, 0), (0, \llbracket \alpha \rrbracket_0))$$

Therefore, the parties can reconstruct the offset input $\langle \delta_{b+2} \rangle = \langle \delta \rangle + \langle \alpha \rangle$ through $\mathcal{F}_{\text{recon}}$, which ensures consistency.

Finally, a corrupted party may maliciously add errors before resharing the $\binom{3}{3}$ -sharing selected secret, resulting in an additive attack. All the above attacks only compromise correctness, not privacy. For the first two attacks, we have implemented a well-formed consistency verification. So far, our protocol is only

limited to adding errors in the re-sharing phase, which can be verified by pre-computing SPDZ-like MAC values. The details are described in Sect. 4.

4 The Proposed FSSTree Scheme

In this section, we present the FSSTree scheme, where the decision tree encoding is similar to previous work [16]. For a tree with depth d and number of nodes m , we follow the encoding from [3] to hide data access patterns. During privacy-preserving evaluation, once a leaf node is reached, the server repeatedly accesses itself until the protocol reaches d steps, treating each leaf node as a decision node with left and right children pointing to itself. This design eliminates the need to fill the tree into a full binary tree with m nodes, saving storage. We encode the decision tree as a multi-dimensional array $\vec{T}$, where each element represents a node. The i -th node contains five values: left child index l , right child index r , feature selection vector $\vec{f}$, threshold value t , and classification value c , denoted as $T[i] \leftarrow l||r||\vec{f}||t||c$. Note that we use a selection vector (for index k , its selection vector $\vec{k}$ comprises all 0s except for a single 1 at k , this vector is also widely used in previous work [3, 17]) to encode the feature value index, since the input features are generally not excessive, which is relevant to reduce subsequent computational overhead.

The model provider performs this encoding as it is lightweight and negligible in cost. For a decision node, the classification value is set randomly. For a leaf node, the left/right child indexes point to itself, and the threshold value is random. This encoding ensures correct evaluation since there is only one leaf node per path, and the decision tree must end at a leaf node.

To address errors in the Π_{cos} protocol used in FSSTree, we use SPDZ-like MACs. We pre-compute the MAC values of tree contents $\vec{T}$ during setup to verify consistency in Π_{cos} . Any error affects the relationship between data and its MAC, detectable through the Batch MAC Check protocol Π_{MacCheck} [3].

Algorithm 3 shows the details of the FSSTree scheme. During the setup phase, the model provider encodes the decision tree T and shares it between parties. Upon receiving $\vec{T}$, the parties jointly compute the MAC values for each node in the decision tree $\vec{T}$ and store them in $\vec{M}$. At this point, the parties are ready to proceed with decision tree inference.

In the inference phase, our algorithm consists of three stages: feature selection, node evaluation, and tree traversal. For the feature selection, we start from the root node and use the selection vector $\vec{f}$ to obtain the required feature value x for the current node. For the node evaluation, by invoking the Π_{cos} protocol, we determine the next direction of traversal based on the comparison result between the feature value x and the threshold t . It is important to note that we simultaneously select both the child selection vector and its MAC value, resulting in a pair of selection outcomes $(\vec{\eta}, \vec{\eta}_m)$. The child selection vector $\vec{\eta}$ is used to choose the next node, and we repeat the above steps to achieve tree traversal. At the end of the entire loop, by calling $\mathcal{F}_{\text{MacCheck}}$, we verify all pairs of selections

Algorithm 3. The FSSTree Scheme (Π_{FSSTree})**Input:** A shared tree model $\langle \vec{T} \rangle$, an input feature vector $\langle \vec{F} \rangle$ **Output:** The RSS sharing result $\langle res \rangle$ **[Setup]:**

- 1: The model provider encodes the decision tree T as a array $\vec{T}$. Then the parties jointly invoke $\mathcal{F}_{\text{share}}$ in which the model provider inputs $\vec{T}$.
- 2: The parties jointly invoke $\mathcal{F}_{\text{rand}}$ to get a shared MAC key $\langle \alpha \rangle$, where $\alpha \in \mathbb{Z}_{2^\ell}$.
- 3: **for** $i = 1$ to m **do**
- 4: Compute MACs: $\langle \vec{M} \rangle = \mathcal{F}_{\text{mul}}^{(\cdot)}(\langle \alpha \rangle, \langle \vec{T}[i] \rangle)$
- 5: **end for**
- 6: The parties run $\Pi_{\text{MacCheck}}(\langle \vec{T} \rangle, \langle \vec{M} \rangle)$. Abort if the check fails.
- 7: Output $\langle \vec{T} \rangle, \langle \vec{M} \rangle$.

[Inference]:Upon receiving $\langle \vec{F} \rangle$ from the client, the party $P_b, b \in \mathbb{Z}_3$ does:

- 8: Parties jointly initialize $\langle res \rangle = \langle 0 \rangle$
- 9: Parse $\langle l \rangle || \langle r \rangle || \langle \vec{f} \rangle || \langle t \rangle || \langle c \rangle \leftarrow \langle \vec{T} \rangle[0]$
- 10: Parse $\langle l_m \rangle || \langle r_m \rangle || \langle \vec{f}_m \rangle || \langle t_m \rangle || \langle c_m \rangle \leftarrow \langle \vec{M} \rangle[0]$
- 11: **for** $i = 1$ to d **do**
- 12: $\langle x \rangle = \sum_{j \in [n]} \langle \vec{F}[j] \rangle \cdot \langle \vec{f}[j] \rangle$
- 13: $l_{\text{pack}} = \langle l \rangle || \langle l_m \rangle$
- 14: $r_{\text{pack}} = \langle r \rangle || \langle r_m \rangle$
- 15: $\langle \eta \rangle || \langle \eta_m \rangle = \Pi_{\text{cos}}((l_{\text{pack}}, r_{\text{pack}}), (\langle x \rangle, \langle t \rangle))$
- 16: $\langle T[\eta] \rangle || \langle \vec{M}[\eta] \rangle = \mathcal{F}_{\text{os}}(\langle \eta \rangle, \langle \vec{T} \rangle || \langle \vec{M} \rangle)$
- 17: Parse $\langle l \rangle || \langle r \rangle || \langle \vec{f} \rangle || \langle t \rangle || \langle c \rangle \leftarrow \langle \vec{T}[\eta] \rangle$
- 18: Parse $\langle l_m \rangle || \langle r_m \rangle || \langle \vec{f}_m \rangle || \langle t_m \rangle || \langle c_m \rangle \leftarrow \langle \vec{M}[\eta] \rangle$
- 19: $\langle res \rangle = \langle c \rangle$
- 20: **end for**
- 21: Use Π_{MacCheck} to check each pair of $\langle \eta \rangle || \langle \eta_m \rangle$ from Π_{cos} and $\langle \vec{T}[\eta] \rangle || \langle \vec{M}[\eta] \rangle$ from $\mathcal{F}_{\text{os}}$. Abort if the check fails.
- 22: Invoke $\mathcal{F}_{\text{recon}}$ to reconstruct res to the client.

during the traversal. If there is any MAC mismatch, the protocol aborts. Finally, only the client can obtain the final result res through $\mathcal{F}_{\text{recon}}$.

Security. We formally prove the security of all our protocols using the proof-by-simulation technique, the details can be found in Appendix A.

5 Experiment

5.1 Setup

Our scheme is implemented with Pytorch based on the NssMPCLib¹. Experiments are conducted on a server running Ubuntu 20.04 with an Intel Core i9-10900k @ 3.7 GHz processor, 128 GB RAM, and an NVIDIA GeForce RTX

¹ <https://github.com/XidianNSS/NssMPCLib>.

3090. We use the Linux *tc* tool to simulate LAN (RTT: 0.1 ms, 1 Gbps), MAN (RTT: 6 ms, 100 Mbps), and WAN (RTT: 80 ms, 40 Mbps) networks. FSSTree operates in the secret sharing domain, with data represented as integers over the ring $\mathbb{Z}_{2^{32}}$ and a security parameter $\lambda = 128$. We use public data sets from the UCI repository as listed in Table 3. We scale rational numbers to integer representation with a factor of 10^6 for the computations.

Table 3. Paramters of Public Datasets

Dataset	Depth d	Features n	#(Nodes) m
wine	5	7	23
breast	7	12	43
digits	15	47	337
spambase	17	57	171
diabetes	28	10	787
Boston	30	13	851
MNIST	20	784	4179

Table 4. Runtimes(s) Comparision of Conditional Oblivious Selection Protocol

Input Size	Scheme	LAN			WAN		
		CPU	GPU	A.R.	CPU	GPU	A.R.
10^2	ASS-based [3]	0.16	0.36	0.45	64.08	64.32	0.99
	Ours	0.03	0.07	0.44	3.24	3.35	0.97
10^4	ASS-based [3]	0.60	0.67	0.89	74.01	74.36	1.00
	Ours	3.55	0.08	43.59	5.16	3.18	1.62
10^6	ASS-based [3]	60.06	51.04	1.18	1290.82	1300.81	0.99
	Ours	44.84	3.22	13.91	102.80	72.17	1.42

* A.R. represents the accelerative ratio.

5.2 Evaluation of Verifiable Conditional Oblivious Selection

Since our work focuses on optimizing conditional oblivious selection, we first benchmark the performance of Π_{COS} as the building blocks used for FSSTree. We perform experiments in different network settings with different input sizes and compare them with the ASS-based protocol used in the state-of-the-art [3] to demonstrate the superiority of our protocol. Since the protocol provided in [3] cannot support GPU acceleration, we re-implement the conditional oblivious selection performed in their protocol based on their open-source code (consisting

of one verifiable comparison and one verifiable dot multiplication) using our experimental environment to make a fair comparison.

Table 4 shows the detailed results of the comparison. In most cases, our protocol performs better. Only in the LAN setting is our protocol slower than ASS-based protocols, especially when the input size is 10^4 . This is because our protocol is based on FSS, which has better communication overhead, but also higher computation overhead compared to the ASS-based protocol, and thus does not have an advantage when computing small batches of data in a good network environment. When the network environment is poor or the input size is large, our protocol performs significantly better than the ASS-based protocol. On the other hand, when the computation is accelerated using GPUs, all computations need to be performed on the Video RAM. However, the data exchanged among parties first needs to be transmitted to the CPU RAM, causing substantial IO overhead and thus impacting the performance of the protocol. Since our protocol has constant communication rounds, it is more suitable for acceleration using GPUs. From the experimental results, it can be seen that the ASS-based protocol cannot be accelerated by GPUs in all network settings, while our protocol can be accelerated by GPUs even in the worst network settings.

5.3 Evaluation of FSSTree Scheme

We evaluate the FSSTree scheme using different public datasets and compare it with the state-of-the-art work Mostree. Compared to Mostree, during the key generation phase, we need to perform an additional key generation and full domain evaluation for DPF keys in the conditional oblivious selection operation, which takes 5 minutes for one FSS key. Since the number of comparisons of the PDTE is only related to the depth of the decision tree and only one value is used for comparison at a time, we consider the additional overhead in this phase to be acceptable. Furthermore, our decision tree encoding and sharing phase is the same as in Mostree. Next, we focus on the performance of the online decision tree evaluation phase.

Table 5 illustrates the runtime and communication costs for the online evaluation phase between Mostree and our scheme in the LAN, MAN, and WAN network settings. Our scheme demonstrates a lower runtime across the various network settings than Mostree, particularly in the WAN setting, where our work improves the evaluation runtime by approximately 1.5–22.3× over Mostree in different datasets. Regarding the MNIST and digits datasets, in the LAN setting, our work has only a 1.5× improvement compared to Mostree, as these two datasets have more features and nodes, leading to plenty of multiplication operations. However, when the network environment becomes worse, e.g., in MAN and WAN settings, our work achieves 11.2–22.3× improvement. This is because the feature comparison in Mostree requires a large number of interactions and is thus greatly affected by communication latency. In contrast, our work has constant communication rounds, and its runtime mainly depends on the depth of the decision tree, making it less susceptible to communication delays.

Table 5. Comparison of Runtime (s) and Communication Cost (KB) with Mostree

Datasets	Runtime						Comm. Costs	
	LAN		MAN		WAN			
	Mostree	Ours	Mostree	Ours	Mostree	Ours	Mostree	Ours
wine	0.56	0.05	25.65	1.34	317.79	14.54	22.82	6.32
breast	0.68	0.06	35.96	1.83	443.38	19.87	31.45	8.36
digits	3.72	2.01	77.94	4.69	954.75	41.81	104.32	54.84
spambase	2.01	0.17	87.14	4.33	1081.71	46.72	130.19	74.11
diabetes	3.07	0.27	143.85	7.12	1782.19	76.18	121.89	29.53
Boston	3.42	0.31	154.11	7.59	1909.18	81.56	136.93	37.96
MNIST	6.10	3.88	106.15	9.43	1281.20	56.05	1175.51	1109.53

Regarding communication overhead, since our Π_{COS} protocol features significantly lower communication rounds and costs, and the main overhead of PDTE stems from conditional selection, FSSTree offers better communication efficiency compared to Mostree, which offers up to $4.1\times$ savings in communication costs.

Table 6. Comparison of Runtime(s) with Mostree on CPU and GPU

Dataset	Scheme	LAN			MAN			WAN		
		CPU	GPU	A.R.	CPU	GPU	A.R.	CPU	GPU	A.R.
digits	Mostree	3.72	3.38	1.10	77.94	78.89	0.99	954.75	960.60	0.99
	Ours	2.01	0.69	2.91	4.69	3.98	1.18	41.82	41.54	1.01
MNIST	Mostree	6.10	4.63	1.32	106.15	105.38	1.01	1281.20	1283.30	1.00
	Ours	3.88	1.04	3.73	9.43	5.46	1.73	56.05	55.33	1.01

For digits and MNIST datasets, we accelerated their computation by the GPUs, with the results illustrated in Table 6. Our work achieves an accelerative ratio of $2.9\text{--}3.5\times$ in the LAN setting, while Mostree is only $1.1\text{--}1.3\times$. Whereas in the MAN setting, our scheme can still be accelerated using the GPUs, Mostree suffers from a communication bottleneck and even runs slower using the GPUs. In the WAN setting, neither scheme will have acceleration using GPUs, and the bottleneck at this point comes entirely from communication. However, our scheme has better performance in this setting.

6 Conclusion

In this paper, we propose FSSTree, a secure three-party PDTE protocol under the malicious model, which enables clients and model providers to evaluate

decision trees without revealing both clients' inputs and model privacy. Technically, we design a verifiable comparison protocol with plaintext input, and based on this, propose a constant-round protocol for securely evaluating conditional oblivious selection. This serves as the foundational bedrock for FSSTree, which enhances the PDTE processes to achieve a considerable decrease in both runtime and communication costs. Extensive experiments demonstrated that compared with the state-of-the-art, FSSTree achieves up to $23.5\times$ better online runtime over a WAN setting, as well as enjoys up to $4.1\times$ savings in communication cost.

Acknowledgement. We are grateful to the anonymous reviewers for their invaluable comments. Ke Cheng is the corresponding author. This research was supported by the National Key R&D Program of China (No. 2023YFB3107500), the National Natural Science Foundation of China (No. 62220106004), the Major Research Plan of the National Natural Science Foundation of China (No. 92267204), the Technology Innovation Leading Program of Shaanxi (No. 2022KXJ-093, 2023KXJ-033), Young Talent Fund of Association for Science and Technology in Shaanxi, China (No. 20240138), the Natural Science Basic Research Program of Shaanxi (Program No. 2024JC-YBQN-0701), the Innovation Capability Support Program of Shaanxi (No. 2023-CX-TD-02), the Shenzhen Science and Technology Program (No. CJGJZD20220517142005013), the Key R&D Program of Shandong Province of China (No. 2023CXPT056), and the Fundamental Research Funds for the Central Universities (No. XJSJ23040, ZDRC2202).

A Security Proofs

We first prove the security of Verifiable Comparison Protocol Π_{VCMF} , then we employ the universal composition (UC) theory [7] to prove that the other protocols satisfy the following definition.

Definition 4 (*Three-party Secure Computation [3]*). Let $\mathcal{F}$ be the three-party functionality. A protocol Π securely computes $\mathcal{F}$ with abort in the presence of one malicious party, if for every party P_b corrupted by a probabilistic polynomial time (PPT) adversary $\mathcal{A}$ in the real world there exists a PPT simulator S in the ideal world with F , such that

$$\{\text{IDEAL}_{\mathcal{F}, S(z), b(x_0, x_1, x_2, n)}\} \stackrel{c}{=} \{\text{REAL}_{\Pi, \mathcal{A}(z), b(x_0, x_1, x_2, n)}\}$$

where $x_b \in \{0, 1\}^*$ is the input provided by P_b for $b \in \mathbb{Z}_3$, and $z \in \{0, 1\}^*$ is the auxiliary information that includes the public input length information $\{|x_b|\}_{b \in \mathbb{Z}_3}$. The protocol Π securely computes $\mathcal{F}$ with abort in the presence of one malicious party with statistical error $2^{-\lambda}$ if there exists a negligible function $\mu(\cdot)$ such that the distinguishing probability of the adversary is less than $2^{-\lambda} + \mu(\kappa)$.

A.1 Security Proof of Verifiable Comparison Protocol Π_{VCMF}

We prove that the verifiable comparison protocol Π_{VCMF} in Sect. 3.1 is secure by the following lemma.

Lemma 1 (*Detection of Malicious Function Shares*). *The verifiable plaintext comparison protocol satisfies Definition 3.*

Proof. We prove this lemma by contradiction. According to Lemma 2 and Lemma 3 in [8], when $\text{VDPF.Verify}(\pi_0, \pi_1)$ outputs **Accept**, then the values $y_0^{(x_i)} + y_1^{(x_i)}$ must be a subset of the truth table of the point function $f_{p,1}^\bullet$. Suppose there exists an input $u \in \{x_i\}_{i=1}^n$ such that u does not satisfy Eq. 3. We categorize the discussion based on the values of u , when $u > p$, based on the assumption, we have $z_0^{(u)} + z_1^{(u)} = 0$. However, according to the definition of Π_{VCMF} , we know that $z_0^{(u)} + z_1^{(u)} = \text{parity}(p[0, u])$, since $u > p$, it follows that $\text{parity}(p[0, u]) = z_0^{(u)} + z_1^{(u)} = 1$. This contradicts the assumption that $z_0^{(u)} + z_1^{(u)} = 0$. For the case where $u \leq p$, we can arrive at a similar conclusion, therefore the verifiable plaintext comparison protocol satisfies Definition 3.

A.2 Security Proof of Verifiable Conditional Oblivious Selection Protocol Π_{cos}

Theorem 1. Π_{cos} securely compute $\mathcal{F}_{\text{cos}}$ in the $\{\mathcal{F}_{\text{recon}}, \mathcal{F}_{\text{mul}}^{\llbracket \cdot \rrbracket}\}$ -hybrid model in the presence of a malicious adversary in the three-party honest-majority setting.

Proof. For any PPT adversary $\mathcal{A}$, we construct a PPT simulator $\mathcal{S}$ that can simulate the adversary's view with accessing the functionality $\mathcal{F}_{\text{cos}}$. In the cases where $\mathcal{S}$ aborts or terminates the simulation, $\mathcal{S}$ outputs whatever $\mathcal{A}$ outputs.

Simulating Offline Phase. When the corrupted party P_b is the DPF key generator, $\mathcal{S}$ acts as honest parties P_{b+1} and P_{b+2} and receives a pair of DPF keys from $\mathcal{A}$. $\mathcal{S}$ checks the keys and aborts if they are incorrect. If an honest party generates the keys, $\mathcal{S}$ creates a pair of VDPF keys and samples a random share $r_b \leftarrow \mathbb{Z}_{2^\ell}$, then sends (k_b, r_b) to $\mathcal{A}$. $\mathcal{S}$ runs the VDPF verification protocol with $\mathcal{A}$ and aborts if $\mathcal{A}$ aborts in the verification. All other messages from honest parties to the corrupted one are randomly sampled by the simulator.

The above simulation is indistinguishable from real-world execution. If $\mathcal{A}$ sends incorrect DPF keys, the real-world execution would abort due to VDPF check. In the ideal world, $\mathcal{S}$ can directly check the keys, causing the ideal world to abort with indistinguishable probability. When the corrupted party is the DPF evaluator, $\mathcal{S}$ honestly generates the DPF keys. If $\mathcal{A}$ follows the VDPF key verification protocol, the check will pass; otherwise, it will abort. Thus, $\mathcal{S}$ runs the check protocol with $\mathcal{A}$ as in the real-world protocol, always aborting with indistinguishable probability.

Simulating Online Phase. If the protocol does not abort after the simulation stage, all DPF keys generated by $\mathcal{A}$ in $\mathcal{S}$ are correct. The only issue in the online phase is whether $\mathcal{A}$ follows the protocol honestly and whether $\mathcal{S}$ successfully extracts the error term. We assume that the shared messages $\langle l \rangle$, $\langle r \rangle$ and conditions $\langle f \rangle$, $\langle t \rangle$ are correctly shared/simulated initially, meaning the honest parties hold correct and consistent RSS shares.

For all local computations, $\mathcal{S}$ is easy to simulate. $\mathcal{S}$ only needs to simulate interactions between honest and corrupted parties and abort with indistinguishable probability. Only a few parts in the selection phase need interaction. First, securely reconstructing δ : since $\langle \delta \rangle = \langle f \rangle - \langle t \rangle + \langle \alpha \rangle$ and $\langle f \rangle$, $\langle t \rangle$, and $\langle \alpha \rangle$ are correct RSS sharings, $\langle \delta \rangle$ is consistent. $\mathcal{S}$ checks if $\mathcal{A}$ sends the correct value using P_{b+2} 's share: 1) if $\mathcal{A}$ sends an incorrect share, $\mathcal{S}$ aborts; 2) if correct, $\mathcal{S}$ samples a random $\delta \in \mathbb{Z}_{2^\ell}$, computes shares for P_{b+1} and P_{b+2} from δ and P_b 's RSS shares, and sends them to $\mathcal{A}$. The parties then open δ to $\mathcal{A}$. Second, from $\mathcal{F}_{\text{B2A}}$, which uses $\mathcal{F}_{\text{mul}}^{[1]}$ with security proved in [28]. Lastly, re-sharing $\llbracket \text{next} \rrbracket$ from $\binom{3}{3}$ -sharing to RSS-sharing. $\mathcal{A}$ can add an error, and $\mathcal{S}$ extracts it: $\mathcal{S}$ gets next_i^* from the corrupted party to an honest party and updates P_{b+1} 's share. $\mathcal{S}$ computes the correct next_i (knowing all shares to compute z_i), then computes $\text{res} \leftarrow \text{next}_i - \text{next}_i^*$ and sends res to $\mathcal{F}_{\text{cos}}$, completing the simulation.

Note that in simulating the open step of δ , δ uses shares of honest parties, either simulated or computed from local data. These shares match those of the corrupted party, making the simulation indistinguishable from real execution. For the re-sharing phase, the simulation randomly samples P_{b+2} 's share and sends it to $\mathcal{A}$. In the real protocol execution, this is generated by a PRF F , so the simulation remains computationally indistinguishable due to the PRF's security. Overall, the simulation is indistinguishable from real execution.

A.3 Security Proof of the FSSTree Evaluation Protocols Π_{FSSTree}

Theorem 2. Π_{FSSTree} securely compute $\mathcal{F}_{\text{FSSTree}}$ in the $\{\mathcal{F}_{\text{recon}}, \mathcal{F}_{\text{os}}, \mathcal{F}_{\text{coin}}, \mathcal{F}_{\text{rand}}, \mathcal{F}_{\text{open}}, \mathcal{F}_{\text{cos}}, \mathcal{F}_{\text{mul}}^{(\cdot)}, \mathcal{F}_{\text{checkZero}}\}$ -hybrid model in the presence of a malicious adversary in the three-party honest-majority setting.

The proof of Theorem 2 is similar to the Π_{pdte} protocol in Mostree. Due to space limitations, please refer to the full version [12] for details.

References

1. Araki, T., Furukawa, J., Lindell, Y., Nof, A., Ohara, K.: High-throughput semi-honest secure three-party computation with an honest majority. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp. 805–817 (2016)
2. Asharov, G., Lindell, Y., Schneider, T., Zohner, M.: More efficient oblivious transfer and extensions for faster secure computation. In: Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security, pp. 535–548 (2013)
3. Bai, J., Song, X., Zhang, X., Wang, Q., Cui, S., Chang, E.C., Russello, G.: Mostree: malicious secure private decision tree evaluation with sublinear communication. In: Proceedings of the 39th Annual Computer Security Applications Conference, pp. 799–813 (2023)
4. Bost, R., Popa, R.A., Tu, S., Goldwasser, S.: Machine learning classification over encrypted data. In: NDSS Symposium 2015, pp. 1–14. Internet Society (2015)

5. Boyle, E., Gilboa, N., Ishai, Y.: Function secret sharing. In: Oswald, E., Fischlin, M. (eds.) EUROCRYPT 2015. LNCS, vol. 9057, pp. 337–367. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-46803-6_12
6. Boyle, E., Gilboa, N., Ishai, Y.: Function secret sharing: improvements and extensions. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp. 1292–1303 (2016)
7. Canetti, R.: Security and composition of multiparty cryptographic protocols. *J. Cryptol.* **13**, 143–202 (2000)
8. de Castro, L., Polychroniadou, A.: Lightweight, maliciously secure verifiable function secret sharing. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022. LNCS, vol. 13275, pp. 150–179. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-06944-4_6
9. Chida, K., et al.: Fast large-scale honest-majority MPC for malicious adversaries. *J. Cryptol.* **36**(3), 15 (2023)
10. Damgård, I., Escudero, D., Frederiksen, T., Keller, M., Scholl, P., Volgushev, N.: New primitives for actively-secure mpc over rings with applications to private machine learning. In: 2019 IEEE Symposium on Security and Privacy (SP), pp. 1102–1120. IEEE (2019)
11. De Cock, M., et al.: Efficient and private scoring of decision trees, support vector machines and logistic regression models based on pre-computation. *IEEE Trans. Dependable Secure Comput.* **16**(2), 217–230 (2017)
12. Fu, J., Cheng, K., Xia, Y., Song, A., Li, Q., Shen, Y.: Private decision tree evaluation with malicious security via function secret sharing. Technical report, Xidian University (2024). <https://github.com/BlackFH/FSSTree/blob/main/FSSTree-full.pdf>
13. Furukawa, J., Lindell, Y., Nof, A., Weinstein, O.: High-throughput secure three-party computation for malicious adversaries and an honest majority. In: Coron, J.-S., Nielsen, J.B. (eds.) EUROCRYPT 2017. LNCS, vol. 10211, pp. 225–255. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-56614-6_8
14. Gupta, K., et al.: Sigma: secure GPT inference with function secret sharing. *Cryptology ePrint Archive* (2023)
15. Ishai, Y., Paskin, A.: Evaluating branching programs on encrypted data. In: Vadhan, S.P. (ed.) TCC 2007. LNCS, vol. 4392, pp. 575–594. Springer, Heidelberg (2007). https://doi.org/10.1007/978-3-540-70936-7_31
16. Ji, K., Zhang, B., Lu, T., Li, L., Ren, K.: UC secure private branching program and decision tree evaluation. *IEEE Trans. Dependable Secure Comput.* **20**(4), 2836–2848 (2023)
17. Kiss, Á., Naderpour, M., Liu, J., Asokan, N., Schneider, T.: SoK: modular and efficient private decision tree evaluation. *Proc. Priv. Enhanc. Technol.* **2**, 187–208 (2019)
18. Lindell, Y.: How to simulate it – a tutorial on the simulation proof technique. In: Lindell, Y. (ed.) *Tutorials on the Foundations of Cryptography*. ISC, pp. 277–346. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-57048-8_6
19. Lindell, Y., Nof, A.: A framework for constructing fast MPC over arithmetic circuits with malicious adversaries and an honest-majority. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, pp. 259–276 (2017)
20. Liu, L., Su, J., Chen, R., Chen, J., Sun, G., Li, J.: Secure and fast decision tree evaluation on outsourced cloud data. In: Chen, X., Huang, X., Zhang, J. (eds.) ML4CS 2019. LNCS, vol. 11806, pp. 361–377. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-30619-9_26

21. Ma, J.P., Tai, R.K., Zhao, Y., Chow, S.S.: Let's stride blindfolded in a forest: sublinear multi-client decision trees evaluation. In: NDSS (2021)
22. Mohassel, P., Rindal, P.: ABY3: a mixed protocol framework for machine learning. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications security, pp. 35–52 (2018)
23. Rathee, D., et al.: CrypTFlow2: practical 2-party secure inference. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, pp. 325–342 (2020)
24. Storrier, K., Vadapalli, A., Lyons, A., Henry, R.: Grotto: screaming fast $(2+1)$ -PC or z_{2n} via $(2, 2)$ -DPFs. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, pp. 2143–2157 (2023)
25. Tai, R.K.H., Ma, J.P.K., Zhao, Y., Chow, S.S.M.: Privacy-preserving decision trees evaluation via linear functions. In: Foley, S.N., Gollmann, D., Sneekenes, E. (eds.) ESORICS 2017. LNCS, vol. 10493, pp. 494–512. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-66399-9_27
26. Wagh, S., Tople, S., Benhamouda, F., Kushilevitz, E., Mittal, P., Rabin, T.: Falcon: honest-majority maliciously secure framework for private deep learning. *Proc. Priv. Enhanc. Technol.* **1**, 188–208 (2021)
27. Wu, D.J., Feng, T., Naehrig, M., Lauter, K.: Privately evaluating decision trees and random forests. *Proc. Priv. Enhanc. Technol.* **4**, 335–355 (2016)
28. Xu, G., et al.: VerifyML: obviously checking model fairness resilient to malicious model holder. *IEEE Trans. Dependable Secure Comput.* (2023)
29. Zheng, Y., Duan, H., Wang, C.: Towards secure and efficient outsourcing of machine learning classification. In: Sako, K., Schneider, S., Ryan, P.Y.A. (eds.) ESORICS 2019. LNCS, vol. 11735, pp. 22–40. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-29959-0_2



Review the Cuckoo Hash-Based Unbalanced Private Set Union: Leakage, Fix, and Optimization

Keyang Liu[✉], Xingxin Li[✉], and Tsuyoshi Takagi

The University of Tokyo, Tokyo, Japan
stephenkobylky2022@e.ecc.u-tokyo.ac.jp, lixingxin93@gmail.com,
takagi@mist.i.u-tokyo.ac.jp

Abstract. A Private Set Union (PSU) protocol involves two participants—the sender and receiver—computing the union of their privately held sets, X and Y , and outputs the result to the receiver. PSU protocols are categorized into balanced ($|X| \approx |Y|$) and unbalanced ($|X| \ll |Y|$ or $|X| \gg |Y|$) settings. Tu et al. (CCS 2023) developed the first efficient unbalanced PSU ($|X| \ll |Y|$) protocol using cuckoo hashing and a novel permuted Reversed Private Membership Test.

In this paper, we reassess Tu et al.’s protocol and present a statistical and computational leakage attack targeting their Hash + RPMT framework. We estimate the lower bound of our attack’s success probability and highlight how Tu’s parameter choices lead to leaks. To counter these vulnerabilities, we offer two mitigation strategies with different trade-offs. Finally, we optimize the p-RPMT protocol by introducing a new shuffled-PMT (s-PMT), which eliminates one permutation round at no extra cost.

Keywords: unbalanced private set union · privacy leakage · cuckoo hash · reversed private membership tests

1 Introduction

The Private Set Union (PSU) protocol allows two parties, a sender $\mathcal{S}$ holding set X and a receiver $\mathcal{R}$ holding set Y , to compute the union exclusively for $\mathcal{R}$. In a secure PSU, $\mathcal{S}$ learns nothing about Y while $\mathcal{R}$ only learns $X \setminus Y$ and cannot infer elements in the intersection. PSU protocols can find applications in cyber risk management [1], blacklist aggregation [23], and private identification systems [9], among others.

Existing PSU research [9, 15] primarily focuses on cases with balanced inputs ($|X| \approx |Y|$), where the state-of-the-art methods achieve linear $O(|X| + |Y|)$ communication and computation complexity [27]. However, this performance becomes impractical when the sender and receiver have unequal communication and computational budgets. This concern leads to research on unbalanced PSU. Theoretically, the unbalanced setting includes two cases: **sender-heavy**

($|X| \gg |Y|$) and **receiver-heavy** ($|X| \ll |Y|$). However, The sender-heavy scenario requires a minimum communication cost of $O(|X|)$, offering minimal optimization potential from balanced protocols. Therefore, existing unbalanced PSU protocols [13, 25] specifically target the receiver-heavy setting.

Jia et al. [13] introduced a shuffle-based unbalanced PSU protocol for the above scenario. However, Jia et al.'s protocol will leak the intersection size $|X \cap Y|$ to the sender. Tu et al. [25] proposed a different solution using cuckoo hashing [14] and the permuted Reversed Private Membership Test (p-RPMT). Our research reviews this approach and introduces a leakage attack vector. Our result shows that Tu et al.'s solution will require more security redundancy to prevent our attack, leading to worse performance.

1.1 Related Works

In 2017, Chen et al. [4] initiated the study of unbalanced Private Set Intersection (PSI), achieving linear communication complexity relative to the size of the smaller set and logarithmic to the larger set. Their approach utilized cuckoo hashing to minimize comparison operations and employed fully homomorphic encryption (FHE) to offload computational burdens to the party with the larger set. Subsequent studies [3, 6] further enhanced performance and expanded the input domains. This groundwork [4] also informed solutions for unbalanced PSI-CA [26] and unbalanced circuit-PSI [24] focusing primarily on sender-heavy scenarios. In addition, these works also imply a symmetric adaptation for receiver-heavy cases.

However, translating these designs to an unbalanced PSU has been challenging due to potential leakages of the intersection distribution to the receiver [15]. Recently, Tu et al. [25] drew the idea of shuffling from Jia et al. [13] and introduced a new PSU protocol that addresses the known leakage issues.

In Tu et al.'s PSU protocol, $\mathcal{S}$ uses a Cuckoo hash to map its set X into a vector $\mathbf{x}'$ of size $L = O(|X|)$. $\mathcal{R}$ uses the same hash functions as Cuckoo hash to map its set Y into a hash table with L bins, denoted by $Y' = \{Y'_i\}_{i \in [L]}$. The two parties pad their hash tables with $\perp$ elements and execute an FHE-based RPMT protocol. The RPMT outputs a bit vector $\mathbf{b}$ to $\mathcal{R}$. $\mathbf{b}$ satisfying:

$$\mathbf{b}[i] = \begin{cases} 1 & x'_i \in Y'_i \\ 0 & x'_i \notin Y'_i \end{cases} \forall i \in [L].$$

Finally, $\mathcal{R}$ uses the Oblivious Transfer (OT) protocol [11] with $\mathbf{b}[i]$ as the choice bit to fetch back the output of difference $X \setminus Y$. Kolesnikov et al. [15] pointed out that the $\mathbf{b}$ can reveal the information of set $X \cap Y$ over Y . Thus, Tu et al. [25] introduce a permuted RMPT (p-RPMT). The output of p-RPMT additionally outputs a random permutation π for $\mathcal{S}$ such that $\mathbf{b}[i] = 1 \iff x'_{\pi(i)} \in Y'_{\pi(i)}$.

Dumas et al. [7] introduce a new RPMT solution, which claims to improve the communication complexity and sender's computation complexity with a cost of higher receiver computation costs. Their work did not use the cuckoo hash

for deterministic protocol. However, if non-determinism is acceptable, our attack and fix to the cuckoo-hash-based protocol might open a new door for combining their RPMT with the cuckoo hash to lower the sender's computation burden further.

1.2 Overview of Our Technique

This work reanalyzes the design proposed by Tu et al. [25] and identifies a new attack vector. Without further notation, we use “the PSU protocol” for the unbalanced PSU protocol from [25]. Our findings indicate that this vulnerability could compromise both PSU and PSI protocols that utilize the Cuckoo Hash [14] and Reverse Private Membership Test (RPMT) framework, like receiver-heavy PSI [4] and balanced PSU [9]. We propose two strategies to prevent this attack with different tradeoffs. Additionally, we offer an optimization to the p-RPMT protocol from [25], which reduces one round of the permutation protocol by altering the computation of random values.

Privacy Leakage Caused by Cuckoo Hash. Our attack comes from a critical observation: upon receiving the PSU results, the receiver can reveal a fuzzy version of $\mathbf{b}$ and identify a subset of positions where $\mathbf{b}[i] = 0$. This discovery underscores that the permutation alone is insufficient to prevent leakage to the receiver.

More concretely, the corrupt receiver can utilize $X \setminus Y$, the input set Y , and simple hashes H to derive an index set $I_A = \{h(e) | \forall e \in X \setminus Y, h \in H\}$. From I_A , the receiver can select any subset $J \subset [L] \setminus I_A$, where $|J| = |Y| - |X| + 1$, and calculate $T_J = \{y | y \in Y \ \& \ \exists h \in H, h(y) \in J\}$. The adversary can then assert $|X \cap T_J| \geq 1$. Compared to the PSU ideal function, $\mathcal{R}$ in the PSU protocol can deduce more about the $X \cap Y$ distribution from all possible T_J . The adversary gains non-trivial information if $|T_J| < |Y| - |X| + |X \setminus Y| + 1$ and $|I_A| < |X| - 1$. Besides this statistical leakage, we outline a special case where the adversary can compute $T = \bigcup T_J$ to measure leakage in polynomial time. In this case, if $\frac{|X| - |I_A|}{|T|} \geq \frac{|X| - |X \setminus Y| + 1}{|Y|}$, then the adversary can computationally distinguish the distribution of $X \cap Y$ from the ideal function in polynomial time.

We also analyze the source of the leakage. In the PSU protocol, the cuckoo hash employs k simple hashes [18] to distribute the set X across $L = O(|X|)$ bins while conducting approximately $k|Y|$ membership tests. Each element in X benefits from fewer comparison operations, and all $|X|$ elements share the redundant comparisons arising from the additional $L - |X|$ bins. Consequently, if any subset of $L - m$ bins lacks sufficient elements from Y , it may result in leakage regarding X . Further details and explanations of this leakage are in Sect. 3.3.

Strategies to Avoid the Leakage. We propose two methods to mitigate the above leakage attack: adjusting cuckoo hash parameters and preventing the computation of I_A .

The first solution comes from our attack condition. The leakage happens when $|T_J|$ is small and $|I_A|$ is large. Therefore, we can increase the cuckoo hash table size L and simple hash number k to stagger the two conditions. Our simulations indicate that these adjustments can significantly reduce the probability of a successful attack, though they also increase communication and computation costs compared to the original protocol. Since the success rate for detecting a statistical leakage depends on $|X|$ and $|Y|$, we must ensure the k and L are secure for the given $|X|$ and $|Y|$ before using them.

The second method involves preventing the adversarial receiver $\mathcal{R}$ from computing I_A . We have developed two approaches for this: pre-processing and post-processing. In pre-processing, $\mathcal{S}$ and $\mathcal{R}$ collaboratively process the set Y before constructing the hash table. This process prevents $\mathcal{R}$ from replaying this pre-process with the received results and cannot compute I_A . For instance, using the Oblivious Pseudo-Random Function (OPRF) protocols [17], $\mathcal{R}$ receives the PRF values $F_{sk}(y)$ for all $y \in Y$, while $\mathcal{S}$ holds the PRF key sk . They then utilize $\{F_{sk}(x)\}_{x \in X}$ and $\{F_{sk}(y)\}_{y \in Y}$ in the PSU protocol until the final Oblivious Transfer (OT) step, where $\mathcal{S}$ still inputs X . This approach can resolve the leakage in balanced PSU settings [9] while preserving communication complexity. However, in receiver-heavy scenarios, it introduces a communication overhead of $O(|Y|)$, contradicting the original intent to reduce communication.

In the post-processing approach, $\mathcal{S}$ transforms the result in a one-way manner, preventing $\mathcal{R}$ from obtaining complete information to compute I_A . For example, applying a one-way collision-resistant hash function G to each element in X during the last OT step ensures that $\mathcal{R}$ only receives a hashed union instead of $X \cup Y$. Alternatively, we can use the Hash + RPMT framework in a Private Computing on Set Intersection (PCSI) protocol to calculate a set function like the cardinal sum or the cardinality of the intersection. While this approach compromises the original functionality of the PSU, it offers a valuable alternative for PCSI in receiver-heavy cases.

Optimized P-RPMT. We identify an optimization of the p-RPMT protocol used in the PSU protocol that can reduce computation and communication costs. Notably, this improvement does not relate directly to our attack and can benefit any other protocols utilizing p-RPMT.

In the original p-RPMT by Tu et al., a permuted equality test compares two matrices of size $\alpha \times L$, specifically $f_{ij}(x) + r_{ij}$ against r_{ij} . For security, the p-RPMT protocol requires two rounds of oblivious permutation for both rows and columns of the two matrices. We simplify this process by modifying how the receiver generates the random values. Our approach requires only a check if α values of $c_{ij}f_{ij}(x) + r_j$ contain r_j . This transformation reduces the original matrix equality test to a sender-heavy Private Membership Test (PMT) problem. We can use a shuffled Private Membership Test (s-PMT) to resolve this problem and eliminate the row permutation. This tweak can lead to a more efficient p-RPMT for PSU and PCSI protocols.

2 Preliminaries

2.1 Notations

We assume X and Y are two subsets of $\mathcal{U} \subset \mathbb{F}_q$ for a large prime $q \approx 2^\lambda$ and $|X| \ll |Y| \approx \text{poly}(\lambda) \ll |\mathcal{U}|$. We use $[L]$ to represent an index set $\{1, 2, \dots, L\}$ and use bold alphabets (e.g., $\mathbf{b}$) for vector and $\mathbf{b}[i]$ to denote i -th element of $\mathbf{b}$.

2.2 Unbalanced Private Set Union

This work studies the unbalanced private set union where the set X held by the sender $\mathcal{S}$ is much smaller than the set Y held by the receiver $\mathcal{R}$. Figure 1 shows the ideal function of an unbalanced PSU.

2.3 Security Model

PSU protocols usually use the semi-honest model [10] as their security requirement. In the semi-honest model, adversaries try to infer any useful information about private data using what they learn during the protocol but cannot deviate from the protocol. A formal definition of security against semi-honest adversaries is as follows:

Parameters: Set size for Sender $\mathcal{S}$ is $|X|$, set size for Receiver $\mathcal{R}$ is $|Y|$; $|X| \ll |Y|$.

Functionality:

- Wait for input $X = \{x_i\}_{i \in [|X|]}$ from the sender $\mathcal{S}$ and input $Y = \{y_i\}_{i \in [|Y|]}$ from the receiver $\mathcal{R}$.
- Give the output $X \cup Y$ to $\mathcal{R}$.

Fig. 1. Ideal function for unbalanced private set union $\mathcal{F}_{\text{uPSU}}$.

Definition 1 (Semi-honest model [10]). Let $f(x, y)$ be a functionality, and $f_0(x, y)$ (resp. $f_1(x, y)$) denote the first (resp. second) element of $f(x, y)$. Let Π be a two-party protocol for computing $f(x, y)$. The view of the first (resp. second) party during the execution of Π on (x, y) is $\text{VIEW}_0(x, y)$ (resp. $\text{VIEW}_1(x, y)$). The protocol Π is said to privately compute function f in the semi-honest model if there exist probabilistic polynomial-time simulators S_0 and S_1 , such that for any input x and y ,

$$S_0(x, f_0(x, y)) \stackrel{c}{\equiv} \text{VIEW}_0(x, y) \quad (1)$$

$$S_1(y, f_1(x, y)) \stackrel{c}{\equiv} \text{VIEW}_1(x, y) \quad (2)$$

where $\stackrel{c}{\equiv}$ represents computational indistinguishability

2.4 Hash Tables

The hashing technique is a practical approach to improve the performance of PSI [18–20]. Hashing techniques can divide a set S into small subsets (bins) according to the hash value. The deterministic nature of hash functions ensures the hash function maps the identical elements to the same bin.

Simple hashing [18] involves a universal hash set $\mathcal{H}_L = h : \mathcal{U} \rightarrow [L]$, where each h is a uniform random map. One can randomly select $h \in \mathcal{H}_L$ to construct a hash table with L bins. For a set S , the distribution of elements per bin follows a binomial distribution $B(|S|, \frac{1}{L})$. In the PSU protocol, the receiver computes k hash tables and combines them bin-wise to form a large hash table.

Cuckoo hashing [14] is an indexing technique that creates a hash table for a set S with $L = O(|S|)$ bins using k distinct simple hashes $H = \{h_i\}_{i \in [k]}$. Unlike traditional hash tables, cuckoo hash ensures each bin stores no more than one element. During construction, we insert elements in S one by one according to the hash h_1 . If a collision happens when inserting $s \in S$ (e.g., $h_1(s') = h_1(s)$), we need to replace the existing element s' with s and reinsert s' into a new bin $h_2(s')$. This recursive process continues until all elements are in the table or reach a predefined time limit. In the latter case, one can select a new H and repeat the procedure. A typical parameter for the cuckoo hash is $k = 3$, and $L \approx 1.48|S|$.

2.5 Permuted Reversed Private Membership Test (p-RPMT)

Multi-query Reversed Private Membership (mq-RPMT) Test is a protocol designed to check if the sender's elements belong to the receiver's set, with results returned to the receiver. To address specific leakage concerns highlighted in [15], the PSU protocol employs a modified version known as permuted RPMT (p-RPMT). Figure 2 illustrates the ideal functions of p-RPMT for the hash-based approach.

Parameters: hash table size L and α

Functionality:

- Wait for input hash table $\mathbf{x} \subset \mathbb{F}_q^L$ and a permutation π over $[L]$ from the sender $\mathcal{S}$ and hash table $\mathbf{Y} \subset \mathbb{F}_q^{\alpha \times L}$ from the receiver $\mathcal{R}$.
- Give the output $\mathbf{b}$ to $\mathcal{R}$, where $\mathbf{b}[i] = 1$ if $\mathbf{x}[\pi(i)] \in \mathbf{Y}[\pi(i)]$ and $\mathbf{b}[i] = 0$ otherwise.

Fig. 2. Ideal function for p-RPMT.

3 Our Attacks

3.1 Overview of Hash + RPMT Framework

Following the divide and test principle from Chen et al. [4], the PSU protocol that leverages the cuckoo hash and RPMT framework comprises several key steps:

- $\mathcal{S}$ and $\mathcal{R}$ agree on the parameters k, L , the set of simple hashes $H = \{h_i\}_{i \in [k]}$, and RPMT-related parameters.
- $\mathcal{S}$ computes a cuckoo hash table for set X using H . Simultaneously, $\mathcal{R}$ constructs a simple hash table for all k hashes and merges them bin-wise into one table. Both parties pad their hash tables with the same $\perp \in \mathbb{F}_q \setminus \mathcal{U}$.
- $\mathcal{S}$ randomly chooses a permutation π over $[L]$. $\mathcal{S}$ and $\mathcal{R}$ initiate the p-RPMT protocol, resulting in $\mathcal{R}$ receiving $\mathbf{b}$.
- $\mathcal{S}$ uses π to permute the cuckoo hash table and employs an Oblivious Transfer (OT) protocol to transfer the permuted table to $\mathcal{R}$ based on $\mathbf{b}$ to learn the set difference $X \setminus Y$. Then $\mathcal{S}$ obtains the union result $X \cup Y$.

The cuckoo hash in this process reduces the computational complexity of RPMT from $O(|X||Y|)$ (comparing each $x \in X$ with Y) to $O(L \frac{k|Y|}{L}) \approx O(|Y|)$.

Kolesnikov et al. [15] noted that using a standard RPMT (non-permuted version) in a PSU protocol might reveal information about $X \cap Y$ to the receiver. Typically, for all i , $\mathbf{b}[i] = 0$ indicates that $\mathbf{Y}[i]$ contains precisely one element from X . The PSU protocol uses a permutation to shuffle $\mathbf{b}$, preventing $\mathcal{R}$ from linking $\mathbf{Y}[i]$ with the results.

However, our result demonstrates that this leakage is also related to the parameters k and L . Thus, altering the RPMT procedure without adjusting the cuckoo hash parameters may not adequately address the vulnerability.

3.2 Attack Vector

Security Model. PSU protocols use the semi-honest model for security proof. Proving a protocol's insecurity in the semi-honest model requires the prover to demonstrate the non-existence of a simulator, which is not straightforward. To address this, we introduce two new security models for the PSU protocol that effectively highlight potential leakages compared to an ideal function. Since our attack focuses on the Hash + RPMT framework, we do not assess the security of the underlying p-RPMT and OT protocols. We assume the PSU protocol can use the ideal functions of both p-RPMT and OT.

Our first security model considers statistical leakage. We require that the receiver $\mathcal{R}$'s view in the PSU protocol provide no more information about $X \cap Y$ than the ideal PSU function for most protocol instances.

Definition 2 (Statistical Security). *An PSU protocol Π is statistical secure to receiver $\mathcal{R}$, if given a security parameter λ_S we have:*

$$\Pr_{X,Y \leftarrow \mathcal{U}}[D(X \cap Y|Y, X \setminus Y) \neq D(X \cap Y|View_{\mathcal{R}}^{\Pi}(X, Y))] < \text{negl}(\lambda_S)$$

Here $D(X \cap Y|C)$ represents given C , the distribution of $X \cap Y$ over Y .

The statistical security model is strong but reasonable. It can detect any information leakage from the protocol to the receiver. This model can identify if an attacker might deduce the distribution of $X \cap Y$ beyond what the ideal function provides—a uniform sample over Y . λ_S within this model allows us to quantify the likelihood of such leakages. An empirical choice of λ_S can be logarithm to the number of sets used in the PSU protocol. Since the p-RPMT and OT protocols are ideally functioning, the statistical security model helps us pinpoint any potential leakage originating solely from the Hash+RPMT framework.

Second, we provide a computational security model. In this model, a security PSU protocol requires all probabilistic polynomial time (PPT) adversaries cannot differentiate the distribution of the intersection from the receiver's view of the protocol and the ideal function.

Definition 3 (Computational Security). *An PSU protocol Π is computationally secure to receiver $\mathcal{R}$, if for a security parameter λ_C , and for all probabilistic polynomial time attacker $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$:*

$$\left| \Pr \left[\begin{array}{c} X, Y \leftarrow \mathcal{A}_1(\mathcal{U}) \text{ s.t. } |X| \ll |Y| \\ b \xleftarrow{\$} \{0, 1\} \end{array} : \begin{array}{c} b' \leftarrow \mathcal{A}_2^{O_b}(Y, X - Y) \\ b = b' \end{array} \right] - \frac{1}{2} \right| \leq \text{negl}(\lambda_C)$$

Where

- O_0 : sample from $D(X \cap Y|Y, X \setminus Y)$
- O_1 : sample from $D(X \cap Y|View_{\mathcal{R}}^{\Pi}(X, Y))$.

We introduce a challenge to measure leakage based on the inference of $X \cap Y$. A computationally secure protocol must prevent a PPT adversary from distinguishing between two oracles, O_1 and O_0 . To establish an attack, it suffices to demonstrate that a PPT adversary can differentiate between $D(X \cap Y|Y, X \setminus Y)$ and $D(X \cap Y|View_{\mathcal{R}}^{\Pi}(X, Y))$. λ_C here should reflect the computation power of the attacker (e.g. $\lambda_C = 64$).

Our security model identifies and quantifies any leakage surpassing the ideal function. In Sect. 3.3, we will elaborate on our attack within the semi-honest model and summarize the distinctions between our computational security model and the semi-honest model. Although our model imposes stricter requirements than the semi-honest model, it remains a practical framework for evaluating the security of PSU protocols. While we have focused on PSU in this discussion, we can define a similar security model for the PSI protocol.

Attack Construction. We adopt the setting from the PSU protocol that $\mathcal{S}$ and $\mathcal{R}$ decide parameters in the offline phase. Denote $A = X \setminus Y$, $a = |A|$, L is the size of the cuckoo hash table, $H = \{h_i : \mathcal{U} \rightarrow [L]\}_{i \in [k]}$ is the set of simple hashes used in cuckoo hash.

For ideal function, we have $D(X \cap Y | X, X \setminus Y)$ is a uniform sample from

$$D_0 = \{B | B \subset Y, |B| = |X| - a\} \quad (3)$$

We now provide study the distribution of intersection $D(X \cap Y | \text{View}_R^H(X, Y))$:

Getting A, Y, H, L from $\text{View}_R^H(X, Y)$, let $I_A = \{h(e) | \forall e \in A, h \in H\}$.
 $\forall J \in [L] \setminus I_A$ s.t. $|J| = L - |X| + 1$, let

$$T_J = \{y | y \in Y \ \& \ \exists h \in H, h(y) \in J\}, T = \bigcup_J T_J$$

We can have the following observations:

- $\forall J \ |T_J \cap X| > 1$
- $|T \cap X| > |X| - |I_A|$

Therefore, we can have $D(X \cap Y | \text{View}_R^H(X, Y))$ as a uniform distribution from a **subset** of

$$D_1 = \{B | B \subset Y, |B| = |X| - a, \forall J, |B \cap T_J| \geq 1, |B \cap T| \geq |X| - |I_A|\} \quad (4)$$

We capture the leakage of the PSU protocol from the additional condition $|B \cap T_J| \geq 1, |B \cap T| \geq |X| - |I_A|$. If $\Pr_{B \subset Y}[|B \cap T_J| \geq 1, |B \cap T| \geq |X| - |I_A|]$ is not overwhelmingly large, the PSU protocol is not secure statistically or even computationally.

Leakage Measurement. Now we measure $\Pr_{B \in Y}[|B \cap T_J| \geq 1, |B \cap T| \geq |X| - |I_A|]$. First, we notice if $|I_A| > |X| - 1$, there is no valid T_J and T . Second, $|B \cap T_J| \geq 1$ is non-trivial as long as exists J such that $|T_J| < |Y| - (|X| - a - 1)$. Therefore, our attack on statistical security succeeds with probability

$$P_S = \Pr[\exists J, |T_J| < |Y| - (|X| - a - 1) \ \& \ |I_A| \leq |X| - 1]. \quad (5)$$

For computational security, we can sample $\text{poly}(\lambda_C)$ times from O_0 and O_1 and test the distribution for $y \in Y$. In O_0 , we have

$$\Pr[y \in X \cap Y | y \in Y] = \frac{|X| - a}{|Y|}.$$

For O_1 , we have

$$\Pr[y \in X \cap Y | y \in T] \geq \frac{|X| - |I_A|}{|T|}.$$

Therefore, we know if $\frac{|X|-|I_A|}{|T|} \geq \frac{|X|-a+1}{|Y|}$ we have the probability difference $\frac{1}{|Y|}$ over T which means we can distinguish O_0 and O_1 in $\text{poly}(|Y|) \approx \text{poly}(\lambda_C)$ samples through χ^2 test. Therefore, our attack succeeds with probability

$$P_C = \Pr \left[|T| < \frac{|Y|(|X| - |I_A|)}{|X| - a + 1} \ \& \ |I_A| \leq |X| - 1 \right]. \quad (6)$$

All above attacks only use $H, Y, X \setminus Y$. Therefore, our attack can apply to all PSUs with Hash + RPMT frameworks, even if the underlying RPMT and OT are ideal functions.

We follow the binomial distribution model from [22] to estimate T_J . We can have, $\Pr_{e \in Y}[e \in T | |I_A| = t] = 1 - (\frac{t}{L})^k$. Then the distribution of $|T|$ follows a binomial distribution $B(|Y|, 1 - (\frac{t}{L})^k)$. Denote the cumulative distribution function of binomial distribution $B(n, p)$ as $\text{cdf}(n, k, p) = \sum_{i=0}^k \binom{n}{i} p^i (1-p)^{n-i}$. Since $\frac{|Y|(|X|-t)}{|X|-a+1}$ is not tight for computational distinguishability, we have the computational lower bound for T_J :

$$\Pr \left[|T| \leq \frac{|Y|(|X| - t)}{|X| - a + 1} \mid |I_A| = t \right] = \text{cdf}(|Y|, \frac{|Y|(|X| - t)}{|X| - a + 1}, 1 - (\frac{t}{L})^k). \quad (7)$$

The combination of J makes the estimation for the statistical condition more complicated. Therefore, we use $\Pr[J \stackrel{[L]}{\$} \mid |T_J| < |Y| - (|X| - a - 1)]$ as the lower bound for $\Pr[\exists J \subset [L] \mid |T_J| |T_J| < |Y| - (|X| - a - 1)]$. Therefore, we get:

$$\Pr[\exists J \subset [L] \mid |T_J| < |Y| - (|X| - a - 1)] > \text{cdf}(|Y|, |Y| - |X| + a, 1 - (\frac{|X| - 1}{L})^k). \quad (8)$$

Next, we estimate $\Pr[|I_A| = t]$. Since h are uniform injections, the whole process becomes assigning ak elements into t out of L slots, each slot having at least one element. This provides us $\binom{L}{t} \binom{ak-1}{t-1}$ possible cases. Thus, we have

$$\Pr[|I_A| = t \mid t > a] = \frac{\binom{L}{t} \binom{ak-1}{t-1}}{\binom{L+ak-1}{ak}}. \quad (9)$$

Combining the two probabilities, we have the two lower bounds:

$$P_S > \text{cdf}(|Y|, |Y| - |X| + a - 2, 1 - (\frac{|X| - 1}{L})^k) \sum_{t=a}^{\min(ak, |X|-1)} \frac{\binom{L}{t} \binom{ak-1}{t-1}}{\binom{L+ak-1}{ak}}. \quad (10)$$

$$P_C > \sum_{t=a}^{\min(ak, |X|-1)} \text{cdf}(|Y|, \frac{|Y|(|X| - |I_A|)}{|X| - a + 1}, 1 - (\frac{t}{L})^k) \frac{\binom{L}{t} \binom{ak-1}{t-1}}{\binom{L+ak-1}{ak}}. \quad (11)$$

We compute the both bound for the setting ($k = 3, L = \text{round}(1.48 \cdot |X|)$) from [25] against different $|X \setminus Y|$ as Fig. 3. When $a = |X| - 1$, the two probabilities match as expected because there is only one possible J . Our result shows that leakage from the PSU protocol is inevitable under their parameter setting. When $|X \setminus Y|$ is close to $|X|$, a PPT adversary can identify the leakage with a high chance. Since our computational security model allows adversaries to choose X, Y adaptively to control $|X \setminus Y|$, the simulation gives our main result:

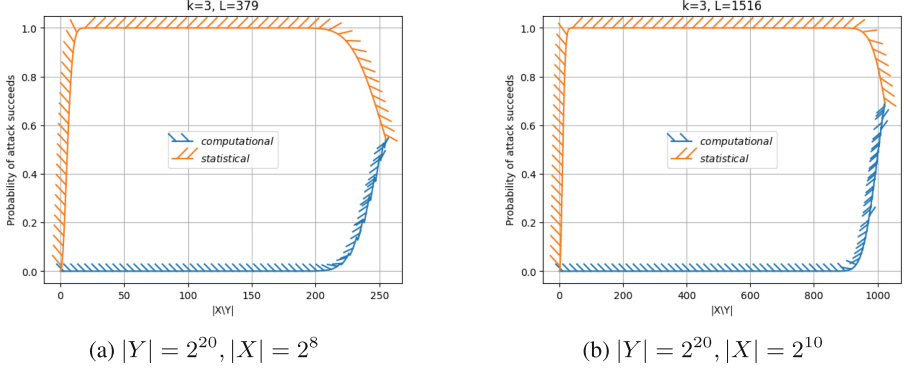


Fig. 3. Lower bound of probability for launching a successful attack.

Theorem 1. *The PSU Protocol in [25] under their parameter ($k = 3, L = \text{round}(1.48 \cdot |X|)$) is not computationally secure in our security model Definition 3.*

We can also apply the above attacks to a receiver-heavy PSI protocol [4]. However, $|U \setminus Y|$ can be too big for launching any meaningful attack in polynomial time. Therefore, our attack only reveals leakage in cuckoo hash-based receiver-heavy PSI protocol [4].

3.3 Explanation of Leakage

Source of Leakage. Our attack success probability P_S serves as a lower bound for initiating two potential attacks:

- 1. The sender $\mathcal{S}$ fails to build a cuckoo hash for X (e.g., when $|I_A| < a$);
- 2. $\mathcal{H}$ can build a cuckoo hash but $|T_J|$ is too small to hide the distribution of $X \cap Y$.

In the first attack, a failure in cuckoo hashing by $\mathcal{S}$ leads to immediate leakage. Therefore, the user should pick k, L according to $|X|$ such that the failure rate is $\text{negl}(\lambda_C)$. The second attack involves $|T_J|$, highlighting T_J as the combination

of redundant $L - |X|$ bins plus one bin containing at least one element in X . The $L - |X|$ redundant bins provide additional entropy to compensate for the reduced comparison in RPMT. Therefore, the receiver may get additional information if any $L - |X|$ bins do not contain sufficient elements. In this case, the choice of k and L additionally depends on $|Y|$, which means preventing the collision attack is insufficient for statistical security.

The attack from Kolesnikov et al. [15] pointed out that knowing the index i of $\mathbf{b}[i] = 1$ (without permutation) will leak the information about the input set $\mathbf{x}[i] \in \mathbf{Y}[i]$. Garimella et al. [9], and Tu et al. [25] use an oblivious permutation to hide the index i during OT protocol. However, our attack points out that the receiver can use the simple hash set H to compute I_A and get knowledge $\mathbf{x}[j] \in \mathbf{Y}[j]$ for $j \notin I_A$. Our method provides a fuzzy inference from $j \notin I_A$. Therefore, our attack is a generalized version of Kolesnikov et al. [15]'s attack.

Computational Leakage and Semi-Honest Model. Now, we discuss the relationship between the semi-honest model and our computational security model.

First, we denote the domain of all possible H , according to given k and L , as $\mathcal{H}$. Then, we define two types of the simple hash set given:

- $\mathbf{H}_0$ represents all possible H that can construct a cuckoo hash table over $X \setminus Y$.
- $\mathbf{H}_1$ includes all possible H that can construct a cuckoo hash table over X .

Notably, $\mathbf{H}_1 \subset \mathbf{H}_0 \subset \mathcal{H}$. When $\mathcal{S}$ and $\mathcal{R}$ agree with H in setup phase, we have $H \xleftarrow{\$} \mathcal{H}$. Therefore, $\frac{|\mathcal{H} \setminus \mathbf{H}_1|}{|\mathcal{H}|}$ denotes the probability of protocol failure.

Moreover, if $\mathcal{S}$ picks $H \in \mathbf{H}_1$ manually to prevent the protocol failure, leakage still occurs since the simulator can only pick H from $\mathbf{H}_0$. Our attack can extract a set T_{sim} from the computationally leaky instance generated by the simulator. However, since T_{sim} and T_{view} are indistinguishable from the semi-honest adversary, our attack cannot break the semi-honest security. Therefore, **a PSU protocol can be computationally leaky while secure in a semi-honest model.**

To enhance the semi-honest model, we can have a stronger adversary:

- We can grant access to a cuckoo hash oracle for checking if H can make a cuckoo hash for X .
- We can allow the adversary to repeat the simulation and protocol for $\text{poly}(\lambda)$ rounds.

The adversary can distinguish the two views by testifying if $H_{sim} \in \mathbf{H}_0 \setminus \mathbf{H}_1$ with the cuckoo hash oracle or by directly deriving the distribution of $\mathbf{H}_1$ and $\mathbf{H}_0$ from $\text{poly}(\lambda)$ pairs of H_{sim} and H_{view} .

Lastly, we find that the collision rate limits the probability of our attack to computational security since $\frac{|\mathbf{H}_1|}{|\mathbf{H}_0|} > \frac{|\mathbf{H}_1|}{|\mathcal{H}|}$. In an extreme scenario where $a = 0$, $\mathbf{H}_0$ equals $\mathcal{H}$. Hence, if collisions occur with a negligible probability, the potential

for computational leakage remains similarly negligible. This outcome shows that even if the PSU protocol allows $\mathcal{S}$ to select the hash H based on X , it is still crucial to select the parameters k and L for preventing leakage.

Remark 1. The original proof in [25] is sufficient for the semi-honest model. However, to achieve computational security, they must select a cuckoo hash parameter that is truly collision-resistant (not those in their experiment). Our statistical security will require a further strict parameter selection to prevent the leakage. We think the semi-honest model is insufficient for proving a protocol is leakage-free.

4 Fix for Hash + RPMT Framework

This section proposes two strategies to avoid our attack vector. The first method aims to prevent the adversary from launching a successful attack by selecting proper parameters for cuckoo hash. By increasing k (the number of hashes) and L (the number of bins), we can achieve computational and statistical security for reasonable secure parameters. This approach directly tackles the root of the leakage by introducing entropy.

The second strategy involves modifying the protocol's structure through a pre-processing phase or transitioning to a different Private Set Operation (PSO) protocol. These changes aim to prevent the receiver from being able to compute I_A after the protocol's execution and thus block our attack vector.

4.1 Parameter Adjustment

The cuckoo hash parameters ($k = 3, L = \text{round}(1.48 \cdot |X|)$) we analyzed, sourced from Tu [25] and Chen [4], claim to provide 2^{-40} bits of collision resistance. However, contrasting findings by Pinkas et al. [21] suggest that for $k = 3$, achieving a collision probability of 2^{-30} requiring $L \approx 166 \cdot |X|$. This gap indicates that the PSU protocol will perform worse with the actual secure k and L .

Considering both computational and statistical leakages, we propose the following enhanced parameter strategy:

- Performance Complexity: Ensure the parameters maintain the unbalanced PSU protocol's computational efficiency, ideally keeping the overall complexity at $O(|Y|)$. The relationship should ideally be $O(1) \approx k \ll |X| < L \approx O(|X|)$.
- Computational Security: Choose (k, L) to ensure a collision-free cuckoo hash by simulations from Pinkas et al. [21].
- Statistical Security: Select (k, L) such that the size of $|T_J|$ is sufficiently large to mitigate the leakage.

We define a new simulation experiment for the third constraint. To reduce the parameter space, we calculate the upper bound of $\Pr[|I_A| \leq |X| - 1]$ based on our derived formula for $\Pr[|I_A| = t]$. This probability negatively related with a ,

allowing us to identify a value a' for which $\Pr[|I_A| \leq |X| - 1 | |A| \geq a'] \leq \text{negl}(\lambda_S)$. Subsequently, we simulate the hash table to compute $\Pr[\min_{J \subset I}(|T_J|) < |Y| - |X| + a' + 1]$. Given that $\min(|T_J|)$ is independent of $|A|$, this enables us to assert that when $|A| < a$, the likelihood of a successful attack remains below $\Pr[\min_J(|T_J|) < |Y| - |X| + a' + 1]$. By iteratively adjusting k and the ratio $\frac{L}{|X|}$, and through repeated simulations, we define a parameter threshold that meets the third requirement. We set the security parameter $\lambda_S = 20$ (leak-free in 1M rounds), with the results detailed in Table 1.

Table 1. Simulation for static attack. Black results simulated 100 rounds, blue results simulated 1 M rounds.

		$ Y = 2^{13}$				$ Y = 2^{15}$			
		$k = 5$	$k = 7$	$k = 9$	$k = 11$	$k = 5$	$k = 7$	$k = 9$	$k = 11$
$ X = 2^5$	$L = 2 X $	100%	100%	72%	0.0000%	100%	100%	100%	44%
	$L = 3 X $	100%	0.0000%	0.0000%		100%	5.9423%	0.0000%	0.0000%
	$L = 4 X $	0.0126%				86%	0.0000%		
$ X = 2^7$	$L = 2 X $	100%	0.1246%	0.0000%	0.0000%	100%	100%	0.0056%	0.0000%
	$L = 3 X $	0.0000%				100%	0.0000%	0.0000%	
	$L = 4 X $					0.0000%			

Our simulation reveals that the probability of a successful attack increases with the size of $|Y|$ and decreases with the size of $|X|$, holding k and L constant. This finding aligns with our previous analyses that our attack is beyond the simple cuckoo hash collision and requires an independent parameter configuration.

Adjusting parameters invariably affects performance. For instance, adopting $k = 9$ and $L = 3|X|$ -a setting we predict will triple the computation cost for the receiver and double the communication cost relative to the benchmarks established by Tu et al. [25]. Furthermore, the optimal parameters depend on the sizes of $|X|$ and $|Y|$, which increase as these variables grow. Thus, users must conduct simulations tailored to their specific use cases before finalizing parameter choices.

Additionally, our simulations consider the statistical leakage. For scenarios primarily concerned with computational security, the parameters suggested by Pinkas et al. [21], such as $k = 5$ and $L \approx 1.2|X|$ for $\lambda_C = 30$, might be sufficient. This configuration maintains a comparable communication cost while increasing the computation cost for the receiver by approximately 70%.

Remark 2. Our attack and analyses assume that the distribution of two sets X and Y are uniformly within $\mathcal{U}$. This setting is aligned with prior works. If X and Y are based on different distributions over $\mathcal{U}$, we predict the leakage probability will increase because the available cuckoo hash H for X will be further limited, and the distribution of Y in the hash table might reveal more information. In

this case, the secure parameter will be even larger, forcing T_J and I_A to be sufficiently large.

4.2 Prevent Computation of I_A

In addition to optimizing parameter choices, modifying the Hash + RPMT protocol to prevent the computation of I_A can further secure the protocol against attacks. We propose two strategies to achieve this:

Preprocessing. Chen et al. [3] introduced the oblivious PRF to sender-heavy PSI protocol for adversary model security. The idea is to let the sender apply a pseudo-random function F_{sk} for all receiver's elements before building the cuckoo hash, while only the sender knows sk . This procedure ensures the receiver cannot recompute F_{sk} post protocol, effectively preventing it from calculating I_A and applying our attack.

Although this pre-processing introduces an entropy layer that secures the hash table, it also incurs an $O(|Y|)$ communication cost in **On-line** phase, making it less suitable for receiver-heavy scenarios. However, it is an effective fix for balanced PSU scenarios using the Hash + RPMT framework, such as those outlined by Garimella et al. [9].

Post-processing. An alternative method uses a public collision-resistant one-way function to prevent computing I_A . The sender hashes all elements of $X \setminus Y$ before the Oblivious Transfer (OT) protocol. Post-protocol, the receiver cannot reconstruct I_A back and still can have a hashed union as the output. The receiver can also hash all his sets to get a hashed union set, which supports further membership tests. The receiver can repeat the union protocol or a similar intersection protocol with other parties in the future. Although this approach limits data availability and increases processing for the receiver, it retains essential membership information for subsequent uses.

Moreover, implementing this in the Hash + RPMT framework is also helpful for receiver-heavy private computing on set intersection (PCSI) protocols that

Parameters: Set size for Sender $\mathcal{S}$ is $|X|$, set size for Receiver $\mathcal{R}$ is $|Y|$.

Functionality:

- Wait for inputs $X = \{x_i\}_{i \in [|X|]}$ and corresponding label $\{l_i\}_{i \in [|X|]}$ from the sender $\mathcal{S}$; wait for an input $Y = \{y_i\}_{i \in [|Y|]}$ from the receiver $\mathcal{R}$.
- **PSI-CA:** Give the output $|X \cap Y|$ to $\mathcal{R}$.
- **Card-Secret-Sharing:** Give the output $k = |X \cap Y|$ and $\{a_i\}_{i \in [k]}$ to $\mathcal{R}$ and $k, \{b_i\}_{i \in [k]}$ to $\mathcal{S}$ where $a_i + b_i \in X \cap Y$.
- **Label-Sum:** Give the output $|X \cap Y|$ and $\sum_{x_i \in Y} l_i$ to $\mathcal{R}, i \in [n_x]$.

Fig. 4. Ideal functionality for PCSI $\mathcal{F}_{\text{PCSI}}$.

do not require revealing original set members to $\mathcal{R}$. Such PCSI protocols, as depicted in Fig. 4, block the computation of I_A and allow for the use of smaller k, L achieving comparable performance compared to other works [5, 25].

5 Improving p-RPMT

This section introduces an enhanced version of the p-RPMT protocol that reduces communication costs compared to the approach in Tu et al. [25].

In the existing method by Tu et al., the receiver $\mathcal{R}$ homomorphically encodes the $\alpha \times L$ membership results from $f_{ij}(x_j)$ into $f_{ij}(x_j) + r_{ij}$. After decryption, the sender $\mathcal{S}$ holds $(f_{ij}(x_j) + r_{ij})$, and $\mathcal{R}$ retains r_{ij} . Then $\mathcal{S}$ and $\mathcal{R}$ need to shuffle the two matrices and perform equality tests for all $\alpha \times L$ elements.

In our proposal, $\mathcal{R}$ selecting L random values r_j and $\alpha \times L$ coefficients c_{ij} . Each $\mathcal{R}$ then homomorphically computes $c_{ij}f_{ij}(x_j) + r_j$ for each $i \in [\alpha], j \in [L]$ and sends these results back to $\mathcal{S}$. The protocol simplifies verifying whether each of the L values r_j is present within the computed $c_{ij}f_{ij}(x_j) + r_j$. Consequently, $\mathcal{S}$ and $\mathcal{R}$ only need to perform a shuffled Private Membership Test (PMT) as Fig. 5. We present an example implementation of s-PMT based on the Decisional Diffie-Hellman assumption in Fig. 6. Compared to the DH-based permuted matrix equality test utilized by Tu et al. [25], our DH-based s-PMT half the communication costs and decreases $O(\alpha)$ group exponentiation without additional costs.

Parameters: number of sets L and the set size α .

Functionality:

- Wait for inputs $\mathcal{S} \subset \mathbb{F}_q^{\alpha \times L}$ and a permutation π over $[L]$ from $\mathcal{S}$, and inputs $e \in \mathbb{F}_q^L$ from $\mathcal{R}$.
- Give the output $\mathbf{b} \subset \{0, 1\}^L$ to $\mathcal{R}$ where $\mathbf{b}[i] = 1$ if $e[\pi(i)] \in \mathcal{S}[\pi(i)]$ and $\mathbf{b}[i] = 0$ otherwise.

Fig. 5. Ideal functionality for Shuffled Private Membership Tests $\mathcal{F}_{\text{s-PMT}}$.

Theorem 2 (Security of s-PMT based on DH). *Protocol $\Pi_{\text{s-PMT}}$ from DH-OPRF in Fig. 6 is secure under the semi-honest model assuming H is a random oracle, and the DDH assumption holds.*

Our s-PMT protocol is secure under the semi-honest model. The proof and an alternative construction of s-PMT based on permutation share techniques are in Appendix A. We will present the full p-RPMT protocol in the full version of this manuscript.

Input: $\mathcal{R}$ inputs a vector $e \in \mathbb{F}_q^L$; $\mathcal{S}$ inputs a vector $S \in \mathbb{F}_q^{\alpha \times L}$ and a permutation π over $[L]$. A cyclic group $\mathbb{G}$ of large prime order p and a random oracle $H : \mathbb{F}_q \rightarrow \mathbb{G}$.

Protocol:

- $\mathcal{R}$ picks a random value sk from $\mathbb{Z}_p$ and sends $(H(e_i)^{sk})_{i \in [L]}$ to $\mathcal{S}$.
- $\mathcal{S}$ picks a random value β from $\mathbb{Z}_p$ and computes $u = (H(e_{\pi(i)})^{sk \cdot \beta})_{i \in [L]}$.
- $\mathcal{S}$ computes $S' = [S'_i = \{H(s)^\beta | s \in S[\pi(i)]\}]$ and shuffle S'_i randomly.
- $\mathcal{S}$ sends u and S' to $\mathcal{R}$.
- $\mathcal{R}$ sets $b_i = 1$ if $u_i^{-sk} \in S'_i$ and $b_i = 0$ otherwise.

Fig. 6. Protocol $\Pi_{s\text{-PMT}}$ from DH-OPRF.

6 Conclusion and Future Works

This work revisits the PSU protocol as described by Tu et al. [25] and introduces attacks against it for statistical and computational security. The attack on statistical security highlights issues with insufficient entropy in the cuckoo hash table, while the attack on computational security primarily concerns collision vulnerabilities. To address these vulnerabilities, we offer two strategies: 1. Adjusting $|k|, |L|$ to maintain performance complexity while mitigating the identified attacks. 2. Incorporating pre-processing or post-processing measures to thwart potential attacks. Finally, we provide a cost-free optimization to the p-RPMT protocol that reduces communication overhead.

Our work also left some open questions for future research:

- 1. Can we have a formula for a practical higher bound of P_S and P_C that can guide the choice k and L without simulation?
- 2. Can we have a stronger semi-honest model for multiparty computation that can detect leakage in this work?

Acknowledgements. This work was done when all authors belonged to The University of Tokyo and was supported by: JST CREST, Grant Number JPMJCR2113; JST SPRING, Grant Number JPMJSP2108; JSPS KAKENHI Grant Number, JP23KJ0385; JSPS Postdoctoral Fellowships for Research in Japan (No. P21073).

A Shuffled PMT (s-PMT)

A.1 Proof of DH Based s-PMT

Theorem 3 (Security of s-PMT based on DH). *Protocol $\Pi_{s\text{-PMT}}$ from DH-OPRF in Fig. 6 is secure under the semi-honest model assuming H is a random oracle, and the DDH assumption holds.*

Proof. We construct simulators $\text{SIM}_{\mathcal{S}}$ and $\text{SIM}_{\mathcal{R}}$ to simulate the view of corrupt $\mathcal{S}$ and $\mathcal{R}$ respectively and argue the indistinguishability of the simulated transcript from the real execution.

Corrupt Sender $\mathcal{S}$. The view of $\mathcal{S}$ in the real protocols is $\{H(e_i)^k\}_{i \in [L]}$. $\text{SIM}_{\mathcal{S}}(\mathcal{S}, \pi, \pi')$ randomly selects L values $\{g_i\}_{i \in [L]}$ from $\mathbb{G}$ to simulate $\mathcal{S}$'s view. We show $\text{SIM}_{\mathcal{S}}(\mathcal{S}, \pi, \pi')$ is indistinguishable from $\mathcal{S}$'s view in the real execution by the following hybrids.

Hybrid₀: $\mathcal{S}$'s view in the real protocol.

Hybrid₁: Same as Hybrid₀ except that $\text{SIM}_{\mathcal{S}}$ selects random group elements g_i instead of $H(e_i)^k$ for $i \in [L]$.

We can learn that Hybrid₁ is computationally indistinguishable from Hybrid₀ based on the DDH assumption and the random oracle model [8, 12].

Corrupt Receiver $\mathcal{R}$. The view of $\mathcal{R}$ in the real protocols consists of $\mathbf{u}$ and $\{S'_i\}_{i \in [L]}$. $\text{SIM}_{\mathcal{R}}(\mathbf{e}, \mathbf{b})$ simulates the view of corrupt Receiver $\mathcal{R}$ as follows. $\text{SIM}_{\mathcal{R}}$ picks a random value k from $\mathbb{Z}_p$. For $b_i = 0, i \in [L]$, $\text{SIM}_{\mathcal{R}}$ chooses $\alpha + 1$ random elements from $\mathbb{G}$ and sets them to be u_i and S'_i respectively. For $b_i = 1$, $\text{SIM}_{\mathcal{R}}$ randomly chooses L values from $\mathbb{G}$ to form S'_i , selects a random index $j \in [\alpha]$ and sets $u_i = s'_{ij}^k$. We show $\text{SIM}_{\mathcal{R}}$ is indistinguishable from $\mathcal{R}$'s view in the real execution by the following hybrids.

Hybrid₀: $\mathcal{R}$'s view in the real protocol.

Hybrid₁: Same as Hybrid₀ except that for $b_i = 0$ $\text{SIM}_{\mathcal{R}}$ chooses $\alpha + 1$ random elements as u_i and S'_i . For $b_i = 1$, $\text{SIM}_{\mathcal{R}}$ randomly chooses L group values to form S'_i , selects a random index $j \in [\alpha]$ and sets $u_i = s'_{ij}^k$. The DDH assumption and the random oracle model ensure that Hybrid₀ and Hybrid₁ are computationally indistinguishable.

A.2 Permute + Share

The Permute + Share functionality $\mathcal{F}_{PS}$ (Fig. 7 [2]) involves two parties $\mathcal{S}$ and $\mathcal{R}$ where $\mathcal{S}$ possesses a set $X = \{x_i\}_{i \in [n]}$ and $\mathcal{R}$ picks a permutation π on n elements. $\mathcal{F}_{PS}$ enables $\mathcal{R}$ to get $\{y_{\pi(i)}\}_{i \in [n]}$ and $\mathcal{S}$ to obtain $\{y'_{\pi(i)}\}_{i \in [n]}$ satisfying $x_{\pi(i)} = y_{\pi(i)} \oplus y'_{\pi(i)}$ for $i \in [n]$. The protocols [2, 16] implementing $\mathcal{F}_{PS}$ have computation and communication complexity $O(n \log n)$.

Parameters: Two parties $\mathcal{S}$, and $\mathcal{R}$; vector size L .

Functionality:

- Wait for a permutation π over $[n]$ from $\mathcal{S}$ and input $\mathbf{x} \in \mathbb{F}_q^L$ from $\mathcal{R}$.
- Give the output $\mathbf{y} \in \mathbb{F}_q^L$ to $\mathcal{R}$ and $\mathbf{y}' \in \mathbb{F}_q^L$ to $\mathcal{S}$ where $\mathbf{y}[i] \oplus \mathbf{y}'[i] = \mathbf{x}[\pi(i)]$.

Fig. 7. Ideal functionality for Permute + Share $\mathcal{F}_{PS}$.

We present the shuffled PMT protocols based on Permute + Share $\mathcal{F}_{\text{PS}}$ in Fig. 8. Compared to DH-based construction, this construction incurs low computation costs with higher communication complexity $O((\alpha + \log L)L)$.

Input: $\mathcal{R}$ inputs $e \in \mathbb{F}_q^L$; $\mathcal{S}$ inputs $S \subset \mathbb{F}_q^{\alpha \times L}$ and a permutation π over $[L]$.

Protocol:

- $\mathcal{R}$ and $\mathcal{S}$ invoke $\mathcal{F}_{\text{PS}}$ where $\mathcal{S}$ as the sender with the input π and $\mathcal{R}$ as the receiver with the input e . $\mathcal{F}_{\text{PS}}$ outputs $\mathbf{u}$ to $\mathcal{S}$ and $\mathbf{v}$ to $\mathcal{R}$ where $\mathbf{u}[i] \oplus \mathbf{v}[i] = e[\pi(i)]$.
- $\mathcal{R}$ and $\mathcal{S}$ invoke an OPRF protocol where $\mathcal{R}$ inputs $\mathbf{v}$ and obtains $[F_k(\mathbf{v}[i])]_{i \in [L]}$ while $\mathcal{S}$ gets the PRF key k .
- $\mathcal{S}$ computes $S' = [S'_i = \{F_k(s \oplus \mathbf{u}[i]) \mid s \in S[\pi(i)]\}]$ and shuffle each S'_i .
- $\mathcal{S}$ sends S' to $\mathcal{R}$.
- $\mathcal{R}$ sets $\mathbf{b}[i] = 1$ if $F_k(\mathbf{v}[i]) \in S'[i]$ and $\mathbf{b}[i] = 0$ otherwise.

Fig. 8. Protocol $\Pi_{\text{s-PMT}}$ from $\mathcal{F}_{\text{PS}}$.

Theorem 4. *Protocol $\Pi_{\text{s-PMT}}$ from $\mathcal{F}_{\text{PS}}$ in Fig. 8 is secure under the semi-honest model assuming semi-honest secure OPRF and Permute + Share.*

Proof. We construct simulators $\text{SIM}_{\mathcal{S}}$ and $\text{SIM}_{\mathcal{R}}$ to simulate the view of corrupt $\mathcal{S}$ and $\mathcal{R}$ respectively and argue the indistinguishability of the simulated transcript from the real execution.

Corrupt Sender $\mathcal{S}$. $\text{SIM}_{\mathcal{S}}(S, \pi)$ randomly selects L values $\mathbf{u}$ and invokes the simulator of Permute + Share $\text{SIM}_{\mathcal{S}}^{\text{PS}}(\pi, \mathbf{u})$ and appends the output to the view. $\text{SIM}_{\mathcal{S}}$ then selects a random key k of PRF, invokes the simulator of OPRF $\text{SIM}_{\mathcal{S}}^{\text{OPRF}}(k)$, and appends the output to the view. We show $\text{SIM}_{\mathcal{S}}(S, \pi, \pi')$ is indistinguishable from $\mathcal{S}$'s view in the real execution by the following hybrids.

Hybrid₀: $\mathcal{S}$'s view in the real protocol.

Hybrid₁: Same as Hybrid₀ except that $\text{SIM}_{\mathcal{S}}$ randomly chooses $\mathbf{u}$ as the output of $\mathcal{F}_{\text{PS}}$ and invokes the simulator $\text{SIM}_{\mathcal{S}}^{\text{PS}}(\pi, \mathbf{u})$ to produce the simulated view for $\mathcal{S}$. The security of Permute Share ensures Hybrid₁ is computationally indistinguishable from Hybrid₀.

Hybrid₂: Same as Hybrid₁ except that $\text{SIM}_{\mathcal{S}}$ randomly chooses a PRF key k and invokes the simulator $\text{SIM}_{\mathcal{S}}^{\text{OPRF}}(k)$ to produce the simulated view for $\mathcal{S}$. The security of OPRF ensures Hybrid₂ is computationally indistinguishable from Hybrid₁.

Corrupt Receiver $\mathcal{R}$. $\text{SIM}_{\mathcal{R}}(\mathbf{e}, \mathbf{b})$ simulates the view of corrupt Receiver $\mathcal{R}$ as follows. $\text{SIM}_{\mathcal{R}}$ randomly selects L values $\mathbf{v}$ and invokes the simulator of $\text{Permute} + \text{Share}$ $\text{SIM}_{\mathcal{R}}^{\text{PS}}(\mathbf{e}, \mathbf{v})$ and appends the output to the view. $\text{SIM}_{\mathcal{R}}$ randomly selects L values $\mathbf{v}'$ and invokes the simulator of OPRF $\text{SIM}_{\mathcal{R}}^{\text{OPRF}}(\mathbf{v}, \mathbf{v}')$ and appends the output to the view. For $\mathbf{b}[i] = 1$, $\text{SIM}_{\mathcal{R}}$ adds $\mathbf{v}'[i]$ to $\mathbf{S}^*[i]$ and pads $\mathbf{S}^*[i]$ with $\alpha - 1$ random values. For $\mathbf{b}[i] = 0$, $\text{SIM}_{\mathcal{R}}$ randomly selects α values to form $\mathbf{S}_i^*$ and appends $\mathbf{S}_i^*$ to the view.

We show $\text{SIM}_{\mathcal{R}}(\mathbf{e}, \mathbf{b})$ is indistinguishable from $\mathcal{R}$'s view in the real execution by the following hybrids.

Hybrid₀: $\mathcal{R}$'s view in the real protocol.

Hybrid₁: Same as Hybrid₀ except that $\text{SIM}_{\mathcal{R}}$ randomly selects $\mathbf{v}$ and invokes the simulator $\text{SIM}_{\mathcal{R}}^{\text{PS}}(\mathbf{e}, \mathbf{v})$ to produce the simulated view for $\mathcal{R}$. The security of $\text{Permute} + \text{Share}$ protocol ensures Hybrid₁ is computationally indistinguishable from Hybrid₀.

Hybrid₂: Same as Hybrid₁ except that $\text{SIM}_{\mathcal{R}}$ randomly selects $\mathbf{v}'$ and invokes the simulator $\text{SIM}_{\mathcal{R}}^{\text{OPRF}}(\mathbf{v}, \mathbf{v}')$ to produce the simulated view for $\mathcal{R}$. The security of OPRF protocol ensures Hybrid₂ is computationally indistinguishable from Hybrid₁.

Hybrid₃: Same as Hybrid₂ except that $\text{SIM}_{\mathcal{R}}$ generates $\mathbf{S}^*$ instead of $\mathbf{S}'$. Since each element of $\mathbf{S}'_i$ is a PRF and $\mathbf{S}^*, \mathbf{S}'$ output the same result, Hybrid₃ is computationally indistinguishable from Hybrid₂.

References

1. Burkhart, M., Strasser, M., Many, D., Dimitropoulos, X.: {SEPIA}:{Privacy-Preserving} aggregation of {Multi-Domain} network events and statistics. In: 19th USENIX Security Symposium (USENIX Security 2010), pp. 223–240 (2010)
2. Chase, M., Ghosh, E., Poburinnaya, O.: Secret-shared shuffle. In: Moriai, S., Wang, H. (eds.) ASIACRYPT 2020. LNCS, vol. 12493, pp. 342–372. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-64840-4_12
3. Chen, H., Huang, Z., Laine, K., Rindal, P.: Labeled psi from fully homomorphic encryption with malicious security. In: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security, pp. 1223–1237. ACM (2018)
4. Chen, H., Laine, K., Rindal, P.: Fast private set intersection from homomorphic encryption. In: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security, pp. 1243–1255. ACM (2017)
5. Chen, Y., Zhang, M., Zhang, C., Dong, M., Liu, W.: Private set operations from multi-query reverse private membership test. Cryptology ePrint Archive (2022)
6. Cong, K., et al.: Labeled psi from homomorphic encryption with reduced computation and communication. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, pp. 1135–1150 (2021)
7. Dumas, J.G., Galan, A., Grenet, B., Maignan, A., Roche, D.S.: Communication optimal unbalanced private set union. arXiv preprint [arXiv:2402.16393](https://arxiv.org/abs/2402.16393) (2024). <https://doi.org/10.48550/arXiv.2402.16393>

8. Freedman, M.J., Ishai, Y., Pinkas, B., Reingold, O.: Keyword search and oblivious pseudorandom functions. In: Kilian, J. (ed.) TCC 2005. LNCS, vol. 3378, pp. 303–324. Springer, Heidelberg (2005). https://doi.org/10.1007/978-3-540-30576-7_17
9. Garimella, G., Mohassel, P., Rosulek, M., Sadeghian, S., Singh, J.: Private set operations from oblivious switching. In: Garay, J.A. (ed.) PKC 2021. LNCS, vol. 12711, pp. 591–617. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-75248-4_21
10. Goldreich, O.: Foundations of Cryptography: Volume 2, Basic Applications. Cambridge university press, Cambridge (2009)
11. Ishai, Y., Kilian, J., Nissim, K., Petrank, E.: Extending oblivious transfers efficiently. In: Boneh, D. (ed.) CRYPTO 2003. LNCS, vol. 2729, pp. 145–161. Springer, Heidelberg (2003). https://doi.org/10.1007/978-3-540-45146-4_9
12. Jarecki, S., Liu, X.: Fast secure computation of set intersection. In: Garay, J.A., De Prisco, R. (eds.) SCN 2010. LNCS, vol. 6280, pp. 418–435. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-15317-4_26
13. Jia, Y., Sun, S.F., Zhou, H.S., Du, J., Gu, D.: Shuffle-based private set union: faster and more secure. In: USENIX Security, p. 18. USENIX Security (2022)
14. Kirsch, A., Mitzenmacher, M., Wieder, U.: More robust hashing: cuckoo hashing with a stash. SIAM J. Comput. **39**(4), 1543–1561 (2009)
15. Kolesnikov, V., Rosulek, M., Trieu, N., Wang, X.: Scalable private set union from symmetric-key techniques. In: Galbraith, S.D., Moriai, S. (eds.) ASIACRYPT 2019. LNCS, vol. 11922, pp. 636–666. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-34621-8_23
16. Mohassel, P., Sadeghian, S.: How to hide circuits in MPC an efficient framework for private function evaluation. In: Johansson, T., Nguyen, P.Q. (eds.) EUROCRYPT 2013. LNCS, vol. 7881, pp. 557–574. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-38348-9_33
17. Pinkas, B., Rosulek, M., Trieu, N., Yanai, A.: SpOT-light: lightweight private set intersection from sparse OT extension. In: Boldyreva, A., Micciancio, D. (eds.) CRYPTO 2019. LNCS, vol. 11694, pp. 401–431. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-26954-8_13
18. Pinkas, B., Schneider, T., Segev, G., Zohner, M.: Phasing: private set intersection using permutation-based hashing. In: USENIX Security Symposium, pp. 515–530 (2015)
19. Pinkas, B., Schneider, T., Weinert, C., Wieder, U.: Efficient circuit-based PSI via cuckoo hashing. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018. LNCS, vol. 10822, pp. 125–157. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-78372-7_5
20. Pinkas, B., Schneider, T., Zohner, M.: Faster private set intersection based on ot extension. In: USENIX Security Symposium, pp. 797–812 (2014)
21. Pinkas, B., Schneider, T., Zohner, M.: Scalable private set intersection based on ot extension. ACM Trans. Priv. Secur. **21**(2), 7:1–7:35 (2011). <https://doi.org/10.1145/3154794>
22. Raab, M., Steger, A.: “Balls into Bins” — a simple and tight analysis. In: Luby, M., Rolim, J.D.P., Serna, M. (eds.) RANDOM 1998. LNCS, vol. 1518, pp. 159–170. Springer, Heidelberg (1998). https://doi.org/10.1007/3-540-49543-6_13
23. Ramanathan, S., Mirkovic, J., Yu, M.: Blag: improving the accuracy of blacklists. In: NDSS (2020)
24. Son, Y., Jeong, J.: Psi with computation or circuit-psi for unbalanced sets from homomorphic encryption. In: Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security, pp. 342–356 (2023)

25. Tu, B., Chen, Y., Liu, Q., Zhang, C.: Fast unbalanced private set union from fully homomorphic encryption. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, pp. 2959–2973. ACM, Copenhagen (2023). <https://doi.org/10.1145/3576915.3623064>
26. Wu, M., Yuen, T.H.: Efficient unbalanced private set intersection cardinality and user-friendly privacy-preserving contact tracing. In: 32nd USENIX Security Symposium (USENIX Security 2023), pp. 283–300 (2023). <https://www.usenix.org/conference/usenixsecurity23/presentation/wu-mingli>
27. Zhang, C., Chen, Y., Liu, W., Zhang, M., Lin, D.: Linear private set union from {Multi-Query} reverse private membership test. In: 32nd USENIX Security Symposium (USENIX Security 2023), pp. 337–354 (2023)



BMI: Bounded Mutual Information for Efficient Privacy-Preserving Feature Selection

David Eklund¹(✉), Alfonso Iacovazzi¹, Han Wang¹, Apostolos Pyrgelis¹,
and Shahid Raza^{1,2}

¹ RISE Research Institutes of Sweden, Box 1263, 16429 Kista, Sweden
{david.eklund,alfonso.iacovazzi,han.wang,apostolos.pyrgelis,
shahid.raza}@ri.se

² Mälardalen University, Västerås, Sweden

Abstract. We introduce low complexity bounds on mutual information for efficient privacy-preserving feature selection with secure multi-party computation (MPC). Considering a discrete feature with N possible values and a discrete label with M possible values, our approach requires $O(N)$ multiplications as opposed to $O(NM)$ in a direct MPC implementation of mutual information. Our experimental results show that for regression tasks, we achieve a computation speed up of over 1,000× compared to a straightforward MPC implementation of mutual information, while achieving similar accuracy for the downstream machine learning model.

Keywords: Feature Selection · Mutual Information · Secure Multi-party Computation · Privacy

1 Introduction

Feature selection is a crucial preparatory data process for machine learning (ML) as it reduces the dimension of the ambient data space and retains the most suitable features for the downstream ML algorithm [23]. With the increased availability of sensitive data in multiple silos and the emergence of collaborative ML solutions, e.g., in the medical [46] and financial [50] sectors, there is an imperative need for enabling feature selection on distributed datasets used for ML purposes while preserving the privacy of the data [39].

Prior solutions for feature selection on distributed datasets provide limited privacy protection as they rely on data holders to share the plain results of statistics or intermediate computations on their data to a coordinating server [7, 20, 59]. On the other hand, earlier works on privacy-preserving machine learning focus primarily on model training and inference [32, 52, 56], and they do not consider feature selection or other data preparation steps, which aim to improve the performance and accuracy of the downstream ML task.

In this paper we address this gap by contributing novel methods to the research of privacy-preserving feature selection using secure multi-party computation (MPC) [1, 39, 43]. We focus on feature selection with the feature filtering method which we instantiate with mutual information. We employ a federated approach that leverages on local computations by the data holders and we combine it with MPC to protect the end-to-end privacy of the feature selection process. To efficiently compute mutual information in MPC, we derive an upper bound based on the concavity of entropy and cross-entropy (Proposition 1), which exploits the similarities between the client distributions and the aggregate global distribution. Assuming a discrete feature with N possible values and a discrete label with M possible values, the upper bound requires $O(N)$ multiplications and it avoids logarithm computations which are expensive to realize in MPC; whereas, a straightforward MPC implementation of mutual information results in $O(NM)$ multiplications (Sect. 5.4).

We implement our protocol in the MP-SPDZ library and we experimentally evaluate it on multiple datasets and ML tasks (Sect. 6.2). Our results show that for regression tasks, our protocol results in a $1,000\times$ computation speed up with retained model accuracy for the feature selection process compared to a direct MPC implementation of mutual information. Similar improvements are achieved in terms of communication as the communication cost of the base MPC protocols employed is linear in the number of multiplications [5].

2 Related Work

We first discuss traditional feature selection methods (Sect. 2.1) and federated feature selection approaches (Sect. 2.2). Then, in Sect. 2.3, we examine the state-of-the-art on privacy-preserving feature selection, and finally, we review privacy-preserving machine learning techniques (Sect. 2.4).

2.1 Feature Selection

In a machine learning pipeline, feature selection refers to the process of selecting a subset of the input variables to train an optimized model for a downstream task [23]. Common approaches for feature selection are: (i) filter methods which rank features based on statistics [8, 35], (ii) wrapper methods which evaluate features based on model performance [33, 57], (iii) embedded methods which integrate model training and feature selection [48, 54], and (iv) unsupervised techniques which find hidden structure in the data using, e.g., similarity [41] or clustering [36]. We refer to [38] for an exhaustive survey on feature selection, and we note that these methods are envisioned for centralized data settings without privacy considerations, i.e., with full access to the dataset.

2.2 Federated Feature Selection

These works perform feature selection on datasets that are distributed among multiple entities (data holders). They work as follows: (a) each data holder

identifies the most relevant features on its own part of the dataset, and (b) a (coordinator) server combines the local feature sets to derive the important features for the global dataset. Yao et al. [59] propose a feature fusion method based on convolutions and weight learning that reduces the high communication costs of federated learning. Banerjee et al. [7] present an information-theoretic approach based on feature-feature and feature-class mutual information. Hu et al. [24] propose a federated evolutionary feature selection method based on particle swarm optimization (PSO), and Fu et al. [20] propose FEAST, a feature selection method for vertical federated learning based on conditional mutual information. Other works tackle the federated feature selection problem for specific use-cases. Qin and Kondo [45] employ a greedy algorithm that selects the best features for intrusion detection by evaluating the model accuracy over multiple feature subsets. Cassara et al. [10] propose a federated feature selection method for autonomous vehicles (AVs): Each AV executes local feature selection and generates a probability distribution of the information in features. Then, the edge server derives the global feature rank with a Bayesian-based aggregation approach. Finally, Zhang et al. [62] propose a federated feature selection method for Internet of Things (IoT) networks. Each IoT device executes a 1-class support vector machine algorithm and eliminates irrelevant features via hierarchical clustering; then, the edge server computes the intersection of the local feature sets and yields the global feature set. While these works employ a federated approach to feature selection, they provide limited privacy protection as they rely on each data holder sharing intermediate results computed on their (sensitive) datasets (e.g., local feature scores, sets, etc.) with the coordinator server.

2.3 Privacy-Preserving Feature Selection

These solutions protect data privacy while performing the feature selection process. Jafer et al. [25] introduce a metric that estimates the private information in a feature and its utility for a downstream classification task and is used to filter out features. However, they assume a trusted party that performs the feature selection. Alishahi and Moghtadaiee [2] evaluate the effects of anonymization on feature selection methods that use Pearson correlation and information gain. Alishahi et al. [3] model the estimation of statistics for feature selection as frequency estimation problems and employ local differential privacy to protect data privacy. Overall, anonymization techniques significantly degrade the utility of the feature selection process [2,3], thus raising the need for accurate solutions.

Other works employ secure multi-party computation (MPC) techniques that perform accurate feature selection without needing trusted parties. Banerjee and Chakravarty [6] design a secure aggregation protocol for feature selection based on virtual dimensionality reduction. Similarly, Sheiksalishahi and Martinelli [53] use secure aggregation to estimate the utility (feature score) and privacy (degree of uncertainty) of each feature in a distributed dataset. Rao et al. [47] employ 2PC to estimate the χ^2 statistic of binary features. But, as these works rely solely on secure aggregation, they leak statistical (aggregate) information about

the input data. To this end, Li et al. [39] implement an end-to-end MPC protocol that reveals neither the feature values nor the feature sets to the computing parties. Similarly, Ono et al. [43] implement a consistency-based feature selection method using homomorphic encryption, and Akavia et al. [1] employ 2PC for recursive feature elimination with sparse linear regression under the L0 constraint. Similar to these works, we design an MPC protocol for feature selection without leaking information in the process. However, we employ a federated approach to improve the efficiency of MPC and focus on different feature selection method; we implement a filter method based on mutual information and contribute efficient approximations to realize it under MPC.

Another line of research combines federated feature selection techniques with MPC to enhance the privacy of the former and the performance of the latter. Dankar et al. [16] employ a distributed statistical learning approach: Each data holder locally computes its feature selection vector and these vectors are aggregated with a secure median protocol. Zhang et al. [61] propose MPC protocols for estimating the Gini impurity of features in vertical federated learning. In the same setting, Li et al. [37] perform feature filtering with partial homomorphic encryption: Each data holder employs Gaussian stochastic dual-gates to approximate the Gini impurity of each feature, and the feature importance is updated while the federated training continues. Finally, Froelicher et al. [18] employ a federated approach to perform principal component analysis over a distributed dataset and they employ MPC to secure any information exchanged in the process. Similar to these works, we employ a federated approach to feature selection and we use MPC to enhance its privacy. However, our focus is on mutual information-based feature filtering for distributed datasets.

2.4 Privacy-Preserving Machine Learning

These works protect the privacy of sensitive data used for machine learning by incorporating privacy-enhancing technologies in the training process. Examples of such solutions employ secure multi-party computation (MPC) [21, 42], homomorphic encryption (HE) [4, 60], or hybrid MPC/HE approaches [19, 51, 52]. While these works pave the way for privacy-preserving machine learning, they do not consider feature selection (or any other pre-processing steps).

3 Preliminaries

We provide background information for relevant concepts used in this work.

3.1 Feature Selection with the Filter Method

We consider the simplest form of feature selection, namely, filter methods which compute a lightweight statistic and use it to score the features one-by-one; the best features are selected based on a given threshold $\theta \in \mathbf{R}$. Commonly used statistics are variance, Pearson correlation coefficients, mutual information, and

Relief [13]. The most appropriate statistic for feature selection depends on the dataset and the available computational resources. In this work, we focus on mutual information (MI), which measures the degree of independence between the feature and the target.

3.2 Cross Entropy and Mutual Information (MI)

Let x and y be discrete random variables with finite ranges of size N and M , respectively. For $1 \leq i \leq N$ and $1 \leq j \leq M$, let $p_i(x)$ be the probability that $x = i$, $p_j(y)$ the probability that $y = j$, and $p_{i,j}(x, y)$ the joint probability that $x = i$ and $y = j$. Let $p(x) = (p_1(x), \dots, p_N(x))$ and $p(y) = (p_1(y), \dots, p_M(y))$ denote the distributions of x and y , and $p(x, y)$ their joint distribution.

The entropy of x is defined by $H(x) = \sum_{i=1}^N -p_i(x) \log p_i(x)$, where $\log$ denotes the natural logarithm and the convention $0 \log 0 = 0$ is followed. If $N = M$, the difference between $p(x)$ and $p(y)$ can be measured by their Kullback-Leibler divergence (KL-divergence):

$$D_{\text{KL}}(p(x), p(y)) = \sum_{i=1}^N p_i(x) \log \frac{p_i(x)}{p_i(y)},$$

skipping terms with $p_i(x) = 0$ and assuming that $p_i(x) = 0$ whenever $p_i(y) = 0$. If $p_i(y) = 0$ but $p_i(x) \neq 0$ for some i , the KL-divergence is defined to be infinite. The divergence is non-negative or infinite and zero if and only if $p(x) = p(y)$. We use the notation $D_{\text{KL}}(x, y) = D_{\text{KL}}(p(x), p(y))$. Still assuming that $N = M$, the cross-entropy between x and y is:

$$C(x, y) = H(x) + D_{\text{KL}}(x, y) = - \sum_{i=1}^N p_i(x) \log p_i(y).$$

Let $p(x) \otimes p(y)$ denote the product distribution of $p(x)$ and $p(y)$. The mutual information (MI) of x and y is defined by

$$I(x, y) = D_{\text{KL}}(p(x, y), p(x) \otimes p(y)) = \sum_{i=1}^N \sum_{j=1}^M p_{i,j}(x, y) \log \frac{p_{i,j}(x, y)}{p_i(x)p_j(y)}.$$

The mutual information is non-negative or infinite and zero if and only if x and y are independent. The mutual information can be expressed as $I(x, y) = H(x) + H(y) - H(x, y)$, where $H(x, y)$ is the entropy of the joint variable (x, y) .

The following example illustrates how, in some cases, mutual information can detect dependence and be used for filtering out relevant features where feature selection based on variance and correlation coefficients cannot.

Example 1. Consider mutually independent random variables $X_1, X_2, \dots, X_m$ for $m > 2$ uniformly distributed on $[-1, 1]$ and $Y = X_1 X_2$. Then, the variance of X_i is the same for all i and the covariance of X_i and Y is zero for all i . This means that feature selection methods based on variance or correlation coefficient cannot distinguish between the features. However, the mutual information of X_i and Y is zero for $i > 2$ and non-zero for $i \in \{1, 2\}$.

3.3 Secure Multi-party Computation

Secure multi-party computation (MPC) protocols enable a set of parties to jointly perform the computation of a desired function (a circuit) on their private inputs without disclosing them [17]. Since the seminal work of Yao [58], researchers have designed multiple MPC protocols with variable techniques, e.g., garbled circuits [58] and secret sharing [22], variable number of parties and circuit support, and different system and threat models, e.g., honest vs. dishonest majority with passive and active adversaries [15, 17]. As a result, there exist multiple public implementations of MPC [9, 30], and it has found application to various domains such as electronic voting [34], medical studies [26], financial trading [44] and fraud detection [50], and machine learning [32].

In this work, we employ the 2-out-of-3 replication secret sharing scheme proposed by Araki et al. [5] which enables 3-party computation of arithmetic circuits over the ring modulo 2^n for the honest majority passive and active adversarial settings. A secret value $x \in \mathbf{Z}_q$, where $q = 2^n$, is split into three shares x_1, x_2 and x_3 such that $x = x_1 + x_2 + x_3$ and shared among 3 parties P_1, P_2 and P_3 as follows: P_1 receives x_1 and x_2 , P_2 receives x_2 and x_3 , and P_3 receives x_3 and x_1 . This sharing scheme enables each party P_i to compute the addition and multiplication of two secret-shared values, where multiplication requires communication as the parties exchanging a randomized version of their shares [5]. Using additions and multiplication over secret-shared values, we can realize more complex operations such as comparison and division by employing the integer truncation protocols proposed in [11, 12].

4 Problem Statement

We aim to achieve efficient and practical privacy-preserving feature selection using a filter method in an MPC setting. While filter methods offer a widely deployed approach for feature selection, their application with MPC becomes computationally expensive for large datasets. In particular, the number of multiplications performed by the protocol used has significant impact on computational and communications costs. The challenge lies in reconciling these aspects: ensuring privacy through MPC while maintaining computational efficiency for practical use. In the next subsections, we describe the assumed system and threat model (Sect. 4.1) and we provide an overview of our solution (Sect. 4.2).

4.1 System and Threat Model

We consider a setting where multiple data holders (e.g., hospitals, companies) desire to collaboratively train a machine learning model on their sensitive data, hence, they employ a set of computing parties to execute a secure computation protocol towards this goal, e.g., [32, 52, 56]. To ensure the generalization of the global model and improve the computation and communication efficiency of the secure model training process, the data holders wish to collectively filter out

irrelevant features from the global dataset in a pre-processing step. However, this pre-processing step should not reveal any information about the sensitive data of each data holder as this would invalidate the privacy protection of the machine learning pipeline. We assume that the global dataset is horizontally partitioned among k data holders and the number of computing parties is three. The data holders have computing power and network connectivity capabilities, hence, they can perform computations on their local datasets and share intermediate results with the computing servers. We assume that there exist secure communication channels between the data holders and the computing servers, and among the computing servers.

The data holders are honest, and they provide legitimate input for the computation. We assume that the majority of the computing parties are honest-but-curious, i.e., they follow the protocol instructions and do not collude with each other; however, they are curious and attempt to learn as much information as possible for the datasets of the data holders by observing the information they process. We consider both the passive and active adversarial setting for the dishonest party.

4.2 Solution Overview

To realize feature filtering on the global dataset in a privacy-preserving manner, our solution relies on a hybrid approach that complements a federated feature selection process with secure multi-party computation. This approach reduces computational costs by leveraging the data holders' ability to perform local pre-processing while protecting shared information during feature selection. This is achieved through a secure computation protocol executed among the computing parties. Each data holder computes statistics on their local (cleartext) data and secret-shares their results among the computing servers. The servers then collaboratively estimate global statistics using a secure multi-party computation protocol, addressing the honest-but-curious majority threat model.

Our approach incorporates two key aspects. First, we define an approximation of mutual information scores that enables data holders to perform locally a significant portion of the feature scoring computation, reducing the workload on the servers. Second, we integrate MPC for the collaborative estimation of global statistics, ensuring data privacy under the honest-but-curious majority threat model.

5 Building Blocks for Privacy-Preserving Feature Selection with Mutual Information

We present secure protocols for privacy-preserving feature selection based on mutual information. We start by introducing the relevant notation (Sect. 5.1) and a baseline protocol for accurately estimating mutual information in MPC (Sect. 5.2). Then, we derive an upper bound on mutual information (Sect. 5.3) which we use to design a more efficient protocol for its estimation in MPC

Algorithm 1 Base: A Secure Protocol for Mutual Information.

Require: For each local dataset $j \in \{1, \dots, k\}$, secret-shared weighted probabilities $[p_{ijl}]$, where $p_{ijl} = \lambda_j p_{il}(x_j, y_j)$ for each $i \in \{1, \dots, N\}$ and $l \in \{1, \dots, M\}$.

Ensure: Secret-shared approximation of MI $I(x, y)$ of the feature x and the target y .

```

1:  $r_3 \leftarrow 0$ 
2: for  $i = 1, \dots, N$  do
3:   for  $l = 1, \dots, M$  do
4:      $[p_{il}] \leftarrow \sum_j [p_{ijl}]$ 
5:      $[r_3] \leftarrow [r_3] - [p_{il}] \log_2 [p_{il}]$ 
6:   end for
7: end for
8: for  $i = 1, \dots, N$  do
9:    $[a_i] \leftarrow \sum_l [p_{il}]$ 
10: end for
11: for  $l = 1, \dots, M$  do
12:    $[b_l] \leftarrow \sum_i [p_{il}]$ 
13: end for
14:  $[r_1] \leftarrow - \sum_i [a_i] \log_2 [a_i]$ 
15:  $[r_2] \leftarrow - \sum_l [b_l] \log_2 [b_l]$ 
16:  $I(x, y) \leftarrow [r_1] + [r_2] - [r_3]$ 

```

(Sect. 5.4). Finally, we conclude with a protocol for computing the mask of selected features under MPC (Sect. 5.5).

5.1 Notation

We consider a feature x and target variable y with finite ranges of size N and M , respectively. The data is horizontally distributed among k data holders (or clients). For $j \in \{1, \dots, k\}$, n_j denotes the number of samples at client j , $n = \sum_j n_j$ is the total number of samples, and $\lambda_j = n_j/n$ is the weight of client j . We assume that n is publicly known by the data holders and that the same holds for a cutoff parameter $0 < \epsilon \leq \min_j (e^{-1/\lambda_j})$. Moreover, x_j and y_j denote the local feature and target variables. Then, (x, y) follows a mixture distribution of (x_j, y_j) with weights λ_j . The total number of data points n can be computed in MPC by summing secret-shares of $n_1, \dots, n_k$.

We use the notation $[z]$ for a secret-shared fix-point number z . We denote addition, subtraction, and multiplication of secret-shared values by $[z] + [w]$, $[z] - [w]$, and $[z][w]$, respectively.

5.2 A Baseline Secure Protocol for Mutual Information

We first design a baseline protocol for computing mutual information in the setting described in Sect. 4.1. Algorithm 1 computes the MI for one feature x and the target y ; the protocol is applied repeatedly for all features to obtain their scores. Following our federated approach, the data holders locally preprocess their data into an appropriate form, i.e., they compute histograms (joint between

feature and target) of the discrete data scaled with the client weights; then, these histograms are secret-shared among the computing servers and they are used as input to the baseline protocol (Algorithm 1).

Note that to compute entropy, cross-entropy, and mutual information under MPC, we need to approximate the logarithm using MPC, see Appendix B for details on how this is done. We use base 2 logarithms instead of natural logarithms to measure information (the unit of information is bits).

In **Base**, line 4 assembles the joint probabilities and lines 9 and 12 the marginal probabilities. The entropies $H(x, y)$, $H(x)$ and $H(y)$ are called r_3 , r_2 and r_1 in the protocol and are computed according to their definitions on lines 5, 14 and 15. The last line uses the expression $H(x) + H(y) - H(x, y)$ for mutual information.

The **Base** protocol yields $O(NMm)$ multiplications, where m is the number of multiplications required to approximate $z \mapsto z \log_2 z$, and $O(NMk)$ additions. Note that for discretized regression problems and classification problems with many classes, M can be large, hence executing the **Base** protocol can be costly (as illustrated in our experiments in Sect. 6). Thus, we propose to compute a lower complexity bound on mutual information and use it to filter out irrelevant features more efficiently.

5.3 Bounded Mutual Information

We derive an upper bound on mutual information of a mixture distribution and we characterize it in terms of Kullback-Leibler divergences of the mixture and its components. We use this upper bound as an alternative feature score to filter out features with small enough mutual information, i.e., we cut off features at a given threshold of the score. Cutting off features with a small mutual information score implies eliminating features with a weak dependence on the target variable. Note that filtering features based on this upper bound of MI is a conservative approach; all features with relatively strong dependence on the target variable are retained, and some extra features may be included in the selected set.

Our main idea is to use the concavity of entropy to bound from below the global joint entropy $H(x, y)$ in terms of the local joint entropies $H(x_i, y_i)$. This gives us an upper bound on the MI of feature x and target y . Furthermore, to avoid computing $H(x)$, we bound it from above in terms of the cross-entropies $C(x, x_i)$, which together with the concavity bound on $H(x, y)$ yields a looser upper bound of MI. The advantage of using the upper bound instead of the mutual information in a multi-party setting is that the entropies $H(x_i, y_i)$ of each data holder can be computed locally and the cross-entropies $C(x, x_i)$ are cheaper to compute than $H(x)$. This is because $C(x, x_i) = -\sum_{j=1}^N p_j(x) \log p_j(x_i)$ and the logarithms $\log p_j(x_i)$ can be computed locally. This gives rise to considerable improvements in computational and communication costs (see Sect. 5.4 and Sect. 6).

We now derive a Lemma that will help us bound MI. Note that the function $z \mapsto -z \log z$, for $z > 0$, is concave since it has a decreasing derivative. It follows by Jensen's inequality that $H(x) \geq \sum_{i=1}^k \lambda_i H(x_i)$. Further-

more, for all i , $D_{\text{KL}}(p(x), p(x_i)) \geq 0$ and hence $H(x) \leq C(x, x_i)$. Hence, $H(x) = \sum_{i=1}^k \lambda_i H(x) \leq \sum_{i=1}^k \lambda_i C(x, x_i)$. In summary,

$$\sum_{i=1}^k \lambda_i H(x_i) \leq H(x) \leq \sum_{i=1}^k \lambda_i C(x, x_i).$$

Lemma 1. *For the gap of the lower bound we have that*

$$H(x) - \sum_{i=1}^k \lambda_i H(x_i) = \sum_{i=1}^k \lambda_i D_{\text{KL}}(x_i, x).$$

Proof. Note that $\sum_i \lambda_i C(x_i, x) = H(x)$ since $\sum_i \lambda_i (\sum_j -p_j(x_i) \log p_j(x)) = \sum_j -(\sum_i \lambda_i p_j(x_i)) \log p_j(x) = \sum_j -p_j(x) \log p_j(x)$. Furthermore, recall that for each i , $D_{\text{KL}}(x_i, x) = C(x_i, x) - H(x_i)$. Hence,

$$\sum_i \lambda_i D_{\text{KL}}(x_i, x) = \sum_i \lambda_i C(x_i, x) - \sum_i \lambda_i H(x_i) = H(x) - \sum_i \lambda_i H(x_i).$$

We apply the above observations to bound mutual information from above.

Proposition 1. *Let $B(x, y) = \sum_{i=1}^k \lambda_i C(x, x_i) + H(y) - \sum_{i=1}^k \lambda_i H(x_i, y_i)$ and let $z_i = (x_i, y_i)$ and $z = (x, y)$ denote the joint variables. Then,*

1. $I(x, y) \leq B(x, y)$,
2. $B(x, y) - I(x, y) = \sum_{i=1}^k \lambda_i (D_{\text{KL}}(x, x_i) + D_{\text{KL}}(z_i, z))$.

Proof. Following the properties of entropy (Sect. 3.2), we have that $I(x, y) = H(x) + H(y) - H(x, y) \leq \sum_{i=1}^k \lambda_i C(x, x_i) + H(y) - \sum_{i=1}^k \lambda_i H(x_i, y_i)$, which proves the first claim. For the second claim, note that $B(x, y) - I(x, y) = \sum_i \lambda_i C(x, x_i) - H(x) + H(x, y) - \sum_i \lambda_i H(x_i, y_i)$. Furthermore, $\sum_i \lambda_i C(x, x_i) - H(x) = \sum_i \lambda_i (C(x, x_i) - H(x)) = \sum_i \lambda_i D_{\text{KL}}(x, x_i)$. Finally, $H(z) - \sum_i \lambda_i H(z_i) = \sum_i \lambda_i D_{\text{KL}}(z_i, z)$ by Lemma 1.

The second part of the proposition expresses the gap of the upper bound as the weighted mean of KL-divergences between components x_i and (x_i, y_i) , and the mixture distributions x and (x, y) . This means that the bound is tighter when the distributions x_i are in the mean similar to their mixture and, hence, similar to each other.

Remark 1. The bound $B(x, y)$ is useful when the local distributions are approximately equal to the global distributions; their similarity is measured implicitly by the bound. If, for a given i , some of the probabilities $p_i(x_j)$ are small or zero (but not all), the upper bound $B(x, y)$ grows as the cross entropy $C(x, x_i)$ becomes large. To remedy this, we modify the bound by cutting off small probabilities at a given level $0 < \epsilon \leq \min_j (e^{-1/\lambda_j})$. This procedure is justified in Appendix A.

We note here that a simple alternative approach is to only use the concavity bound on entropy and not use the cross entropy bound $H(x) \leq \sum_j \lambda_j C(x, x_i)$ but simply compute $H(x) = \sum_i -p_i(x) \log p_i(x)$ directly at a cost of N evaluations of the function $z \mapsto z \log z$.

Algorithm 2 BMI: A Secure Protocol for the Upper Bound of MI.

Require: For each local dataset $j \in \{1, \dots, k\}$, secret-shared weighted histograms $[a_{ij}]$ and $[b_{lj}]$, weighted logarithms $[c_{ij}]$ and weighted local joint entropies $[d_j]$. Here $a_{ij} = \lambda_j \bar{p}_i(x_j)$ where $\bar{p}_i(x_j) = \max(p_i(x_j), \epsilon)$, $b_{lj} = \lambda_j p_l(y_j)$, $c_{ij} = \lambda_j \log_2 \bar{p}_i(x_j)$ and $d_j = \lambda_j H(x_j, y_j)$, for each $i \in \{1, \dots, N\}$ and $l \in \{1, \dots, M\}$.

Require: Approximation of the upper bound $B(x, y)$ of a feature x and target y .

```

1: for  $i = 1, \dots, N$  do
2:    $[a_i] \leftarrow \sum_j [a_{ij}]$ 
3:    $[b_i] \leftarrow \sum_j [b_{ij}]$ 
4:    $[c_i] \leftarrow \sum_j [c_{ij}]$ 
5: end for
6:  $[r_1] \leftarrow -\sum_i [a_i][c_i]$ 
7:  $[r_2] \leftarrow -\sum_i [b_i] \log_2 [b_i]$ 
8:  $[r_3] \leftarrow \sum_j [d_j]$ 
9:  $B(x, y) \leftarrow [r_1] + [r_2] - [r_3]$ 

```

The Mixture Information. In our experiments, we also consider the mixture information $\sum_{i=1}^k \lambda_i I(x_i, y_i)$, which is a straightforward reduction of the global mutual information to local computations. We sample the underlying random variable of the mixture distribution as follows: first, we select a client i randomly with probabilities $0 < \lambda_1, \dots, \lambda_k \leq 1$ independently of $x_1, y_1, \dots, x_k, y_k$, and then we sample (x_i, y_i) . We call this random variable the mixture variable of $(x_1, y_1), \dots, (x_k, y_k)$.

Example 2. Consider local distributions (X_1, Y_1) and (X_2, Y_2) where X_1 and Y_1 are independent and both uniformly distributed in $[-1, 0]$, while X_2 and Y_2 are independent and uniform in $[0, 1]$. Moreover, consider the mixture variable (X, Y) which samples the clients uniformly with probability $1/2$. In this case, the local mutual information of (X_1, Y_1) and (X_2, Y_2) are both zero, and the mixture information is zero. However, the mixtures X and Y are not independent, thus the mutual information of X and Y is not zero.

Example 2 shows that the actual mutual information can diverge from the mixture information when the client distributions are dissimilar from each other (and the mixture distribution). While computing the mixture information incurs minor computational and communication costs in the multi-party setting compared to centralized computation, it may give incorrect results and be misleading, as we shall see in Sect. 6.2.

5.4 A More Efficient Secure Protocol for Mutual Information

Algorithm 2 shows how to compute with MPC the upper bound $B(x, y)$ for a feature x and target y from Proposition 1. Following our federated approach, the data holders locally preprocess their data: They compute weighted histograms, weighted logarithms, and local joint entropies between feature and target. They

also filter out probabilities smaller than the cutoff parameter ϵ . The results are secret-shared among the computing servers and are used as input to Algorithm 2.

The cutoff parameter $0 < \epsilon \leq \min_j (e^{-1/\lambda_j})$ is computed with MPC as follows: As the total number of data points n is publicly known, so are the weights, and hence each data holder can compute e^{-1/λ_j} locally and ϵ can be determined using MPC protocols for comparison, see [11].

In the **BMI** protocol, lines 2 and 3 assemble the marginal probabilities and r_2 denotes $H(y)$ which is computed according to its definition on line 7. The concavity bound of $H(x, y)$ is denoted r_3 and is computed as a weighted mean of local joint entropies on line 8. The cross-entropy bound of $H(x)$ is denoted r_1 and is computed as weighted mean of cross-entropies on line 6. The last line makes use of the expression $H(x) + H(y) - H(x, y)$ for mutual information.

Remark 2. Since $H(y)$ does not depend on x , there is no need to re-compute it for every feature. In fact, we avoid computing $H(y)$ by using $I(x, y) - H(y)$ instead of $I(x, y)$, or the corresponding upper bound, when comparing feature scores. This is, however, a minor gain when there are many features to score.

Remark 3. After an initial approximate feature filtering step using the MI bound, one can perform accurate filtering for the remaining features using the **Base** protocol. This gives an efficient way of filtering features according to mutual information when the local distributions are sufficiently similar.

Table 1. Base vs. BMI: Number of multiplications and additions per feature.

Operation	Base	BMI
Additions	$O(NMk)$	$O(Nk)$
Multiplications	$O(NMm)$	$O(N)$

The **BMI** protocol yields $O(N)$ multiplications and $O(Nk)$ additions. The most important performance measure is the number of multiplications, as this determines communication costs and dominates computational costs in the multi-party protocol we employ [5]. Table 1 shows that **BMI** achieves a considerable reduction in the number of multiplications compared to **Base**, especially if M and m are large. This gives rise to considerable improvements in computation and communication costs as can be seen in our experimental results (Sect. 6).

Remark 4. The requirement on the data holders to perform pre-processing incurs the risk of introducing mistakes that may compromise performance as well as privacy. It is suitable that all data holders use the same script, provided by the computational parties, for preprocessing their data. In addition, simple checks of the internal consistency of the preprocessed data may be performed at a cost of $O(N)$ multiplications: probabilities are between 0 and 1, probabilities sum to 1 and the cutoff at the threshold ϵ has been done correctly. These checks require

that data holders secret-share their weights and probabilities before thresholding. In addition, a small sample of the logarithms can be checked for correctness to detect mistakes or attempted shortcuts taken by the data holders.

5.5 Secure Feature Filtering Based on Mutual Information

Finally, we formulate an MPC protocol for creating a feature mask which can be used to process the dataset by selecting the designated features. The input to the MPC protocol is a secret-shared vector $V = (s_1, \dots, s_f)$ of scores, with one score for each of feature, and a threshold value $\theta \in \mathbf{R}$. The output is the secret-shared feature mask: a vector of zeros and ones, with a zero if $s_i < \theta$ and 1 otherwise. For this purpose, we use an MPC protocol for comparison: $\text{comp}(z, w) = 1$ if $z < w$ and $\text{comp}(z, w) = 0$ otherwise, see [11]. The feature mask may then be expressed as $(1 - \text{comp}(s_1, \theta), \dots, 1 - \text{comp}(s_f, \theta))$.

Feature selection can be done with full privacy-preservation by following [39] or a similar protocol to project the datasets to the selected features. Alternatively, one may choose to leak information by revealing the feature mask, which enables efficient feature projection in cleartext by the data holders themselves.

6 Experimental Evaluation

We report the results of numerical experiments carried out on a number of datasets and we compare the performance of the **Base** and **BMI** protocols.

6.1 Datasets

We first describe the datasets used for the experimental results reported in Sect. 6.2 for both classification and regression tasks.

SpO2: This dataset consists of time-series of peripheral oxygen saturation levels in the blood recorded on sleeping subjects as part of the SIESTA project [31]. In this dataset, the saturation level is 71%–100% restricted to integer values. The time-series are labeled according to whether a desaturation event occurs, which is a drop in oxygen levels possibly caused by sleep apnea. The labeling was done with the ABOSA desaturation detector [28], see also [27, 29]. We preprocess the data by extracting one histogram for each time-series counting saturation levels; this results in 30 features per sample whose values we quantize into 20 bins.

Synth: This is a synthetic dataset consisting of 20,000 samples drawn from the distribution described in Example 1 with $m = 300$. It contains 300 features but only 2 are important for predicting the target variable, i.e., a regression problem. Before feature selection, we preprocess the data by quantization in 35 bins on the interval $[-1, 1]$.

Gisette: The Gisette dataset is a selection of images from the MNIST handwritten digits dataset.¹ The dataset is based on images from the digits 4 and

¹ <http://yann.lecun.com/exdb/mnist/>.

9. The training set consists of 6,000 random example images from these classes, and we extract 5,000 features from each sample. The dataset is part of the NIPS 2003 variable selection Benchmarks.² We preprocess the data by aggregating feature values into 30 bins.

Spambase: This dataset consists of statistics derived from emails such as word counts and character counts (54 features) and lengths of sequences of capital letters (3 features). The emails are labeled as spam or not spam. The dataset is publicly available³, see also [14]. We normalize the data and we aggregate feature values into 10 bins.

Synth2: This is a synthetic dataset consisting of 10,000 points with 10 features $x_0, \dots, x_9$ uniformly distributed in $[-1, 1]$. The target variable y is the sign of x_0 (1 or -1) except that the value is flipped with probability 10%. In other words, $y = \text{sgn}(x_0) \cdot b$, where b is a Bernoulli variable which is 1 with probability 90% and -1 with probability 10% (if $x_0 = 0$, we let $\text{sgn}(x_0) = 1$). We quantize the feature values in 35 bins on the interval $[-1, 1]$.

Table 2 summarizes the characteristics of the datasets, i.e., the number of samples, number of features, and modelling task. Moreover, it displays how the data is discretized with N bins for the feature variables and M bins for the target variable.

Table 2. Characteristics of the employed datasets.

Dataset	# of Samples	# of Features	Task	N	M
SpO2	24,918	30	Classification	20	2
Synth	20,000	300	Regression	35	35
Gisette	6,000	5,000	Classification	30	2
Spambase	3,681	57	Classification	10	2
Synth2	10,000	10	Classification	35	2

6.2 Experimental Results

We train a Support Vector Machine with Radial Basis Function kernel for classification and regression tasks for all datasets except for **Spambase** where we train a logistic regression model. In each case, we randomly select 80% of the dataset for training and the other 20% is used as test set. We perform feature selection on the training data, and we evaluate the model's accuracy on the test set. To evaluate the computational overhead of the **Base** and **BMI** protocols, we implement them in the MP-SPDZ framework [30] and computations are carried out in the ring $\mathbf{Z}/2^m$ with $m = 128$. We consider the 3-party setting with honest majority and passive as well as active adversaries.

² <https://archive.ics.uci.edu/dataset/170/gisette>.

³ <https://archive.ics.uci.edu/dataset/94/spambase>.

Model Accuracy. We begin with the case where the clients have (roughly) the same data distribution, i.e., a homogeneous (iid) setting, except in the case of the SpO2 dataset. We split the other datasets evenly between three clients in a uniform manner with approximately the same number of data points at each client. For the SpO2 dataset we have split the dataset into three different patient groups of roughly equal size, which corresponds to a more realistic non-iid setting. Table 3 displays the model performance on the test set by reporting accuracy for classification and root mean square error (RMSE) for regression tasks. For comparison purposes, we report the metrics for the case of no feature selection (**NoFS**), the baseline algorithm for mutual information (**Base**), and the feature selection using the upper bound (**BMI**). We display the accuracy for some example values of the threshold $\theta \geq 0$; features with MI below this level are filtered out. We also record the number of features selected by the filter methods **Base** and **BMI** (**F-Base** and **F-BMI**). We observe that for the classification problems, both feature selection methods reduce the number of features substantially while retaining a model accuracy close to the **NoFS** baseline. We also see that **BMI** is a bit more conservative than **Base** as it includes some extra features in the selection. For the regression problem, both methods correctly identify the 2 relevant features, while a model without feature selection has very poor performance.

Table 3. Classification accuracy and regression error (iid case for the last three rows).

Dataset	NoFS	Base	BMI	F-Base	F-BMI	θ
SpO2	90%	90%	90%	4	10	0.1
Synth	0.31	0.046	0.046	2	2	0.3
Gisette	97%	95%	97%	84	129	0.12
Spambase	91%	88%	89%	15	17	0.1

Table 4. Classification accuracy and selected features (non-iid case).

Dataset	NoFS	Base	Mix	BMI	f-Base	f-Mix	f-BMI	θ
Synth2	89%	90%	50%	90%	$\{x_0\}$	$\{x_1\}$	$\{x_0, x_1\}$	0.5
Synth2	89%	90%	50%	89%	$\{x_0\}$	$\{x_1\}$	all 10	0.4

Next, we consider a heterogeneous (non-iid) case with the dataset **Synth2** and illustrate the difference between **Base**, **BMI**, and a fully federated approach that uses mixture information as a feature score. To create a skewed client distribution that misleads the federated approach, we split the dataset into three client sets: Client 1 has the points with $y < 0$ and $x_1 < 0$, client 2 has the points

with $y \geq 0$ and $x_1 \geq 0$, and client 3 has the rest of the dataset. The results are shown in Table 4, where **Base**, **BMI**, and the mixture information score (**Mix**) are compared for some threshold values θ . The table also displays the set of features selected in each case in the columns **f-Base**, **f-BMI** and **f-Mix**. We observe that with $\theta = 0.5$, **Base** and **BMI** manage to pick out the relevant feature x_0 and yield high accuracy, but the skew distribution makes **BMI** select the extra feature x_1 . When $\theta = 0.4$, the output of **Base** remains the same while **BMI** does not manage to filter out any features and high accuracy is retained. Meanwhile, **Mix** consistently picks the wrong feature x_1 which results in low accuracy.

Another view on skew distributions is that the number of data points may be too small to yield a good estimate of the feature selection score. We have investigated this issue on the **Synth** data set. In this setup one client has 5% of the data and the rest is split evenly among the other clients. Table 5 shows that our method works well to a point but when the dataset gets too small, the estimate of mutual information becomes too coarse to filter features effectively.

Table 5. Regression error (**Error**) in case one client has a smaller number of data points (**Datasize**).

Datasize	1000	50	35	25	5
Error	0.046	0.049	0.062	0.21	0.31

Table 6. **Base** vs. **BMI**: Computation time (ms) for passive and active adversaries and # of multiplications required.

Dataset	Base			BMI		
	active	passive	mult	active	passive	mult
SpO2	$15.8 \cdot 10^3$	$1.23 \cdot 10^3$	600	186	7.09	20
Synth	$347 \cdot 10^3$	$32.0 \cdot 10^3$	12600	382	31.3	35
Gisette	$25.5 \cdot 10^3$	$2.11 \cdot 10^3$	900	328	21.8	30
Spambase	$8.22 \cdot 10^3$	692	300	109	6.64	10

Performance Results. Table 6 shows the computation and communication costs of the feature selection step with **Base** and **BMI** implemented under MPC. We express the performance in terms of elapsed time per feature (ms) for both active and passive adversaries, as well as the number of multiplications required per feature. Communication costs a directly proportional to the number of multiplications. We observe that, in the passive setting, **BMI** provides a speedup of $100\times$ over **Base** for the classification problems and about $1000\times$ for the regression problem.

7 Conclusion

In this work, we studied the problem of privacy-preserving feature selection on a dataset distributed among multiple data holders. We proposed an efficient protocol for approximate feature filtering using an upper bound on mutual information combined with MPC. We evaluated the performance of our protocol on multiple machine learning tasks and we demonstrated its advantages. As part of future work, we plan to quantify the privacy leakage stemming from revealing the selected features to the federation.

Acknowledgments. This research is funded by the EU Horizon Europe project HARPOCRATES (Grant ID. 101069535) and H2020 project CONCORDIA (Grant ID. 830927). We thank Tuomas Karhu for preparing the SpO2 data as well as help and advice in the process. We would also like to thank the anonymous reviewers for their comments and suggested improvements.

A Appendix: Cutoff Probability

With $x, x_1, \dots, x_k$ and $\lambda_1, \dots, \lambda_k$ as in Sect. 5.1, we shall see that we can modify the bound $H(x) \leq \sum_j \lambda_j C(x, x_j)$ by cutting off probabilities in $C(x, x_j)$ at a given level $0 < \epsilon \leq \min_j (e^{-1/\lambda_j})$. More in detail, $H(x) \leq \sum_j \lambda_j \bar{C}(x, x_j)$ where $\bar{C}(x, x_j) = \sum_i -\bar{p}_i(x) \log \bar{p}_i(x_j)$, $\bar{p}_i(x_j) = \max(p_i(x_j), \epsilon)$ and $\bar{p}_i(x) = \sum_j \lambda_j \bar{p}_i(x_j)$.

Note that $f(z) = -z \log z$, for $0 < z \leq 1$, increases up to $z = e^{-1}$ where it has its maximum $f(e^{-1}) = e^{-1}$ and decreases after that. Let $g(z_1, \dots, z_k) = (\sum_j -\lambda_j z_j) \sum_j \lambda_j \log z_j$ for $0 < z_1, \dots, z_k \leq 1$. Then, $H(x) = \sum_i f(p_i(x))$ and $\sum_j \lambda_j C(x, x_j) = \sum_i g(p_i(x_1), \dots, p_i(x_k))$. In the same way, $\sum_j \lambda_j \bar{C}(x, x_j) = \sum_i g(\bar{p}_i(x_1), \dots, \bar{p}_i(x_k))$ and using the concavity of the logarithm, we have a term wise inequality $f(p_i(x)) \leq g(p_i(x_1), \dots, p_i(x_k))$.

Lemma 2. *We have the alternative bound $H(x) \leq \sum_j \lambda_j \bar{C}(x, x_j)$.*

Proof. We shall see that $f(p_i(x)) \leq g(\bar{p}_i(x_1), \dots, \bar{p}_i(x_k))$ for all i . Fix $1 \leq i \leq N$ and assume that at least one $p_i(x_l) \leq \epsilon$ so that $\bar{p}_i(x_l) = \epsilon$. Suppose first that $\bar{p}_i(x) < e^{-1}$. Since $p_i(x) \leq \bar{p}_i(x)$, we have that $f(p_i(x)) \leq f(\bar{p}_i(x))$ and $f(\bar{p}_i(x)) \leq g(\bar{p}_i(x_1), \dots, \bar{p}_i(x_k))$ by the concavity of the logarithm. Now suppose that $\bar{p}_i(x) \geq e^{-1}$. In this case, $g(\bar{p}_i(x_1), \dots, \bar{p}_i(x_k)) = -\bar{p}_i(x) \sum_j \lambda_j \log \bar{p}_i(x_j) \geq \bar{p}_i(x)(-\lambda_l \log \epsilon) \geq \bar{p}_i(x)$ since $-\lambda_l \log \epsilon \geq 1$. Finally, $\bar{p}_i(x) \geq e^{-1} \geq f(p_i(x))$.

B Appendix: Computing the Logarithm in MPC

To compute entropy, cross-entropy, and mutual information under MPC, we need to evaluate the logarithm, or more precisely the function $z \mapsto z \log z$. We use base 2 logarithms instead of natural logarithms to measure information (the unit

of information is bits). Converting to natural logarithms would cost one extra multiplication by $\ln 2$ per feature.

To approximate $\log_2 z$ we convert z to base-2 floating point $z = w \cdot 2^p$ where p is an integer and $1/2 < w \leq 1$. Note that $\log_2 z = p + \log_2 w$ and employ a Padé rational function approximation to approximate $\log_2 w$ to the desired precision [49]. This approximation applied to a secret-shared value $[z]$ is denoted $\log_2[z]$. Note that there exist alternative ways to approximate the logarithm, for example Taylor polynomials. Compared to Taylor expansions however, Padé approximants are rational functions that allow using lower degree polynomials to approximate the function up to a given order [49]. This means that fewer multiplications are needed in the MPC protocol. For instance, in our experiments in Sect. 6.2, the logarithm is approximated as a quotient of two cubic polynomials while agreeing with the logarithm up to order 6. We have also considered alternatives such as CORDIC type approximations [55], other iterative schemes [40] or using logarithm tables, but have chosen to use Padé approximations based on test implementations and the number of multiplications needed to achieve sufficient precision.

References

1. Akavia, A., Galili, B., Shaul, H., Weiss, M., Yakhini, Z.: Privacy preserving feature selection for sparse linear regression. *Cryptology ePrint Archive*, Paper 2023/1354 (2023). <https://eprint.iacr.org/2023/1354>
2. Alishahi, M., Moghtadaiee, V.: Feature selection on anonymized datasets. In: *IEEE International Conference on Dependable, Autonomic and Secure Computing (DASC)* (2023)
3. Alishahi, M., Moghtadaiee, V., Navidan, H.: Add noise to remove noise: local differential privacy for feature selection. *Comput. Secur.* (2022)
4. Aono, Y., Hayashi, T., Wang, L., Moriai, S., et al.: Privacy-preserving deep learning via additively homomorphic encryption. *IEEE Trans. Inf. Forensics Secur.* (2017)
5. Araki, T., Furukawa, J., Lindell, Y., Nof, A., Ohara, K.: High-throughput semi-honest secure three-party computation with an honest majority. In: *Proceedings of the 2016 ACM Conference on Computer and Communications Security* (2016)
6. Banerjee, M., Chakravarty, S.: Privacy preserving feature selection for distributed data using virtual dimension. In: *Proceedings of the 20th ACM International Conference on Information and Knowledge Management* (2011)
7. Banerjee, S., Elmroth, E., Bhuyan, M.: Fed-FiS: a novel information-theoretic federated feature selection for learning stability. In: Mantoro, T., Lee, M., Ayu, M.A., Wong, K.W., Hidayanto, A.N. (eds.) *ICONIP 2021. CCIS*, vol. 1516, pp. 480–487. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-92307-5_56
8. Battiti, R.: Using mutual information for selecting features in supervised neural net learning. *IEEE Trans. Neural Netw.* (1994)
9. Bogdanov, D., Laur, S., Willemson, J.: Sharemind: a framework for fast privacy-preserving computations. In: Jajodia, S., Lopez, J. (eds.) *ESORICS 2008. LNCS*, vol. 5283, pp. 192–206. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-88313-5_13
10. Cassara, P., Gotta, A., Valerio, L.: Federated feature selection for cyber-physical systems of systems. *IEEE Trans. Veh. Technol.* (2022)

11. Catrina, O., de Hoogh, S.: Improved primitives for secure multiparty integer computation. In: Garay, J.A., De Prisco, R. (eds.) SCN 2010. LNCS, vol. 6280, pp. 182–199. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-15317-4_13
12. Catrina, O., Saxena, A.: Secure computation with fixed-point numbers. In: Sion, R. (ed.) FC 2010. LNCS, vol. 6052, pp. 35–50. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-14577-3_6
13. Chandrashekar, G., Sahin, F.: A survey on feature selection methods. *Comput. Electr. Eng.* (2014)
14. Covert, I.C., Qiu, W., Lu, M., Kim, N.Y., White, N.J., Lee, S.-I.: Learning to maximize mutual information for dynamic feature selection. In: *International Conference on Machine Learning*. PMLR (2023)
15. Cramer, R., Damgård, I.B., et al.: *Secure Multiparty Computation*. Cambridge University Press (2015)
16. Dankar, F.K., Madathil, N., Dankar, S.K., Boughorbel, S.: Privacy-preserving analysis of distributed biomedical data: designing efficient and secure multiparty computations using distributed statistical learning theory. *JMIR Med. Inf.* (2019)
17. Evans, D., Kolesnikov, V., Rosulek, M., et al.: A pragmatic introduction to secure multi-party computation. *Found. Trends® Priv. Secur.* (2018)
18. Froelicher, D., et al.: Scalable and privacy-preserving federated principal component analysis. In: *IEEE Symposium on Security and Privacy (SP)* (2023)
19. Froelicher, D., et al.: Scalable privacy-preserving distributed learning. In: *Proceedings on Privacy Enhancing Technologies* (2021)
20. Fu, R., Wu, Y., Xu, Q., Zhang, M.: FEAST: a communication-efficient federated feature selection framework for relational data. *Proc. ACM Manag. Data* (2023)
21. Gascón, A.: Privacy-preserving distributed linear regression on high-dimensional data. *Proc. Priv. Enhancing Technol.* (2017)
22. Goldreich, O., Micali, S., Wigderson, A.: How to play any mental game, or a completeness theorem for protocols with honest majority, pp. 307–328. *Association for Computing Machinery* (2019)
23. Guyon, I., Elisseeff, A.: An introduction to variable and feature selection. *J. Mach. Learn. Res.* (2003)
24. Hu, Y., Zhang, Y., Gong, D., Sun, X.: Multi-participant federated feature selection algorithm with particle swarm optimization for imbalanced data under privacy protection. *IEEE Trans. Artif. Intell.* (2022)
25. Jafer, Y., Matwin, S., Sokolova, M.: A framework for a privacy-aware feature selection evaluation measure. In: *13th Annual Conference on Privacy, Security and Trust (PST)*. IEEE (2015)
26. Kamm, L., Bogdanov, D., Laur, S., Vilo, J.: A new way to protect privacy in large-scale genome-wide association studies. *Bioinformatics* (2013)
27. Karhu, T.: Desaturation event scoring criteria affect the perceived severity of nocturnal hypoxic load. *Sleep Med.* **100**, 479–486 (2022)
28. Karhu, T., Leppänen, T., Töyräs, J., Oksenberg, A., Myllymaa, S., Nikkonen, S.: ABOSA - freely available automatic blood oxygen saturation signal analysis software: structure and validation. *Comput. Methods Programs Biomed.* (2022)
29. Karhu, T., Myllymaa, S., Nikkonen, S., Mazzotti, D.R., Töyräs, J., Leppänen, T.: Longer and deeper desaturations are associated with the worsening of mild sleep apnea: the sleep heart health study. *Front. Neurosci.* **15**, 657126 (2021)
30. Keller, M.: MP-SPDZ: a versatile framework for multi-party computation. In: *Proceedings of the ACM Conference on Computer and Communications Security* (2020)

31. Klish, G., et al.: The siesta project polygraphic and clinical database. *IEEE Eng. Med. Biol. Mag.* (2001)
32. Knott, B., Venkataraman, S., Hannun, A., Sengupta, S., Ibrahim, M., van der Maaten, L.: CrypTen: secure multi-party computation meets machine learning. In: *Advances in Neural Information Processing Systems* (2021)
33. Kohavi, R., John, G.H.: Wrappers for feature subset selection. *Artif. Intell.* (1997)
34. Küsters, R., Liedtke, J., Müller, J., Rausch, D., Vogt, A.: Ordinos: a verifiable tally-hiding e-voting system. In: *IEEE European Symposium on Security and Privacy (EuroS&P)* (2020)
35. Langley, P., et al.: Selection of relevant features in machine learning. In: *Proceedings of the AAAI Fall Symposium on Relevance* (1994)
36. Law, M.H., Figueiredo, M.A., Jain, A.K.: Simultaneous feature selection and clustering using mixture models. *IEEE Trans. Pattern Anal. Mach. Intell.* (2004)
37. Li, A., et al.: Efficient and privacy-preserving feature importance-based vertical federated learning. *IEEE Trans. Mob. Comput.* (2023)
38. Li, J., et al.: Feature selection: a data perspective. *ACM Comput. Surv. (CSUR)* (2017)
39. Li, X., Dowsley, R., De Cock, M.: Privacy-preserving feature selection with secure multiparty computation. In: *International Conference on Machine Learning*. PMLR (2021)
40. Majithia, J., Levan, D.: A note on base-2 logarithm computations. *Proc. IEEE* **61**(10), 1519–1520 (1973)
41. Mitra, P., Murthy, C., Pal, S.K.: Unsupervised feature selection using feature similarity. *IEEE Trans. Pattern Anal. Mach. Intell.* (2002)
42. Nikolaenko, V., Weinsberg, U., Ioannidis, S., Joye, M., Boneh, D., Taft, N.: Privacy-preserving ridge regression on hundreds of millions of records. In: *IEEE Symposium on Security and Privacy (SP)* (2013)
43. Ono, S., Takata, J., Kataoka, M., Shin, T.I.K., Sakamoto, H.: Privacy-preserving feature selection with fully homomorphic encryption. *Algorithms* (2022)
44. Polychroniadou, A., et al.: Prime match: a privacy-preserving inventory matching system. In: *32nd USENIX Security Symposium* (2023)
45. Qin, Y., Kondo, M.: Federated learning-based network intrusion detection with a feature selection approach. In: *IEEE International Conference on Electrical, Communication, and Computer Engineering* (2021)
46. Raab, R.: Federated electronic health records for the European health data space. *Lancet Digit. Health* (2023)
47. Rao, V., Long, Y., Eldardiry, H., Rane, S., Rossi, R., Torres, F.: Secure two-party feature selection. *arXiv preprint [arXiv:1901.00832](https://arxiv.org/abs/1901.00832)* (2019)
48. Romero, E., Sopena, J.M.: Performing feature selection with multilayer perceptrons. *IEEE Trans. Neural Netw.* (2008)
49. Russell Graves-Morris, P., Saff, E.B., Varga, R.S.: Rational Approximation and Interpolation. *Lecture Notes in Mathematics*. Springer, Heidelberg (1983). <https://doi.org/10.1007/BFb0072395>
50. Sangers, A., et al.: Secure multiparty pagerank algorithm for collaborative fraud detection. In: Goldberg, I., Moore, T. (eds.) *FC 2019*. LNCS, vol. 11598, pp. 605–623. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-32101-7_35
51. Sav, S., Diaa, A., Pyrgelis, A., Bossuat, J.-P., Hubaux, J.-P.: Privacy-preserving federated recurrent neural networks. In: *Proceedings on Privacy Enhancing Technologies* (2023)

52. Sav, S., et al.: POSEIDON: privacy-preserving federated neural network learning. In: 28th Network And Distributed System Security Symposium (NDSS). Internet Society (2021)
53. Sheikhalishahi, M., Martinelli, F.: Privacy-utility feature selection as a privacy mechanism in collaborative data classification. In: IEEE International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprises (2017)
54. Stracuzzi, D.J., Utgoff, P.E.: Randomized variable elimination. *J. Mach. Learn. Res.* (2004)
55. Volder, J.: The CORDIC computing technique. In: International Workshop on Managing Requirements Knowledge. IEEE Computer Society (1959)
56. Wagh, S., Gupta, D., Chandran, N.: SecureNN: 3-party secure computation for neural network training. In: Proceedings on Privacy Enhancing Technologies (2019)
57. Yang, J., Honavar, V.: Feature subset selection using a genetic algorithm. *IEEE Intell. Syst. Their Appl.* (1998)
58. Yao, A.C.: Protocols for secure computations. In: 23rd Annual Symposium on Foundations of Computer Science. IEEE (1982)
59. Yao, X., Huang, T., Wu, C., Zhang, R., Sun, L.: Towards faster and better federated learning: a feature fusion approach. In: IEEE International Conference on Image Processing (ICIP) (2019)
60. Zhang, Q., Wang, C., Wu, H., Xin, C., Phuong, T.V.: GELU-Net: a globally encrypted, locally unencrypted deep neural network for privacy-preserved learning. In: International Joint Conferences on Artificial Intelligence (2018)
61. Zhang, R., Li, H., Hao, M., Chen, H., Zhang, Y.: Secure feature selection for vertical federated learning in eHealth systems. In: IEEE International Conference on Communications (2022)
62. Zhang, X., Mavromatics, A., Vafeas, A., Nejabati, R., Simeonidou, D.: Federated feature selection for horizontal federated learning in IoT networks. *IEEE Internet Things* (2023)



Task-Specific Knowledge Distillation with Differential Privacy in LLMs

Sonakshi Garg^(✉)  and Vicenç Torra 

Umeå University, Umeå, Sweden
{sgarg,vtorra}@cs.umu.se

Abstract. Large Language Models (LLMs) have demonstrated state-of-the-art performance across various applications. However, these models which consists of millions of parameters still face challenges due to their computational inefficiency during inference time. In many real-time tasks, the inference time of the tasks should be in the order of milliseconds to be useful. Deploying LLMs in real-time requires reducing inference time. One effective approach is model compression, which reduces the model's parameters. Also, ensuring data privacy is crucial when using LLMs due to their vulnerability to privacy attacks. To address this, LLMs should be trained with Differential Privacy (DP) on private data. Developing a compact, efficient, and private domain-specific language model is an active area of research. To facilitate the deployment of efficiently compressed models with DP while minimizing any degradation in model utility, we propose our framework, Task-Specific Knowledge Distillation with Differential Privacy. In our approach, we prioritize task-specific distillation that generally enhances the performance of downstream tasks unlike traditional methods. Additionally, we leverage transfer learning by utilizing pre-trained models trained on similar tasks, and then use these models for private fine-tuning. We also emphasized on the initialization of student models with pre-trained models from open domains. We demonstrate the effectiveness of our framework on the GLUE benchmark datasets, employing the BERT-base model as our teacher model, and utilizing BERT-tiny and DistilBERT models as student models. Our framework showcases comparable accuracy to non-private learning methods while also improving the accuracy of student models compared to existing baselines. Remarkably, we achieve this while reducing the parameters of student models by 95%.

Keywords: knowledge distillation · large language models · differential privacy

1 Introduction

The emergence of transformer-based models has revolutionized the field of natural language understanding (NLU), marked prominently by the introduction of BERT [7]. Since its inception, transformers have gained widespread popularity, catalyzing groundbreaking advancements in AI-driven technologies. Recent

advancements, especially with the arrival of GPT models [3], have taken AI to new heights. These models have significantly improved accuracy compared to previous top-notch models. These advancements underscore the transformative potential of transformers, fundamentally reshaping the landscape of language processing tasks.

Nevertheless, as the performance of transformer-based models continues to improve significantly, two notable challenges stand out prominently. Firstly, the computational overhead incurred during inference poses a significant bottleneck, hindering the widespread adoption of these models in real-world production scenarios. The imperative for low-latency, fast inference demands innovative solutions to mitigate the computational cost, ensuring seamless integration into practical applications. For example, deploying LLMs on client devices with limited memory resources poses a challenge. Similarly, in real-time systems like text completion or next-word prediction in text editors, minimizing inference latency is crucial [28]. The model’s response time must be within milliseconds to be considered useful. It’s commonly observed that large deep learning models often contain many redundant parameters that could be removed while still maintaining performance [20]. Thus, addressing this challenge is pivotal to unlocking the full potential of transformer-based technologies and realizing their impact at scale.

Simultaneously, safeguarding privacy emerges as another critical concern in the deployment of language models. The utilization of personal sensitive data for model training makes it susceptible to inadvertent disclosure, raising formidable privacy implications. Recent studies have demonstrated that language models can remember training data, which can lead to privacy attacks [5,6]. That is why it is crucial to strengthen privacy protections with strong safeguards to prevent data exposure and privacy breaches. Addressing this challenge is essential for building trust and responsibly using transformer-based models in sensitive areas.

Differential Privacy (DP) [9] is an industry standard privacy model that is commonly used to prevent such information leakage. To ensure privacy for ML models, Differentially private stochastic gradient descent (DPSGD) is employed [1]. The usual way to train LLMs is to first train them on big public datasets and then fine-tune them on specific tasks using private data. Recent research in differentially private training has used this approach in which LLMs are initialized with pre-trained weights, then fine-tuned via DPSGD. This has significantly improved the performance of models [11,16].

1.1 State of the Art

We describe the state of the art techniques for model compression and private model compression as follows.

Model Compression. Due to the significant computational and memory requirements of LLMs, reducing the inference time is crucial. One effective strategy for achieving this is through model compression [4]. Common model compression methodologies include Quantization [2,30], Pruning [12,13] and Knowledge Distillation [24]. We chose to explore Knowledge Distillation (KD) further

due to its benefits and recent performance. KD distills knowledge from a larger teacher model into a smaller student model. It serves three important aspects: model compression, transfer of knowledge and improving generalization of student models. On the other hand, task-specific knowledge distillation [14, 25] can be considered as an application of traditional KD to a specific task or domain. In this, the teacher model is trained on the specific task of interest. This helps the student model to capture task-specific knowledge from teacher model, which leads to improved performance. Our research specifically targets model compression techniques, with a focus on Task-Specific Knowledge Distillation. This area has been relatively under explored in the existing literature.

Privacy Preserving Model Compression. While model compression in LLMs has been successful in the past [23, 27], there has been limited exploration of compressing LLMs with and DP [18, 29]. Achieving DP or KD for an LLM typically involves a trade-off with utility loss. Attempting both simultaneously can result in significant utility loss, highlighting the complexity of this challenge. Some studies have explored model compression in ensemble learning, aiming to transfer knowledge from a group of teacher models to a single student model in private settings, known as the PATE framework [22]. Additionally, various techniques such as knowledge distillation, pruning, and quantization have been investigated for model compression [18]. Firstly, they employed traditional KD and trained the teacher model with DP-SGD, which is computationally demanding. Also, they highlighted the importance of random initialization of student models. However, their comparison was limited to private models trained on downstream tasks. Another study by [29] delved into selective pre-training, a crucial aspect of model training. This inspired our adoption of transfer learning for the pre-training step in our approach. [10] used two ways to transfer knowledge onto the student model: using synthetic data generated by differentially private teacher LLM and other from the teacher model evaluated on synthetic data. Our work introduces novelty by employing task-specific knowledge distillation, known to enhance performance significantly. Leveraging transfer learning, we efficiently pre-trained our models, a crucial step in our approach. Furthermore, we prioritized initializing student models with pre-trained models. Our comprehensive comparison includes evaluations against existing approaches and non-private models, further validating the effectiveness of our methodology.

1.2 Our Proposal

In this paper, we facilitate the deployment of efficiently compressed models with DP while minimizing any degradation in model utility. To this, we propose a framework “Task-specific Knowledge Distillation with Differential Privacy”. The recent works emphasize on the importance of selective pre-training [29]. Pre-training plays a crucial role in maximising the performance of private learning [11]. Drawing inspiration from these findings, we adopted a distinct approach to pre-train our model. Specifically, we used a transfer learning technique [21] to initiate our pre-training process. This involved utilizing a pre-trained BERT

model (teacher model) that had been fine-tuned on a task similar to our downstream target task. This pre-trained teacher model guides transfer learning, helping the student model adapt to the target domain’s nuances. Once the model is pre-trained, the next step is to perform distillation. So, the knowledge from the teacher model is distilled to create a general student model, capturing the knowledge distilled from the teacher’s learning process. Given that our objective is task-specific knowledge distillation, the distillation process is carried out twice to ensure its alignment with the downstream task. This involves first fine-tuning our general models and then performing distillation again. So, now we fine-tune the general teacher model with DPSGD, using task-specific data to adapt it to the nuances of the downstream task. Finally, we perform private task-specific knowledge distillation with the target task on the fine-tuned teacher model and initialized student model.

The student model parameters are initialized based on the parameters obtained from the general student model. It has been shown that the initialization of a student model affects the final accuracy of the distilled models [15]. We used pre-distilled models such as BERT-tiny and DistilBERT as the initializers for student models. Previous studies have focused on initializing the student model with the weights of the teacher model to boost accuracy [18]. In our approach we utilized open domain pre-distilled models for efficient initialization of student models. This helps the student model learn the details of the downstream task while still maintaining privacy. We performed our experiments on GLUE benchmark datasets. The overall framework reduces the parameters of the student model by 95%, resulting in significantly shorter inference times, while ensuring robust privacy protection at the same time. We compared our approach against non-private models and existing studies, and demonstrated comparable accuracy to non-private models and better performance than existing works.

1.3 Contributions

The main contributions of the paper are as follows:

1. We propose a novel framework that combines Task-specific Knowledge Distillation with Differential Privacy for model compression. Unlike traditional KD methods where the distilled model is fine-tuned for a specific task, our approach distills fine-tuned teacher models directly into student models tailored for the target task.
2. We leverage transfer learning by utilizing pre-trained models that have been fine-tuned on tasks similar to the downstream task. This approach helps the student model adapt to the intricacies of the target domain.
3. We demonstrate that our framework significantly improves the utility of student models compared to existing baselines with strong privacy parameters while reducing the model parameters by 95%.
4. We provide empirical evidence that our framework achieves performance levels comparable to non-private learning approaches, thus promoting the adoption of privacy-preserving techniques.

1.4 Organization

The remaining part of the paper is organized in the following manner. Section 2 describes the important concepts that have been used in the paper. Section 3 presents step-by-step explanations of the proposed approach. Section 4 discusses about the experimental setup and data sets involved. Section 5 provides some results and discussion about them. Finally conclusion and future works are presented in Sect. 6.

2 Preliminaries

In this section we will describe the concepts that are used in this paper.

2.1 Differential Privacy

Differential privacy, as proposed by Dwork [9], is a foundational privacy model emphasizing the importance of preventing the disclosure of whether a specific individual's data is present in a database. It ensures that the inclusion or exclusion of a single database item does not substantially alter the outcome of any analysis, thereby safeguarding individual privacy. In essence, a function F_q is considered differentially private if the presence or absence of a record in a database has minimal impact on the resulting analysis.

Definition 1 (*Differential Privacy*). A function F_q for a query q gives ϵ - differential privacy if for all data sets D_1 and D_2 differing in at most one element, and all $S \subseteq \text{Range}(F_q)$

$$\Pr[F_q(D_1) \in S] \leq e^\epsilon \Pr[F_q(D_2) \in S].$$

2.2 Training Transformers with DP

Training transformer-based models, like BERT and GPT, with differential privacy and utilizing DP-SGD (DP stochastic gradient descent) [1] as the optimizer has garnered attention due to its critical role in safeguarding sensitive data. Differential privacy ensures that the model does not overfit to individual data points, thus minimizing privacy risks. DP-SGD enhances standard stochastic gradient descent optimization method by incorporating two key mechanisms: per-example gradient clipping and gaussian noise addition. These augmentations serve to mitigate privacy risks by constraining the magnitude of individual gradients and introducing randomness during parameter updates. Specifically, per-example gradient clipping limits the size of each gradient to prevent outlier contributions, ensuring that no single data point unduly influences the model parameters. Simultaneously, gaussian noise addition injects randomness

into gradient updates, obscuring any specific data point’s impact on the overall optimization process. Mathematically, DPSGD can be formalized as

$$\tilde{g} \leftarrow \frac{1}{n} \sum_{i=1}^n (\nabla \ell(f(x_i; \theta), y_i) + \text{Noise}) \quad (1)$$

where $\tilde{g}$ is the noisy gradient, ℓ is the loss function, f represents the model, x_i and y_i are input-output pairs, θ denotes model parameters, *Noise* specifies the amount of noise we want to add. This approach affects model performance with stringent privacy guarantees, making it pivotal for ensuring data security in transformative machine learning practices.

2.3 Knowledge Distillation

Knowledge distillation [14] is a method used to transfer knowledge from a large, complex model (the teacher) to a smaller, more lightweight model (the student). This technique is particularly useful when the teacher model has large parameters and computational demands, making it impractical for deployment in resource-constrained environments. The process involves training the student model to mimic the outputs of the teacher model, typically represented as probability distributions over classes.

Let p_i^{teacher} denote the probability assigned by the teacher model to class i and p_i^{student} denote the corresponding probability assigned by the student model. These probabilities are obtained through the softmax function applied to the logits produced by each model. Mathematically,

$$p_i^{\text{teacher}} = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \quad p_i^{\text{student}} = \frac{\exp(\tilde{z}_i/T)}{\sum_j \exp(\tilde{z}_j/T)}$$

where z_i and $\tilde{z}_i$ represent the logits for class i generated by the teacher and student models respectively, and T is the temperature parameter controlling the softness of the distributions. The distillation loss aims to minimize the discrepancy between the teacher and student probabilities. A common formulation for the distillation loss involves the Kullback-Leibler (KL) divergence between the teacher and student distributions

$$\mathcal{L}_{\text{distill}} = \text{KL}(p^{\text{teacher}} || p^{\text{student}}) = \sum_i p_i^{\text{teacher}} \log \frac{p_i^{\text{teacher}}}{p_i^{\text{student}}} \quad (2)$$

This loss encourages the student model to match the soft targets produced by the teacher model, capturing the intricate decision boundaries and nuanced relationships present in the original model’s predictions. By jointly optimizing this distillation loss along with the task-specific loss function (e.g., cross-entropy loss), the student model effectively learns from the teacher’s knowledge, resulting in a compact yet proficient model suitable for deployment in resource-constrained environments.

3 Proposed Methodology

In this section, we present our methodology aimed at enhancing the computational efficiency of model inference time while ensuring robust privacy preservation measures. Our research endeavors to develop a comprehensive framework for compressing large language models (teacher model) into more streamlined models (student model), all while safeguarding the privacy of sensitive data.

One prevalent approach to achieving this goal is to extend the knowledge distillation technique to enable private distillation of the student model. This entails compressing the teacher model’s knowledge into the student model while preserving privacy. Specifically, we train the student model using DPSGD. The training objective involves minimizing the KL divergence loss, as described in Eq. 2. However, such an approach would not align with the principles of DPSGD. The KL loss in Eq. 2 relies on the entire dataset, meaning that if we solely employ DPSGD during distillation and loss computation to generate a student model, the resulting model would not ensure privacy.

To address these challenges, we have proposed our methodology. The methodology aims at constructing a privacy-preserving model with enhanced task-specific performance. It begins with the preparation of a general teacher model, a pre-trained BERT model fine-tuned on a data similar to the target data. It is then followed by distillation to derive a general student model. Subsequently, the general teacher model undergoes fine-tuning DPSGD with task-specific data, ensuring adaptability while upholding privacy guarantees. Upon fine-tuning, the student model is initialized with parameters from the distilled general student model. Finally, task-specific knowledge distillation is performed, leveraging DPSGD to transfer distilled knowledge from the fine-tuned teacher model to the initialized student model. Through this iterative process, the student model acquires task-specific knowledge while maintaining robust privacy protection, enabling effective performance in privacy-sensitive scenarios. These steps are elaborated in the subsequent sections, and step-by-step procedure could be found in Algorithm 1.

3.1 Preparation of General Teacher Model

The first step involves the preparation of a general teacher model. This model serves as a foundational component for the subsequent fine-tuning and distillation processes. Initially, a pre-trained BERT model, which has undergone specific fine-tuning on a task similar to the target task, is selected. This pre-trained teacher model encapsulates valuable knowledge learned from a related domain, facilitating transfer learning. Subsequently, general distillation is performed on the teacher model to obtain a distilled general student model. This distillation process aims to capture the essential knowledge and patterns learned by the teacher model, thereby compressing the information into a more compact form suitable for further adaptation. This step is illustrated in Step 1 of Fig. 1.

3.2 Private Fine-Tuning of General Teacher Model

The fine-tuning process is crucial for enhancing the performance and adaptability of the general teacher model to the specific nuances of the target task. Employing Differential Privacy Stochastic Gradient Descent (DPSGD), the general teacher model undergoes fine-tuning using task-specific data. This involves iteratively updating the model parameters based on computed gradients while incorporating privacy-preserving mechanisms such as gradient clipping and noise addition. This requires a privacy budget of (ϵ_1, δ_1) from the total budget (ϵ, δ) . This step is visually represented in Step 2 of Fig. 1. By fine-tuning with DPSGD, the model learns to adapt to the intricacies of the target task while ensuring robust privacy guarantees, thus striking a balance between performance and privacy preservation (Table 1).

Algorithm 1. Task-Specific Knowledge Distillation with Differential Privacy

Input: Pre-trained teacher model parameters θ_{teacher} , task-specific data, ϵ, δ

Output: Student model parameters $|\theta_{\text{student}}| < \gamma \cdot |\theta_{\text{teacher}}|$ & θ_{student} are (ϵ, δ) -DP

1. General Teacher Model

Take a pre-trained teacher model with θ_{teacher} on source task

$\theta_{\text{general_student}} \leftarrow \text{Distill}(\theta_{\text{teacher}})$

2. Private Fine Tuning

for each mini-batch X_i in task-specific data **do**

 Compute gradients: $\nabla L(\theta_{\text{teacher}}, X_i)$

 Clip gradients: $\nabla_{\text{clipped}} \leftarrow \text{clip}(\nabla L(\theta_{\text{teacher}}, X_i), \text{clip_value})$

 Add noise: $\nabla_{\text{noisy}} \leftarrow \nabla_{\text{clipped}} + \text{noise}(\epsilon_1, \text{noise_scale})$

 Update parameters: $\theta_{\text{teacher}} \leftarrow \theta_{\text{teacher}} - \text{learning_rate} \times \nabla_{\text{noisy}}$

end for

3. Initialization of Student Model

$\theta_{\text{student}} \leftarrow \theta_{\text{general_student}}$

4. Private Task Specific Distillation

for each mini-batch X_i in task-specific data **do**

 Compute gradients: $\nabla L(\theta_{\text{student}}, X_i)$

 Clip gradients: $\nabla_{\text{clipped}} \leftarrow \text{clip}(\nabla L(\theta_{\text{student}}, X_i), \text{clip_value})$

 Add noise: $\nabla_{\text{noisy}} \leftarrow \nabla_{\text{clipped}} + \text{noise}(\epsilon_2, \text{noise_scale})$

 Update parameters: $\theta_{\text{student}} \leftarrow \theta_{\text{student}} - \text{learning_rate} \times \nabla_{\text{noisy}}$

end for

3.3 Initialization of Student Model

After fine-tuning the teacher model, the student model is initialized using the parameters obtained from the previously distilled general student model. Since

Table 1. Notations

θ_{teacher}	teacher model parameters
θ_{student}	student model parameters
ϵ	privacy loss parameter
δ	probability of privacy guarantee being violated
γ	scaling factor (level of compression)
(ϵ_1, δ_1)	privacy budget for private fine-tuning of teacher model
(ϵ_2, δ_2)	privacy budget for task-specific knowledge distillation

the initialization of student model is from publicly pre-trained model, thus privacy cost would be 0. This can be visualized in Step 3 on Fig. 1. This initialization step sets the foundation for the subsequent task-specific knowledge distillation process. By initializing the student model with knowledge distilled from the teacher model, the student model inherits valuable insights and patterns, providing a solid starting point for further refinement and adaptation to the target task. Prior research has primarily emphasized initializing the student model with the weights of the teacher model to enhance accuracy [18]. However, our unique approach to initialization yields superior performance.

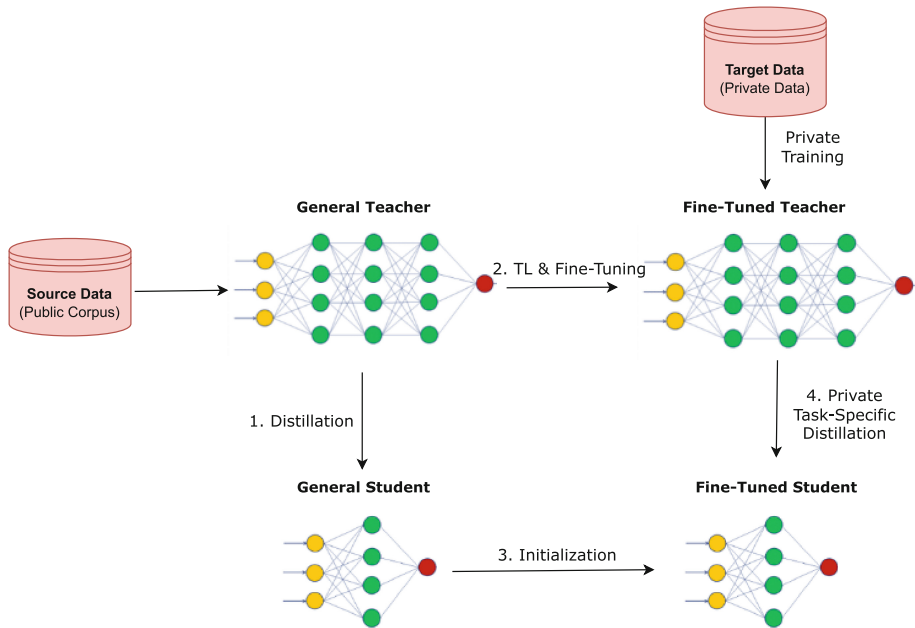


Fig. 1. Overview of the Task-Specific Knowledge Distillation with Differential Privacy framework. Each step of the framework is elaborated in corresponding sections of our methodology (e.g., Step 1 in Sect. 3.1)

3.4 Private Task-Specific Knowledge Distillation

The final step involves task-specific knowledge distillation, aimed at transferring the distilled knowledge from the fine-tuned teacher model to the initialized student model, as seen in Step 4 of Fig. 1. Leveraging DPSGD for privacy-preserving training, the student model is trained on task-specific data to distill task-specific knowledge. This process entails computing gradients, clipping them to mitigate large updates, adding noise to ensure privacy preservation, and updating the student model parameters iteratively. This incurs a privacy budget of (ϵ_2, δ_2) . By distilling task-specific knowledge from the fine-tuned teacher model under privacy constraints, the student model learns to effectively perform the target task while upholding stringent privacy preservation protocols.

4 Experimental Setup

In this section, we outline the experimental setup that we used to perform experiments with our approach.

4.1 Source and Target Data

Firstly, we describe the datasets and the task that we used for our experimentation. GLUE benchmark [26] is an evaluation benchmark designed to measure the performance of natural language understanding models in NLP. We have used it because [16, 18] have also utilized this benchmark for their investigations. We performed experiments with the following tasks: SST-2 (Single-Sentence text classification task), RTE (Recognizing textual entailment), QNLI (Question-answering NLI) and COLA (Corpus of Linguistic Acceptability). Our selection spans a diverse array of tasks, varying in the number of training examples. Notably, QNLI boasts the largest training set with 104K examples, whereas RTE contains 2K examples, representing the smallest training dataset in our experimentation.

We deployed transfer learning mechanism [21] in our approach where a model trained on one task is used as a starting point for a model on a new task. This could be done by transfer of knowledge in terms of information, patterns and representations that the initial model has learned about the features of the data to the second model. TL could be helpful in both same domain where the source and target domains are closely related as well as cross-domain transfers where there is limited available knowledge about the target domain. In our case, since we had enough knowledge in terms of available dataset for target tasks that we considered in the experimentation, so we opted for same domain transfer. This is also known as Inductive transfer learning where source and target domains are the same, even though the tasks performed by the model are different.

So, initially we take the pre-trained models that are trained on a task similar to the downstream target task. For instance, for QNLI task which is one of the large question-answer data, we took source task as SST-2 which is a sentence classification task. For SST-2 task, we took MRPC as a pre-training

task, which is a Microsoft research paraphrase corpus. For RTE task, we took WNLI-a reading comprehension task as a source task. Selecting the appropriate source task for pre-training is crucial for achieving optimal performance in a particular target task. When the source and target tasks are similar, the features learned during pre-training are more likely to be relevant. Pre-training on a dataset from the same or a closely related domain helps the model to learn domain-specific nuances, which can significantly improve the target task performance. Using these large-scale, diverse datasets for pre-training ensures that the model learns rich representations of language that can be effectively transferred to downstream tasks.

4.2 Baselines

In our research, we conducted a comparative analysis of our proposed approach against several baseline methods. The teacher model utilized in our experiments is a pre-trained BERT-base model [8], comprising 12 transformer layers. We performed fine-tuning of the teacher model on various downstream tasks, as well as private fine-tuning using DPSGD.

For the student model, we explored two distinct architectures: BERT-tiny [25], featuring 6 transformer layers, 128 hidden units, and 2 attention heads, and DistilBERT [23], a model distilled from BERT-base. We conducted fine-tuning of BERT-tiny and DistilBERT with the integration of differential privacy techniques [23], as well as without private fine-tuning to provide a fair comparison of our approach with non-private baseline models.

4.3 Privacy Budget and Hyper Parameters

To compare our work with an existing work [18], we adopt the same privacy budget as they used i.e., $\epsilon = 1$, and $\delta = \frac{1}{N}$, where N is number of samples in the dataset. We allocated a privacy budget of (ϵ_1, δ_1) for fine-tuning the teacher model, followed by (ϵ_2, δ_2) for task-specific knowledge distillation. The overall privacy budget utilized was $(\epsilon, \delta) = (1, \frac{1}{N})$. We experimented with learning rate of 10^{-5} , batch size = 64, maximum epochs = 10. The rate of compression in knowledge distillation is controlled by two parameters: α (weighting factor) and *temperature* (softmax temp). α provides a balance between student's own loss and distillation loss (KL loss). Higher alphas put more emphasis on mimicking teacher's prediction, whereas lower alphas give more weight to student's own prediction. In our experiments, we used $\alpha = 0.5$. Parameter *temperature* controls softening the teacher's probability before it is used to train the student. Higher value leads to softer probability distribution making training less reliant on hard targets, whereas lower values leads to sharper probability. We used temperature value of 5 in our experiments. The experimentation were performed on Google Colab with Intel Xeon CPU and 13GB of RAM. The GPU used was NVIDIA Tesla K80 with 12 GB of VRAM.

4.4 Privacy Analysis

We now conduct a privacy assessment of our framework.

Theorem 1. *The student model parameters $\theta_{student}$ obtained from Algorithm 1 are (ϵ, δ) -DP with $\epsilon = \epsilon_1 + \epsilon_2$ and $\delta = \delta_1 + \delta_2$.*

Proof. Our aim is to ensure that the Algorithm 1 guarantees (ϵ, δ) -DP for the sample data **D**. In the first step, the teacher model is a pre-trained model from a public dataset. So, it doesn't incur any privacy loss. In the next step, the teacher model is privately fine-tuned. It upholds a privacy budget of (ϵ_1, δ_1) -DP. In step 3, student model is initialized from open domain models, so it doesn't introduce additional privacy loss. Nevertheless, our model is still privacy preserving, capitalizing on the post-processing characteristic of DP. Finally, in the last step task-specific distillation is performed with the privacy budget of (ϵ_2, δ_2) -DP. Thus, the resultant student model parameters $\theta_{student}$ distilled from teacher model are privacy preserving with a privacy budget of (ϵ, δ) obtained via a composition of (ϵ_1, δ_1) and (ϵ_2, δ_2) .

This completes the proof. $\square$

5 Results and Discussion

In this section we present the results that we have obtained and provide some discussion about them.

5.1 A Comparative Analysis with Differentially Private Fine-Tuned Models

We compared our proposed approach Task-Specific Knowledge Distillation with Differential Privacy with open source models. In this analysis, we can evaluate the effectiveness of our algorithm by comparing the accuracy achieved through compressing the model using our proposed approach for downstream tasks against the accuracy obtained by fine-tuning a large teacher model and a student model on the same downstream task via DP. For this, we selected the BERT-base model and fine-tuned it for downstream tasks using DPSGD with a privacy parameter, ϵ , set to 1. We chose this privacy parameter value to enable easy comparison with existing studies. A smaller privacy parameter value offers stronger privacy guarantees but results in more loss of utility. Using the BERT-base model, we achieved an accuracy of 92.2% on the SST-2 dataset. In comparison, with our proposed approach, we attained an accuracy of 86.23%. These findings are presented in Table 2 of the paper. Additionally, we employed the BERT-tiny model (also serving as our student model) and fine-tuned it for downstream tasks using DPSGD with the same privacy budget. This resulted in an accuracy of 78.89% on the SST-2 dataset. From this analysis, we deduce that our approach reduces the number of model parameters from 109 million (in BERT-base) to 5 million (in our approach) while sacrificing only 6% model accuracy.

Table 2. Comparison of our approach with 12-layer BERT (BERT-base) and 6-layer BERT (BERT-tiny) which are fine-tuned using DPSGD with privacy budget $\epsilon = 1$

Model	Training	Teacher	Num-Params	SST-2	RTE	QNLI	COLA	Avg
BERT-base	DP-Finetune	-	109M	92.2	59.56	87.8	81.30	80.21
BERT-tiny	DP-Finetune	-	5M	78.89	55.23	81.42	69.12	71.16
BERT-tiny	Our Approach	BERT-base	5M	86.23	59.92	83.17	72.38	75.42

5.2 A Comparative Analysis with Fine-Tuned Models in a Privacy-Agnostic Context

We also conducted a comparison of our proposed approach with non-private models. Typically, privacy entails a trade-off with utility. In certain applications where model utility is paramount, prioritizing individual privacy may not be desirable. In this analysis, we utilized the same models, namely BERT-base, and fine-tuned them for downstream tasks without any privacy considerations. This resulted in an accuracy of 92.31% on the SST-2 dataset, which is slightly better than the accuracy achieved with the private fine-tuned model. Conversely, our proposed approach yielded an accuracy of 86.23%. By compromising only 6% of model accuracy, one can attain a differentially private model. These findings are summarized in Table 3. These results are consistent with existing research that have shown that large pre-trained models fine-tunes via DPSGD are as good as non-private models [17, 19].

Table 3. Comparison of our approach with 12-layer BERT (BERT-base) and 6-layer BERT (BERT-tiny) which are fine-tuned without preserving privacy

Model	Training	Teacher	Num-Params	SST-2	RTE	QNLI	COLA	Avg
BERT-base	Finetune	-	109M	92.31	61.01	87.9	80.72	80.48
BERT-tiny	Finetune	-	5M	79.93	58.48	81.96	69.12	72.37
BERT-tiny	Our Approach	BERT-base	5M	86.23	59.92	83.17	72.38	75.42

5.3 Initialization of Student Models with Pre-distilled Models

We also investigated whether better initialization of student models leads to the improved performance. In this experiment as shown in Table 4, we initialized student models with a pre-distilled DistilBERT model. When we employed this initialized student model in our approach, we observed an increase in model accuracy. Specifically, we achieved an accuracy of 89.90%. Next, we directly fine-tuned the DistilBERT model without utilizing our approach, first without using DPSGD and then with DPSGD. The resulting accuracies were 90.90% and

90.71%, respectively. In this analysis, our approach of Task-specific Knowledge Distillation with Differential Privacy demonstrates comparable performance to non-private learning and is also comparable to baseline models such as BERT-base.

Table 4. Comparison of our approach with 6-layer DistilBERT which are fine-tuned with and without using with privacy budget $\epsilon = 1$

Model	Training	Teacher	Num-Params	SST-2	RTE	QNLI	COLA	Avg
DistilBERT	Finetune	-	66M	90.90	59.12	87.33	81.49	79.71
DistilBERT	DP-Finetune	-	66M	90.71	58.12	88.46	77.08	78.59
DistilBERT	Our Approach	BERT-base	66M	89.90	60.35	84.29	68.16	75.67

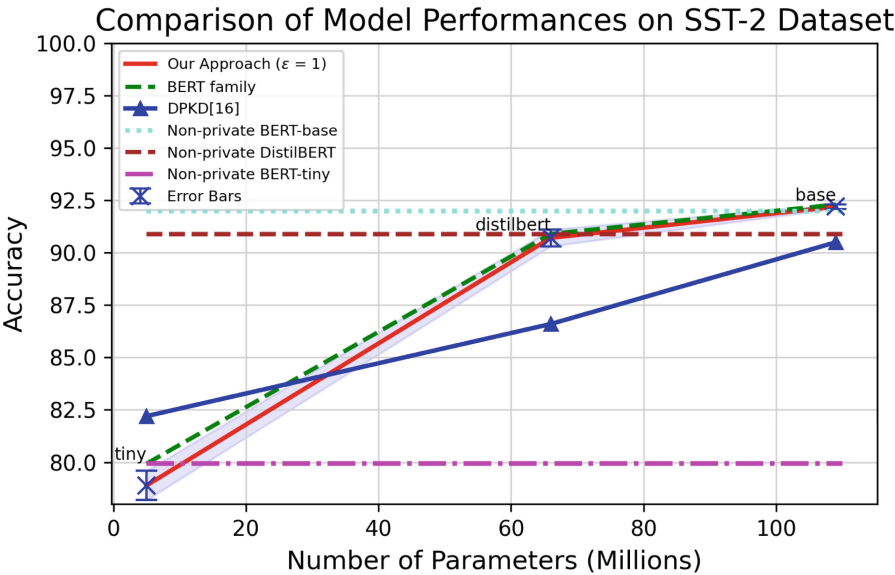


Fig. 2. Comparison of Model Performance on SST-2 Dataset: We plot the accuracy of BERT-base, DPKD, and our approach (epsilon = 1) against the number of parameters.

The Fig. 2 illustrates the performance of various models on the SST-2 dataset concerning accuracy versus the number of parameters. Three primary models are compared: BERT-base, DPKD [18], and an approach labeled “Our Approach” with epsilon equal to 1. Each model’s accuracy is plotted against the number of parameters, showcasing their relative performance. Error bars represent the variability in accuracy for the “Our Approach” model. Additionally, three specific points on the BERT-base curve are labeled as “bert-tiny”, “distilbert”, and “base”, indicating performance benchmarks.

6 Conclusion and Future Works

In our paper, we introduce a new method to speed up transformer-based models and protect data privacy. Our approach, called Task-Specific Knowledge Distillation with Differential Privacy, starts with a pre-trained model from a source task and uses transfer learning. This pre-training step is crucial for improving performance on downstream tasks. We focus on task-specific distillation, which further boosts performance compared to traditional methods. Additionally, we initialize student models from pre-distilled ones. Our framework achieves similar performance to both non-private and private baselines while reducing model parameters by 95%, making it suitable for real-time applications.

In the future, we aim to explore additional model compression techniques to further decrease inference time while maintaining or even enhancing performance compared to larger models.

Acknowledgement. This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation.

References

1. Abadi, M., et al.: Deep learning with differential privacy. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp. 308–318 (2016)
2. Bondarenko, Y., Nagel, M., Blankevoort, T.: Understanding and overcoming the challenges of efficient transformer quantization. arXiv preprint [arXiv:2109.12948](https://arxiv.org/abs/2109.12948) (2021)
3. Brown, T., et al.: Language models are few-shot learners. In: Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901 (2020)
4. Buciluă, C., Caruana, R., Niculescu-Mizil, A.: Model compression. In: Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 535–541 (2006)
5. Carlini, N., Liu, C., Erlingsson, Ú., Kos, J., Song, D.: The secret sharer: evaluating and testing unintended memorization in neural networks. In: 28th USENIX Security Symposium (USENIX Security 19), pp. 267–284 (2019)
6. Carlini, N., et al.: Extracting training data from large language models. In: 30th USENIX Security Symposium (USENIX Security 21), pp. 2633–2650 (2021)
7. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: BERT: pre-training of deep bidirectional transformers for language understanding. arXiv preprint [arXiv:1810.04805](https://arxiv.org/abs/1810.04805) (2018)
8. Devlin, J., Chang, M., Lee, K., Toutanova, K.: BERT: pre-training of deep bidirectional transformers for language understanding. CoRR abs/1810.04805 (2018). <http://arxiv.org/abs/1810.04805>
9. Dwork, C.: Differential privacy. In: Bugliesi, M., Preneel, B., Sassone, V., Wegener, I. (eds.) ICALP 2006. LNCS, vol. 4052, pp. 1–12. Springer, Heidelberg (2006). https://doi.org/10.1007/11787006_1
10. Flemings, J., Annavaram, M.: Differentially private knowledge distillation via synthetic text generation. arXiv preprint [arXiv:2403.00932](https://arxiv.org/abs/2403.00932) (2024)

11. Ganesh, A., et al.: Why is public pretraining necessary for private model training? In: International Conference on Machine Learning pp. 10611–10627. PMLR (2023)
12. Gordon, M.A., Duh, K., Andrews, N.: Compressing BERT: studying the effects of weight pruning on transfer learning. arXiv preprint [arXiv:2002.08307](#) (2020)
13. Han, S., Pool, J., Tran, J., Dally, W.: Learning both weights and connections for efficient neural network. In: Advances in Neural Information Processing Systems, vol. 28 (2015)
14. Hinton, G., Vinyals, O., Dean, J.: Distilling the knowledge in a neural network. arXiv preprint [arXiv:1503.02531](#) (2015)
15. Kim, Y.J., Awadalla, H.H.: FastFormers: highly efficient transformer models for natural language understanding. arXiv preprint [arXiv:2010.13382](#) (2020)
16. Li, X., Tramer, F., Liang, P., Hashimoto, T.: Large language models can be strong differentially private learners. arXiv preprint [arXiv:2110.05679](#) (2021)
17. Mehta, H., Thakurta, A., Kurakin, A., Cutkosky, A.: Large scale transfer learning for differentially private image classification. arXiv preprint [arXiv:2205.02973](#) (2022)
18. Mireshghallah, F., Backurs, A., Inan, H.A., Wutschitz, L., Kulkarni, J.: Differentially private model compression. In: Advances in Neural Information Processing Systems, vol. 35, pp. 29468–29483 (2022)
19. Mireshghallah, F., Taram, M., Vepakomma, P., Singh, A., Raskar, R., Esmaeilzadeh, H.: Privacy in deep learning: a survey. arXiv preprint [arXiv:2004.12254](#) (2020)
20. Najafabadi, M.M., Villanustre, F., Khoshgoftaar, T.M., Seliya, N., Wald, R., Muharemagic, E.: Deep learning applications and challenges in big data analytics. *J. Big Data* **2**, 1–21 (2015)
21. Pan, S.J., Yang, Q.: A survey on transfer learning. *IEEE Trans. Knowl. Data Eng.* **22**(10), 1345–1359 (2009)
22. Papernot, N., Song, S., Mironov, I., Raghunathan, A., Talwar, K., Erlingsson, Ú.: Scalable private learning with pate. arXiv preprint [arXiv:1802.08908](#) (2018)
23. Sanh, V., Debut, L., Chaumond, J., Wolf, T.: DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv preprint [arXiv:1910.01108](#) (2019)
24. Tang, R., Lu, Y., Liu, L., Mou, L., Vechtomova, O., Lin, J.: Distilling task-specific knowledge from BERT into simple neural networks. arXiv preprint [arXiv:1903.12136](#) (2019)
25. Turc, I., Chang, M.W., Lee, K., Toutanova, K.: Well-read students learn better: on the importance of pre-training compact models. arXiv preprint [arXiv:1908.08962v2](#) (2019)
26. Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., Bowman, S.R.: GLUE: a multi-task benchmark and analysis platform for natural language understanding. arXiv preprint [arXiv:1804.07461](#) (2018)
27. Wang, J., Bao, W., Sun, L., Zhu, X., Cao, B., Philip, S.Y.: Private model compression via knowledge distillation. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 33, pp. 1190–1197 (2019)
28. Xu, Z., et al.: Federated learning of Gboard language models with differential privacy. arXiv preprint [arXiv:2305.18465](#) (2023)
29. Yu, D., et al.: Selective pre-training for private fine-tuning. arXiv preprint [arXiv:2305.13865](#) (2023)
30. Zafir, O., Boudoukh, G., Izsak, P., Wasserblat, M.: Q8BERT: quantized 8bit BERT. In: 2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing-NeurIPS Edition (EMC2-NIPS), pp. 36–39. IEEE (2019)



Exploiting Internal Randomness for Privacy in Vertical Federated Learning

Yulian Sun^{1,3} , Li Duan^{1,2} , Ricardo Mendes¹ , Derui Zhu^{1,4} ,
Yue Xia^{1,4} , Yong Li¹ , and Asja Fischer³

¹ Huawei Technologies Düsseldorf, Düsseldorf, Germany
{yulian.sun1,li.duan,ricardo.mendes1,derui.zhu,yue.xia1,
yong.li1}@huawei.com

² Paderborn University, Paderborn, Germany
liduan@mail.upb.de

³ Ruhr University Bochum, Bochum, Germany
yulian.sun@edu.ruhr-uni-bochum.de, asja.fischer@rub.de

⁴ Technical University Munich, Munich, Germany
{derui.zhu,yue1.xia}@tum.de

Abstract. Vertical Federated Learning (VFL) is becoming a standard collaborative learning paradigm with various practical applications. Randomness is essential to enhancing privacy in VFL, but introducing too much external randomness often leads to an intolerable performance loss. Instead, as it was demonstrated for other federated learning settings, leveraging internal randomness – as provided by variational autoencoders (VAEs) – can be beneficial. However, the resulting privacy has never been quantified so far, nor has the approach been investigated for VFL.

We therefore propose a novel differential privacy (DP) estimate, denoted as distance-based empirical local differential privacy (dELDP). It allows us to empirically bound DP parameters of models or model components, quantifying the internal randomness with appropriate distance and sensitivity metrics. We apply dELDP to investigate the DP of VAEs and observe values up to $\epsilon \approx 6.4$ and $\delta = 2^{-32}$. Based on this, to link the dELDP parameters to the privacy of VAE-including VFL systems in practice, we conduct comprehensive experiments on the robustness against state-of-the-art privacy attacks. The results illustrate that the VAE system is robust against feature reconstruction attacks and outperforms other privacy-enhancing methods for VFL, especially when the adversary holds 75% of the features during label inference attacks.

Keywords: privacy · vertical federated learning · distance-based empirical differential privacy · variational autoencoder

1 Introduction

Vertical Federated Learning (VFL) is a distributed learning paradigm that allows different participants to collaboratively train a joint model without sharing their

Y. Sun and L. Duan—These authors contributed equally to this work.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024
J. Garcia-Alfaro et al. (Eds.): ESORICS 2024, LNCS 14983, pp. 390–409, 2024.

https://doi.org/10.1007/978-3-031-70890-9_20

raw data. In VFL, *different* features of each sample are held by different participants. A typical VFL model consists of multiple *bottom models* and one *top model* [38]. In principle, the participants feed their own features to the bottom models, whereas those with labels hold the top model(s) and coordinate the updates. VFL has been widely deployed in commercial services spanning various industrial sectors, and it is seen as prominent by leading AI players, including Google, Amazon, and Huawei, as VFL promotes data-based collaborations [18]. An overview of the VFL workflow is shown in Fig. 1.

Despite the remarkable success of VFL in real-world applications, there are increasing concerns about the vulnerabilities, especially input and model leakage of the basic VFL architecture. Besides attacking the VFL training [25, 31], an adversary can also extract information during the inference phase if a trained VFL model is released as a public service, threatening the privacy of input features of other participants and even the entire VFL model [25, 29]. For instance, Luo *et al.* [26] show how to utilize the model output to approximate the input features. Moreover, Geng *et al.* [15] illustrate how the released model outputs can help an adversary reconstruct the entire model by employing transfer learning.

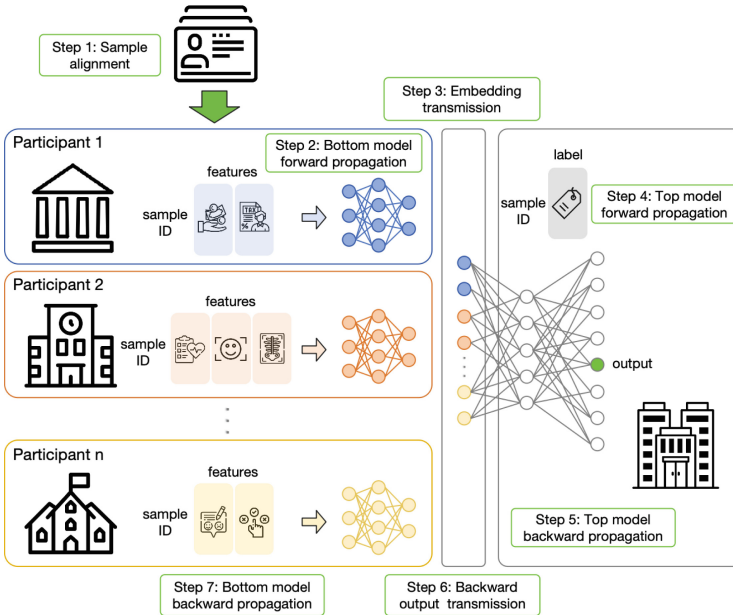


Fig. 1. VFL, the general workflow

1.1 Randomness for Privacy in VFL

Technically, enhancing privacy often relates to processing the input data or intermediate states in VFL with randomness. Sources of randomness can be (1) exter-

nal, e.g., resulting from adding independent random noise, or (2) internal, resulting from random variables that depend on the structure of the network and the input data. Classical differential privacy (DP) uses external randomness. Quantifying privacy from external randomness is rather straightforward. For example, in the case of Laplace and Gaussian mechanisms in DP, predefined diversity b or variance σ can determine the privacy parameters (ϵ, δ) . However, major downsides of external randomness also exist: it hinders performance or utility [16], and the users may be concerned with the trustworthiness of centralized noise adding mechanisms. As a countermeasure, Geng *et al.* [15] suggested to use a trusted third party (TTP) to introduce noise into the output throughout training and inference. However, the existence of a TTP is usually considered an unjustified strong assumption in real-world scenarios.

In contrast to external noise, taking advantage of structural and data dependent randomness can lead to more compact and efficient solutions. This is because the structures such as variational autoencoder [19], global attention [9], and U-noise [21] are already parts of the machine learning model, i.e., the randomness is *internal*. Solutions such as PRECODE [31] and GATN [9] have demonstrated the merit of using internal randomness for privacy. On the other hand, quantifying the privacy resulting from this internal randomness is far more challenging than in the external case. First, the structural randomness may not have an analytic form [21]. If the query structure spans several linear and non-linear layers, directly deriving a generic analytic form for the random distribution is intractable [19]. Second, the extracted privacy parameters may not be comparable or compatible with other mechanisms. For example, as pointed out by Burchard *et al.* [7], Duan’s noise-less privacy [11] cannot be combined with DP directly.

1.2 Contributions and Paper Outline

This paper proposes a novel way to estimate differential privacy, that can be applied to model internal noise, and employs it to quantify the privacy in VAE based VFL systems. More specifically, we make the following contributions.

1. We propose a new differential privacy estimate called distance-based Empirical Local Differential Privacy (dELDP) that forms a good foundation for *empirical* estimation of internal randomness in VFL. When used with appropriate distance metric, sensitivity and estimators, the dELDP privacy parameters can be efficiently estimated for concrete structural parameters, trained model, and data sets.
2. We demonstrate the applicability of dELDP by estimating the (ϵ, δ) values of trained VAEs. Randomly initialized VAEs are integrated into an existing VFL architecture, and the whole model is then trained with the original optimization goal and methods. We denote the VFL pipeline extended with VAE as SAIR, as it SAVes the cost with Internal Randomness. Besides quantifiable privacy guarantee in the sense of dELDP, we also show that SAIR can enjoy minimal performance loss and be empirically composed with other privacy-enhancing technologies for improved resilience against attacks.

3. To link dELDP parameters to privacy in practice, we also conduct investigate the robustness of VAE equipped VFL systems against state-of-the-art privacy attacks during VFL inference. Besides showing high robustness against feature reconstruction attacks, the results also illustrate that SAIR outperforms other privacy-preserving methods in VFL, especially when the adversary holds more than 50% of features in label inference attacks and model stealing attacks.

Outline. After the introduction of background knowledge and a survey of related work in Sect. 2, Sect. 3 elaborates dELDP definitions and its application. Section 4 contains detailed implementation, experiment description and highlights of empirical results for privacy and performance.

2 Background and Related Work

Distance-Based Local Differential Privacy. The main idea of Local differential privacy (LDP) is to perturb individual data samples with noises, and the usage of LDP is frequently seen in FL. Truex *et al.* [35] present LDP-Fed, a federated learning system with a quantifiable privacy guarantee using LDP. Erlingsson *et al.* [13] described an algorithm whose privacy cost is polylogarithmic in the number of user value changes. The major drawback of LDP is its negative impact on accuracy, but efforts are being made to reduce it. In the VFL setting, besides conventional DP against honest adversaries [30], Li *et al.* [24] propose a federated tree boosting framework based on order-preserving desensitization with distance-based LDP (dLDP). The distance metric introduced by dLDP reduces the amount of noise needed, as it provides a reasonable way to ignore the extreme cases where two samples are too far away, i.e., too easy to be distinguished by an adversary. However, existing dLDP mechanisms still rely on external randomness source and complete analytic forms of queries.

Empirical Differential Privacy. In 2009, Duan [11] introduced the notion of privacy without noise, which estimates the corresponding (ϵ, δ) from the inherent randomness in the dataset and the query. Although this approach needs strong assumptions about data distribution, i.e., independent identically distributed data, it highlights that non-uniform internal noise can also achieve reasonable DP. Moreover, Duan’s research also inspired Burchard *et al.* to formalize empirical differential privacy (EDP) in 2019 [7]. EDP introduces a method to empirically estimate the privacy of a given query $f()$ applied to a random database S , which iterates over all adjacent data bases S_i for each sample x_i to estimate the centralized privacy risk. However, a large gap still exists between EDP and the evaluation of privacy in VFL, especially the privacy provided by each individual bottom model. First, formed as centralized DP, the estimation in EDP needs complete knowledge about every feature of each sample, contradicting with the settings of VFL where each bottom model owner knows only its own features.

Second, a large number of the numerical integration of the estimated probability density function are necessary, leading to high computational cost and accumulated error even if the data set is of moderate size.

Other Existing Defenses. Besides homomorphic encryption (HE) and multi-party computation [20], differential Privacy (DP) [12] has also been applied to the training process: Differentially Private-Stochastic Gradient Descent (**DP-SGD**) [1]. DP-SGD has been widely used as an effective and provable privacy protection mechanism to protect training data. However, and similarly to LDP, it can have a significant impact on utility. In the context of VFL, simpler mechanisms have also been proposed that can protect privacy and improve communication costs. Particularly, Noisy Gradient (**NG**) [40] adds Gaussian noise to the backward propagation, Gradient Compression (**GC**) [18] retains only the highest gradients, and Discrete SGD (**DSGD**) [4] discretizes gradients into a small number of bins. We mainly use these non-cryptographic defenses as comparison in the experiments.

Although VAE and its variants are widely used and studied in machine learning [2, 19, 27, 36], to the best of our knowledge Scheliga *et al.* [31] first proposed the use of VAE for privacy in training in 2022 as PRECODE, an VAE-based privacy-enhancing module for existing models. Although PRECODE’s privacy protection against gradient inversion attack is empirically illustrated in training, Scheliga *et al.* did not give any formal interpretation of the privacy guarantee or conduct any attack experiments for the inference phase.

3 Distance-Based Empirical Local Differential Privacy

3.1 Formal Definition of dELDP

We consider a threat model for the inference, where a malicious adversary $\mathcal{A}$ controls some participants and holds partial datasets. Moreover, $\mathcal{A}$ can interact with the trained model via normal interfaces. Due to the sample alignment in VFL¹, it is trivial to see whether an individual is in the training dataset or not. Thus, in the sense of centralized DP [12] or EDP [7], VFL training cannot have any meaningful privacy. Moreover, an adversary $\mathcal{A}$ trying to steal information from a trained VFL model M^* has two extra advantages: (1) $\mathcal{A}$ can get the label produced by M^* if $\mathcal{A}$ participates in the inference; (2) $\mathcal{A}$ can have partial information of the sample x . Due to (1) and (2), even if we apply LDP in VFL and $\mathcal{A}$ ’s own features are identical for each sample, $\mathcal{A}$ can trivially distinguish two samples with *different* labels with a probability determined by the accuracy of M^* . Therefore, besides the range of the output, we have to consider the partial information held by $\mathcal{A}$ (leakage, especially labels), which can be formulated as the distance between two samples. We start by recalling the distance-based LDP definition and discussing its limitations.

¹ In Step 1 in Fig. 1, features of the same individual have to be aligned by the identifier. If $\mathcal{A}$ participates in training, $\mathcal{A}$ sees the (quasi-)identifier of every sample.

Definition 1 (Distance-Based LDP, dLDP). We say that a dataset S has (ϵ, δ) -distance-based local differential privacy against an adversary $\mathcal{A}$ with regard to statistical function $f : S \rightarrow \mathcal{R}$, distance metric $\text{dist}()$, and distance threshold t , if $\forall x, x' \in S, \forall y \in \mathcal{R}$:

$$\text{dist}(x, x') \leq t \Rightarrow \Pr[f(x) = y] \leq e^{t\epsilon} \Pr[f(x') = y] + \delta . \quad (1)$$

Given an appropriate $\text{dist}()$, if we have an input-independent $f()$ with an efficiently computable analytic form, we can derive the dLDP parameters directly.

However, in VFL, the complexities inside the parameterized function $f_\theta()$ hinders the use of dLDP for quantifying the corresponding internal randomness. The value $f_\theta(x)$ depends on both θ and x , and the trained model parameters θ depend on x and other samples (features) in the training data set. As mentioned before, when $f_\theta()$ spans several linear and non-linear layers, deriving the analytic form of $f_\theta()$ or the probability function of it is intractable [19]. Moreover, samples are split into features and the bottom models are trained for different features. Therefore, not only the value, the function $f_\theta()$ itself may also vary for different x , adding another layer of complexity.

Thus, we opt for an *empirical* approach to circumvent the theoretical obstacles. Intuitively, we can turn it into an estimate of the privacy parameters, as formalized by the following definition.

Definition 2 (Distance-Based Empirical LDP, dELDP). We say that a dataset S has (ϵ, δ) - distance-based *empirical* local differential privacy against an adversary $\mathcal{A}$ with regard to statistical function $f : S \rightarrow \mathcal{R}$, distance metric $\text{dist}()$, distance threshold t , and *estimator* $\text{esmt}()$, if

$$\begin{aligned} \forall x, x' \in S, \forall y \in \mathcal{R} : \text{dist}(x, x') \leq t \Rightarrow \\ \text{esmt}(\Pr[f(x) = y]) \leq e^{t\epsilon} \cdot \text{esmt}(\Pr[f(x') = y]) + \delta . \end{aligned} \quad (2)$$

Note that if the same $\text{dist}()$ and threshold t are used, and if the distribution can be estimated exactly, i.e., $\forall x : \text{esmt}(\Pr[f(x) = y]) = \Pr[f(x) = y]$, then (ϵ, δ) -dELDP implies (ϵ, δ) -dLDP.

In the application of dELDP in practice, besides well-defined $\text{dist}()$ and threshold, the estimation algorithm provides another degree of freedom and it can be adapted to individual VFL task for improved efficiency. Next we elaborate how dELDP can be applied to Gaussian-type internal randomness.

3.2 Application: dELDP for Trained VAEs in VFL

The posterior sampling process inside VAE has a Gaussian form, so we can estimate the *sensitivity* of the function and the *variance* instead. To see how this can proceed, first recall the Gaussian Mechanism in LDP and the posterior sampling in VAE.

Definition 3 (Gaussian Mechanism for LDP [32]). If a function $f(x)$ of an input $x \in S$ is to be released, the Gaussian release mechanism is defined as

$$G(x) := f(x) + \mathcal{N}(0, \sigma^2 I) = f(x) + \sigma \mathcal{N}(0, I) . \quad (3)$$

Theorem 1 ((ϵ, δ) of Classical Gaussian Mechanism [12,32]). *If the sensitivity of the function is bounded by Δ_f , i.e., $\forall x, x' \in \mathcal{S}$, $\|f(x) - f(x')\|_2 \leq \Delta_f$, then for any $\delta \in [0, 1]$, the Gaussian mechanism $G()$ satisfies (ϵ, δ) -LDP, where*

$$\epsilon = \frac{\Delta_f}{\sigma} \sqrt{2 \log \frac{1.25}{\delta}} . \quad (4)$$

Definition 4 (Posterior Sampling in VAE[19]). *Let $x \in \mathcal{S}$ be the input to VAE. The posterior sampling of $z(x)$ is defined as*

$$z(x) := \mu(x) + \Sigma(x)^{\frac{1}{2}} \mathcal{N}(0, I) , \quad (5)$$

where $\mu(x)$ is the mean function and $\Sigma(x)$ the covariance matrix dependent on x learned by the VAE.

The key difference between (5) and the original Gaussian mechanism (3) is that the perturbation in (5) is a function of the input value x . Following the approach taken by [11] for non-uniform perturbation, we can generalize the Gaussian mechanism as follows.

Definition 5 (Extended Gaussian Mechanism, EG). *Let $\mathbb{R}$ be the set of real numbers. For $f : \mathcal{S} \rightarrow \mathcal{R} \subset \mathbb{R}^m$, if a function $f(x)$ of an input $x \in \mathcal{S}$ is to be released, the extended Gaussian release mechanism is defined as*

$$EG(x) := f(x) + M(x) \mathcal{N}(0, I) , \quad (6)$$

where $M : \mathcal{S} \rightarrow \mathbb{R}^{m \times m}$ maps x to a diagonal matrix with non-negative entries.

By taking $f() := \mu()$, VAE posterior sampling is an instance of EG. As $\Sigma(x)^{\frac{1}{2}}$ is a diagonal matrix in VAE [19], we can collect the diagonal elements into a vector $V(x)$. The $EG(x)$ implemented by VAE can then be re-written as

$$EG(x) = f(x) + V(x) \odot \mathcal{N}(0, I) . \quad (7)$$

We keep this $V(x)$ notation and omit the element-wise product symbol $\odot$ henceforth. What remains to be quantified for dLDP for VAE is an appropriate distance function and sensitivity. As we consider samples with the same label, we define

$$\text{dist}(x, x') = \begin{cases} 1, \text{Label}(x) = \text{Label}(x') \\ \infty, \text{otherwise} \end{cases} . \quad (8)$$

Definition 6 (Local Sensitivity, Distance-Based). *For $f : \mathcal{S} \rightarrow \mathcal{R} \subset \mathbb{R}$ and $\forall x, x' \in \mathcal{S}$ with $\text{Label}(x) = \text{Label}(x') = \ell$, the local sensitivity $\text{Sens}_{f,\ell}$ of $f()$ with respect to ℓ is defined as*

$$\text{Sens}_{f,\ell} = \max_{x, x'} (\|f(x) - f(x')\|_2) . \quad (9)$$

Using (4), (7), (8) and (9), we arrive at the bound of dLDP parameters of VAE in Theorem 2. Let $\text{Label}(\mathcal{S})$ be the set of labels of samples in $\mathcal{S}$.

Theorem 2 (dLDP for VAE of Each Bottom Model). *Given dataset $\mathcal{S}$ and query $\mu : \mathcal{S} \rightarrow \mathcal{R} \subset \mathbb{R}^m$, if $\forall \ell \in \text{Label}(\mathcal{S}), x, x' \in \mathcal{S}, \text{Label}(x) = \text{Label}(x') = \ell$ $\exists \Delta_\ell, \sigma_\ell : \left(\text{Sens}_{\mu, \ell} \leq \Delta_\ell \wedge \min(\|V(x)\|_2, \|V(x')\|_2) \geq \sigma_\ell \right)$, then there exists Δ_μ and σ_μ for VAE posterior sampling for fixed (μ, V) that satisfy (ϵ, δ) -dLDP in the sense of Definition 1 for $t = 1$, $\delta \in [0, 1]$, and*

$$\epsilon = \frac{\Delta_\mu}{\sigma_\mu} \sqrt{2 \log \frac{1.25}{\delta}}. \quad (10)$$

Proof. (Sketch) Δ_ℓ is the upper bound of the sensitivity, and σ_ℓ is the lower bound of the (standard) variance. The intuition is that if for a label ℓ , the minimum noise is no less than $\mathcal{N}(0, \sigma_\ell^2 I)$, then the privacy guarantee is at least as good as provided by $\mathcal{N}(0, \sigma_\ell^2 I)$ with the corresponding $(\epsilon_\ell, \delta_\ell)$. And Δ_μ / σ_μ corresponds to the worst ϵ_ℓ and δ_ℓ , i.e., the maximum among all ℓ 's. The complete proof is in Appendix A.

The last gap between dLDP and dELDP is an efficient estimation. If the size of the dataset and the number of parameters of a VAE can be bounded by a polynomial of the input length, the sensitivity and the variance can be efficiently estimated. Thus, we have the following lemma for dELDP of VAE.

Lemma 1 (dELDP for VAE). *The posterior sampling in VAE has (ϵ, δ) -dELDP in the sense of Definition 2 with*

$$\epsilon = \text{esmt} \left(\frac{\Delta_\mu}{\sigma_\mu} \right) \sqrt{2 \log \frac{1.25}{\delta}}, \quad (11)$$

if $\text{esmt}()$ can be computed in polynomial time.

Algorithm 1 plays the central role in $\text{esmt}()$ which bounds the sensitivity and the variance in the trained VAE with respect to the given partial dataset. Let the number of labels be a constant, and the number of data items be n_x . The time complexity of Algorithm 1 is $\mathcal{O}(n_x^2)$. The computation includes only inner-product and comparison. Once $\{(\Delta_\ell, \sigma_\ell)\}$ is collected, the term $\text{esmt} \left(\frac{\Delta_\mu}{\sigma_\mu} \right)$ can be computed as $\max \left(\frac{\Delta_\ell}{\sigma_\ell} \right)$. The empirical results will be presented in Table 3 in Sect. 4.

Algorithm 1. dELDP for VAE

```

1: Input:  $P_i$ 's partial data  $S_i$  and  $\text{Label}(S)$ ;
2: Output:  $\{(\Delta_\ell/\sigma_\ell)\}$ , the maximum sensitivity over the minimum variance pairs of
   each label  $\ell \in \text{Label}(S)$  with respect to  $S_i$ .
3: Algorithm Starts:
4: for each label  $\ell \in \text{Label}(S)$  do
5:    $\Delta_\ell = 0, \sigma_\ell = 0$ 
6:   for each pair  $(x, x')$  in  $S_i$  do
7:     if  $\text{Label}(x) = \text{Label}(x') = \ell$  then
8:        $\Delta = \|\mu_x - \mu_{x'}\|_2, \sigma = \min(\|V(x)\|_2, \|V(x')\|_2);$ 
9:        $\Delta_\ell = \max(\Delta_\ell, \Delta), \sigma_\ell = \min(\sigma_\ell, \sigma);$ 
10:    end if
11:  end for
12: end for
13: Output:  $\{(\Delta_\ell/\sigma_\ell)\}$  for  $S_i$ 

```

4 Experiments

We first describe the setup of our experiments that were all run on a TeslaTMP100 GPU with 16 GB RAM. Next, we discuss the results.

4.1 Setup

Datasets. We use six real-world multiclass classification datasets for evaluation: Sensorless Drive Diagnosis (**SDD**) [3], **Criteo** [22], **MNIST** [8], **MedicalMNIST** [37], Look and Listen (**LL**, a multimodal dataset with image-audio modalities) [5], and **AG's News** Topic Classification Dataset [39]. The features are split into two groups and then fed into VFL systems. The feature distribution and statistics of the six datasets are shown in Table 1.

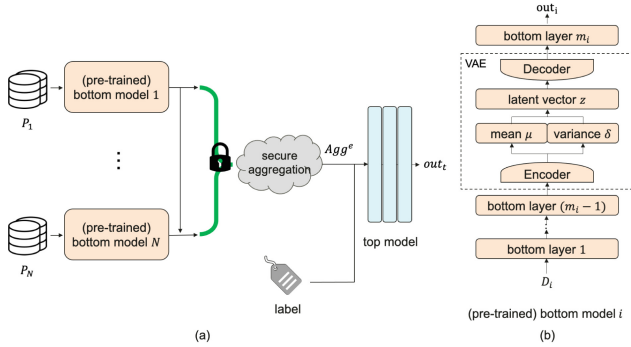
Baseline Models. All baseline models have no privacy-preserving techniques. We employed VFL systems with two bottom models and one top model. For **SDD**, **Criteo**, **MNIST**, and **MedicalMNIST**, each bottom model consists of two linear layers of neural network. The top model as well consists of two linear layers. For **LL**, one bottom model contains a VGG-16 [34] backbone for the image modality, the other contains 4 linear layers for the audio modality. The top model contains two linear layers. For **AG's News**, each bottom contains a BERT [10] backbone, and the top model contains one linear layer. Table 1 shows the parameters of the top model and bottom models.

SAIR Models. Rather than testing existing models that contain VAE-like structures, we add a VAE to our VFL baseline models that do not contain any other sources of randomness, since this allows for direct comparison and the effect of adding internal randomness can be observed without interference. Specifically, we augment each baseline model by adding a VAE with 3 layers (corresponding

Table 1. Bottom and Top Model Parameters. Different clients hold different ratios of features, for example, client 0 and client 1 have 24 features in **SDD**.

Dataset	#Samples	#Labels	#Params. Bottom Model Feature Distribution	#Params. Top Model
SDD	58.5K	11	[18.5K , 18.5K] (24, 24)	6K
Criteo	44841K	2	[17.5K , 18.1K] (13, 26)	5K
MNIST	70K	10	[72.4K , 72.4K] (14, 14)	10.8K
MedicalMNIST	59K	6	[315K , 315K] (16, 16)	10.8K
LL	17.2K	9	[294M , 733K] (image, audio)	1.7K
AG’s News	128K	4	[1.5M , 1.5M] (‘title’, ‘description’)	32.3K

to an encoder and a decoder with one linear hidden layer each) on top of each bottom model. We denote the VFL pipeline extended with a VAE as **SAIR**, as it **SA**ves the cost with **I**nternal **R**andomness. The algorithm used for training the models is described in Algorithm 2. The model inference is identical to the forward pass except that the loss function is not computed (Fig. 2).

**Fig. 2.** Overview of VAE-enhanced VFL: (a) data flow; (b) VAE in bottom model. Secure aggregation [6] against adversaries is used.

Performance Experiments. To evaluate the performance of different systems, we measure the total time needed until convergence. We compare **SAIR** with VAEs trained from scratch and **SAIR** with pre-trained models to the baseline models.

dELDP Experiments. We estimate the dELDP of the VAEs using Algorithm 1. The algorithm is executed over each label and each bottom model trained on **SDD**, **MedicalMNIST**, **LL**, and **AG’s News**. The primary results from the experiments are the maximum values $\frac{\Delta_\ell}{\sigma_\ell}$ for each label-model combination.

Algorithm 2. Training Process in SAIR

```

1: Input and Parameters:
2: Top model owner  $\mathcal{S}$  and  $N$  bottom model owners, a dataset  $\mathbf{S}$ , and labels  $\text{Label}(\mathbf{S})$ ;
3: Bottom model, owner  $P_i$ , has partial data  $\mathbf{S}_i, \forall i \in [N]$ ;
4: (Pre-trained) bottom model  $\mathbf{M}_B$  with VAE, top model  $\mathbf{M}_T$ ;
5: Top and bottom model parameters  $\theta_S, \{\theta_{C_i}, \forall i \in [N]\}$ , resp.;
6: Loss function  $\mathcal{L}$  of final outputs, loss functions  $\{\mathcal{L}_{(\mu_i, \sigma_i)}\}$  of bottom models, parameters  $\{\mu_i\}$  and  $\{\sigma_i\}$  of VAEs;
7: Learning rate  $\eta$ , total training epochs  $\mathcal{E}$ .
8: Algorithm Starts:
9: for  $e = 1, 2, \dots, \mathcal{E}$  do
10:   // Training for bottom model
11:   for each bottom model owner  $P_i$  do
12:     // 1. Forward propagation for bottom models
13:     // Train  $\mathbf{M}_B$  in parallel;
14:      $out_i^e = \mathbf{M}_B(\mathbf{S}_i)$ ;
15:     Compute  $\mathcal{L}_{(\mu_i, \sigma_i)}$ ;
16:     Send  $out_i^e$  to  $\mathcal{S}$ ; // All  $out_i^e$  have the same shape.
17:     // 2. Backpropagation and stochastic gradient descent for bottom models
18:     Compute  $\partial \mathcal{L}_{(\mu_i, \sigma_i)}$ , update  $\mu_i$  and  $\sigma_i$ ;
19:     // Until the server finishes backward propagation
20:     Receive  $\frac{\partial \mathcal{L}(out_T, \text{Label})}{\partial out_i^e}$ ;
21:     Compute  $\frac{\partial \mathcal{L}(out_T, \text{Label})}{\partial \theta_{C_i}} = \frac{\partial \mathcal{L}(out_T, \text{Label})}{\partial out_i^e} \times \frac{\partial out_i^e}{\partial \theta_{C_i}}$ ;
22:     Update  $\theta_{C_i} = \theta_{C_i} - \eta \frac{\partial \mathcal{L}(out_T, \text{Label})}{\partial \theta_{C_i}}$ ;
23:   end for
24:   // Training for top model
25:   // 1. Forward propagation for top model
26:    $out_T = \mathcal{M}_T(out_i^e)$ ;
27:   Compute  $\mathcal{L}(out_T, \text{Label}(\mathbf{S}))$ ;
28:   // 2. Backward propagation for top model;
29:    $\theta_S = \theta_S - \eta \frac{\partial \mathcal{L}(out_T, \text{Label}(\mathbf{S}))}{\partial \theta_S}$ ;
30:   Compute and send  $\frac{\partial \mathcal{L}(out_T, \text{Label})}{\partial out_i^e}$  to all bottom model owners;
31: end for

```

Attack Experiments. We evaluate the robustness of SAIR in comparison with that of the baseline models with respect to the following state-of-the-art attacks applicable during model inference. As mentioned in Sect. 3.1, the malicious adversary $\mathcal{A}$ tries to extract private information by accessing the trained model and the partial datasets during the inference. $\mathcal{A}$'s power may be strengthened or altered for each concrete attack.

- **Feature Reconstruction (FR) Attack.** In FR attacks, $\mathcal{A}$ has access to the trained model. We implemented an FR attack under the *white-box* setting [17], which provides a stronger adversary and an upper bound of privacy loss. In this attack, the adversary $\mathcal{A}$ uses backpropagation to update an input composed of $\mathcal{A}$'s own and random features, as to minimize the distance between

the true confidence scores and the output for this input. $\mathcal{A}$ collects the true confidence scores of the trained model during inference, meanwhile forming pairs of partial inputs and respective outputs.

- **Label Inference (LI) Attack.** As an example of an LI attack, we implement the Passive Label Inference Attack through Model Completion [14] without access to the complete trained model. In this LI attack, the adversary $\mathcal{A}$ tries to infer the labels of new samples by using its own bottom model M_B^A , and complementing that model with an *inference head*, which corresponds to randomly initialized additional layers that augment the local model. $\mathcal{A}$ trains this head by freezing M_B^A and using samples containing only $\mathcal{A}$'s training data (i.e., part of the features) and the respective labels.
- **Model Stealing (MS) Attack.** MS attackers train shadow models and need access to the trained model. In addition, the adversary $\mathcal{A}$ has some input data with all features, which is referred to as the *background information*. Specifically,
 - for stealing M^* trained with **MNIST**, we use the **MedicalMNIST** dataset to pre-train the adversarial model M' ,
 - and for stealing M^* trained with **LL**, we use the **CIFAR10** dataset to pretrain M' .

Following the approach taken by [28], we use three different background information for MS:

1. the full training set S , i.e., $\mathcal{A}$'s background information covers all the (private) training data S . It should be noted that in this case $\mathcal{A}$ can train the model from scratch. Therefore this is an unrealistic scenario to provide an upper bound on the attack performance;
2. the validation set S_V , i.e., $\mathcal{A}$'s background information has *no* intersection with S ;
3. a randomly sampled subset from the S of the size $|S_V|$, i.e., $\mathcal{A}$'s background information intersects with part of S .

We refer to [14] and consider four privacy-preserving methods in our experimental comparison: Noisy Gradient (**NG**) [40], Gradient Compression (**GC**) [18], Differentially Private - Stochastic Gradient Descent (**DP-SGD**) [1], and Discrete Stochastic Gradient Descent (**DSGD**) [4]. **NG** is a common strategy that adds noise to the gradients [40]. As in previous work [14] we consider three **NG** ratios for comparisons, namely 10^{-4} , 10^{-3} and 10^{-2} . **GC** is a method for privacy protection and communication efficiency [18]. We consider 75%, 50%, 25% and 10% as compression rates. **DP-SGD** is widely used in privacy-preserving deep learning [1]. We choose ϵ to be 0.1, 1 and 10. **DSGD** protects the signs of gradients. Thus, it benefits not only the privacy protection but also contributes to the communication efficiency [4]. We choose **DSGD** bins to be 6, 12, 18, and 24 as suggested by previous work [14].

4.2 Results

Performance. To analyze the performance, we measure the accuracy change over epochs displayed in Fig. 3 and the total execution time shown in Table 2. Overall, the results confirm that using SAIR with a pre-trained VAE to replace the insecure baseline model has minimum impact on the training and inference cost. Figure 3 shows that integrating a VAE does not affect the convergence of training. Moreover, employing pre-trained VAEs accelerates the speed of convergence. Table 2 shows that until convergence the basic SAIR consumes a maximum of up to 1.43 times more training time than baseline. The acceleration from a pre-trained model is also significant. The time cost is c.a. 1.04 to 1.40 of that of the baseline.

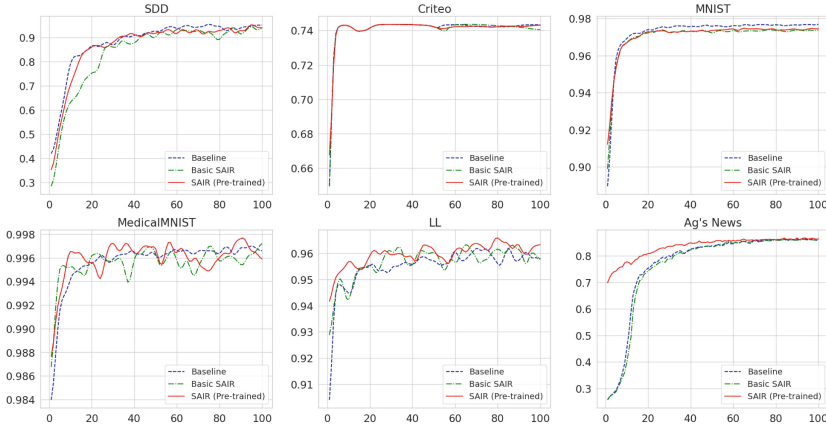


Fig. 3. Accuracy changes in training over different datasets. Models with pre-trained VAE and correlated randomness converge faster than baseline and those without pre-trained models. All experiments run 100 epochs.

Table 2. Time consumption (in minutes) till convergence in each setting. The time used by the Baseline is taken as 1.00x.

Settings/Datasets	SDD	Criteo	MNIST	MedicalMNIST	LL	AG's News
Baseline (not secure)	44.04 (1.00x)	251.13 (1.00x)	31.31 (1.00x)	39.00 (1.00x)	109.17 (1.00x)	296.71 (1.00x)
Basic SAIR	57.68 (1.30x)	258.21 (1.03x)	44.96 (1.43x)	47.85 (1.23x)	141.38 (1.29x)	370.57 (1.24x)
SAIR (Pre-trained)	50.20 (1.14x)	266.20 (1.06x)	43.41 (1.40x)	46.78 (1.21x)	118.89 (1.09x)	309.08 (1.04x)

The dELDP of VAE. Table 3 shows the results for $\frac{\Delta_\ell}{\sigma_\ell}$ computed by Algorithm 1 as described in in Sect. 4.1. The ratio $\frac{\Delta_\ell}{\sigma_\ell}$ lies between 0.1 and 2.1. For the VAE trained on **SDD** we observe the maximum ratio at client 0 for label 6.

Meanwhile, for the VAE trained on **MedicalMNIST** the minimum value is found for client 1 and label 1. The ratios are more evenly distributed around 0.5 for the VAE trained on **LL**. Besides, the VAE trained on **AG's News** obtains ratios above 1.1 for all labels besides label 4. By setting $\delta = 2^{-32}$, we obtain ϵ values between 2.59 and 10.23 for the different models based on the ratios presented in the table and we estimate $\epsilon \approx 6.4$ with $\delta = 2^{-32}$ on average.

Table 3. dELDP results, the maximum $\frac{\Delta_\ell}{\sigma_\ell}$ values for each label-model combination. The largest value on each client is marked in bold.

Dataset	Labels & Clients	Ratio										
SDD	Label	1	2	3	4	5	6	7	8	9	10	11
	Client 0	0.5381	0.5611	0.2431	0.4871	0.4032	2.1041	0.2155	0.5046	0.7639	0.4732	0.4926
	Client 1	0.6210	0.5126	0.3827	0.5971	0.6307	1.9711	0.4227	0.7162	0.8985	0.4009	0.4071
MedicalMNIST	Label	1	2	3	4	5	6					
	Client 0	0.2430	0.3360	0.8323	0.1992	1.1190	1.8725					
	Client 1	0.1092	0.2079	1.2161	0.1474	0.8595	0.3430					
LL	Label	1	2	3	4	5	6	7	8	9		
	Client 0	0.4781	0.7276	0.5979	0.7690	0.7464	0.4585	0.7487	0.7345	0.6337		
	Client 1	0.4869	0.4273	0.4070	0.5339	0.4711	0.4410	0.4846	0.4188	0.4644		
AG's News	Label	1	2	3	4							
	Client 0	1.3804	1.7998	1.6909	0.4732							
	Client 1	1.1874	1.3442	1.3543	0.4009							

Resilience Against Attacks. As aforementioned, we executed three attacks. The results on the robustness of models trained on MNIST against the feature reconstruction (FR) and the label inference (LI) attack are presented in Table 4, and the results w.r.t. model stealing (MS) in Table 5. For comparison, we consider both the model performance and the resilience, as a model with low accuracy is usually more robust against attacks but not useful. The baseline model achieves an accuracy of 97%, 86%. This accuracy is only met by models using the **GC** defense and reduced for all other privacy-enhancing techniques. The performance is most heavily reduced for **DP-SGD** and **NG** with a noise scale of 10^{-2} or 10^{-3} . For SAIR we only observe a very slight performance loss.

In Table 4, we compare the robustness of SAIR against FR and LI attacks with that of models protected by **NG**, **GC**, **DSGD** and **DP-SGD**. We estimate the robustness against the FR attack by measuring the mean squared error (MSE) between the real and the reconstructed features in dependency of the percentage of features owned by the adversary. That is, the larger the MSE, the higher the robustness. For the LI attack we measure the accuracy of the label inferred by the adversary w.r.t. the true label. The higher the robustness of the model, the lower this accuracy will be. **DP-SGD** has a more powerful defense than **GC**, **DSGD** and **NG**, as clearly observed from the label inference accuracy, but it also incurs a significant utility loss. In comparison with using no defense and $\epsilon = 10$, SAIR has higher accuracy while having the efficiency of **DP-SGD** with $\epsilon = 10$. In addition, SAIR achieves notable efficiency, even when the adversary

holds a large proportion of features. When the adversary has 75% features and no protection, the label extraction accuracy is 97.02%. Meanwhile, SAIR has a label extraction accuracy of only 87.68%. The last row of Table 4 also shows that the internal randomness in VAE can be empirically combined with **DP-SGD** and achieve better resilience against attacks.

Table 4. MNIST: Adversary performance for the feature and label extraction attacks against the different defenses, and considering an adversary holding different ratios of features. The adversaries hold 25%, 50%, and 75% of the total feature space. Note that we use the Mean Square Error (MSE) for feature extraction, whereas for label extraction, we use the Accuracy (Acc).

Defense Approach	Parameter	Parameter Set Value	Model Accuracy (50%)	FR; MSE			LI; Acc		
				25%	50%	75%	25%	50%	75%
No Defense			97.86%	2.46	2.32	1.37	35.05%	88.30%	97.02%
NG	Noise Scale	1e-4	97.34%	2.32	2.12	1.40	10.20%	82.74%	95.56%
		1e-3	96.88%	2.34	2.13	1.57	9.11%	82.48%	95.52%
		1e-2	94.17%	2.71	3.07	2.13	10.14%	76.43%	82.70%
GC	Compression Rate	75%	97.85%	2.33	2.34	1.39	9.54%	86.71%	95.01%
		50%	97.87%	2.33	2.32	1.39	10.74%	84.07%	95.50%
		25%	97.85%	2.35	2.31	1.38	11.49%	86.65%	95.92%
		10%	97.82%	2.35	2.32	1.41	9.82%	85.64%	95.56%
DSGD	Bin	24	97.42%	2.34	2.34	1.41	9.86%	85.14%	95.12%
		18	97.32%	2.36	2.38	1.50	9.62%	85.59%	92.90%
		12	97.42%	2.37	2.44	1.38	11.26%	79.43%	95.09%
		6	97.43%	2.61	2.29	1.45	6.74%	80.72%	94.12%
DP-SGD	ϵ	0.1	91.70%	2.28	2.18	1.37	9.31%	57.50%	71.85%
		1	95.57%	2.38	2.24	1.35	12.62%	72.81%	79.15%
		10	96.80%	2.40	2.30	1.44	10.07%	80.38%	87.03%
Ours			97.68%	2.34	2.15	1.36	9.67%	81.66%	87.68%
Ours & DP-SGD($\epsilon=10$)			96.94%	2.42	2.28	1.43	9.63%	77.18%	86.15%

When facing MS, SAIR in general outperforms **DP-SGD** in both the utility and protection metrics. In comparison with the baseline, SAIR has an accuracy loss below 0.5%, while **DP-SGD** drops up to 16% in the in **MedicalMNIST** (83.06% versus the baseline 99.9%). With regard to the attack, SAIR also outperforms **DP-SGD** in provided protection when using the validation data or the training subsamples, with a minimum observed attacker accuracy of 11.52% in the **MNIST** dataset. We use the full training data as the strongest attacker reference, which shows two special cases in **MedicalMNIST**. When the task accuracy is 83.06%, the adversary is not able to achieve better result (82.36%). However, when the task accuracy is very good (99.96%), the adversary is then also powerful to achieve a good result (97.63%).

Table 5. Model Stealing Attacks

Dataset	Protection	Task Accuracy	Attack Accuracy		
			Full Training Data	Validation Data	Training Subsamples
MNIST	Baseline	97.68%	77.42%	33.44%	33.61%
	DP-SGD($\epsilon = 10$)	93.83%	76.95%	32.83%	32.82%
	Ours	97.53%	67.80%	11.52%	11.54%
MedicalMNIST	Baseline	99.90%	98.93%	97.06%	96.73%
	DP-SGD ($\epsilon = 0.1$)	83.06%	82.36%	80.15%	80.60%
	Ours	99.96%	97.63%	79.96%	80.43%
LL	Baseline	95.63%	92.81%	90.14%	90.43%
	DP-SGD ($\epsilon = 0.1$)	94.72%	91.51%	89.98%	89.54%
	Our Solution	95.16%	88.28%	86.55%	86.14%

5 Conclusion and Future Work

We proposed an empirical estimate of distance-based local differential privacy that we refer to as dELDP. We demonstrate that it can be employed to quantify the privacy of variational auto encoders (VAEs). VAEs can be used as components of vertical federated learning (VFL) systems with the goal of inducing internal randomness. Our experimental study shows that such VAE containing VFL systems display increased robustness against various state-of-the-art privacy attacks while maintaining good overall performance.

We still wonder whether it is possible to compose the parameters of both worlds, i.e., form a generic bound given both the LDP and dELDP mechanisms robustly in theory. Moreover, developing concrete algorithms for other types of model internal randomness, e.g., components within Bayesian networks [33], can be meaningful future work. In addition, SAIR can be combined with orthogonal privacy-enhancing mechanisms. For example, when each individual embedding must be protected against strong adversaries, secure aggregation with malicious security [23] can be applied in addition. If the secrecy of top model parameters is critical, using multi-party computation [20] for the top model is a good option. Analyzing the overall privacy of other extended pipelines is intriguing, too.

Acknowledgments. This research is funded by the European Research Center of Huawei Technologies. Asja Fischer acknowledges support by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany’s Excellence Strategy - EXC 2092 CASA - 390781972. We thank anonymous reviewers for the various constructive comments and suggestions.

A Proof of Theorem 2

Here is the complete proof of the dLDP main theorem (Theorem 2).

Proof. We start from one-dimensional case and upgrade it to the high dimensional case. If the distance $< \infty$, it is 1 by definition for x, x' with $\text{Label}(x) =$

$\text{Label}(x')$ in (8). We start from one-dimensional case and real valued $\mu(x), V(x)$. Fix a label $\ell = \text{Label}(x) = \text{Label}(x')$, The sensitivity is

$$\Delta_\ell = \max_{x, x'} (|\mu(x) - \mu(x')|)$$

We then consider the absolute value of the privacy loss for x, x' :

$$\left| \ln \left(\frac{e^{(-1/2\sigma_1^2)x^2}}{e^{(-1/2\sigma_2^2)(x+\Delta_\ell)^2}} \right) \right|, \quad (12)$$

where $\sigma_1 = \|V(x)\|_2$ and $\sigma_2 = \|V(x')\|_2$, and $V()$ is fixed.²

Without loss of generality, we assume $\sigma_1 \leq \sigma_2$. Then we have

$$\left| \ln \left(\frac{e^{(-1/2\sigma_1^2)x^2}}{e^{(-1/2\sigma_1^2)(x+\Delta_\ell)^2}} \right) \right| \geq \left| \ln \left(\frac{e^{(-1/2\sigma_1^2)x^2}}{e^{(-1/2\sigma_2^2)(x+\Delta_\ell)^2}} \right) \right|. \quad (13)$$

So to bound (12), we can bound the left-hand side of (13) instead, i.e., we need an ϵ_1 , such that

$$\epsilon_1 \geq \left| \ln \left(\frac{e^{(-1/2\sigma_1^2)x^2}}{e^{(-1/2\sigma_1^2)(x+\Delta_\ell)^2}} \right) \right| = \left| \frac{1}{2\sigma_1^2} (2x\Delta_\ell - \Delta_\ell/2) \right|. \quad (14)$$

Thus, if we iterate over all x with $\text{Label}(x) = \ell$ and let

$$\sigma_\ell := \min(\{\sigma_i\}), \sigma_i = V(x_i),$$

the bound for the privacy loss w.r.t. label ℓ is

$$\epsilon_\ell \geq \left| \ln \left(\frac{e^{(-1/2\sigma_\ell^2)x^2}}{e^{(-1/2\sigma_\ell^2)(x+\Delta_\ell)^2}} \right) \right| = \left| \frac{-1}{2\sigma_\ell^2} (2x\Delta_\ell - \Delta_\ell/2) \right| \quad (15)$$

With a similar argument in the proof of Theorem A.1 in [12], and since $|x|$ is identical to $\|x\|_2$ if $x \in \mathbb{R}$, then we can have the relation

$$\epsilon_\ell = \frac{\Delta_\ell}{\sigma_\ell} \sqrt{2 \log \frac{1.25}{\delta}}. \quad (16)$$

By taking $\epsilon = \max(\{\epsilon_\ell\})$, we can conclude that Theorem 2 is correct for one-dimensional posterior sampling in VAE.

We use a convention $\mu_x := \mu(x)$. For the high dimension case, i.e., $\mu_x, x, V(x) \in \mathbb{R}^m$, the sensitivity is changed to

$$\Delta_\ell = \max_{x, x'} (\|\mu_x - \mu_{x'}\|_2).$$

² $V(x) \in \mathbb{R}$, so $|V(x)| = \|V(x)\|_2$. The VAE parameter θ has been trained.

Similarly, we are interested in the privacy loss w.r.t. label ℓ

$$\left| \ln \left(\frac{e^{(-1/(2\sigma_1^\top \sigma_1))\|x\|_2^2}}{e^{(-1/(2\sigma_2^\top \sigma_2))\|(x+\mu_x-\mu_{x'})\|_2^2}} \right) \right|, \quad (17)$$

For $\sigma_1 = V(x)$ and $\sigma_2 = V(x')$, with $\sigma_1^\top \sigma_1 \leq \sigma_2^\top \sigma_2$, we can have

$$\left| \ln \left(\frac{e^{(-1/(2\sigma_1^\top \sigma_1))\|x\|_2^2}}{e^{(-1/(2\sigma_2^\top \sigma_2))\|(x+\mu_x-\mu_{x'})\|_2^2}} \right) \right| \geq \left| \ln \left(\frac{e^{(-1/(2\sigma_1^\top \sigma_1))\|x\|_2^2}}{e^{(-1/(2\sigma_1^\top \sigma_1))\|(x+\mu_x-\mu_{x'})\|_2^2}} \right) \right|. \quad (18)$$

If we assume that $\mu_x - \mu_{x'}$ with the maximum norm is aligned with x (which gives the biggest denominator), we are back to the one-dimensional case, i.e., for

$$\sigma_\ell = \arg \min_{\sigma_i} (\sigma_i^\top \sigma_i),$$

the bound ϵ_ℓ for the privacy loss must fulfill

$$\epsilon_\ell \geq \left| \ln \left(\frac{e^{(-1/(2\sigma_\ell^\top \sigma_\ell))\|x\|_2^2}}{e^{(-1/(2\sigma_\ell^\top \sigma_\ell))(\|x\|_2 + \Delta_\ell)^2}} \right) \right| = \left| \frac{-1}{2\sigma_\ell^\top \sigma_\ell} (2\|x\|_2 \Delta_\ell - \Delta_\ell/2) \right|. \quad (19)$$

So we have (20) in high dimension. Theorem 2 holds.³

$$\epsilon_\ell = \frac{\Delta_\ell}{\sqrt{\sigma_\ell^\top \sigma_\ell}} \sqrt{2 \log \frac{1.25}{\delta}}. \quad (20)$$

□

References

1. Abadi, M., et al.: Deep learning with differential privacy. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp. 308–318 (2016)
2. Bai, J., Wang, W., Gomes, C.P.: Contrastively disentangled sequential variational autoencoder. In: Advances in Neural Information Processing Systems, vol. 34, pp. 10105–10118 (2021)
3. Bator, M.: Dataset for Sensorless Drive Diagnosis. UCI Machine Learning Repository (2015)
4. Bernstein, J., Wang, Y.X., Azizzadenesheli, K., Anandkumar, A.: signSGD: compressed optimisation for non-convex problems. In: International Conference on Machine Learning, pp. 560–569. PMLR (2018)
5. Bird, J.J., Faria, D.R., Premebida, C., Ekárt, A., Vogiatzis, G.: Look and listen: a multi-modality late fusion approach to scene classification for autonomous machines. In: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 10380–10385. IEEE (2020)

³ We abuse the notation by letting $\sigma_\ell = \sqrt{\sigma_\ell^\top \sigma_\ell}$ so we have the form in the theorem.

6. Bonawitz, K., et al.: Practical secure aggregation for privacy-preserving machine learning. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, pp. 1175–1191 (2017)
7. Burchard, P., Daoud, A., Dotterer, D.: Empirical differential privacy. arXiv preprint [arXiv:1910.12820](https://arxiv.org/abs/1910.12820) (2019)
8. Cohen, G., Afshar, S., Tapson, J., Van Schaik, A.: EMNIST: extending MNIST to handwritten letters. In: 2017 International Joint Conference on Neural Networks (IJCNN), pp. 2921–2926. IEEE (2017)
9. Dai, Y., et al.: Improving adversarial robustness of medical imaging systems via adding global attention noise. *Comput. Biol. Med.* **164**, 107251 (2023)
10. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: Bert: pre-training of deep bidirectional transformers for language understanding. arXiv preprint [arXiv:1810.04805](https://arxiv.org/abs/1810.04805) (2018)
11. Duan, Y.: Privacy without noise. In: Proceedings of the 18th ACM Conference on Information and Knowledge Management, pp. 1517–1520 (2009)
12. Dwork, C., Roth, A., et al.: The algorithmic foundations of differential privacy. *Found. Trends® Theor. Comput. Sci.* **9**(3–4), 211–407 (2014)
13. Erlingsson, Ú., Feldman, V., Mironov, I., Raghunathan, A., Talwar, K., Thakurta, A.: Amplification by shuffling: From local to central differential privacy via anonymity. In: Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms, pp. 2468–2479. SIAM (2019)
14. Fu, C., et al.: Label inference attacks against vertical federated learning. In: 31st USENIX Security Symposium (USENIX Security 2022), pp. 1397–1414 (2022)
15. Geng, J., et al.: Towards general deep leakage in federated learning. arXiv preprint [arXiv:2110.09074](https://arxiv.org/abs/2110.09074) (2021)
16. Grining, K., Klonowski, M.: Towards extending noiseless privacy: dependent data and more practical approach. In: Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security, pp. 546–560 (2017)
17. Jiang, X., Zhou, X., Grossklags, J.: Comprehensive analysis of privacy leakage in vertical federated learning during prediction. *Proc. Priv. Enhancing Technol.* **2022**(2), 263–281 (2022)
18. Kairouz, P., et al.: Advances and open problems in federated learning. *Found. Trends Mach. Learn.* **14**(1–2), 1–210 (2021)
19. Kingma, D.P., Welling, M.: Auto-encoding variational bayes. arXiv preprint [arXiv:1312.6114](https://arxiv.org/abs/1312.6114) (2013)
20. Knott, B., Venkataraman, S., Hannun, A., Sengupta, S., Ibrahim, M., van der Maaten, L.: Crypten: secure multi-party computation meets machine learning. In: Advances in Neural Information Processing Systems, vol. 34, pp. 4961–4973 (2021)
21. Koker, T., Mireshghallah, F., Titcombe, T., Kaissis, G.: U-noise: learnable noise masks for interpretable image segmentation. In: 2021 IEEE International Conference on Image Processing (ICIP), pp. 394–398. IEEE (2021)
22. Criteo dataset (2021). <https://labs.criteo.com/2014/02/download-kaggle-display-advertising-challenge-dataset/>
23. Li, H., Lin, H., Polychroniadou, A., Tessaro, S.: LERNA: secure single-server aggregation via key-homomorphic masking. In: Guo, J., Steinfeld, R. (eds.) ASIACRYPT 2023. LNCS, vol. 14438, pp. 302–334. Springer, Cham (2023). https://doi.org/10.1007/978-981-99-8721-4_10
24. Li, X., et al.: Opboost: a vertical federated tree boosting framework based on order-preserving desensitization. *Proc. VLDB Endow.* **16**(2), 202–215 (2022). <https://doi.org/10.14778/3565816.3565823>

25. Liu, Y., et al.: Vertical federated learning. arXiv preprint [arXiv:2211.12814](https://arxiv.org/abs/2211.12814) (2022)
26. Luo, X., Wu, Y., Xiao, X., Ooi, B.C.: Feature inference attack on model predictions in vertical federated learning. In: 2021 IEEE 37th International Conference on Data Engineering (ICDE), pp. 181–192. IEEE (2021)
27. Neumeier, M., Botsch, M., Tollkühn, A., Berberich, T.: Variational autoencoder-based vehicle trajectory prediction with an interpretable latent space. In: 2021 IEEE International Intelligent Transportation Systems Conference (ITSC), pp. 820–827. IEEE (2021)
28. Orekondy, T., Schiele, B., Fritz, M.: Knockoff nets: stealing functionality of black-box models. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 4954–4963 (2019)
29. Ou, W., Zeng, J., Guo, Z., Yan, W., Liu, D., Fuentes, S.: A homomorphic-encryption-based vertical federated learning scheme for risk management. *Comput. Sci. Inf. Syst.* **17**(3), 819–834 (2020)
30. Ranbaduge, T., Ding, M.: Differentially private vertical federated learning. arXiv preprint [arXiv:2211.06782](https://arxiv.org/abs/2211.06782) (2022)
31. Scheliga, D., Mäder, P., Seeland, M.: Precode-a generic model extension to prevent deep gradient leakage. In: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, pp. 1849–1858 (2022)
32. Seif, M., Tandon, R., Li, M.: Wireless federated learning with local differential privacy. In: 2020 IEEE International Symposium on Information Theory (ISIT), pp. 2604–2609. IEEE (2020)
33. Shridhar, K., Laumann, F., Liwicki, M.: A comprehensive guide to Bayesian convolutional neural network with variational inference. arXiv preprint [arXiv:1901.02731](https://arxiv.org/abs/1901.02731) (2019)
34. Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. arXiv preprint [arXiv:1409.1556](https://arxiv.org/abs/1409.1556) (2014)
35. Truex, S., Liu, L., Chow, K.H., Gursoy, M.E., Wei, W.: LDP-fed: federated learning with local differential privacy. In: Proceedings of the Third ACM International Workshop on Edge Systems, Analytics and Networking, pp. 61–66 (2020)
36. Yang, C., Wang, X., Mao, S.: Autotag: recurrent variational autoencoder for unsupervised apnea detection with RFID tags. In: 2018 IEEE Global Communications Conference (GLOBECOM), pp. 1–7. IEEE (2018)
37. Yang, J., Shi, R., Ni, B.: Medmnist classification decathlon: a lightweight automl benchmark for medical image analysis. In: IEEE 18th International Symposium on Biomedical Imaging (ISBI), pp. 191–195 (2021)
38. Yang, Q., Liu, Y., Chen, T., Tong, Y.: Federated machine learning: concept and applications. *ACM Trans. Intell. Syst. Technol. (TIST)* **10**(2), 1–19 (2019)
39. Zhang, X., Zhao, J., LeCun, Y.: Character-level convolutional networks for text classification. In: Advances in Neural Information Processing Systems, vol. 28 (2015)
40. Zhu, L., Liu, Z., Han, S.: Deep leakage from gradients. In: Advances in Neural Information Processing Systems, vol. 32 (2019)



Code Voting: When Simplicity Meets Security

Véronique Cortier, Alexandre Debant, and Florian Moser^(✉)

CNRS, Université de Lorraine, INRIA, Nancy, France
{veronique.cortier,alexandre.debant,florian.moser}@inria.fr

Abstract. The Swiss Chancellery has developed highly demanding requirements in terms of security for electronic voting. A few protocols have been proposed that meet the requirements.

We propose a very simple protocol that satisfies the Swiss requirements while achieving an additional property: secrecy against a dishonest voting device, thanks to code voting. One key feature of our protocol is to use very short codes (typically one or two digits), so that it can be easily used by voters. Moreover, it only relies on standard primitives. Using the tool ProVerif, we formally prove vote privacy and end-to-end verifiability under the trust model of the Swiss Chancellery, assuming a dishonest voting device.

Keywords: Internet Voting · Code Voting · Privacy · Verifiability · Switzerland

1 Introduction

Electronic voting is used in several countries, such as Australia [8], Estonia [22], Switzerland [39], or France [17]. It aims at preserving the same properties as traditional paper-ballots elections and in particular two main properties, namely vote privacy and verifiability: the result of the election should reflect the votes of all legitimate voters. Verifiability is often divided into several sub-properties such as cast-as-intended, individual, universal, and eligibility verifiability [29].

All these properties come with trust assumptions on the parties involved in the protocol. Such assumptions vary a lot depending on the voting system. For example, some systems such as Helios [1], Belenios [16], or Civitas [12] consider a fully dishonest voting server but assume a public Bulletin Board that anyone can see and trust [23]. Another approach followed in Estonia [22] and Switzerland [39] is to maintain a private bulletin board, distributed over several online servers, among which only one is trusted. Another important element of a voting system is the voting device, which is assumed to be trusted or untrusted depending on the protocol.

This work benefited from funding managed by the French National Research Agency under the France 2030 program with the reference ANR-22-PECY-0006. It was also partly supported by the ANR Chair IA ASAP (ANR-20-CHIA-0024) with support from the region Grand Est, France.

Dishonest Voting Device. Securing Internet voting against a dishonest voting device is a challenging task. A versatile approach is the Benaloh’s challenge [2], which allows a voter to check that their vote has been properly encrypted. This approach is, however, hard to use in practice [33]. In the Estonian system [22] or with CAISED [36], voters are offered the possibility to use a second device to control the behavior of their main voting device. In Switzerland, the voters are given a voting sheet before the start of the election that contains a *return code* for each voting option [39]. Then, when a voter selects some voting option v on their voting device, they should be displayed the corresponding return code. This mechanism guarantees their vote has been correctly transmitted since the voting device cannot guess the return codes.

However, all these solutions only focus on cast-as-intended: even if the voting device is compromised, the voter’s *intended* vote should be correctly encrypted and sent to the voting server. None of these approaches protect against vote privacy: a dishonest voting device immediately learns the voter’s vote since the vote is entered in clear by the voter. This yields an uncomfortable situation where a device is trusted for one key property (vote privacy) but not the other one (verifiability). A few systems overcome this issue using *code voting* introduced by Chaum [10]: a voter no longer enters their vote in clear but transmits a voting code instead, so the voting device does not learn the vote.

Swiss Context. The Swiss Chancellery [19] is highly demanding in terms of security for Internet voting. The specification should be fully public, as well as the code of the system. Public scrutiny is encouraged through bug bounty programs. In the Swiss approach, the voting sheets distributed to voters are generated by a trusted **Setup Component**, in collaboration with **Control Components**. When a voter votes, their **Voting Device** encrypts their vote and sends the ciphertext to the control components, which intuitively play the role of a distributed voting server. The control components collaboratively compute the corresponding return code, which is sent back to the voter, who will confirm their vote only if the return code is correct.

As defined by the Swiss Chancellery, verifiability should hold even if the attacker controls the network, all the online control components but one, and the voting device. For vote privacy, the trust assumptions are similar, except the voting device is now trusted. Our goal is to suppress this assumption.

Our Contribution. Our main contribution is a voting protocol that satisfies the security requirements of the Swiss Chancellery (vote privacy and verifiability) without assuming a trusted voting device, even for vote privacy. A recent proposal [20] aims at the same goal, relying on BLS signatures [7] and long voting codes. Comparing to it, we show that it is possible to obtain the same security guarantees using a radically different approach, with very short voting codes of typically 1 or 2 digits. Interestingly, our protocol is very simple in terms of design and cryptographic primitives. Besides homomorphic encryption or verifiable mixnets for the tally, it only uses very standard cryptographic primitives,

such as signatures and hashes. We believe our protocol can easily be implemented and maintained, even by programmers with a light background in cryptography.

In addition to removing the trust in the voting device, we mitigate one of the strongest trust assumptions of the Swiss context: the trusted setup component. This component is trusted both for vote privacy and verifiability. We keep this assumption but our protocol is designed in such a way that the computations of SC are minimal and easy to audit. In particular, it does not need to generate any randomness. Hence, it is easy to check that the output corresponds to the input, with no additional information or bias introduced through the randomness. So, if the SC misbehaves, it can be caught through random audits. Moreover, a malicious setup could use well-chosen randomness to leak information. In our design, we remove the use of randomness in the core design of the setup, hence limiting the risk of covert channels. The use of randomness is limited to external components, e.g., to implement authenticated channels with signatures.

Finally, we provide a full security proof of both vote privacy and verifiability using the ProVerif [5] tool, for an arbitrary number of voters and an arbitrary number of elections. For verifiability, we rely on a recent framework [13], that allows to directly prove end-to-end verifiability, that is, proving that the final result accounts for all the votes of honest voters, plus at most one vote per dishonest voters. Directly verifying end-to-end verifiability avoids the need to consider several sub-properties (e.g., cast-as-intended, recorded-as-cast), at the risk of missing an important property [31].

Outline of the Paper. In the next section, we further discuss other voting protocols based on code voting. Our protocol is presented in Sect. 3, and we explain its security model and proof in ProVerif in Sect. 4. Some concluding remarks are in Sect. 5.

2 Related Work

In the Swiss context, two main protocols have been proposed, namely CHVote [21] and the one developed by Swiss Post [39]. However, both assume that the voting device is trusted for vote privacy. In the literature, a few systems have been proposed to achieve vote privacy against a dishonest voting device, using code voting, following the seminal work of Chaum [10]. Among them, only one, HKL [20], has been designed specifically for the Swiss context, i.e., with the aim to satisfy the Swiss trust assumptions. Others have been designed in a more general context. Hence, their different trust assumptions led to different design compromises. We discuss here the systems that aim at protecting vote privacy against a dishonest voting device.

Comparison to HKL Protocol [20]. In the HKL protocol, voters receive a voting sheet in which each voting option is associated to a QR-code, that encodes the encrypted vote. Thanks to a verifiable mixnet during the setup, the relation between the ciphertext and its plain vote remains private for anyone without

access to the voting sheet. BLS signatures [7], based on pairing, are then used to prevent ballot stuffing.

One major drawback of this approach is its usability. Indeed, in Switzerland, a voter typically needs to answer about 6 questions, and for several questions, 3 options can be selected (yes, no, abstain). This would amount to selecting 6 QR-codes out of 18 for a typical election. But some elections have more questions (e.g., 13 questions in Zurich in autumn 2022 [27, 38], therefore amounting to 39 QR-codes) and some elections even offer voters to select *several* options for each question [26]. Moreover, because the camera used to scan the QR-code(s) shall not see the other cryptographic data printed onto the voting sheet for security purposes, the voter is subject to additional constraints when manipulating the voting sheet. Recent studies [34, 41] conclude that QR-codes are usable for electronic voting when voters select exactly *one* voting option in total, but it seems cumbersome to use such a technology in elections where voters have to select more than a few options.

Compared to HKL, our protocol is QR-code-free (exception, optionally, for the first step). Voters only need to enter a short code for each question: if a question has 15 possible answers, then the voter simply needs to enter a number between 1 and 15. The association between these small codes and the corresponding encrypted votes is committed by the setup component and audited by auditors before the start of the election. The return codes guarantee that the voting device cannot modify the voting code without being detected. Studies have also explored the usability of entering voting codes [30, 34], and even though the voting codes were longer than in our proposal (8 numbers in [34], and 3 characters in [30]), reached acceptable usability scores.

Comparison to Non-swiss Specific Systems. Outside the Swiss context, BeleniosVS [15] follows the same approach as HKL: the code is simply the encrypted vote. However, BeleniosVS assumes in addition that voters can access a public Bulletin Board. Unfortunately, existing techniques to implement a secure public Bulletin Board [23] typically assume a signature mechanism. Since a voter cannot check the validity of a signature by themselves, they need to rely on their voting device to do so. Assuming the voting device dishonest as in our setting, it may simply lie to the voter. Therefore, an approach based on a public Bulletin Board does not seem to be a realistic option under our trust assumptions.

Pretty Good Democracy (PGD) [37] uses long, hard to guess, codes for each voting choice, that intuitively points to the right encrypted vote in the data prepared during setup. In PGD, if the attacker knows another code, it may swap votes. While long codes may be acceptable when only one vote is expressed, this does not fit well in the Swiss context. Furthermore, the control components (the trustees in the PGD setting) need to be trusted for verifiability, as they may lie on how they shifted the ciphertexts during the setup. Pretty Understandable Democracy (PUD) [9] also uses pointers to ciphertexts. The voter needs to enter several codes for each voting option, received from different voting sheets, one from each server (these servers intuitively correspond to the Swiss control components). This may again be very cumbersome in practice. The VeryVote

system [25] shifts the trust from the voting device to the voting server that (intuitively) encrypts the vote for the voter, but in a verifiable manner. The authors later extend their approach to remove the trust in the voting server [24], but at the cost of a secure hardware token on the voter side, hence trusting again some part of the voting device.

All these systems use either voting codes which need to be resistant to random guessing or encode directly the ciphertext or signatures. This results into long voting codes in the order of 128 bits or even more. To our knowledge, the only scheme that uses short voting codes, hence in the order of number of candidates, is D-Demos [11]. As in our proposal, the voting code simply points to the right ciphertext, and the number of needed voting codes is simply the number of voting options. D-DEMOS uses a double-ballot system, where the voter chooses which ballot to audit and which one to use for voting, which leads to a slightly more complex system. Moreover, D-Demos assumes a public Bulletin Board, which tends to discard it for real-world usages under our trust assumptions as discussed above. Finally, their setup component has to generate a lot of cryptographic, randomized, material, which may be hard to audit.

Table 1 displays a comparison of the existing code voting systems that do not assume any trust in the voting device. We first compare their security (privacy and verifiability) for different trust models. We highlight the trust model of the Swiss Chancellery and our trust model. For the sake of comparison, we also display the two main systems that have been designed for the Swiss context, namely Swiss Post [39] and CHVote [21]. None of them achieve privacy against an insecure voting device but in contrast, they achieve a better verifiability level than code voting when the setup component is dishonest but the voting device is trusted (a scenario outside the Swiss trust model).

We also compare the systems in terms of usability and auditability. We pinpoint which systems use short voting codes and which are “vote and go”; that is, a voter simply needs to interact with the system during the voting phase, and does not need to come back afterward, e.g., to check that their vote has been counted. In terms of auditability and simplicity, we highlight systems that use a deterministic setup and do not use cryptography on the voting device. We also remark that most papers focus on the design of the system and do not accompany it with a security proof of verifiability, vote privacy, or both.

Table 1. Comparison of code voting systems. Block one and two describe scenarios with different trust assumptions for vote privacy and verifiability corresponding to the control components CC, the setup component SC and the voting device VD. The more components need to be trusted, the less secure is the protocol. In *italic* the Swiss trust model, and in gray our stronger trust model (we no longer assume a trusted voting device for privacy). **c**: these protocols assume a public bulletin board that voters can securely access. This implicitly assumes some trusted device. In PGD, all control components need to be honest for verifiability, as they may lie on how they shifted the ciphertexts during the setup. In VeryVote, the server is trusted to encrypt the candidate for the voter, which is a stronger assumption than the ones we consider here.

	Swiss Setting				Voting Codes Systems				
	SPP	CHVote	HKL	Proposal	PGD	PUD	D-DEMOS	BeleniosVS	VeryVote
	[39]	[21]	[20]		[37]	[9]	[11]	[15]	[25]
Trust for Vote privacy									
<i>1/n CC, SC, VD</i>	•	•	•	•	•	•	•	•	
<i>1/n CC, SC</i>			•	•	•	•	•	•	
1/n CC, VD								•	
1/n CC									
Trust for Verifiability									
1/n CC, SC, VD	•	•	•	•		•	•	•	•
<i>1/n CC, SC</i>	•	•	•	•		•	c	c	c
1/n CC, VD	•	•							•
1/n CC									c
Security Proof									
Privacy	•			•			•	•	
Verifiability	•	•		•			•	•	
Construction									
Short Voting Codes				•			•		
Deterministic SC		•	•	•	•				
Vote and go	•	•	•	•	•	•			
No Crypto on VD			•	•	•	•	•		

3 Protocol

We first recall the voting infrastructure of the Swiss context. We then describe our protocol and its main security properties.

3.1 The Swiss Context

The Swiss Chancellery has precisely defined the main actors of a voting system in the Swiss context, with associated communication channels. We summarize the given model as follows.

The **Setup Component** is active during the setup. It computes the voting material and distributes it to the voters through postal mail. It also prepares the material used to tally the votes.

The **Control Components** are entities that are active during all the voting phases. They collectively generate the randomness used during the setting, register the votes and compute return codes during the voting phase, and gather the encrypted ballots for the tally phase.

The **Decryption Authorities** are in charge of generating the public key of the election and decrypting during the tally. Typically, a threshold of t out of n authorities is necessary to decrypt.

The **Voting Device** interacts with the **Voter** to help them cast a vote over the Internet.

The voting device does not communicate directly with the control components. Instead, all communications go through a **Voting Server** that is fully untrusted and simply sends the messages back and forth from the voting devices and the control components. Therefore, we will omit its description in what follows.

Trust Assumptions. We consider the trust model as defined by the Swiss Chancellery [19].

The network is fully under the control of the attacker. In particular, the security of the protocol cannot rely on TLS. However, postal mail from the setup component to the voter is assumed to be trusted: the attacker can neither read nor tamper with the voting sheets sent to the voters.

The setup component is assumed to be fully trusted. It is only active during the setup phase and does not need to be online. The voting material (the voting sheets) is sent over secure channels to the voters (in practice, by Post). The control components are typically a set of 4 entities, among which only one is trusted for verifiability and vote privacy. The decryption authorities are trusted as a whole, with a threshold of trusted authorities. Finally, the voting device is untrusted for verifiability but trusted for privacy in the trust model of the Chancellery. This is, however, the point where our trust assumptions differ: we no longer assume the voting device to be honest for privacy.

Moreover, the Chancellery's model places a lot of trust in the setup component since it is trusted both for privacy and verifiability. This is something that we mitigate in our protocol by proposing a fully deterministic process, easy to audit.

3.2 Voter's View

To introduce the protocol, we first present the voter's view. The voter is given a ballot sheet with a QR code to login, and codes to cast and confirm their vote. See Fig. 1 for an example using realistic parameters (e.g., length of codes). First, the voter scans the QR code to login ① with her identifier Id . The QR code contains the URL of the voting server, reducing the chance that the voter ends up on the wrong page. Then, the voter enters the short voting codes $c_1, \dots, c_n$

of the voting choices they intend to vote for ②. The voting codes are sent to the voting system, which responds with return codes $rc_1, \dots, rc_n$, one for each selected voting option (codes for blank votes shall be provided). If and only if these codes match what is printed on the voting sheet ②, the voter is instructed to confirm the vote ③ by sending the authentication code ac . This is the only long secret value that authenticates the voter. The voter finally receives a finalization code fc , that guarantees them that their vote has been counted.

The voting client essentially needs to forward values to the voting system and back to the user. It does not need to encrypt or sign the vote. The voting client may guide the user with basic validation (e.g., ensuring the voting code entered for the first question is between 1 and 3).

① **Start**

Scan to open the election platform and login:



Then you can cast your Vote ②.

③ **Confirm**

If Verify letters match, enter Authentication:

F28Q nAJcj 6W3f C5CXt vwfa

Afterwards, check Finalization is shown:

JQHS UMKQ

If Finalization shown, your vote is stored.
If Finalization is not shown, call 114.

② **Vote**

Do you want to accept the popular initiative
"Lower Voting Age to Sixteen Years"

	Vote	Verify
Yes	3	ADEH
No	1	KYSO
Abstain	2	KKHA

Do you want to accept the federal law
"Electronic Identification (eID Act)"

	Vote	Verify
Yes	4	FSUM
No	6	PINQ
Abstain	5	UAQM

Enter Vote numbers to cast the vote.
 Afterwards, check Verify letters are shown.

If Verify letters match, then Confirm ③.
If Verify letters do not match, abort.

Fig. 1. An example of a voting sheet using realistic parameters. ① contains a QR code with the voter id. ② contains the voting codes and their respective return code. ③ contains the authentication code and the respective finalization.

3.3 Protocol Description

The protocol is divided into three phases. In the setup phase, all parties receive the required cryptographic material to run the voting and tally phase. In the voting phase, the voter submits their vote and performs their individual verifiability checks. In the final tally phase, the votes are decrypted and counted. For

simplicity, we present our protocol for an election with one question only, but it can easily be extended to several questions (e.g., the voting sheet presented in Fig. 1 has two questions).

Notations. We denote $\mathbb{Z}_n$ or $[1, n]$ the set of integers between 1 and n and $\oplus_n$ the addition modulo n . If n is obvious from the context, we may omit it. The set of all possible permutations of $\mathbb{Z}_n$ is denoted S_n . The composition of two permutations π_1 and π_2 is denoted $\pi_1 \circ \pi_2$.

The set of voting options is denoted $\mathcal{V}$ and is of size n_v . The correspondence between voting options and voting codes will be denoted by a substitution (or a table) σ from $\mathcal{V}$ to $\mathbb{Z}_{n_v}$. The application of a substitution σ to an element $v \in \mathcal{V}$ is denoted $\sigma[v]$. We assume an *initial correspondence* σ_0 , chosen publicly by the authorities, that maps each voting option to some integer in $\mathbb{Z}_n$. This substitution is fixed for all voters. For example, if $\mathcal{V} = \{a, b, c\}$, the initial substitution could be $\sigma_0 = \{a \rightarrow 1, b \rightarrow 2, c \rightarrow 3\}$.

Cryptographic Primitives. Our protocol relies on three standard primitives.

- a hash function `hash`,
- a signature scheme. We denote $\sigma = \text{sign}(sk, m, r)$ the signature with signing key sk , message m and randomness r . Similarly, we denote $\text{verify}(pk, m, \sigma)$ the verification applied to public key pk , message m and signature σ . We write `sign`(sk, m) when the source of randomness r is clear from the context.
- an encryption scheme compatible with a homomorphic tally or a mixnet, such as the one used in Helios [1]. We denote $E = \text{enc}(pk, m, r)$ for the encryption of message m with public key pk and randomness r .

Setup. Before the setup, we assume that the authorities have agreed on a list of identifiers Id , one for each eligible voter. The identifiers are used to keep track of the cryptographic material per voter, and they are not assumed to be secret.

We also assume given n_{rc} the number of return codes, n_a the number of authentication codes, and n_f the number of finalization codes. For return and finalization codes, the adversary has a single try to convince the voter, hence the values do not need to be hard to guess. Typically, n_{rc} and n_f will be of medium size (e.g., 4 digits, that is $n_{rc} = n_f = 10^4$). Regarding authentication codes, corrupted parties will have access to their hashes, so this set must be large enough for picked values to be hard to brute-force. Hence typically $n_a = 2^{128}$. Lastly, we assume given $\mathbb{Z}_{n_r}$, the set from which encryption randomness is drawn.

Key Generation. The decryption authorities jointly generate the election public key pk_E , using a Distributed Key Generation, for example, the one used in Belenios [16] or ElectionGuard [3]. They each have a share of the decryption key such that k out of n authorities can decrypt.

We also assume that each control component CC_i has a signing key sk_i , with associated public verification key vk_i .

Preparation of the Voting Sheets. During the setup, the setup component is in charge of sending to each voter:

- their identifier Id ,
- a correspondence σ between voting options and voting codes,
- a table t that maps each voting option to a return code,
- an authentication code ac ,
- a finalization code fc .

To obtain a deterministic setup component, each control component generates for each $Id \in \mathbf{Id}$:

- a randomly chosen correspondence π_i between voting options and voting codes,
- a table t_i that maps each voting option to a randomly chosen return code,
- a table r_i that maps each voting code to a randomly chosen encryption randomness,
- a random authentication code ac_i ,
- a random finalization code fc_i .

Then, the setup component simply combines the contributions of all m control components:

- $\sigma = \sigma_0 \circ \pi$ where $\pi = \circ_{i=1}^m \pi_i$: the initial correspondence σ_0 is permuted using the combination π of the permutations π_i chosen by the CC_i ,
- $t = \Sigma_{i=1}^m t_i$,
- $r = \oplus_{i=1}^m r_i$,
- $ac = \Sigma_{i=1}^m ac_i$,
- $fc = \Sigma_{i=1}^m fc_i$,

where $\oplus$ denotes the bitwise exclusive-OR operator, and $\Sigma_{i=1}^m x_i = x_1 + \dots + x_m \bmod N$ with N defined from the context (e.g., $N = n_{rc}$ when computing t , $N = n_a$ for ac , and $N = n_f$ for fc). By abuse of notation, when operands are tables, the operator is applied for each index.

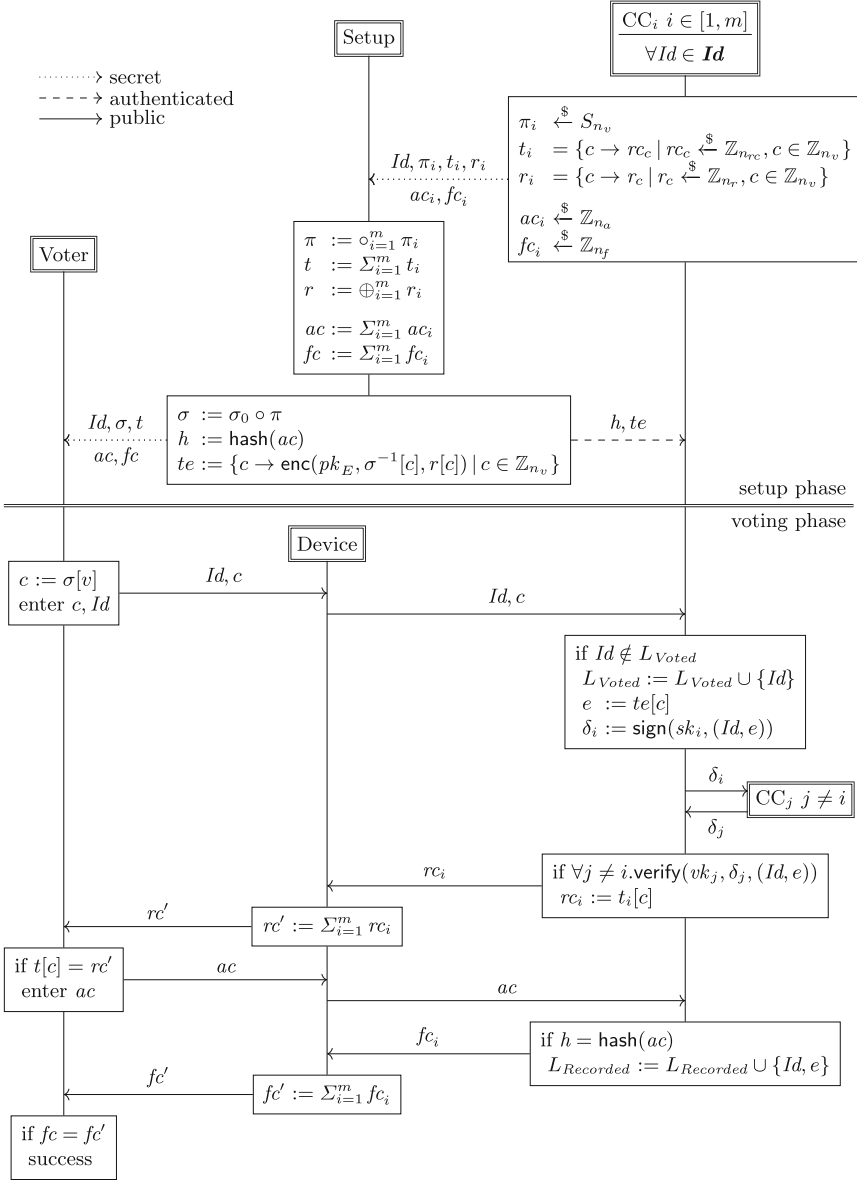
Additionally, the setup returns to the control components:

- $h = \text{hash}(ac)$, the hashed authentication code,
- a table te from the voting codes to the corresponding (encrypted) voting options

$$\{c \rightarrow \text{enc}(pk_E, \sigma^{-1}[c], r[c]) \mid c \in \mathbb{Z}_{n_v}\}$$

using the XORed randomness received from the CC_i .

The setup phase is depicted in Fig. 2, as well as the remainder of the protocol.

**Fig. 2.** Protocol.

Audit of the Setup Component. The setup component needs to be fully trusted. Notably, it could switch the vote and verification codes of plain votes, such that a different voting option is voted for than the one intended. We introduce an audit procedure to improve the detection of a misbehaving setup component.

To make an audit of the setup component practical, we ensure it only runs deterministic algorithms. Note that if at least one control component performs honestly, the adversary has no advantage in guessing the ballots.

To audit the setup component, we first note that the identifiers in $\mathbf{Id}$ are not tied to voters. They are just numbers (possibly randomly chosen). Hence, when preparing the election, the authorities simply add n additional entries to $\mathbf{Id}$, for n chosen in such a way that if n voters are audited, the risk is sufficiently minimized. After the setup component finishes execution, each control component i chooses some subset of voters to audit $\mathbf{Id}_A^{(i)} \subset \mathbf{Id}$ such that $|\cup_i \mathbf{Id}_A^{(i)}| = n$. Any corresponding $\mathbf{Id}$ can no longer be used to cast votes, but instead, the control components publish all values they generated for this $\mathbf{Id}$. The control components then assert that the voting sheet, h , and te have been generated correctly. Finally, the remaining voting sheets are randomly assigned to voters, to prevent targeted attacks on specific $\mathbf{Ids}$ from the setup and control components.

Voting Phase. The voter cast their vote by interacting with the control components.

- The voters first select the voting code c corresponding to their vote v and they simply enter c together with their identifier $\mathbf{Id}$ to their voting device.
- Each control component checks that they did not receive $\mathbf{Id}$ yet, signs $\mathbf{Id}$ together with the ciphertext e corresponding to c in their table te , and sends this signature to the other control components. Once the control component has received corresponding signatures back from all other control components, it sends their share rc_i of the return code.
- The voting device combines the return codes and displays $rc = \sum_{i=1}^m rc_i$ to the voter. If the voter agrees, they send their authentication code ac .
- Each control component checks that the code ac is valid by comparing it to its hashed value h . They now record the ciphertext e and $\mathbf{Id}$ in the list $L_{Recorded}$ of recorded votes and they each send their share fc_i of the finalization code.
- The voting device displays the combination of the fc_i to the voter, which ends the voting phase.

At any time, if the voter detects some inconsistency, they should contact the voting authorities, who then offer other means of voting.

Multiple Voting Options. For simplicity, we considered the selection of one voting option only. In case the voter has to select k out of n options, then the voter simply sends the k corresponding voting codes, the control components return k return codes and the protocol runs accordingly. In case the voter may select up to k out of n options, we use the same approach as for other Swiss systems:

we add k blank options so that the voter always selects k options, possibly with some blank ones. The voter hence has to send k voting codes and check k return codes, even if they vote for less than k actual candidates.

Several questions can easily be supported using different code voting names. For example, the voting sheet provided in Fig. 1 corresponds to an election with two questions, of 3 voting options each. How to adapt our protocol to cover more complex electoral rules, where a ballot may be further constrained (e.g., when at least one non-blank candidate needs to be selected) is left as future work.

Tally Phase. At the end of the voting phase, each control component has a list of recorded votes $L_{Recorded}$ which need to be tallied. Ideally, all control components should have the same $L_{Recorded}$ in their local state. However, dishonest control components might have added, modified, or dropped entries in their local $L_{Recorded}$, hence the control components need to agree on their list $L_{Recorded}$ of encrypted votes.

Agreement Procedure. To establish L_{Agreed} , each control component CC_i sends $L_{Recorded}$ with the “proof” for each entry (that is, ac and signatures over (Id, e)) to all other control components CC_j . Each control component then defines L_{Agreed} to be $L_{Recorded}$, in which they add any e such that

- they received a valid signature $\text{sign}(sk_j, (Id, e))$ from each CC_j ,
- h is associated to Id and $h = \text{hash}(ac)$.

After this step, all control components agree, or they can detect who is misbehaving.

Tally Procedure. After establishing L_{Agreed} , the decryption authorities execute the tally procedure with the ciphertext given by L_{Agreed} as input. We leave this step abstract as our protocol can support both a homomorphic tally [1] or a mixnet-based tally [18]. In both cases, the decryption authorities provide verifiable proof of correct tally.

3.4 Security Claims

Our protocol satisfies all the security requirements of the Swiss Chancellery, even if the voting device is dishonest. More precisely, our protocol guarantees vote privacy, even when the attacker controls the voting device, all but one control component, and less than t decryption authorities. Our protocol also ensures verifiability, even when the attacker controls the voting device and all but one control component. In particular, it guarantees cast-as-intended and vote privacy against a dishonest voting device.

3.5 Design Rationale

In this section, we give an intuitive explanation why our protocol guarantees the verifiability and privacy in our trust model. We provide formal proofs in Sect. 4.

Verifiability. A correct return code rc in the first roundtrip guarantees that the voting code c cast by the voter Id has been received by all control components. rc can only be reconstructed if all control components publish their rc_i . The honest control component will only publish the corresponding rc_i if c is indeed the first vote of that voter, and it has received signatures over (Id, e) for $e = te[c]$ of all other control components. We assume the adversary unable to guess rc_i or forge signatures.

A correct finalization code fc in the second roundtrip guarantees that only votes are tallied for which the first roundtrip was successful. fc can only be reconstructed if all control components publish their fc_i . The honest control component will only publish fc_i if it has seen ac , which is in turn only published by the voter when rc of the previous roundtrip was correct. We assume the adversary unable to guess fc_i or ac .

Finally, for the vote e to enter the tally, there are two conditions: there must exist a signature over (Id, e) from all control components, and ac must be known. If the voter has seen a correct finalization code, the honest control component is in possession of these values, therefore guarantees inclusion of e in the tally. Furthermore, the fact that ac is known implies that rc was valid, which in turn implies that the corresponding c has been received by all control components. The verifiable tally procedure then ensures the vote is not modified during the tally.

Privacy. We first collect which values learned by the adversary depend on the selected voting option. When the voter has chosen their plain vote v , they then map it to the voting code $c := \sigma[v]$, hence c clearly depends on v . The other values that depend on v are the corresponding partial return codes $rc_i := t_i[c]$, and the encrypted vote $e := te[c]$. All the other values, including those from other voters, are independent (i.e. remain the same if the voter would chose a different v').

The adversary cannot relate c to its corresponding v , as it does not know σ , and cannot reconstruct π (as π_i of the honest control component is unknown to the adversary). The adversary cannot learn additional information from rc_i , as they were randomly chosen. Further, the adversary is unable to decrypt e as at least one decryption authority is assumed honest. The privacy-preserving tally procedure then ensures that the adversary does not learn anything about v , apart from the result of the election.

4 Security Analysis

In order to study the security of our protocol, we conducted a formal security analysis using the automatic prover ProVerif.

4.1 ProVerif in a Nutshell

ProVerif [5,6] is an automatic tool designed to conduct security analyses of cryptographic protocols. Protocols are modeled in a symbolic model: messages

are abstracted with terms modeling an idealized cryptography. For instance, as soon as the attacker does not know the decryption key, a ciphertext is perfectly hiding its plain message. Protocol roles are modeled with processes that define how messages are exchanged between the participants and which checks are done upon receiving a message. Processes are made of commands that describe the different actions of a participant: $\text{in}(c, x)$ models the input of a message x on a channel c , $\text{out}(c, u)$ the output of message u on channel c , $\text{if } b \text{ then } P \text{ else } Q$ models a test. Other commands exist to model protocol states, parallel execution or replication of roles, fresh term generation, and more. Finally, ProVerif models a Dolev-Yao attacker controlling the network: the attacker learns all messages exchanged between participants on public channels and can modify them, block them, or forge new ones. This attacker usually over-approximates the behavior of a concrete attacker. It can also behave as a dishonest participant of the protocol as soon as it is given the corresponding secret material.

Regarding security properties, ProVerif supports both reachability and equivalence properties. In the context of electronic voting, reachability properties are used in particular to model verifiability: for all executions, the outcome of the election corresponds to the intended votes. Equivalence properties model that an attacker cannot distinguish between two different scenarios. This is typically used to model vote privacy.

Other tools, such as Tamarin [35] or CPSA [32], have also been developed for the analysis of security protocols. We chose ProVerif because it has already been successfully applied to voting protocols in the Swiss setting [40] and because it offers a framework [13] for proving end-to-end verifiability. Unfortunately, such a framework does not exist to prove vote privacy. Hence, we defined our own model, combining the protocol roles previously defined in the verifiability framework.

4.2 Modeling the Protocol and Security Properties

Protocol. The first step to prove the security of the protocol is to describe its different roles (voter, setup component and control component). Because the voting device is always assumed dishonest in the scenarios we want to consider, we do not model it. Instead, this role will be under the control of the implicit attacker of the symbolic model. In the same way, we model only one control component, the others being assumed dishonest, too. Moreover, for the sake of simplicity, we model a setup component that interacts with only one dishonest control component, which mimics the behavior of all the dishonest ones.

We provide an excerpt of the voter role in Fig. 3. We model the voting sheet as a table filled by the setup component, which the voter can read from using `get` in order to retrieve the appropriate voting code. Then, the voter sends the voting code together with their id. The voter expects back a return code, and if this return code is as expected, the voter sends the authentication code. The voter again expects back a finalization code, and again checks if this finalization code is as expected. After voting and after verification, the voter raises events which we use in the queries of our verifiability proof.

```

1      (* voter who casts and verifies *)
2      let Voter(id: id_t, voting_choice: voting_choice_t) =
3
4          (* read out voting material *)
5          get VoteState(=id, =voting_choice, vote_code, return_code) in
6          get AuthenticateState(=id, authentication_code, finalization_code) in
7
8          (* cast the vote and raise Voted event *)
9          out(c, (id, vote_code));
10         event Voted(id, voting_choice);
11
12         (* if the return code matches, then authenticate the vote *)
13         in(c, return_code': xor_t);
14         if return_code' = return_code then (
15             out(c, authentication_code);
16
17             (* if the finalization code matches, then raise Verified event *)
18             in(c, finalization_code': xor_t);
19             if finalization_code' = finalization_code then (
20                 event Verified(id, voting_choice)
21             )
22         )
23     .

```

Fig. 3. The voters' process.

Verifiability. The framework developed by Cheval et al. in [13] aims at proving end-to-end verifiability: an electronic voting protocol satisfies end-to-end verifiability if for all execution traces,

$$result = V_{HV} \uplus V_{HNV} \uplus D$$

where *result* is the multiset of tallied votes, V_{HV} is the multiset of votes of honest voters who verify, V_{HNV} is a sub-multiset of the votes cast by honest voters who do not verify, and D contains at most one vote per dishonest voter.

Instantiating this definition to return code protocols, we say that a voter has voted if they have cast their voting code and received the expected return code. Moreover, we say that a voter has verified if they have sent the confirmation code and received the expected finalization code. Informally, end-to-end verifiability guarantees that the attacker cannot modify the plain vote cast by the voter, nor confirm a vote on their behalf. Moreover, the attacker cannot stuff the ballot box by impersonating voters or casting multiple ballots in their name.

Vote Privacy. In [28], Kremer et al. define vote privacy as an equivalence property: an electronic voting protocol satisfies vote privacy if an attacker is not able to distinguish whether Alice votes for 0 and Bob for 1, or conversely, i.e.

$$\text{Voter}(\text{alice}, 0) \mid \text{Voter}(\text{bob}, 1) \mid P \approx \text{Voter}(\text{alice}, 1) \mid \text{Voter}(\text{bob}, 0) \mid P$$

where P denotes the roles of the protocol other than Alice's and Bob's, such as the setup component or the control component. For the sake of generality, we prove vote privacy assuming that P models an arbitrary number of dishonest voters.

Modeling Choices. Even if our modeling is close to the protocol description provided in Sect. 3, the ProVerif model differs in a few aspects.

Exclusive-OR and Sum. ProVerif does not support the exclusive-or operator. It does not support addition between variables either¹ Therefore, we had to abstract these two operators to make the analysis possible. We chose to abstract it through a unique uninterpreted function of symbol `xor_combine(·, ·)` that combines two shares into a unique element. We then provide to the attacker some minimal deduction capabilities, common to both exclusive-OR and sum, using the two following reduction rules:

$$\begin{aligned} \text{getL}(\text{xor_combine}(x, y), y) &= x \\ \text{getR}(\text{xor_combine}(x, y), x) &= y. \end{aligned}$$

Unfortunately, this modeling does not reflect the algebraic properties of the operators. Going beyond this limitation in ProVerif is still an open problem and out of scope of this paper. Despite this limitation, our analysis provides a substantial level of confidence in the security of the protocol.

Permutations. In the protocol, permutations are used to keep the voter’s choice private. Permutations are encountered in other contexts, such as mixnets, but unfortunately, no faithful model exists at the moment for mixnets, they are instead abstracted away in existing analysis (see e.g., [4, 15]). Hence we cannot rely on these approaches. In our protocol, we use the fact that permutations are independent from a voter to another. They can therefore be soundly abstracted as follows to prove vote privacy: first, do not apply any permutation for voting material associated to dishonest voters. Then, when forging Alice’s and Bob’s voting material, apply the following permutation: assuming that the permutation σ is applied to the plain votes 0 and 1 on the left side of the equivalence, apply the permutation $\sigma' = \{0 \mapsto 1, 1 \mapsto 0\} \circ \sigma$ on the right side. If σ is a randomly sampled permutation, then σ' is a randomly sampled permutation too. We can thus soundly apply the first transformation and abstract away the exact permutation σ . It makes ProVerif succeed in proving equivalence in the scenarios we consider.

4.3 Results

The ProVerif tool confirmed the expected security of the protocol: both privacy and verifiability hold as soon as the setup component and one control component are trusted. The analysis took about 224s for verifiability and about 11s for vote privacy. The corresponding ProVerif files are available at [14]. Experiments have been conducted on a standard laptop (MacBook Pro - M2 Pro - 32GB RAM) using the ProVerif development branch `improved_scope_lemma`² as requested by the framework used to prove verifiability [13].

¹ Since v2.01, ProVerif supports natural numbers. However, the operator $+$ can only be used between a variable and a constant or two constants, which is too restrictive to model our protocol.

² https://gitlab.inria.fr/bblanche/proverif/-/tree/improved_scope_lemma, with the most recent commit at the time of writing being a7ff60b1.

5 Conclusion

We have provided a simple voting protocol that satisfies both verifiability and privacy, even against a dishonest device. Its simplicity makes it easy to implement, with no scalability issue related to cryptographic computations. Our proposal also demonstrates that the Swiss requirements can be fulfilled with a minimal protocol, providing some conceptual answer on the “minimal” building blocks needed in this setting. It also opens the discussion on which other properties are also needed. For example, we noticed that the SwissPost protocol still guarantees privacy when only the setup component and one auditor are dishonest, and all other components, including the communication channels and the voting server, are honest. This is due to the fact that the voting device encrypts the vote. In contrast, our code-voting protocol does not offer vote privacy against an honest-but-curious setup component that has access to the ballot box. This highlights the fact that the Swiss Chancellor, in charge of defining the required properties, may want to further strengthen its requirements. For example, it may want to require that an honest-but-curious setup component may not, alone, break vote privacy. On the other hand, this will make the design of code voting schemes much more complex. Hence, on the contrary, another desirable requirement could be vote privacy against a voting device.

References

1. Adida, B.: Helios: web-based open-audit voting. In: USENIX (2008)
2. Benaloh, J.: Simple verifiable elections. EVT (2006)
3. Benaloh, J., Naehrig, M.: ElectionGuard. Design Specification Version 2.0.0. Technical report, Microsoft Research (2023). https://github.com/microsoft/electionguard/releases/download/v2.0/EG_Spec_2_0.pdf
4. Bernhard, D., Cortier, V., Gaudry, P., Turuani, M., Warinschi, B.: Verifiability Analysis of CHVote. Cryptology ePrint Archive (2018). <https://eprint.iacr.org/2018/1052>
5. Blanchet, B.: An efficient cryptographic protocol verifier based on prolog rules. In: Computer Security Foundations Workshop (CSFW) (2001)
6. Blanchet, B., Cheval, V., Cortier, V.: ProVerif with lemmas, induction, fast subsumption, and much more. In: Symposium on Security and Privacy (SP) (2022)
7. Boneh, D., Lynn, B., Shacham, H.: Short signatures from the weil pairing. In: Advances in Cryptology (2001)
8. Brightwell, I., Cucurull, J., Galindo, D., Guasch, S.: An Overview of the iVote 2015 Voting System. New South Wales Electoral Commission, Australia (2015). <https://elections.nsw.gov.au/getmedia/4279ab0e-5db0-451e-9e3d-87d81ed82d9c/overview-of-the-ivote-2015-voting-system.pdf>
9. Budurushi, J., Neumann, S., Olembo, M.M., Volkamer, M.: Pretty understandable democracy - a secure and understandable internet voting scheme. In: International Conference on Availability, Reliability and Security (ARES) (2013)
10. Chaum, D.: Sure vote: technical overview. In: Workshop on trustworthy elections (WOTE) (2001). <http://www.vote.caltech.edu/wote01/pdfs/surevote.pdf>

11. Chondros, N., et al.: D-DEMOS: a distributed, end-to-end verifiable, internet voting system. In: International Conference on Distributed Computing Systems (ICDCS) (2016)
12. Clarkson, M.R., Chong, S., Myers, A.C.: Civitas: Toward a secure voting system. In: Symposium on Security and Privacy (SP) (2008)
13. Cortier, V., Debant, A., Cheval, V.: Election verifiability with ProVerif. In: Computer Security Foundations Symposium (CSF) (2023)
14. Cortier, V., Debant, A., Moser, F.: ProVerif proof of Code voting: when simplicity meets security (2024). https://inria.hal.science/hal-04627733v1/file/proof_proverif.zip
15. Cortier, V., Filipiak, A., Lallemand, J.: BeleniosVS: secrecy and verifiability against a corrupted voting device. In: Computer Security Foundations Symposium (CSF) (2019)
16. Cortier, V., Gaudry, P., Glondy, S.: Belenios: a simple private and verifiable electronic voting system. Foundations of Security, Protocols, and Equational Reasoning: Essays Dedicated to Catherine A. Meadows (2019)
17. Debant, A., Hirschi, L.: Reversing, breaking, and fixing the French legislative election e-voting protocol. In: USENIX Security Symposium (2023)
18. Wikström, D.: Open Verificatum. <https://www.verificatum.org/>
19. Federal Chancellery: Federal Chancellery Ordinance on Electronic Voting (2013). <https://www.fedlex.admin.ch/eli/cc/2022/336/en>
20. Haenni, R., Koenig, R.E., Dubuis, E.: Private internet voting on untrusted voting devices. In: International Conference on Financial Cryptography and Data Security (2023)
21. Haenni, R., Koenig, R.E., Locher, P., Dubuis, E.: CHVote Protocol Specification. Cryptology ePrint Archive (2017). <https://eprint.iacr.org/2017/325.pdf>
22. Heiberg, S., Willemson, J.: Verifiable internet voting in Estonia. In: International Conference on Electronic Voting (EVOTE) (2014)
23. Hirschi, L., Schmid, L., Basin, D.A.: Fixing the achilles heel of e-voting: the bulletin board. In: Computer Security Foundations Symposium (CSF) (2021)
24. Joaquim, R., Ferreira, P., Ribeiro, C.: EVIV: an end-to-end verifiable Internet voting system. Comput. Secur. **32**, 170–191 (2013)
25. Joaquim, R., Ribeiro, C., Ferreira, P.: Veryvote: a voter verifiable code voting system. In: International Conference on E-Voting and Identity (VOTE-ID) (2009)
26. Kanton Zürich: National- und Ständeratswahlen. <https://www.zh.ch/de/politik-staat/wahlen-abstimmungen/national-staenderatswahlen.html>
27. Kanton Zürich: Abstimmungsvorlagen vom 25. September 2022 (2022). <https://www.zh.ch/de/news-uebersicht/medienmitteilungen/2022/06/abstimmungsvorlagen-vom-25-september-2022.html>
28. Kremer, S., Ryan, M.: Analysis of an electronic voting protocol in the applied pi calculus. In: European Symposium on Programming (ESOP) (2005)
29. Kremer, S., Ryan, M.D., Smyth, B.: Election verifiability in electronic voting protocols. In: Symposium on Research in Computer Security (2010)
30. Kulyk, O., Neumann, S., Budurushi, J., Volkamer, M.: Nothing comes for free: how much usability can you sacrifice for security? IEEE Secur. Priv. **15**(3), 24–29 (2017)
31. Küsters, R., Truderung, T., Vogt, A.: Clash attacks on the verifiability of e-voting systems. In: Symposium on Security and Privacy (SP) (2012)
32. Liskov, M.D., Ramsdell, J.D., Guttman, J.D., Rowe, P.D.: The Cryptographic Protocol Shapes Analyzer: A Manual. The MITRE Corporation (2016)

33. Marky, K., Kulyk, O., Renaud, K., Volkamer, M.: What did i really vote for? On the usability of verifiable e-voting schemes. In: Conference on Human Factors in Computing Systems (CHI) (2018)
34. Marky, K., Schmitz, M., Lange, F., Mühlhäuser, M.: Usability of code voting modalities. In: Conference on Human Factors in Computing Systems, CHI 2019. ACM (2019)
35. Meier, S., Schmidt, B., Cremers, C., Basin, D.: The tamarin prover for the symbolic analysis of security protocols. In: International Conference on Computer Aided Verification (CAV) (2013)
36. Müller, J., Truderung, T.: A protocol for cast-as-intended verifiability with a second device. In: International Joint Conference on Electronic Voting (EVOTE) (2023)
37. Ryan, P.Y., Teague, V.: Pretty good democracy. In: International Workshop on Security Protocols (2009)
38. Stadt Zürich: 25. September 2022: fünf kommunale Sachvorlagen (2022). https://www.stadt-zuerich.ch/portal/de/index/politik_u_recht/abstimmungen_u_wahlen/archiv_abstimmungen/vergangene_termine/220925.html
39. Swiss Post: Swiss Post Voting System: System Specification. Version 1.1.1. Technical report, Swiss Post (2022). https://gitlab.com/swisspost-evoting/e-voting/e-voting-documentation/-/blob/afbd7555b3f87092596a5203f022c00dcc0fd390/System/System_Specification.pdf
40. Swiss Post: Symbolic Analysis of the Swiss Post Voting System. Version 1.6.0.0 (2024). <https://gitlab.com/swisspost-evoting/e-voting/e-voting-documentation/-/tree/20d8d707/Symbolic-models>
41. Volkamer, M., Kulyk, O., Ludwig, J., Fuhrberg, N.: Increasing security without decreasing usability: a comparison of various verifiable voting systems. In: Symposium on Usable Privacy and Security (SOUPS) (2022)



Speedy Privacy-Preserving Skyline Queries on Outsourced Data

Yu Chen¹, Lin Liu^{1(✉)}, Rongmao Chen^{1(✉)}, Shaojing Fu¹, Yuexiang Yang¹,
Jiangyong Shi¹, and Liangzhong He²

¹ National University of Defense Technology, Changsha 410073, Hunan, China
{yuchen,liulin16,chromao,fushaojing,yyx,shijiangyong}@nudt.edu.cn

² China Mobile (Suzhou) Software Technology Co., Ltd., Suzhou, China
heliangzhong@cmss.chinamobile.com

Abstract. Supporting efficient and secure skyline query services on outsourced data remains an ongoing challenge, given the fact that existing solutions either incur significant overhead due to the underlying algorithm traversing the entire dataset or raise privacy issues when specific data structures are introduced. Motivated by addressing these limitations, this work proposes the Speedy and Privacy-Preserving Skyline Query scheme (SPSQ) to support efficient skyline queries that preserve the privacy requirements of datasets, skylines, queries, and indirect privacy with low online computational costs. SPSQ is an optimized distributed skyline diagram constructed by a novel privacy-preserving set intersection with the deduplication algorithm. Moreover, it converts the skyline queries into a lightweight polynomial-based private information retrieval for diagrams to reduce user-perceived latency. The quadrant skyline is extracted directly while the retrieval scale of the dynamic skyline is rapidly reduced by around 100×. We also carefully analyze its security and conduct experimental evaluations on different datasets. The results show that our SPSQ is more efficient than current works, and it is at least an order of magnitude faster in computational complexity than the state-of-the-art with the same security level.

Keywords: secure skyline query · distributed skyline diagram · private information retrieval · privacy protection

1 Introduction

Outsourcing data to cloud servers is gaining popularity as it allows data owners to use tremendous computation capabilities and storage resources. However, the cloud servers might not be fully trusted [5]. Hence, it has been widely considered

This work was supported in part by the National Natural Science Foundation of China (Grant No. 62102430, No. 62122092, No. 62032005, No. 62072466), Science Research Plan Program by NUDT (Grant No. ZK22-50), and Education Ministry-China Mobile Research Funding under Grant No. MCM20170404.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024
J. Garcia-Alfaro et al. (Eds.): ESORICS 2024, LNCS 14983, pp. 430–450, 2024.
https://doi.org/10.1007/978-3-031-70890-9_22

practically essential to preserve privacy when one uses cloud services for data storage, data analysis, online query services, etc.

In this work, we focus on one of the most prevailing query-based services, namely *skyline queries* on outsourced data, which is helpful for many applications, e.g., multi-attribute similarity retrieval and multi-objective decision [17]. Defining the exact scoring preferences for K-Nearest Neighbor (KNN) queries is complex [15]. To this end, skyline queries act as a filter to obtain all 1NN records by involving all possible relative preferences. Roughly, the skyline consists of all the points not dominated by any other point concerning the query point, which always is the clear favorite, regardless of how the preferences of each attribute are weighed [1]. However, an untrusted environment reveals the following privacy.

- *Direct privacy.* Datasets are valuable assets subject to legal outsourcing restrictions and confidential to others. Users are sensitive to directly disclosing their query data and results, which usually contain personal attributes [4].
- *Indirect privacy.* Protecting access patterns and skyline dominance relationships that may indirectly reveal information is also important [14].

Table 1. Summary of existing privacy-preserving dynamic skyline query schemes

Works	DP	IP	KEF	Method	Query Process	Accuracy	CC
Liu [14]	✓	✓	✗	Paillier	Direct search	Lossless	$\mathcal{O}(dn\theta + \ln\theta)$
Ding [7]	✓	✗	✓	HRES	Partial pruning	Lossless	$\mathcal{O}(2dn(1 - \frac{1}{2^\theta}))$
Wang [24]	✓	✗	✗	ORE+AES	Direct search	Lossless	$\mathcal{O}(n^2)$
Zeighami [26]	✓	✗	✗	OPE	Voronoi diagram	Lossy	$\mathcal{O}(d^2 \lceil \frac{n}{l_r} \rceil \log n)$
Wang [25]	✓	✗	✗	ORE+Paillier	R-tree	Lossy	$\mathcal{O}(3dn)$
Zheng [30]	✓	✓	✓	Secret Sharing	Direct search	Lossless	$\mathcal{O}(dn(\theta + 1))$
Ours	✓	✓	✓	Secret Sharing	Skyline diagram	Lossless	$\mathcal{O}(dn + d\bar{n}\theta)$

^a DP/IP, KEF, CC, HRES, AES, ORE/OPE: Direct/Indirect Privacy, Key Escrow Freeness, Computational Complexity, Homomorphic Re-Encryption Scheme, Order-Revealing/Preserving Encryption, Advanced Encryption Standard.

^b $d, n, \bar{n}, \theta, l_r, l$: number of dimensions, points, global skyline, dynamic skyline, intersect regions, and bits. Notice that $n \gg \bar{n}$.

As Table 1 shows, various works have aimed to protect the above privacy issues. Unfortunately, we found that existing solutions either *incur significant overhead* due to the underlying algorithm traversing the entire dataset or *raise privacy issues* by introducing specific data structures to reduce overhead. Order-preserving/revealing encryption (OPE/ORE) schemes have been applied to various queries over encrypted databases. However, ORE leaks the order relationships on each dimension and fails to hide access patterns, which may incur inference attacks [24]. Zeighami et al. used result materialization for dynamic skyline queries by OPE and ORE [26]. This scheme requires index negotiation or incurs

a small measurable leakage when comparing two values, and the approximate result is lossy. Liu et al. first proposed a secure skyline query protocol (FSSP) on the encrypted data [14]. However, one server has the encrypted data, and the other holds the private key. This distribution will inevitably bring huge privacy risks, e.g., key escrow freeness, where a single point of disclosure occurs, and the capabilities of the two servers differ. Later, Ding et al. improved the efficiency of FSSP by constructing a noise matrix, which aims to hide the dominated position [7]. However, this scheme still leaks some indirect privacy since its data assistance provider knows the number of dominated tuples with fewer iterations due to plaintext zeros on the matrix. In particular, Wang et al. focused on the location-based skyline with two spatial attributes [25]. Like the FSSP, however, one server is authorized with the private key. Moreover, its providers decrypt and know the privacy of dominance when calling the location-based secure dominance algorithm, and the query result will reveal data privacy due to ORE and path selection. Furthermore, Zheng et al. combined the strengths of the previous works and applied them to secret sharing [30], but their preprocessing for multiplicative triples is relatively impractical due to the increasing unfixed parameters. Some schemes fail to protect direct privacy. Zheng et al. mainly focused on the secure data merging technique and did not consider the privacy and access patterns in computing the skyline [29]. Qaosar et al. ensured that each party computes the local dynamic skylines in advance, however, the query tuple is exposed to all participating parties [20]. Zhang et al. focused on the constrained and interval skyline query rather than the generic skyline [27, 28]. However, it fails to protect the access patterns and requires a fresh key per user [6, 11]. Moreover, skylines can be queried in trusted execution environments [23], but this is orthogonal to the outsourced data. Most schemes require traversing the dataset with high computational complexity [15], even with pruning operations [7].

To address these limitations, this work aims to design an efficient approach that significantly reduces user-perceived online latency while protecting all privacy. Our work is initially inspired by the so-called *skyline diagram* from [15]. Such a method uses space partitioning to generate skyline diagrams offline and then extracts the accurate skyline through query points. Below, we briefly describe how skyline diagrams work through an example and then explain why achieving privacy requirements under skyline diagrams is challenging.

Skyline Diagram. As depicted in Fig. 1, there are eight points with two dimensions: age and blood pressure (BP), and the user with index $q = (45, 80)$ wishes to find close records, which encompasses four quadrants. If the first quadrant skyline (Definition 2) is considered, i.e. the older people with higher blood pressure, the result is the points p_6 and p_8 . The deviating point p_8 is also valuable as a filtering feature in skyline queries that is not available in the 1NN. All quadrant skylines in each cell are computed offline through the diagram, and the points are extracted from the dynamic query cell.

At first glance, one could trivially extend all processes in diagram queries by using the existing secure (comparison and intersection) protocols. Unfortunately,

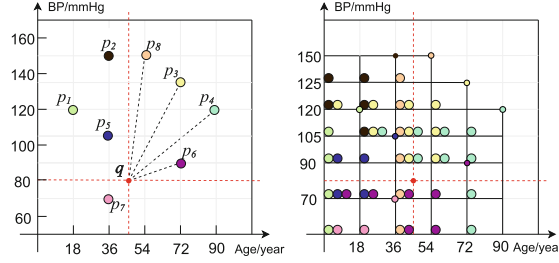


Fig. 1. Sample for skyline query in diagram

as explained below, achieving secure diagram generation and skyline extraction efficiently under outsourced data turns out to be non-trivial.

Challenge 1: *How to compute a protected diagram for secure skyline queries while guaranteeing data confidentiality?* The generation of skyline diagrams relies on set operations involving duplicate elements, the current methods often face challenges in handling duplicated results or inadvertently disclosing dominance relationships under skyline change. Considering the trivial union operations, we propose a novel skyline point generation that realizes a secure set intersection combined with deduplication, ultimately enabling blind skyline generation.

Challenge 2: *How to safely and efficiently extract skylines from many non-public diagram cells?* The position of the extracted cell involves an access pattern, however, performing secure secret shared multiplications naively on all cells is inefficient. To avoid the expensive cost of its distributed garbled circuits [3], we propose a newly developed secure skyline extraction merely using random polynomials and hash function with database-copies idea, where the position and skyline are hidden so that the query unlinkability is also preserved.

Challenge 3: *How to further accelerate the efficiency of diagram generation?* Observe that the dominance relationships are transitive, each independent region can be computed separately. We devise an optimized construction by considering the border cell of adjacent regions, which dominate all quadrant points so that the dominated points are pruned without new costly comparison operations.

Putting all the challenges together, we design a privacy-preserving skyline query scheme based on an optimized distributed skyline diagram, significantly reducing the search size. Our contributions are summarized as follows:

1. We propose a novel privacy-preserving skyline query scheme based on the enhanced distributed skyline diagram. It preserves the privacy of datasets, skylines, queries, dominance relationships, and access patterns. It also supports both offline data owners and real-time queries for multiple users.
2. We propose a secure skyline point generation method consisting of privacy-preserving set intersection with deduplication since the servers could not directly handle the mixed skyline set on the distributed skyline diagram. We also propose a privacy-preserving skyline retrieval method using random

polynomials and hash functions to reduce search complexity. Some processes can be pre-computed and parallelized to improve efficiency further.

3. We evaluate the experiments on the equivalent dataset. It shows that, with the same security, our scheme improves computational complexity by at least an order of magnitude over the state-of-the-art approach.

The rest is organized as follows. Section 2 gives some primitives and models. Section 3 presents the concrete scheme. We provide the security analysis and performance evaluation in Sects. 4 and 5. Finally, we conclude this paper.

Table 2. Main Notations

Notations	Descriptions
$\mathcal{P}, d$	dataset of n points $\{p_1, \dots, p_n\}$, number of dimensions
$p_i[k], q[k]$	the k^{th} value of data point p_i and query point q
$n, \bar{n}, \theta$	number of points, global skyline and dynamic skyline
n_1, n_2	length of each coordinate in skyline diagram
$\mathbf{G}_{k,j_k}$	j_k^{th} coordinate of k^{th} dimension in diagram
$\mathbf{C}_{j_1,j_2}, \mathbf{S}_{j_1,j_2}$	cell with bottom left corner $(\mathbf{G}_{1,j_1}, \mathbf{G}_{2,j_2})$, its skyline
$\mathbf{P}_{j_1,j_2}, \mathbf{I}_{j_1,j_2}, \mathbf{IV}_{j_1,j_2}$	point, point indicator and index vector at coordinate $(\mathbf{G}_{1,j_1}, \mathbf{G}_{2,j_2})$
$\langle \cdot \rangle, \langle \cdot \rangle^C, \langle \cdot \rangle^E$	shares of elements, the specific share of party CS and ES
π_c, π_e	randomly sampled elementary matrix
$f_t(\cdot), h_t(\cdot)$	t -degree polynomial with coefficients $\mathbf{a}$ or $\mathbf{b}$
$\mathbf{u}_k, \eta_k, \mathbf{m}_k$	query indicator, retrieval position and mask at k^{th} coordinate

2 Preliminaries and Models

For references, the main notations are given in Table 2.

2.1 Preliminaries

Additive Secret Sharing (ASS). We adopt 2-out-of-2 additive secret sharing scheme over the rings $\mathbb{Z}_{2^l}$ that the randomly sampled shares $\langle x \rangle^C$ and $\langle x \rangle^E$ satisfy $\langle x \rangle^C + \langle x \rangle^E = x \pmod{2^l}$. For simplicity, we omit modulus 2^l . We mainly leverage four underlying protocols where all instances of multiplication triples are temporarily generated by oblivious transfer during protocol execution [21].

Secure Sharing Comparison. Two parties hold shared elements $\langle x \rangle$ and $\langle y \rangle$, and they compute shared output $\langle x > y \rangle$ by invoking **SSC**(x, y).

Secure Sharing Equal. Two parties hold shared elements $\langle x \rangle$ and $\langle y \rangle$, and they compute shared output $\langle x = y \rangle$ by **SSE**(x, y).

Algorithm 1: Skyline Diagram

Input: a set of n points and skyline cells C_{j_1, j_2}

Output: skyline S_{j_1, j_2}

```

1 for  $j_1 = n_1 - 1, \dots, 0$  do
2   for  $j_2 = n_2 - 1, \dots, 0$  do
3     if a point  $p_i$  is on the upper right corner of  $C_{j_1, j_2}$  then
4        $S_{j_1, j_2} = \{p_i\}$ . // condition 1
5     else
6        $S_{j_1, j_2} = S_{j_1+1, j_2} + S_{j_1, j_2+1} - S_{j_1+1, j_2+1}$ . // condition 2

```

Secure Sharing Hadamard Product. Two parties hold shared matrices $\langle X \rangle$ and $\langle Y \rangle$, and they compute the shared element-wise product $\langle X \circ Y \rangle$ by **SSH**(X, Y).

Secure Sharing Matrix Multiplication. Two parties hold shared matrices $\langle X \rangle$ and $\langle Y \rangle$, and they compute the shared matrix product $\langle X \cdot Y \rangle$ by **SSM**(X, Y).

Skyline Query. We adopt the related definitions and their properties from [15].

Definition 1 (*Dynamic Skyline Query*). Given a dataset $\mathcal{P}$ of n points and a query point q in d -dimensional space. Let p_1, p_2 are two different points, p_1 dominates p_2 with regard to q , denoted by $p_1 \prec_q p_2$, if $\forall k \in [1, d], |p_1[k] - q[k]| \leq |p_2[k] - q[k]|$ and $|p_1[k] - q[k]| < |p_2[k] - q[k]|$ at least one k [14]. The dynamic skylines are those points that are not dominated by any other point.

Definition 2 (*Quadrant Skyline*). The query point q divides the d -dimensional space into 2^d quadrants according to the coordinates in each dimension, forming hyperplanes perpendicular to the coordinate axes. The quadrant skylines are not dominated by any other point with q under the same quadrant [15].

Definition 3 (*Global Skyline*). The global skylines are the union of all quadrant skyline points, encompassing the query results across all quadrants [15].

The dynamic skyline is a subset of the global skyline, which is used for the dynamic skyline query that efficiently filters the global skyline from the diagram and extracts the dynamic skyline, consequently minimizing the search size [15].

Skyline Diagram. The skyline diagram is the subdivision of the plane into a set of polyominoes that any query points in the same polyomino have the same quadrant skyline. As Algorithm 1 shows, for any cell with a point p_i as its upper right corner, its quadrant skyline is $\{p_i\}$ because this point dominates all its upper right regions. For other cells, C_{j_1, j_2} , the quadrant skyline is a set by union of the upper and right regions after the deduplication on the upper right region, i.e., $S_{j_1, j_2} = S_{j_1+1, j_2} + S_{j_1, j_2+1} - S_{j_1+1, j_2+1}$ [15]. As shown in Fig. 1, q is inside the cell $C_{2,1}$ so its (first) quadrant skyline points are p_6 and p_8 .

Discussion. This paper focuses on generic queries under two-dimensional spaces, which have significance in reality, such as longitude and latitude in geography, gene location and expression levels in bioinformatics, and price and demand in economics. Moreover, high dimension is feasible based on multi-set operations [15]. This scheme works well in non-generic query scenarios, such as location, which contain two-dimensional dynamic location and other non-spatial attributes. The diagram is built on location offline, and only filters points with non-spatial attributes in a smaller set when extracting dynamic skyline online.

Floram. Floram transforms data access to the secret index into revealing accesses by maintaining identical masked copies [8]. Given the plaintext database S , where the j -th block is S_j , the masked database S' can be computed as follows.

$$S'_j = S_j \oplus \text{PRF}_{k_1}(j) \oplus \text{PRF}_{k_2}(j). \quad (1)$$

where PRF is a pseudorandom function, P_1 holds k_1 and P_2 holds k_2 respectively. In Floram, two bit-vectors u^1 and u^2 are securely generated such that $u_\eta^1 \oplus u_\eta^2 = 1$, where η is the retrieved index, and the rest corresponds to 0. Each party P_i locally computes $\bigoplus_j u_j^i \times S'_j$, so their XOR is S'_η . Finally, two parties use a garbled circuit to remove the masked PRFs so that S_η is shared secretly.

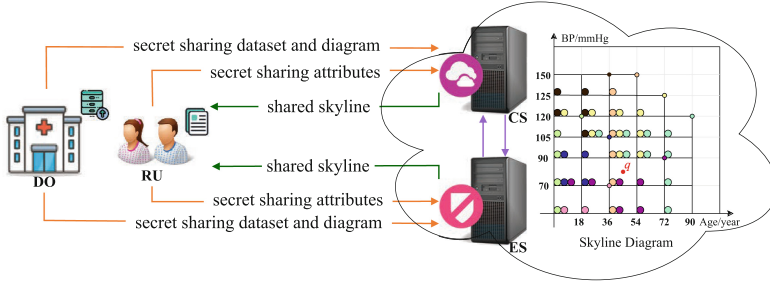


Fig. 2. System model

2.2 System Model

As Fig. 2 shows, our system comprises a Requested User (RU), Data Owner (DO), and two collusion-resistant servers (cloud server, CS and evaluation server, ES). The RU queries the skyline from DO's datasets under two servers.

- The DO shares the dataset and indicators with servers.
- Any RU shares query attributes and expects skyline.
- The CS and ES perform skyline queries over shared data and return the shared results to the RU.

The two-server model is realistic, where their collusion would damage the reputation and revenue [16]. This model is a typical distributed setting used by previous works [7, 25, 30], and can be applied to server clusters under replicated secret sharing or threshold sharing [18, 22].

2.3 Security Model and Goal

All entities are *honest-but-curious*, following honestly the protocol while attempting to gather extra data under curiosity. Specifically, an external adversary can only launch *passive attacks*, such as eavesdropping without data forgeries. Furthermore, the solution enables datasets, queries and results to be commendably protected, and the dominated points associated with queries cannot be reversed.

3 Our Scheme

Two servers jointly generate the distributed skyline diagrams and extract the skyline through the shared index. Moreover, an optimized construction is designed to accelerate the diagram generation further.

3.1 System Initialization

The DO initializes the skyline diagram, mainly sorting data in each dimension and selecting the random numbers under plaintext operations. The offline overhead of the skyline diagram is one-time and supports incremental updates.

1. The DO sorts the k -dimension values $(\{p_1[k], \dots, p_n[k]\})$ in increasing order to form a coordinate vector $\mathbf{G}_k$ of length n_k and shares them to servers based on the random construction of ASS, that is $\langle \mathbf{G}_{k,j_k} \rangle^C + \langle \mathbf{G}_{k,j_k} \rangle^E = \mathbf{G}_{k,j_k}$, where $j_k \in [1, n_k], k \in \{1, 2\}$.
2. The DO generates the point diagram $\mathbf{P}$, divided by the horizontal and vertical line over each point in $\mathcal{P}$. The DO shares all the points of grid intersection in the diagram, that is, $\langle \mathbf{P}_{j_1,j_2} \rangle^C + \langle \mathbf{P}_{j_1,j_2} \rangle^E = \mathbf{P}_{j_1,j_2} = (\mathbf{G}_{1,j_1}, \mathbf{G}_{2,j_2})$.
3. The DO generates the point indicator $\mathbf{I}$. If one point in $\mathcal{P}$ is on the upper right corner of the cell $\mathbf{C}_{j_1,j_2}$, $\mathbf{I}_{j_1,j_2} = 1$. Otherwise, $\mathbf{I}_{j_1,j_2} = 0$. The DO shares them to servers, that is $\langle \mathbf{I}_{j_1,j_2} \rangle^C + \langle \mathbf{I}_{j_1,j_2} \rangle^E = \mathbf{I}_{j_1,j_2}$. It is a matrix.

The RU shares the query point to servers, that is, $\langle q[k] \rangle^C + \langle q[k] \rangle^E = q[k]$.

3.2 Skyline Generation

Based on Algorithm 1, we construct a protected diagram by Skyline Diagram Generation (**SDG**) algorithm with Skyline Point Intersection (**SPI**).

Skyline Point Intersection. As Algorithm 2 shows, the skyline is securely generated through set intersection and deduplication for the skyline set $\mathbf{S}_{j_1,j_2} =$

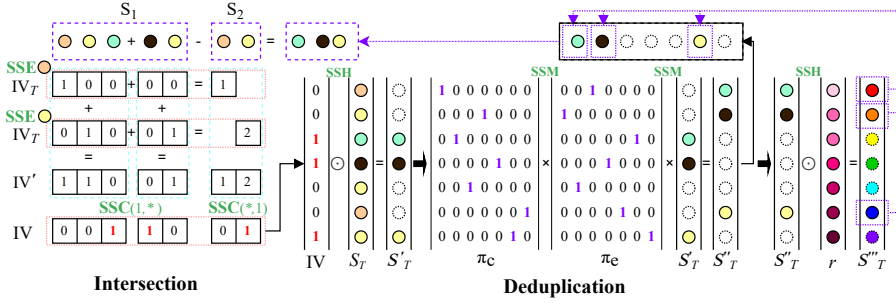


Fig. 3. Skyline Point Intersection

Algorithm 2: Skyline Point Intersection

Input: $\langle S_{j_1+1,j_2} \rangle, \langle S_{j_1,j_2+1} \rangle, \langle S_{j_1+1,j_2+1} \rangle$
Output: $\langle S_{j_1,j_2} \rangle = \langle S_{j_1+1,j_2} + S_{j_1,j_2+1} - S_{j_1+1,j_2+1} \rangle$

- 1 Set $\langle S_1 \rangle = \langle S_{j_1+1,j_2} \rangle \parallel \langle S_{j_1,j_2+1} \rangle, \langle S_2 \rangle = \langle S_{j_1+1,j_2+1} \rangle, \xi_1 = |\langle S_1 \rangle|, \xi_2 = |\langle S_2 \rangle|$
- 2 **for** $\mu_2 = 1, \dots, \xi_2$, *i.e.*, each element in $\langle S_2 \rangle$ **do**
- 3 **for** $\mu_1 = 1, \dots, \xi_1$, *i.e.*, each element in $\langle S_1 \rangle$ **do**
- 4 CS and ES compute: $\langle IV_T \rangle [\mu_1] = \text{SSE}(S_1[\mu_1], S_2[\mu_2])$.
- 5 $\langle IV'_{j_1,j_2} \rangle [\mu_1] = \langle IV'_{j_1,j_2} \rangle [\mu_1] + \langle IV_T \rangle [\mu_1]$.
- 6 $\langle IV'_{j_1,j_2} \rangle [\xi_1 + \mu_2] = \sum_{\mu_1=1}^{\xi_1} \langle IV_T \rangle [\mu_1]$.
- 7 **for** $\mu_1 = 1, \dots, \xi_1$ **do**
- 8 CS and ES compute: $\langle IV_{j_1,j_2} \rangle [\mu_1] = \text{SSC}(1, IV'_{j_1,j_2}[\mu_1])$.
- 9 **for** $\mu_2 = 1, \dots, \xi_2$ **do**
- 10 CS and ES compute: $\langle IV_{j_1,j_2} \rangle [\xi_1 + \mu_2] = \text{SSC}(IV'_{j_1,j_2}[\xi_1 + \mu_2], 1)$.
- 11 CS and ES locally merges $\langle S_T \rangle = \langle S_1 \rangle \parallel \langle S_2 \rangle$, and set $\xi = \xi_1 + \xi_2$.
- 12 CS and ES compute: $\langle S'_T \rangle = \text{SSH}(IV_{j_1,j_2}, S_T)$.
- 13 CS chooses $\pi_c, \langle \pi_c \rangle^C \in_R \mathbb{Z}_N^{\xi \times \xi}$ and $\langle r \rangle^C \in_R \mathbb{Z}_N^\xi$, and shares $\langle \pi_c \rangle^E = \pi_c - \langle \pi_c \rangle^C$.
- 14 ES chooses $\pi_e, \langle \pi_e \rangle^E \in_R \mathbb{Z}_N^{\xi \times \xi}$ and $\langle r \rangle^E \in_R \mathbb{Z}_N^\xi$, and shares $\langle \pi_e \rangle^C = \pi_e - \langle \pi_e \rangle^E$.
- 15 CS and ES compute: $\langle S''_T \rangle = \text{SSM}(\text{SSM}(\pi_c, \pi_e), S'_T)$.
- 16 CS and ES compute: $\langle S'''_T \rangle = \text{SSH}(r, S''_T)$, and reconstruct S'''_T .
- 17 CS and ES locally adds all the $\langle S''_T \rangle [\mu]$ into $\langle S_{j_1,j_2} \rangle$ if $S'''_T[\mu] \neq 0$.

$S_{j_1+1,j_2} + S_{j_1,j_2+1} - S_{j_1+1,j_2+1}$ in three cells $C_{j_1+1,j_2}, C_{j_1,j_2+1}$ and C_{j_1+1,j_2+1} . Referring to Fig. 3, set operations involve duplicate elements. The **Intersection** picks out the indexes of non-intersecting elements by μ_1 and maintains the duplicates by μ_2 . The **Deduplication** blindly removes the 0 elements by the permutation matrices and the random numbers.

1. **Intersection.** If the point $S_1[\mu_1]$ is not equal to any point in S_2 (under ξ_2 iterations), $IV'_{j_1,j_2}[\mu_1] = 0$ since all $IV_T[\mu_1] = 0$ in line 5, and its final index is set to $\langle 1 \rangle$ in line 8. Otherwise, this point is the intersection to be discarded. Note that the point $S_2[\mu_2]$ may appear in both S_{j_1+1,j_2} and S_{j_1,j_2+1} , S_1 will

Algorithm 3: Skyline Diagram Generation**Input:** point indicator $\langle I \rangle$, point diagram $\langle P \rangle$ **Output:** skyline $\langle S_{j_1, j_2} \rangle$

```

1 for  $j_1 = n_1 - 1, \dots, 0$  do
2   for  $j_2 = n_2 - 1, \dots, 0$  do
3      $\langle S'_2 \rangle = \text{SPI}(\mathbf{S}_{j_1+1, j_2}, \mathbf{S}_{j_1, j_2+1}, \mathbf{S}_{j_1+1, j_2+1})$ .
4      $\langle S'_1 \rangle = \langle P_{j_1+1, j_2+1} \rangle || \langle \mathbf{0} \rangle_{|S'_2|-1}$ .
5      $\langle S_{j_1, j_2} \rangle = \text{SSH}(\mathbf{I}_{j_1+1, j_2+1}, \mathbf{S}'_1 - \mathbf{S}'_2) + \langle S'_2 \rangle$ .

```

be removed repeatedly. For this, its index $\text{IV}'_{j_1, j_2}[\xi_1 + \mu_2] \geq 1$ in line 6, and becomes $\langle 1 \rangle$ in line 10. As a result, the index $\langle \text{IV}_{j_1, j_2} \rangle$ links the final skyline.

2. **Deduplication.** The generated skyline $\langle S'_T \rangle$ that contains 0 points requires deduplication. Each server chooses the local elementary matrix $\pi_{c(e)}$ so that each server is unaware of new positions after two independent permutations by **SSM**. Hidden by r , the valid skylines are securely picked from S''_T .

Discussion. Simply applying the existing set intersection would miss duplicate elements and leak the dominance relationship due to the change in set [12]. Moreover, since the private order needs to be synchronized under shared servers, one server permutes the encrypted index while the other decrypts it to return the index of 0 is unavailable [7]. Therefore, we use two independent permutations.

Example 1 : Referring to Fig. 1 and Fig. 3, $S_{2,3} = \{a, b, c\}$, $S_{1,4} = \{d, b\}$ and $S_{2,4} = \{a, b\}$. Then, $S_1 = \{a, b, c, d, b\}$ and the index IV_T is $\{1, 0, 0, 0, 0\}$ (equals a) and $\{0, 1, 0, 0, 1\}$ (equals b). Their sum is $\{1, 1, 0, 0, 1; 1, 2\}$, thus IV is $\{0, 0, 1, 1, 0, 0, 1\}$ after **SSC**. As a result, new skylines are $S'_T = [0, 0, c, d, 0, 0, b] = [0, 0, 1, 1, 0, 0, 1] \circ [a, b, c, d, b, a, b]$. Through π_c and π_e , $S''_T = [c, d, 0, 0, 0, b, 0]$. By blind selection, $S_{1,3} = [c, d, b]$ is equal to the set $S_{2,3} + S_{1,4} - S_{2,4} = \{c, d, b\}$.

Skyline Diagram Generation. Referring to Algorithm 1, for the cell C_{j_1, j_2} , the skyline is a single point if a data point in $\mathcal{P}$ is on cell's upper right corner (where $\mathbf{I}_{j_1+1, j_2+1} = 1$) or an intersection set S_{j_1, j_2} . As Algorithm 3 shows, $|S'_2| - 1$ dummy points ensure that a point and a set are statistically indistinguishable under sharing form, this number is not fixed but dynamically allocated according to SPI results. Since the upper right point's indicator $\mathbf{I}_{j_1+1, j_2+1}$ is 0 or 1, only one value (S'_2 or S'_1) is correctly extracted by **SSH**. The different conditional branches in Algorithm 1 are chosen blindly, point domination is not compromised.

$$\mathbf{S}_{j_1, j_2} = \mathbf{I}_{j_1+1, j_2+1} \times (\mathbf{S}'_1 - \mathbf{S}'_2) + \mathbf{S}'_2 = \mathbf{I}_{j_1+1, j_2+1} \times \mathbf{S}'_1 + (1 - \mathbf{I}_{j_1+1, j_2+1}) \times \mathbf{S}'_2 \quad (2)$$

3.3 Skyline Extraction

We propose a novel and efficient retrieval method to extract the skyline S_{η_1, η_2} from the corresponding query cell (position) C_{η_1, η_2} under many non-public diagram cells without leaking the result privacy and access pattern.

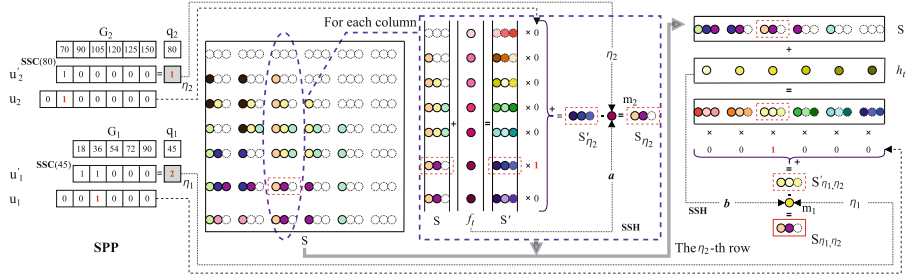


Fig. 4. Skyline Point Extraction

Skyline Point Position (SPP). Compare the query $\langle q \rangle$ with skyline diagram $\langle G_k \rangle$ to determine the position C_{η_1, η_2} . Referring to Fig. 4, the comparison result $u'_k[j_k]$ is obtained by **SSC** for each dimension, and its sum is the position $\langle \eta_k \rangle$. Due to the construction of the vector u'_k , the subtraction of neighboring elements extracts the position of the last 1 such that $\langle u_k[\eta_k] \rangle = \langle 1 \rangle$ while the rest is $\langle 0 \rangle$. The length of u_k is added to 1, i.e., $u[0] = 0$, is used to match cells.

Example 2 : Given the coordinate is $G_1 = [18, 36, 54, 72, 90]$ and the query is $q_1 = 45$, the comparison is $u'_1 = [1, 1, 0, 0, 0]$ by **SSC**($q_1, G_1[k]$), then its sum is $\eta_2 = 2$ where the index is $u_1 = [0, 1 - 1, 1 - 0, 0 - 0, 0 - 0, 0] = [0, 0, 1, 0, 0, 0]$.

Skyline Point Extraction. With the masked copies of Floram, the skyline extraction is transformed into private information retrieval. Instead of expensive distributed PRFs, we use two t -degree polynomials f_t^c and f_t^e under a hash function $H : \{0, 1\}^l \rightarrow \{0, 1\}^l$. By randomly choosing the coefficients $\{a_0^c, \dots, a_t^c\}$ and $\{a_0^e, \dots, a_t^e\}$, CS locally computes $f_t^c(j) = a_0^c + a_1^c \times (j)^1 + \dots + a_t^c \times (j)^t$ while ES locally computes $f_t^e(j) = a_0^e + a_1^e \times (j)^1 + \dots + a_t^e \times (j)^t$. Given the element S_{j_1, j_2} in j_2 -th cell under j_1 -th column, the servers choose the global random number r_{j_2} , then the masked block S'_{j_1, j_2} can be computed as follows.

$$S'_{j_1, j_2} = S_{j_1, j_2} + r_{j_2} + H(f_t^c(j_2)) + H(f_t^e(j_2)), 0 < j_k < n_k, k \in [1, 2] \quad (3)$$

As Algorithm 4 shows, servers construct the diagram copies S'_{j_1, j_2} by the random polynomials and negotiating the hash function. Since only $u_2[\eta_2] = 1$, the servers obtain $S'_{j_1, \eta_2} = \sum_{j_2=0}^{n_2} \langle u_2[j_2] \rangle \times S'_{j_1, j_2}$ for all j_1 , then remove the mask $m_2^c = f_t^c(\eta_2)$ and $m_2^e = f_t^e(\eta_2)$ by **SSH** and the local share of r_{j_1} to obtain $\langle S_{j_1, \eta_2} \rangle$ as line 11, i.e., the η_2 -th row. Likewise, the servers retrieves the skyline S_{η_1, η_2} from the row extraction where the mask $m_1 = h_t(\eta_1)$. Both servers hold the share of quadrant skylines $\langle S_{\eta_1, \eta_2} \rangle$ and return it to the RU in a secure channel.

Example 3 : We describe an example of lines 12 to 17 in Algorithm 4, the servers randomly choose $t = 2, b^c = [1, 2, 3], b^e = [3, 2, 1], r' = 3$, then they get $h_t^c = [1, 6, 17, 34, 57, 86]$ by the polynomial $h_t^c(j_1) = b_0^c + b_1^c j_1 + b_2^c (j_1)^2$ and $h_t^e = [3, 6, 11, 18, 27, 38]$ by $h_t^e(j_1) = b_0^e + b_1^e j_1 + b_2^e (j_1)^2$, where $j_1 \in \{0, \dots, 5\}$. Given the index $u_1 = [0, 0, 1, 0, 0, 0]$, $\eta_1 = 2$, the elements $S_{\eta_2} = [a, b, c, d, e, f]$,

Algorithm 4: Skyline Point Extraction

Input: $\langle S_{j_1, j_2} \rangle, \langle G \rangle, \langle q \rangle$
Output: $\langle S_{\eta_1, \eta_2} \rangle$

- 1 CS and ES jointly compute: $(\langle u_1 \rangle, \langle u_2 \rangle, \langle \eta_1 \rangle, \langle \eta_2 \rangle) = \mathbf{SPP}(G, q)$.
- 2 CS/ES generates the random number $\langle r' \rangle$.
- 3 **for** $j_1 = 0, \dots, n_1$ **do**
- 4 CS and ES locally chooses the random number $\langle r_{j_1} \rangle$.
- 5 **for** $j_2 = 0, \dots, n_2$ **do**
- 6 CS and ES locally generates $f_t^{c(e)}(j_2) = a_0^{c(e)} + \dots + a_t^{c(e)} \times (j_2)^t$.
- 7 CS and ES locally computes $\langle S'_{j_1, j_2} \rangle = \langle S_{j_1, j_2} \rangle + \langle r_{j_1} \rangle + H(f_t^{c(e)}(j_2))$.
- 8 CS and ES jointly reconstructs S'_{j_1, j_2} .
- 9 CS and ES jointly compute $\langle m_2^{c(e)} \rangle$ by **SSH**:

$$\langle m_2^{c(e)} \rangle = a_0^{c(e)} + a_1^{c(e)} \times \langle \eta_2 \rangle + \dots + a_t^{c(e)} \times \langle (\eta_2)^t \rangle$$
- 10 CS sends $\langle m_2^c \rangle$ to ES while ES sends $\langle m_2^e \rangle$ to CS.
- 11 CS and ES reconstructs $m_2^{c(e)}$ and removes the column-mask:

$$\langle S_{j_1, \eta_2} \rangle = \sum_{j_2=0}^{n_2} \langle u_2[j_2] \rangle \times S'_{j_1, j_2} - \langle r_{j_1} \rangle - H(m_2^{c(e)})$$
- 12 CS and ES locally computes: $h_t^{c(e)}(j_1) = b_0^{c(e)} + \dots + b_t^{c(e)} \times (j_1)^t$.
- 13 CS and ES locally computes $\langle S'_{j_1, \eta_2} \rangle = \langle S_{j_1, \eta_2} \rangle + \langle r' \rangle + H(h_t^{c(e)}(j_1))$.
- 14 CS and ES jointly reconstruct S'_{j_1, η_2} .
- 15 CS and ES jointly compute: $\langle m_1^{c(e)} \rangle$ by **SSH**:

$$\langle m_1^{c(e)} \rangle = b_0^{c(e)} + b_1^{c(e)} \times \langle \eta_1 \rangle + \dots + b_t^{c(e)} \times \langle (\eta_1)^t \rangle$$
- 16 CS sends $\langle m_1^c \rangle$ to ES while ES sends $\langle m_1^e \rangle$ to CS.
- 17 CS/ES reconstructs $m_1^{c(e)}$ and removes the row-mask:

$$\langle S_{\eta_1, \eta_2} \rangle = \sum_{j_1=0}^{n_1} \langle u_1[j_1] \rangle \times S'_{j_1, \eta_2} - \langle r' \rangle - H(m_1^{c(e)})$$

then servers compute the copies S'_{η_2} by $S_{\eta_2} + r' + h_t^c + h_t^e = [a + 7, b + 15, c + 31, d + 55, e + 87, f + 127]$ (omit Hash) in line 13 and the mask $m_1^c = h_t^c(\eta_1) = 17$, $m_1^e = h_t^e(\eta_1) = 11$ in line 15. So the servers jointly compute the final skyline $S_{\eta_1, \eta_2} = S'_{\eta_2}[\eta_1] - r' - m_1^c - m_1^e = c + 31 - 3 - 17 - 11 = c$ in line 17.

Discussion. Due to the hash function with random numbers, the masks are not linked to the polynomial, achieving a confusing effect. After all quadrant skylines are aggregated, the dynamic skyline is its subset by secure dominated tuples filtering [10, 30]. Therefore, we safely reduce dynamic query from the size of the dataset, n , to the number of skylines, $\bar{n}$, about $\mathcal{O}(\frac{n}{\bar{n}})$ times acceleration.

3.4 Optimized Construction

To further accelerate the efficiency, we present a Quadrant Skyline Parallel Optimization (**QSPO**) algorithm to partition the skyline diagram that considers the local skyline points of independent regions with skyline additivity.

Local Skyline. The skyline intersections in the diagram are serial in a single dimension and parallel in all dimensions. Therefore, two coordinates can parallel after generating the skyline in each upper right cell, which cuts the time in half.

Inherit Skyline. The left region inherits its right region's border skyline with **SSH**, i.e., $S'_{i,j} = \mathbf{SSH}(1 - \mathbf{I}_{i,j}, S_{i+1,j})$. This property means that if $\mathbf{I}_{i,j} = 0$, the new skyline is added because no other point can dominate it in this region. Otherwise, 0 is added for statistical indistinguishability. It is similar for the bottom region with $S'_{i,j} = \mathbf{SSH}(1 - \mathbf{I}_{i,j}, S_{i,j+1})$. Finally, the border skyline S_{l_1, b_1} in the upper right region is the extra skyline that affects the bottom left region. All additional skylines are inherited without comparison operations.

Discussion. Only the skyline in the adjacent border affects the local region, while these border skylines dominate its quadrant region. Therefore, the number of regions speeds up diagram generation but also increases the number of inherited skylines, affecting extraction performance. It is a custom trade-off.

4 Correctness and Security Analysis

Our SPSQ scheme is a sequential composition of **SDG** and **SPE** protocols.

4.1 Correctness

Theorem 1. *Our scheme is correct under semi-honest and passive attacks.*

Proof. Firstly, the index and coordinates are shared on servers under the same modulus, and the shares recover the original value so that all inputs are correct.

We then propose four main protocols: **SPI**, **SDG**, **SPP**, and **SPE**. Based on **SSE** and **SSC**, the valid points are maintained, and the other points become 0. The order of sequence elements is replaced without changing the values based on **SSH** and **SSM**. In particular, the blind factor may make the product of correct skylines equal 0, but this probability is negligible, and this problem can be addressed by working over a prime field. In **SDG** protocol, the 1-out-of-2 selection operations are performed based on **SSH**, then the correct skyline is generated. Since **SPP** is responsible for the comparisons between the query and diagram, $\mathbf{SSC} \mapsto \mathbf{SPP}$ is correct due to the correctness of **SSC**.

In **SPE**, servers locally construct the polynomial to mask the skyline. Based on **SSH**, η_2 is addressed for computing the polynomial $f_t(\eta_2)$. Since there is only one element satisfies $u_2[\eta_2] = 1$, S_{j_1} is the correct skyline after removing mask m_2 . Similar to the mask m_1 , the correct skyline S_{η_1, η_2} is shared on two servers.

4.2 Privacy and Leakage

To analyze security, we follow the framework of the simulation paradigm [13].

Theorem 2. ***SDG** protocol protects the privacy of dataset, result, and dominance relationships if the involved numbers are pseudorandom and existing sub-protocols **SSE**, **SSC**, **SSM**, and **SSH** are secure under passive attacks.*

Proof. In the **SDG**, servers and external adversary $\mathcal{A}$ cannot obtain extra data from the skyline diagram except for $n_1 \times n_2$ local shared points. The inputs and outputs are shared and computed by existing secure protocols **SSH**, **SSM**, **SSE** and **SSC**. Meanwhile, since π and r are pseudorandom, the intermediate result in **SDG** is indistinguishable for $\mathcal{A}$. Details are presented in Appendix A.

By the same argument, we have that the simulation for **SPE** is indistinguishable from a real execution based on the proven security of **SSC** and **SSH**.

Theorem 3. ***SPE** protects the privacy of query and skyline if the coefficients are pseudorandom and existing **SSC** and **SSH** are secure under passive attacks.*

The access patterns involve only **SPE**. For each query, the shared index η is different by **SSC**, and $\mathbf{m}$ is not associated with η since coefficients a and b are pseudorandom locally. The server cannot independently collect index and skyline at a given time for statistical analysis, and the queries are not linked or reversed because η or $\mathbf{m}$ is indistinguishable from random numbers.

Based on the composition theorem [2], the security of our scheme is proved through the security of **SDG** and **SPE**, and we give the following theorem.

Theorem 4. *Our SPSQ scheme protects the privacy of the dataset, query, skyline, and indirect privacy if the random numbers involved are pseudorandom and existing **SSE**, **SSC**, **SSH**, and **SSM** protocols are secure under passive attacks.*

Proof. Overall, the entities use the randomness of random numbers and secret sharing that prevents privacy leakage in communication. As for $\mathcal{A}$, nothing can be obtained except for the data distribution and diagram size, where the specific value is indistinguishable and satisfies the dataset privacy requirements.

5 Performance Analysis

C++ developed our method from Ezpc [21] on a server with two 2.40GHz Intel Xeon Silver 4210R CPUs. We simulated the real communication channel with sockets, and the code is available at <https://github.com/youzichaa/Skyline>.

5.1 Theoretical Analysis

Communication Cost. Note that $n_k \leq n$ is the length of k^{th} coordinate in the diagram, dimension $d = 2$, $\bar{n}$ is the maximum number of skylines in all cells, and l is the length of bits. In addition, the communication cost in the protocol **SSC**, **SSE**, **SSH** and **SSM** are denoted C_c, C_e, C_h and C_m respectively.

Firstly, the DO shares the index $\mathbf{I}$, points $\mathbf{P}$ and coordinate $\mathbf{G}$, which needs $2 \prod_{k=1}^d n_k l + \sum_{k=1}^d n_k l$ bits. The RU shares dl bits of points.

There are $\prod_{k=1}^d n_k$ iterations. In each iteration, $2\bar{n}^2 C_e + 3\bar{n} C_c$ bits are required for the index vector $\mathbf{IV}$, $(21\bar{n}^2 l + 6\bar{n} C_h + 12\bar{n}^2 C_m)$ bits are required for skyline duplication in **SPI**, and $\bar{n} C_h$ extra bits are required in **SDG**. As a result, the total cost is $\prod_{k=1}^d n_k (21\bar{n}^2 l + 2\bar{n}^2 C_e + 3\bar{n} C_c + 7\bar{n} C_h + 12\bar{n}^2 C_m)$ bits.

The shares of position need $\sum_{k=1}^d n_k C_c$ bits in **SPP**, the skyline copies need two reconstructions with $\prod_{k=1}^d n_k \bar{n} l + n_1 \bar{n} l$ bits, and $(n_1 + 2)(2tC_h + l)$ bits for mask generation and transmission, where tC_h bits to generate $(\eta_k)^t$.

In general, $\mathcal{O}(n^d \bar{n}^2 (C_c + C_m))$ bits are required for the diagram offline. Furthermore, $\mathcal{O}(dnC_c + n^d \bar{n} l + ntC_h)$ bits are required for each query.

Computational Cost. All the computation mainly lies in secure share comparisons and multiplications. The cost in **SSC**, **SSE**, **SSH**, and **SSM** are denoted T_c, T_e, T_h and T_m seconds respectively. Similarly, in each iteration, $2\bar{n}^2 T_e + 3\bar{n} T_c$ seconds for **IV**, $(6\bar{n} T_h + 18\bar{n}^2 T_m)$ seconds for final **S** in **SPI**, and $\bar{n} T_h$ seconds are required in **SDG**. **SPP** needs $\sum_{k=1}^d n_k T_c$ seconds and skyline extraction $(n_1 + 2)2tT_h$ seconds while omitting the local share time.

In general, $\mathcal{O}(n^d \bar{n}^2 (T_c + T_m))$ offline, and $\mathcal{O}(dnT_c + ntT_h)$ seconds for queries.

5.2 Evaluation and Comparison

Parameter Settings. We evaluate the efficiency by varying the number of tuples from 1000 to 9000 while $l = 32$ and $d = 2$. Note that the query points are randomly chosen; the cost is calculated by averaging the measurements.

Datasets. We produced correlated (CORR), independent (INDE), and anti-correlated (ANTI) datasets similar to work [1]. Furthermore, we adopted two real two-dimensional datasets ($n = 1000$) about correlated diabetic medications and independent obesity levels from the UCI Machine Learning repository [9].

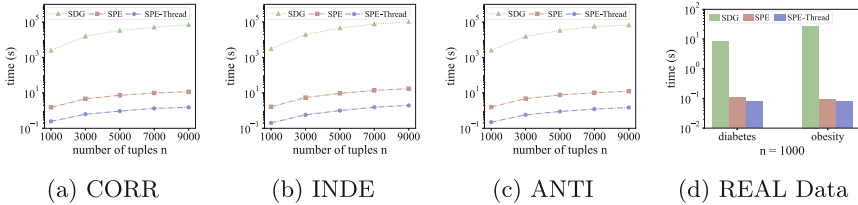


Fig. 5. Time of Quadrant Skyline Diagram

As Fig. 5 shows, the diagram time differs, and the gap becomes more pronounced as n increases. The INDE dataset consumes more time due to the most cells, directly affecting the diagrams' size. The most crucial query time, even for $n = 9000$, is less than 20s. We parallel the diagram generation and skyline copies under 32 threads. Time is linear and is reduced to **one second**. For two real datasets, the coordinate range is limited and slightly smaller. Details are presented in Appendix B. These diagrams take less than 30s for the diagram and less than a **hundred milliseconds** to query the quadrant skyline.

In the dynamic skyline, we compare recent works by listing our advantages.

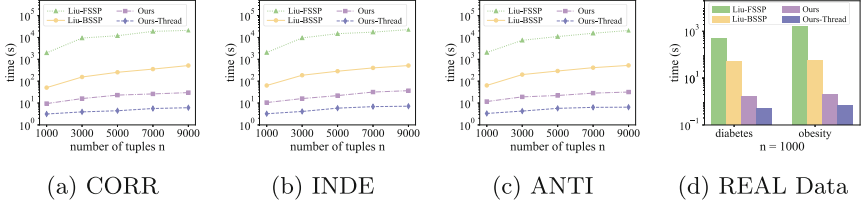


Fig. 6. Time of Dynamic Skyline Query

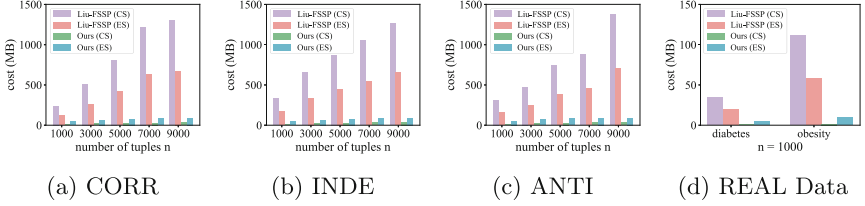


Fig. 7. Cost of Dynamic Skyline Query

1. **FSSP** and **BSSP** [14]. BSSP is a weakened FSSP, revealing dominance relationships. Figure 6 shows that our time is significantly lower even with BSSP. Our diagram copies mainly produce communication, while FSSP communication involves iterative picking, so our communication cost is lower as Fig. 7. Their 512-bit key is too small to meet practical security [19].
2. **SMSQ** [7]. The SMSQ effectively improves the FSSP by pruning the dominated tuples. However, it is less efficient than the BSSP because of the extra privacy matrix. Compared to the efficiency between the BSSP and the FSSP in their analysis, our scheme is naturally more efficient than the SMSQ.
3. **BVLSQ** [25]. BVLSQ's efficiency is approximately equal to BSSP's cost, and the gains are reduced by increasing node capacity for solid security. Since their R-tree process is unavailable, we can only conduct comparative experiments with BSSP on their HOTE dataset. As Fig. 8 shows, the global skyline is retrieved quickly, costing about one second, while the dynamic skyline requires domination filtering of global data. The skyline diagram can effectively alleviate the dimension growth of BVLSQ.
4. **SecSkyline** [30]. By generating a lot of secret-shared Beaver triple offline that increases dramatically with n and is not fixed, SecSkyline saves online overhead. Our secret-sharing is OT-based, accounting for most of the time and communication. That is why we consider that computational complexity is a fair comparison. SecSkyline requires $nd + (\theta + 1) \times n + \theta \times (nd + n)$ secure comparison operations. Precisely, we only need nd **SSC** operations find the position, and take $(\theta + 1) \times (\bar{n} - 1) + \theta \times (\bar{n}d + \bar{n})$ **SSC** for query. Our theoretical time is much shorter ($\mathcal{O}(\frac{n}{\bar{n}})$) than SecSkyline. From Fig. 9, the gain is more prominent as n increases. Details are presented in Appendix B.

In real datasets, FSSP requires at least 506 s while we need about 1.68 s with $301\times$ improvement, the communication cost is reduced by nine times. In parallel execution, the query time is reduced to 3–7 s in synthetic datasets and less than **0.7 s** in real datasets. This suggests a skyline diagram is more suitable for real applications with limited cells.

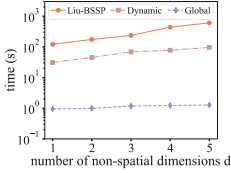


Fig. 8. Time on the HOTE

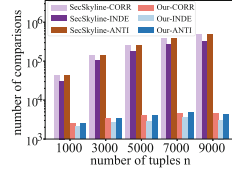


Fig. 9. Number of Comparisons

Discussion. Considering the database updates, the skyline diagram only requires incremental changes. For instance, the deleting only requires that the point in involved cells be set to 0. Inserting requires adding dn cells but essentially performing the inherited process in **QSPO** algorithm.

6 Conclusion and Future Work

This work proposed an efficient and privacy-preserving skyline query by designing a novel secure skyline diagram generation with parallelism and trade-off private information retrieval method under the federated clouds, achieving desirable privacy protections of dataset, skyline and query. The evaluation shows that our skyline query is practical, especially for real-time query and location-based applications. In the future, it is interesting to consider authentication and better efficiency on higher dimensional datasets involving skyline variants.

Appendix

Appendix A Security Analysis with Simulation

We construct a polynomial-time simulator $\mathcal{S}$ with the ideal process for $\mathcal{F}_{\text{SDG}}$ such that for any probability polynomial time (PPT) adversary $\mathcal{A}$, no environment $\mathcal{Z}$ can tell with non-negligible probability whether it is interacting with $\mathcal{A}$ and the real protocol or with $\mathcal{S}$ in the ideal process for $\mathcal{F}_{\text{SDG}}$. We describe the real view $\mathbf{Real}_{\mathcal{A}}$ and the simulated view $\mathbf{Ideal}_{\mathcal{A},\mathcal{S}}$ as follows.

In the $\mathbf{Real}_{\mathcal{A}}$, given the inputs P_{j_1+1,j_2+1} and I_{j_1+1,j_2+1} , CS and ES calculate the index vector $\mathbf{IV}_{j_1,j_2}$ with three generated skylines $\mathbf{S}_{j_1+1,j_2}$, $\mathbf{S}_{j_1,j_2+1}$ and $\mathbf{S}_{j_1+1,j_2+1}$ by **SSE** and **SSC**, and exchanges the obfuscated skyline shares $\mathbf{S}'''_T$ with some pseudorandom numbers π and r by **SSM** and **SSH**. CS and ES select

the final set intersection and compute the skyline $\mathbf{S}_{j_1, j_2}$ with inputs $\mathbf{P}_{j_1+1, j_2+1}$, $\mathbf{I}_{j_1+1, j_2+1}$ and $\mathbf{0}$ by **SSH**. Finally, they hold the shared skyline $\langle \mathbf{S}_{j_1, j_2} \rangle$.

In the **Ideal** $_{\mathcal{A}, \mathcal{S}}$, recall that $\mathcal{S}$ interacts with the ideal functionality $\mathcal{F}_{\text{SDG}}$ and with the environment $\mathcal{Z}$. Simulator $\mathcal{S}$ starts by invoking a copy of $\mathcal{A}$ and running a simulated interaction of $\mathcal{A}$ with $\mathcal{Z}$ and parties running the protocol [3].

1. Simulating the communication with $\mathcal{Z}$: Every input value that $\mathcal{S}$ receives from $\mathcal{Z}$ is written on $\mathcal{A}$'s input tape (as if coming from $\mathcal{A}$'s environment). Likewise, each output written by $\mathcal{A}$ on its output tape is copied to $\mathcal{S}$'s output tape (to be read by $\mathcal{S}$'s environment $\mathcal{Z}$).
2. Simulating the case where CS only is corrupted: $\mathcal{S}$ begins by activating $\mathcal{A}$ and internally sending it the message that $\mathcal{A}$ (controlling CS) expects to receive from ES in a real execution. Simulator $\mathcal{S}$ externally sends CS's input (i, j) and $(\langle \mathbf{S}_{j_1+1, j_2} \rangle^C, \langle \mathbf{S}_{j_1, j_2+1} \rangle^C, \langle \mathbf{S}_{j_1+1, j_2+1} \rangle^C)$ to $\mathcal{F}_{\text{SDG}}$ and receives back the output (i, j, ξ_1, ξ_2) . Then, $\mathcal{S}$ simulates $\langle \mathbf{S}_1 \rangle^E \in_R \mathbb{Z}_N^{\xi_1}$, $\langle \mathbf{S}_2 \rangle^E \in_R \mathbb{Z}_N^{\xi_2}$ and simulates ES interacting with CS in the internal interactions to receive $\langle \mathbf{IV} \rangle^E$. Next, when $\mathcal{A}$ sends the message $\langle \pi_c \rangle^E$ from CS to ES, $\mathcal{S}$ simulates $\langle \pi_e \rangle^E \in_R \mathbb{Z}_N^{\xi \times \xi}$ and sends $\langle \pi_e \rangle^C$ to CS. $\mathcal{S}$ simulates $\langle r \rangle^E \in_R \mathbb{Z}_N^\xi$ and simulates ES interacting with CS in the internal interactions to receive $\langle \mathbf{S}_T''' \rangle^E$ for computing the final set intersection. Finally, $\mathcal{S}$ simulates $\langle \mathbf{P}_{j_1+1, j_2+1} \rangle^E \in_R \mathbb{Z}_N$ and $\langle \mathbf{I}_{j_1+1, j_2+1} \rangle^E \in_R \mathbb{Z}_N$, and simulates ES interacting with CS to construct $\langle \mathbf{0} \rangle$ to output the final skyline $\langle \mathbf{S}_{j_1, j_2} \rangle$.
3. Simulating the case where ES only is corrupted: $\mathcal{S}$ begins by activating $\mathcal{A}$ and internally sending it the message that $\mathcal{A}$ (controlling ES) expects to receive from CS in a real execution. Simulator $\mathcal{S}$ externally sends ES's input (i, j) and $(\langle \mathbf{S}_{j_1+1, j_2} \rangle^E, \langle \mathbf{S}_{j_1, j_2+1} \rangle^E, \langle \mathbf{S}_{j_1+1, j_2+1} \rangle^E)$ to $\mathcal{F}_{\text{SDG}}$ and receives back the output (i, j, ξ_1, ξ_2) . Then, $\mathcal{S}$ simulates $\langle \mathbf{S}_1 \rangle^C \in_R \mathbb{Z}_N^{\xi_1}$, $\langle \mathbf{S}_2 \rangle^C \in_R \mathbb{Z}_N^{\xi_2}$ and simulates CS interacting with ES in the internal interactions to receive $\langle \mathbf{IV} \rangle^C$. Next, when $\mathcal{A}$ sends the message $\langle \pi_e \rangle^C$ from ES to CS, $\mathcal{S}$ simulates $\langle \pi_c \rangle^E \in_R \mathbb{Z}_N^{\xi \times \xi}$ and sends $\langle \pi_c \rangle^E$ to ES. $\mathcal{S}$ simulates $\langle r \rangle^C \in_R \mathbb{Z}_N^\xi$ and simulates CS interacting with ES in the internal interactions to receive $\langle \mathbf{S}_T''' \rangle^C$ for computing the final set intersection. Finally, $\mathcal{S}$ simulates $\langle \mathbf{P}_{j_1+1, j_2+1} \rangle^C \in_R \mathbb{Z}_N$ and $\langle \mathbf{I}_{j_1+1, j_2+1} \rangle^C \in_R \mathbb{Z}_N$, and simulates CS interacting with ES to construct $\langle \mathbf{0} \rangle$ to output the final skyline $\langle \mathbf{S}_{j_1, j_2} \rangle$.
4. Simulating the case that neither party is corrupted: In this case, $\mathcal{S}$ receives a message $\langle \mathbf{S}_{j_1, j_2} \rangle$ signaling it that CS and ES concluded an ideal execution with $\mathcal{F}_{\text{SDG}}$. $\mathcal{S}$ then generates a simulated transcript of messages between the real parties. That is, $\mathcal{S}$ obtains the length of inputs L . In each cell, $\mathcal{S}$ simulates the input $\mathbf{P}_{j_1+1, j_2+1}^* \in_R \mathbb{Z}_N$ and $\mathbf{I}_{j_1+1, j_2+1}^* \in_R \mathbb{Z}_N$ of length 1, simulates the dummy points $\mathbf{0}$ of length $L - 1$, and extracts the simulated history $\mathbf{S}_{j_1+1, j_2}^*$, $\mathbf{S}_{j_1, j_2+1}^*$ and $\mathbf{S}_{j_1+1, j_2+1}^*$ of corresponding length. After that, $\mathcal{S}$ forms the simulated input $\mathbf{P}_{j_1+1, j_2+1}^*$ and $\mathbf{I}_{j_1+1, j_2+1}^*$, and runs $\mathcal{F}_{\text{SDG}}$ to output the result $\mathbf{S}_{j_1, j_2}^*$ by the experiment.

The shared inputs and outputs are indistinguishable from uniformly chosen random numbers. Based on the simulator $\mathcal{S}$, no PPT adversary can distinguish the output of $\mathbf{Ideal}_{\mathcal{A},\mathcal{S}}$ from that of $\mathbf{Real}_{\mathcal{A}}$. That is, $\mathbf{Ideal}_{\mathcal{A},\mathcal{S},\mathcal{Z}} \stackrel{c}{\approx} \mathbf{Real}_{\mathcal{A},\mathcal{Z}}$.

In $\mathcal{F}_{\mathbf{SDG}}$, the adversary only knows the final size of each cell since adding some dummy points to the intermediate result, and then there is some leakage about the distribution of points in the dataset. However, the adversary cannot derive values from the data distribution because the skylines are invisible.

The adversary knows nothing in $\mathcal{F}_{\mathbf{SPE}}$ since we add dummy points to all the cells to make the size the same. Opening $\mathfrak{m}$ does not make more information leakage occur in η due to the random coefficients. Therefore, The leakage of $\mathcal{F}_{\mathbf{SPE}}$ is only related to the degree t of the polynomial, but $t = 2$ ensures safety when the skylines in the cell are not identical and visible.

Appendix B Details for Skyline Diagram

Table 3 illustrates examples of averages, $\bar{n}$ is at least two orders of magnitude smaller than n , thus effectively reducing the retrieval size. Concretely, when $n = 9000$, ours 22491 secure operations are ($22\times$) less than SecSkyline's 495000 in CORR, ($15\times$) in INDE and ($22\times$) in ANTI. When $n = 9000$, ($9.6\times$) in CORR, ($7.6\times$) in INDE and ($9.4\times$) in ANTI, ($5\times$) and ($10.7\times$) in REAL, respectively.

Table 3. Size of diagram (n_1, n_2) with $[\bar{n}, \theta]$ when $d = 2$

n	CORR	INDE	ANTI
1000	(910, 902)[60,10]	(950, 958)[71,7]	(927, 899)[62,10]
3000	(2333, 2354)[76,11]	(2613, 2594)[80,8]	(2321, 2340)[76,11]
5000	(3372, 3417)[83,12]	(3919, 3597)[87,8]	(3356, 3409)[81,12]
7000	(4163, 4161)[84,13]	(5017, 5060)[95,9]	(4109, 4135)[90,13]
9000	(4707, 4728)[85,13]	(5920, 5952)[94,8]	(4710, 4690)[82,13]
^a $n = 1000$. Diabetes: (60, 30)[22,2], Obesity: (50, 114)[31,7].			

References

1. Börzsönyi, S., Kossmann, D., Stocker, K.: The skyline operator. In: 17th IEEE International Conference on Data Engineering, ICDE, pp. 421–430. IEEE Computer Society (2001)
2. Canetti, R., Lindell, Y., Ostrovsky, R., Sahai, A.: Universally composable two-party and multi-party secure computation. In: Proceedings on 34th Annual ACM Symposium on Theory of Computing, 19–21 May 2002, Montréal, Québec, Canada, pp. 494–503. ACM (2002)
3. Chen, H., Chillotti, I., Dong, Y., Poburinnaya, O., Razenshteyn, I.P., Riazi, M.S.: SANNS: scaling up secure approximate k-nearest neighbors search. In: 29th USENIX Security Symposium, USENIX Security 2020, 12–14 August 2020, pp. 2111–2128. USENIX Association (2020)

4. Chen, J., Liu, L., Chen, R., Peng, W., Huang, X.: Secrec: a privacy-preserving method for the context-aware recommendation system. *IEEE Trans. Dependable Secur. Comput.* **19**(5), 3168–3182 (2022)
5. Chen, W., Liu, M., Zhang, R., Zhang, Y., Liu, S.: Secure outsourced skyline query processing via untrusted cloud service providers. In: 35th Annual IEEE International Conference on Computer Communications, INFOCOM 2016, USA, April, 2016, pp. 1–9. IEEE (2016)
6. Choi, S., Ghinita, G., Lim, H., Bertino, E.: Secure KNN query processing in untrusted cloud environments. *IEEE Trans. Knowl. Data Eng.* **26**(11), 2818–2831 (2014)
7. Ding, X., Wang, Z., Zhou, P., Choo, K.R., Jin, H.: Efficient and privacy-preserving multi-party skyline queries over encrypted data. *IEEE Trans. Inf. Forensics Secur.* **16**, 4589–4604 (2021)
8. Doerner, J., Shelat, A.: Scaling ORAM for secure computation. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, 2017, pp. 523–535. ACM (2017)
9. Dua, D., Graff, C.: UCI machine learning repository (2017). <http://archive.ics.uci.edu/ml>
10. Hose, K., Vlachou, A.: A survey of skyline processing in highly distributed environments. *VLDB J.* **21**(3), 359–384 (2012)
11. Kermanshahi, S.K., et al.: Multi-client cloud-based symmetric searchable encryption. *IEEE Trans. Dependable Secur. Comput.* **18**(5), 2419–2437 (2021)
12. Kerschbaum, F., Blass, E., Mahdavi, R.A.: Faster secure comparisons with offline phase for efficient private set intersection. In: 30th Annual Network and Distributed System Security Symposium, NDSS 2023, San Diego, California, USA, 27 February–3 March 2023. The Internet Society (2023)
13. Lindell, Y.: How to simulate it – a tutorial on the simulation proof technique. In: Lindell, Y. (ed.) *Tutorials on the Foundations of Cryptography*. ISC, pp. 277–346. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-57048-8_6
14. Liu, J., Yang, J., Xiong, L., Pei, J.: Secure and efficient skyline queries on encrypted data. *IEEE Trans. Knowl. Data Eng.* **31**(7), 1397–1411 (2019)
15. Liu, J., et al.: Skyline diagram: efficient space partitioning for skyline queries. *IEEE Trans. Knowl. Data Eng.* **33**(1), 271–286 (2021)
16. Liu, L., et al.: Towards strong privacy protection for association rule mining and query in the cloud. *IEEE Trans. Cloud Comput.* **11**(3), 3211–3225 (2023)
17. Liu, X., Yang, D., Ye, M., Lee, W.: U-skyline: a new skyline query for uncertain databases. *IEEE Trans. Knowl. Data Eng.* **25**(4), 945–960 (2013)
18. Mohassel, P., Rindal, P.: Aby^3 : a mixed protocol framework for machine learning. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, 15–19 October 2018, pp. 35–52. ACM (2018)
19. Paillier, P.: Public-key cryptosystems based on composite degree residuosity classes. In: Stern, J. (ed.) *EUROCRYPT 1999*. LNCS, vol. 1592, pp. 223–238. Springer, Heidelberg (1999). https://doi.org/10.1007/3-540-48910-X_16
20. Qaasar, M., Zaman, A., Siddique, M.A., Annisa, Morimoto, Y.: Privacy-preserving secure computation of skyline query in distributed multi-party databases. *Information* **10**(3), 119 (2019)
21. Rathee, D., et al.: Cryptflow2: practical 2-party secure inference. In: CCS 2020: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, pp. 325–342. ACM (2020)

22. Wagh, S., Tople, S., Benhamouda, F., Kushilevitz, E., Mittal, P., Rabin, T.: Falcon: honest-majority maliciously secure framework for private deep learning. *Proc. Priv. Enhancing Technol.* **2021**(1), 188–208 (2021)
23. Wang, J., Du, M., Chow, S.S.M.: Stargazing in the dark: secure skyline queries with SGX. In: Nah, Y., Cui, B., Lee, S.-W., Yu, J.X., Moon, Y.-S., Whang, S.E. (eds.) *DASFAA 2020. LNCS*, vol. 12114, pp. 322–338. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-59419-0_20
24. Wang, W., et al.: SCALE: an efficient framework for secure dynamic skyline query processing in the cloud. In: Nah, Y., Cui, B., Lee, S.-W., Yu, J.X., Moon, Y.-S., Whang, S.E. (eds.) *DASFAA 2020. LNCS*, vol. 12114, pp. 288–305. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-59419-0_18
25. Wang, Z., Ding, X., Jin, H., Zhou, P.: Efficient secure and verifiable location-based skyline queries over encrypted data. *Proc. VLDB Endow.* **15**(9), 1822–1834 (2022)
26. Zeighami, S., Ghinita, G., Shahabi, C.: Secure dynamic skyline queries using result materialization. In: 37th IEEE International Conference on Data Engineering, ICDE, pp. 157–168. IEEE (2021)
27. Zhang, S., Ray, S., Lu, R., Zheng, Y., Guan, Y., Shao, J.: Towards efficient and privacy-preserving interval skyline queries over time series data. *IEEE Trans. Dependable Secur. Comput.* **20**(2), 1348–1363 (2023)
28. Zhang, S., Ray, S., Lu, R., Zheng, Y., Guan, Y., Shao, J.: Towards efficient and privacy-preserving user-defined skyline query over single cloud. *IEEE Trans. Dependable Secur. Comput.* **20**(2), 1319–1334 (2023)
29. Zheng, Y., Lu, R., Li, B., Shao, J., Yang, H., Choo, K.R.: Efficient privacy-preserving data merging and skyline computation over multi-source encrypted data. *Inf. Sci.* **498**, 91–105 (2019)
30. Zheng, Y., Wang, W., Wang, S., Jia, X., Huang, H., Wang, C.: Secskyline: fast privacy-preserving skyline queries over encrypted cloud databases. *IEEE Trans. Knowl. Data Eng.* **35**(9), 8955–8967 (2023)



RedactBuster: Entity Type Recognition from Redacted Documents

Mirco Beltrame¹, Mauro Conti^{1,2}, Pierpaolo Guglielmin¹,
Francesco Marchiori^{1(✉)}, and Gabriele Orazi^{1,3}

¹ Department of Mathematics, University of Padua, Padua, Italy
{mirco.beltrame.1,pierpaolo.guglielmin}@studenti.unipd.it,
mauro.conti@unipd.it, {francesco.marchiori.4,gabriele.orazi}@phd.unipd.it

² Faculty of Electrical Engineering, Mathematics and Computer Science, Delft
University of Technology, Delft, The Netherlands

³ FDM Business Services, Milan, Italy

Abstract. The widespread exchange of digital documents in various domains has resulted in abundant private information being shared. This proliferation necessitates redaction techniques to protect sensitive content and user privacy. While numerous redaction methods exist, their effectiveness varies, with some proving more robust than others. As such, the literature proposes several deanonymization techniques, raising awareness of potential privacy threats. However, while none of these methods are successful against the most effective redaction techniques, these attacks only focus on the anonymized tokens and ignore the sentence context.

In this paper, we propose **RedactBuster**, the first deanonymization model using sentence context to perform Named Entity Recognition on redacted text. Our methodology leverages fine-tuned state-of-the-art Transformers and Deep Learning models to determine the anonymized entity types in a document. We test RedactBuster against the most effective redaction technique and evaluate it using the publicly available Text Anonymization Benchmark (TAB). Our results show accuracy values up to 0.985 regardless of the document nature or entity type. In raising awareness of this privacy issue, we propose a countermeasure we call *character evasion* that helps strengthen the secrecy of sensitive information. Furthermore, we make our model and testbed open-source to aid researchers and practitioners in evaluating the resilience of novel redaction techniques and enhancing document privacy.

Keywords: Document Redaction · Privacy · Personally Identifiable Information · Named Entity Recognition · Information Leakage

1 Introduction

Increasing document digitalization efforts have been adopted in several domains, such as the corporate sector, healthcare, and government [39]. These procedures allow for streamlined workflows, improving processing times and reducing

reliance on manual manipulation. As a side effect, the amount of data exchanged has also seen a significant rise [29]. To illustrate the magnitude of the phenomenon, in 2022 and in Italy alone, the car and motorcycle insurance market generated more than 39 million documents between contracts and claims for a market of about 22.6 billion euro [19]. Each of these 39 billion documents contains sensitive information that needs to be protected. While the process of digitalization can improve efficiency and optimization in specific tasks thanks to the recent advancements of Artificial Intelligence (AI) and data-driven approaches, it can cause privacy concerns. Indeed, the lack of obfuscation of sensitive content may lead to unfair profiling of certain individuals or discrimination of specific groups [17]. For these reasons, *anonymization* defined as “the process of rendering personal data anonymous” has been mandated for General Data Protection Regulation (GDPR) compliance in the EU [14].

Several anonymization techniques have been proposed, given the importance of protecting user privacy and complying with regulations. In the context of privacy protection, *redaction* refers to the process of selectively editing or obscuring sensitive information from documents to prevent unauthorized access or disclosure [2]. It allows for removing only specific parts of a sentence while preserving the overall content of the text. In the Italian case, document redaction is required for all public administrations. The Personal Data Protection Guarantor (GDPD) obliges public organizations to publish how public resources were used by anonymizing Personally Identifiable Information (PII) that refers to individuals. However, this anonymization process might leave some information about the users if only sensitive words are redacted. For example, inferring the entity type of redacted content can pose a privacy threat as it could be cross-referenced with other open-source intelligence. This can also lead to security risks, as the knowledge of specific locations or individuals involved in sensitive documents could be leveraged for malicious purposes. Some examples of redaction techniques are shown in Fig. 1. One redaction technique blurs a specific text part to make it unreadable (Fig. 1b). Another considers specific parts of the document as images and pixelates them by reducing their image quality (Fig. 1c). Blackout or whiteout redaction techniques (when applied correctly) are more effective, as they allow to discard entirely the content to be anonymized and substitute it with black or white boxes (Fig. 1d).

While redaction techniques are commonly used in many domains, their security in protecting user privacy is uncertain. Indeed, several works in the literature propose attacks for determining redacted content in sensitive documents. For example, blurred and pixelated content can be unmasked with high accuracy [18]. For these reasons, other redaction techniques, such as blackout and whiteout, should be preferable. While the effectiveness of other redaction techniques has not yet been discussed, no attacks in the literature have been proposed. Furthermore, the existing attacks focus exclusively on the redacted tokens. Since with more effective anonymization techniques the sensitive content is erased from the document, this strategy is not effective anymore.

The case originated in an application (no. 1518/36) against the PROSEC Research Group lodged with the Court under Article 34 of the Convention for the Protection of Human Rights and Fundamental Freedoms ("the Convention") by an Italian national, Ms Dolores B.S. ("the applicant"), on 1 January 2000.

The case originated in an application (no. [REDACTED] against the [REDACTED] lodged with the Court under Article 34 of the Convention for the Protection of Human Rights and Fundamental Freedoms ("the Convention") by an [REDACTED] national, [REDACTED] ("the applicant"), on [REDACTED]

(a) Plain text.

(b) Blurring.

The case originated in an application (no. [REDACTED] against the [REDACTED] lodged with the Court under Article 34 of the Convention for the Protection of Human Rights and Fundamental Freedoms ("the Convention") by an [REDACTED] national, [REDACTED] ("the applicant"), on [REDACTED]

The case originated in an application (no. [REDACTED] against the [REDACTED] lodged with the Court under Article 34 of the Convention for the Protection of Human Rights and Fundamental Freedoms ("the Convention") by an [REDACTED] national, [REDACTED] ("the applicant"), on [REDACTED]

(c) Pixelation.

(d) Blackout.

Fig. 1. Examples of redaction techniques.

Contribution. In this paper, we present **RedactBuster**, the first document deanonymization attack against the most effective redaction technique. We open-source and evaluate our framework on the most comprehensive dataset, obtaining results of up to 0.985. We also discuss the implementation of several countermeasures to our attack and propose a novel technique to enhance redaction efforts. Our contributions can be summarized as follows.

- We present **RedactBuster**, the first deanonymization attack against the most effective redaction technique. Our methodology leverages state-of-the-art Machine Learning (ML) and Deep Learning (DL) models for computing sentence embeddings and performing classification.
- We evaluate our framework on Text Anonymization Benchmark (TAB), the most extensive dataset publicly available on document redaction [32]. This dataset contains 1268 court cases in English, which have been properly annotated, labeled, and redacted. Our results show baseline accuracy values of 0.958, which can reach 0.985 by fine-tuning the embedding model on our specific task. We test several ML and DL models, comprehensively evaluating different architectures' capabilities and limitations.
- We propose *character evasion* as an effective countermeasure against our attack and test them against our model. This technique involves the exchange of specific homoglyphs, allowing users to read and process the document effortlessly but preventing malicious parties from extracting the redacted entity types. We also demonstrate the effectiveness of our countermeasure on the dataset and show how swapping only five characters can decrease the attack success rate to 0.195.
- We open-source our framework at: <https://github.com/spritz-group/RedactBuster>.

Organization. Our paper is organized as follows. In Sect. 2, we discuss related works on anonymization and attacks on redaction techniques. Section 3 presents our system and threat model, while technical details on the methodology are provided in Sect. 4. In Sect. 5, we evaluate our framework, and we propose countermeasures in Sect. 6. Finally, Sect. 7 concludes our work.

2 Related Works

The importance of redaction for protecting documents containing sensitive data or PII does not have a widely shared standard among parties to date. In June 2023, a litigation between Microsoft and the Federal Trade Commission demonstrated that (i) documents can still be redacted by hand using a Sharpie marker and (ii) redacted document represents a threat to organizations because of possible and expensive data leaks¹. This event highlights how sensitive information circulating through documents is still subject to human error today. The unstructured nature of a written document makes it impossible to apply analytical methods to arrive at a specific result. Despite the current tools to support this problem, manual validation is simultaneously the only solution to curb the deficiency and the source of possible issues. Therefore, in this section, we describe which are the current solutions for document redaction, how PII can be recognized within a text, and which unredaction techniques have been found to date.

Redaction Techniques. Many studies and patents tackle the redaction phase per se, covering some parts of the text. In [21], the procedure is specifically crafted to work in synergy with Microsoft Office Word. In [27] a rule-set based automatic redaction system is presented, while [12] proposes a dynamic redaction based on tags specified by the user. Instead of covering, random generation of sensitive characters and numbers has also been patented [26]. A similar approach is presented in [1], which includes a redaction and storage system for redacted documents. Another valuable approach is presented in [33], which implies the concept of blockchain to preserve the security of an already-redacted document. Unfortunately, such studies neglect the core of the redaction process, which is identifying the sensitive characters of a text that must be covered.

Named Entity Recognition (NER). NER is a crucial component of Natural Language Processing (NLP). It identifies predefined categories of objects in text, such as names of individuals, organizations, locations, expressions of time, and quantities. In [10], the authors present a novel approach in the field by employing bidirectional Long Short-Term Memory (LSTM) and Convolutional Neural Network (CNN) architecture. The insight of the latter work influenced the creation of the well-known BERT [13], a milestone on which much work in NLP is still based today. BERT has also proven to be highly effective in developing techniques for NER [24,25]. Nevertheless, BERT models are also adopted as hyper-specialized models for particular domains such as [40] materials and aerospace engineering [36]. The scientific community has shown approaches for automation of NER and subsequent redaction of sensitive parts, albeit supervised by an end user [2]. On the same line, Microsoft developed and maintains an open source tool called PRESIDIO [28], which analyzes a text and, based on multiple recognizers, can detect and anonymize PII.

¹ <https://www.theverge.com/2023/6/28/23777298/sony-ftc-microsoft-confidential-documents-marker-pen-scanner-oops>.

Unredaction. Despite exciting developments in text redaction, studies are pointing out that some methods of information coverage are ineffective and vulnerable to possible data leaks. In [5], the authors assess standard PDF redaction tools vulnerability. The study enlightens the possibility of recovering first and last names just because of subpixel-sized shifts of characters. Instead, the authors of [18] demonstrate the inadequacy of mosaicing, blurring, and pixelating methods as graphical coverage of sensitive characters. To further demonstrate how dangerous such methods are, the security engineer Dan Petro released a proof-of-concept [31]. Although the outcome might be preferred to solid black rectangles, such aberrations represent an actual information leakage for a potential attacker.

3 System and Threat Model

We now discuss the assumptions defining both the redaction system’s functionality and potential attackers’ capabilities. In particular, Sect. 3.1 delves into the system model and the functionalities of a redaction pipeline in an adversary-free environment. In Sect. 3.2, we analyze the attacker’s knowledge of the target model in real-world scenarios.

3.1 System Model

Digital documents are commonly used online to share knowledge, sign contracts, or write reports. Due to the nature of documents, it is expected to share such files while preserving the sensitive information. *Redaction* is the process that allows companies and organizations to identify and protect the most sensitive parts of a document by covering such portions of the text. To date, no objective standard has been certified as the best methodology for digital document redaction. The security of such an operation falls in the hands of individuals or companies that may adopt more or less virtuous methods.

A general pipeline that organizations adopt is the one shown in Fig. 2 and includes the following main steps:

1. *Automated Entity Recognition.* The text in a document is fed into a NER software, which produces an intermediate document version with highlighted entities that will potentially be redacted. Entities can be differentiated by using different highlighting colors depending on the kind of entity (e.g., proper name, date, location). If needed, entities that can or must be disclosed safely can be included in a whitelist.
2. *Human validation.* An operator takes charge of the intermediate document and manually reviews the highlighted entities to assess that there are no missed detections. Once the validation is complete, the final redacted version of the document can be generated.

Although redaction is a lossy process of the original digital document, many details can still be used to empower inference techniques for disclosure purposes.

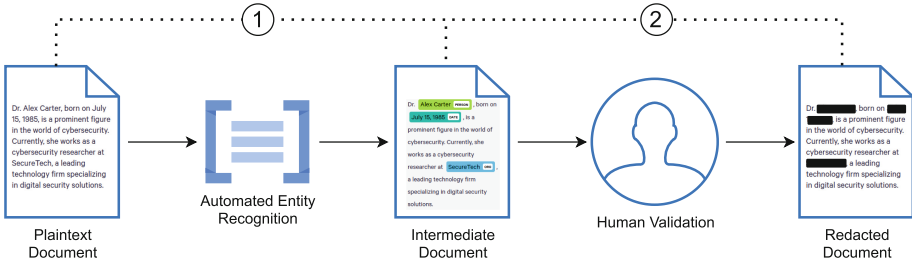


Fig. 2. General pipeline for document redaction.

3.2 Threat Model

RedactBuster aims to predict which kinds of entities are hidden underneath a redacted text. Even though working with unstructured text contained in documents is usually a challenge, our model wants to leverage the complexity of sentences to achieve its goal.

The attacker employing RedactBuster is interested in reconstructing the redacted text, ideally being able to rebuild the original unredacted document. In other words, they want to revert the pipeline described in Sect. 3.1. Documents are redacted to be shared with a broader audience that might not otherwise view the information in plain text. For this reason, we assume that the attacker has access to a redacted document that, if deanonymized, could lead to real gain. For instance, the attacker may want to access plaintext documents to extort, threaten, or ransom the victim.

Formal Definition. We will now outline the context in which the attacker operates, considering four key criteria [3]: their *goals*, *knowledge*, *capabilities*, and *strategies*.

- *Goal:* The attacker wants to recover the original version of a document that has been redacted.
- *Knowledge:* The attacker knows the area in which the document of interest falls. A list of examples includes fiscal, legal, or technical scope.
- *Capability:* The attacker knows how and where to find redacted documents they aim to deanonymize.
- *Strategy:* The attacker extracts the redacted text from the target document and processes it through the proposed model. With the entity predictions, they can leverage other kinds of attacks (such as social engineering attacks or data leaks) or knowledge bases to fully unredacted the target document.

Practical Scenario. To accomplish the goal, the attacker of our threat model must progress through different stages: *collection*, *processing* and *recomposition*:

1. *Collection:* The attacker gains access to redacted documents. Optionally, the attacker can use other publicly available datasets similar to the target document’s scope to fine-tune the unredaction system.

2. *Processing*: After being extracted, the redacted text is processed by the proposed model to gain predictions on covered entity types.
3. *Recomposition*: Combining the results of the previous step and additional knowledge (personal/public domain or combined attacks), the attacker recovers the original document’s plain text.

The main focus of this paper is mainly encapsulated in the *processing* step. The *collection* step is dependent on the target that the attacker sets. Similarly, in the *recomposition* step, the attacker has a plethora of possibilities that are impossible to describe exhaustively and are, in fact, not primarily of interest to the purpose of the paper. Some essential insights may already be contained within the redacted document: one example is the length of the redacted block, which can determine with a reasonable degree of approximation how many characters the hidden text consists of.

Along the same lines, the step of extracting text from the target document (typically a PDF file) is beyond the scope of this paper. The documents are usually structured, and the text can be extracted even with a simple copy-paste from any file reader. In cases where the document is unstructured because perhaps it is the result of scanning, Optical Character Recognition (OCR) can be involved. This technology has been extensively examined by the academic community [9,22] as well as being widely adopted in the consumer world. In addition, there are OCR open source projects such as Tesseract² or Apache Tika³, both widely adopted by the developers.

4 Methodology

We now detail our methodology and the techniques we use for unredaction. In Sect. 4.1, we provide the specifics of the dataset and how it generalizes real-world scenarios. Section 4.2 discusses the data processing steps applied, constituting a crucial component of our contribution. In Sect. 4.3, we detail the used models and their hyper-parameters. A complete overview of our framework is shown in Fig. 3.

4.1 Dataset

Publicly available datasets present several limitations that make their application to our study particularly challenging. In particular, most datasets are restricted to clinical records and describe only a single individual [20]. Furthermore, many are related to public figures, include only short sentences, are composed of images, or have been subject to prior redaction or anonymization techniques [30].

² <https://github.com/tesseract-ocr/tesseract>.

³ <https://tika.apache.org/>.

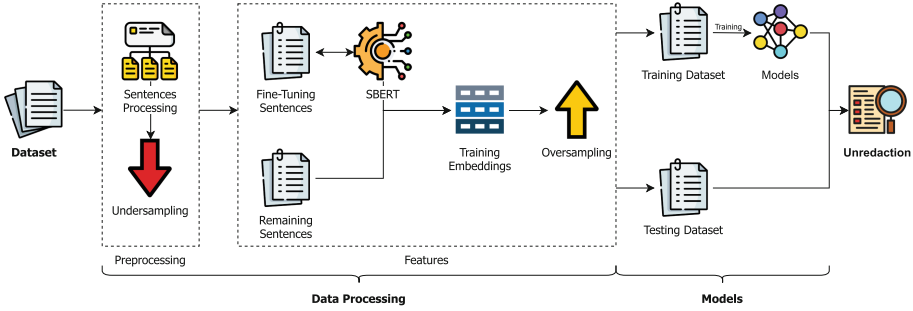


Fig. 3. RedactBuster framework overview.

Text Anonymization Benchmark. For our study, we use the TAB dataset, an open-source annotated corpus consisting of 1268 English-language court cases from the European Court of Human Rights (ECHR). Twelve university students at the University of Oslo have annotated the documents of this corpus after being trained on their classification. In particular, multiple users have annotated each document to cross-check their output. In case of disagreements, conflicts were resolved among students based on the guidelines provided by the authors. These guidelines indicate that each annotator should redact *direct identifiers* and *quasi-identifiers*. The former refers to unique values given to an individual, e.g., full names, email addresses, and social security numbers. The latter refers to publicly known information of an individual that doesn’t enable identification when isolated, e.g., gender, nationality, postal code. Each document is then distributed in JSON format and contains the following data.

- Text of the court case used during the annotation.
- Document annotation.
- The target of the anonymization task.
- Whether another annotator revised the document.

Labels. The redacted text and its properties will constitute the starting point for extracting the features needed for classification. Instead, the annotations represent our study’s labels and ground truth. In particular, each redacted entity belongs to one of eight classes: datetime (e.g., “15 December 1993”), organization (e.g., “Court of Cassation”), person (e.g., “Dr Price”), demographic (e.g., “police officer”), location (e.g., “London”), miscellaneous (e.g., “overdose of paracetamol”), quantity (e.g., “2,000,000 Swedish kronor (SEK)”), and code (e.g., “36619/03”).

4.2 Data Processing

When dealing with classification tasks, features and their correlation are the most significant variables models can use to compute predictions. For this reason, we first need to split the data for each annotation and define its ground truth. Then,

features must be extracted from the redacted sentences, and eventual biases in the dataset must be removed.

Preprocessing. The first step is to clean the text of each document and remove artifacts that might tamper with the classification process. In this step, we handle end-of-line characters and particular abbreviations. For instance, court texts in the dataset always include section titles (e.g., “*THE FACTS*”), which are followed by a series of `\n` characters. We handle them by substituting them with dot characters and an appropriate number of spaces since their removal would make the redaction offset wrong. We present more details of the textual-level modifications we employ in Appendix A.1. Since each document contains several redacted entities, we separate them into single samples. This facilitates classification, as ML and DL models predict one label for each sample. Therefore, we split the text into sentences using NLTK PunktSentenceTokenizer [4]. Each sentence is then associated with the calculated offset and entity type of the redactions that are present inside. After this procedure, however, we might still encounter sentences containing multiple redacted entities. For this reason, we duplicate the sentence and switch the redaction target until all confidential entities are redacted once. This also serves as a data augmentation technique, as a single sentence can represent multiple labels depending on the redacted data. An example is shown in Table 1.

Table 1. Example of sentence splitting and redaction.

Sentence	Redaction	Offset	Type
It happened on 19/10/2004.	It happened on *****.	(15, 24)	DATETIME
Paolo was in Amsterdam.	***** was in Amsterdam.	(0, 4)	PERSON
	Paolo was in *****.	(13, 21)	LOC

At this stage, the dataset comprises the redacted text, the redaction offset, and the redaction type. However, we notice a substantial imbalance between the class distribution of the labels. Indeed, the most frequent class is datetime with 34280 samples, while code comprises only 2781 samples. This heavy skew in the data distribution can negatively impact a ML model performance, as it would create bias in classification. Therefore, we perform random undersampling on the data to balance the distribution.

Features. After the preprocessing steps, the training and testing dataset mostly contain samples in text format. However, ML and DL traditionally require a constant number of features from each sample to compute a prediction. For this reason, we use a widespread technique in the NLP literature: computing embeddings [23]. Given the importance of sentence data in our dataset, we use SentenceTransformers (SBERT), a popular Python framework for generating text and image embeddings [34]. This Python package provides several Transformer

models with different sizes and performances. We use `all-mpnet-base-v2` as it provides the best performances overall. Indeed, in our threat model scenario, resources or computational overhead do not represent an obstacle, as document unredaction is not a time-constrained task.

SBERT models are generated and trained for general purposes and do not focus on specific domains. As such, using an out-of-the-box model yields lower accuracy values with respect to a specifically trained model. For this reason, we resort to fine-tuning. With this procedure, we can retain the knowledge acquired by the model during the original training procedure and adapt it to our particular domain to increase performance. In the case of SBERT Transformers, fine-tuning is performed by comparing sentences with one another and providing a similarity score for the couple. This score is the cosine similarity. Given two vectors A and B of length n , this metric is defined as follows.

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}. \quad (1)$$

The output of this score is then normalized on a scale from 0 to 1, where higher values indicate higher similarity between the sentences. Therefore, we extract a subset of the training dataset containing 250 samples for each label (i.e., 2000 total samples). For each label, we fine-tune the model as follows: (i) we group sentences in pairs (i.e., 125 for each label) and provide a similarity score of 0.8, and (ii) we randomly pick a sentence from that label and a sentence from another random label (until obtaining 125 couples) and provide a similarity score of 0.2. We select those values since we want our model to comprehend the similarity of the sentence context around the redaction without losing their individual properties. Indeed, since embeddings are created from redacted sentences, our Transformer model can only process the remaining sentence tokens. Therefore, phrase context provides the knowledge the classifier requires to compute predictions. It is worth noting that the 2000 samples used for fine-tuning the model are then discarded from the dataset. We do this to ensure that our Transformer model and the classifier are presented with new data and to prevent overfitting.

After fine-tuning, our Transformer generates embeddings from the remaining training sentences (i.e., 2531 for each label). The embedding size is determined by the Transformer model used for computation, which, in our case, is 768. We now oversample the data at the embedding level to compensate for the under-sampling performed during preprocessing. This is a widespread data augmentation technique used in ML literature, as it can compensate for the possible lack of data in a dataset and allow the classifier to train on more samples [15]. It is also worth noting that, in our scenario, this procedure can be applied only after the embedding computation, as oversampling textual data can provide several artifacts that can instead decrease the classifier performance. In particular, we use Synthetic Minority Over-sampling Technique (SMOTE) as oversampling technique [7]. This technique selectively generates synthetic samples through a

nearest-neighbors approach for the minority class (which, in our case, are all the classes) by interpolating between existing instances. We generate samples for each label until we obtain 3500 total samples for each class. An overview of the data balancing process is shown in Appendix A.2. We then split our dataset in training and testing respective size percentages of 85% and 15%.

4.3 Models

After dealing with the dataset and its feature extraction, we design a classifier for the unredaction task. We present a comprehensive study of different state-of-the-art models by considering three ML models and two DL models. For the ML models, we perform GridSearch on different hyper-parameters configurations to find the best values for them. In particular, we perform a 5-fold cross-validation, removing the need to include a validation set in the train and test dataset split. GridSearch is performed on a balanced subset of the original dataset (~ 10000 samples). Then, the found hyper-parameters are applied to the model to train on the whole training dataset. DL architectures are instead validated manually due to the need to test multiple architectures and layers. Furthermore, being GPU-accelerated, they are significantly faster with respect to the ML executed on the CPU.

ML Models. The first model we test is the Random Forest (RF) [6], an ensemble learning method that constructs multiple Decision Tree (DT) during training. Each tree is trained on a random subset of the dataset (bagging), and a random subset of features is considered for each split, which adds randomness and reduces overfitting. Hyper-parameter search is performed on the number of estimators, the criterion, and the max depth. The first variable determines the number of trees in the forest (tested with 150, 200, and 300). The second represents the function of measuring the quality of a split (tested with gini, entropy, and log loss). The third describes the maximum depth of each DT in the forest, thus controlling the complexity of the model (tested with 3, 5, or unlimited).

Another widespread model used in the literature is Support Vector Machine (SVM) [11]. It works by finding the hyperplane that best separates classes in a high-dimensional space. For nonlinear classification tasks, SVM uses a kernel trick to map the original features into a higher-dimensional space where a separating hyperplane exists. Hyper-parameter search is performed on the kernel and the C parameter. The first variable specifies the kernel type for the algorithm (tested with polynomial and radial basis function). In the case of the polynomial kernel, we also search for its optimal degree (tested with 3 and 4). The second represents the regularization parameter that controls the trade-off between maximizing the margin and minimizing the classification error (tested with 3, 5, and 7). We also use bagging (bootstrap aggregation) to make the training process faster and reduce overfitting.

Finally, we test eXtreme Gradient Boosting (XGBoost) [8]. This model implements gradient-boosting algorithms designed for speed and performance. It

builds multiple decision trees iteratively and tries to correct the errors of the previous models. It uses a gradient descent algorithm to minimize the loss function when adding new models. Hyper-parameter search is performed on the learning rate, the L2 regularization, and the max depth. The first parameter prevents overfitting by shrinking the feature weights to make the boosting process more conservative (tested with 0.01, 0.1, and 0.3). The second variable is a regularization term on weights that penalizes the complexity of the model, preventing overfitting (tested with 0, 0.125, and 0.25). The third represents the maximum depth of each tree (tested with 3, 5, or unlimited).

DL Models. The first DL model we test is a feedforward Deep Neural Network (DNN) composed of multiple layers of neurons. Our specific implementation consists of five fully connected layers. The input layer consists of 768 neurons, corresponding to the data's input features. We include four hidden layers with 512, 256, 128, and 64 neurons, respectively. Each hidden layer applies a linear transformation followed by a Rectified Linear Unit (ReLU) activation function. Finally, the output layer consists of 8 neurons, representing the final output classes. The model is trained with cross-entropy as the loss function and uses Adam as an optimizer with a learning rate of 0.00005 for 200 epochs with a batch size of 100.

The other model architecture we use is the CNN. It is a type of neural network particularly effective for processing grid-like data, such as images or, in this case, one-dimensional sequences. CNNs utilize convolutional layers to automatically and adaptively learn spatial hierarchies of features from the input data. The reason why we select this specific architecture is for its capability to detect patterns in data samples. Indeed, in text data, embedding captures semantic information and relationships between words or tokens [37]. The convolutional layers can effectively process these to detect specific sequences at different abstract levels, such as character-level, word-level, and higher-level semantic features. In our implementation, the first two layers are convolutional. The first layer applies 16 filters of size 16 to the input sequence, producing feature maps. The second layer applies 16 filters of size 16 with a stride of 2, reducing the spatial dimensions. After each convolutional layer, a max pooling operation is applied with a kernel size of 8 and a stride of 2, which reduces the spatial dimensions while retaining significant features. Following the convolutional layers, there are four fully connected layers with 1376, 688, 344, and 172 neurons, respectively. These layers further process the extracted features. Finally, the output layer consists of 8 neurons, representing the final output classes. The model is trained with cross-entropy as the loss function and uses Adam as an optimizer with a learning rate of 0.0001 for 200 epochs with a batch size of 100.

5 Evaluation

We now evaluate the entity-type recognition capabilities of our proposed methodology. First, in Sect. 5.1, we disclose the metrics we use for the evaluation and

their definition. We then provide in Sect. 5.2 a baseline evaluation of our models in scenarios in which the Transformer model has not been fine-tuned on the corpus. We finally show the effectiveness of our fine-tuning procedure in Sect. 5.3.

5.1 Metric

From an ML perspective, our task is a multiclass classification task in which each label has the same importance. This is different from a binary classification task, where there is usually a positive and negative class. For this reason, in our scenario, it is not possible to define False Positives (FP), False Negatives (FN), and True Negatives (TN). Instead, only two events can occur when the model predicts the redacted entity type.

- *True Positive (TP)*: the predicted entity type matches the original entity type.
- *Misclassification (Err)*: the predicted entity type does not match the original entity type.

For this reason, we use accuracy as our evaluation metric, defined as the ratio of correctly classified instances (across all classes) to the total number of samples in the dataset.

$$Accuracy = \frac{\sum_{i=1}^8 TP_i}{\sum_{i=1}^8 (TP_i + Err_i)} = \frac{\# \text{ Correct Predictions}}{\# \text{ Total Predictions}}. \quad (2)$$

5.2 Baseline

We now present the evaluation of our models in baseline performance. This implies that each model is trained on embeddings generated from a non-finetuned Transformer model. As such, after processing the document sentences as detailed in Sect. 4.2 and undersampling them based on the number of classes, we directly compute the embeddings with the out-of-the-box model provided by SBERT. Since, in this scenario, the 2000 samples for fine-tuning the Transformer are not discarded, we reduce the oversampling percentage to reach the same number of total data samples. Training and testing datasets are then generated with the same split percentages. The results of this evaluation are shown in Table 2. DL models outperform all ML models on the test dataset, with the DNN model obtaining the best score. It is also worth noting that ML models are overfitting in most cases. This can be observed by the high scores on the training dataset, which, however, are not representative of the models' capabilities on unforeseen data. The ML models' behavior in this task can be attributed to two main factors: (i) the shallower architectures employed with respect to the DL models, and (ii) the variance and complexity of the embeddings. Indeed, the DL models reach high scores without overfitting, making their more complex architectures

advantageous in this task. Furthermore, high training scores but lower testing scores indicate that the models are memorizing the embeddings, which causes a loss of generalization capabilities. This indicates that the vectorial representation of the sentences is too statistically varied, making smaller models unable to classify them.

Table 2. Baseline evaluation of the models.

Model	Accuracy	
	Train	Test
RF	1.000	0.697
SVM	0.892	0.747
XGBoost	0.970	0.714
DNN	0.979	0.958
CNN	0.981	0.924

5.3 Finetuning

To address the shortcomings of the baseline evaluations, we proposed using a fine-tuned Transformer model for embedding computation. In Table 3, we provide an evaluation of our models trained and tested on the fine-tuned dataset presented in Sect. 4.2. Most of the models present an increased score in both test and train datasets with respect to the baseline of Table 2. Indeed, the best-performing model is now the CNN. Furthermore, we can also notice a significant improvement in the ML models' scores. This growth shows that fine-tuning the Transformer model makes the embedding more homogeneous, allowing for using less complex models. Regardless, the deeper architectures provided by DL should be preferred, as they can provide consistently high scores and prevent overfitting.

Table 3. Evaluation of the models on fine-tuned embeddings.

Model	Accuracy	
	Train	Test
RF	1.000	0.823
SVM	0.983	0.840
XGBoost	0.940	0.866
DNN	0.946	0.945
CNN	0.986	0.985

6 Countermeasures

Given the high accuracies demonstrated by our evaluation in Sect. 5, it is clear that sentence context can provide enough information for an attacker to infer the entity type of a redacted token. This threat can constitute a privacy breach since, through the entity type, data linkage can be performed, which might lead to bias and discrimination. Therefore, it is important to develop effective countermeasures depending on the field of application of the redaction. For example, if sensitive documents are shared in PDF format, an option is to disable printing functionalities, thus preventing possible attackers from detecting and extracting textual information. However, this can be counter-effective, as it also impedes legitimate parties from engaging with the document. Unfortunately, this side effect also applies to other scenarios where text is extracted through OCR systems. For instance, when redacted documents are stored in paper format, their digitalization involves scanning and extracting textual data from retrieved images. Thus, to prevent an attacker from being able to extract the text, adversarial attacks against ML-based OCR systems can be crafted [35, 38]. However, this also prevents legitimate parties from obtaining the document’s contents.

Character Evasion. To address the shortcomings of traditional countermeasures against document unredaction, we propose a technique we call *character evasion*. This method involves substituting specific ASCII characters at test time to fool the entire unredaction pipeline. Taking inspiration from adversarial and evasion techniques, the perturbations applied to the textual data are imperceptible to the human eye but are substantial enough to fool the ML systems [16]. Indeed, one of our study’s assumptions is that the attacker can access (or generate) a dataset of redacted documents. As such, the attacker fine-tunes its Transformer model and trains the DL models to process the text automatically. Traditional evasion techniques would use the models’ gradients to compute adversarial attacks at the embedding level to cause misclassification. However, the attacker owns the unredaction pipeline; thus, legitimate parties cannot access the models’ parameters and architectures. Therefore, countermeasures must be applied at the document level to make the attacker misclassify redacted entity types. In this case, the best strategy is to substitute characters in the text with graphically identical ones, which, however, are processed differently by models. For example, each “a” character in the text can be swapped with the “а” character. These characters are homoglyphs, i.e., they appear visually similar but are distinct characters with different Unicode code points. In particular, the former is a Latin character (U+0061), while the latter is a Cyrillic character (U+0430). We only swap the five most common letters in the English alphabet (for which an identical Cyrillic or Armenian character is available) and show them graphically with their Unicode code in Appendix B. This technique is particularly effective in cases where human readability is necessary while preserving the content’s privacy. Indeed, while humans can perfectly read the text, the unredaction attack is no longer effective. An evaluation of this countermeasure is shown in Table 4. When using character evasion, the accuracy of the models drops to values close to random

guessing. This behavior can be attributed to the Transformer model, which, by processing foreign characters at test time, cannot generate embeddings statistically close to the ones on which the DL models are trained. ML models obtain similar results, with an average accuracy of 0.238. Furthermore, we can notice that fine-tuning the Transformer does not improve the results, making character evasion a robust countermeasure against our attack. It is also worth noting that several Unicode homoglyphs exist for each Latin character. Therefore, the attacker cannot simply substitute the evaded characters one-to-one before inputting the text into the unredaction pipeline. Moreover, most homoglyphs are regular characters of different alphabets, making their detection increasingly difficult when redacting text in other languages.

Table 4. Evaluation of the models with the *character evasion* countermeasure.

Model	Accuracy	
	Baseline	Fine-tuned
DNN	0.182	0.195
CNN	0.181	0.183

7 Conclusions

With the increase in document digitalization and the consequent rise in the exchange of data, it is essential to ensure that sensitive content stays private. For this reason, several redaction techniques have been proposed and used to mask private tokens inside textual documents. With the recent discoveries on the insecurity of specific techniques such as blurring and pixelation, the complete removal of the token is often preferred as, in this way, the entity is erased from the document. However, sentence context can still leak information on the entity type that has been redacted.

Contribution. This paper presented **RedactBuster**, the first entity type extraction attack on redacted tokens. Our attack leverages state-of-the-art DL models for the processing and classification of redacted sentences. We evaluated our attack on a real-world dataset, obtaining an accuracy of 0.958. Furthermore, we proposed an effective countermeasure called *character evasion*, which can aid practitioners in defending against our discovered attack.

Future Works. For future endeavors, we aim to create a new dataset that can aid this line of research. Indeed, while our dataset included several types of documents, more entity types, and document styles can further solidify the contributions provided in this paper. This opportunity also opens up the possibility

of implementing OCR system in the experimentations, thus simulating a complete document digitalization and redaction pipeline. Access to a more realistic framework can aid us in detecting new vulnerabilities and developing more robust solutions to ensure user privacy. Finally, further experiments, such as adversarially training the Transformer with evaded characters, can enhance the applicability of our countermeasure in more varied scenarios.

A Dataset

In this Appendix section, we give more details on some of the processing steps that the text must go through before being fed to the Transformer for embedding computation.

A.1 Character Preprocessing

After handling the section titles, we substitute all `\n` characters with spaces, as some sentences might be contained in multiple lines. This procedure also ensures that the overall number of characters in the text remains the same. We also address possible abbreviations that are common in specific tokens. For example, “no.” is often used for “number” (or “nos.” for “numbers”). These words cause the tokenizer to end a sentence after the dot character. To fix this, we swap the dot character with an underscore. Nevertheless, the tokenization process introduces several artifacts in the text, as it often removes spaces after a dot character at the end of a sentence. This causes the sentences’ offsets to be shifted with respect to the original text. We treat sentences separately and compute the redaction offsets w.r.t. to the sentence start to solve this. Finally, once we find the word to censor inside the sentence, we substitute it with the right amount of asterisk characters.

A.2 Data Balancing

To ensure that the models do not have any biases for classification, we balance the number of data samples for each class with different techniques. An overview is shown in Table 5. First, we randomly undersample data to obtain a balanced distribution. Then, we use a fixed amount of samples for each class for fine-tuning the Transformer model. Finally, with the remaining dataset, we perform oversampling.

Table 5. Number of samples for each class at different stages of the unredaction pipeline.

Class	Dataset	Undersampling	Fine-Tuning	Oversampling
DATE/TIME	34280	2781	2531	3500
ORG	28828	2781	2531	3500
PERSON	13393	2781	2531	3500
DEM	6325	2781	2531	3500
LOC	6224	2781	2531	3500
MISC	5169	2781	2531	3500
QUANTITY	2963	2781	2531	3500
CODE	2781	2781	2531	3500
Total	122963	22248	20248	28000

B Character Evasion

In Table 6, we show all the characters we use for our *character evasion* technique and their respective Unicode code.

Table 6. List of character evasion substitutions.

Original		Evasion	
Char	Code	Char	Code
a	U+0061	a	U+0430
e	U+0065	e	U+0435
i	U+0069	i	U+0456
n	U+006E	n	U+0578
o	U+006F	o	U+043E

C Hardware and Software Configuration

All experiments have been conducted on a workstation with the following configurations.

- **CPU:** AMD Ryzen 5 3600X.
- **RAM:** 32 GB at 3200 MT/s
- **Operating System:** Ubuntu 20.04.4 LTS.
- **Software:** Python 3.8.10.

All ML models are implemented with the **Scikit-learn** Python package, which does not natively support GPU acceleration. Instead, DL models are implemented with Pytorch 1.7.1.

References

1. Bendersky, M., et al.: Information redaction from document data. US Patent 9,734,148 (2017)
2. Bier, E., Chow, R., Gollé, P., King, T.H., Staddon, J.: The rules of redaction: identify, protect, review (and repeat). *IEEE Secur. Priv.* **7**(6), 46–53 (2009)
3. Biggio, B., Roli, F.: Wild patterns: ten years after the rise of adversarial machine learning. *Pattern Recogn.* **84**, 317–331 (2018)
4. Bird, S.: NLTK: the natural language toolkit. In: *Proceedings of the COLING/ACL 2006 Interactive Presentation Sessions*, pp. 69–72 (2006)
5. Bland, M., Iyer, A., Levchenko, K.: Story beyond the eye: glyph positions break pdf text redaction. In: *Proceedings on Privacy Enhancing Technologies* (2023)
6. Breiman, L.: Random forests. *Mach. Learn.* **45**, 5–32 (2001)
7. Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P.: Smote: synthetic minority over-sampling technique. *J. Artif. Intell. Res.* **16**, 321–357 (2002)
8. Chen, T., Guestrin, C.: Xgboost: a scalable tree boosting system. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794 (2016)
9. Chen, X., Jin, L., Zhu, Y., Luo, C., Wang, T.: Text recognition in the wild: a survey. *ACM Comput. Surv. (CSUR)* **54**(2), 1–35 (2021)
10. Chiu, J.P., Nichols, E.: Named entity recognition with bidirectional LSTM-CNNs. *Trans. Assoc. Comput. Linguist.* **4**, 357–370 (2016)
11. Cortes, C., Vapnik, V.: Support-vector networks. *Mach. Learn.* **20**, 273–297 (1995)
12. Cottrille, S.C.: Selective document redaction. US Patent 7,913,167 (2011)
13. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: Bert: pre-training of deep bidirectional transformers for language understanding. *arXiv preprint [arXiv:1810.04805](https://arxiv.org/abs/1810.04805)* (2018)
14. European Parliament, Council of the European Union: Regulation (EU) 2016/679 of the European Parliament and of the Council. <https://data.europa.eu/eli/reg/2016/679/oj>
15. Fernández, A., Garcia, S., Herrera, F., Chawla, N.V.: Smote for learning from imbalanced data: progress and challenges, marking the 15-year anniversary. *J. Artif. Intell. Res.* **61**, 863–905 (2018)
16. Goodfellow, I.J., Shlens, J., Szegedy, C.: Explaining and harnessing adversarial examples. *arXiv preprint [arXiv:1412.6572](https://arxiv.org/abs/1412.6572)* (2014)
17. Hajian, S., Domingo-Ferrer, J., Monreale, A., Pedreschi, D., Giannotti, F.: Discrimination-and privacy-aware patterns. *Data Min. Knowl. Disc.* **29**(6), 1733–1782 (2015)
18. Hill, S., Zhou, Z., Saul, L.K., Shacham, H.: On the (in) effectiveness of mosaicing and blurring as tools for document redaction. *Proc. Priv. Enhancing Technol.* **2016**(4), 403–417 (2016)
19. IVASS: I principali numeri delle assicurazioni in italia (2022). https://www.ivass.it/pubblicazioni-e-statistiche/statistiche/numeri-assicurazioni/2022/Focus-I_principali_numeri_2022.pdf. Accessed 17 Apr 2024
20. Johnson, A.E., et al.: Mimic-iv, a freely accessible electronic health record dataset. *Sci. Data* **10**(1), 1 (2023)
21. Kelly, D.G., Foster, B.R.: Process for electronic document redaction. US Patent 8,456,654 (2013)
22. Li, M., et al.: TrOCR: transformer-based optical character recognition with pre-trained models. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, pp. 13094–13102 (2023)

23. Li, Y., Yang, T.: Word embedding for understanding natural language: a survey. In: *Guide to Big Data Applications*, pp. 83–104 (2018)
24. Liu, Y., et al.: Roberta: a robustly optimized BERT pretraining approach. arXiv preprint [arXiv:1907.11692](https://arxiv.org/abs/1907.11692) (2019)
25. Luoma, J., Pyysalo, S.: Exploring cross-sentence contexts for named entity recognition with BERT. In: *Proceedings of the 28th International Conference on Computational Linguistics*, pp. 904–914 (2020)
26. Mane, S.: Method and system for advanced document redaction. US Patent 11,562,134 (2023)
27. Matichuk, B., Rebstock, J., Kraft, M.: Redaction engine for electronic documents with multiple types, formats and/or categories. US Patent 10,853,570 (2020)
28. Microsoft: Presidio: Data protection and de-identification SDK (2022). <https://microsoft.github.io/presidio/>. Accessed 17 Apr 2024
29. Nabbosa, V., Kaar, C.: Societal and ethical issues of digitalization. In: *Proceedings of the 2020 International Conference on Big Data in Management*, pp. 118–124 (2020)
30. Papadopoulos, C., Pletschacher, S., Clausner, C., Antonacopoulos, A.: The impact dataset of historical document images. In: *Proceedings of the 2nd International Workshop on Historical Document Imaging and Processing*, pp. 123–130 (2013)
31. Petro, D.: GitHub - BishopFox/unredacter: never ever ever use pixelation as a redaction technique — github.com (2022). <https://github.com/bishopfox/unredacter>. Accessed 17 Apr 2024
32. Pilán, I., Lison, P., Øvrelid, L., Papadopoulou, A., Sánchez, D., Batet, M.: The text anonymization benchmark (TAB): a dedicated corpus and evaluation framework for text anonymization. *Comput. Linguist.* **48**(4), 1053–1101 (2022)
33. Ramos, I.S., Dickenson, M., Nair, S.: Document redaction and reconciliation. US Patent App. 16/438,439 (2020)
34. Reimers, N., Gurevych, I.: Sentence-BERT: sentence embeddings using siamese BERT-networks. arXiv preprint [arXiv:1908.10084](https://arxiv.org/abs/1908.10084) (2019)
35. Song, C., Shmatikov, V.: Fooling OCR systems with adversarial text images. arXiv preprint [arXiv:1802.05385](https://arxiv.org/abs/1802.05385) (2018)
36. Tikayat Ray, A., Pinon-Fischer, O.J., Mavris, D.N., White, R.T., Cole, B.F.: aeroBERT-NER: named-entity recognition for aerospace requirements engineering using BERT. In: *AIAA SCITECH 2023 Forum*, p. 2583 (2023)
37. Xu, H., Dong, M., Zhu, D., Kotov, A., Carcone, A.I., Naar-King, S.: Text classification with topic-based word embedding and convolutional neural networks. In: *Proceedings of the 7th ACM International Conference on Bioinformatics, Computational Biology, and Health Informatics*, pp. 88–97 (2016)
38. Xu, X., Chen, J., Xiao, J., Gao, L., Shen, F., Shen, H.T.: What machines see is not what they get: fooling scene text recognition models with adversarial text images. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 12304–12314 (2020)
39. Zhang, R., Yang, Y., Wang, W.: Research on document digitization processing technology. In: *MATEC Web of Conferences*, vol. 309, p. 02014. EDP Sciences (2020)
40. Zhao, X., Greenberg, J., An, Y., Hu, X.T.: Fine-tuning BERT model for materials named entity recognition. In: *2021 IEEE International Conference on Big Data (Big Data)*, pp. 3717–3720. IEEE (2021)

Author Index

A

Aguilar-Melchoro, Carlos 24

B

Bailleux, Thomas 24

Beltrame, Mirco 451

C

Cai, Chengjun 247

Cascavilla, Giuseppe 144

Chen, Rongmao 430

Chen, Rui 289

Chen, Yu 430

Cheng, Ke 310

Cheng, Zhenyu 162

Chiroiu, Mihai 202

Conti, Mauro 451

Cortier, Véronique 410

D

De Pascale, Daniel 144

Debant, Alexandre 410

Deng, Yunjie 85

Dietrich, Sven 3

Ding, Yong 45

Dong, Yifan 123

Du, Ruochen 289

Duan, Li 390

E

Eklund, David 353

F

Fischer, Asja 390

Fong, Philip W. L. 104

Fu, Jiaxuan 310

Fu, Shaojing 430

G

Garg, Sonakshi 374

Ge, Jingquan 85

Goertzen, Jason 24

Guglielmin, Pierpaolo 451

Guinet, Adrien 24

Guo, Li 45

H

Han, Qilong 289

He, Junlin 268

He, Liangzhong 430

Homsí, Soamar 182

I

Iacovazzi, Alfonso 353

J

Jafarov, Isa 3

Jia, Xiaohua 247

Jing, Jiwu 268

Joseph, David 24

L

Lee, Choongin 3

Lee, Heejo 3

Lei, Lingguang 268

Li, Baiyang 45

Li, Qianxing 310

Li, Shuhao 162

Li, Xingxin 331

Li, Yong 390

Lian, Rui 247

Liu, Keyang 331

Liu, Lin 430

Liu, Qingyun 45, 162

Luo, Ning 225

Luo, Yihao 85

M

Mantu, Radu 202
 Marchiori, Francesco 451
 Mendes, Ricardo 390
 Ming, Yulong 247
 Moser, Florian 410

N

Ning, Zhenyu 85

O

Orazi, Gabriele 451

P

Piskac, Ruzica 225
 Pyrgelis, Apostolos 353

R

Rahman, Md Mushfekur 104
 Raiciu, Costin 202
 Raykova, Mariana 225
 Raza, Shahid 353
 Robben, Jeroen 65

S

Shen, Yulong 310
 Shi, Jiangyong 430
 Singh, Jaspal 225
 Song, Anxiao 310
 Song, Zhenyu 123
 Stebila, Douglas 24
 Sun, Peishuai 162
 Sun, Yong 45
 Sun, Yulian 390

T

Takagi, Tsuyoshi 331
 Tamburri, Damian A. 144
 Tan, Gefei 225

Tang, Peng 289
 Torra, Vicenç 374

V

Van Den Heuvel, Willem Jan 144
 Vanhoef, Mathy 65

W

Wang, Chenxu 162
 Wang, Cong 247
 Wang, Han 353
 Wang, Pingjian 268
 Wang, Yuewu 268
 Weng, Chenkai 225
 Wu, Wenxuan 182

X

Xia, Yue 390
 Xia, Yuheng 310
 Xie, Jiang 162
 Xu, Hongbo 162

Y

Yan, Yukun 289
 Yang, Yafang 123
 Yang, Yuexiang 430

Z

Zhang, Fengwei 85
 Zhang, Yuedong 45
 Zhang, Yupeng 182
 Zhang, Zhenhao 123
 Zhao, Yunlei 123
 Zheng, Jieyu 123
 Zheng, Yifeng 247
 Zhu, Derui 390
 Zhu, Haoliang 123
 Zhu, Yujia 45